2025 EDITION

FREE

SYSTEM DESIGN

THE BIG ARCHIVE



ByteByteGo

The 9 Algorithms That Dominate Our World....8
How does gRPC work?....12
Docker vs. Kubernetes. Which one should we use?....14
What happens when you type google .com into a browser?....18
A Cheatsheet on Infrastructure as Code Landscape....20
A Crash Course on Architectural Scalability....22
The Open Source AI Stack....24
Cookies Vs Sessions Vs JWT Vs PASETO....26
How TikTok Manages a 200K File Frontend MonoRepo?....28
The Ultimate API Learning Roadmap....30
30 Useful AI Apps That Can Help You in 2025....32
10 Essential Components of a Production Web Application....34
Session, Cookie, JWT, Token, SSO, and OAuth 2.0 Explained in One Diagram....36
DeepSeek 1-Pager....38
My Favorite 10 Books for Software Developers....40
What is an AI Agent?....42
Git vs GitHub....44
A Cheatsheet on Database Performance....46
18 Common Ports Worth Knowing....48
Generative Adversarial Network (GAN) - AI by Hand ✍️....50
JWT 101: Key to Stateless Authentication....51
A handy cheat sheet for the most popular cloud services....53
0 to 1.5 Billion Guests: Airbnb's Architectural Evolution....57
12 Algorithms for System Design Interviews....59
How Kubernetes Works?....61
PostgreSQL 101: The Everything Database....63
Top 12 Tips for API Security....65
24 Good Resources to Learn Software Architecture in 2025....67
SQS vs SNS vs EventBridge vs Kinesis....69
Top 5 common ways to improve API performance....71
xAI's Grok-3 One-Pager....73
30 Free APIs for Developers....75
The Generative AI Learning Roadmap....77
HTTP/1 -> HTTP/2 -> HTTP/3....79
Structure of URL....81
A Cheatsheet on Comparing Key-Value Stores....83
Which Database Should I Use on AWS?....85
What is MCP?....87
How to Design a System like Instagram....89
How to load your websites at lightning speed....91
What is SSO (Single Sign-On)?....93

How Java Virtual Threads Work? 95
Redis VS Memcached 97
The Shopify Tech Stack 98
What is Retrieval-Augmented Generation (RAG)? 102
How Two-factor Authentication (2FA) Works? 104
How Amazon S3 Works? 106
How to Learn Backend Development? 108
A Simplified Git Workflow 110
Virtualization vs Containerization 112
How Netflix Built a Distributed Counter? 114
How to Learn API Development? 116
Must-Know Network Protocol Dependencies 118
Top AI Coding Tools for Developers You Can Use in 2025 120
18 Key Design Patterns Every Developer Should Know 122
The Data Engineering Roadmap 124
What do version numbers mean? 128
Everyone talks about Transformers. How Transformers Architecture Works? 130
Top YouTube Channels and Blogs for AI Learning in 2025 132
Top 20 System Design Concepts You Should Know 134
9 OOP Design Patterns You Must Know 139
A Cheatsheet on REST API Design Best Practices 143
Top 30 AWS Services That Are Commonly Used 145
The Large-Language Model Glossary 147
9 Clean Code Principles To Keep In Mind 149
The 4 Types of SQL Joins 151
How to Learn Cloud Computing? 153
Visualizing a SQL query 155
Explaining JSON Web Token (JWT) with simple terms 156
12 MCP Servers You Can Use in 2025 158
How to Deploy Services 160
The System Design Topic Map 162
How Transformers Architecture Works? 164
JWT Simply Explained 166
The 5 Pillars of API Design 168
How Computer Memory Works? 170
Top Kubernetes Scaling Strategies You Must Know 172
AI Agent versus MCP 174
How does HTTPS work? 176
How to Learn Kubernetes? 178
Top 6 most commonly used Server Types 180
Amazon Key Architecture with Third Party Integration 182

How Do Companies Ship Code to Production?.....184
What is Event Sourcing? How is it different from normal CRUD design?.....186
How Data Lake Architecture Works?.....188
How Netflix Built a Distributed Counter.....190
Top 20 AI Concepts You Should Know.....194
The AI Application Stack for Building RAG Apps.....196
Shopify Tech Stacks and Tools.....198
AI Vs Machine Learning Vs Deep Learning Vs Generative AI.....200
How SQL Query Executes In A Database?.....202
Top 20 AI Agent Concepts You Should Know.....204
How RabbitMQ Works?.....206
RAG vs Agentic RAG.....208
A Cheatsheet on Kubernetes.....210
6 Data Structures to Save Storage.....212
5 Database Normal Forms Every Developer Should Know.....214
How CQRS Works?.....218
How MongoDB Works?.....220
The Generative AI Tech Stack.....222
24 Good Resources to Learn Software Architecture in 2025.....224
Database Index Types Every Developer Should Know.....226
The Agentic AI Learning Roadmap.....228
12 MCP Servers You Can Use in 2025.....230
Top 5 common ways to improve API performance.....232
REST API Vs. GraphQL.....234
Tokens vs API Keys.....236
The AWS Tech Stack.....238
5 Data Structures That Make DB Queries Super Fast.....240
12 MCP Servers You Can Use in 2025.....242
MCP Versus A2A Protocol.....244
How can Cache Systems go wrong?.....246
8 System Design Concepts Explained in 1 Diagram.....248
16 Coding Patterns That Make Interviews Easy.....250
How to Learn Databases?.....252
What happens when you type a URL into a browser?.....254
How HTTPS Works?.....256
The Evolution of Scaling at Netflix.....258
What is the SOLID Principle?.....260
Common HTTP Status Codes.....262
How Clean Architecture Works?.....264
How does Docker Work?.....265
How Large Language Models Work?.....266

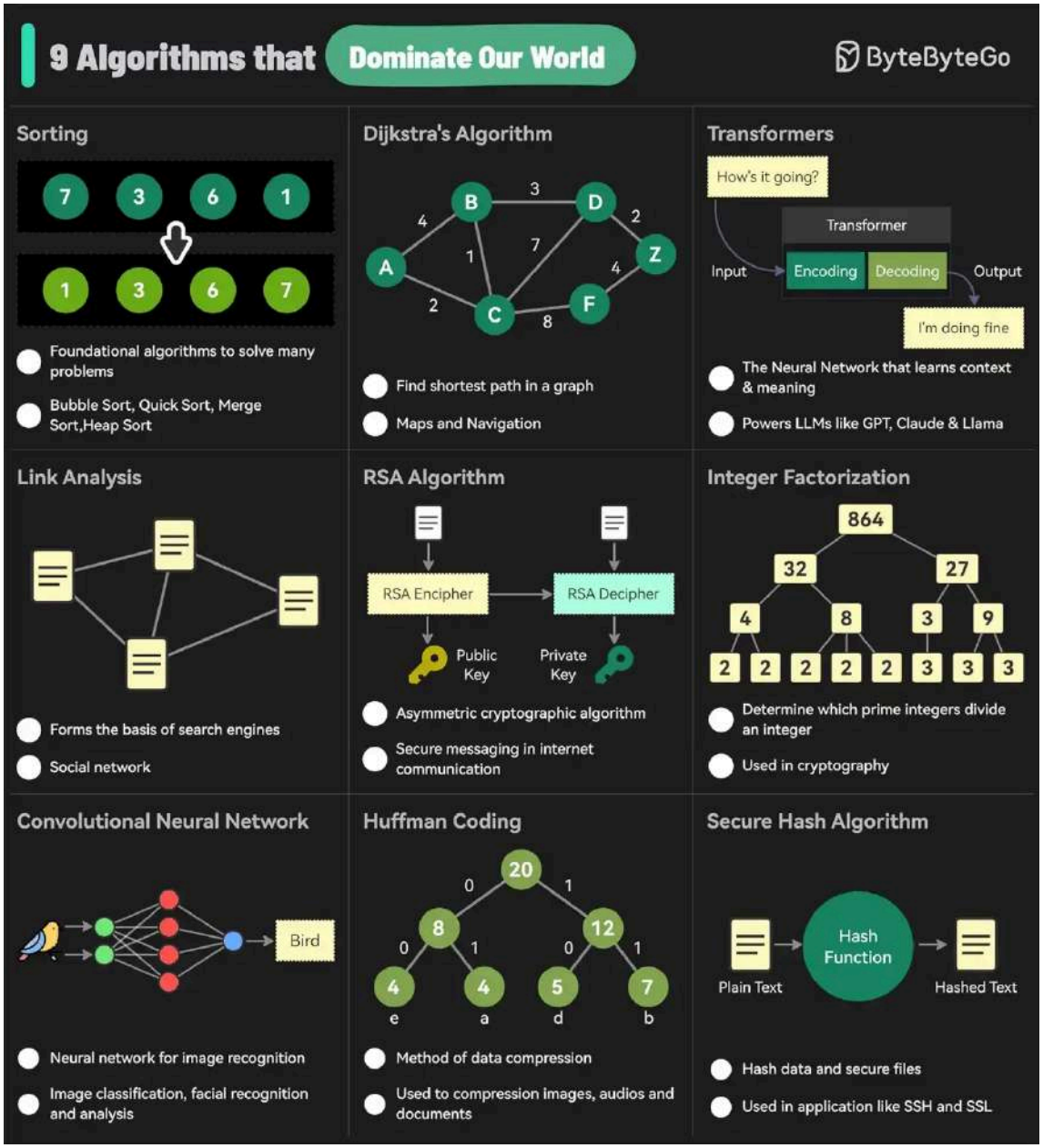
How Does SSO Work? 268
Best Practices in API Design 270
Key Terms in Domain-Driven Design 272
Top AI Agent Frameworks You Should Know 274
How OpenAI's GPT-OSS 120B and 20B Models Work? 276
The Modern Software Stack 278
Concurrency is NOT Parallelism 280
JWT vs PASETO: The Two Players of Token-Based Authentication 282
The Linux Cron Cheatsheet 284
AI Agent versus MCP 286
The Lifecycle of a Kubernetes Pod 288
CI/CD Pipeline Explained 290
The Open Source RAG Stack 291
What are some of the most popular versioning strategies? 293
The Testing Pyramid 294
9 Docker Best Practices You Should Know 295
Kubernetes Explained 297
N8N versus LangGraph 299
Where Do We Cache Data? 301
Python vs Java 303
Design Patterns Cheat Sheet 305
CI/CD Simplified Visual Guide 308
How Apache Kafka Works? 310
Load Balancers vs API Gateways vs Reverse Proxy! And how can they Work Together? 312
How Java Works 314
How Gitflow Branching Works? 316
The Life of a Redis Query 318
6 Steps to Create a New AI Model 320
Cookies vs Sessions 322
Access Control Clearly Explained 324
Full Fine-Tuning vs LoRA vs RAG 326
How Git Reset Works? 328
Apache Kafka Explained (At the high level) 330
MCP vs API: what's the difference? 332
TCP vs UDP 334
A piece of knowledge each day: AI and Machine Learning (ML) 335
How Python Works 337
API Vs SDK! 339
SQL Injection (SQLi) 341
Types of AI Agents 343
24 Good Resources to Learn Software Architecture in 2025 345

Cross-Site Scripting (XSS) Attacks....347
Docker vs Kubernetes....349
Batch vs Stream Processing....351
What are Modular Monoliths?....353
Popular interview question: What is the difference between Process and Thread?....355
How AI Agents Chain Tools, Memory, and Reasoning?....356
Latency vs. Throughput....358
Top 20 System Design Concepts You Should Know....360
How to Debug a Slow API?....362
How LLMs See the World....364
RAG vs Fine-tuning: Which one should you use?....366
IP Address Cheat Sheet Every Engineer Should Know....369
Which Protocols Run on TCP and UDP....371
Why is DeepSeek-OCR such a BIG DEAL?....373
Servers You Should Know in Modern Systems....375
The Building Blocks of Modern Networking....376
Network Services That Power Modern Connectivity....378
How to Design Good APIs....380
Big Data Pipeline Cheatsheet for AWS, Azure, and Google Cloud....382
How to Learn AWS?....384
The AI Agent Tech Stack....386
How to Build a Basic RAG Application on AWS?....388
Types of Virtualization....390
Cloudflare vs. AWS vs. Azure....392
Popular Backend Tech Stack....394
HTTP vs. HTTPS....395
Forward Proxy versus Reverse Proxy....397
Things Every Developer Should Know: Concurrency is NOT parallelism....399
Virtualization vs. Containerization....401
5 REST API Authentication Methods....403
How do AirTags work?....405
What is a Firewall?....407
Modem vs. Router....409
What is a REST API?....411
How Java HashMaps Work?....413
Virtualization Explained: From Bare Metal to Hosted Hypervisors....415
Database Types You Should Know in 2025....417
Apache Kafka vs. RabbitMQ....419
The HTTP Mindmap....421
How DNS Works....423
Can a web server provide real-time updates?....425

Evolution of HTTP....427
System Performance Metrics Every Engineer Should Know....429
Why Is Nginx So Popular?....431
Network Debugging Commands Every Engineer Should Know....433
Hub, Switch, & Router Explained....435
Common Network Protocols Every Engineer Should Know....437
8 Popular Network Protocols....439
A picture is worth a thousand words: 9 best practices for developing microservices....441

The 9 Algorithms That Dominate Our World

The diagram below shows the most commonly used algorithms in our daily lives. They are used in internet search engines, social networks, WiFi, cell phones, and even satellites.



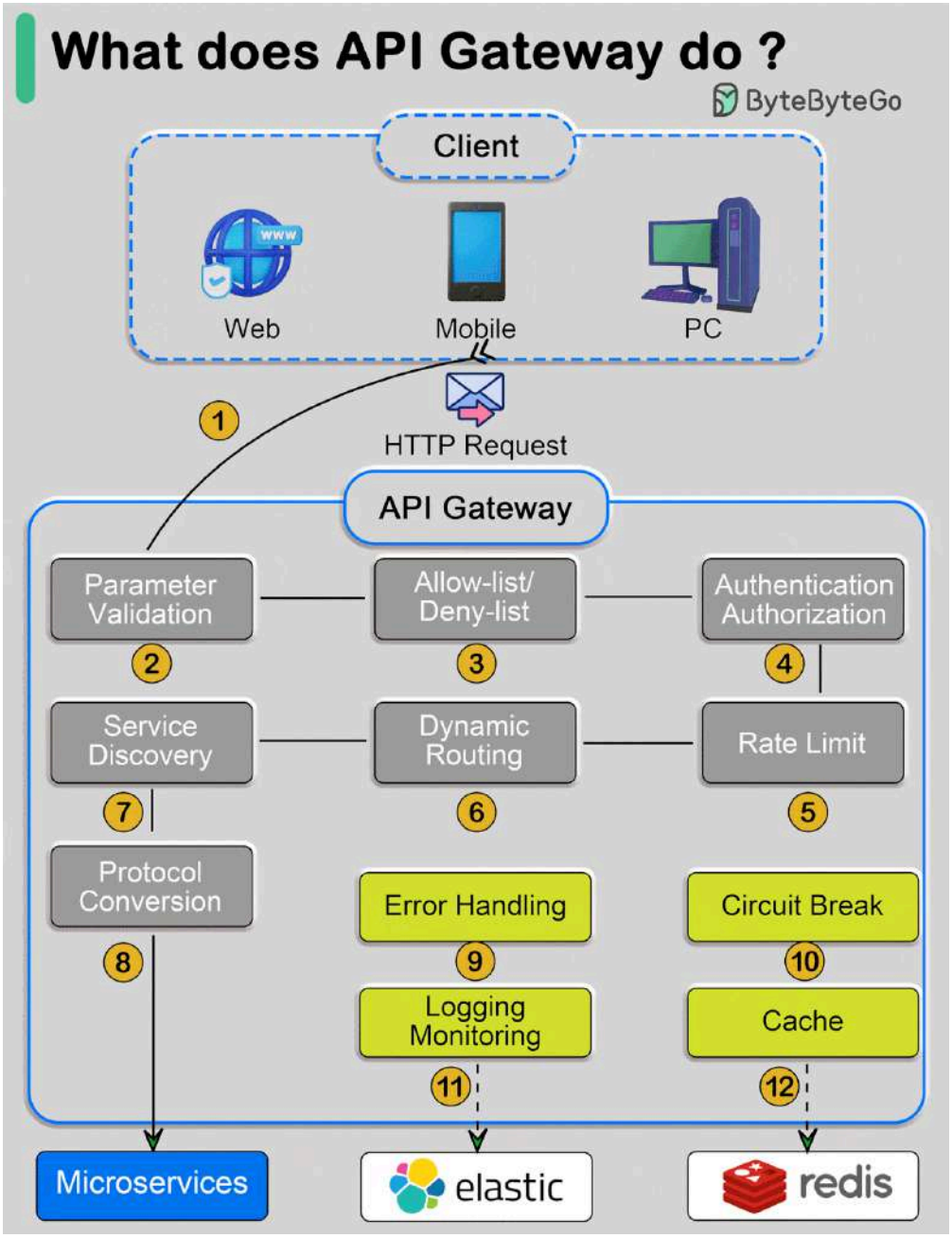
1. Sorting
2. Dijkstra’s Algorithm
3. Transformers
4. Link Analysis
5. RSA Algorithm
6. Integer Factorization

7. Convolutional Neural Networks
8. Huffman Coding
9. Secure Hash Algorithm

Over to you: Are there any other commonly used algorithms we missed?

What does API gateway do?

The diagram below shows the detail.



Step 1 - The client sends an HTTP request to the API gateway.
Step 2 - The API gateway parses and validates the attributes in the HTTP request.

Step 3 - The API gateway performs allow-list/deny-list checks.

Step 4 - The API gateway talks to an identity provider for authentication and authorization.

Step 5 - The rate limiting rules are applied to the request. If it is over the limit, the request is rejected.

Steps 6 and 7 - Now that the request has passed basic checks, the API gateway finds the relevant service to route to by path matching.

Step 8 - The API gateway transforms the request into the appropriate protocol and sends it to backend microservices.

Steps 9-12: The API gateway can handle errors properly, and deals with faults if the error takes a longer time to recover (circuit break). It can also leverage ELK (Elastic-Logstash-Kibana) stack for logging and monitoring. We sometimes cache data in the API gateway.

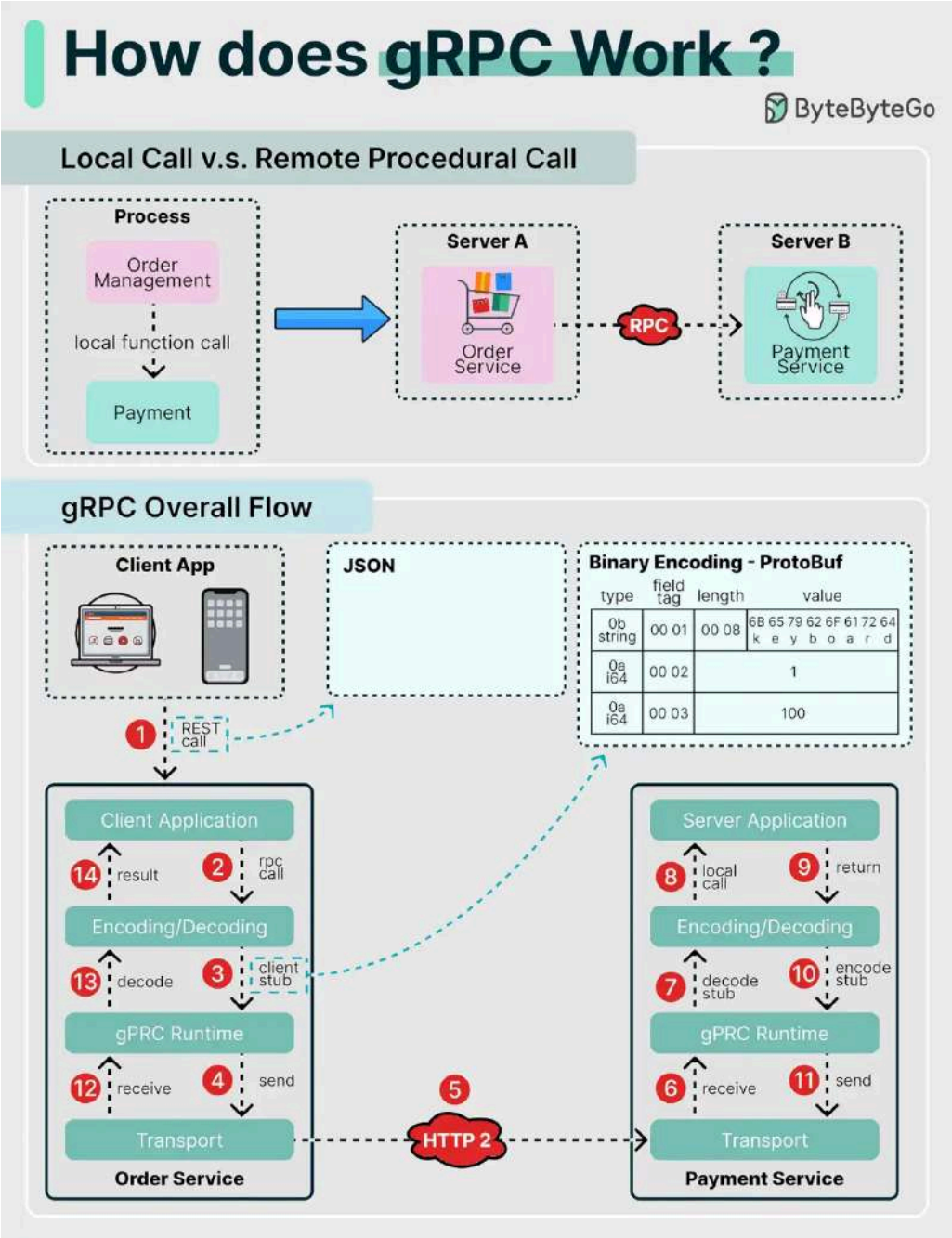
Over to you:

1. What's the difference between a load balancer and an API gateway?
2. Do we need to use different API gateways for PC, mobile and browser separately?

How does gRPC work?

RPC (Remote Procedure Call) is called “remote” because it enables communications between remote services when services are deployed to different servers under microservice architecture. From the user’s point of view, it acts like a local function call.

The diagram below illustrates the overall data flow for gRPC.



Step 1: A REST call is made from the client. The request body is usually in JSON format.

Steps 2 - 4: The order service (gRPC client) receives the REST call, transforms it, and makes an RPC call to the payment service. gPRC encodes the client stub into a binary format and sends it to the low-level transport layer.

Step 5: gRPC sends the packets over the network via HTTP2. Because of binary encoding and network optimizations, gRPC is said to be 5X faster than JSON.

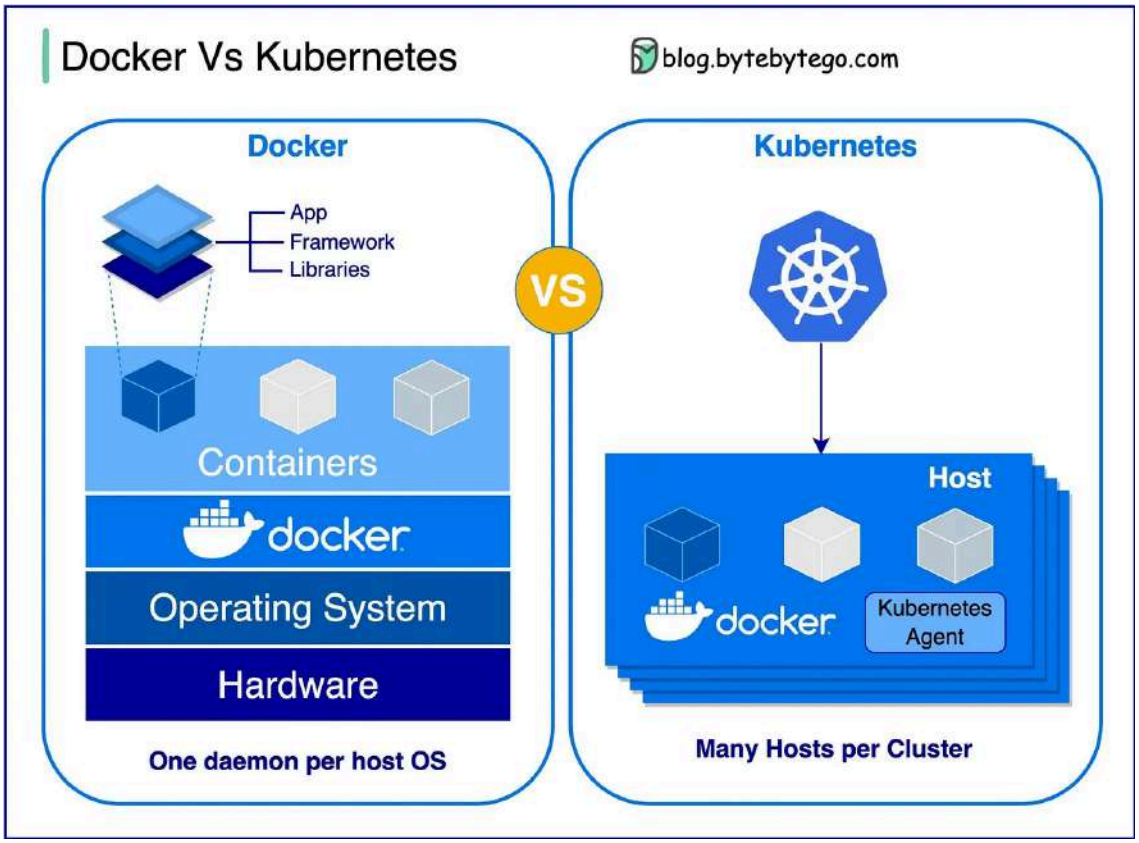
Steps 6 - 8: The payment service (gRPC server) receives the packets from the network, decodes them, and invokes the server application.

Steps 9 - 11: The result is returned from the server application, and gets encoded and sent to the transport layer.

Steps 12 - 14: The order service receives the packets, decodes them, and sends the result to the client application.

Over to you: Have you used gPRC in your project? What are some of its limitations?

Docker vs. Kubernetes. Which one should we use?



What is Docker ?

Docker is an open-source platform that allows you to package, distribute, and run applications in isolated containers. It focuses on containerization, providing lightweight environments that encapsulate applications and their dependencies.

What is Kubernetes ?

Kubernetes, often referred to as K8s, is an open-source container orchestration platform. It provides a framework for automating the deployment, scaling, and management of containerized applications across a cluster of nodes.

How are both different from each other ?

Docker: Docker operates at the individual container level on a single operating system host.

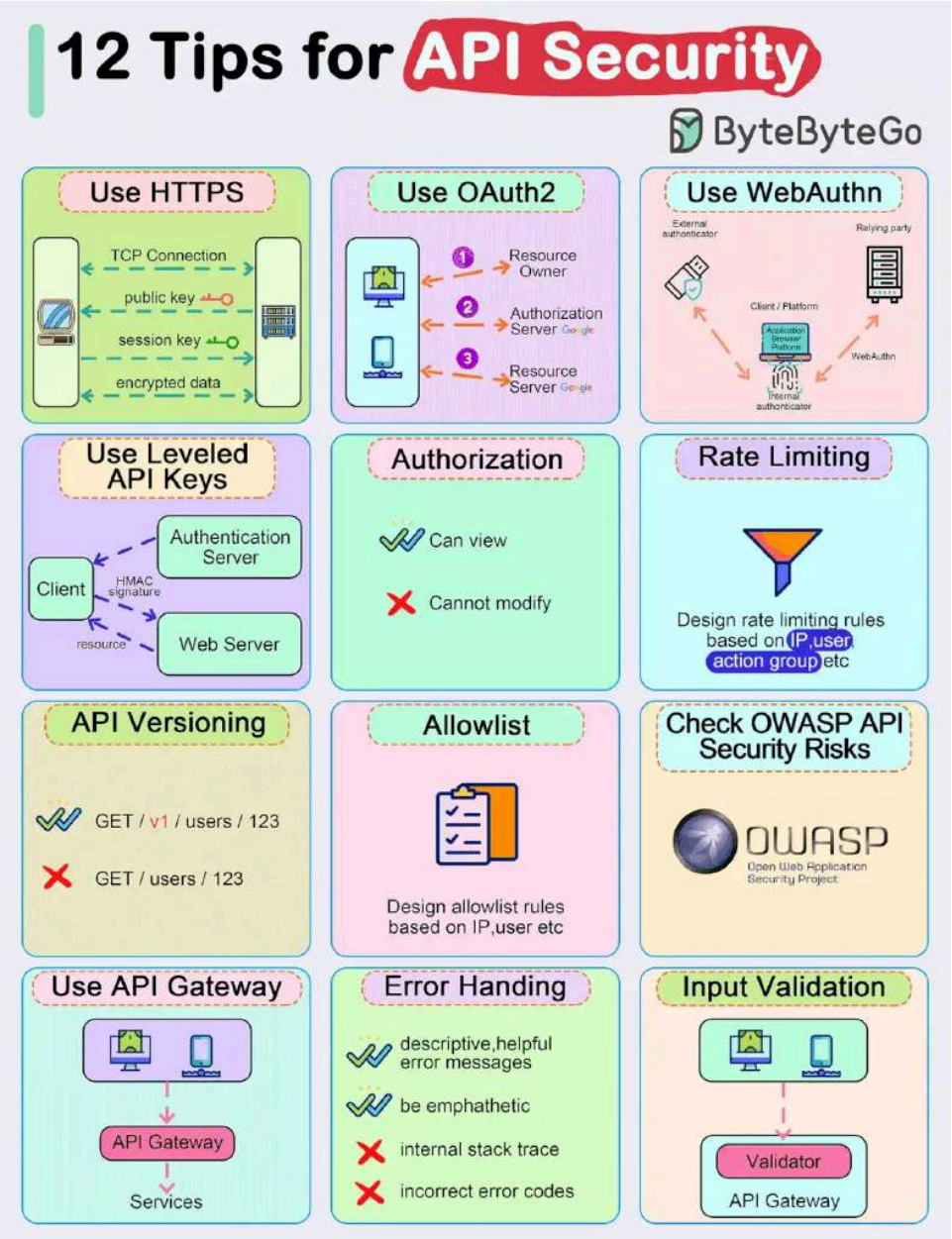
You must manually manage each host and setting up networks, security policies, and storage for multiple related containers can be complex.

Kubernetes: Kubernetes operates at the cluster level. It manages multiple containerized applications across multiple hosts, providing automation for tasks like load balancing, scaling, and ensuring the desired state of applications.

In short, Docker focuses on containerization and running containers on individual hosts, while Kubernetes specializes in managing and orchestrating containers at scale across a cluster of hosts.

Over to you: What challenges prompted you to switch from Docker to Kubernetes for managing containerized applications?

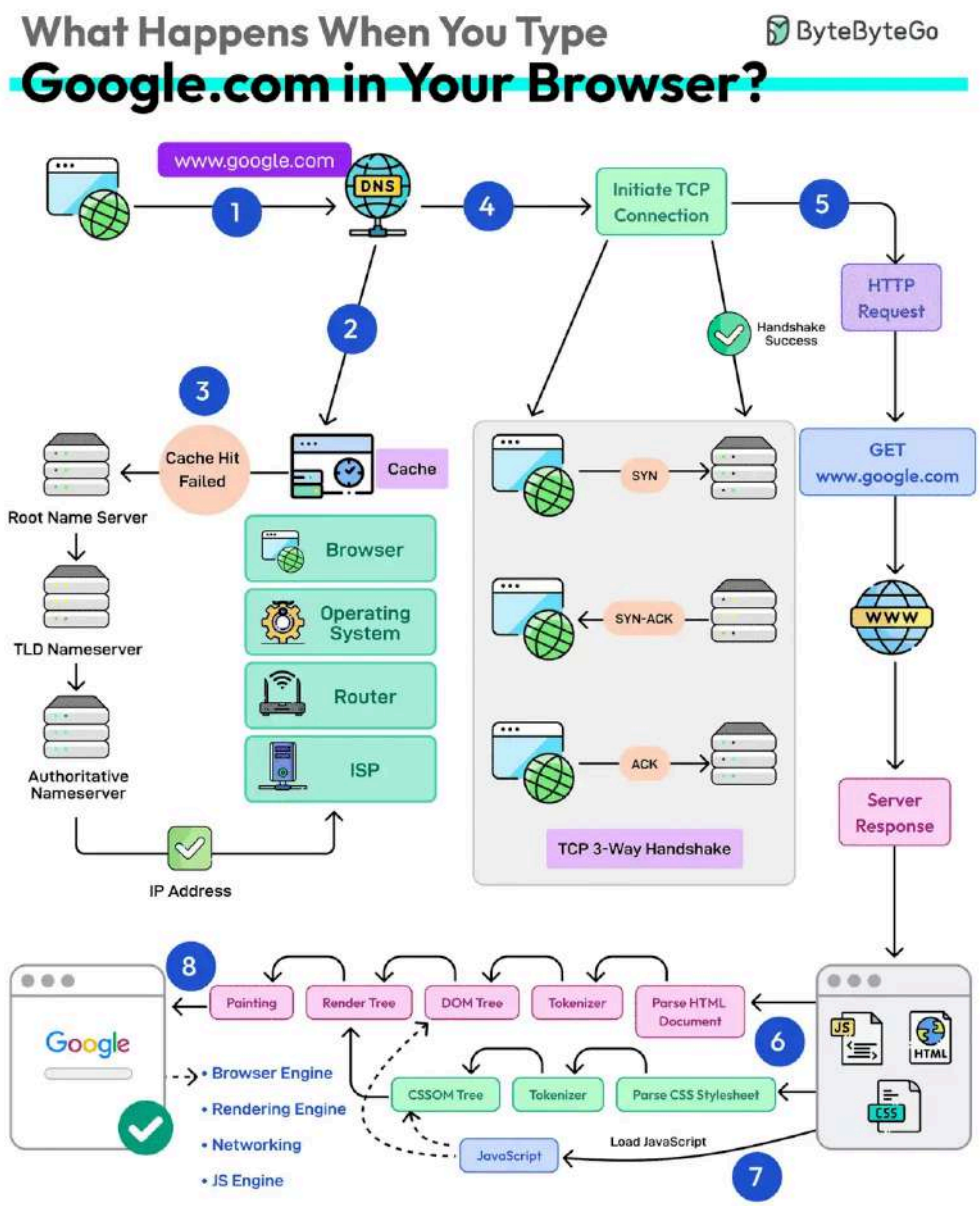
Top 12 Tips for API Security



- Use HTTPS
- Use OAuth2
- Use WebAuthn
- Use Leveled API Keys
- Authorization

- Rate Limiting
- API Versioning
- Whitelisting
- Check OWASP API Security Risks
- Use API Gateway
- Error Handling
- Input Validation

What happens when you type google .com into a browser?

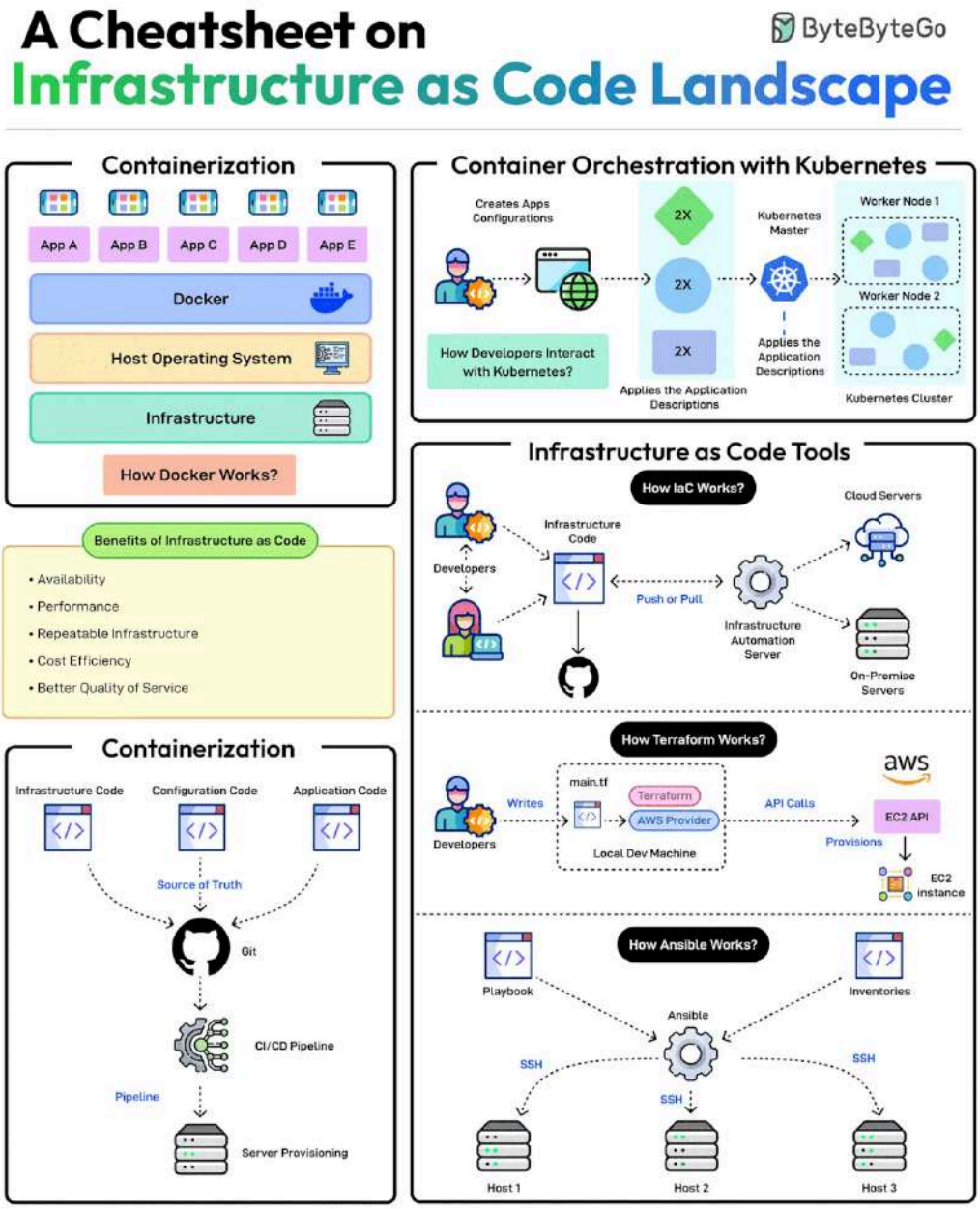


1. First up, you type the website address in the browser's address bar.
2. The browser checks its cache first. If there's a cache miss, it must find the IP address.
3. DNS lookup begins (think of it as looking up a phone number). The request goes through different DNS servers (root, TLD, and authoritative). Finally, the IP address is retrieved.

4. Next, your browser initiates a TCP connection like a handshake. For example, in the case of HTTP 1.1, the client and server perform a TCP three-way handshake with SYN, SYN-ACK, and ACK messages.
5. Once the handshake is successful, the browser makes an HTTP request to the server and the server responds with HTML, CSS, and JS files.
6. Finally, the browser processes everything. It parses the HTML document and creates DOM and CSSOM trees.
7. The browser executes the JavaScript and renders the page through various steps (tokenizer, parser, render tree, layout, and painting).
8. Finally, the webpage appears on your screen.

Over to you: Which other step will you add to the overall process?

A Cheatsheet on Infrastructure as Code Landscape



Scalable infrastructure provisioning provides several benefits related to availability, scalability, repeatability, and cost-effectiveness.

But how do you achieve this?

Provisioning infrastructure using code is the key to scalable infra management.
There are multiple strategies that can help:

1. Containerization is one of the first strategies to make application deployments based on code. Docker is one of the most popular ways to containerize the application.

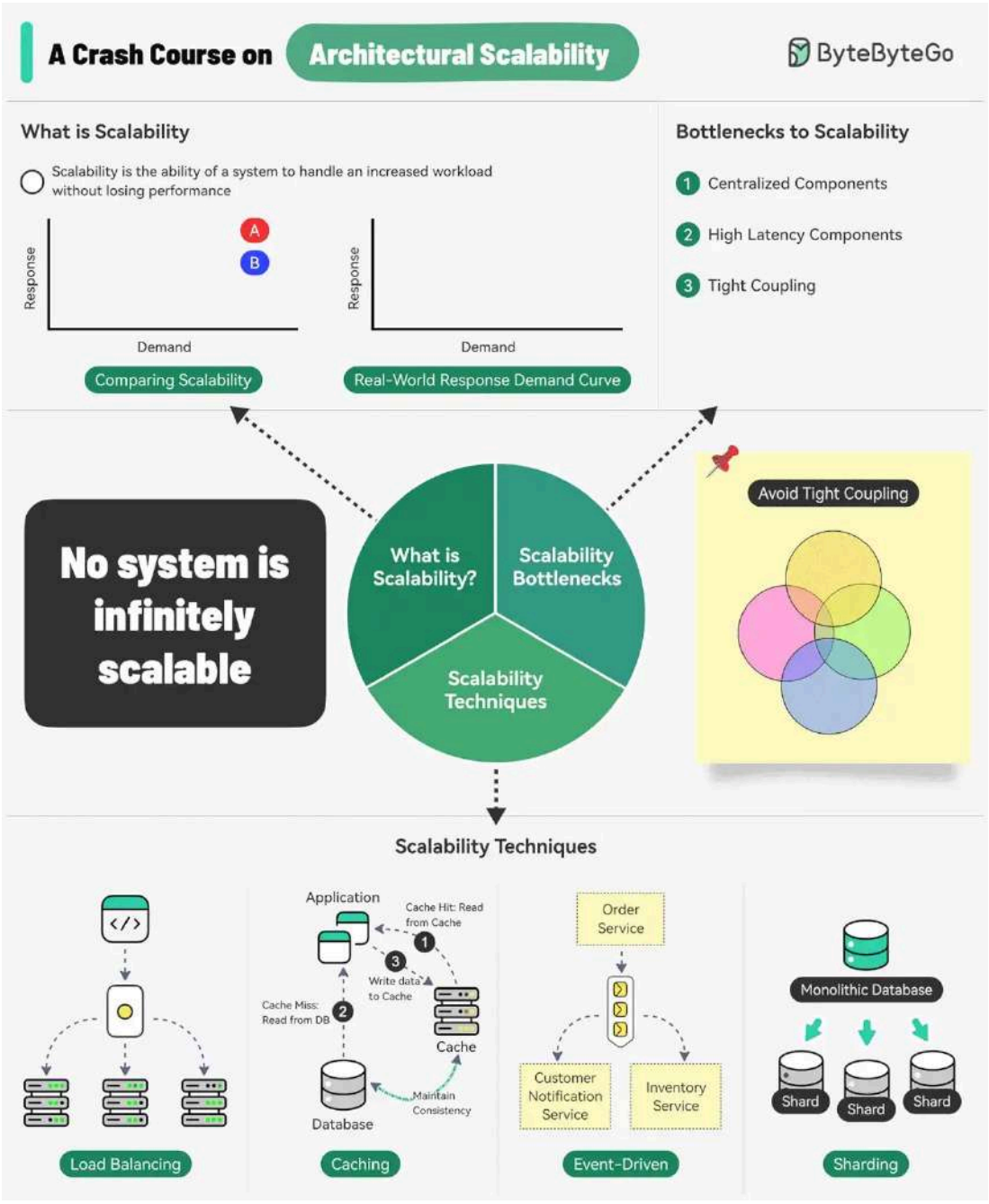
2. Next, container orchestration becomes a necessity when dealing with multiple containers in an application. This is where container orchestration tools like Kubernetes become important.

3. IaC treats infrastructure provisioning and configuration as code, allowing developers to define the application infrastructure in files that can be versioned, tested, and reused. Popular tools such as Terraform, AWS CloudFormation, and Ansible can be used. Ansible is more of a configuration tool.

4. GitOps leverages a Git workflow combined with CI/CD to automate infrastructure and configuration updates.

Over to you: Have you used Infrastructure as Code for your projects?

A Crash Course on Architectural Scalability



Scalability is the ability of a system to handle an increased workload without losing performance. However, we can also look at scalability in terms of the scaling strategy.

Scalability is the system's ability to handle an increased workload by repeatedly applying a cost-effective strategy. This means it can be difficult to scale a system beyond a certain point if the scaling strategy is not financially viable.

Three main bottlenecks to scalability are:

1. Centralized components: This can become a single point of failure
2. High Latency Components: These are components that perform time-consuming operations.
3. Tight Coupling: Makes the components difficult to scale

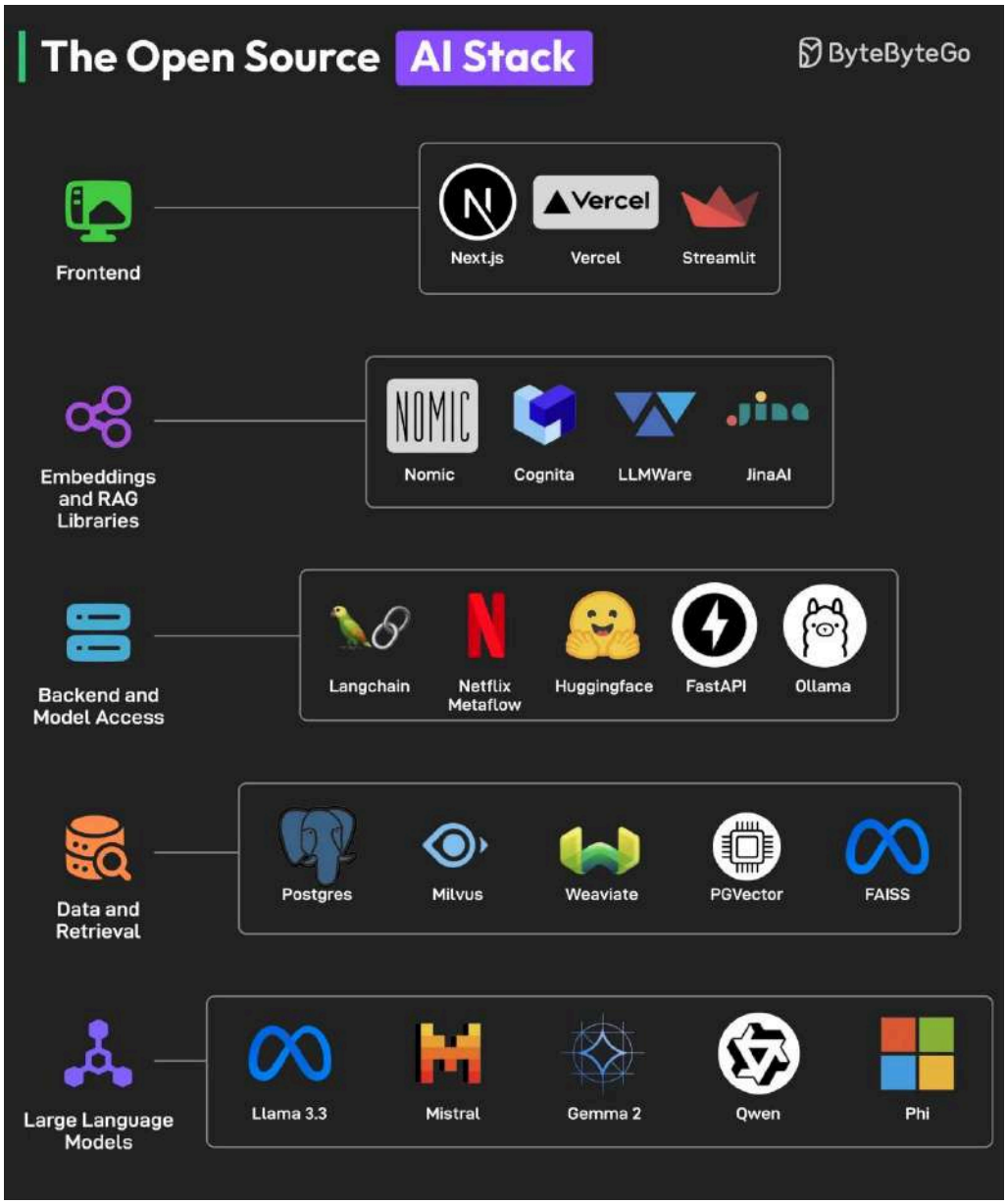
Therefore, to build a scalable system, we should follow the principles of statelessness, loose coupling, and asynchronous processing.

Some common techniques for improving scalability are as follows:

- Load Balancing: Spread requests across multiple servers to prevent a single server from becoming a bottleneck.
- Caching: Store the most commonly requested information in memory.
- Event-Driven Processing: Use an async processing approach to process long-running tasks.
- Sharding: Split a large dataset into smaller subsets called shards for horizontal scalability.

Over to you: How do you improve a system's scalability?

The Open Source AI Stack



You don’t need to spend a fortune to build an AI application. The best AI developer tools are open-source, and an excellent ecosystem is evolving that can make AI accessible to everyone. The key components of this open-source AI stack are as follows:

1. Frontend
   To build beautiful AI UIs, frameworks like NextJS and Streamlit are extremely useful. Also, Vercel can help with deployment.

2. Embeddings and RAG libraries
   Embedding models and RAG libraries like Nomic, JinaAI, Cognito, and LLMAware help developers build accurate search and RAG features.

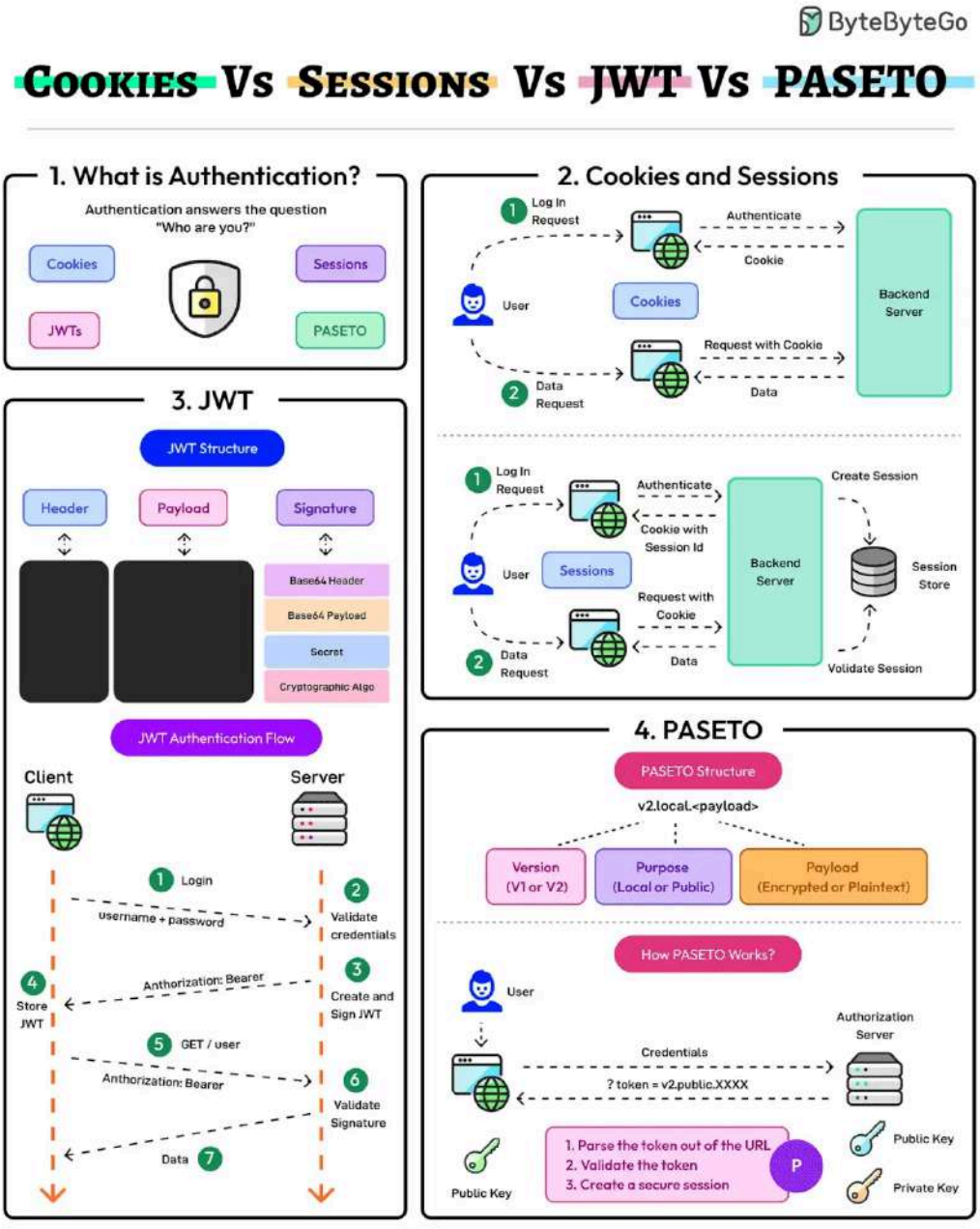
3. Backend and Model Access
   For backend development, developers can rely on frameworks like FastAPI, Langchain, and Netflix Metaflow. Options like Ollama and Huggingface are available for model access.

4. Data and Retrieval
   For data storage and retrieval, several options like Postgres, Milvus, Weaviate, PGVector, and FAISS are available.

5. Large-Language Models
   Based on performance benchmarks, open-source models like Llama, Mistral, Qwen, Phi, and Gemma are great alternatives to proprietary LLMs like GPT and Claude.

   Over to you: Which other tool will you add to the Open Source AI Stack?

Cookies Vs Sessions Vs JWT Vs PASETO



Authentication ensures that only authorized users gain access to an application's resources. It answers the question of the user's identity i.e. "Who are you?"
The modern authentication landscape has multiple approaches: Cookies, Sessions, JWTs, and PASETO. Here's what they mean:

1. Cookies and Sessions
   Cookies and sessions are authentication mechanisms where session data is stored on the server and referenced via a client-side cookie.

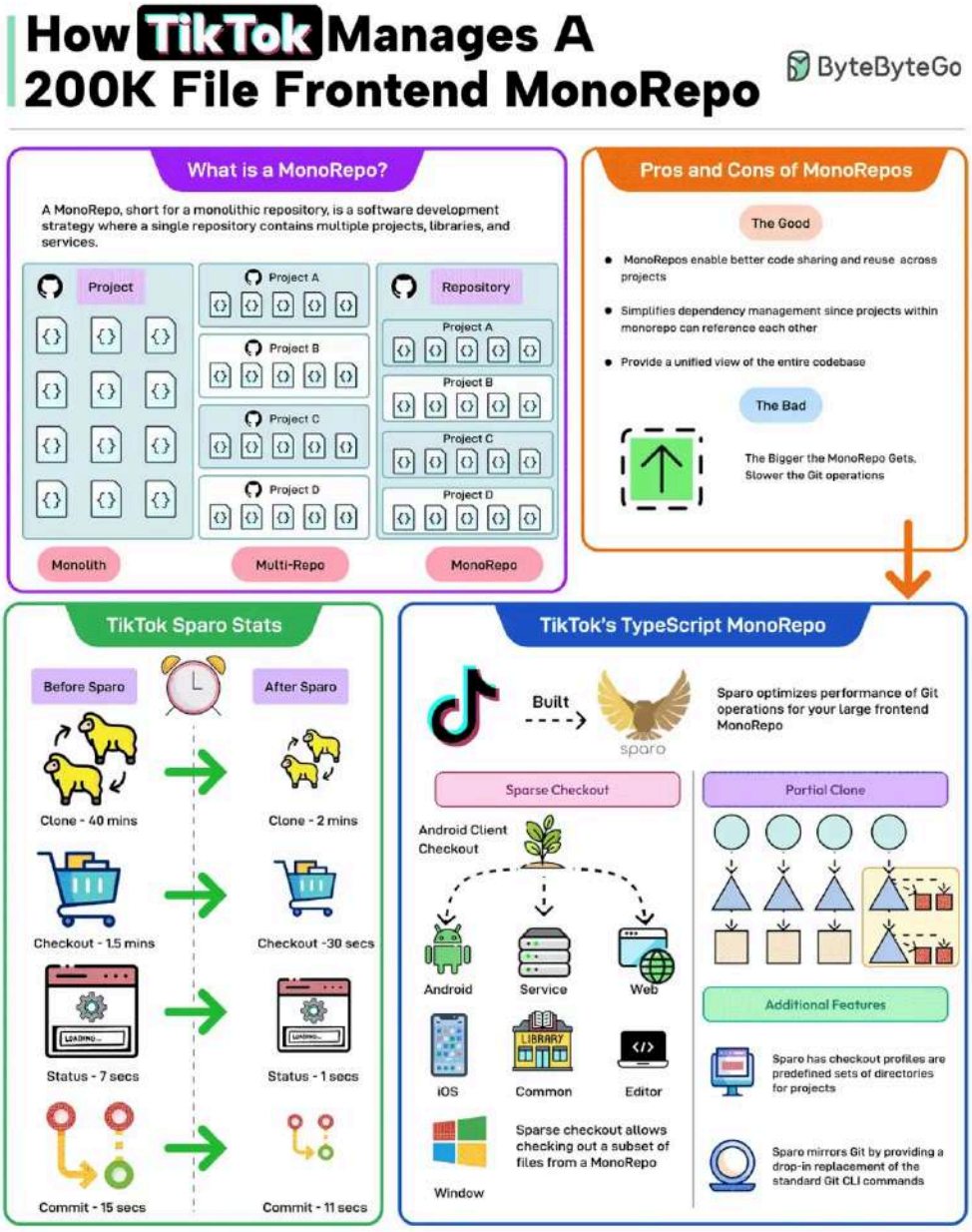
Sessions are ideal for applications requiring strict server-side control over user data. On the downside, sessions may face scalability challenges in distributed systems.

2. JWT
   JSON Web Token (JWT) is a stateless, self-contained authentication method that stores all user data within the token.
   JWTs are highly scalable but require careful handling to mitigate the chances of token theft and manage token expiration.

3. PASETO
   Platform-Agnostic Security Tokens or PASETO improve upon JWT by enforcing stronger cryptographic defaults and eliminating algorithmic vulnerabilities.
   PASETO simplifies token implementation by avoiding the risks associated with misconfiguration.

Over to you: Which authentication approach have you used?

How TikTok Manages a 200K File Frontend MonoRepo?



A MonoRepo, short for a monolithic repository, is a software development strategy where a single repository contains multiple projects, libraries, and services.

The good parts of a MonoRepo are:

- Better code sharing

- Simplified dependency management
- A unified view of the code base

However, the bigger the MonoRepo gets, the slower the various Git operations.

TikTok faced a similar change with its frontend TypeScript MonoRepo with 200K files.

To deal with this, TikTok built a tool named Sparo that optimizes the performance of Git operations for large frontend MonoRepos.

Sparo dramatically improved the performance of Git operations. Some stats are as follows

- Git clone time went from 40 mins to just 2 mins.
- Checkout went from 1.5 minutes to 30 seconds.
- Status went from 7 seconds to 1 second.
- Git commit time went from 15 seconds to 11 seconds.

Over to you: Have you used MonoRepos in your projects?

The Ultimate API Learning Roadmap



APIs are the backbone of communication over the Internet. Every developer needs to learn about APIs. Here's a roadmap that covers the most important topics:

1. Introduction to APIs
   API is a set of protocols and tools for building applications. Different types of APIs exist, such as public, private, and partner.

2. API Terminologies
   Various API terminologies, such as HTTP versions, cookies, and caching, need to be understood.

3. API Styles
   The most common API styles are REST, SOAP, GraphQL, gRPC, and WebSockets

4. API Authentication
   API Authentication techniques like Basic Auth, Token, JWTs, OAuth, and Session Auth

5. API Documentation
   A good API is understandable. API Documentation tools like Swagger, Postman, Redoc, and DapperDox make it possible.

6. API Features
   Key API features include pagination, parameters, idempotency, API versioning, HATEOAS, and content negotiation

7. API Performance Techniques
   Common API performance techniques are caching, rate limiting, load balancing, pagination, DB indexing, scaling, and performance testing.

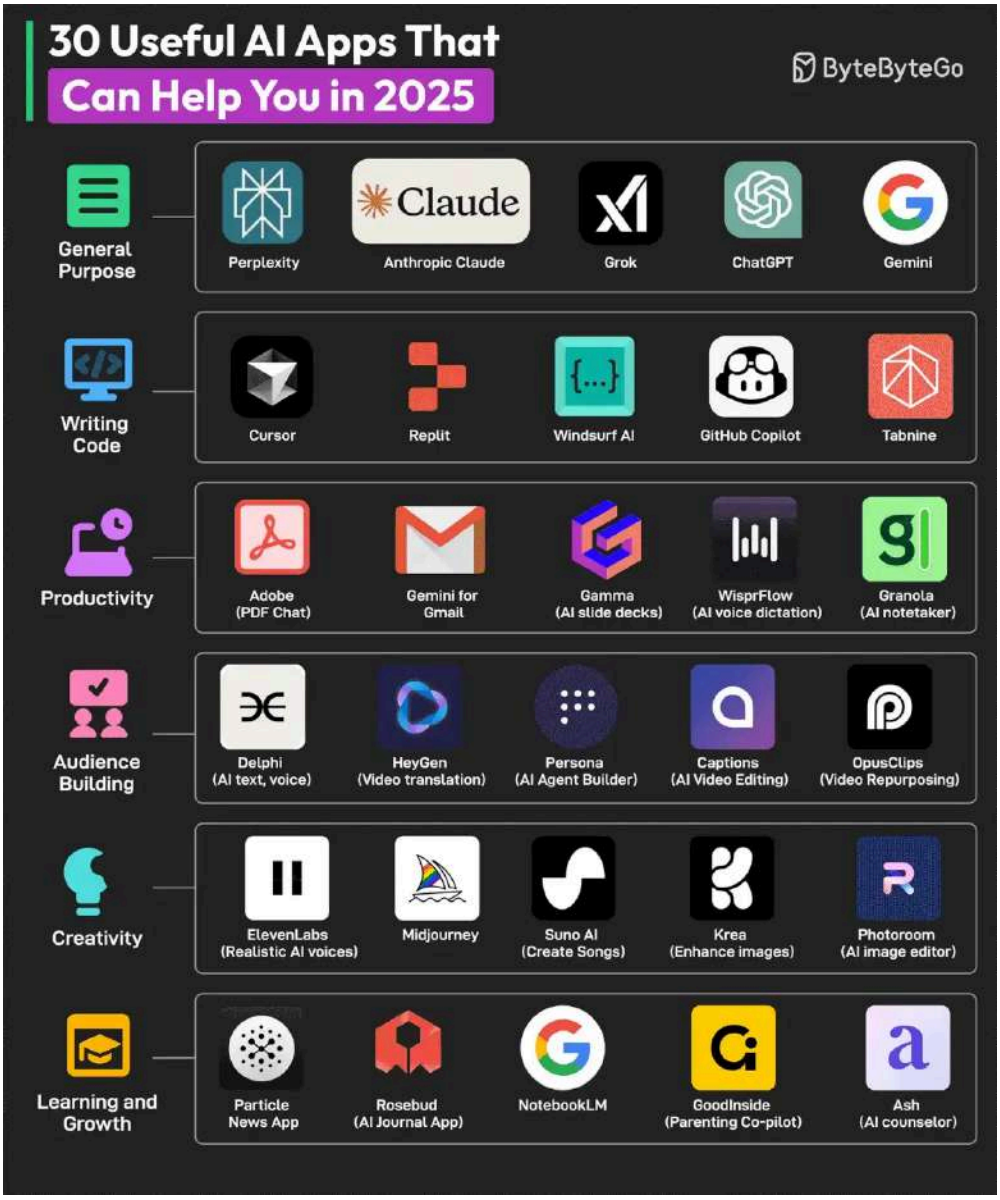
8. API Gateways
   Learn about API Gateways such as Amazon API Gateway, Azure API Services, Kong, Nginx, etc.

9. API Implementation Frameworks
   The most popular API development frameworks are Node.js, Spring, Flask, Django, and FastAPI

10. API Integration Patterns
    Learn about various API integration patterns such as gateways, event-driven, webhook, polling, and batch processing.

Over to you: What else will you add to the API learning roadmap?

30 Useful AI Apps That Can Help You in 2025



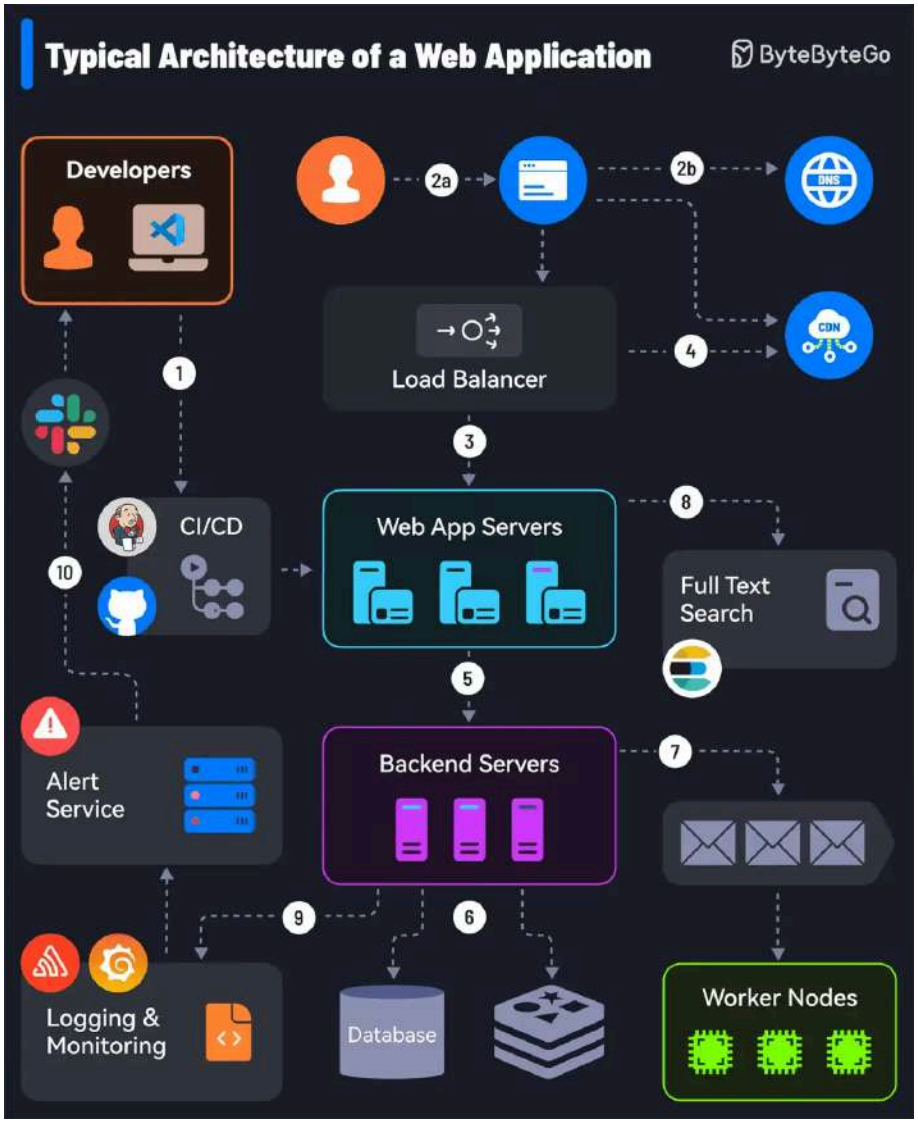
AI apps are taking over the world. There's an AI app for every conceivable use case. Here are some AI apps for different categories:

1. General Purpose: Perplexity, Anthropic Claude, Grok, ChatGPT, and Gemini
2. Writing Code: Cursor, Replit, Windsurf AI, Github Copilot, and Tabnine
3. Productivity: Adobe (PDF Chat), Gemini for Gmail, Gamma (AI slide deck), WisprFlow (AI voice dictation), and Granola (AI notetaker)
4. Audience Building: Delphi (AI text, voice), HeyGen (video translation), Persona (AI agent builder), Captions (AI video editing), and OpusClips (Video repurposing)

5. Creativity: ElevenLabs (realistic AI voices), Midjourney, Suno AI (music generation), Krea (enhance images), and Photoroom (AI image editing)
6. Learning and Growth: Particle News App, Rosebud (AI journal app), NotebookLM, GoodInside (parenting co-pilot), and Ash (AI counselor).

Over to you: Which other AI app will you add to the list?

10 Essential Components of a Production Web Application

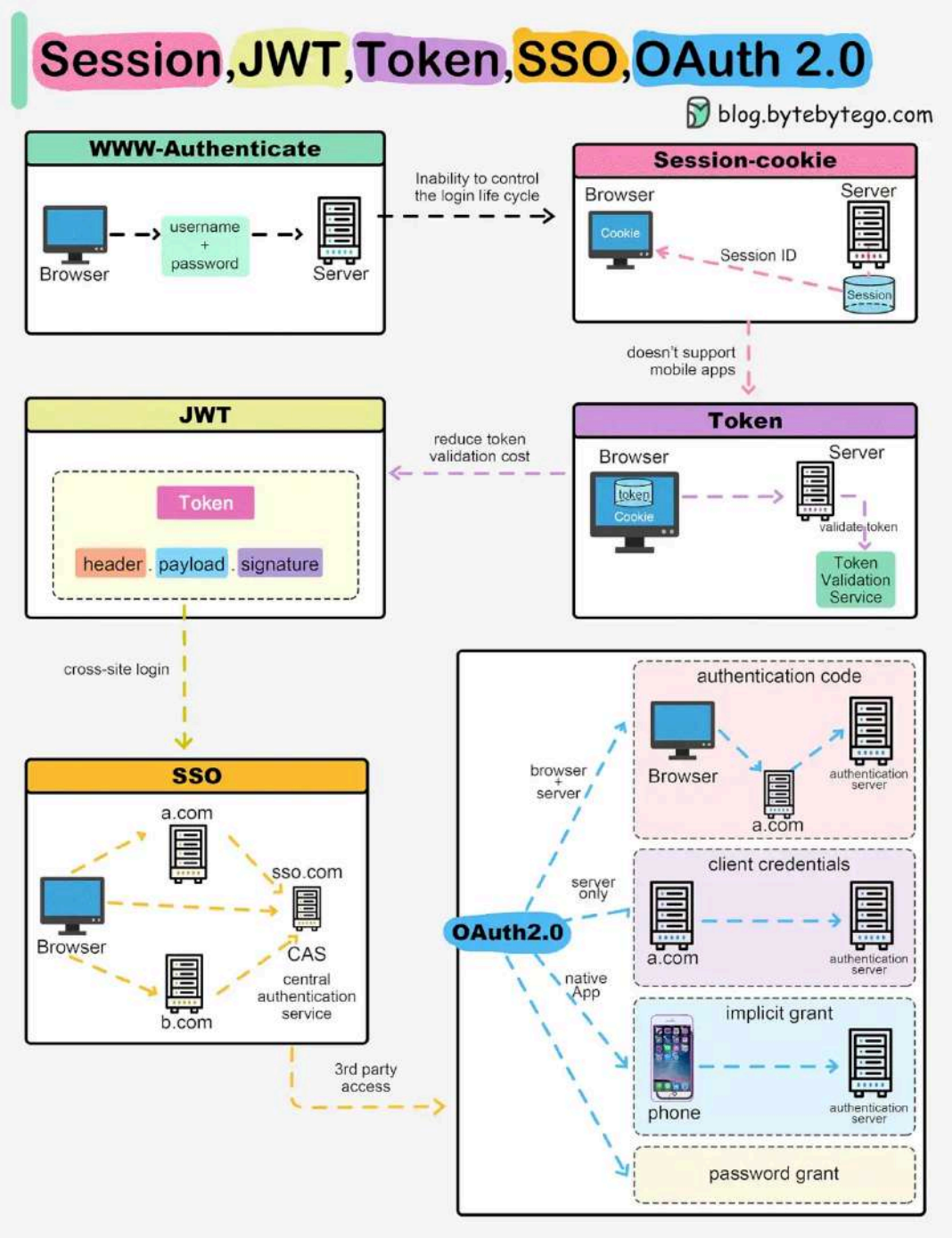


1. It all starts with CI/CD pipelines that deploy code to the server instances. Tools like Jenkins and GitHub help over here.
2. The user requests originate from the web browser. After DNS resolution, the requests reach the app servers.
3. Load balancers and reverse proxies (such as Nginx & HAProxy) distribute user requests evenly across the web application servers.
4. The requests can also be served by a Content Delivery Network (CDN).
5. The web app communicates with backend services via APIs.

6. The backend services interact with database servers or distributed caches to provide the data.
7. Resource-intensive and long-running tasks are sent to job workers using a job queue.
8. The full-text search service supports the search functionality. Tools like Elasticsearch and Apache Solr can help here.
9. Monitoring tools (such as Sentry, Grafana, and Prometheus) store logs and help analyze data to ensure everything works fine.
10. In case of issues, alerting services notify developers through platforms like Slack for quick resolution.

Over to you: What other components would you add to the architecture of a production web app?

Session, Cookie, JWT, Token, SSO, and OAuth 2.0 Explained in One Diagram

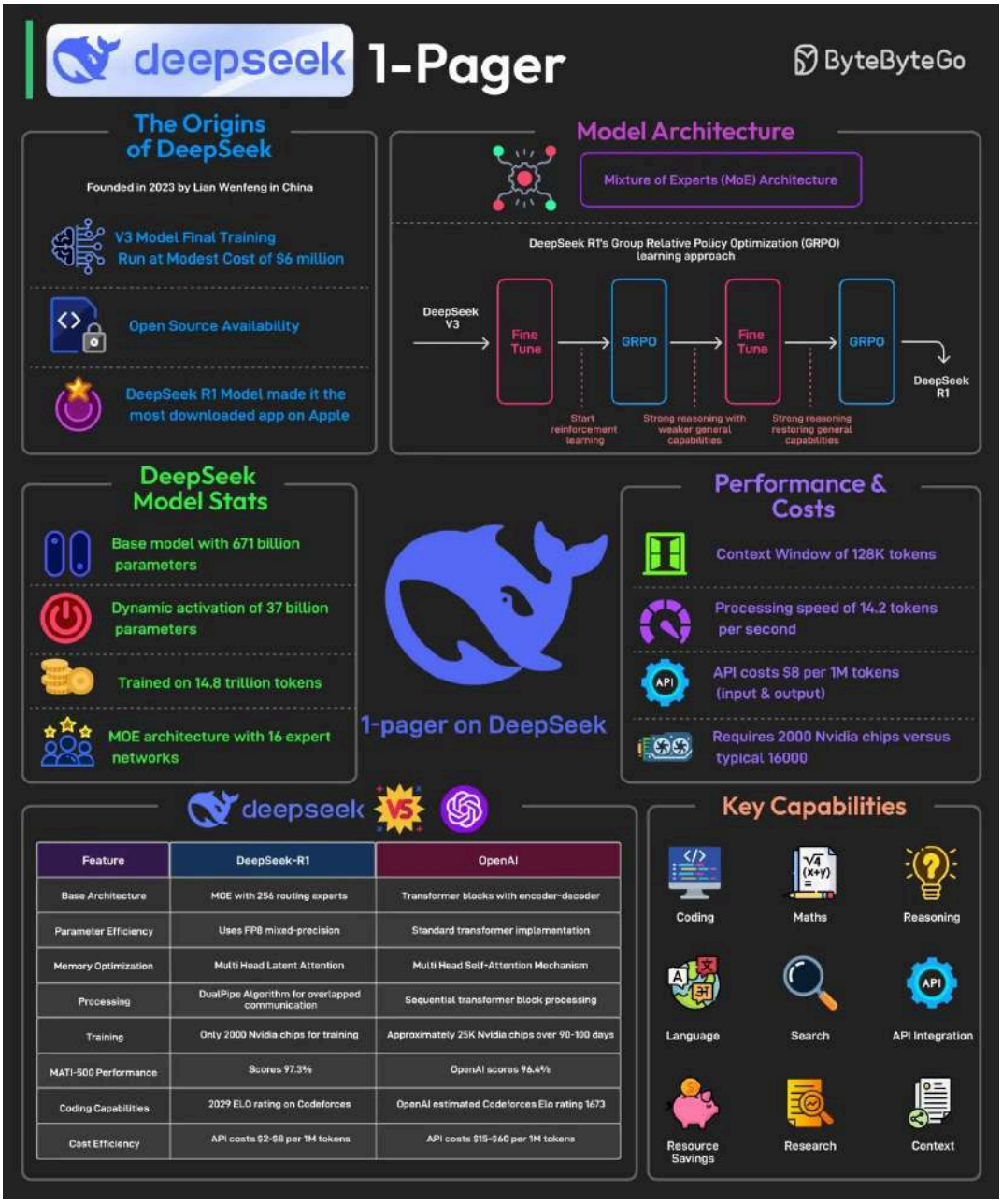


When you login to a website, your identity needs to be managed. Here is how different solutions work:

- Session - The server stores your identity and gives the browser a session ID cookie. This allows the server to track login state. But cookies don't work well across devices.
- Token - Your identity is encoded into a token sent to the browser. The browser sends this token on future requests for authentication. No server session storage is required. But tokens need encryption/decryption.
- JWT - JSON Web Tokens standardize identity tokens using digital signatures for trust. The signature is contained in the token so no server session is needed.
- SSO - Single Sign On uses a central authentication service. This allows a single login to work across multiple sites.
- OAuth2 - Allows limited access to your data on one site by another site, without giving away passwords.
- QR Code - Encodes a random token into a QR code for mobile login. Scanning the code logs you in without typing a password.

Over to you: QR code logins are gaining popularity. Do you know how it works?

DeepSeek 1-Pager



It is said to have developed a powerful AI model at a remarkably low cost, approximately $6 million for the final training run. In January 2025, it is said to have released its latest reasoning-focused model known as DeepSeek R1.

The release made it the No. 1 downloaded free app on the Apple Play Store.

Most AI models are trained using supervised fine-tuning, meaning they learn by mimicking large datasets of human-annotated examples. This method has limitations.

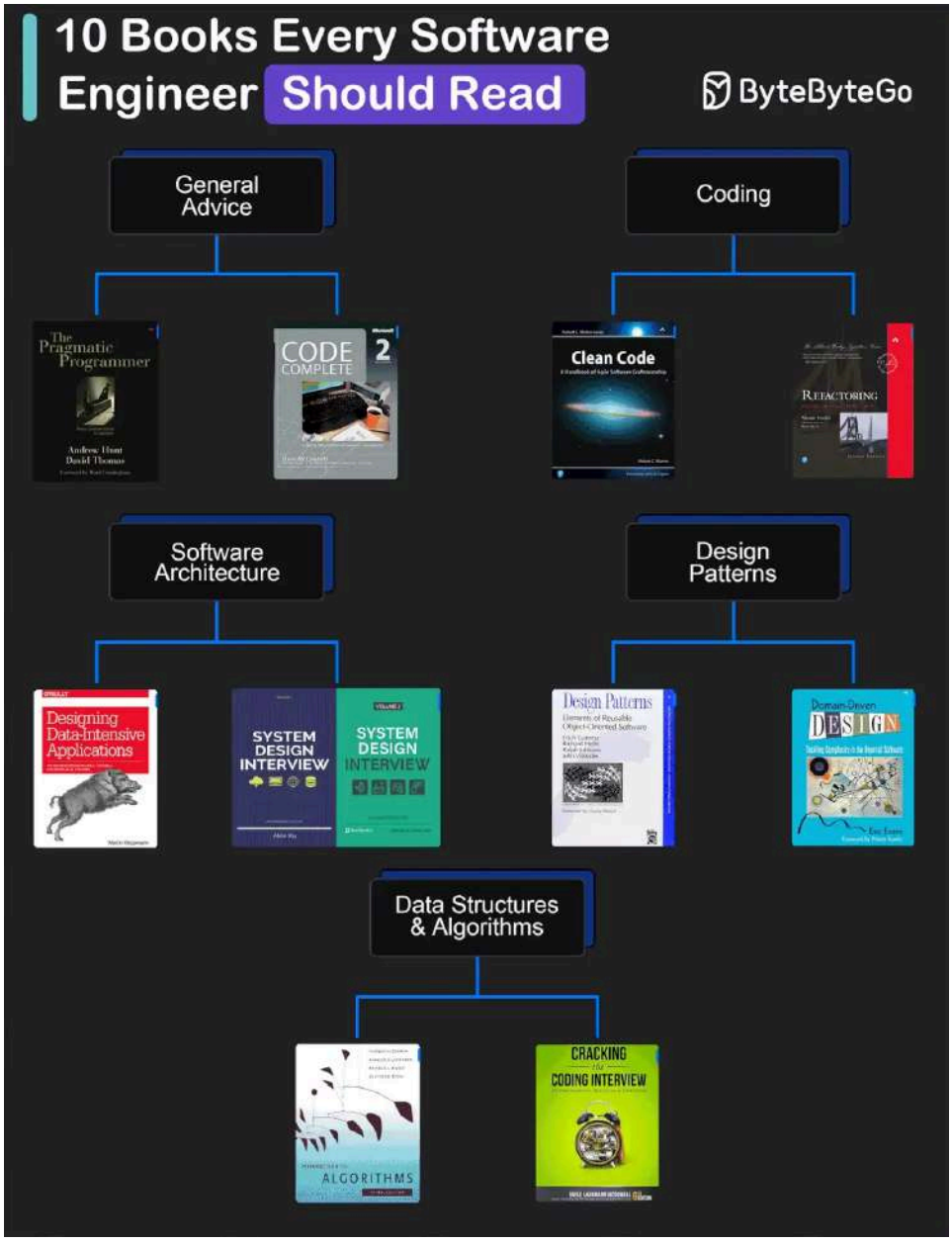
DeepSeek R1 overcomes these limitations by using Group Relative Policy Optimization (GRPO), a reinforcement learning technique that improves reasoning efficiency by comparing multiple possible answers within the same context.

Some facts about DeepSeek's R1 model are as follows:

1. DeepSeek-R1 uses a Mixture-of-Experts (MoE) architecture with 671 billion total parameters, activating only 37 billion parameters per task.
2. It employs selective parameter activation through MoE for resource optimization.
3. The model is pre-trained on 14.8 trillion tokens across 52 languages.
4. DeepSeek-R1 was trained using just 2000 Nvidia GPUs. By comparison, ChatGPT-4 needed approximately 25K Nvidia GPUs over 90-100 days.
5. The model is 85-90% more cost-effective than competitors.
6. It excels in mathematics, coding, and reasoning tasks.
7. Also, the model has been released as open-source under the MIT license.

Over to you: Have you tried DeepSeek?

My Favorite 10 Books for Software Developers



General Advice

1. The Pragmatic Programmer by Andrew Hunt and David Thomas
2. Code Complete by Steve McConnell: Often considered a bible for software developers, this comprehensive book covers all aspects of software development, from design and coding to testing and maintenance.

Coding

1. Clean Code by Robert C. Martin
2. Refactoring by Martin Fowler

Software Architecture

1. Designing Data-Intensive Applications by Martin Kleppmann
2. System Design Interview (our own book :))

Design Patterns

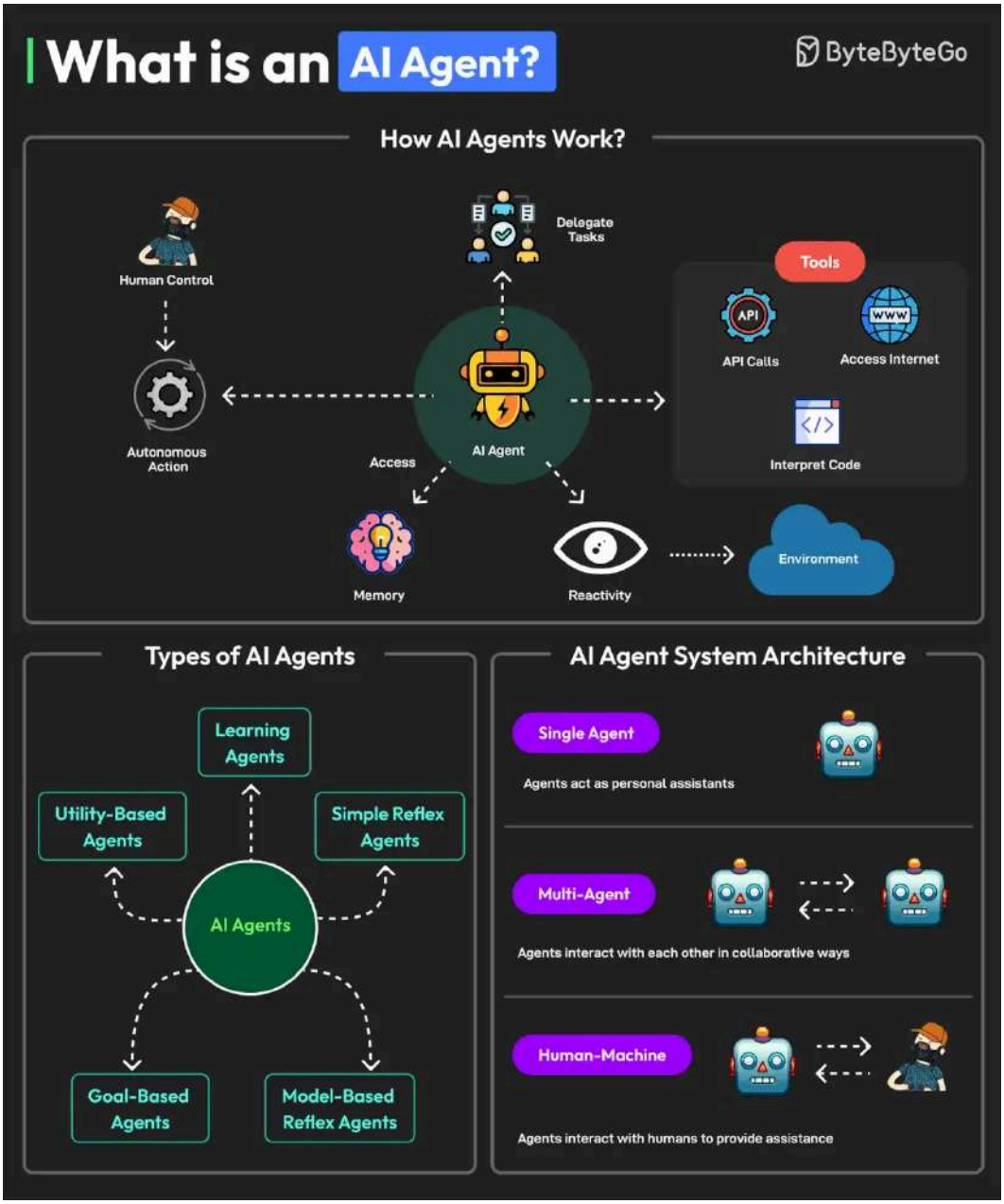
1. Design Patterns by Eric Gamma and Others
2. Domain-Driven Design by Eric Evans

Data Structures and Algorithms

1. Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein
2. Cracking the Coding Interview by Gayle Laakmann McDowell

Over to you: What is your favorite book?

What is an AI Agent?



An AI agent is a software program that can interact with its environment, gather data, and use that data to achieve predetermined goals. AI agents can choose the best actions to perform to meet those goals.

Key characteristics of AI agents are as follows:

An agent can perform autonomous actions without constant human intervention. Also, they can have a human in the loop to maintain control.

- Agents have a memory to store individual preferences and allow for personalization. It can also store knowledge. An LLM can undertake information processing and decision-making functions.
- Agents must be able to perceive and process the information available from their environment.
- Agents can also use tools such as accessing the internet, using code interpreters and making API calls.
- Agents can also collaborate with other agents or humans.

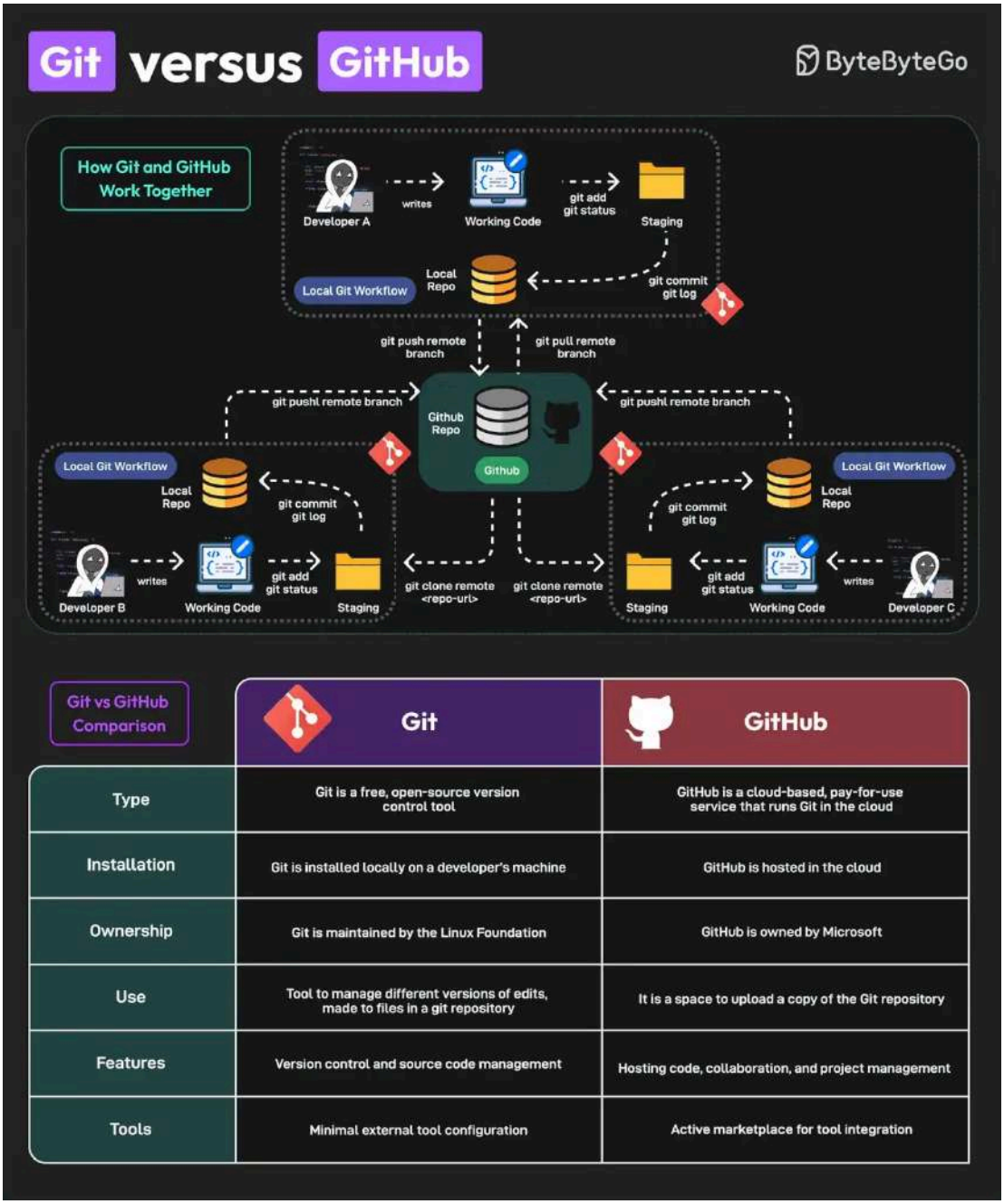
Multiple types of AI agents are available such as learning agents, simple reflex agents, model-based reflex agents, goal-based agents, and utility-based agents.

A system with AI agents can be built with different architectural approaches.

1. Single Agent: Agents can serve as personal assistants.
2. Multi-Agent: Agents can interact with each other in collaborative or competitive ways.
3. Human Machine: Agents can interact with humans to execute tasks more efficiently.

Over to you: Have you used AI Agents?

Git vs GitHub



Git and GitHub are popular tools for version control. They work together and complement each other to provide effective source control management.

On a high level, Git is focused on version control and code sharing, whereas GitHub is focused on centralized source code hosting for sharing with other developers.

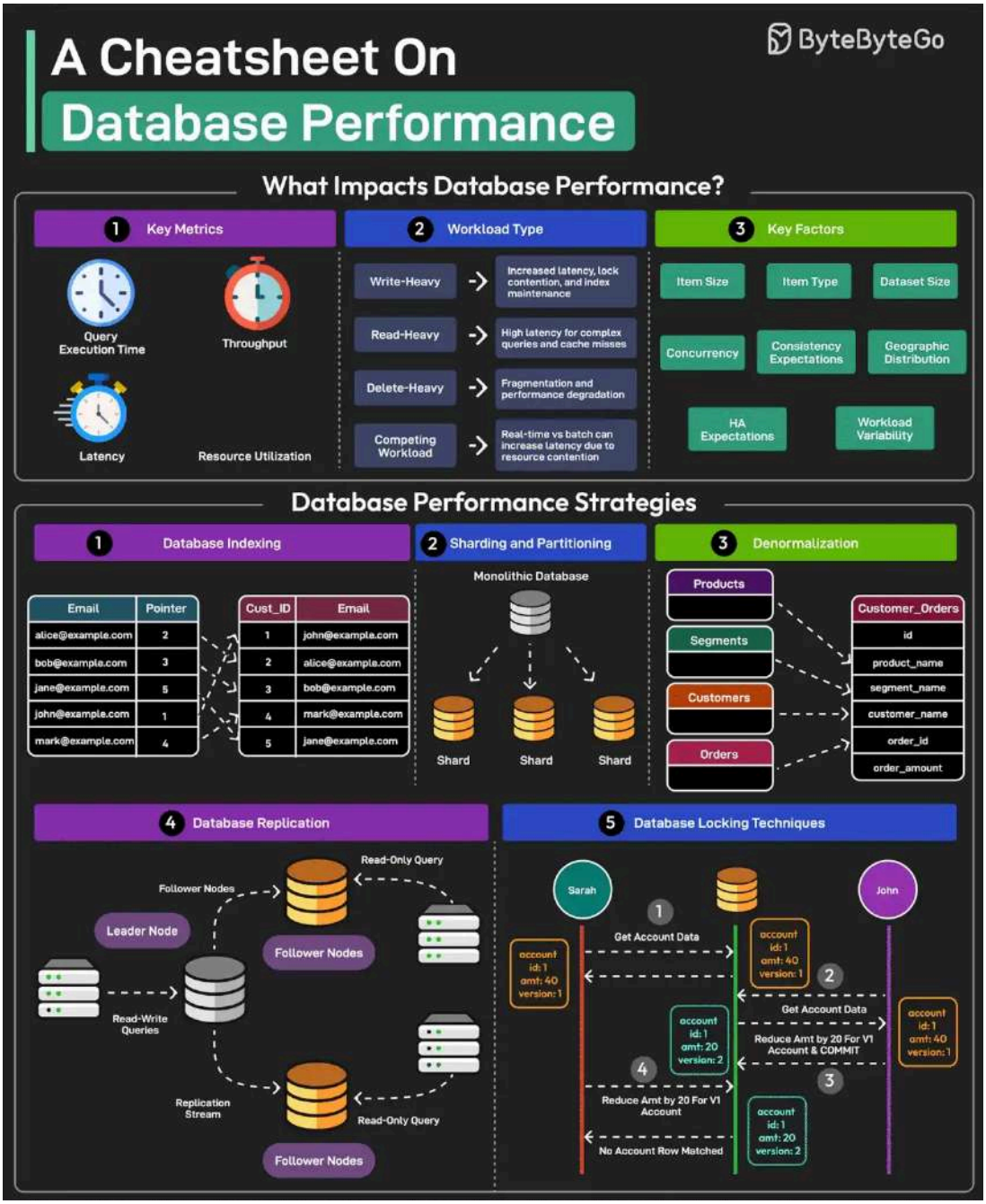
However, they have some key differences

1. Git is a free, open-source version control tool. GitHub is a cloud-based, pay-for-use service that runs Git in the cloud.
2. Git is installed locally on a developer's machine. GitHub is hosted in the cloud.
3. The Linux Foundation maintains Git. Microsoft owns GitHub.
4. Git can manage different versions of edits, made to files in a git repository. GitHub is a space to upload a copy of the Git repository.
5. Git supports version control and source code management. GitHub can be used for hosting code, collaboration, and project management.
6. Git has minimal external tool configuration. GitHub provides an active marketplace for tool integration.

Lastly, you can use Git without GitHub but you cannot use GitHub without Git.

Over to you: What else will you add to distinguish between Git and GitHub?

A Cheatsheet on Database Performance



Good database performance is critical since it directly impacts user experience, operational costs, and scalability.

But what impacts database performance?

Evaluating database performance depends on key metrics such as query execution time, throughput, latency, and resource utilization.

Workload Types such as write-heavy, read-heavy, delete-heavy, and competing workloads pose unique challenges.

Other factors that impact performance are item size, item type, dataset size, concurrency expectations, consistency requirements, HA expectations, and geographic distribution.

Multiple strategies exist to improve database performance. Some of the most important ones are as follows:

1. Database Indexing
   Indexes are important for speeding up database queries by reducing the amount of data scanned. Also, choosing the right index type is crucial.

2. Sharding and Partitioning
   Divide the data into smaller, more manageable chunks known as shards. Each shard is also stored on a different server.

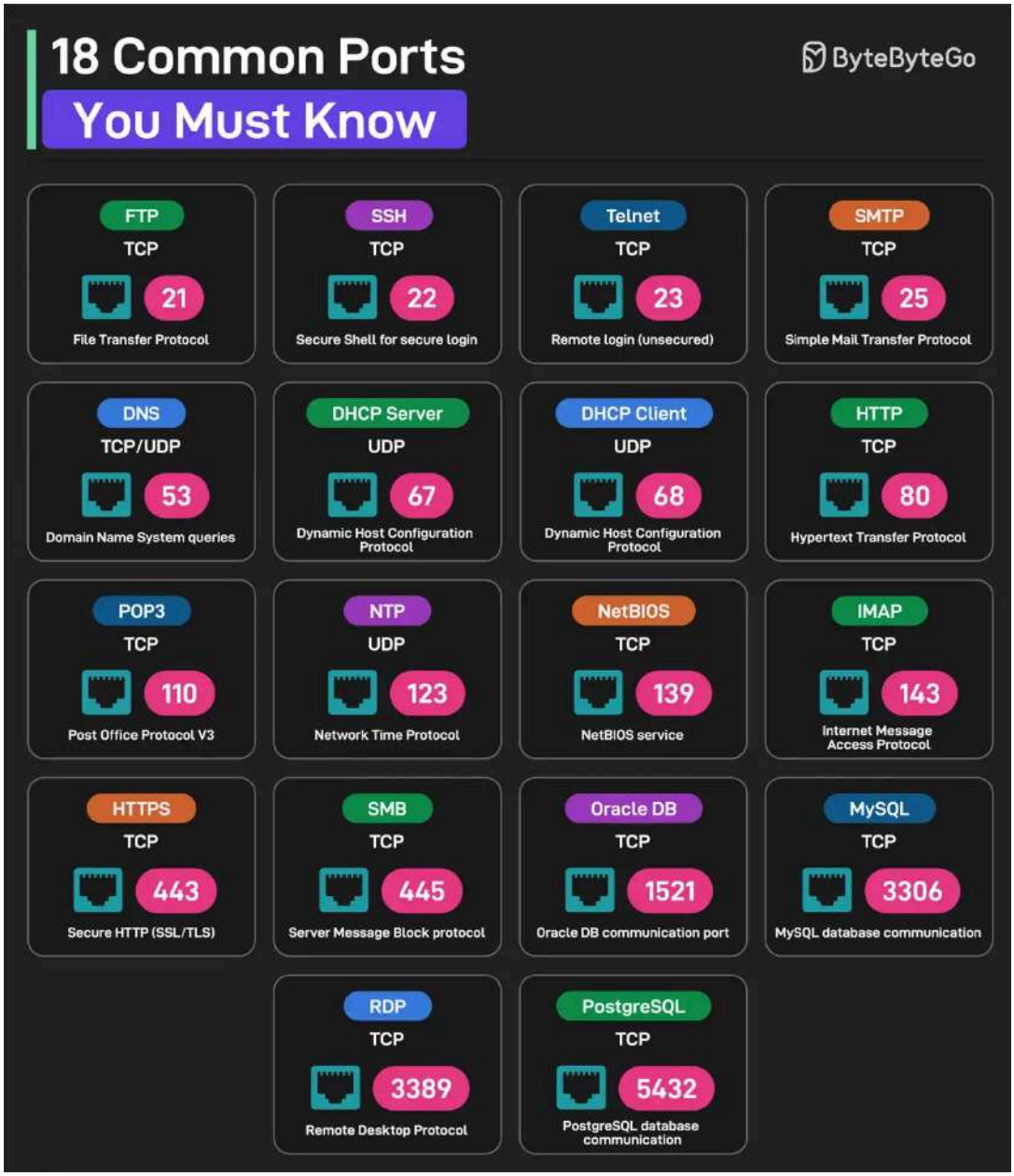
3. Denormalization
   Denormalization combines data into fewer tables to reduce the overhead of joins, improving read performance.

4. Database Replication
   Replication involves maintaining multiple copies of the same database, typically with a primary node for writes (and critical reads) and secondary nodes for most read operations.

5. Database Locking Techniques
   Use locking techniques like pessimistic and optimistic locking to manage concurrency levels and resource contention.

Over to you: Which other database performance strategy will you add to the list?

18 Common Ports Worth Knowing

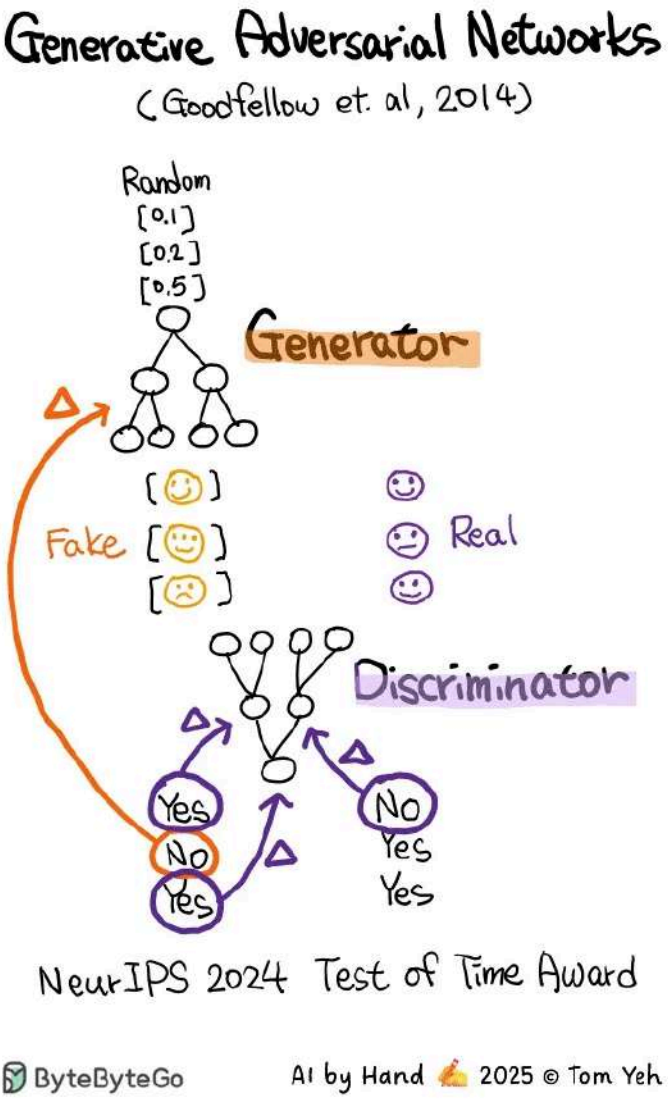


1. FTP (File Transfer Protocol): Uses TCP Port 21
2. SSH (Secure Sheel for Login): Uses TCP Port 22
3. Telnet: Uses TCP Port 23 for remote login
4. SMTP (Simple Mail Transfer Protocol): Uses TCP Port 25
5. DNS: Uses UDP or TCP on Port 53 for DNS queries

6. DHCP Server: Uses UDP Port 67
7. DHCTP Client: Uses UDP Port 68
8. HTTP (Hypertext Transfer Protocol): Uses TCP Port 80
9. POP3 (Post Office Protocol V3): Uses TCP Port 110
10. NTP (Network Time Protocol): Uses UDP Port 123
11. NetBIOS: Uses TCP Port 139 for NetBIOS service
12. IMAP (Internet Message Access Protocol): Uses TCP Port 139
13. HTTPS (Secure HTTP): Uses TCP Port 443
14. SMB (Server Message Block): Uses TCP Port 445
15. Oracle DB: Uses TCP Port 1521 for Oracle database communication port
16. MySQL: Uses TCP Port 3360 for MySQL database communication port
17. RDP: Uses TCP Port 3389 for Remote Desktop Protocol
18. PostgreSQL: Uses TCP Port 5432 for PostgreSQL database communication

Over to you: Which other port will you add to the list?

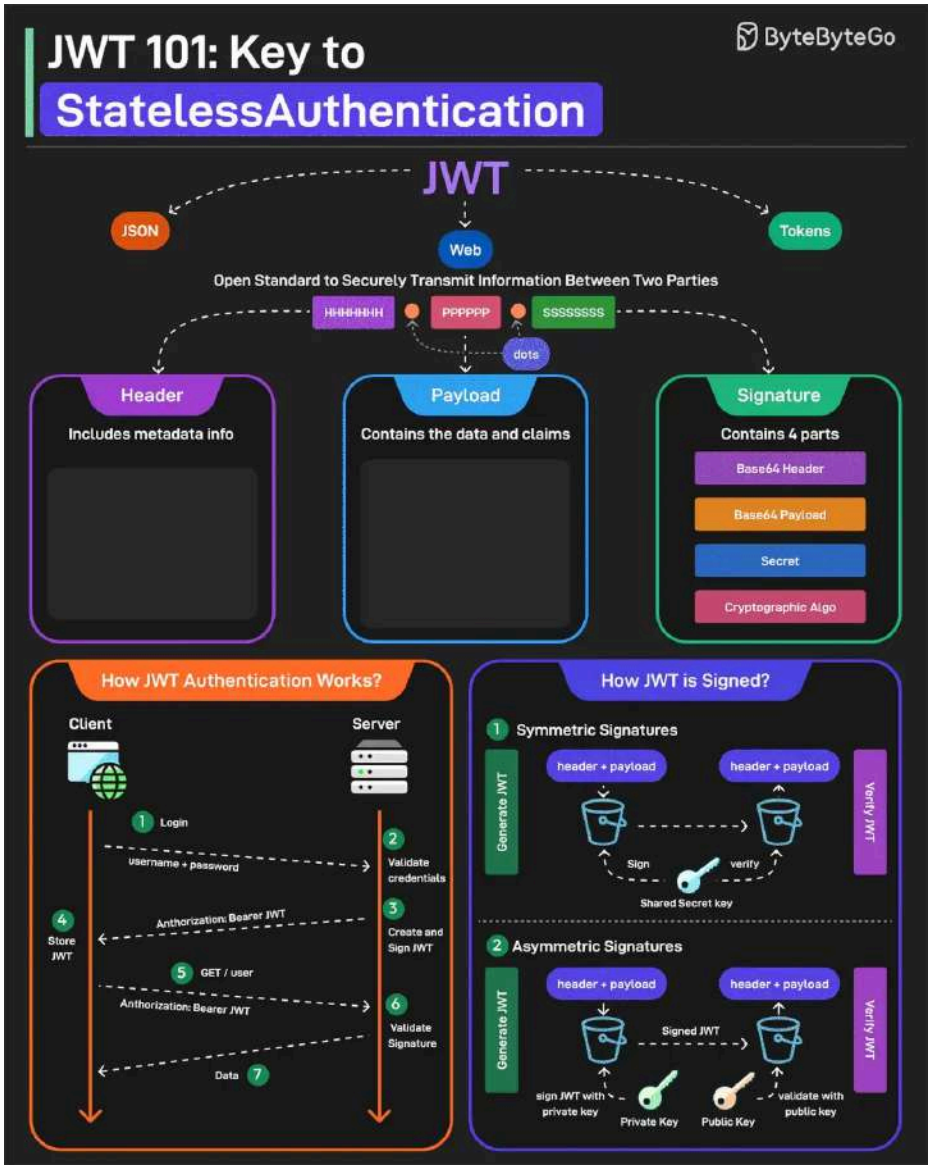
Generative Adversarial Network (GAN) - AI by Hand ✍️



This animation is hand drawn by Prof. Tom Yeh exclusively for the readers of the ByteByteGo Newsletter..

If you want to learn more about GAN, check out Chapter 7 in the System Design Interview book.

JWT 101: Key to Stateless Authentication



JWT or JSON Web Tokens is an open standard for securely transmitting information between two parties. They are widely used for authentication and authorization.

A JWT consists of three main components:

1. Header
   Every JWT carries a header specifying the algorithms for signing the JWT. It's written in JSON format.

2. Payload
   The payload consists of the claims and the user data. There are different types of claims such as registered, public, and private claims.

3. Signature
   The signature is what makes the JWT secure. It is created by taking the encoded header, encoded payload, secret key, and the algorithm and signing it.

JWTs can be signed in two different ways:

1. Symmetric Signatures
   It uses a single secret key for both signing the token and verifying it. The same key must be shared between the server that signs the JWT and the system that verifies it.

2. Asymmetric Signatures
   In this case, a private key is used to sign the token, and a public key to verify it. The private key is kept secure on the server, while the public key can be distributed to anyone who needs to verify the token.

Over to you: Do you use JWTs for authentication?

A handy cheat sheet for the most popular cloud services

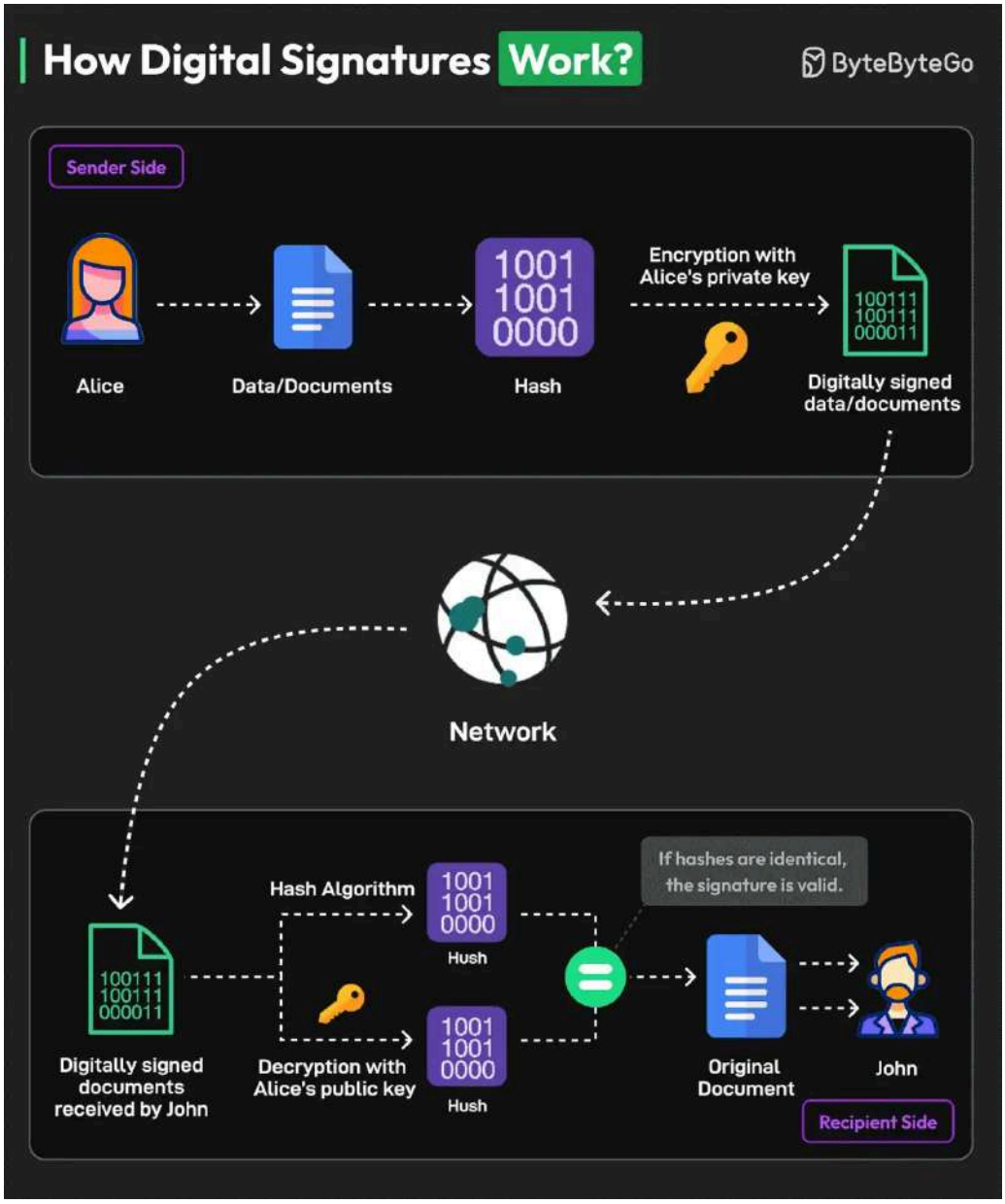


What’s included?

- AWS, Azure, Google Cloud, Oracle Cloud, Alibaba Cloud
- Cloud servers
- Databases
- Message queues and streaming platforms
- Load balancing, DNS routing software
- Security
- Monitoring

Over to you - which company is the best at naming things?

How Digital Signatures Work?



A digital signature is a specific kind of electronic signature to sign and secure electronically transmitted documents.

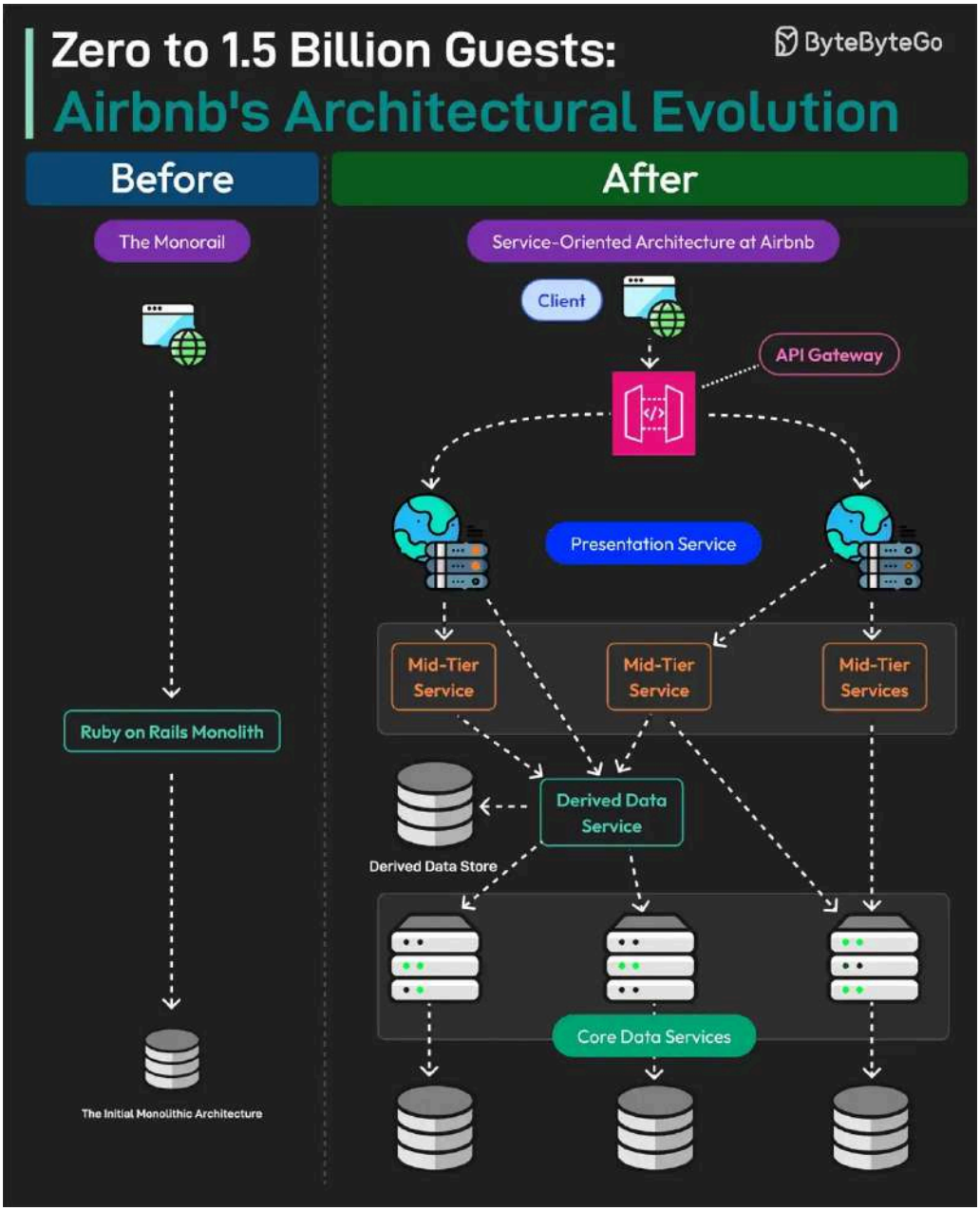
Digital signatures are similar to physical signatures since they are unique to every person. They identify the identity of the signer.

Here's an example of the working process of a digital signature with Alice as the sender and John as the recipient:

1. Alice generates a cryptographic key pair consisting of a private key and a corresponding public key. The private key remains confidential and is known only to the signer, while the public key can be shared openly.
2. The signer (Alice) uses a hash function to create a unique fixed-length string of numbers and letters, called a hash, from the document. This hash value represents the contents of the document.
3. Alice uses their private key to encrypt the hash value of the message. This hash value is known as the digital signature.
4. The digital signature is attached to the original document, creating a digitally signed document. It is transmitted over the network to the recipient.
5. The recipient (John) extracts both the digital signature and the original hash value from the document.
6. The recipient uses Alice's public key to decrypt the digital signature. This produces a hash value that was originally encrypted with the private key.
7. The recipient calculates a new hash value for the received message using the same hashing algorithm as the signer. They then compare this recalculated hash with the decrypted hash value obtained from the digital signature.
8. If the hash values are equal, the digital signature is valid, and it is determined that the document has not been tampered with or altered.

Over to you: Have you used digital signatures?

0 to 1.5 Billion Guests: Airbnb’s Architectural Evolution



Airbnb operates in 200+ countries and has helped 4 million hosts welcome over 1.5 billion guests across the world.

What powers Airbnb technically?

Airbnb started as a monolithic application. It was built using Ruby-on-Rails and was internally known as the Monorail.

The monolith was a single-tier unit responsible for both client and server-side functionality.

However, as Airbnb went into hypergrowth, the Monorail started facing issues. This is when they began a migration journey to move from monolithic to Service-Oriented Architecture.

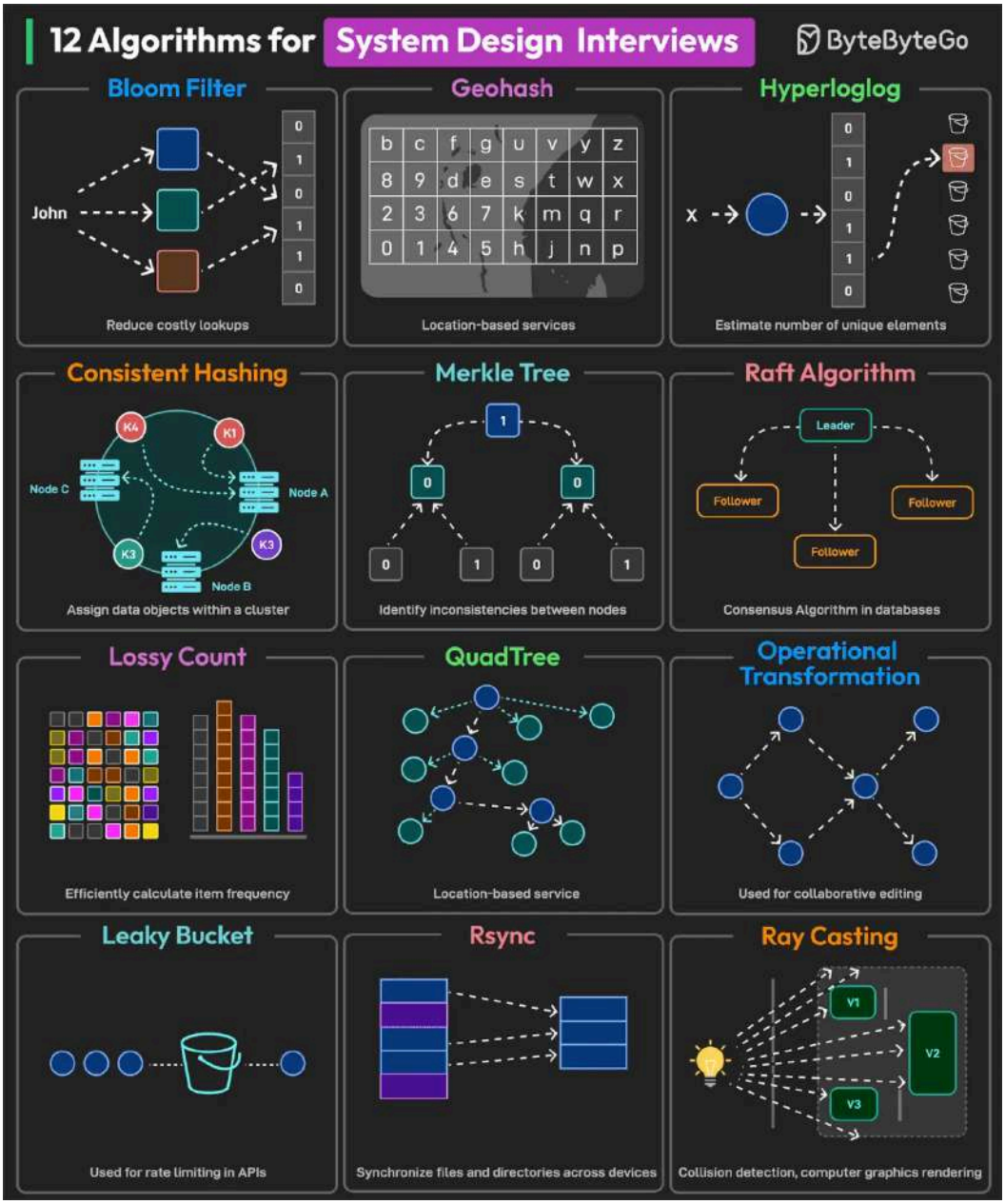
For Airbnb, SOA is a network of loosely coupled services where clients make their requests to a gateway and the gateway routes these requests to multiple services and databases.

Various types of services were built such as:

1. Data Service: This is the bottom layer and acts as the entry point for all read and write operations on the data entities.
2. Derived Data Service: These services read from data services and apply basic business logic.
3. Middle Tier Service: They manage important business logic that doesn't fit at the data service level or derived data service level.
4. Presentation Service: They aggregate data from all other services and also apply some frontend-specific business logic.

After the migration, the Monorail was eliminated and all reads/writes were migrated to the new services.

12 Algorithms for System Design Interviews

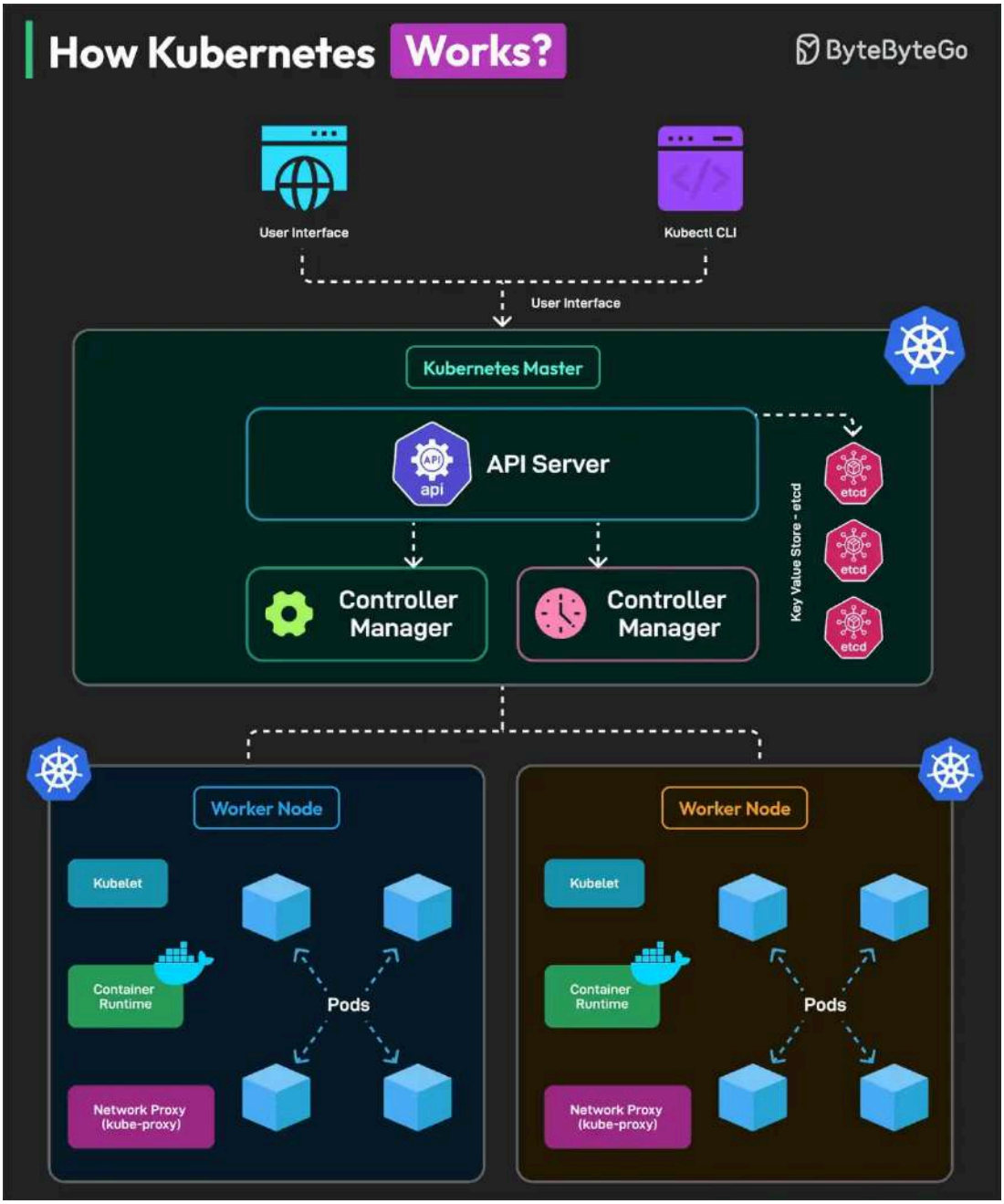


1. Bloom Filter: Check if a requested item is in the cache before performing disk operations.
2. Geohash: Used to build location-based services.
3. HyperLogLog: Counting unique queries performed by users in a search.
4. Consistent Hashing: Used for efficient data distribution between the cluster's nodes.
5. Merkle Tree: Used to detect inconsistencies between data replicas across multiple nodes.

6. Raft Algorithm: Used to achieve consensus on log replication.
7. Lossy Count: Network traffic analysis, web analytics, and identifying heavy hitters.
8. QuadTree: Used to build location-based services.
9. Operational Transformation: Used to support collaborative editing systems.
10. Leaky Bucket: Used for rate limiting
11. Rsync: Synchronizing files and directories between two different systems.
12. Ray Casting: Used for geospatial analysis, collision detection in video games, and computer graphics.

Over to you: Which other algorithm will you add to the list?

How Kubernetes Works?



Kubernetes (K8S) is an open-source container orchestration platform originally developed by Google and now maintained by the Cloud Native Computing Foundation (CNCF).

Here's how developers interact with Kubernetes:

1. Developers create manifest files describing the application.

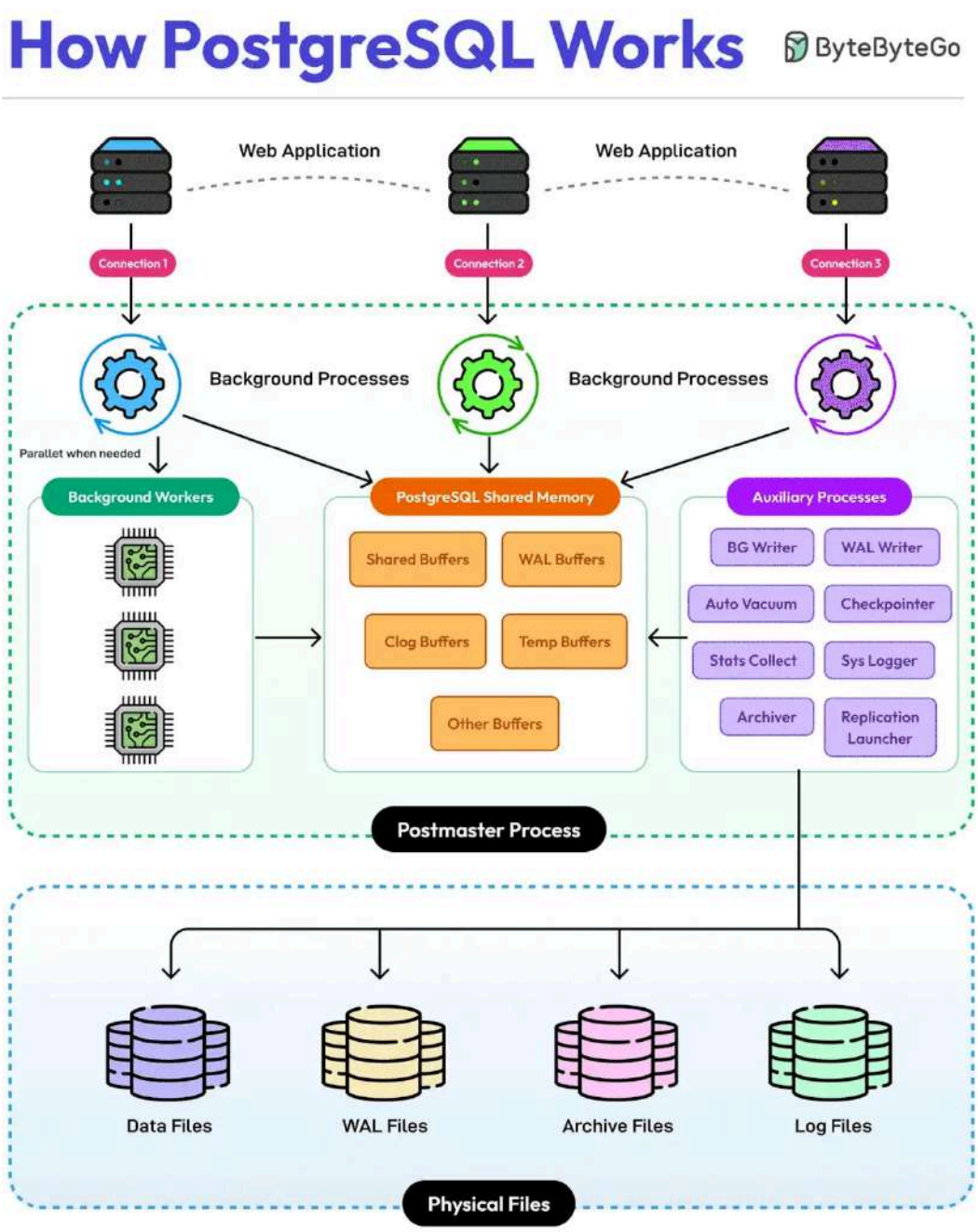
2. Kubernetes takes these manifest files, validates them, and deploys the applications across its cluster of worker nodes.
3. Kubernetes manages the entire lifecycle of the application.

Kubernetes is made up of two main components:

1. Control Plane: It is like the brain of Kubernetes and consists of the following parts:
    - API Server: It receives all incoming requests from users or CLI.
    - Scheduler: It decides where to run new pods based on resource availability and constraints.
    - Controller Manager: It ensures that the cluster's desired state matches the actual state.
    - Etcd: The key-value store where Kubernetes keeps all of its data, including configurations, cluster states, and desired states of applications.
2. Worker Nodes: They are the workers in a cluster. Each node has the below sub-components:
    - Kubelet: Main worker on each node. It communicates with the API server, gets instructions, and ensures that the containers are running as expected.
    - Kube-proxy: Handles networking on each node.
    - Container runtime

Over to you: Have you used Kubernetes for your projects?

PostgreSQL 101: The Everything Database



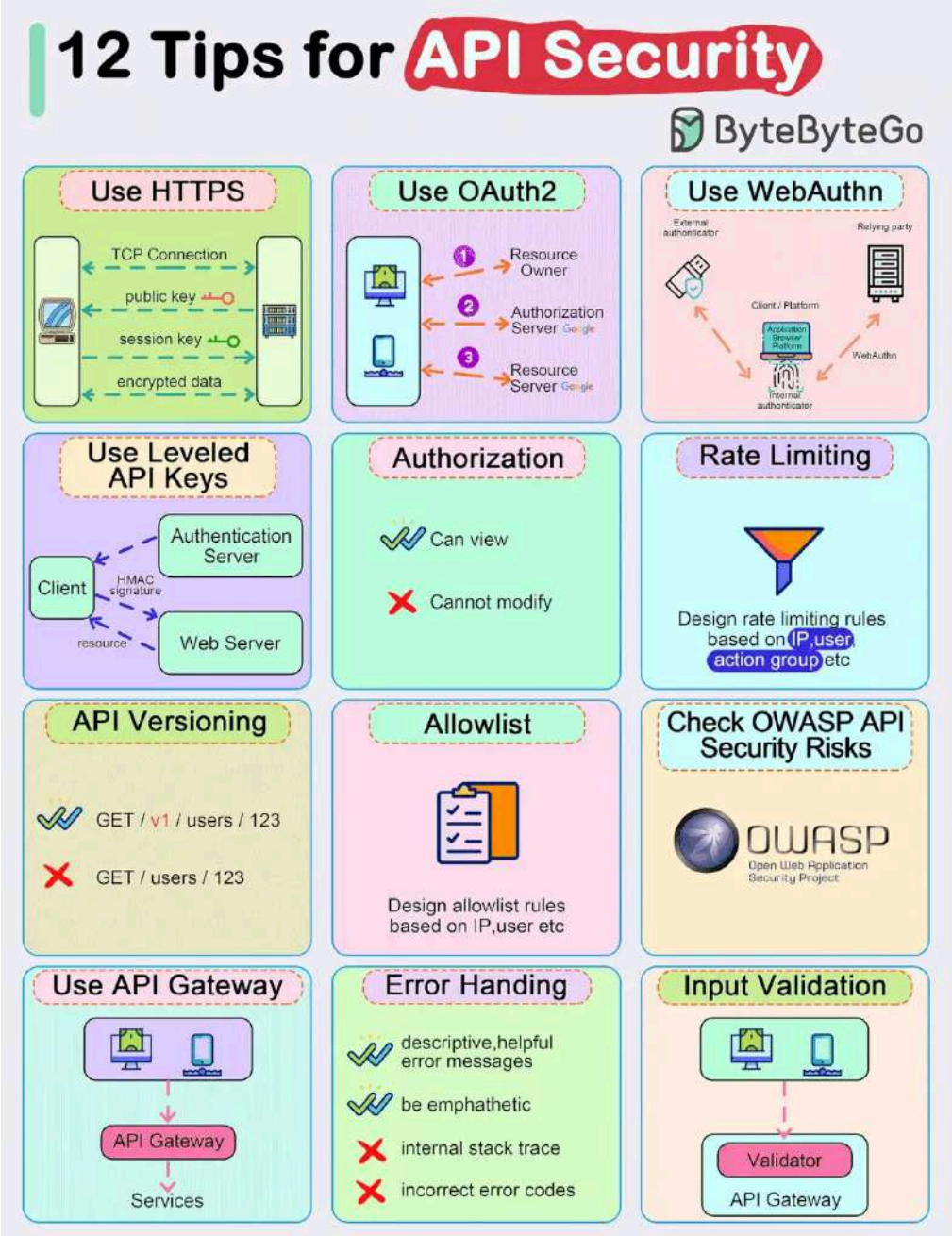
Built using C language, PostgreSQL is the most popular choice of database from small web apps to enterprise systems. It runs as a multiprocess system and follows ACID principles.

The key points about PostgreSQL's Architecture are as follows:

1. PostgreSQL supports concurrent client connections independently. Each client connection to PostgreSQL creates a dedicated server process.
2. The Postmaster Process is the main supervisor that manages all other PostgreSQL processes. It controls the entire database instance.
3. Background workers run parallel processes when needed to handle specialized tasks.
4. PostgreSQL shared memory is a central memory area containing multiple buffers such as Shared, WAL, Clog, and Temporary buffers. All components communicate through this shared memory.
5. PostgreSQL also has several auxiliary processes such as:
    - BG Writer: Manages background writing
    - WAL Writer: Handles write-ahead logging
    - Auto Vacuum: Maintains database cleanliness
    - Checkpointer: Ensures data consistency
    - Stats Collector: Gathers statistics
    - System Logger: Manages Logging
    - Archiver: Handles archiving
    - Replication launcher: Manages replication
6. PostgreSQL has different types of physical files for varied needs such as:
    - Data Files: Stores actual database data
    - WAL Files: Write-ahead log storage
    - Archive Files: Backup and recovery data
    - Log Files: System and error logs

Over to you: What else would you add to understand more about PostgreSQL's architecture?

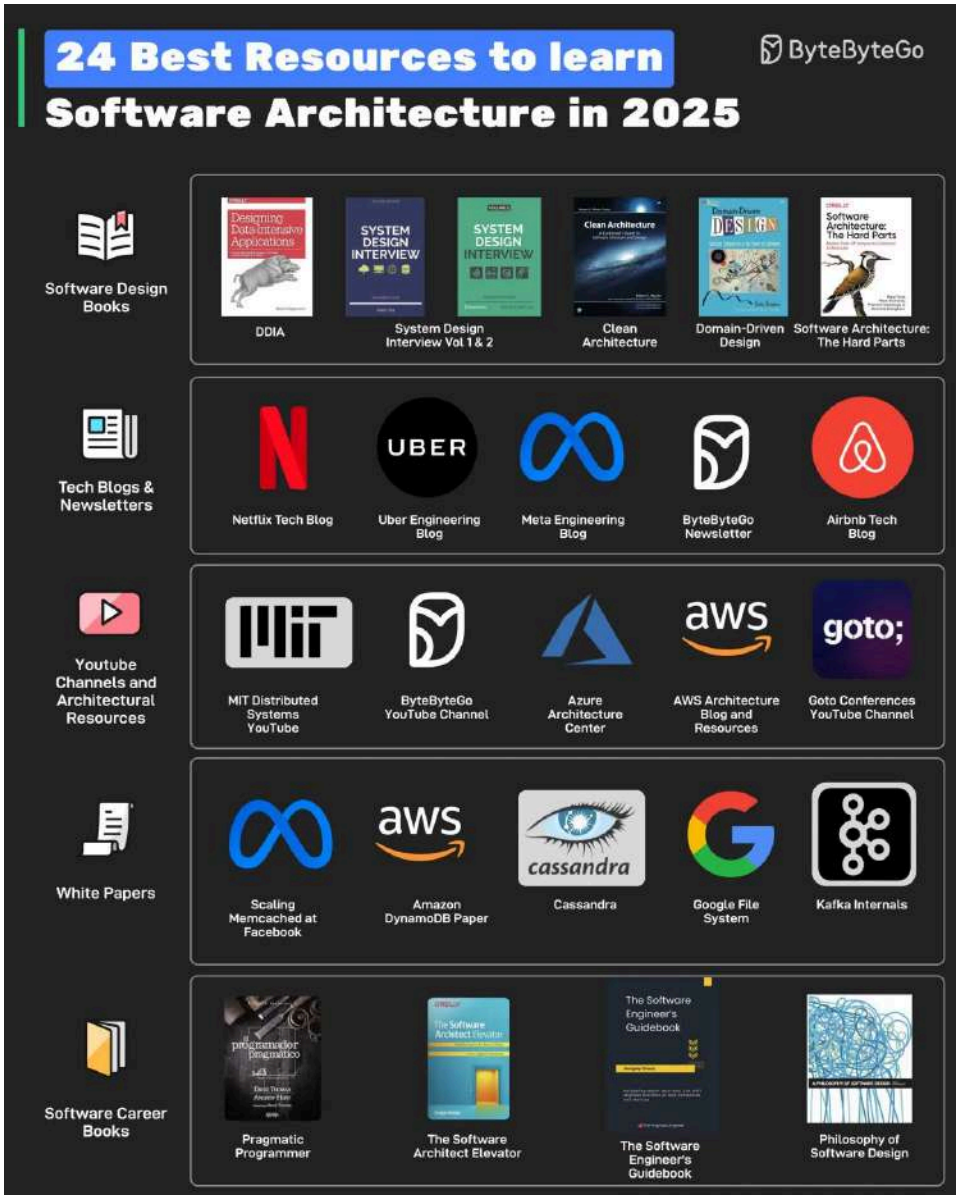
Top 12 Tips for API Security



- Use HTTPS
- Use OAuth2
- Use WebAuthn
- Use Leveled API Keys

- Authorization
- Rate Limiting
- API Versioning
- Whitelisting
- Check OWASP API Security Risks
- Use API Gateway
- Error Handling
- Input Validation

24 Good Resources to Learn Software Architecture in 2025



The resources can be divided into different types such as:

1. Software Design Books
   Some books that can help are DDIA, System Design Volume 1 & 2, Clean Architecture, Domain-Driven Design, and Software Architecture: the Hard Parts

2. Tech Blogs and Newsletters
   Read technical blogs by companies like Netflix, Uber, Meta, and Airbnb. Also, the ByteByteGo newsletter provides insights into software design every week.

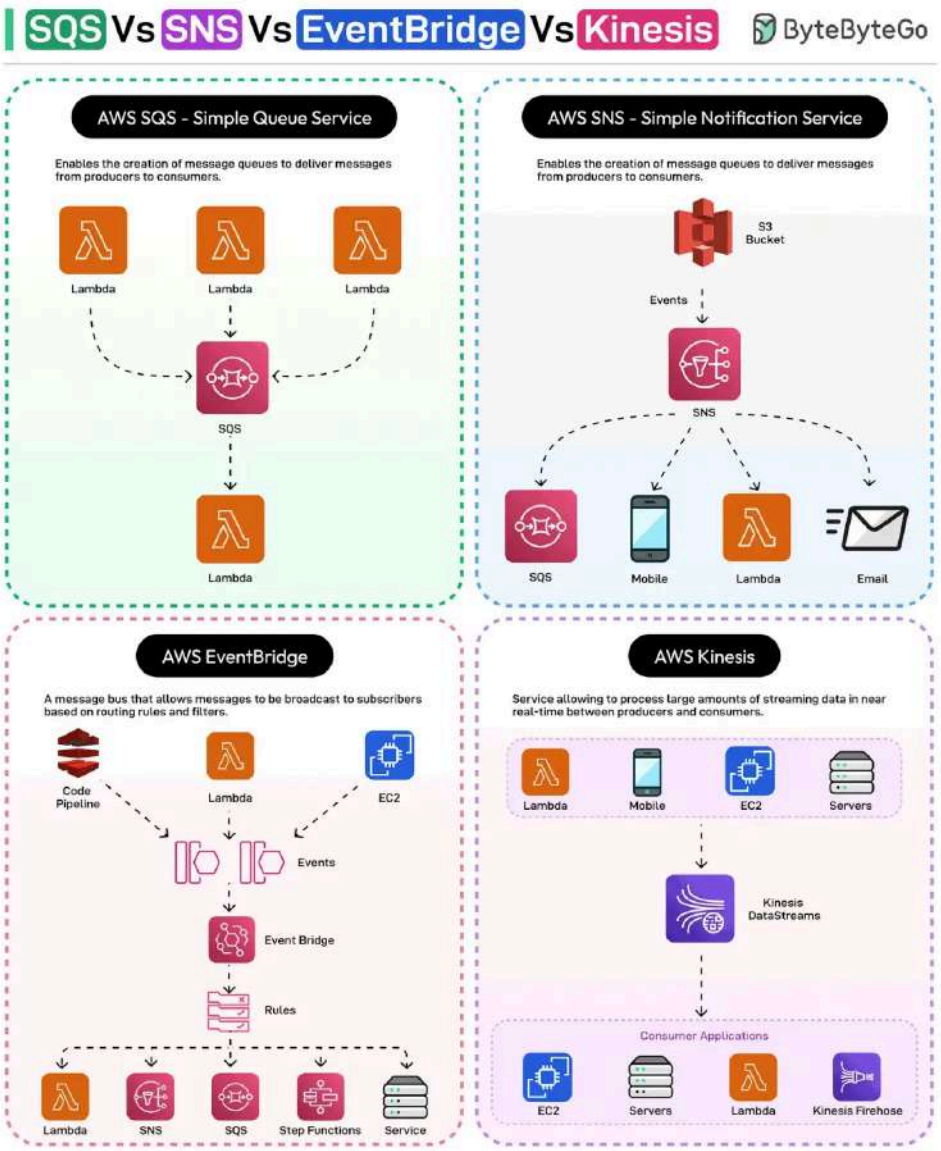
3. YouTube Channels and Architectural Resources
   YouTube channels like MIT Distributed Systems, Goto Conferences, and ByteByteGo can help with software architecture and system design. Azure Architecture Center and AWS Architecture Blog are other important resources.

4. WhitePapers
   For deeper insights, read whitepapers like Facebook Memcache Scaling, Cassandra, Amazon DynamoDB, Kafka, and Google File System.

5. Software Career Books
   A Software Architect also needs to develop holistic skills. Books about software career aspects such as Pragmatic Programmer, The Software Architect Elevator, The Software Engineer's Guidebook, and Philosophy of Software Design can help.

Over to you: Which other resources will you add to the list?

SQS vs SNS vs EventBridge vs Kinesis



1. AWS SQS: It is a fully managed message queuing service that helps decouple communication between distributed components.
2. AWS SNS: A fully managed publish/subscribe messaging service using topics to send messages to multiple subscribers or endpoints (such as email, Lambda, SQS, and so on).
3. AWS EventBridge: It is a serverless event bus that facilitates the routing of events between AWS services, and applications based on specific routing rules and filters.

4. AWS Kinesis Data Streams: A real-time data streaming service that captures, processes, and analyzes data in near real-time.

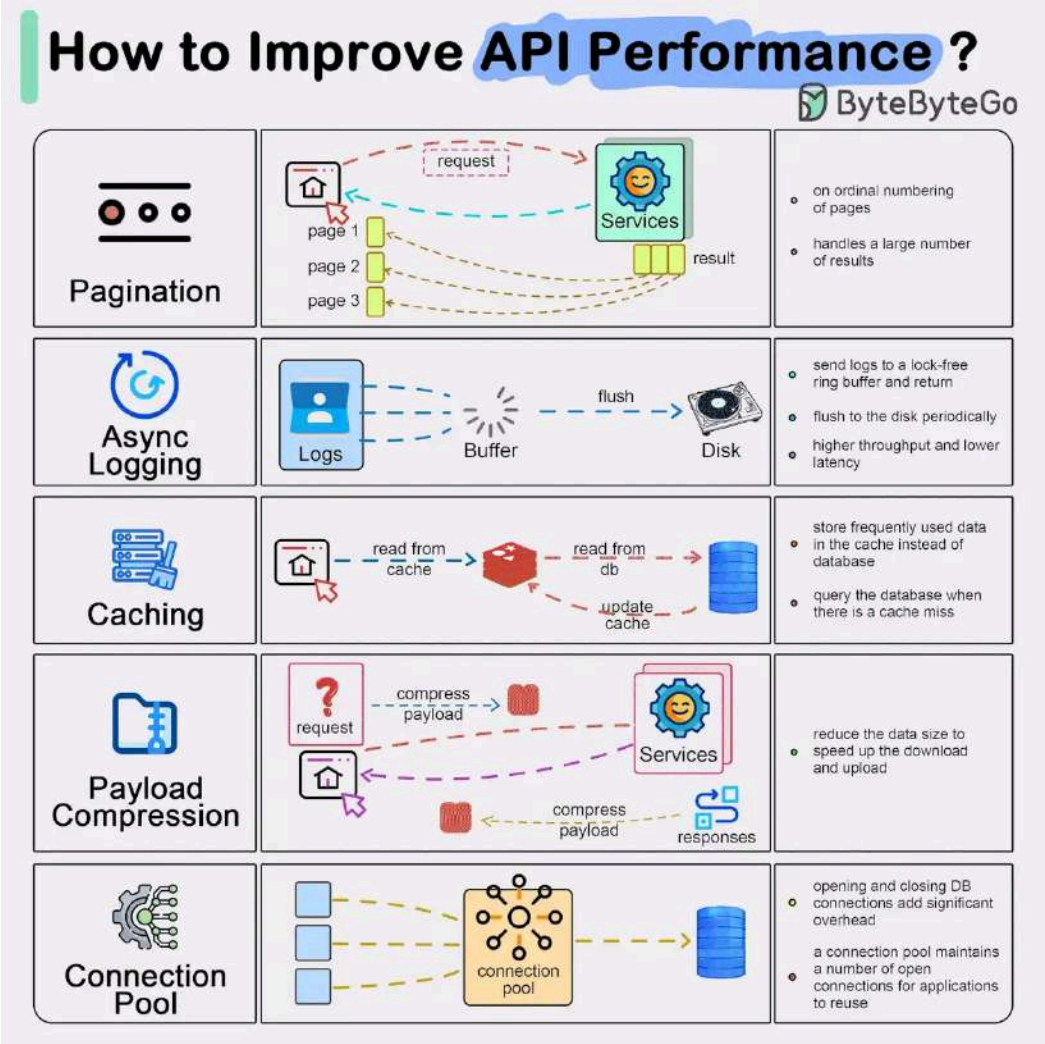
How can you choose which service to use?

Here are some basic tips and examples:

- For massive amounts of events like a clickstream from an application use Kinesis Data Stream
- For events processed by a single consumer like an image processing job, SQS is a better choice.
- For sending messages and notifications, SNS is a great choice.
- For system events such as a new user registration that multiple services must receive and react to, use EventBridge.

Over to you: Have you used SQS, SNS, EventBridge, or Kinesis?

Top 5 common ways to improve API performance



Result Pagination:

This method is used to optimize large result sets by streaming them back to the client, enhancing service responsiveness and user experience.

Asynchronous Logging:

This approach involves sending logs to a lock-free buffer and returning immediately, rather than dealing with the disk on every call. Logs are periodically flushed to the disk, significantly reducing I/O overhead.

Data Caching:

Frequently accessed data can be stored in a cache to speed up retrieval. Clients check the cache before querying the database, with data storage solutions like Redis offering faster access due to in-memory storage.

Payload Compression:

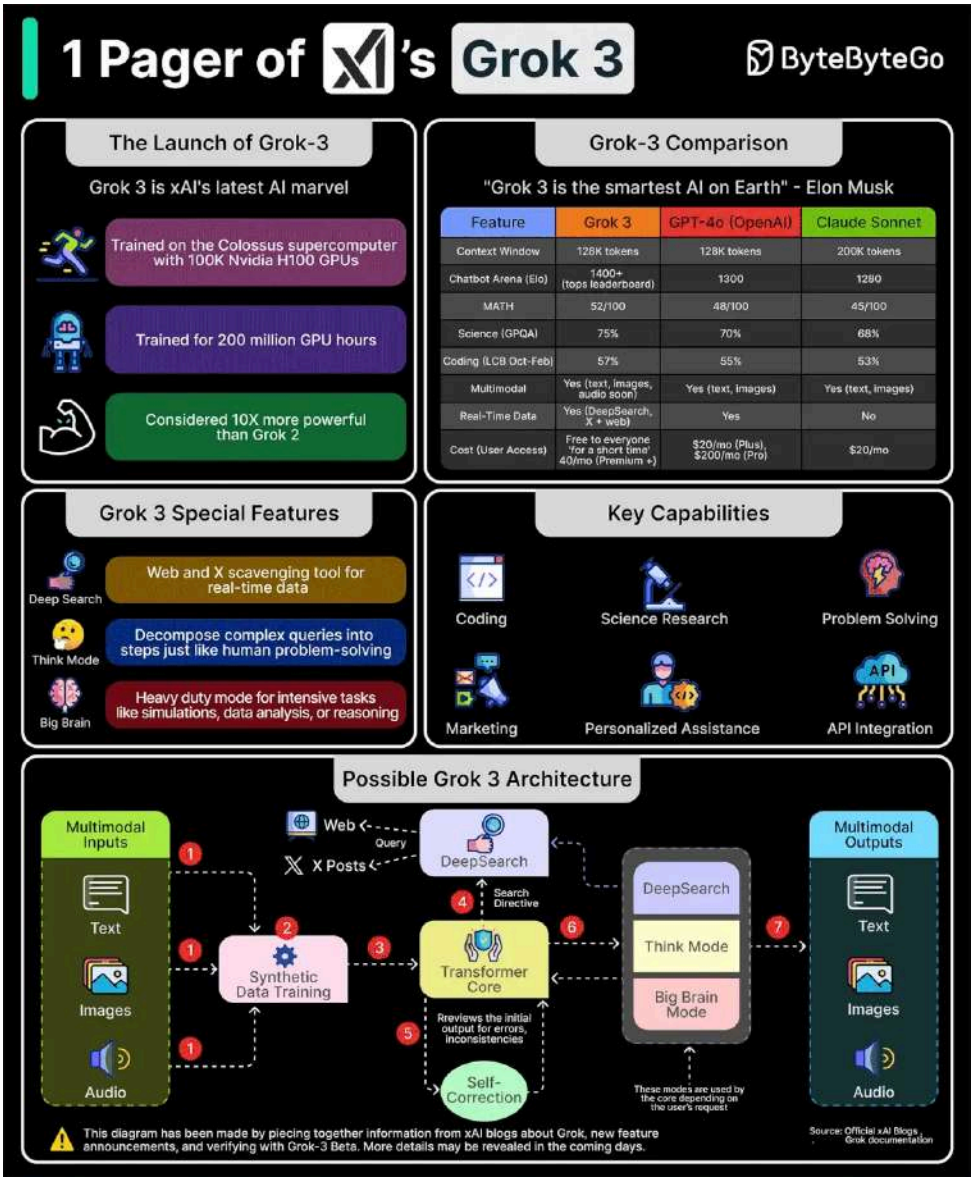
To reduce data transmission time, requests and responses can be compressed (e.g., using gzip), making the upload and download processes quicker.

Connection Pooling:

This technique involves using a pool of open connections to manage database interaction, which reduces the overhead associated with opening and closing connections each time data needs to be loaded. The pool manages the lifecycle of connections for efficient resource use.

Over to you: What other ways do you use to improve API performance?

xAI’s Grok-3 One-Pager



Grok 3 was unveiled on February 17. According to Elon Musk, it is “the smartest AI on Earth.”

But what makes it so powerful?

Grok 3 has been trained on the Colossus supercomputer with 100K Nvidia H100 GPUs. According to one stat, it has been trained for 200 million GPU hours. The key capabilities of Grok 3 are coding, science research, problem-solving, creating marketing campaigns, personalized assistance, and API integration.

Some special features of Grok 3 are:

1. DeepSearch: Allows Grok to search the web and X posts for real-time data and information.
2. Think Mode: Decompose complex queries into steps just like human problem-solving.
3. Big Brain Mode: A heavy-duty mode for intensive tasks like simulations, data analysis, or reasoning.

Here's how Grok 3 possibly works:

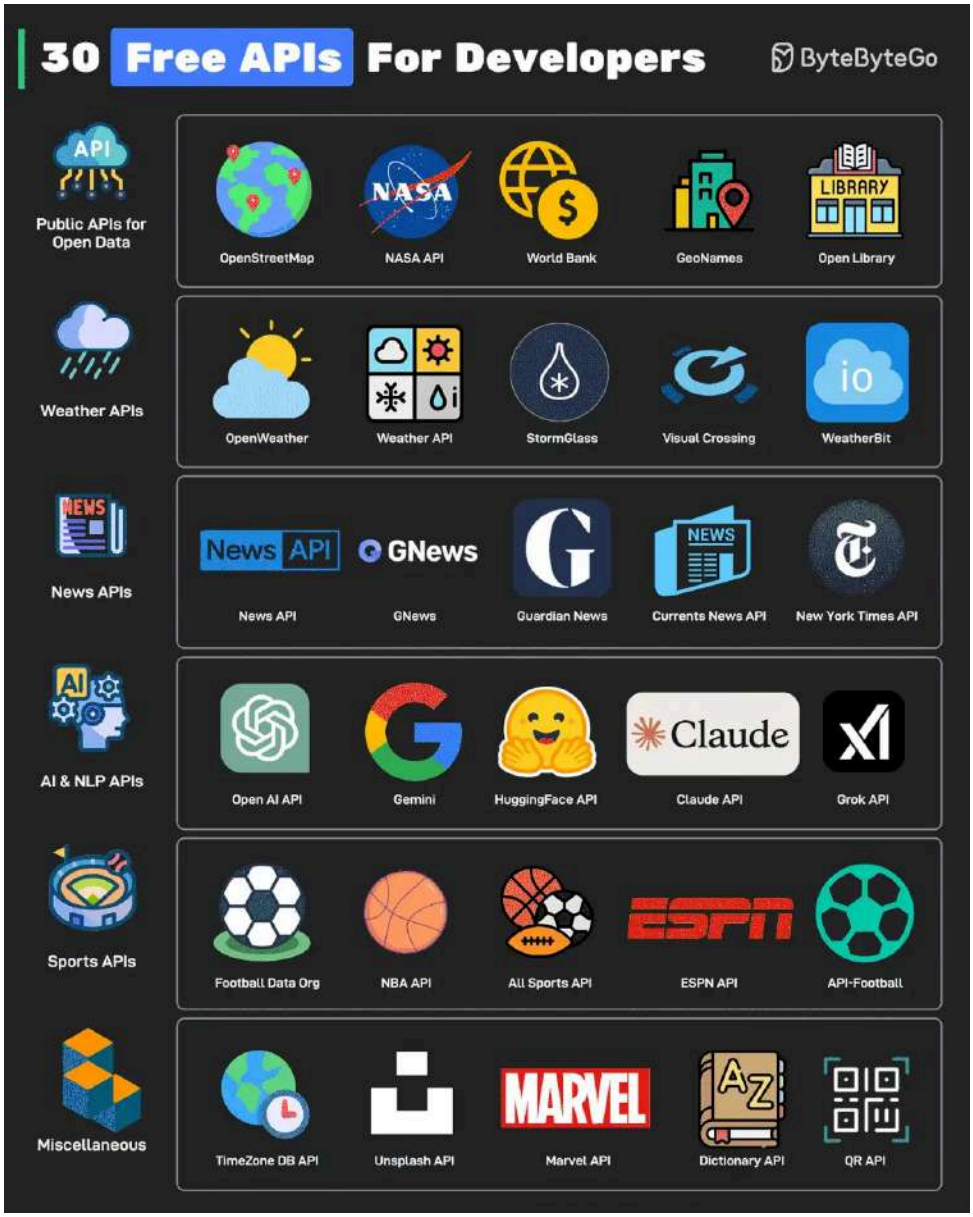
1. Grok 3 supports multi-modal inputs. The user can ask questions as text, upload images, or talk to Grok. The inputs are tokenized to initiate processing.
2. Synthetic data training enhances the query interpretation.
3. This is fed to the core transformer model that analyzes the input.
4. The core transformer issues a search directive to DeepSearch to retrieve necessary information from the web or real-time X data.
5. Next, a self-correct process ensures accuracy through an internal feedback loop.
6. Depending on the user's input, the Think Mode or Big Brain Mode refines the responses as required. It might again use DeepSearch to find more information.
7. The answers are delivered to the user in a multimodal manner.

Note that these details are based on piecing together information from xAI blogs about Grok and new feature announcements. More details may be revealed in the coming days.

Over to you: Have you used xAI's Grok 3?

30 Free APIs for Developers

APIs are the backbone of modern software development. Whether it is a hobby project or a real-world application, developers need APIs. These APIs (some completely free and some with free tiers) can help kickstart development.



1. Public APIs for Open Data
   OpenStreetMap, NASA, World Bank, GeoNames, and Open Library APIs provide a lot of useful data.

2. Weather APIs
   OpenWeather, Weather API, StormGlass, Visual Crossing, and WeatherBit are some APIs to fetch weather-related information

3. News APIs
   The News API, GNews, Guardian News, Current News API, and New York Times API can help developers fetch the latest news.

4. AI & NLP APIs
   Open AI API, Gemini, HugginFace API, Claude API, and Grok API can help developers experiment with AI models and tools.

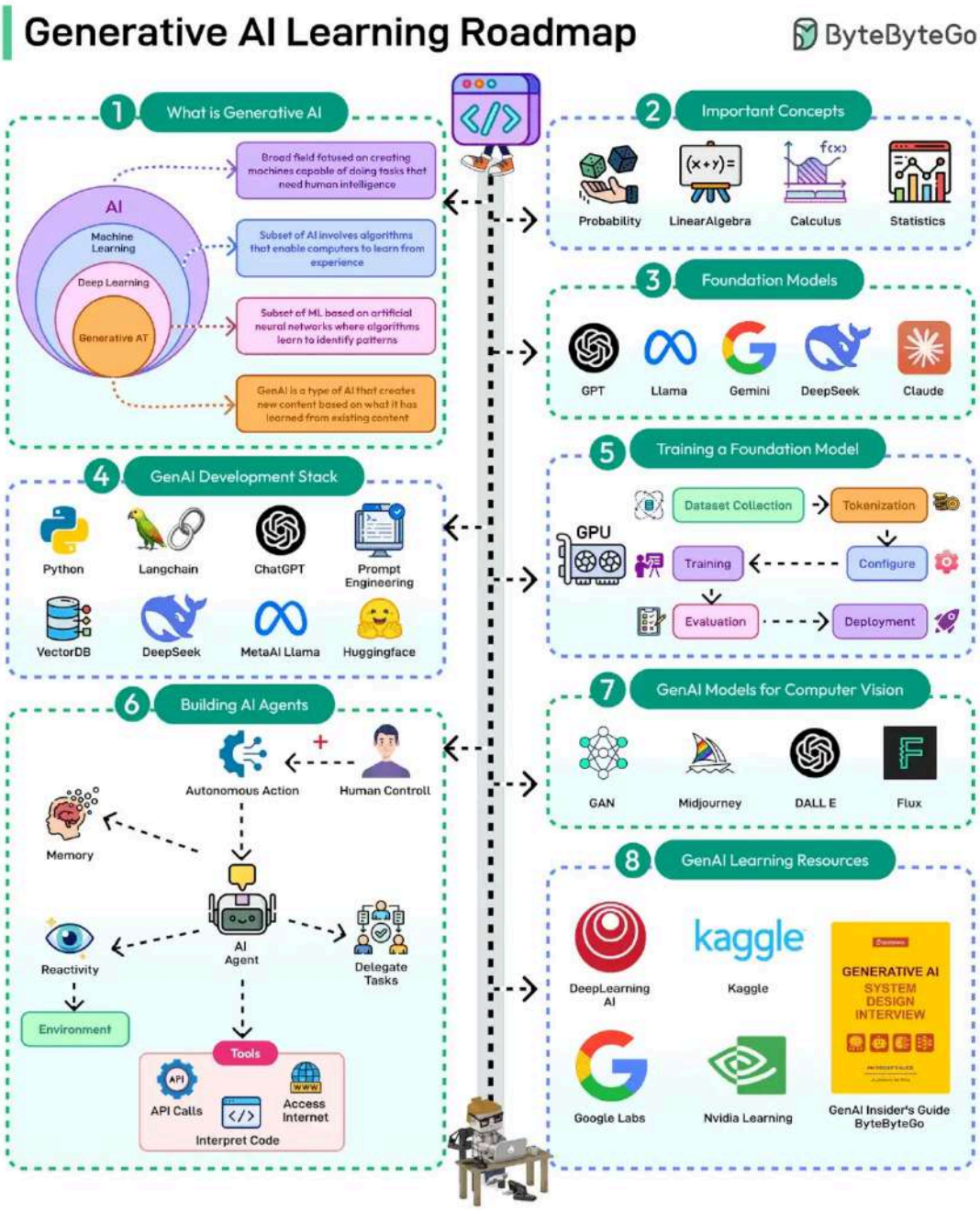
5. Sports API
   Football Data Org, NBA API, All Sports API, ESPN API, and API-Football can help fetch sports-related information.

6. Miscellaneous
   Some interesting miscellaneous APIs are TimeZone API, Unsplash API, Marvel API, Dictionary API, and QR Generation API.

Over to you: Which other API will you add to the list?

The Generative AI Learning Roadmap

Generative AI is a type of AI that can create new content based on what it has learned from existing knowledge. It has the potential to revolutionize human learning.



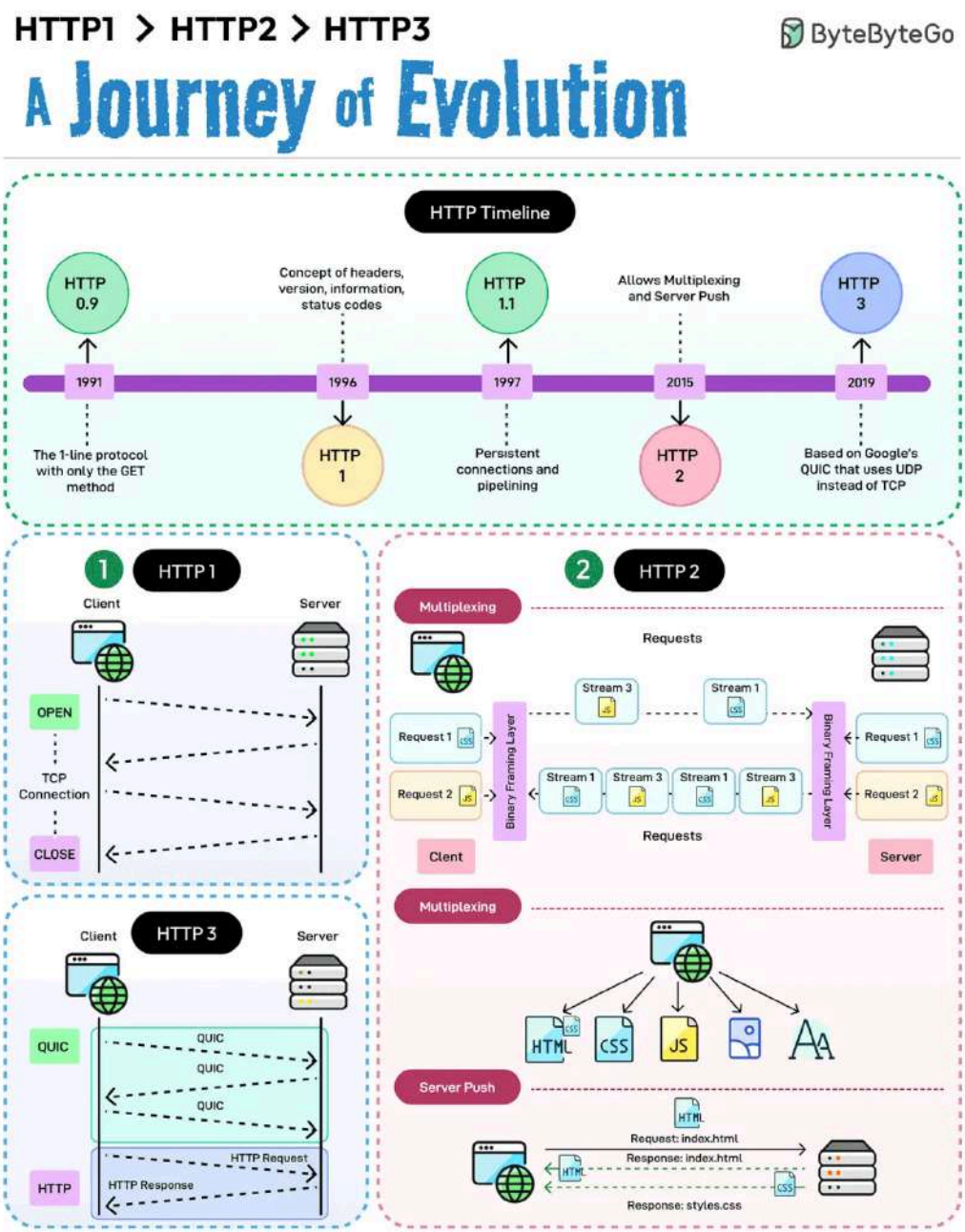
Here's a GenAI roadmap with learning resources:

1. Learn about important concepts like Probability, Statistics, Calculus, and Linear Algebra.

2. Understand the working of foundational models like GPT, MetaAI's Llama, Gemini, DeepSeek, and Claude.
3. Learn the GenAI development stack that includes Python, Language, ChatGPT APIs, Prompt Engineering, VectorDB, DeepSeek, Llama, and Huggingface.
4. Learn how to train and fine-tune a foundation model.
5. Understand the role of AI Agents and how to build one using GenAI tools.
6. Learn about GenAI models for computer vision such as GAN (Generative Adversarial Networks), MidJourney, DALL E, Flux, and so on.
7. Make use of GenAI Learning Resources such as DeepLearning AI platform, Kaggle, Generative AI Insider's Guide by ByteByteGo, Google Labs, and Nvidia Learning platforms.

Over to you: What else will you add to the GenAI learning roadmap?

HTTP/1 -> HTTP/2 -> HTTP/3



HTTP 1 started in 1996 followed by HTTP 1.1 the very next year. In 2015, HTTP 2 came about and in 2019 we got HTTP 3.

With each iteration, the protocol has evolved in new and interesting ways.

1 - HTTP 1 (and its sub-versions) introduced features like persistent connections, pipelining, and the concept of headers. The protocol was built on top of TCP and provided a reliable way of communication over the World Wide Web. It is still used despite being over 25 years old.

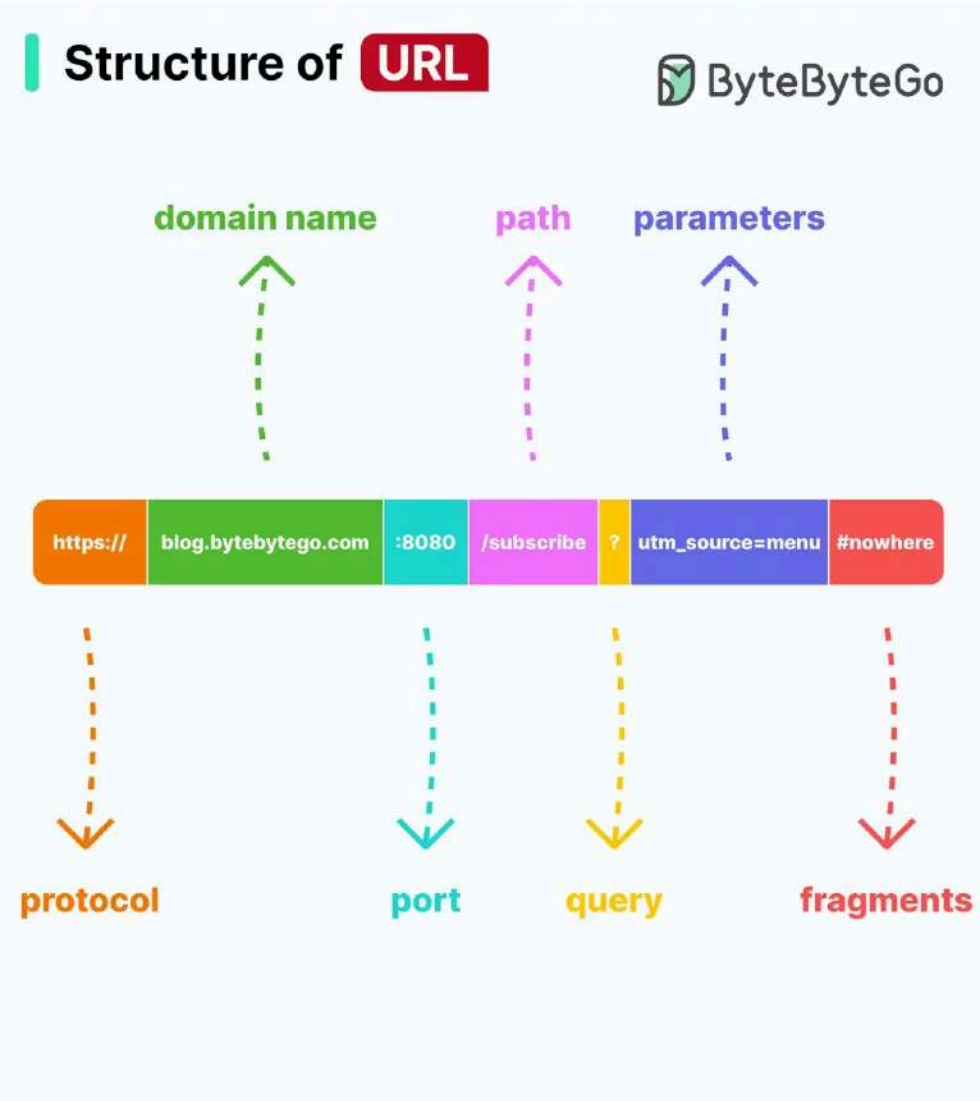
2 - HTTP 2 brought new features such as multiplexing, stream prioritization, server push, and HPACK compression. However, it still used TCP as the underlying protocol.

3 - HTTP 3 uses Google's QUIC, which is built on top of UDP. In other words, HTTP 3 has moved away from TCP.

Over to you: What would you add to understand the evolution of HTTP over the years?

Structure of URL

Do you know all the components of a URL?



Uniform Resource Locator (URL) is a term familiar to most people, as it is used to locate resources on the internet. When you type a URL into a web browser's address bar, you are accessing a "resource", not just a webpage.

URLs comprise several components:

- The protocol or scheme, such as http, https, and ftp.
- The domain name and port, separated by a period (.)
- The path to the resource, separated by a slash (/)

- The parameters, which start with a question mark (?) and consist of key-value pairs, such as a=b&c=d.
- The fragment or anchor, indicated by a pound sign (#), which is used to bookmark a specific section of the resource.

A Cheatsheet on Comparing Key-Value Stores



Database	Type	Replication Style	Consistency Protocol	Node Discovery	Partitioning Approach
Redis	Key-value	Single-leader	Eventual Consistency	Sentinel	Hash slots & Consistent Hashing
MongoDB	Document	Single-leader	Raft Algorithm	MongoDB Config Server	Range-based & Hash-based
Cassandra	Key-value	Leaderless	Quorum (Tunable)	Gossip Protocol	Hash-based
DynamoDB	Key-value & Document	Leaderless	Eventual Consistency (Tunable)	Gossip Protocol	Consistent Hashing
Neo4j	Graph Database	Single-leader	Raft	Cluster Server Discovery	Range-based & Hash-based
Couchbase	Document	Leaderless	Strong Consistency with MVCC	Gossip Protocol	Hash-based
CosmosDB	Multi-model	Leaderless	Tunable Consistency	Proprietary	Hash-based
HBase	Wide-column	Single-leader	Consensus	Zookeeper	Range-based
Google Firestore	Document	Multi-region	Strong Consistency	Proprietary	Automatic
ScyllaDB	Wide-column	Leaderless	Tunable Consistency	Gossip Protocol	Consistent Hashing
CouchDB	Document	Leaderless	MVCC	Cluster-aware	Hash-based
Zookeeper	Key-value	Single-leader	ZAB Protocol	Coordinator Node	Ensemble-based
ETCD	Key-value	Multi-leader	Raft	Discovery Service	Consistent Hashing
Riak	Key-value	Leaderless	Vector Clocks	Gossip Protocol	Hash-based
Foundation DB	Key-value	Leaderless	Strong Consistency	Coordinator-based	Range-based

A key-value store is a database that stores data as a collection of key-value pairs. We compare 15 key-value stores across different parameters like replication style, consistency protocol, node discovery approach, and partitioning approach.

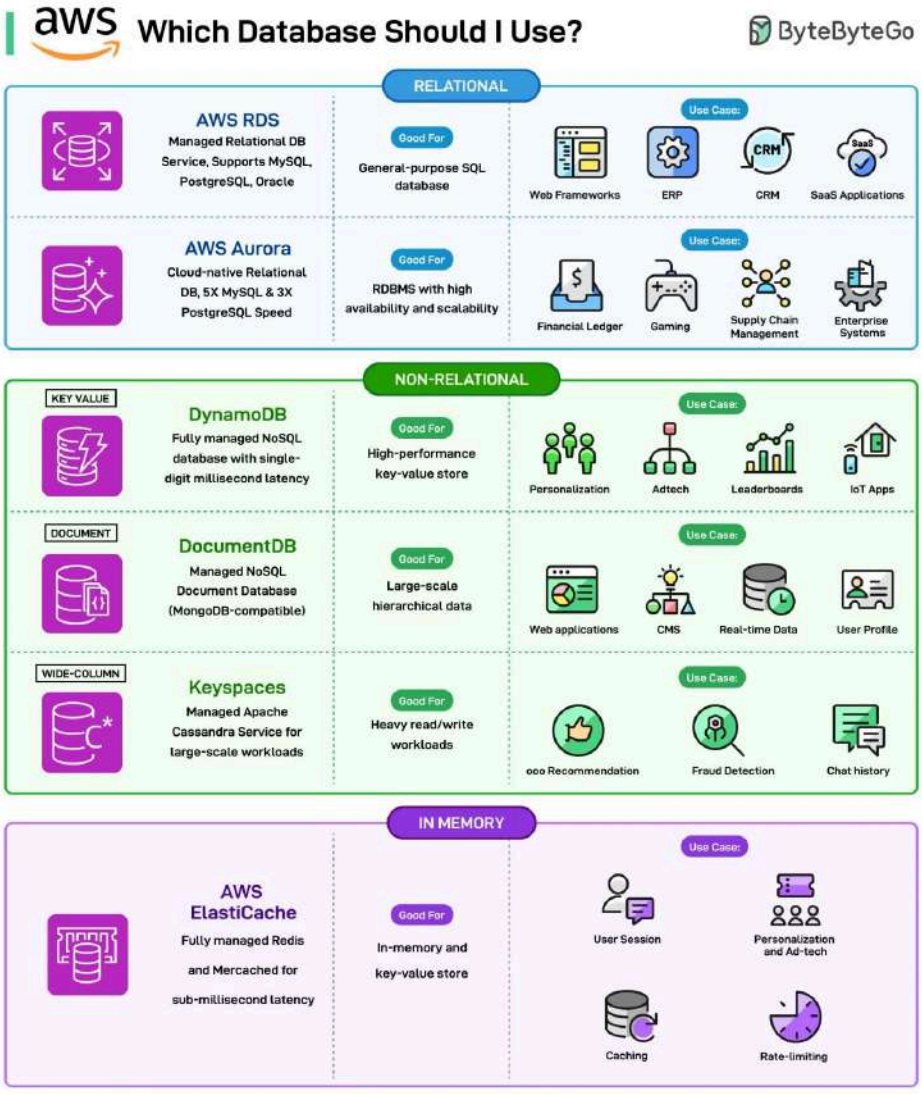
The databases covered are:

1. Redis
2. MongoDB
3. Cassandra
4. DynamoDB
5. Neo4j
6. Couchbase
7. CosmosDB
8. HBase
9. Firestore
10. ScyllaDB
11. CouchDB
12. Zookeeper
13. ETCD
14. Riak
15. FoundationDB

Over to you: Which other database will you add to the list?

Which Database Should I Use on AWS?

This question often comes up when we start deploying an application on AWS. There are many options available to choose from.



Broadly, there are three categories of databases:

1. Relational
   The two main choices in this space are AWS RDS and AWS Aurora.

   AWS RDS is a managed relational DB service that supports MySQL, PostgreSQL, Oracle, etc. AWS Aurora is a cloud-native relational database that supports high availability and scalability

2. Non-Relational
   The three main choices in this category are DynamoDB, DocumentDB, and AWS

Keyspaces.

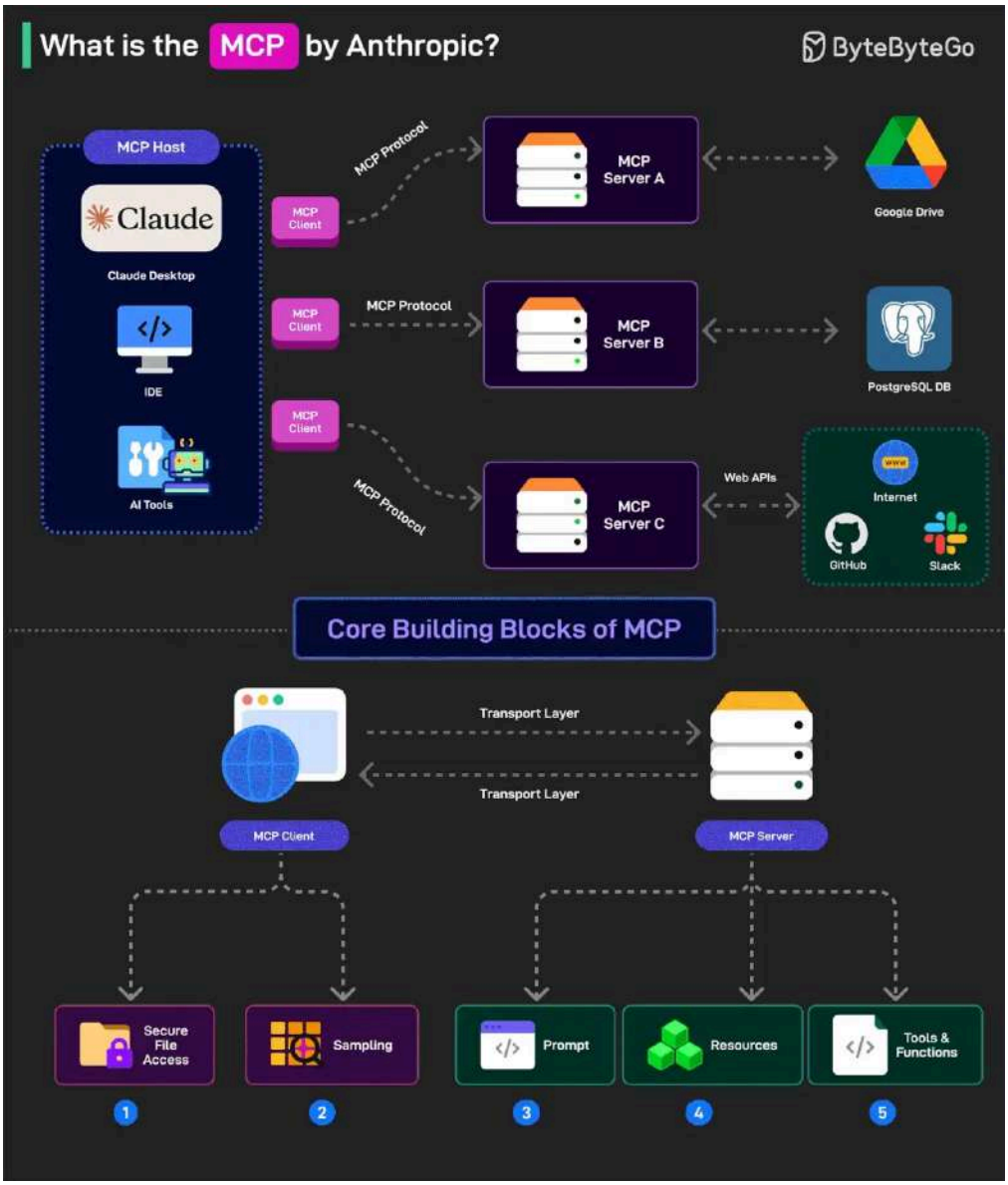
DynamoDB is a fully managed NoSQL key-value datastore. DocumentDB is a managed document database compatible with MongoDB. AWS Keyspaces is a managed, wide-column Apache Cassandra service for large-scale workloads.

3. In-memory
   AWS Elasticache is the most popular option in this category. It is a fully managed Redis or Memcached service for sub-millisecond latency.

Over to you: Which AWS Databases have you used?

What is MCP?

Why is everyone talking about it? Let's take a closer look.



Model Context Protocol (MCP) is a new system introduced by Anthropic to make AI models more powerful.

It is an open standard (also being run as an open-source project) that allows AI models (like Claude) to connect to databases, APIs, file systems, and other tools without needing custom code for each new integration.

MCP follows a client-server model with 3 key components:

1. Host: AI applications like Claude that provide the environment for AI interactions so that different tools and data sources can be accessed. The host runs the MCP Client.

2. MCP Client: The MCP client is the component inside an AI model (like Claude) that allows it to communicate with MCP servers. For example, if the AI model wants data from PostgreSQL, the MCP client formats the request into a structured message to send to the MCP Server

3. MCP Server: This is the middleman that connects an AI model to an external system like PostgreSQL, Google Drive, or an API. For example, if Claude analyzes sales data from PostgreSQL, the MCP Server for PostgreSQL acts as the connector between Claude and the database.

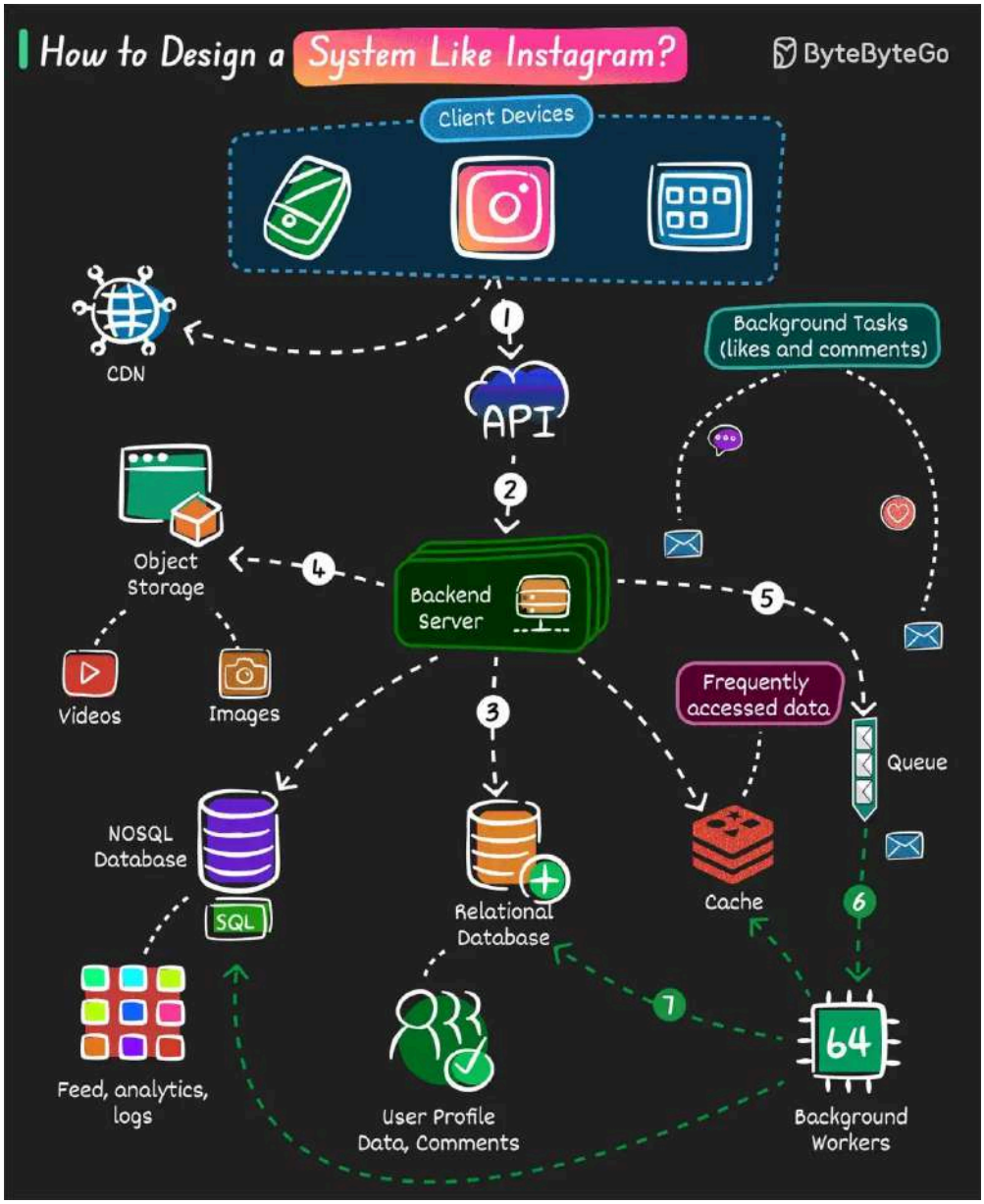
MCP has five core building blocks (also known as primitives). They are divided between the client and server.

1. For the clients, the building blocks are Roots (secure file access) and Sampling (ask the AI for help with a task such as generating a DB query).

2. For the servers, there are Prompts (instructions to guide the AI), Resources (Data Objects that the AI can reference) and Tools (functions that the AI can call such as running a DB query).

Over to you: Have you explored Anthropic's Model Context Protocol?

How to Design a System like Instagram



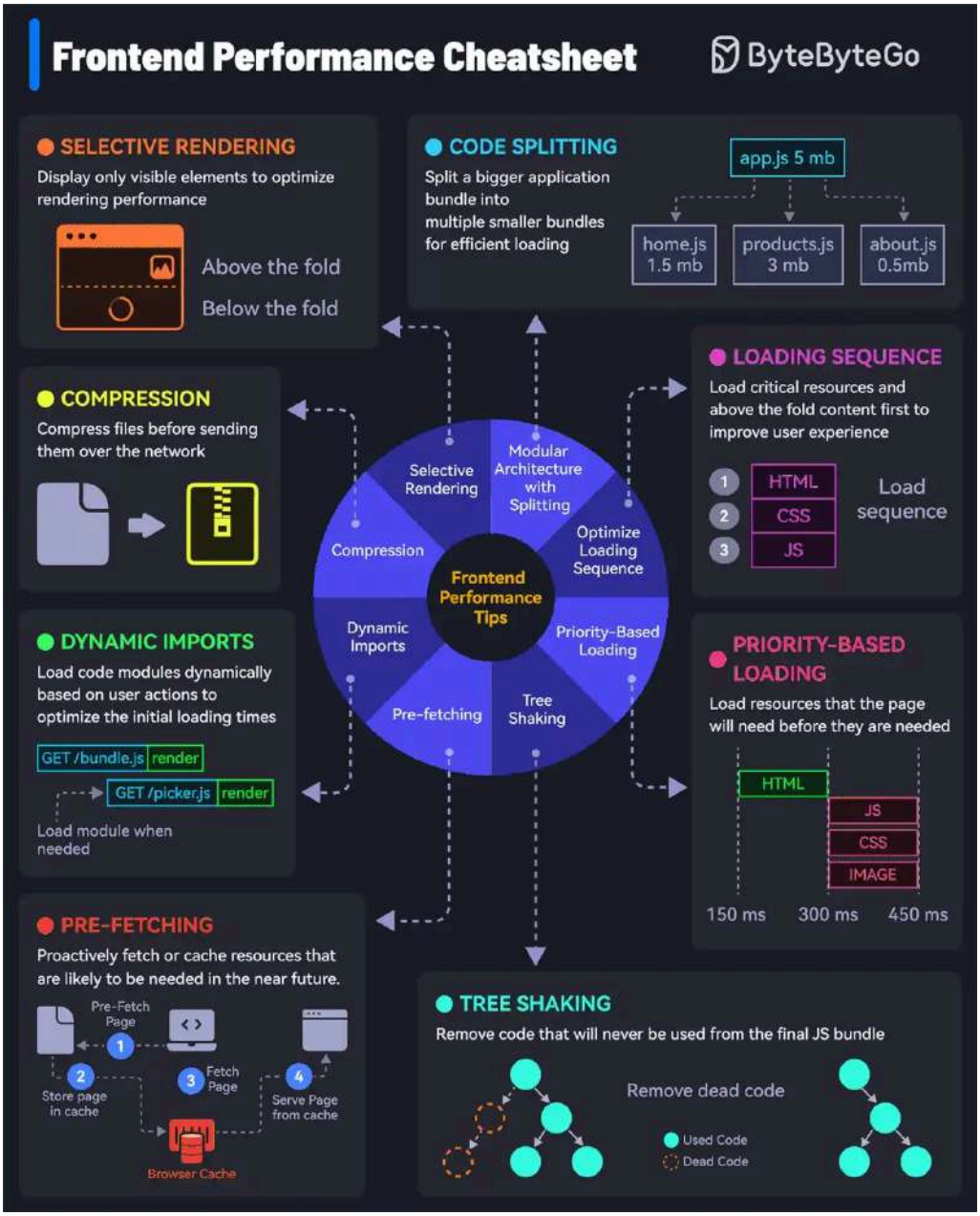
Here's a simple step-by-step approach to build the core functionality:

1. Users perform an action on their device like tapping the like button, uploading a picture, or following another user.
2. The request goes through a gateway to the backend server built using frameworks like Django, Express, or anything else.
3. The data is stored in the database.

    - Structured data such as user profiles, comments, and relationships are stored in relational databases like Postgres.
    - Data items requiring high write throughput such as user feeds and activity logs can be stored in eventually consistent databases like Cassandra.
    - Frequently accessed data is stored in cache (for example, Redis or Memcached).
4. Images and videos are stored in an object storage and corresponding metadata is stored in the database.
5. Asynchronous tasks like sending notifications are sent to a queue.
6. Background workers (like Celery) retrieve the task from the queue and perform the necessary processing.
7. Celery worker also performs the necessary updates in the databases.

Over to you: What else will you add to build an Instagram-like system?

How to load your websites at lightning speed



Check out these 8 tips to boost frontend performance:

1. Compression
   Compress files and minimize data size before transmission to reduce network load.

2. Selective Rendering/Windowing
   Display only visible elements to optimize rendering performance. For example, in a dynamic list, only render visible items.

3. Modular Architecture with Code Splitting
   Split a bigger application bundle into multiple smaller bundles for efficient loading.

4. Priority-Based Loading
   Prioritize essential resources and visible (or above-the-fold) content for a better user experience.

5. Pre-loading
   Fetch resources in advance before they are requested to improve loading speed.

6. Tree Shaking or Dead Code Removal
   Optimize the final JS bundle by removing dead code that will never be used.

7. Pre-fetching
   Proactively fetch or cache resources that are likely to be needed soon.

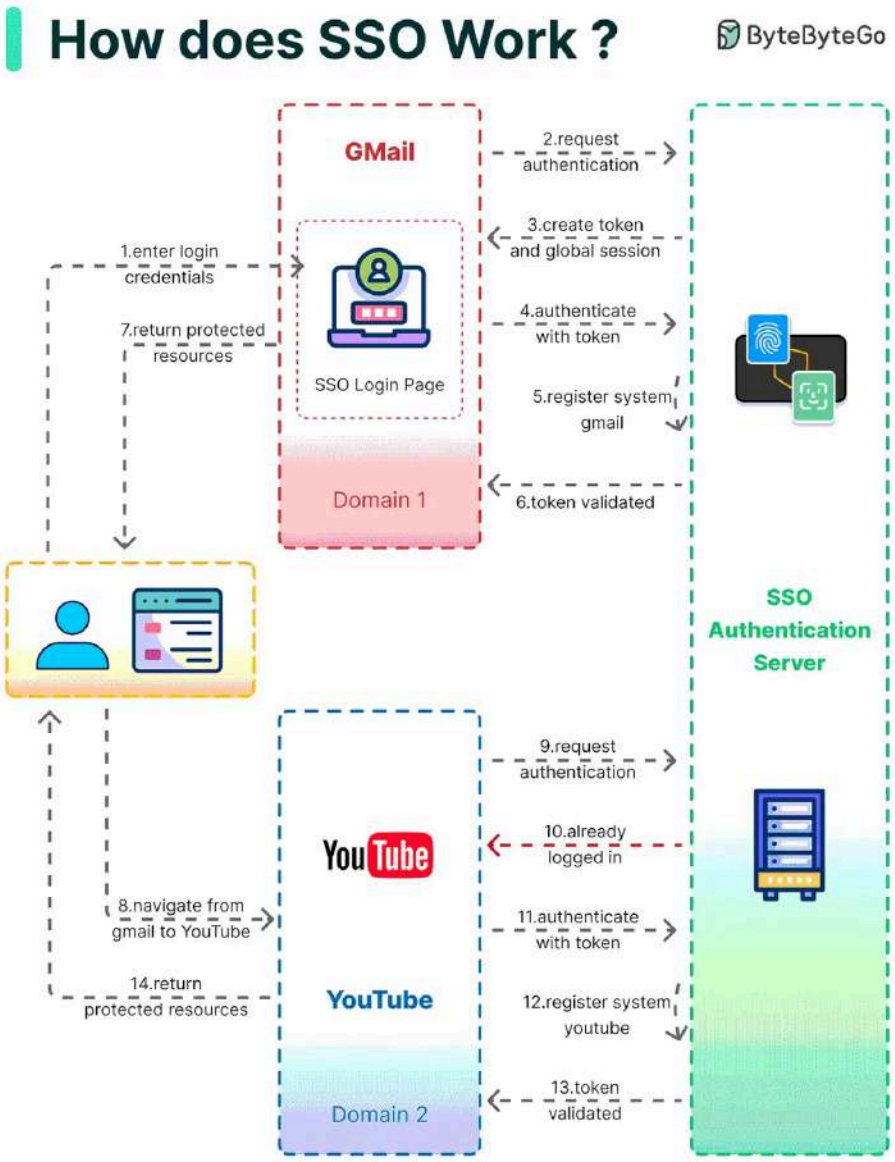
8. Dynamic Imports
   Load code modules dynamically based on user actions to optimize the initial loading times.

Over to you: What other frontend performance tips would you add to this cheat sheet?

What is SSO (Single Sign-On)?

Basically, Single Sign-On (SSO) is an authentication scheme. It allows a user to log in to different systems using a single ID.

The diagram below illustrates how SSO works.



- Step 1: A user visits Gmail, or any email service. Gmail finds the user is not logged in and so redirects them to the SSO authentication server, which also finds the user is not logged in. As a result, the user is redirected to the SSO login page, where they enter their login credentials.

- Steps 2-3: The SSO authentication server validates the credentials, creates the global session for the user, and creates a token.
- Steps 4-7: Gmail validates the token in the SSO authentication server. The authentication server registers the Gmail system, and returns “valid.” Gmail returns the protected resource to the user.
- Step 8: From Gmail, the user navigates to another Google-owned website, for example, YouTube.
- Steps 9-10: YouTube finds the user is not logged in, and then requests authentication. The SSO authentication server finds the user is already logged in and returns the token.
- Step 11-14: YouTube validates the token in the SSO authentication server. The authentication server registers the YouTube system, and returns “valid.” YouTube returns the protected resource to the user.

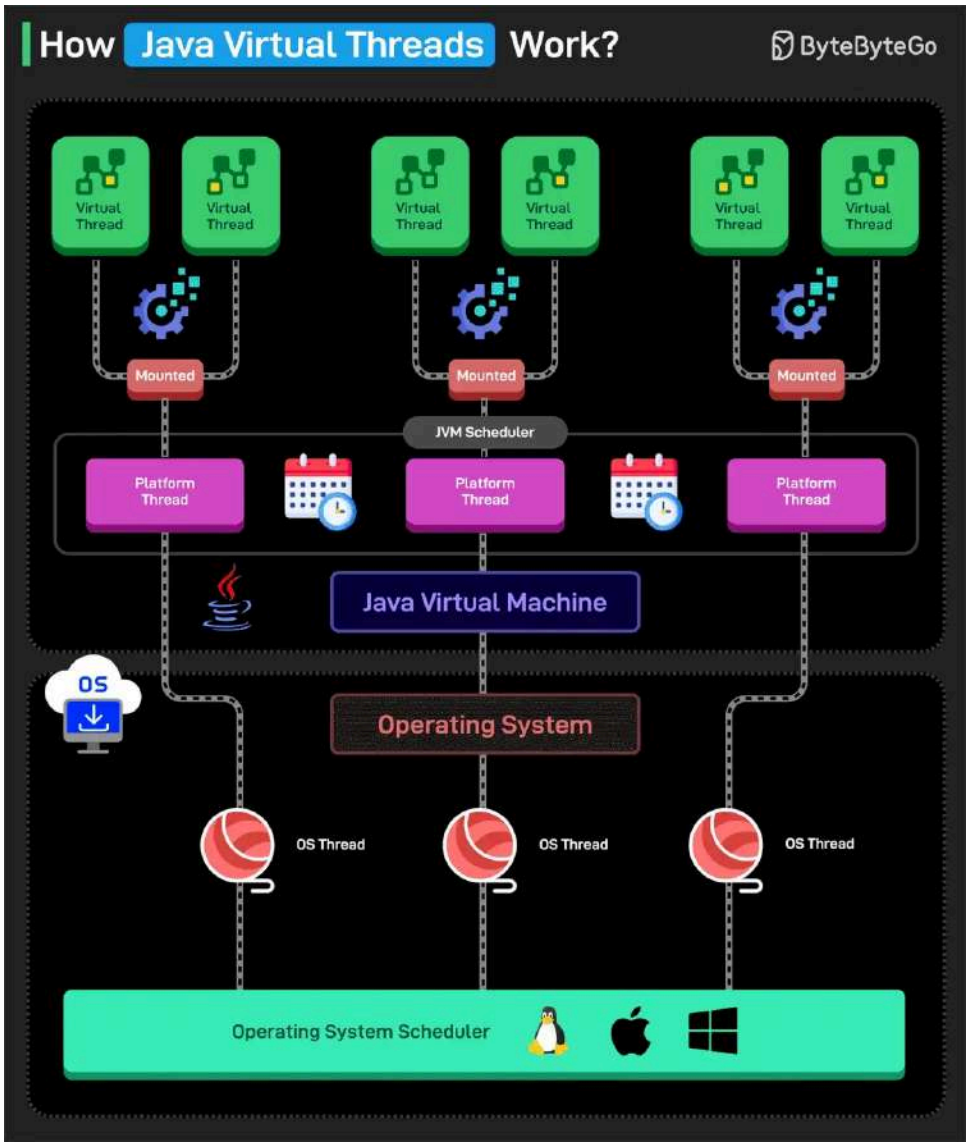
The process is complete and the user gets back access to their account.

Over to you:

- Have you implemented SSO in your projects? What is the most difficult part?
- What’s your favorite sign-in method and why?

How Java Virtual Threads Work?

Virtual Threads are lightweight threads introduced in Java 19 (Preview) and Java 21 (Stable). They allow Java to create millions of threads efficiently, helping handle concurrent tasks without wasting memory or CPU.



Virtual Threads do not map 1:1 to OS Threads and do not replace the original Platform Threads. The Platform Threads are backed by the OS Threads and are sometimes also known as Carrier Threads in this context.

Think of the Platform Threads as a small group of workers, and Virtual threads as tasks. With Virtual threads, tasks are given to workers only when needed, allowing one worker to handle thousands of tasks efficiently.

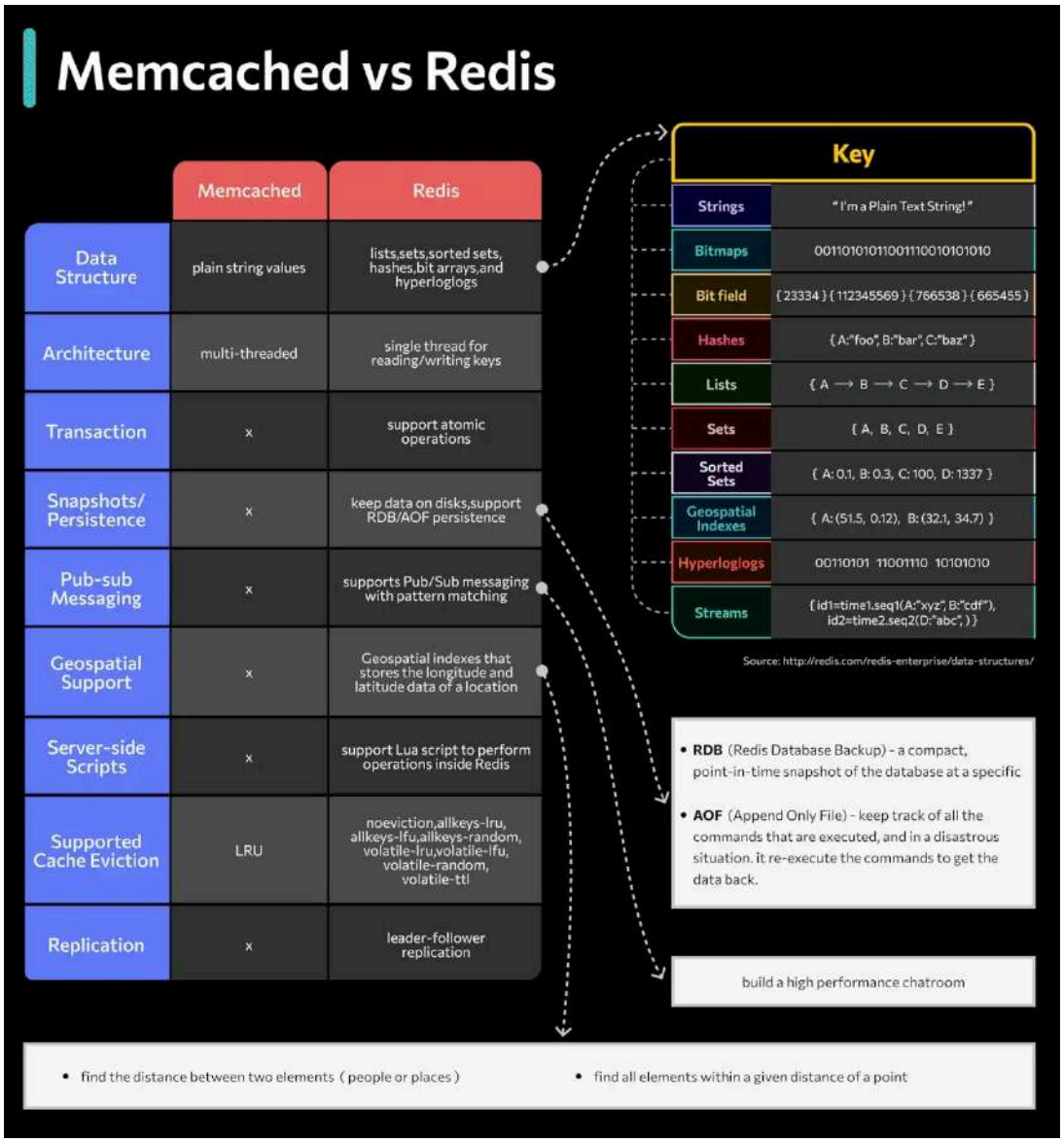
Here's how Virtual Threads work:

1. Virtual Threads run on top of Platform Threads. The JVM schedules them onto a small number of Platform Threads.
2. When a Virtual Thread starts, the JVM assigns it to a normal OS-backed Platform Thread.
3. Virtual Threads can also handle CPU-intensive work, but their real advantage is in scenarios with a high number of I/O-bound or concurrent tasks.
4. If the Virtual Thread performs a blocking operation (like I/O, database call, sleep, etc.), the JVM unmounts it from the Platform Thread. However, this does not block the underlying OS Thread as such.
5. The Platform thread is freed up to handle another Virtual Thread.
6. When the blocking operation finishes, the Virtual Thread is rescheduled on any available Platform thread.

Over to you: Have you used Virtual Threads?

Redis VS Memcached

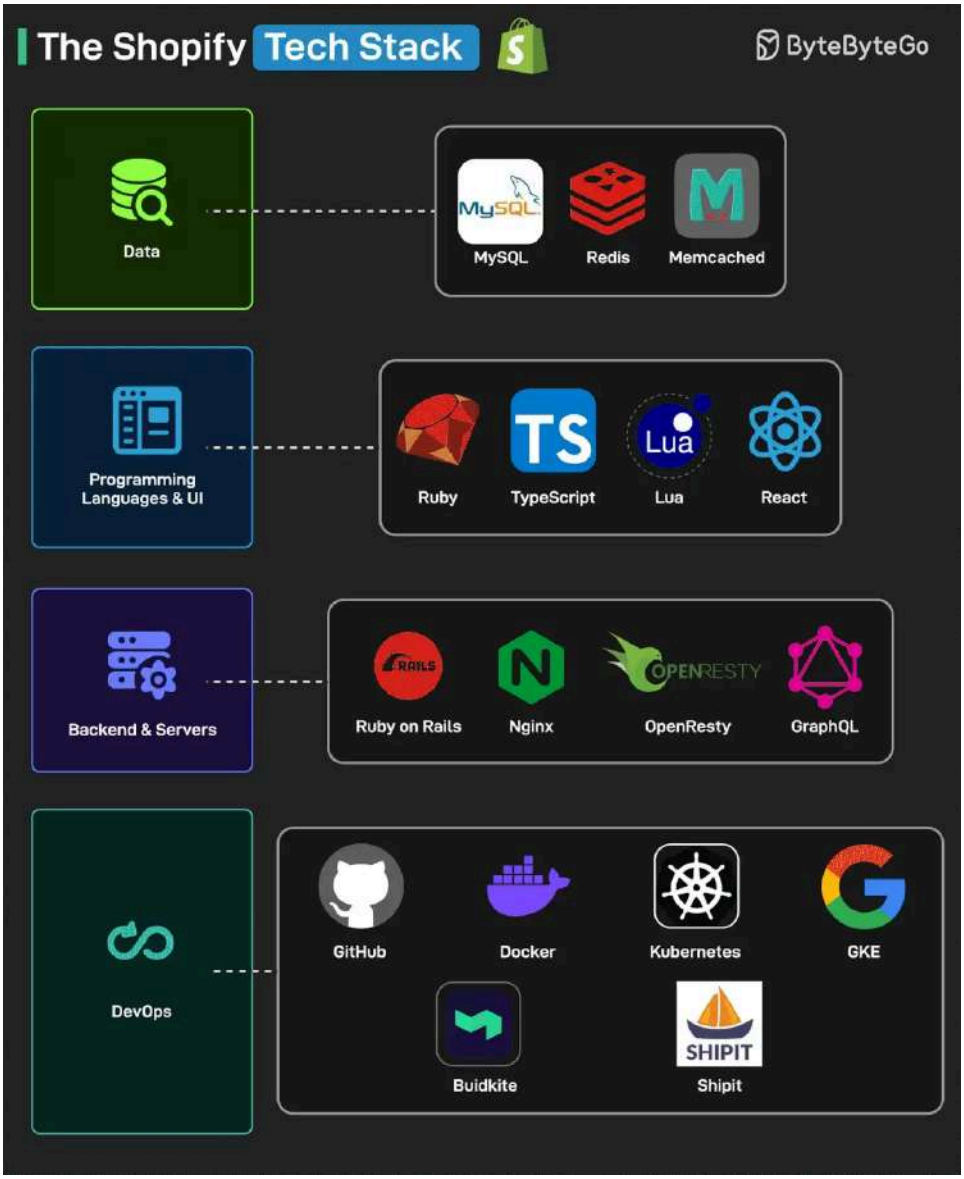
The diagram below illustrates the key differences.



The advantages of data structures make Redis a good choice for:

- Recording the number of clicks and comments for each post (hash)
- Sorting the commented user list and deduping the users (zset)
- Caching user behavior history and filtering malicious behaviors (zset, hash)
- Storing boolean information of extremely large data into small space. For example, login status, membership status. (bitmap)

The Shopify Tech Stack



Shopify is a multi-channel commerce platform for small and medium businesses. It allows merchants to create a shop and sell products wherever they want.

Here's the Shopify Tech Stack, which powers more than 600,000 merchants and serves 80,000 requests per second during peak traffic.

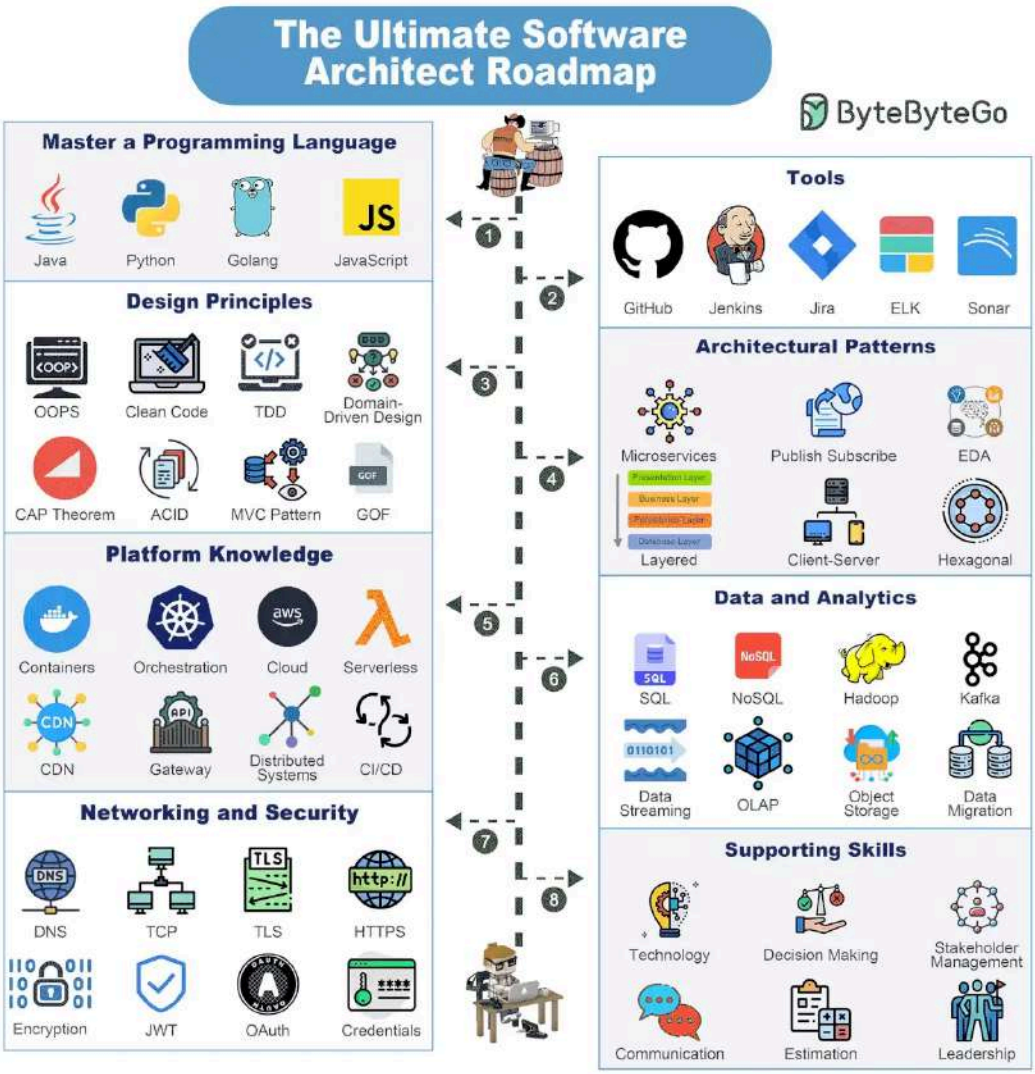
1. Programming Languages & UI: They use Ruby, Typescript, Lua, and React
2. Backend & Servers: They use Ruby on Rails, Nginx, OpenResty, and GraphQL.
3. Data: They use MySQL, Redis, and Memcached.

4. DevOps: They use GitHub, Docker, Kubernetes, GKE, BuildKite, and ShipIt. ShipIt has been also made open source.

Reference: E-Commerce at Scale: Inside Shopify's Tech Stack

Software Architect Knowledge Map

Becoming a Software Architect is a journey where you are always learning. But there are some things you must definitely strive to know.



1. Master a Programming Language
   Look to master 1-2 programming languages such as Java, Python, Golang, JavaScript, etc.

2. Tools
   Build proficiency with key tools such as GitHub, Jenkins, Jira, ELK, Sonar, etc.

3. Design Principles
   Learn about important design principles such as OOPS, Clean Code, TDD, DDD, CAP Theorem, MVC Pattern, ACID, and GOF.

4. Architectural Principles
   Become proficient in multiple architectural patterns such as Microservices, Publish-Subscribe, Layered, Event-Driven, Client-Server, Hexagonal, etc.

5. Platform Knowledge
   Get to know about several platforms such as containers, orchestration, cloud, serverless, CDN, API Gateways, Distributed Systems, and CI/CD

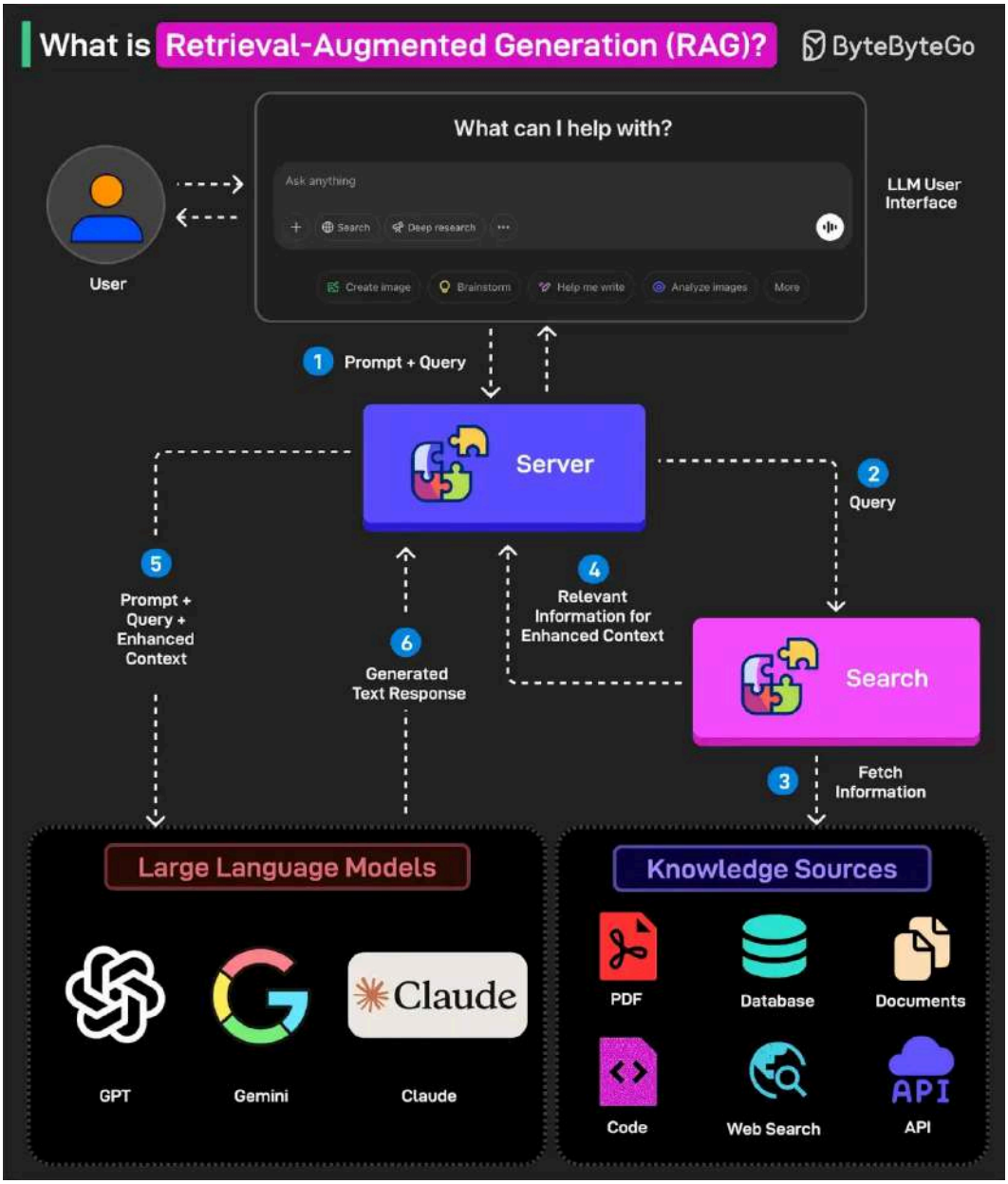
6. Data Analytics
   Build a solid knowledge of data and analytics components like SQL and NoSQL databases, data streaming solutions with Kafka, object storage, data migration, OLAP, and so on.

7. Networking and Security
   Learn about networking and security concepts such as DNS, TCP, TLS, HTTPS, Encryption, JWT, OAuth, and Credential Management.

8. Supporting Skills
   Apart from technical, software architects also need several supporting skills such as decision-making, technology knowledge, stakeholder management, communication, estimation, leadership, etc.

Over to you - What else would you add to the roadmap?

What is Retrieval-Augmented Generation (RAG)?



RAG is the process of optimizing an LLM's output to reference a specific knowledge base that may not have been part of its training data before generating a response.

In other words, RAG helps extend the powerful capabilities of LLMs to specific domains or knowledge bases without the need for additional training.

Here's how RAG works:

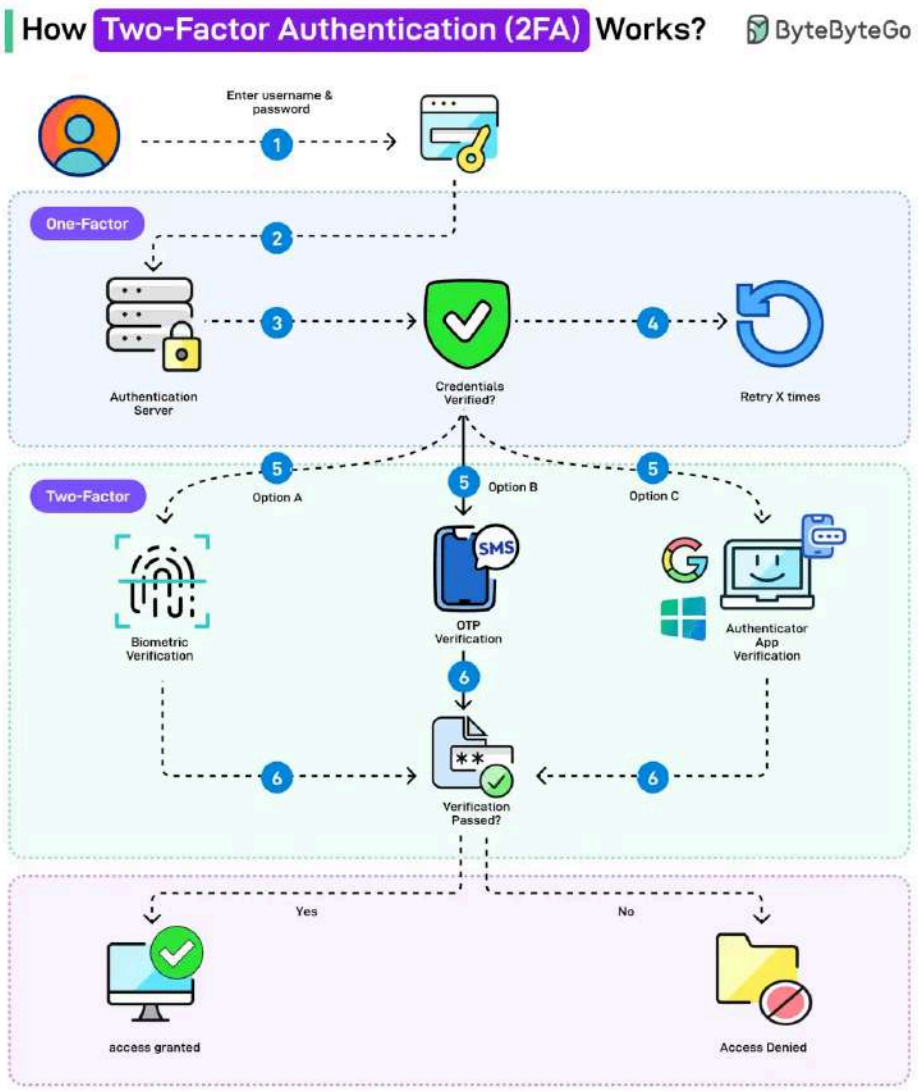
1. The user writes a query prompt in the LLM's user interface. This query is passed to the backend server where it is converted to a vector representation.

2. The query is sent to a search system.
3. This search system can refer to various knowledge sources such as PDFs, Web Search, Code-bases, Documents, Database, or APIs to fetch relevant information to answer the query.
4. The fetched information is sent back to the RAG model.
5. The model augments the original user input by adding the fetched information to the context and sending it to the LLM endpoint. Various LLM options are Open AI's GPT, Claude Sonnet, Google Gemini, and so on.
6. The LLM generates an answer based on the enhanced context and provides a response to the user.

Over to you: Have you used RAG with your favorite LLM?

How Two-factor Authentication (2FA) Works?

Two-factor authentication (2FA) is a type of multi-factor authentication that makes accounts more secure.



The idea behind 2FA is that you need to provide two identifying factors to prove your identity and gain access to your account.

The first factor is the knowledge factor, where a user needs to know a secret, such as a password. The second factor depends on the possession factor (such as RSA, authenticator app) or inherence factor (biometrics).

Here's how 2FA works:

1. The user enters username and password. This is for the first level of authentication, also known as single-factor.
2. The authentication request goes to the authentication server.
3. The authentication credentials are verified.
4. In case of any incorrect credentials, a certain number of retries may be allowed.
5. If the credentials are correct, the two-factor authentication kicks in. There are multiple options available: biometric verification, OTP verification, or Authenticator App Verification. Organizations like Google and Microsoft also provide such apps.
6. The user verifies using the chosen option.
7. If the verification fails, access is denied. However, if verification succeeds, access is granted.

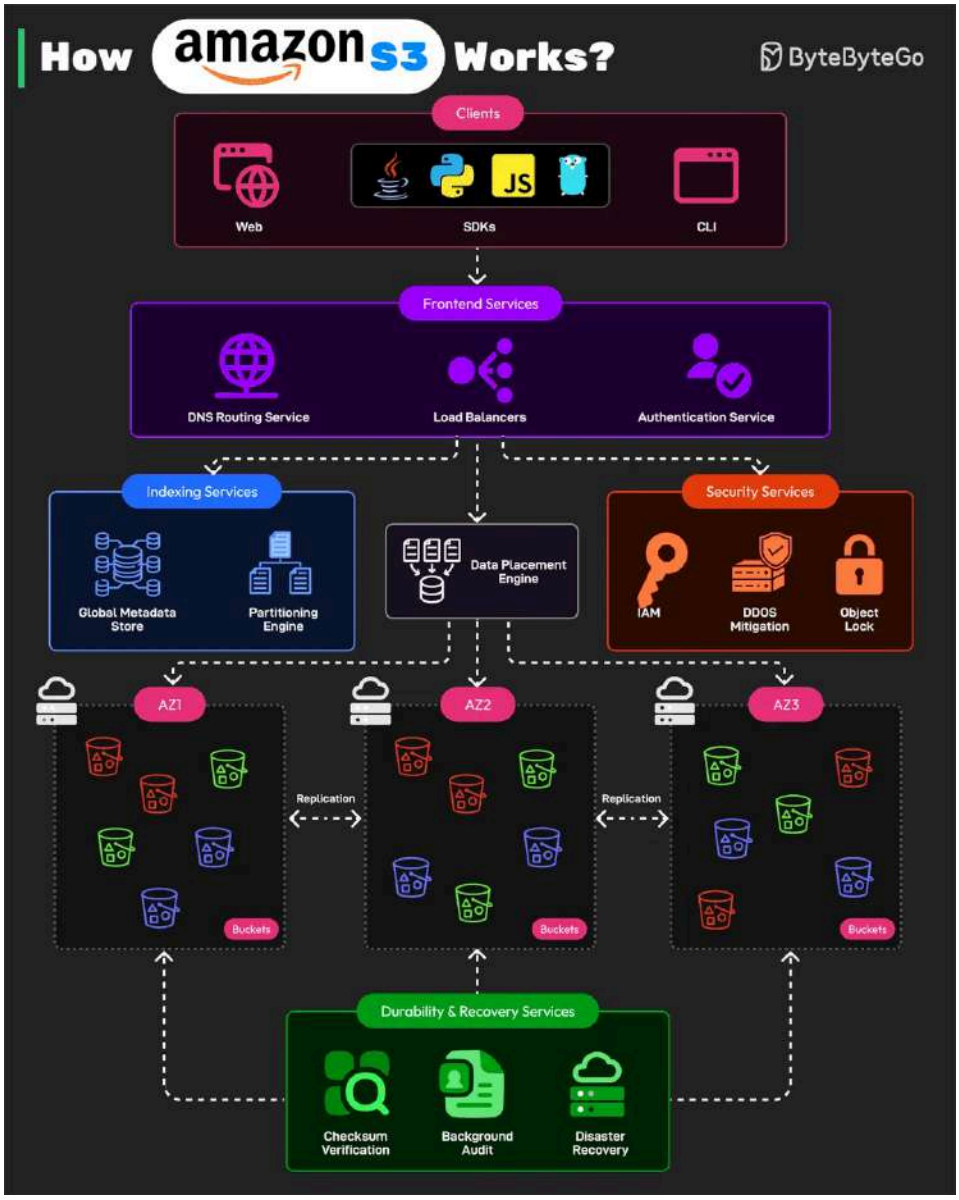
Note that the OTP verification using SMS is often considered less secure as a 2FA mechanism.

Over to you - Have you used two-factor authentication?

How Amazon S3 Works?

This post is based on:

1. AWS re:Invent 2023 - Dive deep on Amazon S3 video
2. Building and operating a pretty big storage system called S3 published on All Things Distributed.



Amazon S3 is one of the largest and most complex distributed storage systems in the world. It processes millions of requests per second and stores over 350 trillion objects while maintaining 99.999999999% durability.

AWS architected S3 using a microservices-based design

Here's how it works:

1. Front-end Request Handling Services
   These services receive API requests from clients. The clients can be web, CLI, and SDKs for programming languages like Java, Python, JS, or Golang. After receiving the request, the services authenticate users, validate requests, and route them to the correct storage service. The services consist of DNS routing, authentication service, and load balancing.

2. Indexing and Metadata Services
   Every object stored in S3 is assigned a unique identifier and metadata. The indexing services track object locations. They consist of a global metadata store and partitioning engine.

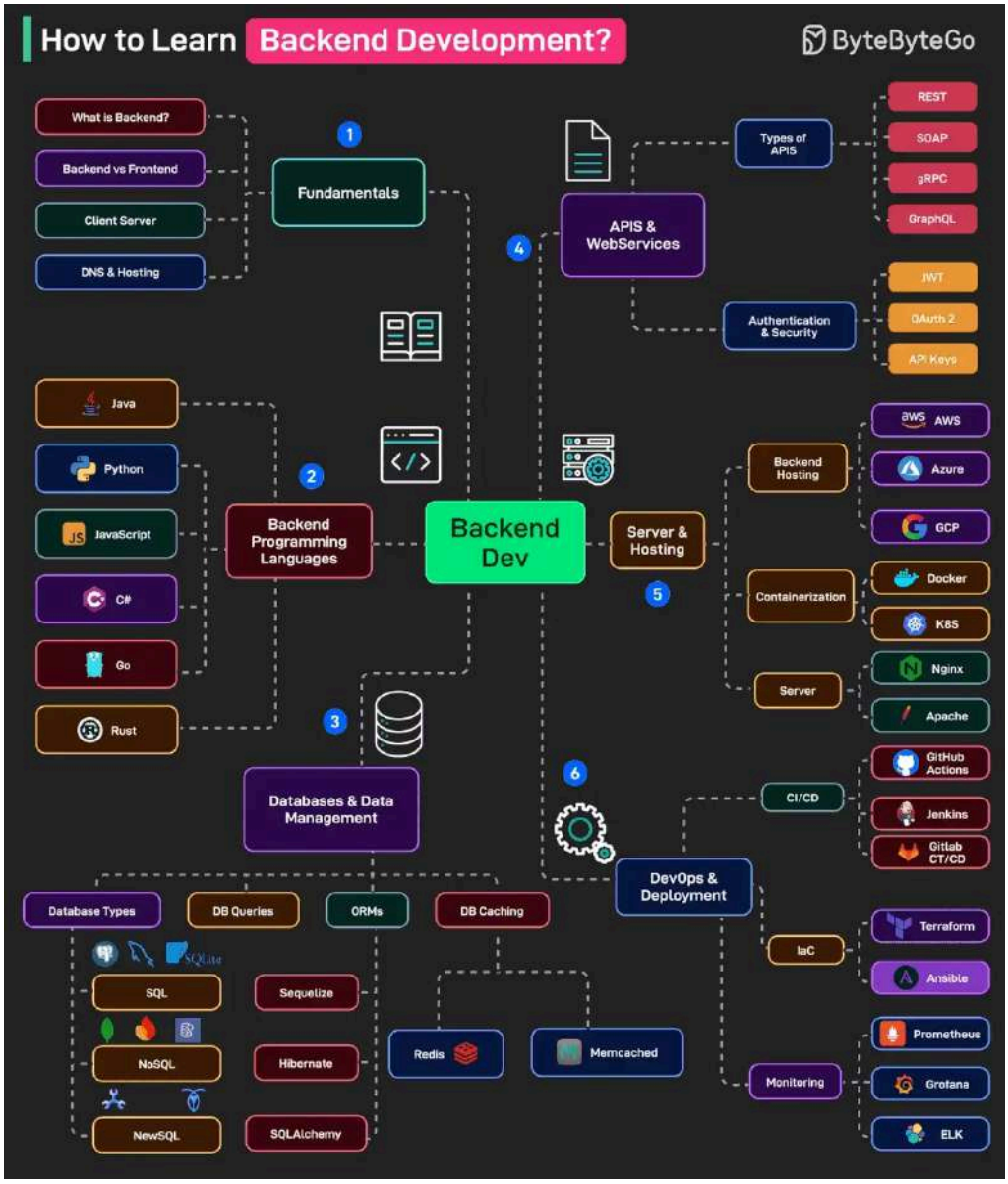
3. Storage and Data Placement Services
   This part handles the physical storage of objects across multiple S3 nodes. To protect against data loss, it uses erasure coding and multi-AZ replication.

4. Durability and Recovery Services
   These services ensure data integrity and fault recovery. They consist of checksum verification, background auditing, and disaster recovery.

5. Security and Compliance Services
   These services protect S3 from unauthorized access and support features such as IAM and bucket Policy, DDoS Mitigation, and Object Lockiand versioning.

Over to you: Have you used Amazon S3?

How to Learn Backend Development?



Backend Development requires knowledge of multiple aspects. Here's a mind map of what all things a developer should learn:

1. Fundamentals
   This includes topics like backend vs frontend, client-server, DNS, etc.

2. Backend Programming Languages
   Choose between one or more programming languages like Java, Python, JS, Go, Rust, and C#.

3. Databases
   This includes topics like types of databases such as SQL (Postgres, MySQL, SQLite), NoSQL (MongoDB, Firebase, DynamoDB), NewSQL (CockroachDB, Spanner). Other topics include working with ORMs and Database Caching.

4. APIs and Web Services
   Learn about API types (REST, GraphQL, gRPC, SOAP) and authentication techniques (like JWT, OAuth 2, API keys).

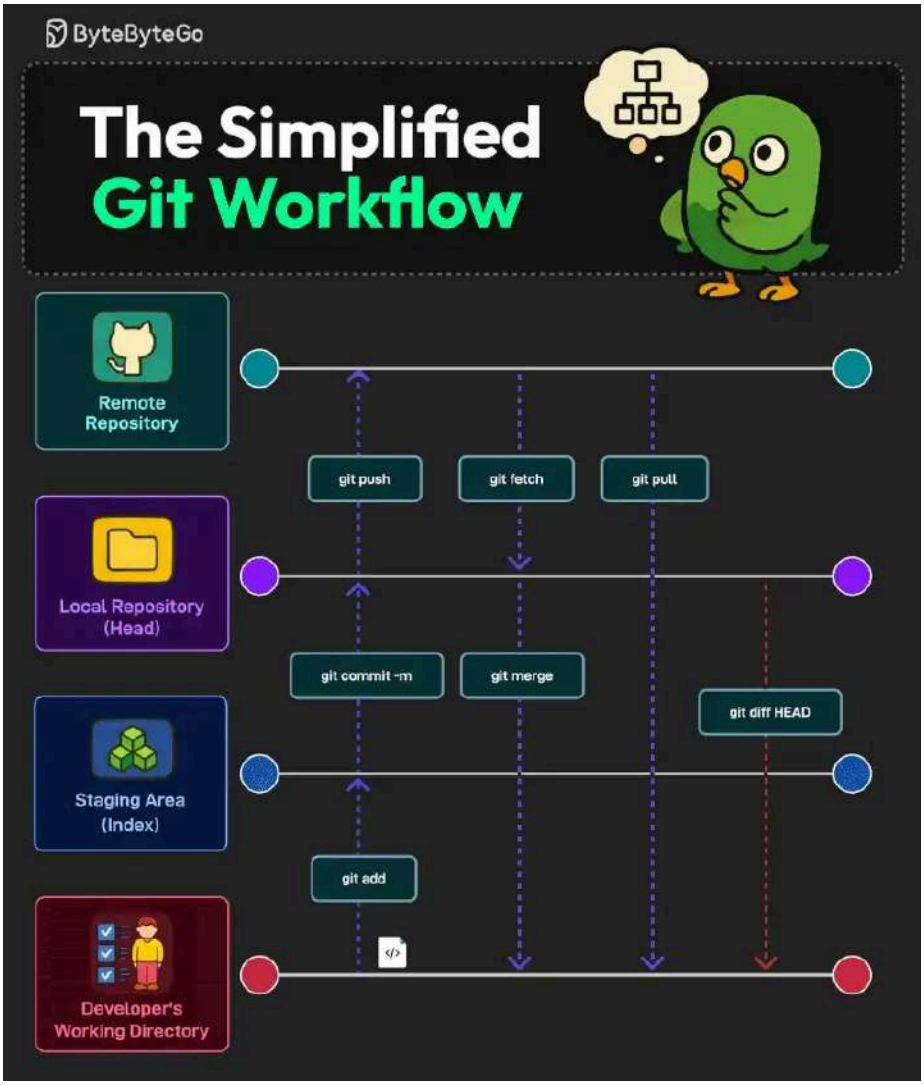
5. Server and Hosting
   This involves topics like backend hosting services (AWS, Azure, GCP), Containerization using Docker & Kubernetes, and Server Setup for Nginx, Apache, etc.

6. DevOps
   Learn about CI/CD Pipelines using GitHub Actions and Jenkins, IaC (Terraform, Ansible) and Monitoring with tools like Prometheus, Grafana, ELK.

Over to you: What else will you add to the list for learning backend development?

A Simplified Git Workflow

Learning Git is one of the fundamental skills for every developer out there. Here are the steps within a simple and basic Git workflow.



1. Developer’s Working Directory (Untracked) to Staging Area (Index)
   The command for the same is “git add”. The files go from untracked to staged.

2. Staging Area to Local Repository (Head)
   The command for this move is “git commit -m “message””. It saves changes to the local repository (HEAD), marking a version history.

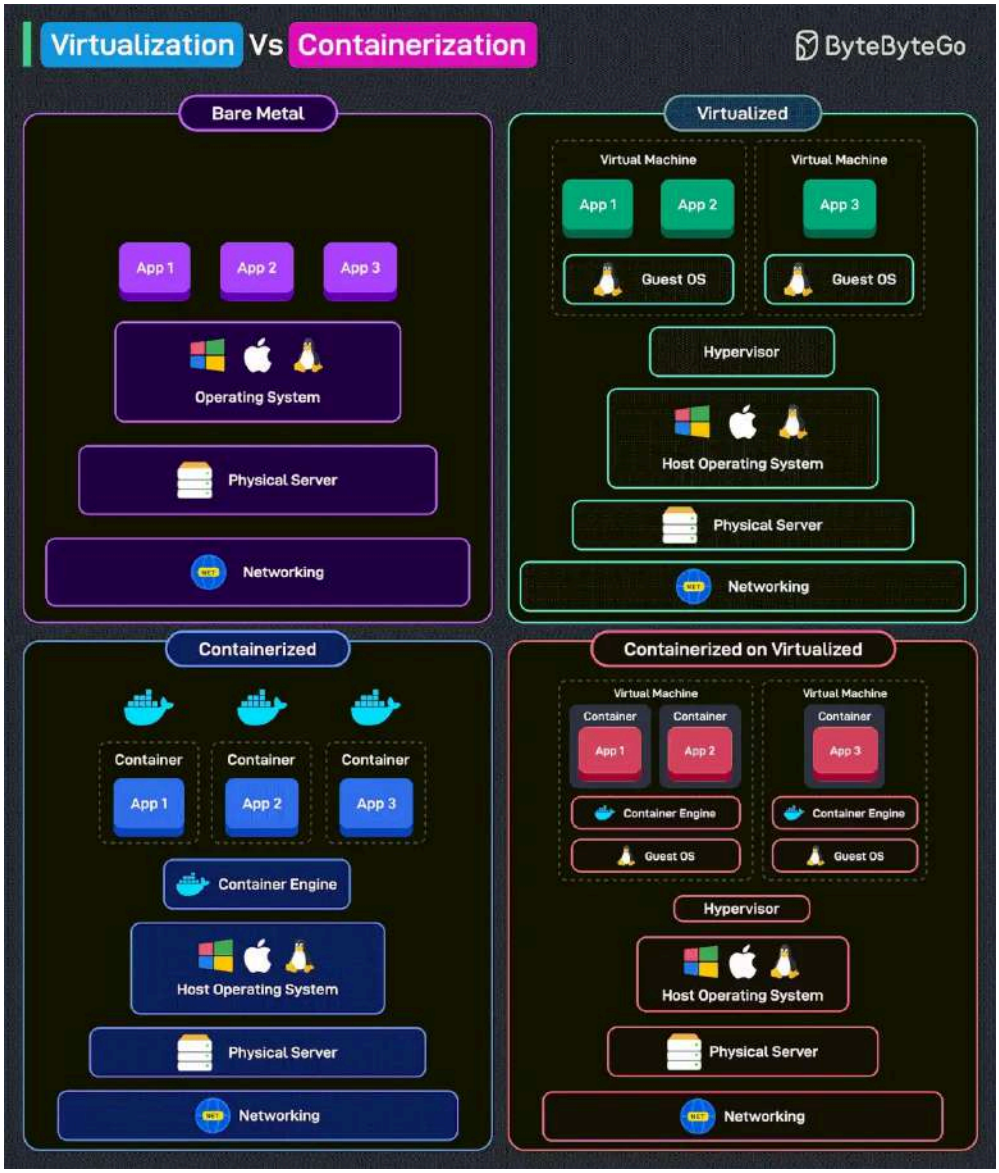
3. Local Repository to Remote Repository (Remote)
   The command for this move is “git push”. It uploads the committed changes to the remote repository (such as on GitHub) for collaboration.

4. Remote Repository to Local Repository
   The commands “git pull” and “git fetch” help with this. “git pull” updates local files with remote changes. On the other hand, “git fetch” retrieves remote changes but does not merge them. The command “git merge” combines changes from different branches.

5. Checking the Differences
   The command “git diff HEAD” shows the differences between the developer's working directory and the latest commit.

Over to you: Which other step do you follow in your Git workflow?

Virtualization vs Containerization



Virtualization creates multiple VMs on a single physical server, each with its operating system, using a hypervisor.

Containerization is a lightweight virtualization method that runs applications in isolated environments (containers) sharing the same operating system.

Let's look at the different possibilities in more detail.

1. Bare Metal: Applications run directly on the operating system. No virtualization or containerization is used. It provides high performance but lacks flexibility.

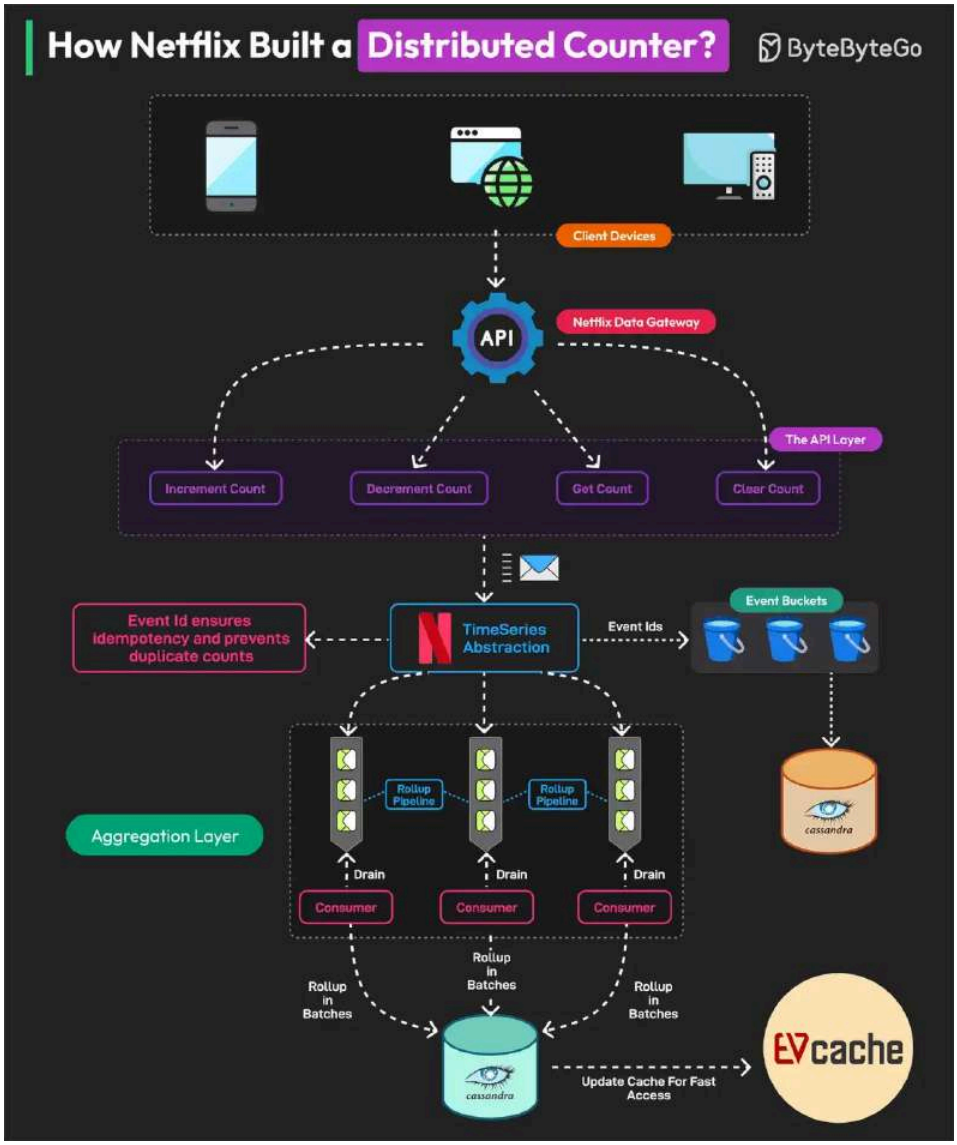
2. Virtualized: Uses a hypervisor to create VMs. Each VM has its guest OS, making it heavier on resources. Provides strong isolation but adds overhead.

3. Containerized: Uses a container engine instead of a hypervisor. Containers share the host operating system, making them lightweight and efficient. It is faster and more scalable than VMs.

4. Containerized on Virtualized: Runs containers inside VMs. Provides both flexibility and isolation, common in hybrid cloud environments. It balances resource efficiency with security.

Over to you: Have you used VMs and Containers to deploy workloads?

How Netflix Built a Distributed Counter?



A Distributed Counter is a system where the responsibility of counting events is spread across multiple servers or nodes in a network. Netflix needs to track and measure multiple user interactions to make real-time decisions and optimize its infrastructure.

For this reason, they built a Distributed Counter Abstraction.

Netflix's Distributed Counter Abstraction operates in four main layers, ensuring high performance, scalability, and eventual consistency.

1. Client API Layer
   Users interact with the system by sending AddCount, GetCount, or ClearCount requests. The Netflix Data Gateway efficiently processes and routes these requests.

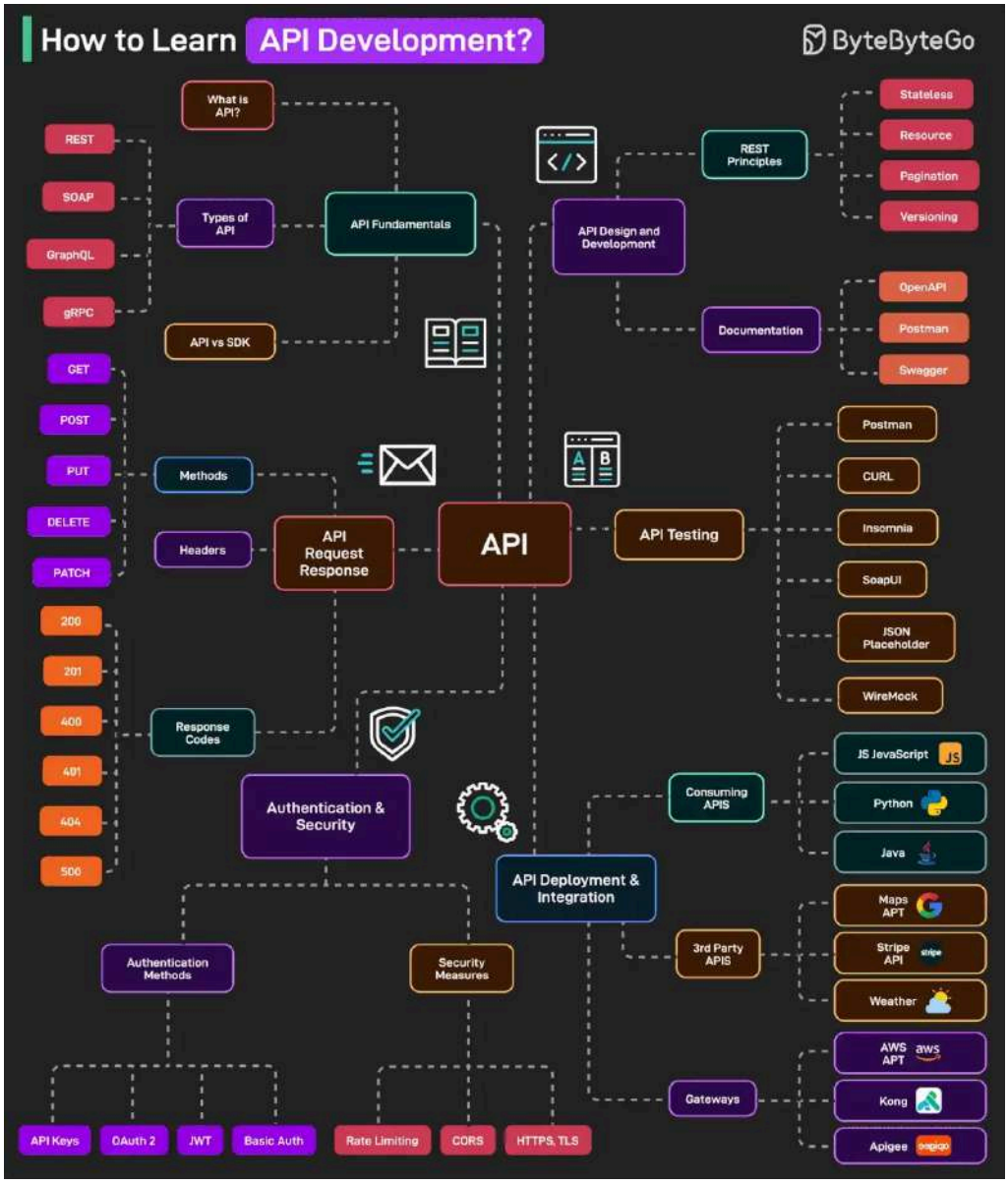
2. Event Logging and TimeSeries Storage
   Events are stored in Netflix TimeSeries Abstraction for scalability. Each event is tagged with an Event ID to ensure idempotency. To avoid database contention, events are grouped into time partitions known as buckets. Data is stored in Cassandra.

3. Rollup Pipeline or Aggregation
   Rollup Queues collect event changes and process them in batches. Aggregation occurs in immutable time windows, ensuring accurate rollup calculations. Data is stored in the Cassandra Rollup Store for eventual consistency.

4. Read Optimization (Cache & Query Handling)
   Aggregated counter values are cached in EVCache for ultra-fast reads. If a cache value is stale, a background rollup refresh updates it. This model allows Netflix to process 75K requests per second with single-digit millisecond latency.

How to Learn API Development?

Learning how to develop APIs is an important skill for modern-day developers. Here's a mind map of what all you need to learn about API development:



1. API Fundamentals
   What is an API, types of API (REST, SOAP, GraphQL, gRPC, etc.), and API vs SDK.

2. API Request/Response
   HTTP Methods, Response Codes, and Headers.

3. Authentication and Security
   Authentication mechanisms (JWT, OAuth 2, API Keys, Basic Auth) and security strategies.

4. API Design and Development
   RESTful API principles include stateless, resource-based URL, versioning, and pagination. Also, API documentation tools like OpenAPI, Postman, Swagger.

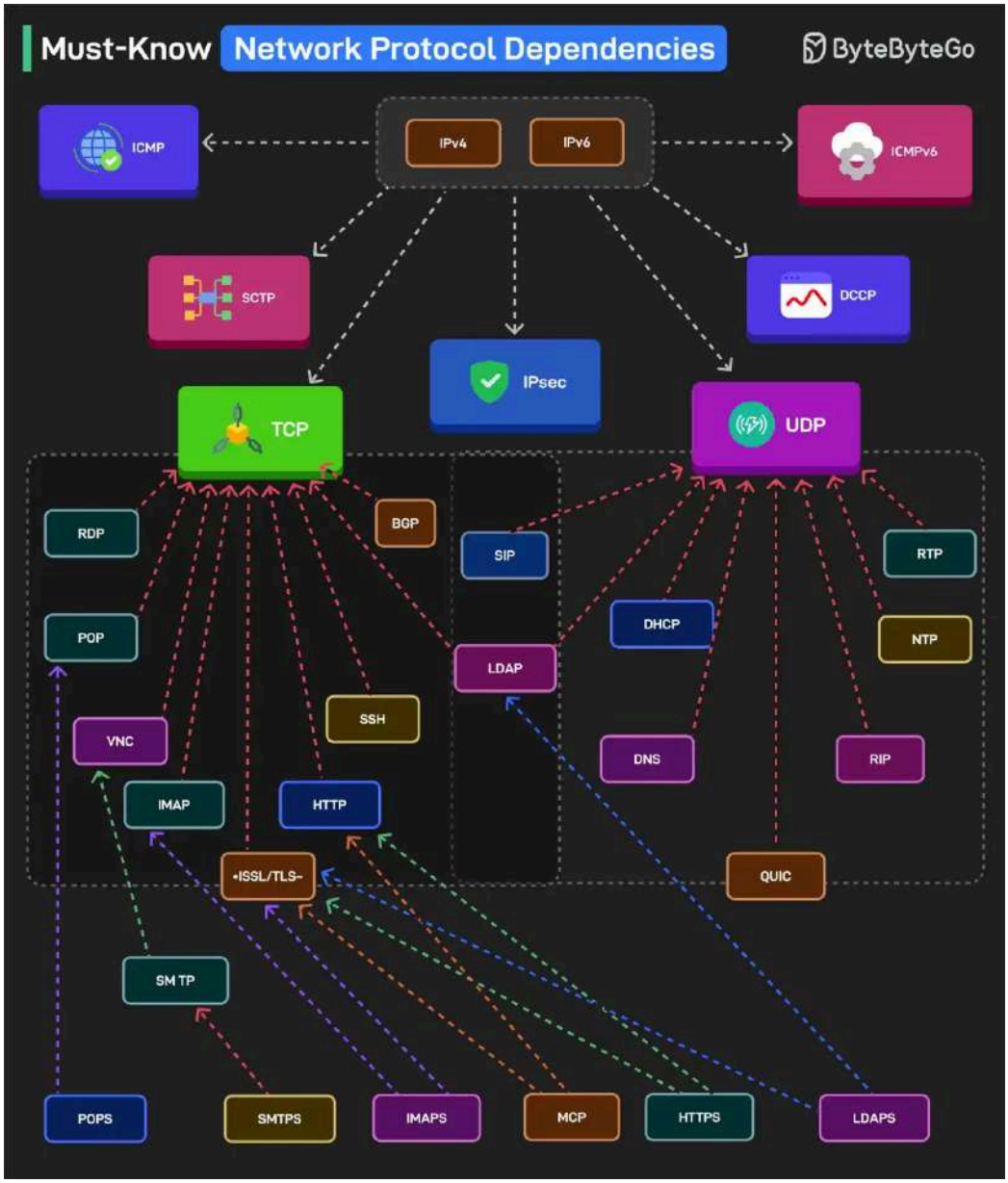
5. API Testing
   Tools for testing APIs such as Postman, cURL, SoapUI, and so on.

6. API Deployment and Integration
   Consuming APIs in different languages like JS, Python, and Java. Also, working with 3rd party APIs like the Google Maps API and the Stripe API. Learn about API Gateways like AWS, Kong, Apigee.

Over to you: What else will you add to the list for learning API development?

Must-Know Network Protocol Dependencies

Understanding network protocol dependencies is essential for cybersecurity and networking. Here's a quick understanding of the same:

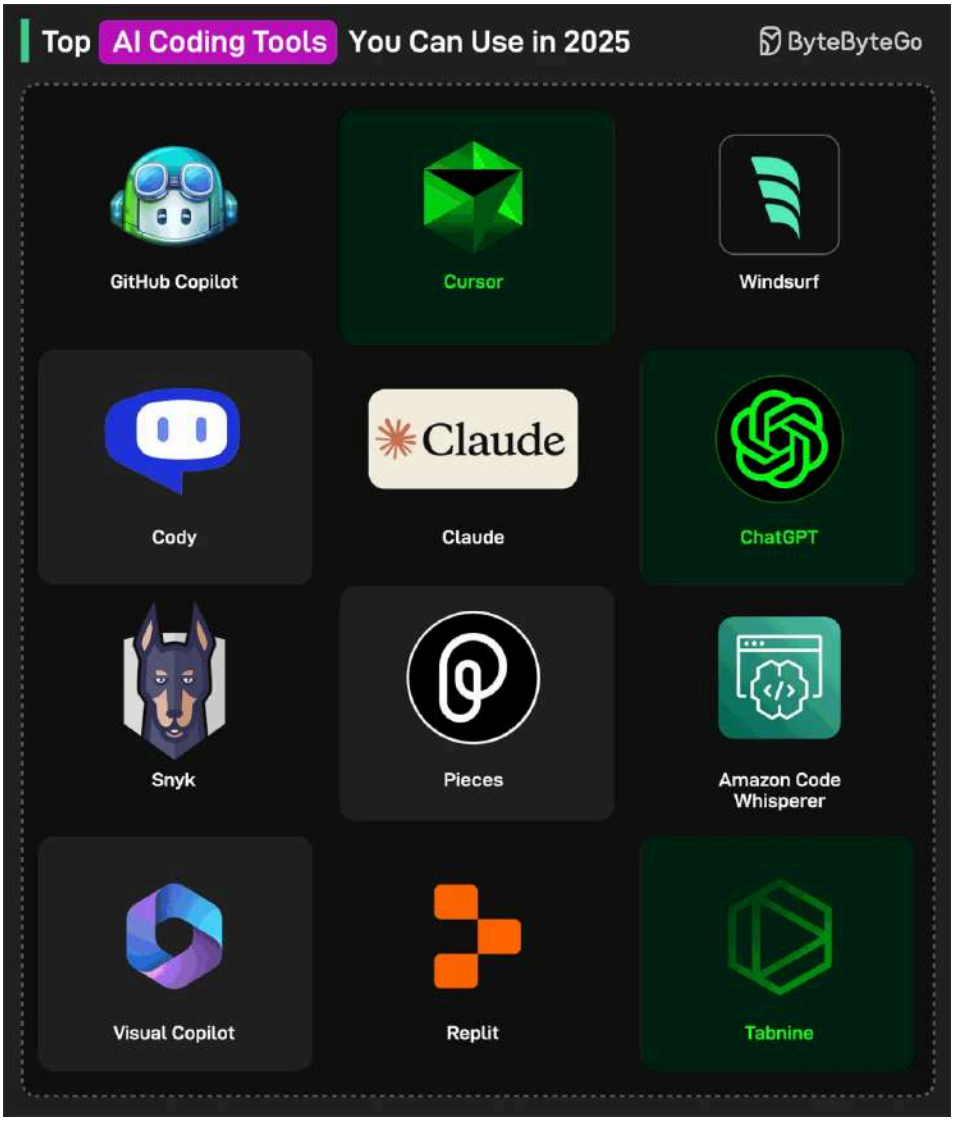


1. IPv4 and IPv6 are the foundation of all networking. ICMP and ICMPv6 handle diagnostics, while IPsec ensures secure communication.
2. TCP and UDP support various protocols. SCTP and DCCP serve specific cases.
3. Some TCP-based protocols are HTTP, SSH, BGP, RDP, IMAP, SMTP, POP, etc.
4. UDP-based protocols are DNS, DHCP, SIP, RTP, NTP, etc.

5. SSL/TLS encrypts HTTPS, IMAPS, and SMTPS.
6. LDAP and LDAPs are used for directory services over TCP and secured with SSL/TLS.
7. QUIC is a UDP-based replacement for TCP+TLS for faster, encrypted connections.
8. MCP or Model Context Protocol is an emerging standard for communicating with LLMs.

Over to you: Which other network protocol will you add to the list?

Top AI Coding Tools for Developers You Can Use in 2025



1. AI Code Assistants
   GitHub Copilot: Code completion and automatic programming tool.

   ChatGPT: Helps write and debug code with the latest models.

   Claude: Recent and specialized coding knowledge to generate accurate and up-to-date code.

   Amazon CodeWhisperer: AI Assistant in the IDE

2. AI-Powered IDEs
   Cursor: AI-powered IDE for Windows, macOS, and Linux.

   Windsurf: AI-powered IDE that tackles complex tasks independently.

   Replit: Create fully working apps to go live fast.

3. Team Productivity
   Cody: The enterprise AI code assistant for writing, fixing, and maintaining code.

   Pieces: AI-enabled productivity tool to help developers manage code snippets.

   Visual Copilot: Convert Figma designs into React, Vue, Svelte, Angular, or HTML code.

4. Code Quality and Completion
   Snyk: Real-time vulnerability scanning of human and AI-generated code.

   Tabnine: A code completion tool to accelerate software development.

Over to you: Which other AI Coding Tool will you add to the list? What's your favorite?

18 Key Design Patterns Every Developer Should Know



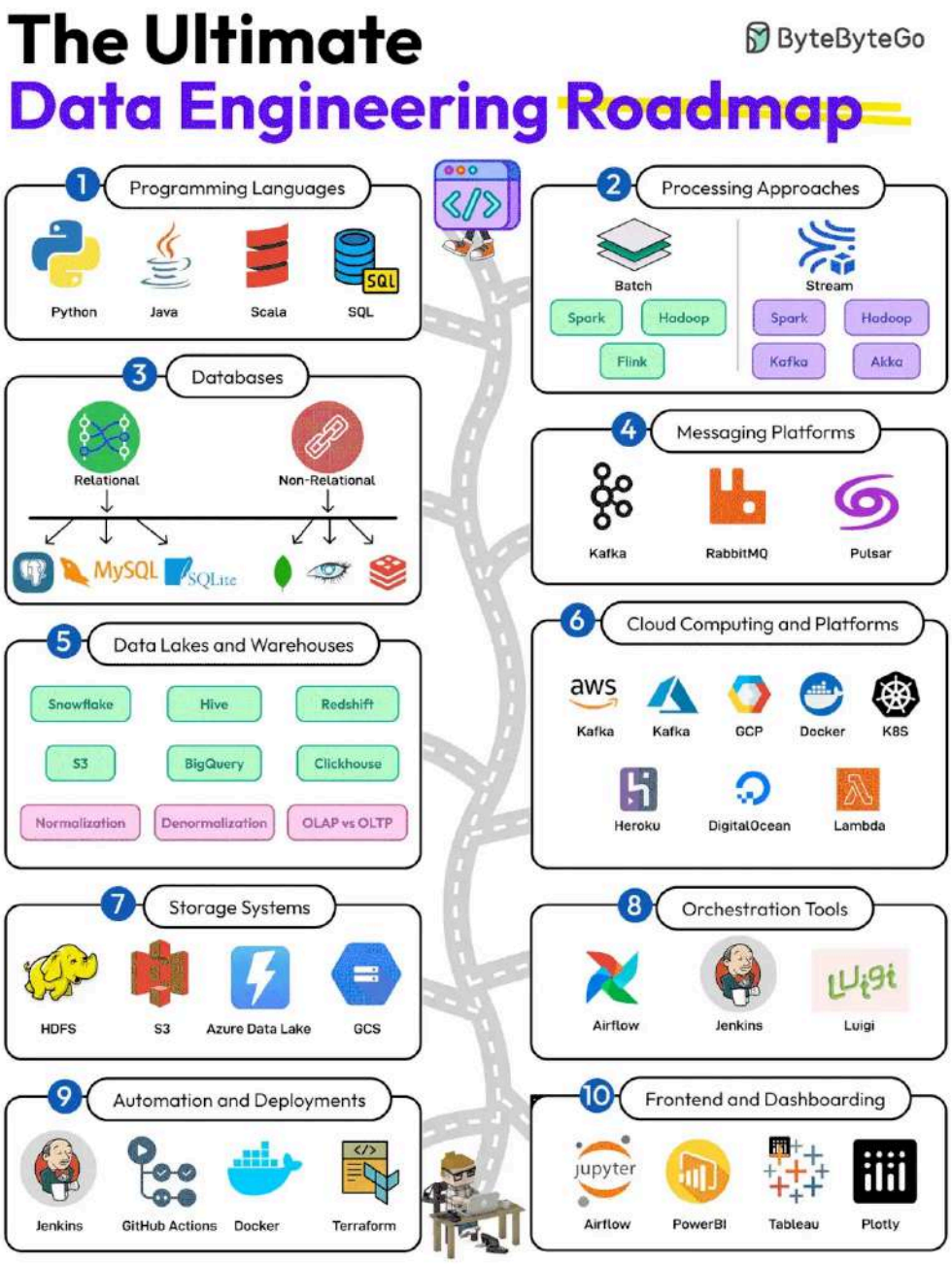
Patterns are reusable solutions to common design problems, resulting in a smoother, more efficient development process. They serve as blueprints for building better software structures. These are some of the most popular patterns:

- Abstract Factory: Family Creator - Makes groups of related items.
- Builder: Lego Master - Builds objects step by step, keeping creation and appearance

- Prototype: Clone Maker - Creates copies of fully prepared examples.
- Singleton: One and Only - A special class with just one instance.
- Adapter: Universal Plug - Connects things with different interfaces.
- Bridge: Function Connector - Links how an object works to what it does.
- Composite: Tree Builder - Forms tree-like structures of simple and complex parts.
- Decorator: Customizer - Adds features to objects without changing their core.
- Facade: One-Stop-Shop - Represents a whole system with a single, simplified interface.
- Flyweight: Space Saver - Shares small, reusable items efficiently.
- Proxy: Stand-In Actor - Represents another object, controlling access or actions.
- Chain of Responsibility: Request Relay - Passes a request through a chain of objects until handled.
- Command: Task Wrapper - Turns a request into an object, ready for action.
- Iterator: Collection Explorer - Accesses elements in a collection one by one.
- Mediator: Communication Hub - Simplifies interactions between different classes.
- Memento: Time Capsule - Captures and restores an object's state.
- Observer: News Broadcaster - Notifies classes about changes in other objects.
- Visitor: Skillful Guest - Adds new operations to a class without altering it.

The Data Engineering Roadmap

Data engineering has become the backbone of effective data analysis. It involves managing, processing, and optimizing data to derive actionable insights.



Here's a roadmap that can help you get better at data engineering:

1. Programming Languages
   Learn SQL and a few programming languages like Python, Java, and Scala.

2. Processing Techniques
   Learn batch processing tools like Spark and Hadoop and stream processing tools like Flink and Kafka.

3. Databases
   Focus on both relational and non-relational databases. Some examples are MySQL, Postgres, MongoDB, Cassandra, and Redis.

4. Messaging Platforms
   Master the use of platforms like Kafka, RabbitMQ, and Pulsar.

5. Data Lakes and Warehouses
   Learn about various data lake and warehousing solutions such as Snowflake, Hive, S3, Redshift, and Clickhouse. Also, learn about Normalization, Denormalization, and OLTP vs OLAP.

6. Cloud Computing Platforms
   Master the use of cloud platforms like AWS, Azure, Docker, and K8S

7. Storage Systems
   Learn about the key storage systems like S3, Azure Data Lake, and HDFS

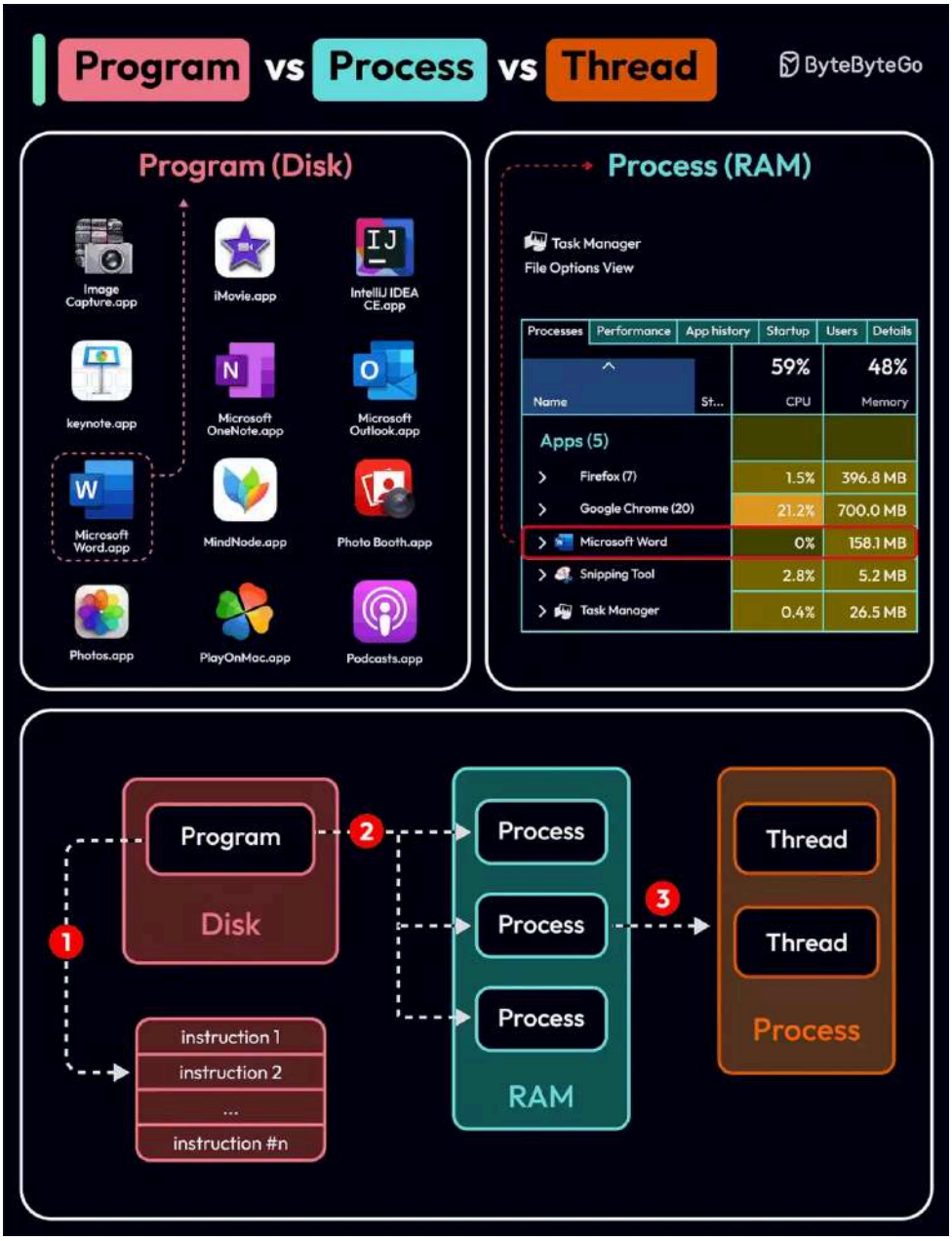
8. Orchestration Tools
   Learn about orchestration tools like Airflow, Jenkins, and Luigi

9. Automation and Deployments
   Learn automation tools such as Jenkins, Github Actions, and Terraform.

10. Frontend and Dashboarding
    Master the use of tools like Jupyter Notebooks, PowerBI, Tableau, and Plotty

Over to you: What else will you add to the Data Engineering Roadmap?

Popular interview question: What is the difference between Process and Thread?



To better understand this question, let’s first take a look at what is a Program. A Program is an executable file containing a set of instructions and passively stored on disk. One program can have multiple processes. For example, the Chrome browser creates a different process for every single tab.

A Process means a program is in execution. When a program is loaded into the memory and becomes active, the program becomes a process. The process requires some essential resources such as registers, program counter, and stack.

A Thread is the smallest unit of execution within a process.

The following process explains the relationship between program, process, and thread.

1. The program contains a set of instructions.
2. The program is loaded into memory. It becomes one or more running processes.
3. When a process starts, it is assigned memory and resources. A process can have one or more threads. For example, in the Microsoft Word app, a thread might be responsible for spelling checking and the other thread for inserting text into the doc.

Main differences between process and thread:

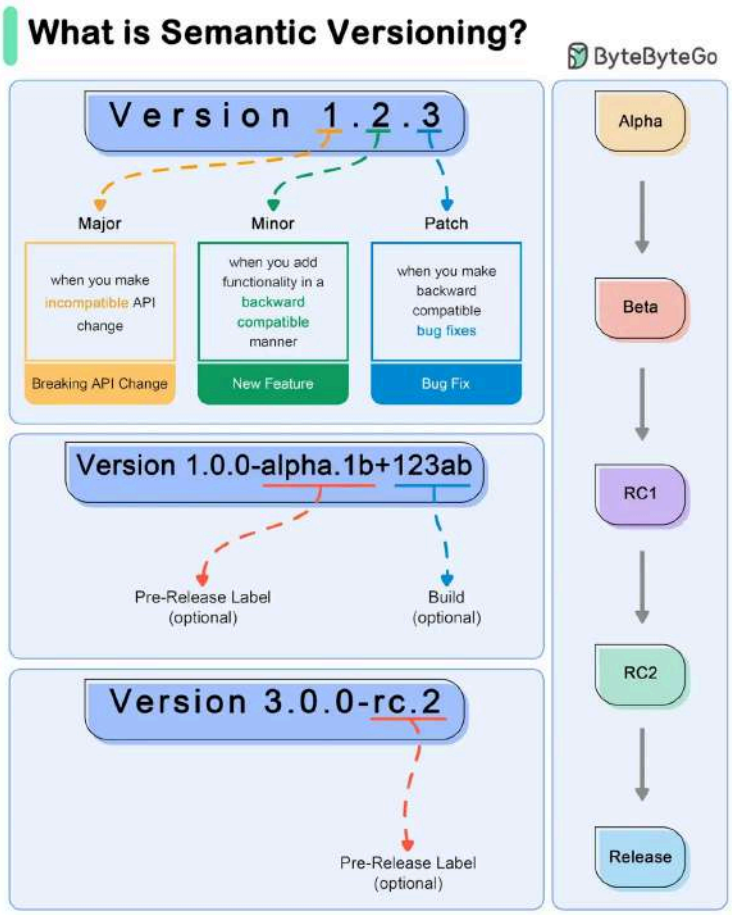
- Processes are usually independent, while threads exist as subsets of a process.
- Each process has its own memory space. Threads that belong to the same process share the same memory.
- A process is a heavyweight operation. It takes more time to create and terminate.
- Context switching is more expensive between processes.
- Inter-thread communication is faster for threads.

Over to you:

1. Some programming languages support coroutines. What is the difference between coroutine and thread?
2. How to list running processes in Linux?

What do version numbers mean?

Semantic Versioning (SemVer) is a versioning scheme for software that aims to convey meaning about the underlying changes in a release.



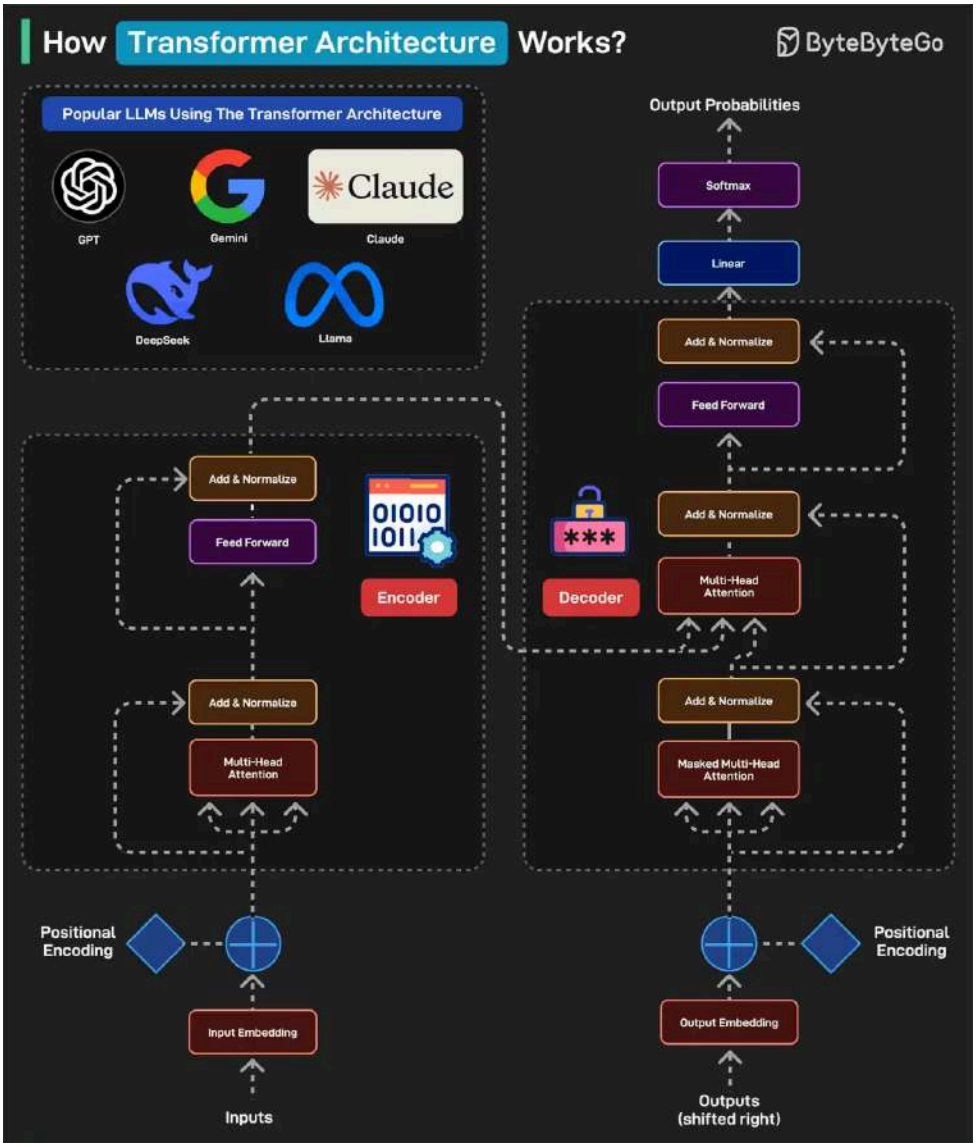
- SemVer uses a three-part version number: MAJOR.MINOR.PATCH.
    1. MAJOR version: Incremented when there are incompatible API changes.
    2. MINOR version: Incremented when functionality is added in a backward-compatible manner.
    3. PATCH version: Incremented when backward-compatible bug fixes are made.
- Example Workflow
    1. Initial Development Phase
       Start with version 0.1.0.
    2. First Stable Release
       Reach a stable release: 1.0.0.

3. Subsequent Changes
   Patch Release: A bug fix is needed for 1.0.0. Update to 1.0.1.
   Minor Release: A new, backward-compatible feature is added to 1.0.3. Update to 1.1.0.
   Major Release: A significant change that is not backward-compatible is introduced in 1.2.2. Update to 2.0.0.

4. Special Versions and Pre-releases
   Pre-release Versions: 1.0.0-alpha, 1.0.0-beta, 1.0.0-rc.1.
   Build Metadata: 1.0.0+20130313144700.

Everyone talks about Transformers. How Transformers Architecture Works?

Transformers Architecture has become the foundation of some of the most popular LLMs including GPT, Gemini, Claude, DeepSeek, and Llama.



Here's how it works:

1. A typical transformer-based model has two main parts: encoder and decoder. The encoder reads and understands the input. The decoder uses this understanding to generate the correct output.
2. In the first step (Input Embedding), each word is converted into a number (vector) representing its meaning.

3. Next, a pattern called Positional Encoding tells the model where each word is in the sentence. This is because the word order matters in a sentence. For example “the cat ate the fish” is different from “the fish ate the cat”.

4. Next is the Multi-Head Attention, which is the brain of the encoder. It allows the model to look at all words at once and determine which words are related. In the Add & Normalize phase, the model adds what it learned from attention back into the sentence.

5. The Feed Forward process adds extra depth to the understanding. The overall process is repeated multiple times so that the model can deeply understand the sentence.

6. After the encoder finishes, the decoder kicks into action. The output embedding converts each word in the expected output into numbers. To understand where each word should go, we add Positional Encoding.

7. The Masked Multi-Head Attention hides the future words so the model predicts only one word at a time.

8. The Multi-Head Attention phase aligns the right parts of the input with the right parts of the output. The decoder looks at both the input sentence and the words it has generated so far.

9. The Feed Forward applies more processing to make the final word choice better. The process is repeated several times to refine the results.

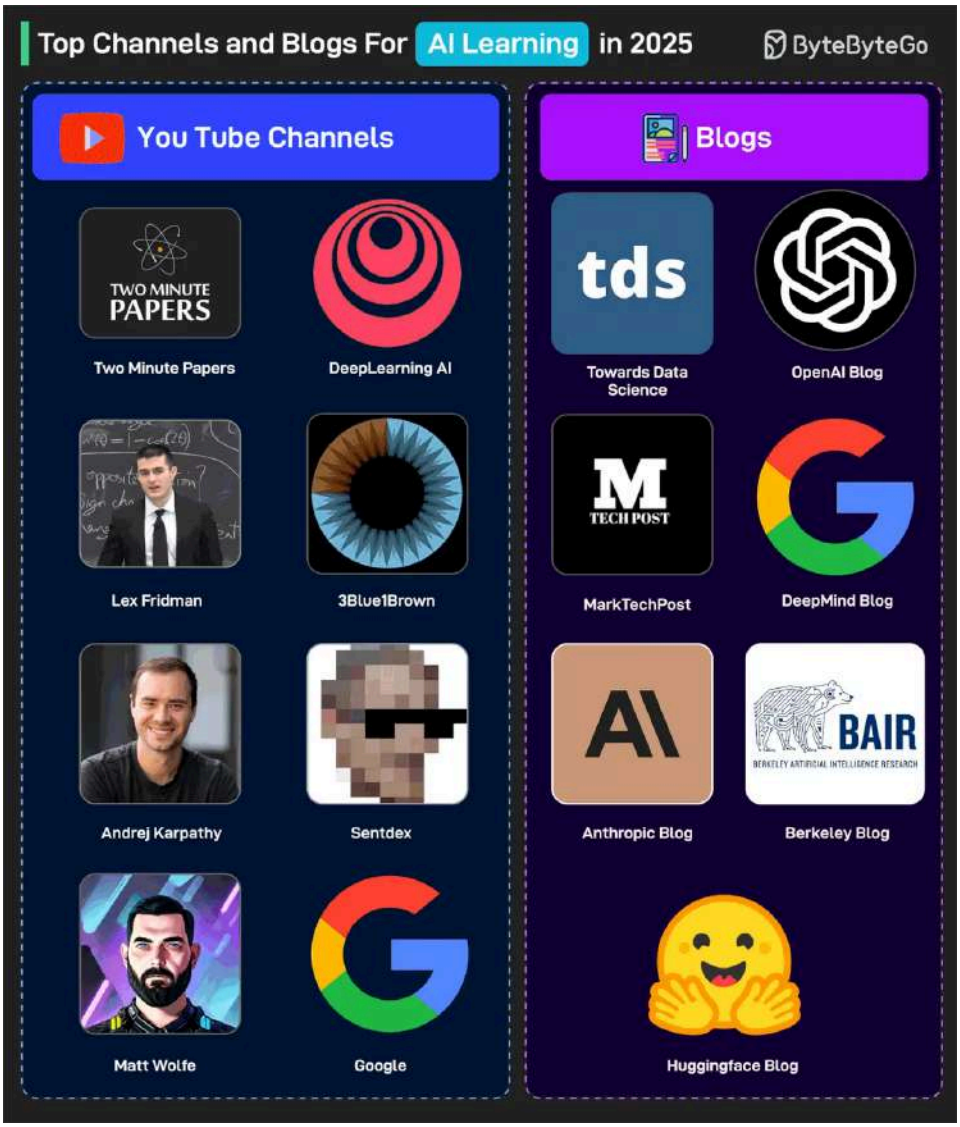
10. Once the decoder has predicted numbers for each word, it passes them through a Linear Layer to prepare for output. This layer maps the decoder’s output to a large set of possible words.

11. After the Linear Layer generates scores for each word, the Softmax layer converts those scores into probabilities. The word with the highest probability is chosen as the next word.

12. Finally, a human-readable sentence is generated.

Over to you: What else will you add to understand the Transformer Architecture?

Top YouTube Channels and Blogs for AI Learning in 2025



Some great YouTube Channels are:

1. Two Minute Papers
2. DeepLearning AI
3. Lex Fridman
4. 3Blue1Brown

5. Andrej Karpathy
6. Sentdex
7. Matt Wolfe
8. Google YouTube Channel

Also, here are some great blogs focusing on AI:

1. TowardsDataScience
2. OpenAI Blog
3. MarkTechPost
4. DeepMind Blog
5. Anthropic Blog
6. Berkeley Bair
7. Huggingface Blog

Over to you: Which other channel and blog will you add to the list?

Top 20 System Design Concepts You Should Know

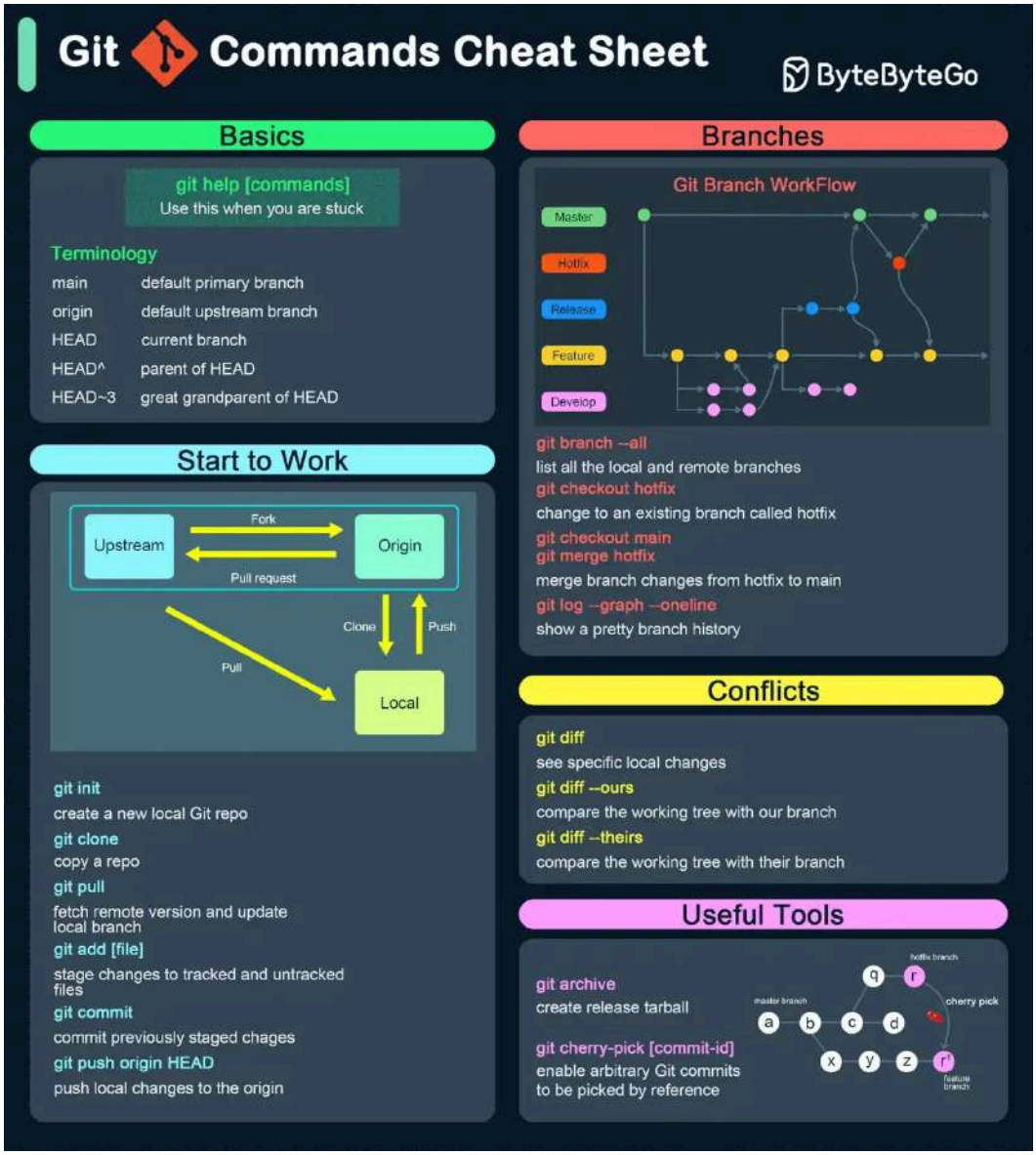


1. Load Balancing: Distributes traffic across multiple servers for reliability and availability.
2. Caching: Stores frequently accessed data in memory for faster access.
3. Database Sharding: Splits databases to handle large-scale data growth.
4. Replication: Copies data across replicas for availability and fault tolerance.
5. CAP Theorem: Trade-off between consistency, availability, and partition tolerance.
6. Consistent Hashing: Distributes load evenly in dynamic server environments.
7. Message Queues: Decouples services using asynchronous event-driven architecture.

8. Rate Limiting: Controls request frequency to prevent system overload.
9. API Gateway: Centralized entry point for routing API requests.
10. Microservices: Breaks systems into independent, loosely coupled services.
11. Service Discovery: Locates services dynamically in distributed systems.
12. CDN: Delivers content from edge servers for speed.
13. Database Indexing: Speeds up queries by indexing important fields.
14. Data Partitioning: Divides data across nodes for scalability and performance.
15. Eventual Consistency: Guarantees consistency over time in distributed databases
16. WebSockets: Enables bi-directional communication for live updates.
17. Scalability: Increases capacity by upgrading or adding machines.
18. Fault Tolerance: Ensures system availability during hardware/software failures.
19. Monitoring: Tracks metrics and logs to understand system health.
20. Authentication & Authorization: Controls user access and verifies identity securely.

Over to you: Which other System Design concept will you add to the list?

Essential Git Cheatsheet!



🔧 Basic Commands

git init – Initialize a new Git repository.

git clone <repo_url> – Clone a remote repository.

git status – Check the status of your working directory.

git add <file> – Stage changes for commit.

git commit -m "message" – Commit staged changes with a message.

git push – Push your local commits to the remote repository.

git pull – Fetch and merge changes from the remote repo.

git diff – Show changes in the working directory (uncommitted changes).

git diff --staged – Show changes between the staging area and last commit.

🛠️ Branching & Merging

git branch – List branches.

git branch <branch_name> – Create a new branch.

git checkout <branch_name> – Switch to another branch.

git checkout -b <branch_name> – Create and switch to a new branch.

git merge <branch_name> – Merge a branch into the current one.

git branch -d <branch_name> – Delete a branch after merging.

git branch -D <branch_name> – Forcefully delete a branch, even if it hasn't merged.

🔄 Synchronization

git fetch – Download changes from remote without merging.

git rebase <branch> – Reapply commits on top of another branch to maintain linear history.

git pull --rebase – Fetch and reapply your changes on top of the latest remote changes.

git remote add <name> <url> – Add a new remote repository.

🎯 Advanced Git

git stash – Temporarily save changes without committing.

git stash pop – Reapply stashed changes.

git cherry-pick <commit> – Apply a specific commit to your current branch.

git log --oneline – View simplified commit history.

git reflog – Show the history of your reference changes (e.g., checkout, resets).

git log --graph --decorate --all – Show a visual commit history.

🚨 Undoing Changes

git reset <file> – Unstage a file.

git reset --soft <commit> – Reset to a commit but keep changes in the working directory.

git reset --hard <commit> – Completely reset to a previous commit, discarding changes.

git revert <commit> – Create a new commit that undoes a specific commit.

⚙️ Collaborating with Others

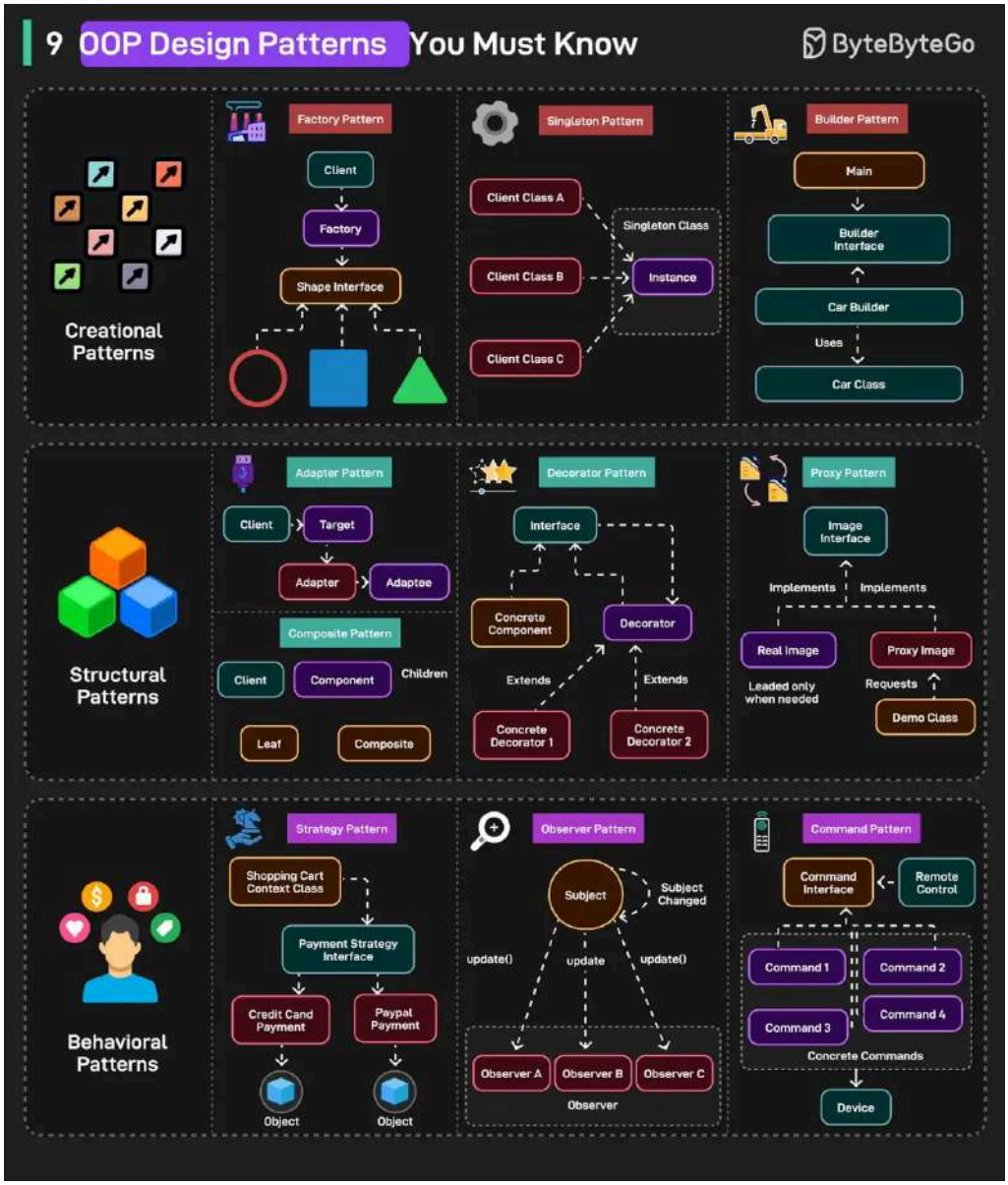
git fork – Fork a repository on GitHub (via UI) to start contributing.

git pull origin <branch> – Pull changes from the original remote branch.

git push origin <branch> – Push your branch to the original repository for collaboration.

Over to you: did we miss anything?

9 OOP Design Patterns You Must Know



These patterns can be divided into 3 categories:
A - Creational Patterns
Deal with object creation mechanism to decouple the client code from concrete classes.

1. Factory Pattern: Centralizes object creation logic and returns different subclasses based on input

2. Singleton Pattern: Ensures only one instance of a class exists and provides global access to it.

3. Builder Pattern: Constructs complex objects step-by-step, allowing optional configuration.

B - Structural Patterns
Help compose classes and objects into larger structures.

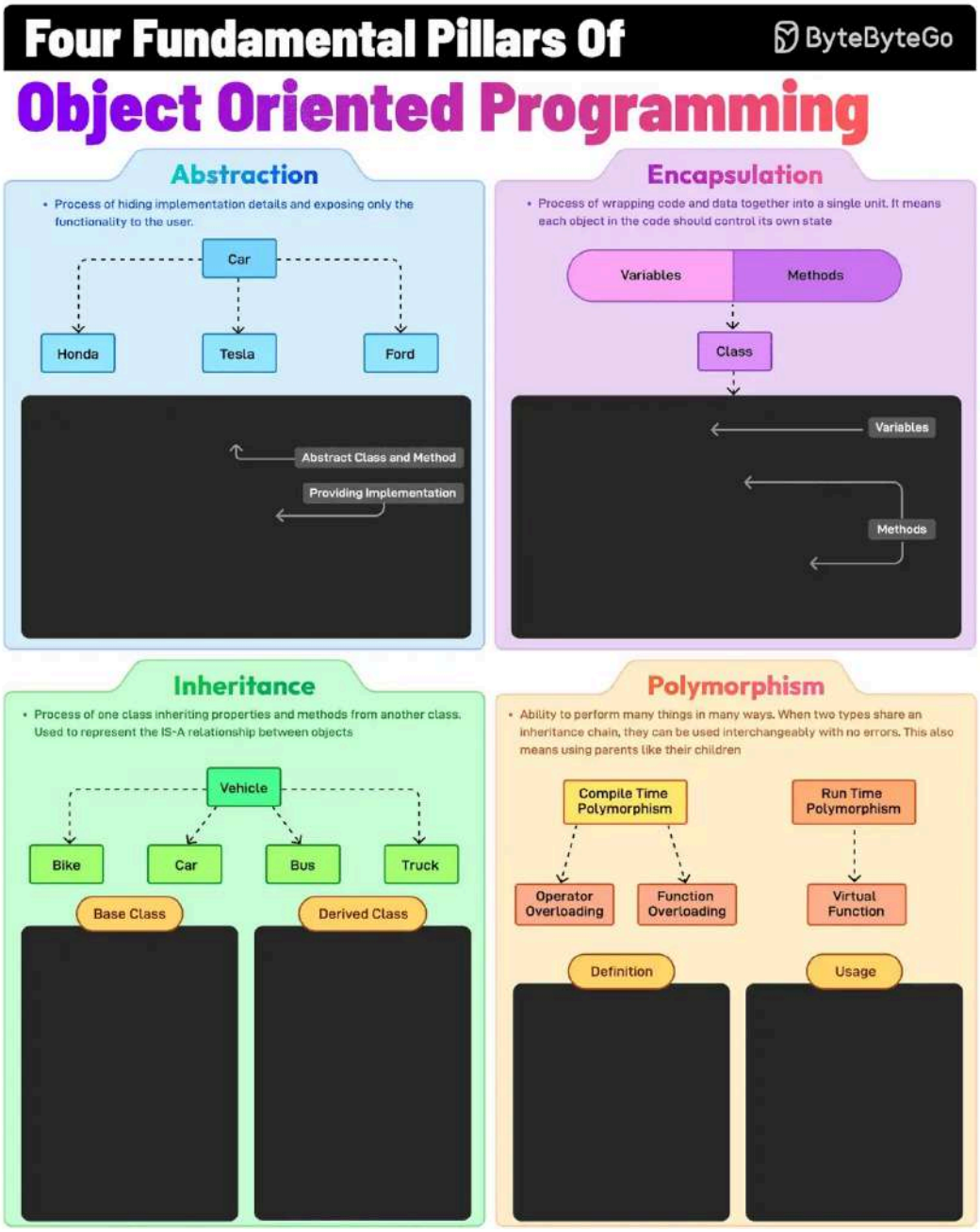
1. Adapter Pattern: Allows incompatible interfaces to work together by translating one interface into another.
2. Decorator Pattern: Adds new behavior to objects dynamically without altering their original structure.
3. Proxy Pattern: Acts as a placeholder for accessing another object.

C - Behavioral Patterns
Focus on communication and interaction between objects.

1. Strategy Pattern: Allows selecting an algorithm or behavior from a family of interchangeable strategies at runtime.
2. Observer Pattern: Enables a one-to-many dependency so that when one object changes state, all its dependents are notified.
3. Command Pattern: An object encapsulates all information needed to perform an action or trigger an event.

Over to you: Which of these patterns have you used?

The Fundamental Pillars of Object-Oriented Programming



Abstraction, Encapsulation, Inheritance, and Polymorphism are the four pillars of object-oriented programming. What do they mean?

1. Abstraction
   This is the process of hiding implementation details and showing only the essential features

of an object. For example, a Vehicle class with an abstract stop method.

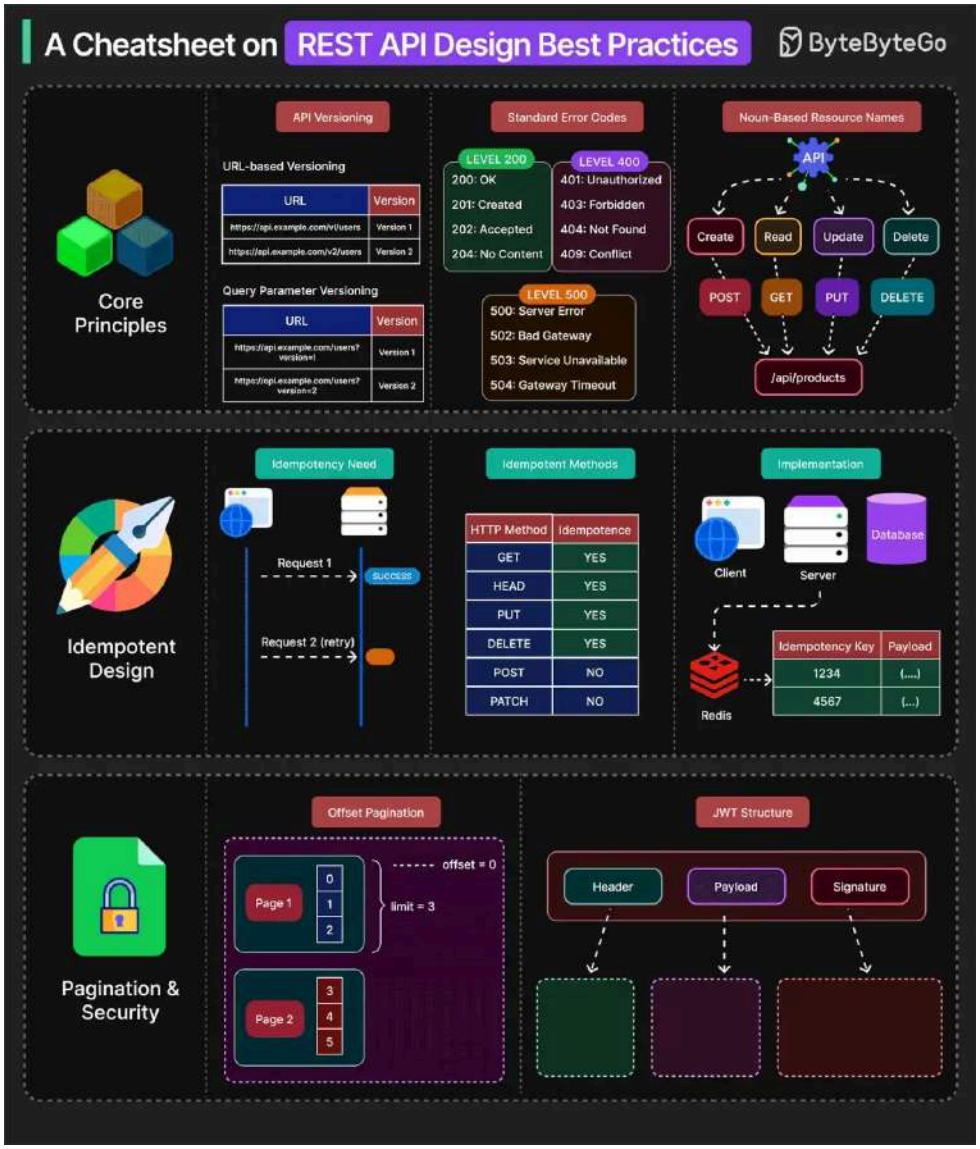
2. Encapsulation
   It involves wrapping data (fields) and methods in a single unit (class) and restricting direct access using access modifiers. For example, private fields with public getters and setters.

3. Inheritance
   The process of creating a new class (child) that inherits attributes and methods from an existing class (parent), thereby promoting code reuse. For example, a Car class inherits from a Vehicle class.

4. Polymorphism
   It allows methods to perform differently based on the object they are invoked on. When two types share an inheritance chain, they can be used interchangeably with no errors.

Over to you: Do you use object-oriented programming?

A Cheatsheet on REST API Design Best Practices



Well-designed APIs behave consistently, fair predictably, and grow without friction. Some best practices to keep in mind are as follows:

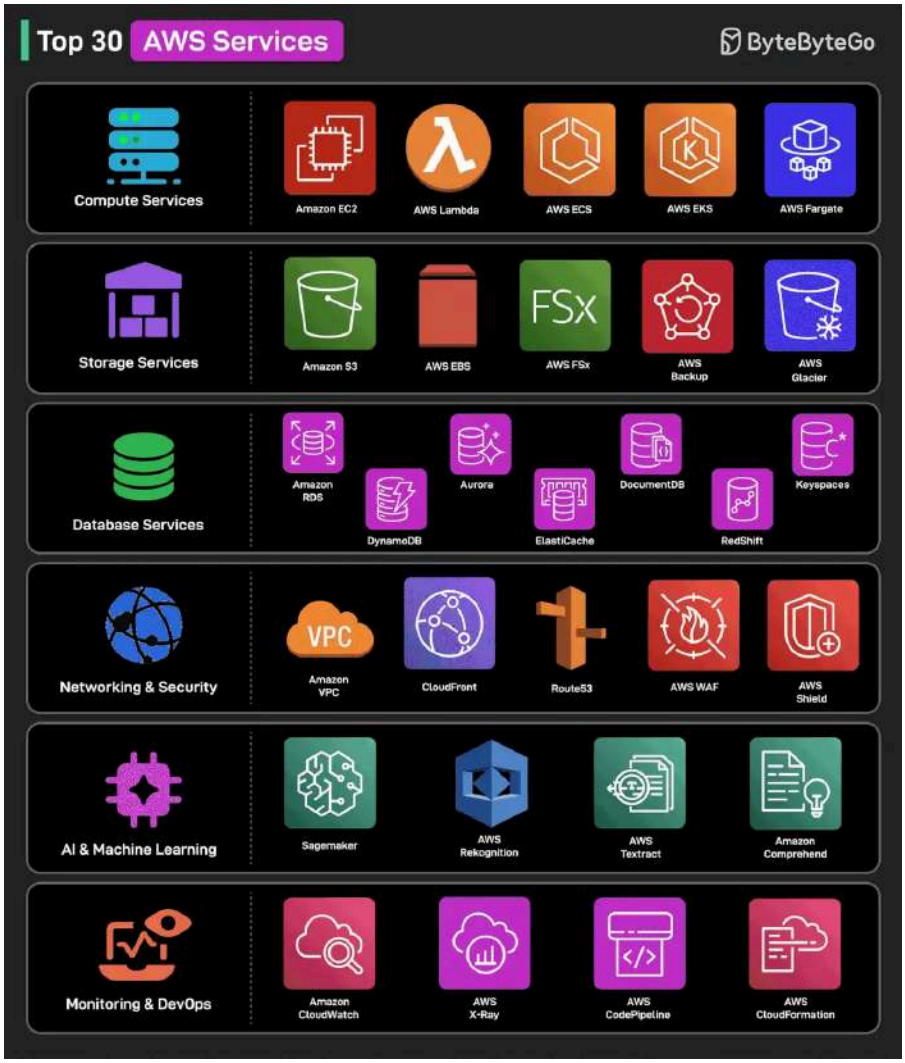
1. Resource-oriented paths and proper use of HTTP verbs help APIs align with standard tools.
2. Use a proper API versioning approach.
3. Use standard error codes while generating API responses.
4. APIs should be idempotent. They ensure safe retries by making repeated requests to produce the same result, especially for POST operations.
5. Idempotency keys allow clients to safely deduplicate operations with side effects.

6. APIs should support pagination to prevent performance bottlenecks and payload bloat. Some common pagination strategies are offset-based, cursor-based, and keyset-based.

7. API security is mandatory for well-designed APIs. Use proper authentication and authorization with APIs using API Keys, JWTs, OAuth2, and other mechanisms. HTTPS is also a must-have for APIs running in production.

Over to you: Which other best practices do you follow while designing APIs?

Top 30 AWS Services That Are Commonly Used



We group them by category and understand what they do.

Compute Services
1 - Amazon EC2: Virtual servers in the cloud
2 - AWS Lambda: Serverless functions for event-driven workloads
3 - Amazon ECS: Managed container orchestration
4 - Amazon EKS: Kubernetes cluster management service
5 - AWS Fargate: Serverless compute for containers

Storage Services
6 - Amazon S3: Scalable secure object storage
7 - Amazon EBS: Block storage for EC2 instances
8 - Amazon FSx: Fully managed file storage
9 - AWS Backup: Centralized backup automation

10 - Amazon Glacier: Archival cold storage for backups

Database Services
11 - Amazon RDS: Managed relational database service
12 - Amazon DynamoDB: NoSQL database with low latency
13 - Amazon Aurora: High-performance cloud-native database
14 - Amazon Redshift: Scalable data warehousing solution
15 - Amazon Elasticache: In-memory caching with Redis/Memcached
16 - Amazon DocumentDB: NoSQL document database (MongoDB-compatible)
17 - Amazon Keyspaces: Managed Cassandra database service

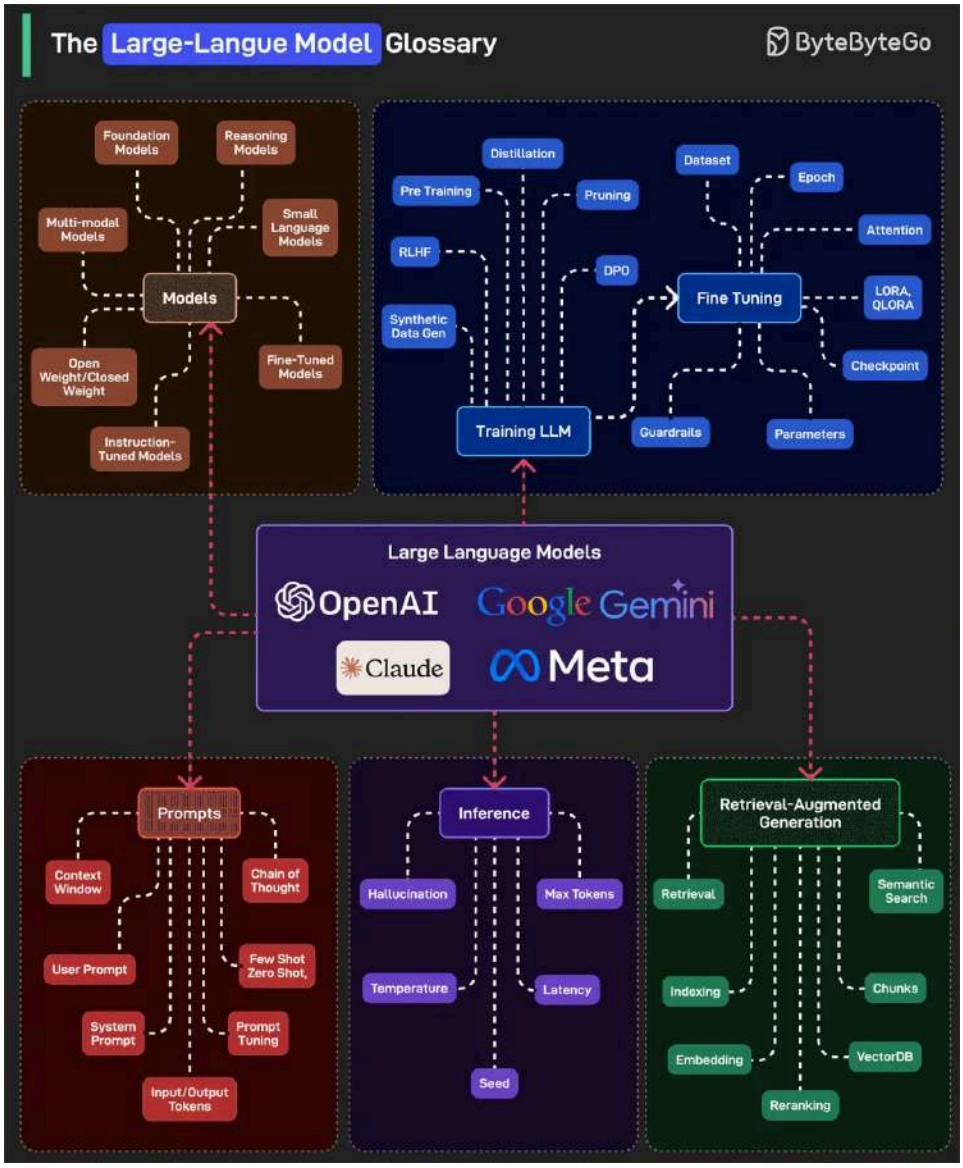
Networking & Security
18 - Amazon VPC: Secure cloud networking
19 - AWS CloudFront: Content Delivery Network
20 - AWS Route53: Scalable domain name system (DNS)
21 - AWS WAF: Protects web applications from attacks
22 - AWS Shield: DDoS protection for AWS workloads

AI & Machine Learning
23 - Amazon SageMaker: Build, train, and deploy ML models
24 - AWS Rekognition: Image and video analysis with AI
25 - AWS Textract: Extracts text from scanned documents
26 - Amazon Comprehend: AI-driven natural language processing

Monitoring & DevOps
27 - Amazon CloudWatch: AWS performance monitoring and alerts
28 - AWS X-Ray: Distributed tracing for applications
29 - AWS CodePipeline: CI/CD automation for deployments
30 - AWS CloudFormation - Infrastructure as Code (IaC)

Over to you: Which other AWS service will you add to the list?

The Large-Language Model Glossary



This glossary can be divided into high-level categories:

1. Models: Includes the types of models such as Foundation, Instruction-Tuned, Multi-modal, Reasoning, and Small Language Model.

2. Training LLM: Training begins with pretraining RLHF, DPO, and Synthetic Data. Fine-Tuning adds control with datasets, checkpoints, LoRA/QLoRA, guardrails, and parameter tunings.

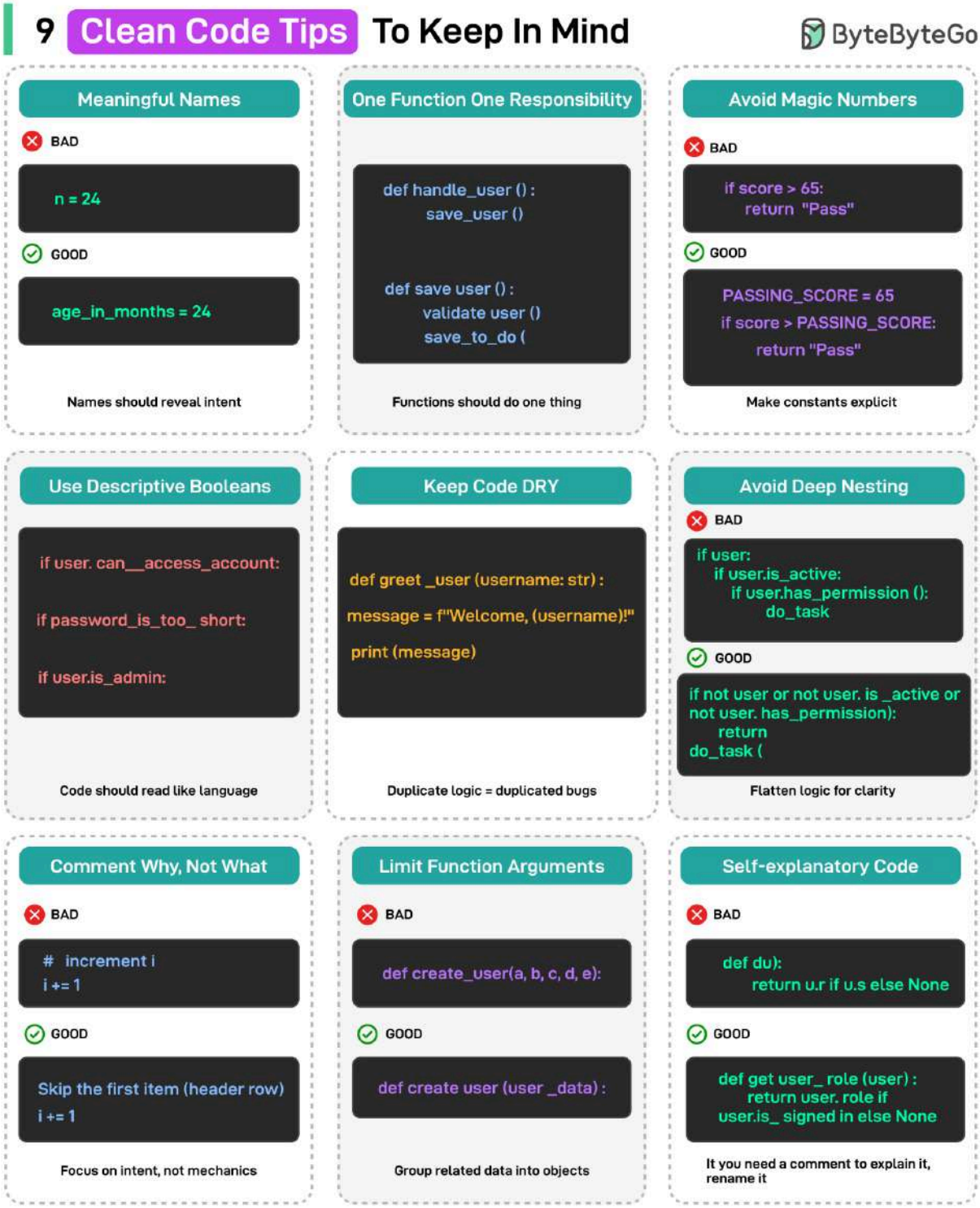
3. Prompts: Prompts drive how models respond using User/System Prompts, Chain of Thought, of Few/Zero-Shot learning. Prompt Tuning and large Context Windows help shape more precise, multi-turn conversations.

4. Inference: This is how models generate responses. Key factors include Temperature, Max Tokens, Seed, and Latency. Hallucination is a common issue here, where the model makes things up that sound real.

5. Retrieval-Augmented Generation: RAG improves accuracy by fetching real-world data. It uses Retrieval, Semantic Search, Chunks, Embeddings, and VectorDBs. Reranking and Indexing ensure the best answers are surfaced, not just the most likely ones.

Over to you: What else will you add to the LLM glossary?

9 Clean Code Principles To Keep In Mind

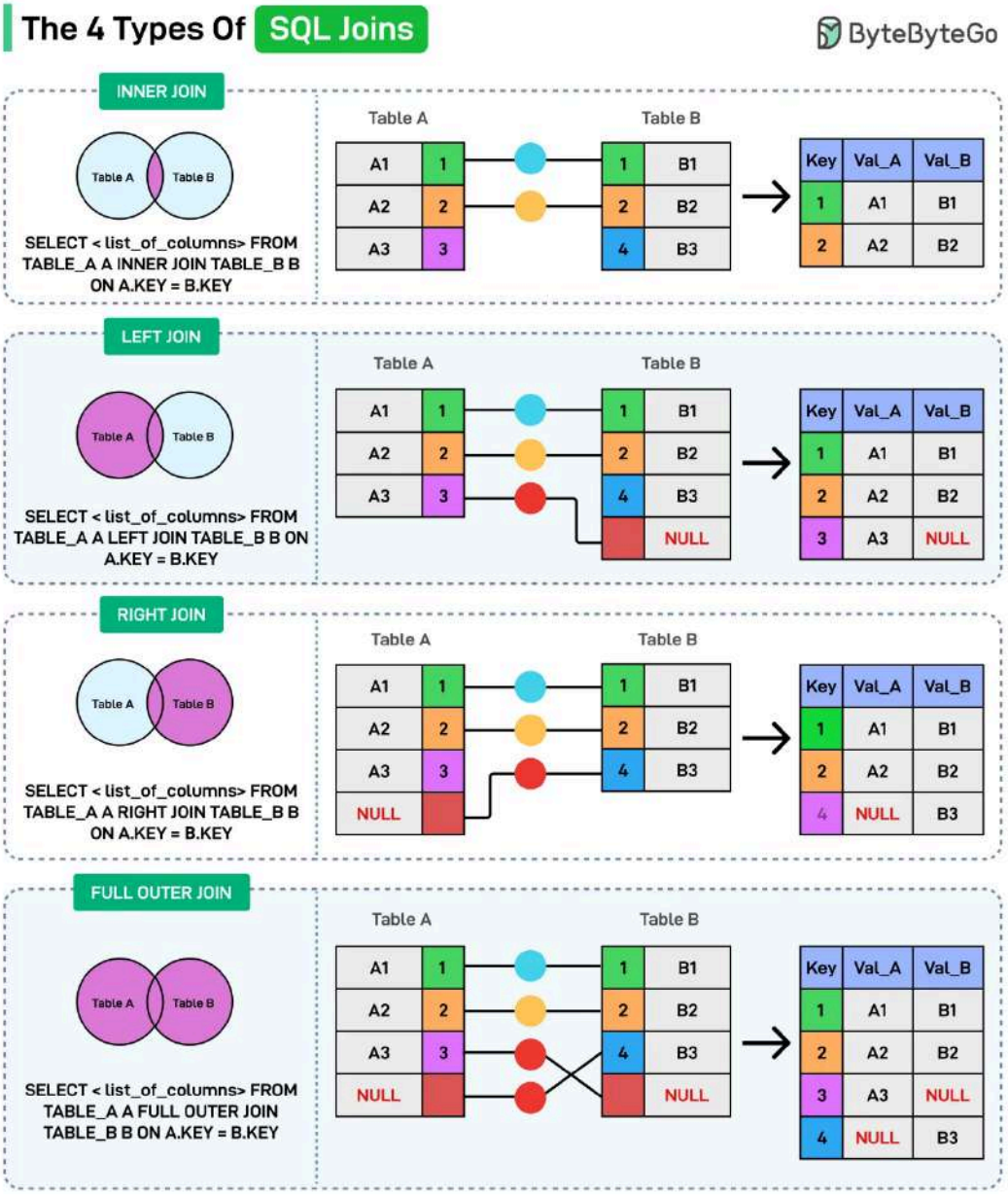


1. Meaningful Names: Name variables and functions to reveal their purpose, not just their value.

2. One Function, One Responsibility: Functions should do one thing.
3. Avoid Magic Numbers: Replace hard-code values with named constants to give them meaning.
4. Use Descriptive Booleans: Boolean names should state a condition, not just its value.
5. Keep Code DRY: Duplicate code means duplicate bugs. Try and reuse logic where it makes sense.
6. Avoid Deep Nesting: Flatten your code flow to improve clarity and reduce cognitive load.
7. Comment Why, Not What: Explain the intention behind your code, not the obvious mechanics.
8. Limit Function Arguments: Too many parameters confuse. Group related data into objects.
9. Code Should Be Self-Explanatory: Well-written code needs fewer comments because it reads like a story.

Over to you: Which other clean code principle will you add to the list?

The 4 Types of SQL Joins



SQL joins combine rows from two or more tables based on a related column. Here are the different types of joins you can use:

1. Inner Join
   Returns only the matching rows between both tables. It keeps common data only.

2. Left Join
   Returns all rows from the left table and matching rows from the right table. If a row in the left

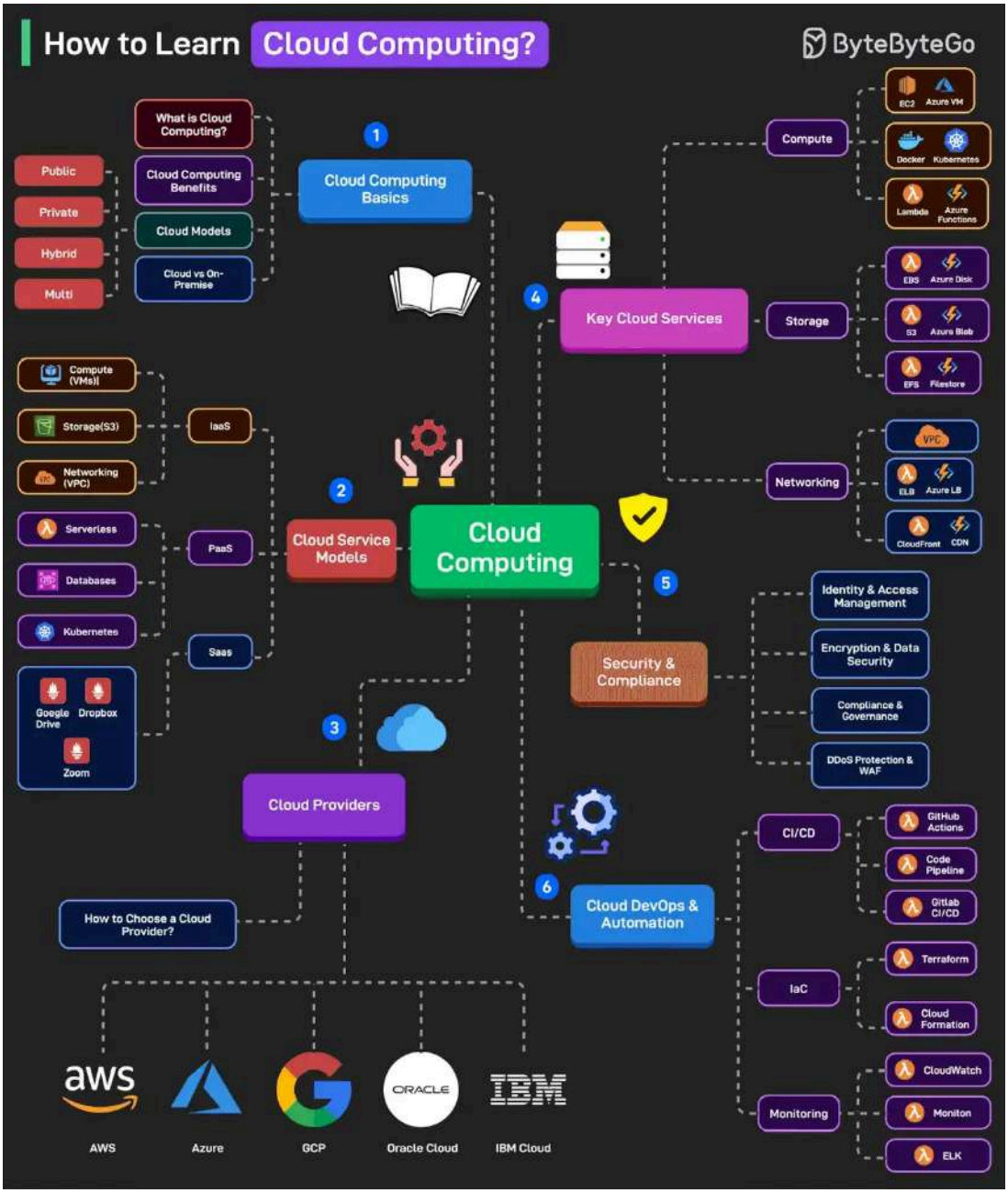
table doesn’t have a match in the right table, the right table’s columns will contain NULL values in that row.

3. Right Join
   Returns all rows from the right table and matching rows from the left table. If no matching record exists in the left table for a record in the right table, the columns from the left table in the result will contain NULL values.

4. FULL OUTER JOIN
   Returns all rows from both tables, filling in NULL for missing matches.

Over to you: Which SQL Join have you used the most?

How to Learn Cloud Computing?



Cloud computing is a vast field with an ever-growing footprint. It can often get tricky for a new developer to understand where to start. Here's a learning map:

1. Cloud Computing Basics
   This includes topics such as "what is cloud computing," its benefits, cloud models (public, private, hybrid, and multi), and a comparison of cloud vs. on-premise.

2. Cloud Service Models
   Learn about cloud service models such as IaaS, PaaS, and SaaS.

3. Cloud Providers
   Explore the various popular cloud platforms such as AWS, Azure, GCP, Oracle Cloud, IBM Cloud, etc. Also, learn how to choose a cloud provider.

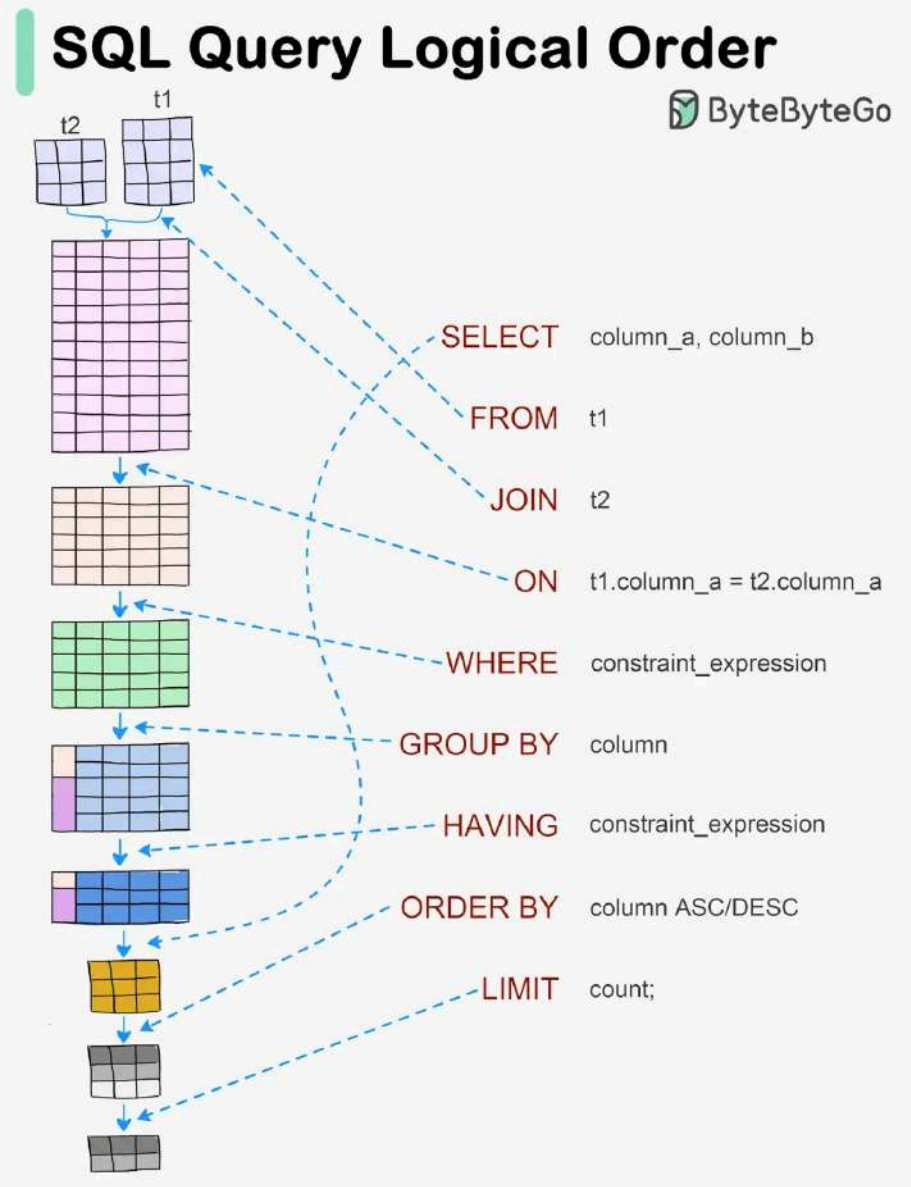
4. Key Cloud Services
   Learn the key cloud services related to Compute (EC2, Azure VM, Docker, Kubernetes, Lambda, etc), Storage (EBS, Azure Disk, S3, Azure Blob, EFS, etc.), and Networking (VPC, ELB, Azure LB, Cloudfront, and Azure CDN).

5. Security & Compliance
   Learn about the critical security and compliance points related to identity, access management, encryption, data security, DDoS protection, and WAF.

6. Cloud DevOps & Automation
   Learn Cloud DevOps and automation in specific areas such as CI/CD, IaC, and Monitoring.

Over to you: What else will you add to the list for learning cloud computing?

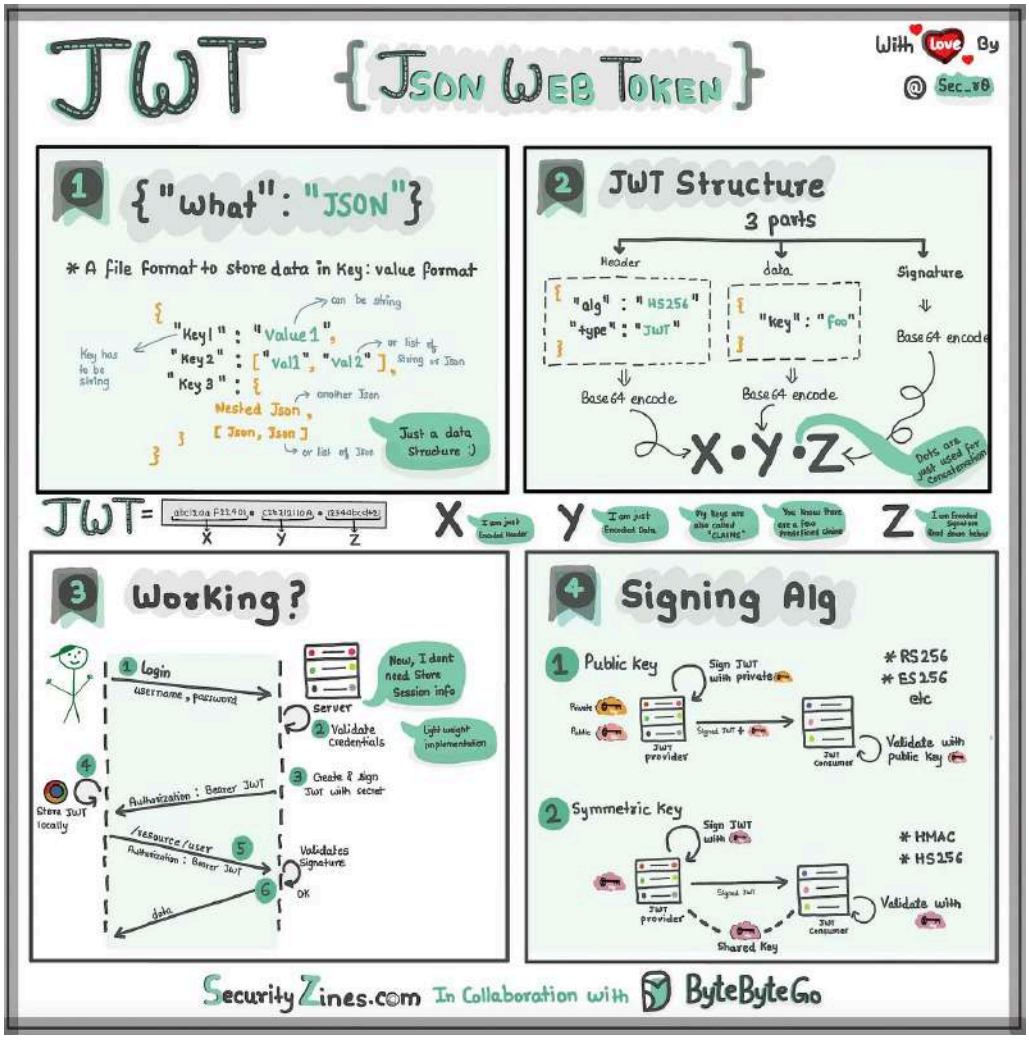
Visualizing a SQL query



SQL statements are executed by the database system in several steps, including:

- Parsing the SQL statement and checking its validity
- Transforming the SQL into an internal representation, such as relational algebra
- Optimizing the internal representation and creating an execution plan that utilizes index information
- Executing the plan and returning the results

Explaining JSON Web Token (JWT) with simple terms



Imagine you have a special box called a JWT. Inside this box, there are three parts: a header, a payload, and a signature.

The header is like the label on the outside of the box. It tells us what type of box it is and how it's secured. It's usually written in a format called JSON, which is just a way to organize information using curly braces { } and colons : .

The payload is like the actual message or information you want to send. It could be your name, age, or any other data you want to share. It's also written in JSON format, so it's easy to understand and work with.

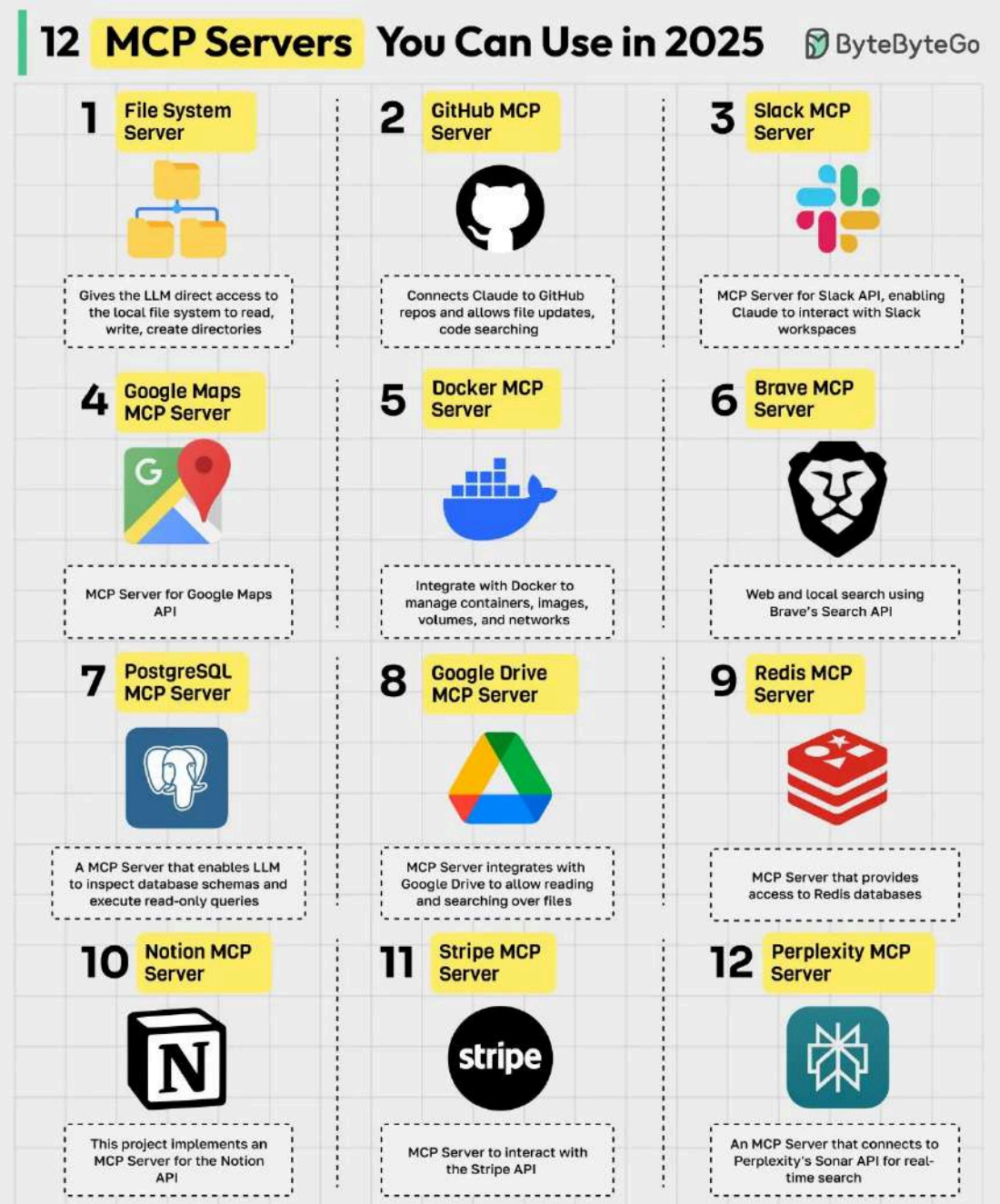
Now, the signature is what makes the JWT secure. It's like a special seal that only the sender knows how to create. The signature is created using a secret code, kind of like a password. This signature

ensures that nobody can tamper with the contents of the JWT without the sender knowing about it.

When you want to send the JWT to a server, you put the header, payload, and signature inside the box. Then you send it over to the server. The server can easily read the header and payload to understand who you are and what you want to do.

Over to you: When should we use JWT for authentication? What are some other authentication methods?

12 MCP Servers You Can Use in 2025



MCP (Model Context Protocol) is an open standard that simplifies how AI models, particularly LLMs, interact with external data sources, tools, and services. An MCP server acts as a bridge between these AI models and external tools. Here are the top MCP servers:

1. File System MCP Server
   Allows the LLM to directly access the local file system to read, write, and create directories.

2. GitHub MCP Server
   Connects Claude to GitHub repos and allows file updates, code searching.

3. Slack MCP Server
   MCP Server for Slack API, enabling Claude to interact with Slack workspaces.

4. Google Maps MCP Server
   MCP Server for Google Maps API.

5. Docker MCP Server
   Integrate with Docker to manage containers, images, volumes, and networks.

6. Brave MCP Server
   Web and local search using Brave's Search API.

7. PostgreSQL MCP Server
   An MCP server that enables LLM to inspect database schemas and execute read-only queries.

8. Google Drive MCP Server
   An MCP server that integrates with Google Drive to allow reading and searching over files.

9. Redis MCP Server
   MCP Server that provides access to Redis databases.

10. Notion MCP Server
    This project implements an MCP server for the Notion API.

11. Stripe MCP Server
    MCP Server to interact with the Stripe API.

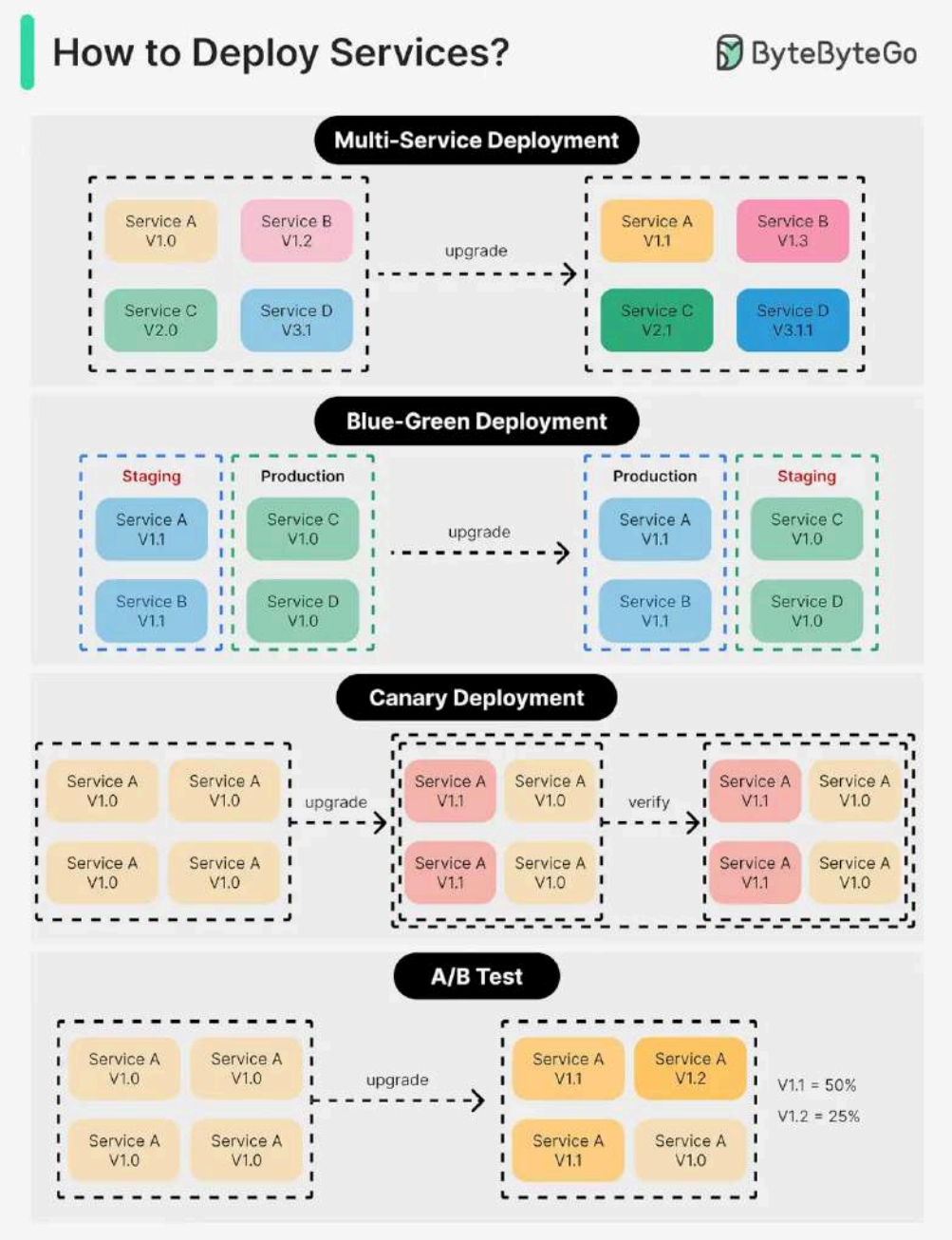
12. Perplexity MCP Server
    An MCP Server that connects to Perplexity's Sonar API for real-time search.

Over to you: Which other MCP Server will you add to the list?

How to Deploy Services

Deploying or upgrading services is risky. In this post, we explore risk mitigation strategies.

The diagram below illustrates the common ones.



Multi-Service Deployment
In this model, we deploy new changes to multiple services simultaneously. This approach is easy to implement. But since all the services are upgraded at the same time, it is hard to manage and test dependencies. It's also hard to rollback safely.

Blue-Green Deployment
With blue-green deployment, we have two identical environments: one is staging (blue) and the other is production (green). The staging environment is one version ahead of production. Once testing is done in the staging environment, user traffic is switched to the staging environment, and the staging becomes the production. This deployment strategy is simple to perform rollback, but having two identical production quality environments could be expensive.

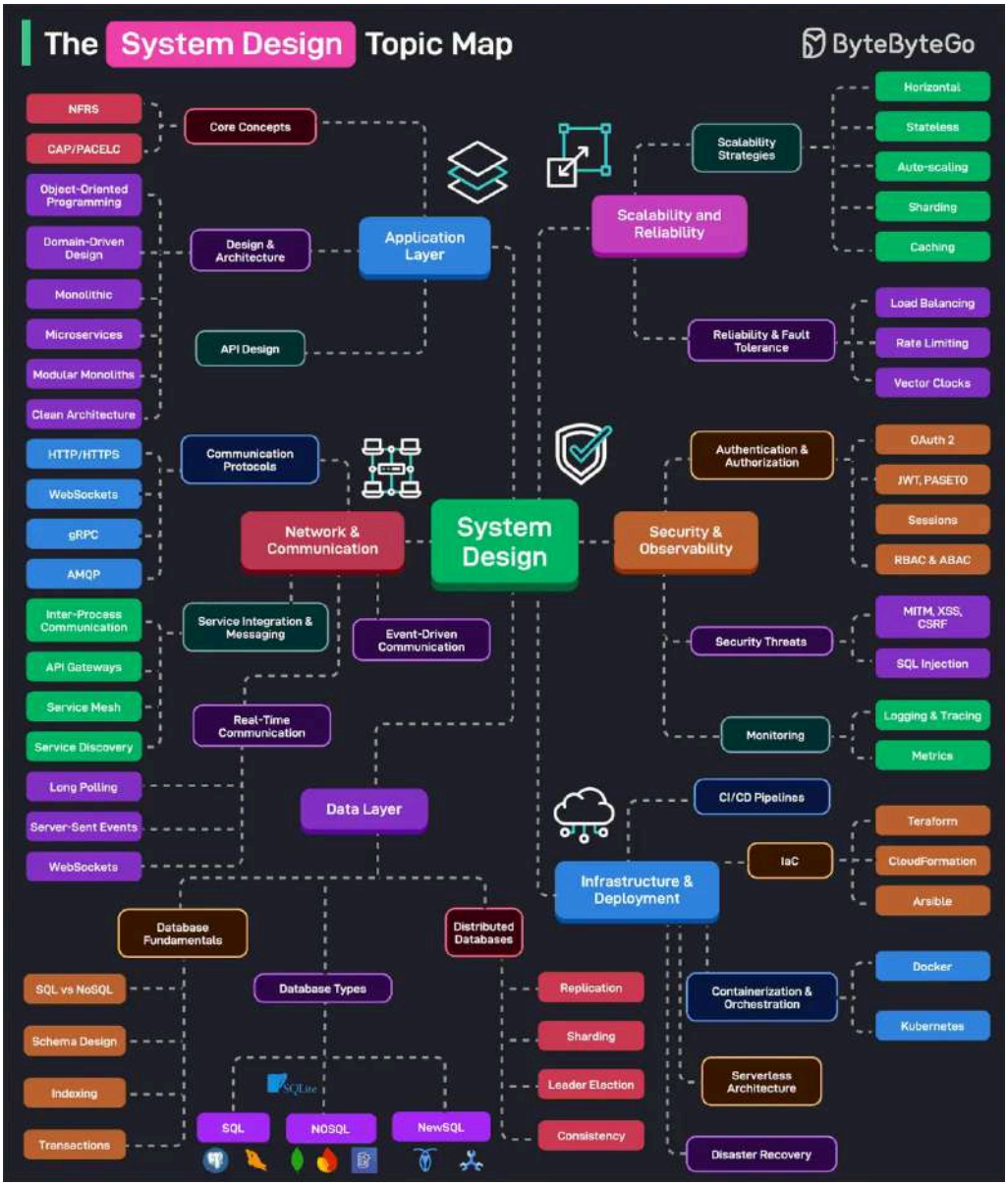
Canary Deployment
A canary deployment upgrades services gradually, each time to a subset of users. It is cheaper than blue-green deployment and easy to perform rollback. However, since there is no staging environment, we have to test on production. This process is more complicated because we need to monitor the canary while gradually migrating more and more users away from the old version.

A/B Test
In the A/B test, different versions of services run in production simultaneously. Each version runs an “experiment” for a subset of users. A/B test is a cheap method to test new features in production. We need to control the deployment process in case some features are pushed to users by accident.

Over to you - Which deployment strategy have you used? Did you witness any deployment-related outages in production and why did they happen?

The System Design Topic Map

Effective system design is a game of trade-offs and requires a broad knowledge base to make the best decisions. This topic map categorizes the essential system design topics based on categories.



1. Application Layer: It consists of the core concepts such as availability, scalability, reliability, and other NFRs. Also covers design and architectural topics such as OOP, DDD, Microservices, Clean Architecture, Modular Monoliths, and so on.

2. Network & Communication: It covers communication protocols, service integration, messaging, real-time communication, and event-driven architecture.

3. Data Layer: It covers the basics of database systems (schema design, indexing, SQL vs NoSQL, transactions, etc), the various types of databases, and the nuances of distributed databases (replication, sharding, leader election, etc.)

4. Scalability & Reliability: This covers scalability strategies (horizontal, stateless, caching, partitioning, etc) and reliability strategies like load balancing, rate limiting, and so on.

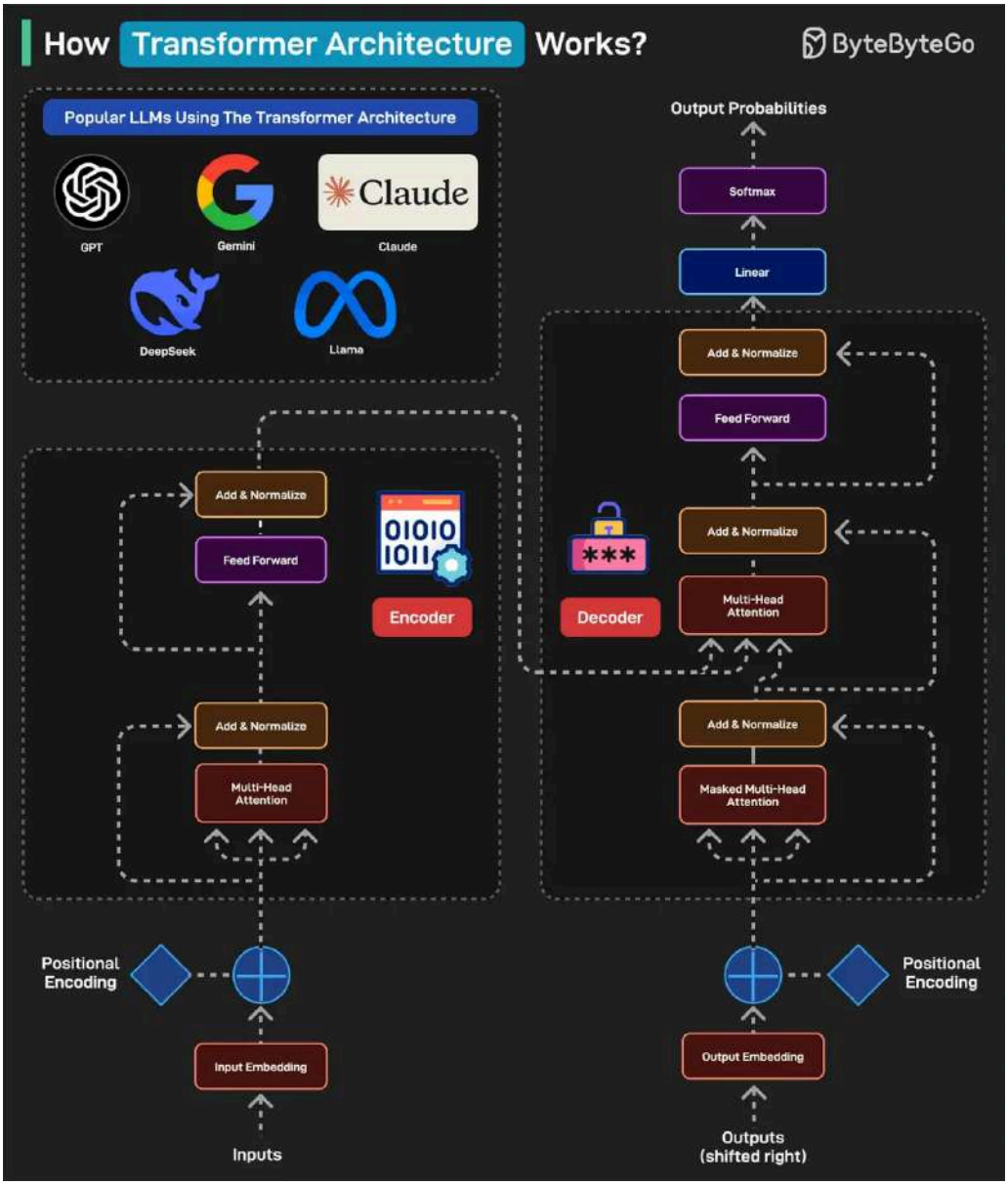
5. Security & Observability: It covers authentication and authorization techniques (OAuth 2, JWT, PASETO, Sessions, Cookies, RBAC, etc.) and security threats. The observability area deals with topics like monitoring, tracing, and logging.

6. Infrastructure & Deployments: Deals with CI/CD pipelines, containerization and orchestration, serverless architecture, IaC, and disaster recovery techniques.

Over to you: What else will you add to the list?

How Transformers Architecture Works?

Transformers Architecture has become the foundation of some of the most popular LLMs including GPT, Gemini, Claude, DeepSeek, and Llama.



Here's how it works:

1. A typical transformer-based model has two main parts: encoder and decoder. The encoder reads and understands the input. The decoder uses this understanding to generate the correct output.

2. In the first step (Input Embedding), each word is converted into a number (vector) representing its meaning.

3. Next, a pattern called Positional Encoding tells the model where each word is in the sentence. This is because the word order matters in a sentence. For example “the cat ate the fish” is different from “the fish ate the cat”.

4. Next is the Multi-Head Attention, which is the brain of the encoder. It allows the model to look at all words at once and determine which words are related. In the Add & Normalize phase, the model adds what it learned from attention back into the sentence.

5. The Feed Forward process adds extra depth to the understanding. The overall process is repeated multiple times so that the model can deeply understand the sentence.

6. After the encoder finishes, the decoder kicks into action. The output embedding converts each word in the expected output into numbers. To understand where each word should go, we add Positional Encoding.

7. The Masked Multi-Head Attention hides the future words so the model predicts only one word at a time.

8. The Multi-Head Attention phase aligns the right parts of the input with the right parts of the output. The decoder looks at both the input sentence and the words it has generated so far.

9. The Feed Forward applies more processing to make the final word choice better. The process is repeated several times to refine the results.

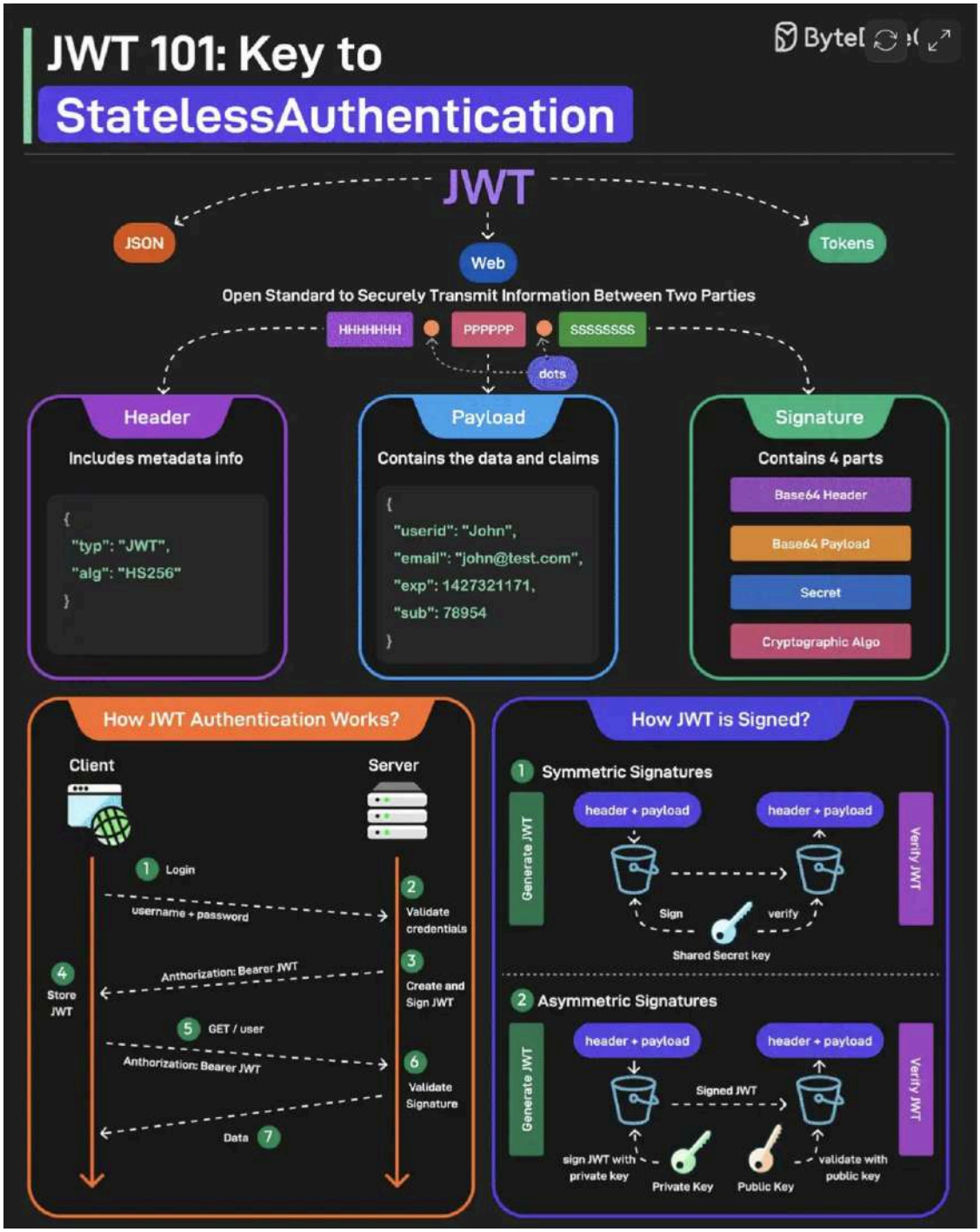
10. Once the decoder has predicted numbers for each word, it passes them through a Linear Layer to prepare for output. This layer maps the decoder’s output to a large set of possible words.

11. After the Linear Layer generates scores for each word, the Softmax layer converts those scores into probabilities. The word with the highest probability is chosen as the next word.

12. Finally, a human-readable sentence is generated.

JWT Simply Explained

JWT or JSON Web Tokens is an open standard for securely transmitting information between two parties. They are widely used for authentication and authorization.



A JWT consists of three main components:

1. Header
   Every JWT carries a header specifying the algorithms for signing the JWT. It's written in JSON format.

2. Payload
   The payload consists of the claims and the user data. There are different types of claims such as registered, public, and private claims.

3. Signature
   The signature is what makes the JWT secure. It is created by taking the encoded header, encoded payload, secret key, and the algorithm and signing it.

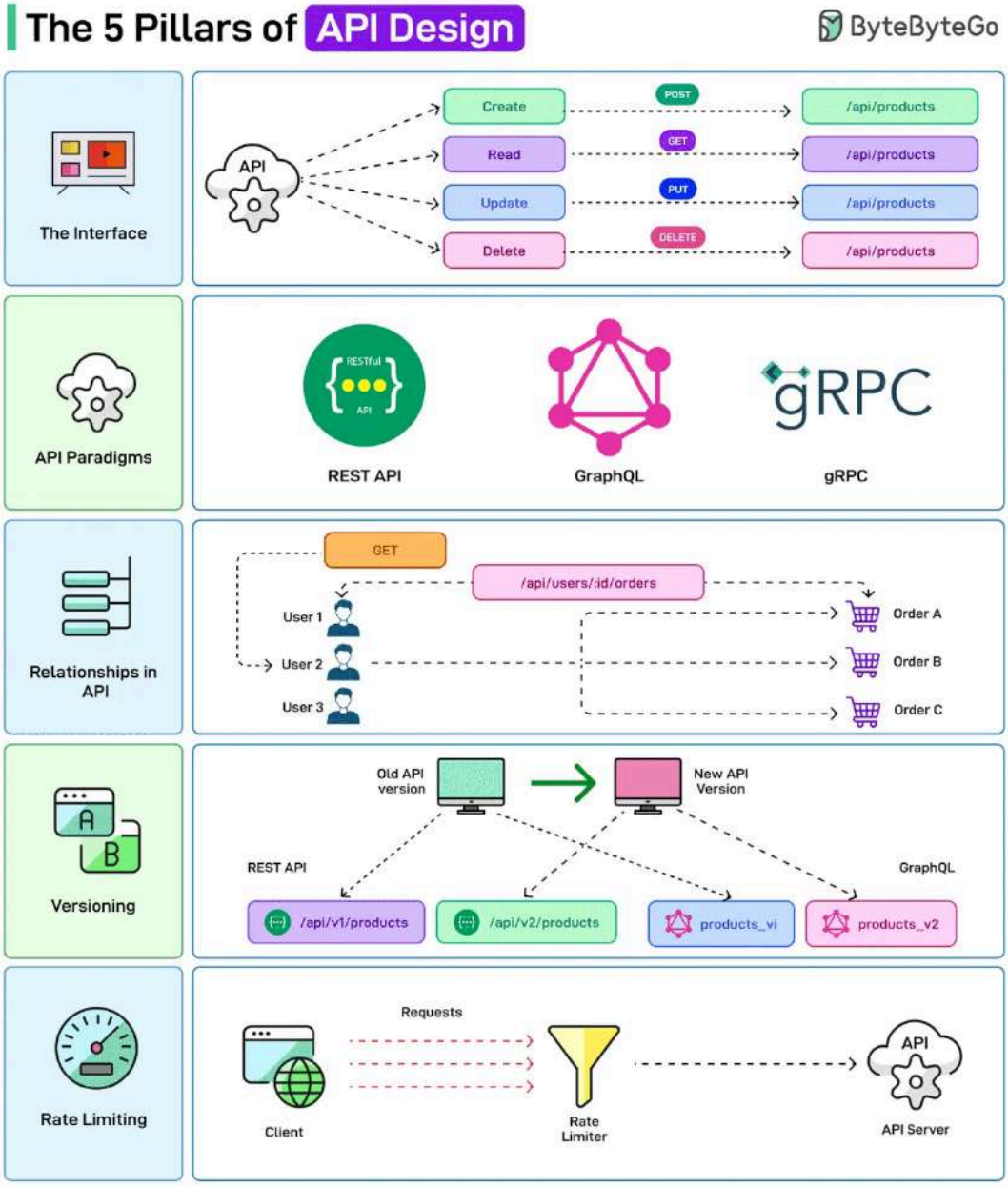
JWTs can be signed in two different ways:

1. Symmetric Signatures
   It uses a single secret key for both signing the token and verifying it. The same key must be shared between the server that signs the JWT and the system that verifies it.

2. Asymmetric Signatures
   In this case, a private key is used to sign the token, and a public key to verify it. The private key is kept secure on the server, while the public key can be distributed to anyone who needs to verify the token.

Over to you: Do you use JWTs for authentication?

The 5 Pillars of API Design



APIs are the backbone of modern systems. But it is also important to design them in the right way.

Here are a few things that a developer should consider while designing APIs

1. The Interface
   API Design is concerned with defining the inputs and outputs of an API. For example, defining how the CRUD operations may be exposed to the user or the client.

2. API Paradigms
   APIs can be built following different paradigms, each with its own set of protocols and standards. Some options are REST, GraphQL, and gRPC.

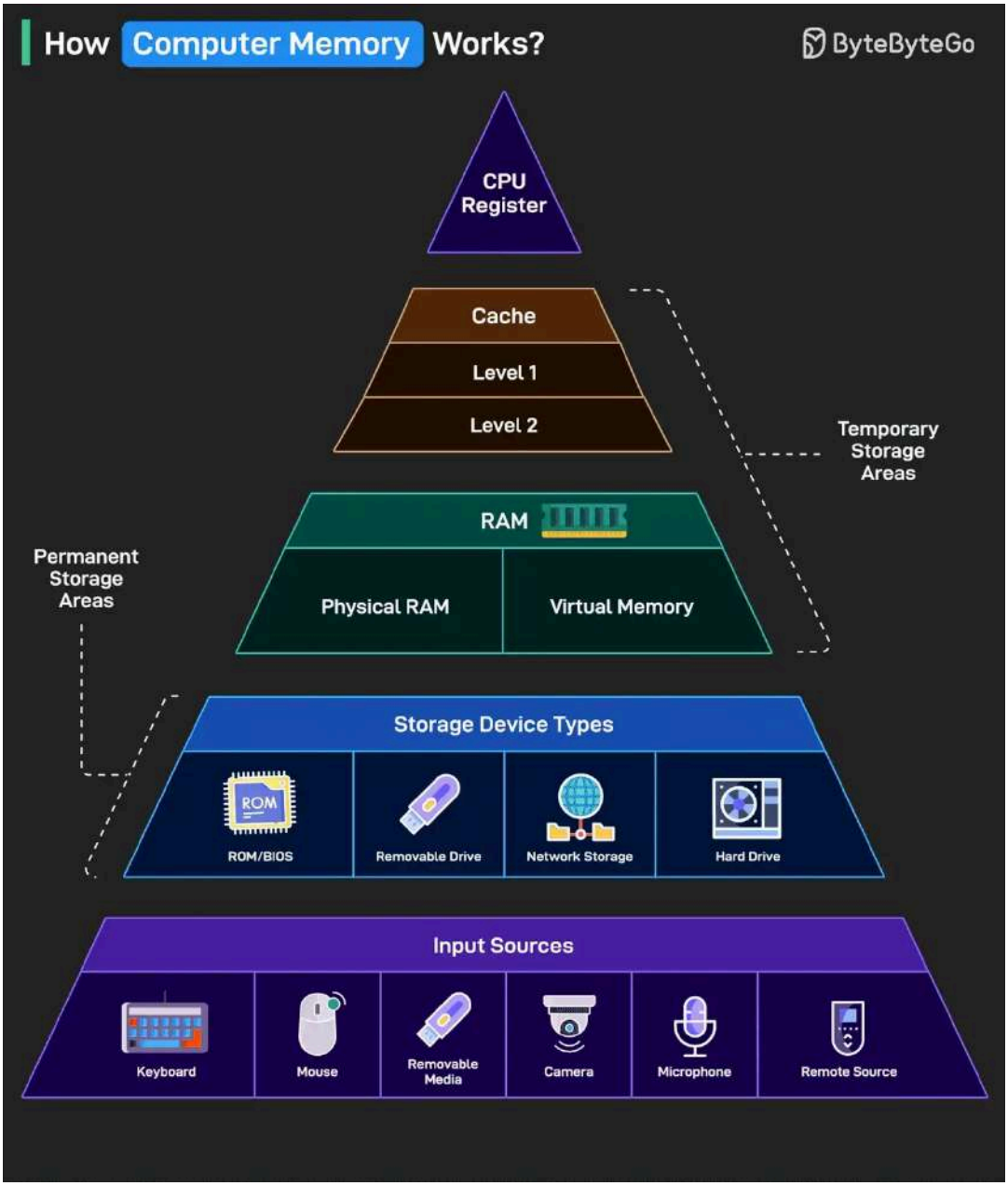
3. Relationships in API
   APIs often need to establish relationships between the various entities. For example, a user might have multiple orders related to their account. The API endpoints should reflect these relationships for a better client experience.

4. Versioning
   When modifying API endpoints, proper versioning and supporting backward compatibility are important.

5. Rate Limiting
   Rate limiting is used to control the number of requests a user can make to an API within a certain timeframe. This is crucial for maintaining the reliability and availability of the API.

Over to you: Which other API Design principle will you add to the list?

How Computer Memory Works?



Here's a simple breakdown that shows how data moves through a system from input to processing to storage.

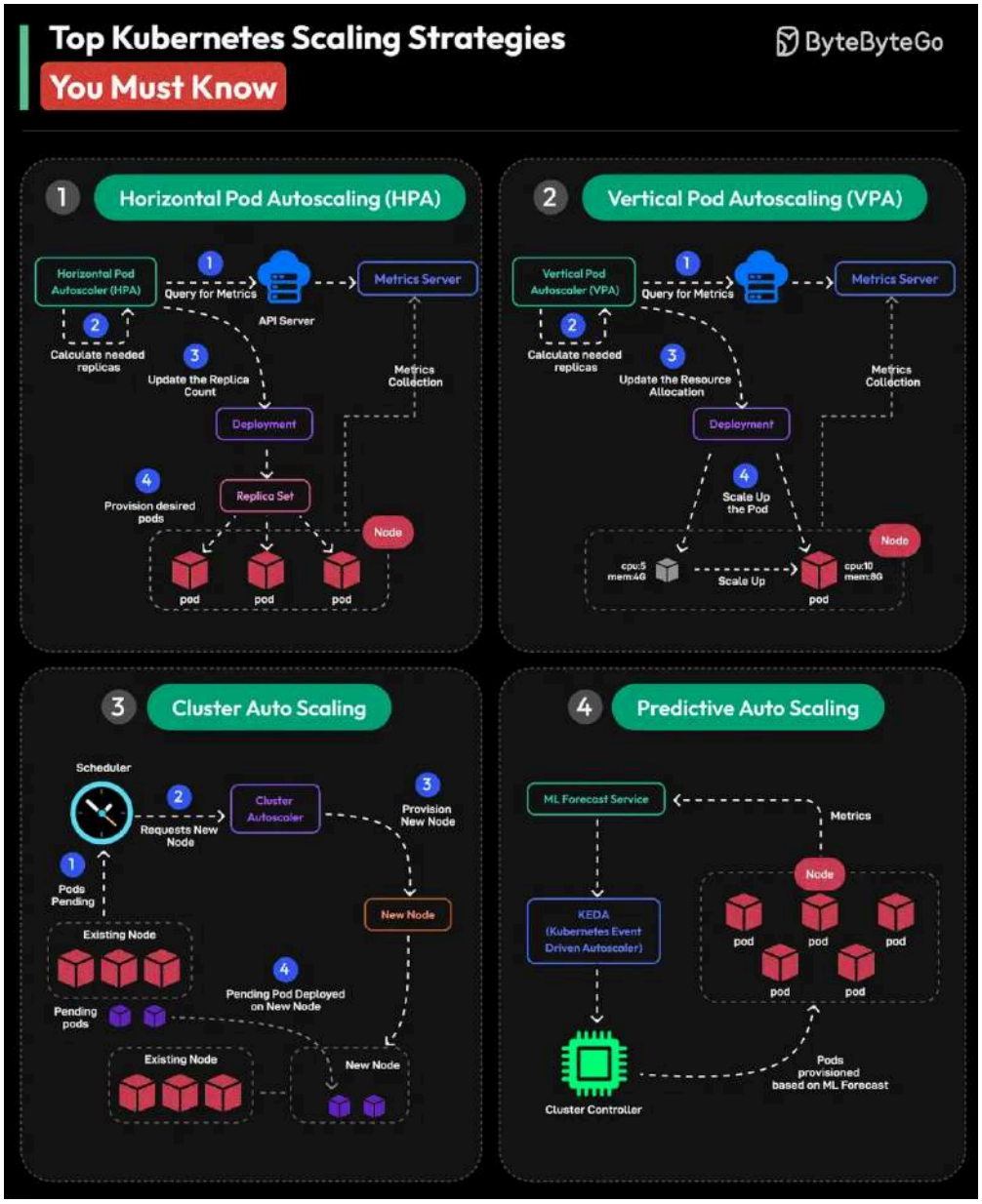
1. Data enters through input sources like keyboard, mouse, camera, or remote systems.
2. Permanent storage holds your system files, apps, and media. This includes hard drives, USB drives, ROM/BIOS, and network-based storage.

3. RAM is the workspace of your computer. It includes physical memory and virtual memory, which temporarily store data and programs while you're using them.
4. Cache memory sits closer to the CPU and is split into Level 1 and Level 2. It helps speed up access to frequently used data.
5. CPU registers are the fastest and smallest memory units. They're used directly by the processor to execute instructions almost instantly.

The higher you go in the memory pyramid, the faster and smaller the storage.

Over to you: What else will you add to improve the understanding of a computer memory's working?

Top Kubernetes Scaling Strategies You Must Know



1. Horizontal Pod Autoscaling or HPA
   Horizontal Pod Autoscaler automatically scales the number of Pods in a Deployment, ReplicaSet, or StatefulSet based on observed CPU utilization, memory usage, or custom metrics.

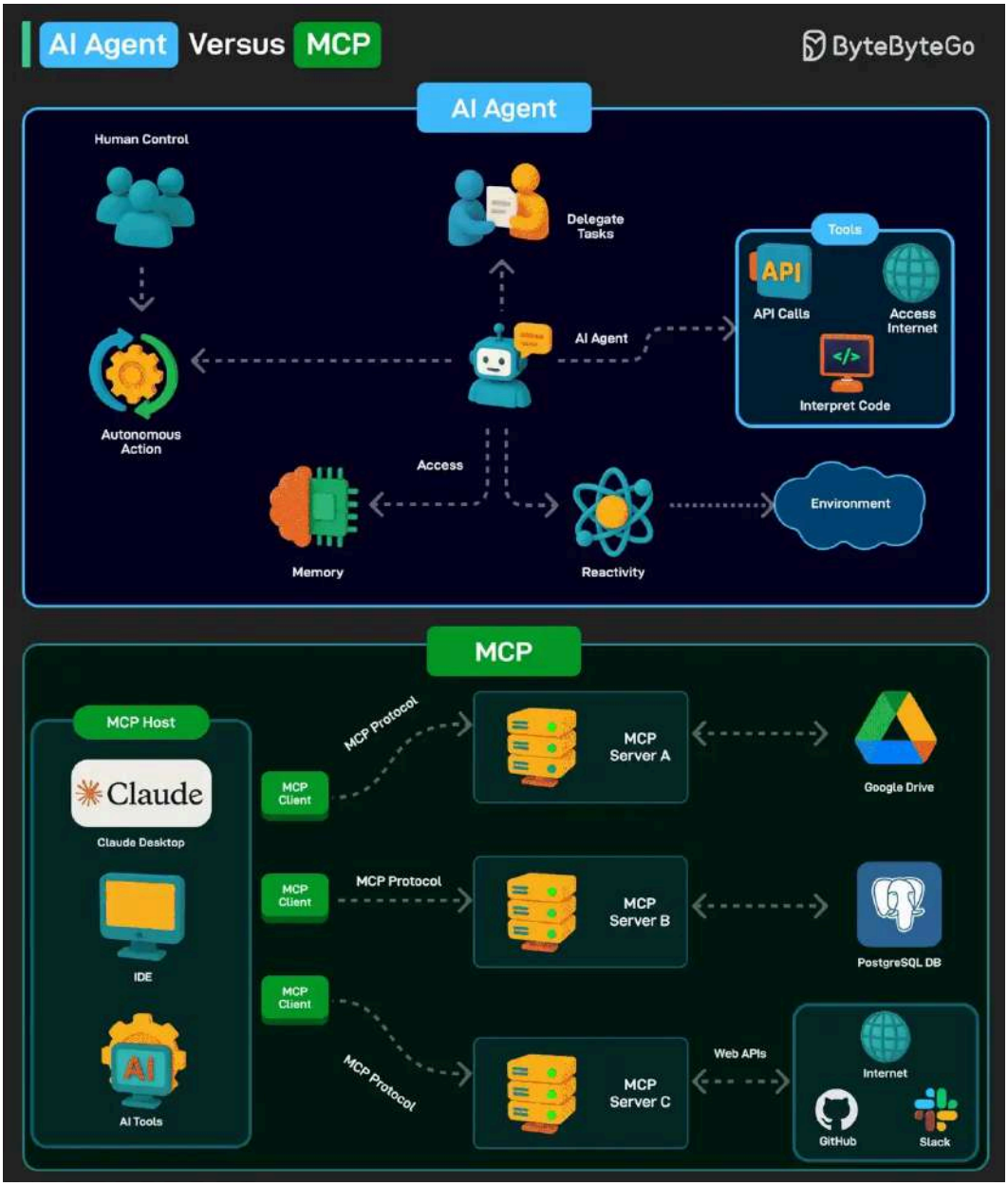
2. Vertical Pod Autoscaling or VPA
   Based on application requirements, VPA adjusts the resources allocated to individual pods, such as CPU and memory. This approach dynamically changes pod resource settings based on workload metrics.

3. Cluster Auto Scaling
   The Cluster Autoscaler automatically adjusts the number of nodes in a Kubernetes cluster. It interacts with the cloud provider to add or remove nodes based on requirements. This is important to maintain a balanced cluster.

4. Predictive Auto Scaling
   Predictive Autoscaling uses machine learning to forecast future resource requirements. It helps Kubernetes adjust resources by anticipating workload demands.

Over to you: Which other Kubernetes Scaling Strategy will you add to the list?

AI Agent versus MCP



An AI agent is a software program that can interact with its environment, gather data, and use that data to achieve predetermined goals. AI agents can choose the best actions to perform to meet those goals.

Key characteristics of AI agents are as follows:

1. An agent can perform autonomous actions without constant human intervention. Also, they can have a human in the loop to maintain control.

2. Agents have a memory to store individual preferences and allow for personalization. It can also store knowledge. An LLM can undertake information processing and decision-making functions.

3. Agents must be able to perceive and process the information available from their environment.

Model Context Protocol (MCP) is a new system introduced by Anthropic to make AI models more powerful.

It is an open standard that allows AI models (like Claude) to connect to databases, APIs, file systems, and other tools without needing custom code for each new integration.

MCP follows a client-server model with 3 key components:

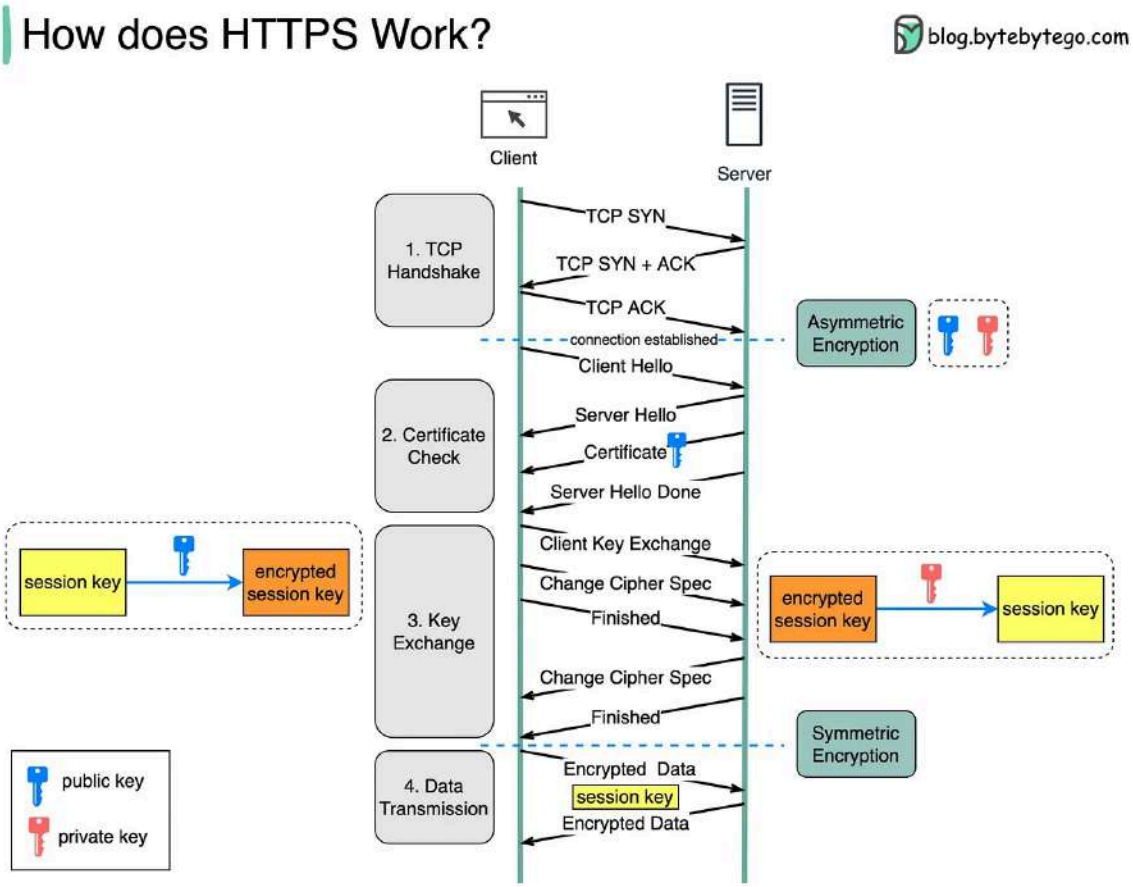
1. Host: AI applications like Claude

2. MCP Client: Component inside an AI model (like Claude) that allows it to communicate with MCP servers

3. MCP Server: Middleman that connects an AI model to an external system

Over to you: Have you used AI Agents or MCP?

How does HTTPS work?



Hypertext Transfer Protocol Secure (HTTPS) is an extension of the Hypertext Transfer Protocol (HTTP.) HTTPS transmits encrypted data using Transport Layer Security (TLS.) If the data is hijacked online, all the hijacker gets is binary code.

How is the data encrypted and decrypted?

Step 1 - The client (browser) and the server establish a TCP connection.

Step 2 - The client sends a “client hello” to the server. The message contains a set of necessary encryption algorithms (cipher suites) and the latest TLS version it can support. The server responds with a “server hello” so the browser knows whether it can support the algorithms and TLS version.

The server then sends the SSL certificate to the client. The certificate contains the public key, host name, expiry dates, etc. The client validates the certificate.

Step 3 - After validating the SSL certificate, the client generates a session key and encrypts it using the public key. The server receives the encrypted session key and decrypts it with the private key.

Step 4 - Now that both the client and the server hold the same session key (symmetric encryption), the encrypted data is transmitted in a secure bi-directional channel.

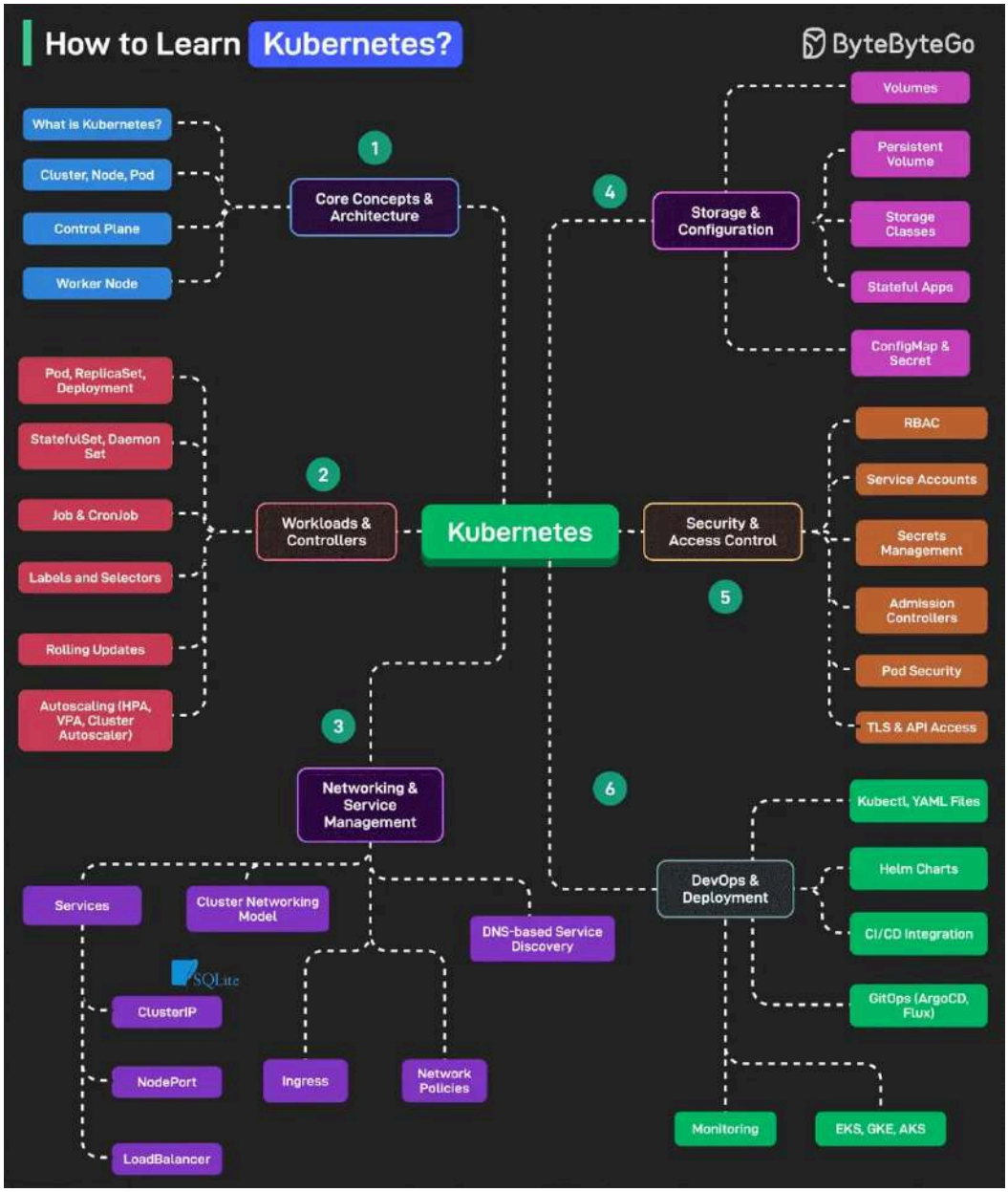
Why does HTTPS switch to symmetric encryption during data transmission? There are two main reasons:

1. Security: The asymmetric encryption goes only one way. This means that if the server tries to send the encrypted data back to the client, anyone can decrypt the data using the public key.
2. Server resources: The asymmetric encryption adds quite a lot of mathematical overhead. It is not suitable for data transmissions in long sessions.

Over to you: how much performance overhead does HTTPS add, compared to HTTP?

How to Learn Kubernetes?

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications.



1. Core Concepts and Architecture
   This includes topics like What is Kubernetes, Cluster, Node, Pod, Control Plane, and Worker Node.

2. Workloads and Controllers
   This includes topics like Pod, ReplicaSet, Deployment, StatefulSet, Job, CronJob, Labels, Selectors, and Autoscalers (HPA, VPA, and Cluster Autoscaler).

3. Networking and Service Management
   This involves topics like Cluster Networking Model, Services (ClusterIP, NodePort, LoadBalancer), Ingress, and Network Policies.

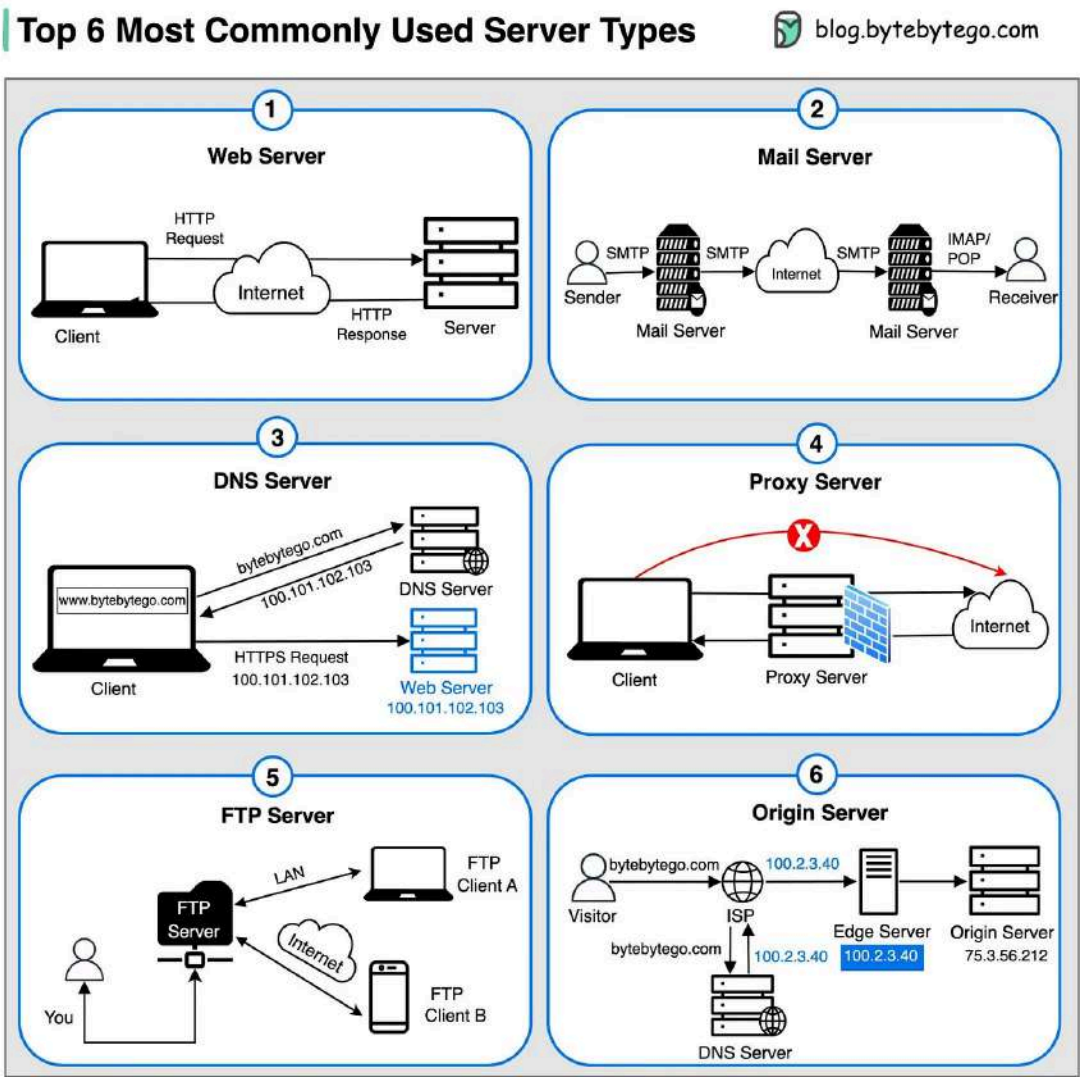
4. Storage and Configuration
   This includes topics like Volumes, PersistentVolume, PersistentVolumeClaim, Storage Classes, ConfigMap, Secret, and Stateful Applications.

5. Security & Access Control
   This involves topics like RBAC, Service Accounts, Secrets Management, Admission Controllers, Pod Security, TLS, and API Access.

6. Tools, Observability & Ecosystem
   This includes topics like kubectl, YAML files, Helm Charts, CI/CD Integration, GitOps, Logging, Monitoring, and EKS.

Over to you: What else will you add to the list for learning Kubernetes?

Top 6 most commonly used Server Types



1. Web Server:
   Hosts websites and delivers web content to clients over the internet

2. Mail Server:
   Handles the sending, receiving, and routing of emails across networks

3. DNS Server:
   Translates domain names (like bytebytego. com) into IP addresses, enabling users to access websites by their human-readable names.

4. Proxy Server:
   An intermediary server that acts as a gateway between clients and other servers, providing additional security, performance optimization, and anonymity.

5. FTP Server:
   Facilitates the transfer of files between clients and servers over a network

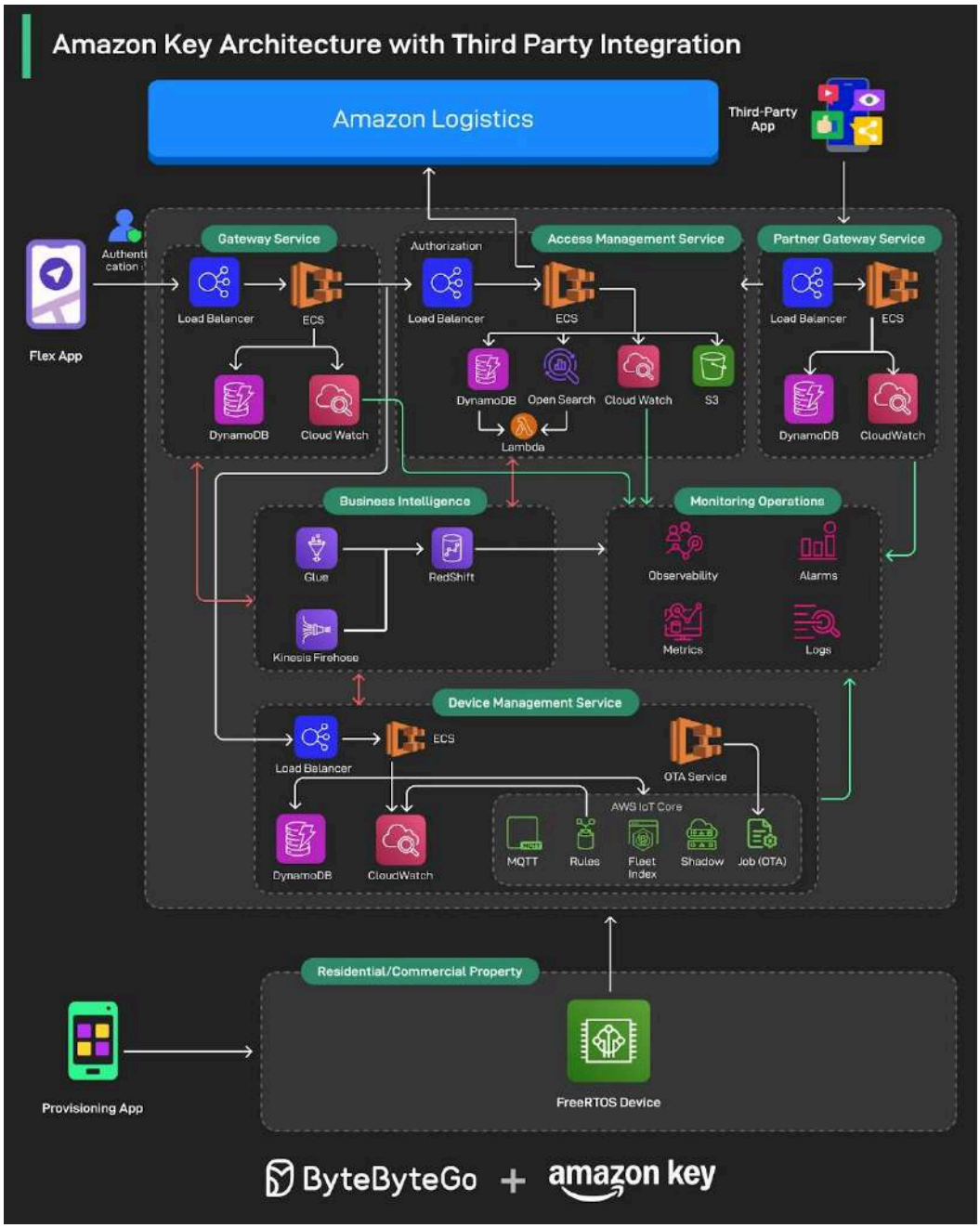
6. Origin Server:
   Hosts central source of content that is cached and distributed to edge servers for faster delivery to end users.

Over to you: Which type of server do you find most crucial in your online experience?

Amazon Key Architecture with Third Party Integration

Amazon Key is a REALLY convenient way to have your packages securely delivered right inside your garage or multifamily property.

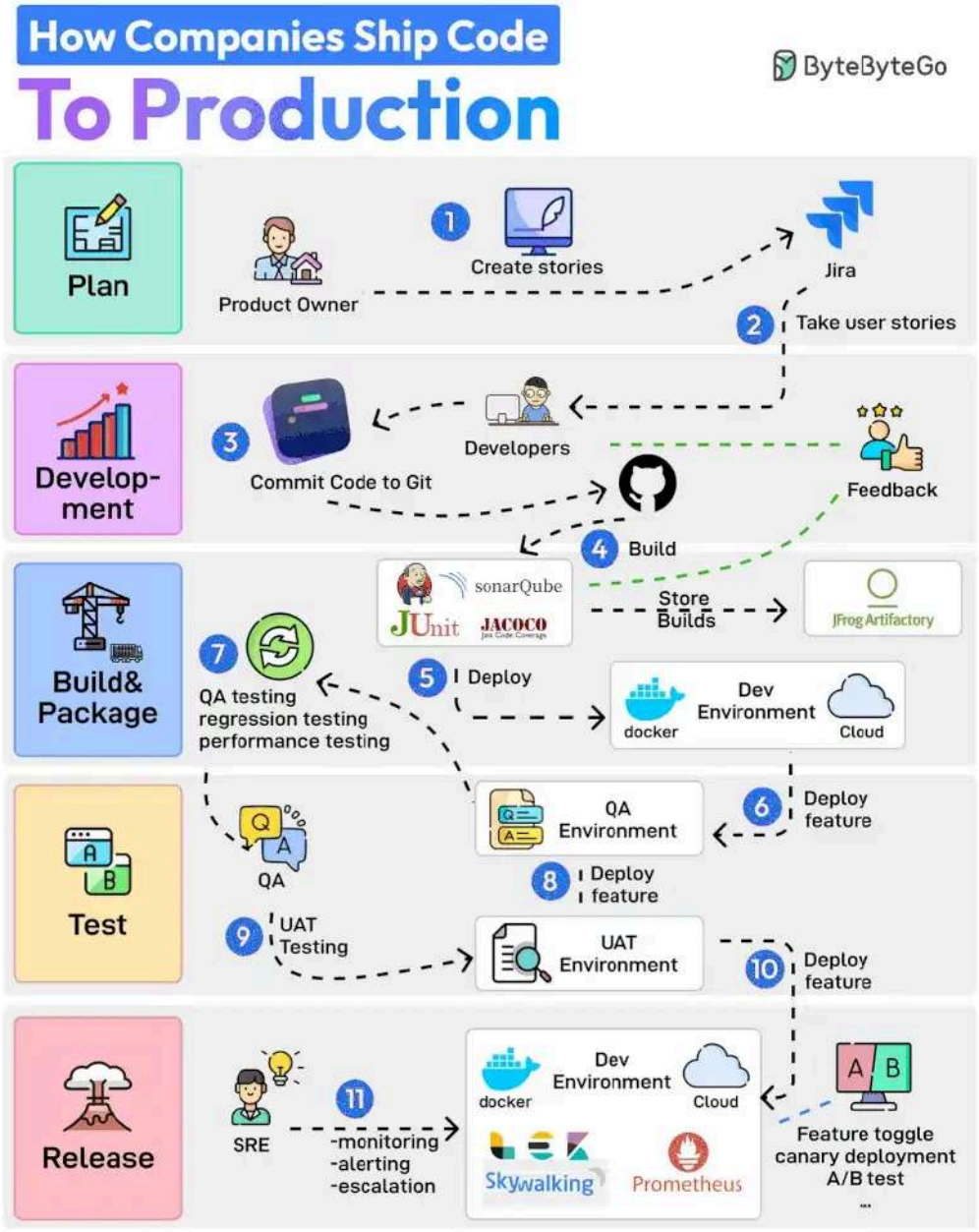
We sat down with Kaushik Mani and Vijayakrishnan Nagarajan to learn how a small side project evolved into a global platform: now powering over 100 million secure door unlocks every year.



This story is packed with valuable lessons on:

- Scaling IoT infrastructure across thousands of locations
- Building resilient, field-ready hardware for real-world environments
- Overcoming cold starts and connectivity challenges
- Designing a microservices architecture for global expansion
- Creating a secure, partner-ready platform to support third-party integrations
- And many more

How Do Companies Ship Code to Production?



Here are 11 steps from planning to production:

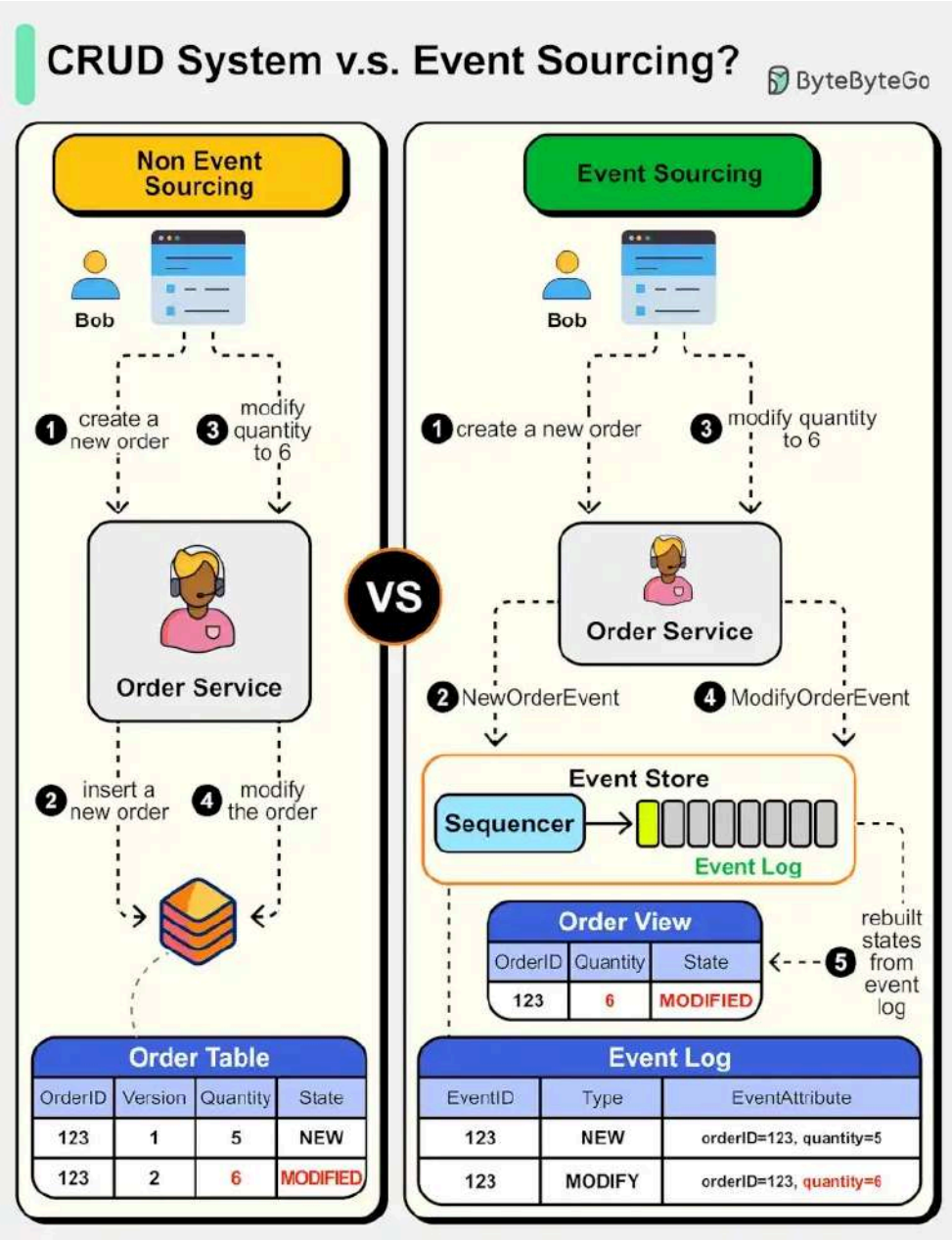
1. The Product Owner starts the entire process by creating user stories in a tool like Jira.
2. The Developer Team performs Sprint Planning activity and adds the user stories to the sprint.
3. Developers work on the assigned stories. Once a story is finished, they commit the code to Git and push it to GitHub.

4. Jenkins builds and runs the code through testing and quality check tools such as JUnit, Jacoco, and SonarQube.
5. If the build is successful, it is stored in the artifactory such as JFrog. Jenkins also deploys the build to the Dev Environment via Docker.
6. Next up, the feature gets deployed to the QA environment. Since multiple teams may be working on the same code base, multiple QA environments will be created
7. The QA team uses a particular QA environment and runs multiple test types such as QA, regression, and performance.
8. Once the QA verification is complete, features are deployed to the UAT (User Acceptance Testing) environment.
9. UAT testing verifies whether the feature satisfies the user's requirements.
10. Once the UAT testing is successful, the builds become release candidates. They are deployed to the production environment based on a specific schedule.
11. The SRE team uses tools like ELK and Prometheus to monitor the production environment and handle alerts in case of issues.

Over to you: Will you add any other steps to the process?

What is Event Sourcing? How is it different from normal CRUD design?

The diagram below shows a comparison of normal CRUD system design and event sourcing system design. We use an order service as an example.



The event sourcing paradigm is used to design a system with determinism. This changes the philosophy of normal system designs.

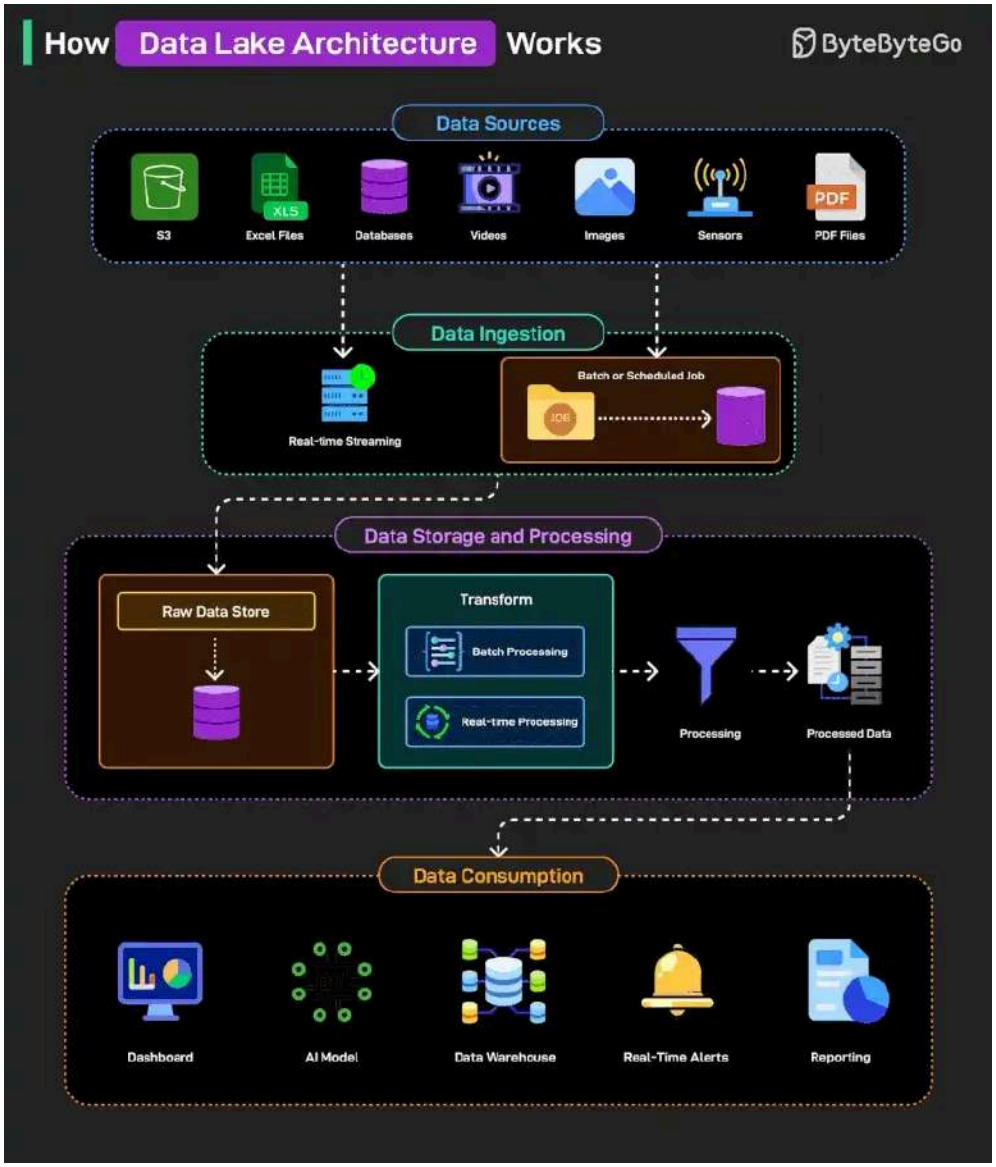
How does this work? Instead of recording the order states in the database, the event sourcing design persists the events that lead to the state changes in the event store. The event store is an append-only log. The events must be sequenced with incremental numbers to guarantee their ordering. The order states can be rebuilt from the events and maintained in OrderView. If the OrderView is down, we can always rely on the event store which is the source of truth to recover the order states.

Let's look at the detailed steps.

- Non-Event Sourcing
    - Steps 1 and 2: Bob wants to buy a product. The order is created and inserted into the database.
    - Steps 3 and 4: Bob wants to change the quantity from 5 to 6. The order is modified with a new state.
- Event Sourcing
    - Steps 1 and 2: Bob wants to buy a product. A NewOrderEvent is created, sequenced, and stored in the event store with eventID=321.
    - Steps 3 and 4: Bob wants to change the quantity from 5 to 6. A ModifyOrderEvent is created, sequenced, and persisted in the event store with eventID=322.
    - Step 5: The order view is rebuilt from the order events, showing the latest state of an order.

Over to you: Which type of system is suitable for event sourcing design? Have you used this paradigm in your work?

How Data Lake Architecture Works?



A data lake is a centralized storage system that holds large volumes of structured, semi-structured, and unstructured data in its raw form. It allows organizations to store data at scale and process it later for different requirements.

Here's how it works on a high level:

1. Data from diverse sources like S3, Excel files, databases, videos, images, sensors, and PDFs is collected for ingestion.

2. Ingestion happens via real-time streaming or batch/scheduled jobs, depending on the data type and source.

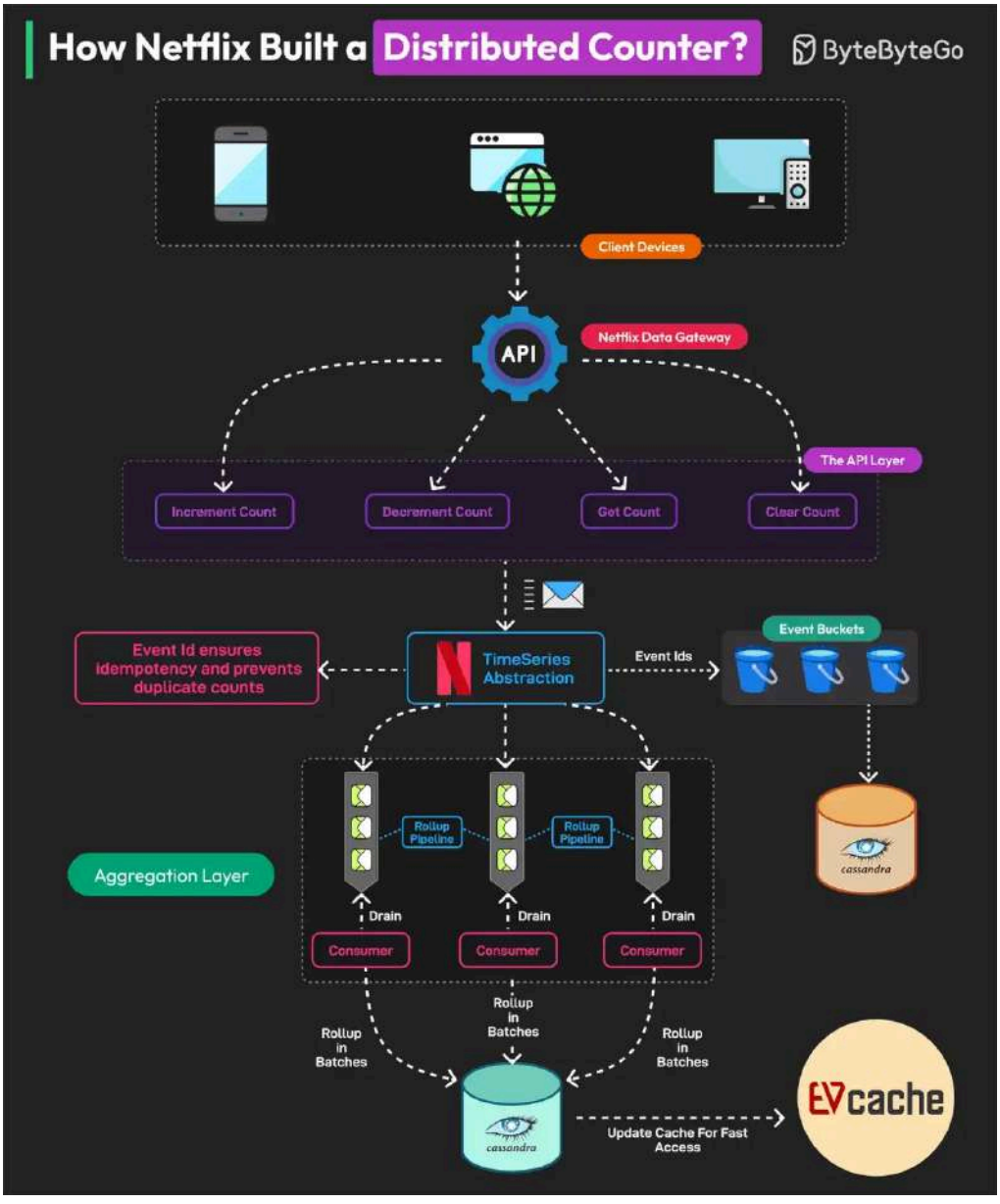
3. Ingested data is first stored in a raw data store before any transformation or processing occurs.
4. Data is then processed through batch or real-time processing systems to clean, transform, and prepare it for use.
5. The processed data is stored and made ready for downstream consumption by various tools and platforms.
6. Final data consumption happens through dashboard, AI models, data warehouse, real-time alerts, and reporting systems.

Over to you: What else will you add to better understand the Data Lake Architecture?

How Netflix Built a Distributed Counter

The Netflix distributed counter is a masterclass for learning system design.

A Distributed Counter is a system where the responsibility of counting events is spread across multiple servers or nodes in a network. Netflix needs to track and measure multiple user interactions to make real-time decisions and optimize its infrastructure.



For this reason, they built a Distributed Counter Abstraction.

Netflix's Distributed Counter Abstraction operates in four main layers, ensuring high performance, scalability, and eventual consistency.

1. Client API Layer
   Users interact with the system by sending AddCount, GetCount, or ClearCount requests. The Netflix Data Gateway efficiently processes and routes these requests.

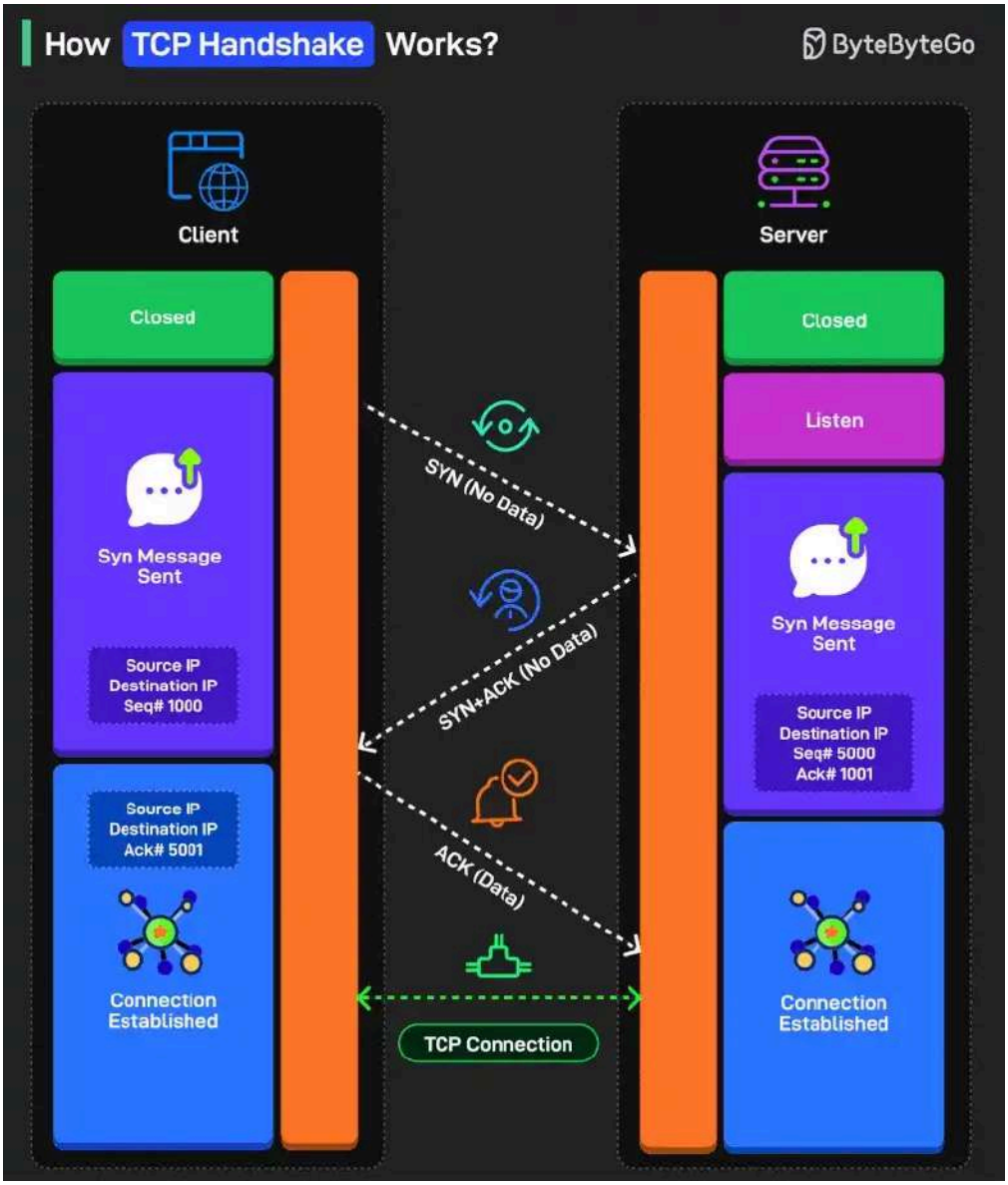
2. Event Logging and TimeSeries Storage
   Events are stored in Netflix TimeSeries Abstraction for scalability. Each event is tagged with an Event ID to ensure idempotency. To avoid database contention, events are grouped into time partitions known as buckets. Data is stored in Cassandra.

3. Rollup Pipeline or Aggregation
   Rollup Queues collect event changes and process them in batches. Aggregation occurs in immutable time windows, ensuring accurate rollup calculations. Data is stored in the Cassandra Rollup Store for eventual consistency.

4. Read Optimization (Cache & Query Handling)
   Aggregated counter values are cached in EVCache for ultra-fast reads. If a cache value is stale, a background rollup refresh updates it. This model allows Netflix to process 75K requests per second with single-digit millisecond latency.

Reference: Netflix's Distributed Counter Abstraction

How TCP Handshake Works?



There are three steps in the handshake process:

1. SYN (Synchronize)
   Think of it as one party extending its hand for a handshake. The process starts with the client sending a TCP segment to the server with the SYN flag. Also, an initial sequence number (ISN) is sent along with the message.

2. SYN-ACK
   This is like the second party agreeing to the handshake. The server receives the SYN segment and responds by sending a TCP segment to the client. It sets the SYN flag and the

ACK flag in the segment. The ACK flag is meant to acknowledge the receipt of the client's SYN segment by incrementing the ISN.

3. ACK
   This is where the two parties shake hands. The client receives the SYN-ACK segment and acknowledges the server's response by sending another TCP segment with the ACK flag set. The sequence number is set to the server's ISN after incrementing.

Some more points worth considering:

1. SYN and ACK are control flags used to manage different aspects of the communication process.
2. Sequence numbers (ISN) help the client and server keep track of the message order so that nothing gets missed.

Over to you: What else will you add to understand the TCP Handshake process in a better way?

Top 20 AI Concepts You Should Know

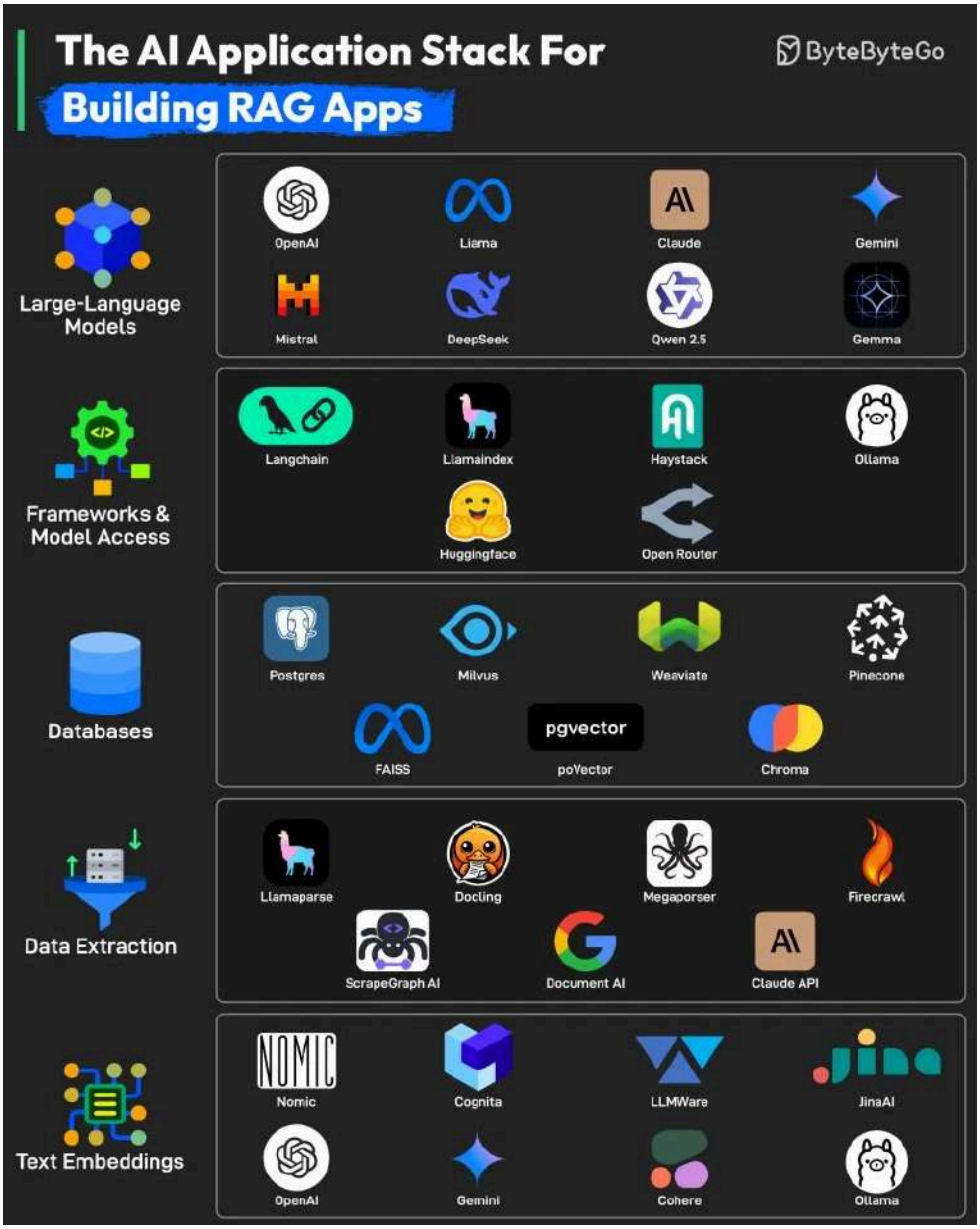


1. Machine Learning: Core algorithms, statistics, and model training techniques.
2. Deep Learning: Hierarchical neural networks learning complex representations automatically.
3. Neural Networks: Layered architectures efficiently model nonlinear relationships accurately.
4. NLP: Techniques to process and understand natural language text.
5. Computer Vision: Algorithms interpreting and analyzing visual data effectively
6. Reinforcement Learning: Distributed traffic across multiple servers for reliability.
7. Generative Models: Creating new data samples using learned data.

8. LLM: Generates human-like text using massive pre-trained data.
9. Transformers: Self-attention-based architecture powering modern AI models.
10. Feature Engineering: Designing informative features to improve model performance significantly.
11. Supervised Learning: Learns useful representations without labeled data.
12. Bayesian Learning: Incorporate uncertainty using probabilistic model approaches.
13. Prompt Engineering: Crafting effective inputs to guide generative model outputs.
14. AI Agents: Autonomous systems that perceive, decide, and act.
15. Fine-Tuning Models: Customizes pre-trained models for domain-specific tasks.
16. Multimodal Models: Processes and generates across multiple data types like images, videos, and text.
17. Embeddings: Transforms input into machine-readable vector formats.
18. Vector Search: Finds similar items using dense vector embeddings.
19. Model Evaluation: Assessing predictive performance using validation techniques.
20. AI Infrastructure: Deploying scalable systems to support AI operations.

Over to you: Which other AI concept will you add to the list?

The AI Application Stack for Building RAG Apps



1. Large Language Models
   These are the core engines behind Retrieval-Augmented Generation (RAG), responsible for understanding queries and generating coherent and contextual responses. Some common LLM options are OpenAI GPT models, Llama, Claude, Gemini, Mistral, DeepSeek, Qwen 2.5, Gemma, etc.

2. Frameworks and Model Access
   These tools simplify the integration of LLMs into your applications by handling prompt orchestration, model switching, memory, chaining, and routing. Common tools are Langchain, LlamaIndex, Haystack, Ollama, Hugging Face, and OpenRouter.

3. Databases
   RAG applications rely on storing and retrieving relevant information. These vector databases are optimized for similarity search, while relational options like Postgres offer structured storage. Tools are Postgres, FAISS, Milvus, pgVector, Weaviate, Pinecone, Chroma, etc.

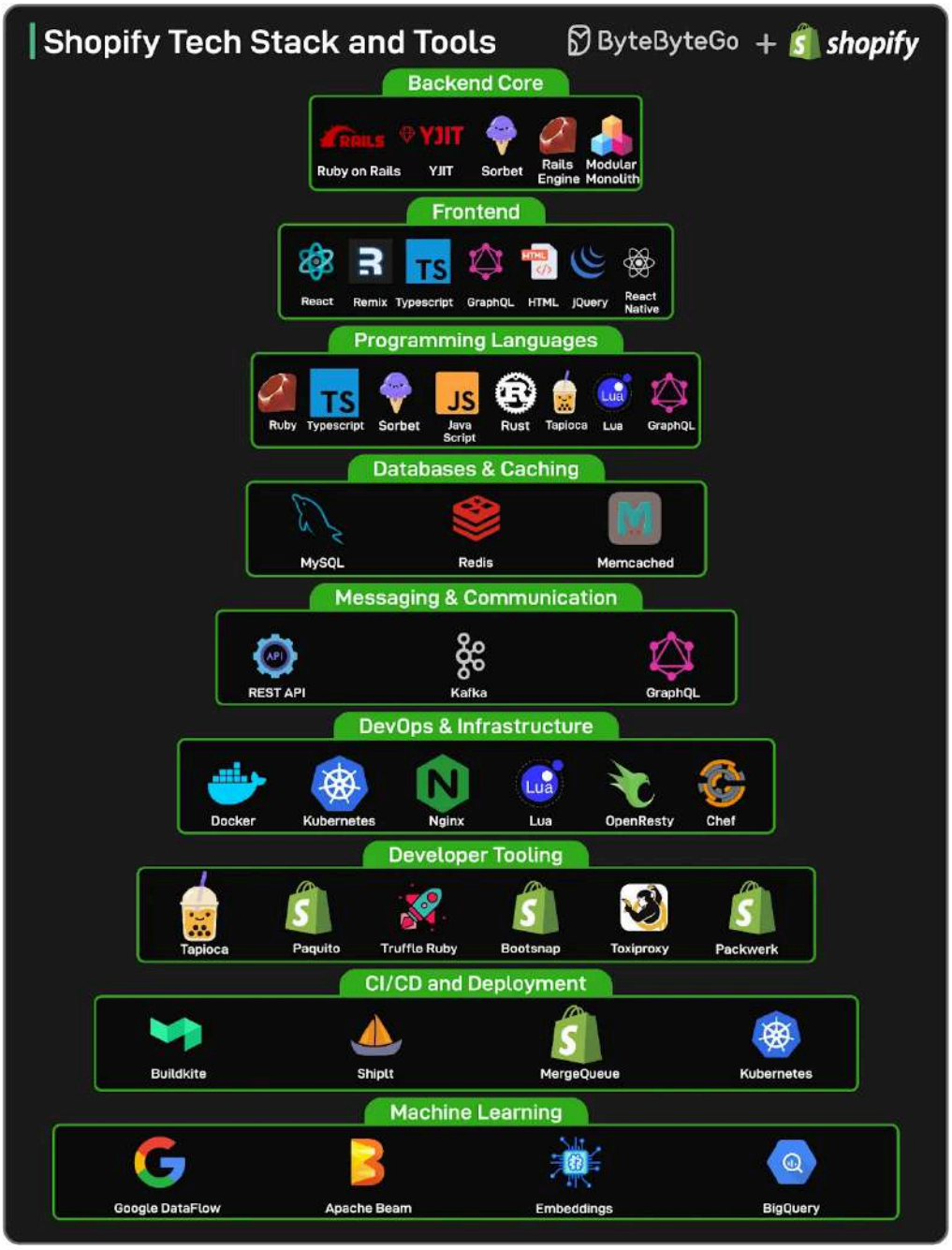
4. Data Extraction
   To populate your knowledge base, these tools help extract structured information from unstructured sources like PDFs, websites, and APIs. Some common tools are Llamaparse, Docking, Megaparser, Firecrawl, ScrapeGraph AI, Document AI, and Claude API.

5. Text Embeddings
   Embeddings convert text into high-dimensional vectors that enable semantic similarity search, which is a critical step for connecting queries with relevant context in RAG. Common tools are Nomic, OpenAI, Cognita, Gemini, LLMWare, Cohere, JinaAI, and Ollama.

Over to you: What else will you add to the list to build RAG apps?

Shopify Tech Stacks and Tools

Shopify handles scale that would break most systems.



On a single day (Black Friday 2024), the platform processed 173 billion requests, peaked at 284 million requests per minute, and pushed 12 terabytes of traffic every minute through its edge.

These numbers aren't anomalies. They're sustained targets that Shopify strives to meet. Behind this scale is a stack that looks deceptively simple from the outside: Ruby on Rails, React, MySQL, and Kafka.

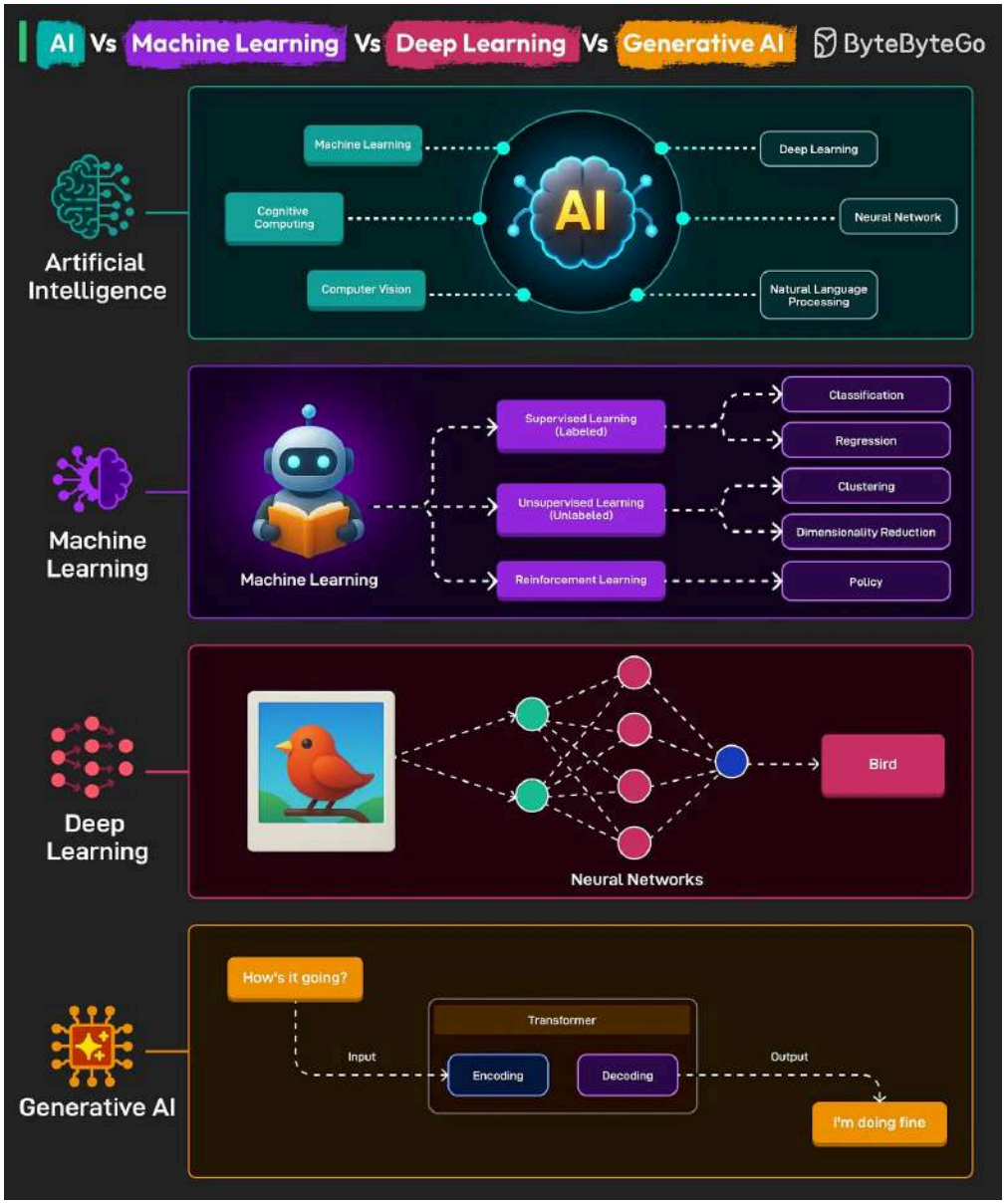
But that simplicity hides sharp architectural decisions, years of refactoring, and thousands of deliberate trade-offs.

In this newsletter, we map the tech stack powering Shopify from:

- the modular monolith that still runs the business,
- to the pods that isolate failure domains,
- to the deployment pipelines that ship hundreds of changes a day.
- It covers the tools, programming languages, and patterns Shopify uses to stay fast, resilient, and developer-friendly at incredible scale.

A huge thank you to Shopify's world-class engineering team for sharing their insights and for collaborating with us on this deep technical exploration.
🔗 Dive into the full newsletter here.

AI Vs Machine Learning Vs Deep Learning Vs Generative AI



1. Artificial Intelligence (AI)
   It is the overarching field focused on creating machines or systems that can perform tasks typically requiring human intelligence, such as reasoning, learning, problem-solving, and language understanding. AI consists of various subfields, including ML, NLP, Robotics, and Computer Vision

2. Machine Learning (ML)
   It is a subset of AI that focuses on developing algorithms that enable computers to learn from and make decisions based on data.

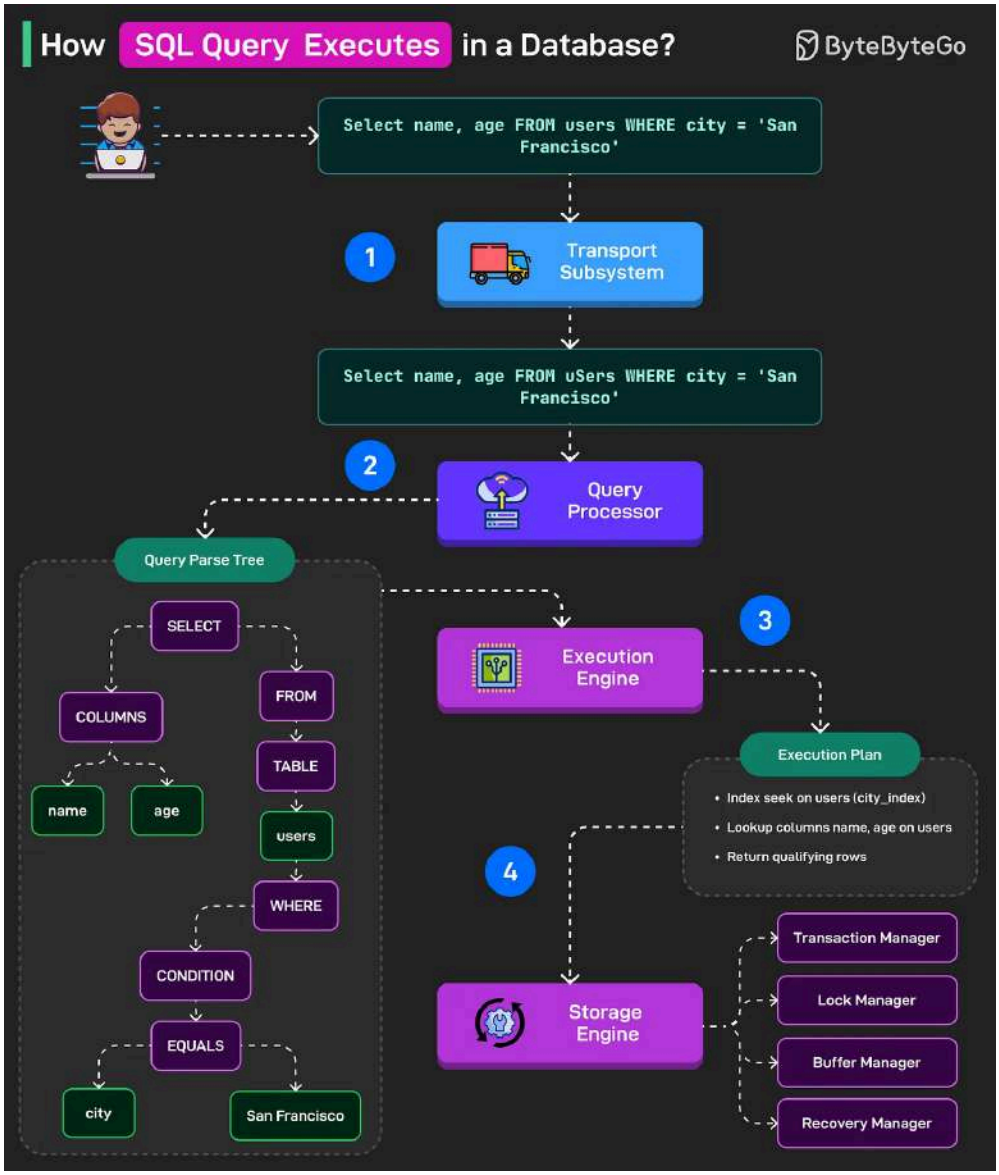
Instead of being explicitly programmed for every task, ML systems improve their performance as they are exposed to more data. Common applications include spam detection, recommendation systems, and predictive analytics.

3. Deep Learning
   It is a specialized subset of ML that utilizes artificial neural networks with multiple layers to model complex patterns in data.
   Neural networks are computational models inspired by the human brain's network of neurons. Deep neural networks can automatically discover representations needed for future detection. Use cases include image and speech recognition, NLP, and autonomous vehicles.

4. Generative AI
   It refers to AI systems capable of generating new content, such as text, images, music, or code, that resembles the data they were trained on. They rely on the Transformer Architecture.
   Notable generative AI models include GPT for text generation and DALL-E for image creation.

Over to you: What else will you add to understand these concepts better?

How SQL Query Executes In A Database?



STEP 1

The query string first reaches the Transport Subsystem of the database. This subsystem manages the connection with the client. Also, it performs authentication and authorization checks, and if everything looks fine, it lets the query go to the next step.

STEP 2

The query now reaches the Query Processor subsystem, which has two parts: Query Parser and Query Optimizer.

The Query Parser breaks down the query into sub-parts (such as SELECT, FROM, WHERE). It checks for any syntax errors and creates a parse tree.

Then, the Query Optimizer goes through the parse tree, checks for semantic errors (for example, if the “users” table exists or not), and finds out the most efficient way to execute the query.

The output of this step is the execution plan.

STEP 3

The execution plan goes to the Execution Engine. This plan is made up of all the steps needed to execute the query.

The Execution Engine takes this plan and coordinates the execution of each step by calling the Storage Engine. It also collects the results from each step and returns a combined or unified response to the upper layer.

STEP 4

The Execution Engine sends low-level read and write requests to the Storage Engine based on the execution plan.

This is handled by the various components of the Storage Engine, such as the transaction manager (for transaction management), lock manager (acquires necessary locks), buffer manager (checks if data pages are in memory), and recovery manager (for rollback or recovery).

Over to you: What else will you add to understand the execution of an SQL Query?

Top 20 AI Agent Concepts You Should Know



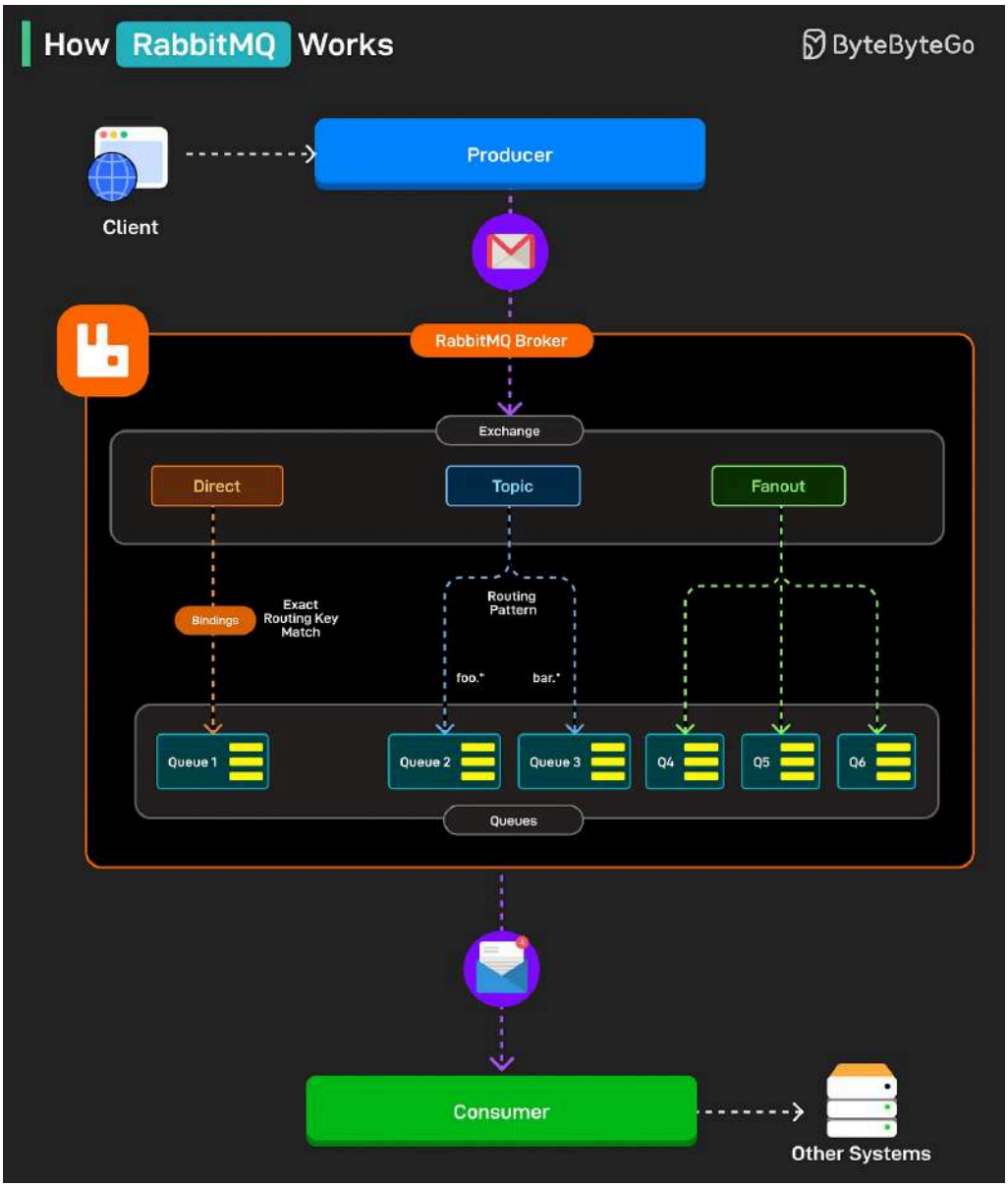
1. Agent: An autonomous entity that perceives, reasons, and acts in an environment to achieve goals.
2. Environment: The surrounding context or sandbox in which the agent operates and interacts.
3. Perception: The process of interpreting sensory or environmental data to build situational awareness.
4. State: The agent’s current internal condition or representation of the world.

5. Memory: Storage of recent or historical information for continuity and learning.
6. Large Language Models: Foundation models powering language understanding and generation.
7. Reflex Agent: A simple type of agent that makes decisions based on predefined “condition-action” rules.
8. Knowledge Base: Structured or unstructured data repository used by agents to inform decisions.
9. CoT (Chain of Thought): A reasoning method where agents articulate intermediate steps for complex tasks.
10. ReACT: A framework that combines step-by-step reasoning with direct environmental actions.
11. Tools: APIs or external systems that agents use to augment their capabilities.
12. Action: Any task or behavior executed by the agent as a result of its reasoning.
13. Planning: Devising a sequence of actions to reach a specific goal.
14. Orchestration: Coordinating multiple steps, tools, or agents to fulfill a task pipeline.
15. Handoffs: The transfer of responsibilities or tasks between different agents.
16. Multi-Agent System: A framework where multiple agents operate and collaborate in the same environment.
17. Swarm: Emergent intelligent behavior from many agents following local rules without central control.
18. Agent Debate: A mechanism where agents argue opposing views to refine or improve outcomes.
19. Evaluation: Measuring the effectiveness or success of an agent’s actions and outcomes.
20. Learning Loop: The cycle where agents improve performance by continuously learning from feedback or outcomes.

Over to you: Which other AI agent concept will you add to the list?

How RabbitMQ Works?

RabbitMQ is a message broker that enables applications to communicate by sending and receiving messages through queues. It helps decouple services, improve scalability, and handle asynchronous processing efficiently.



Here's how it works:

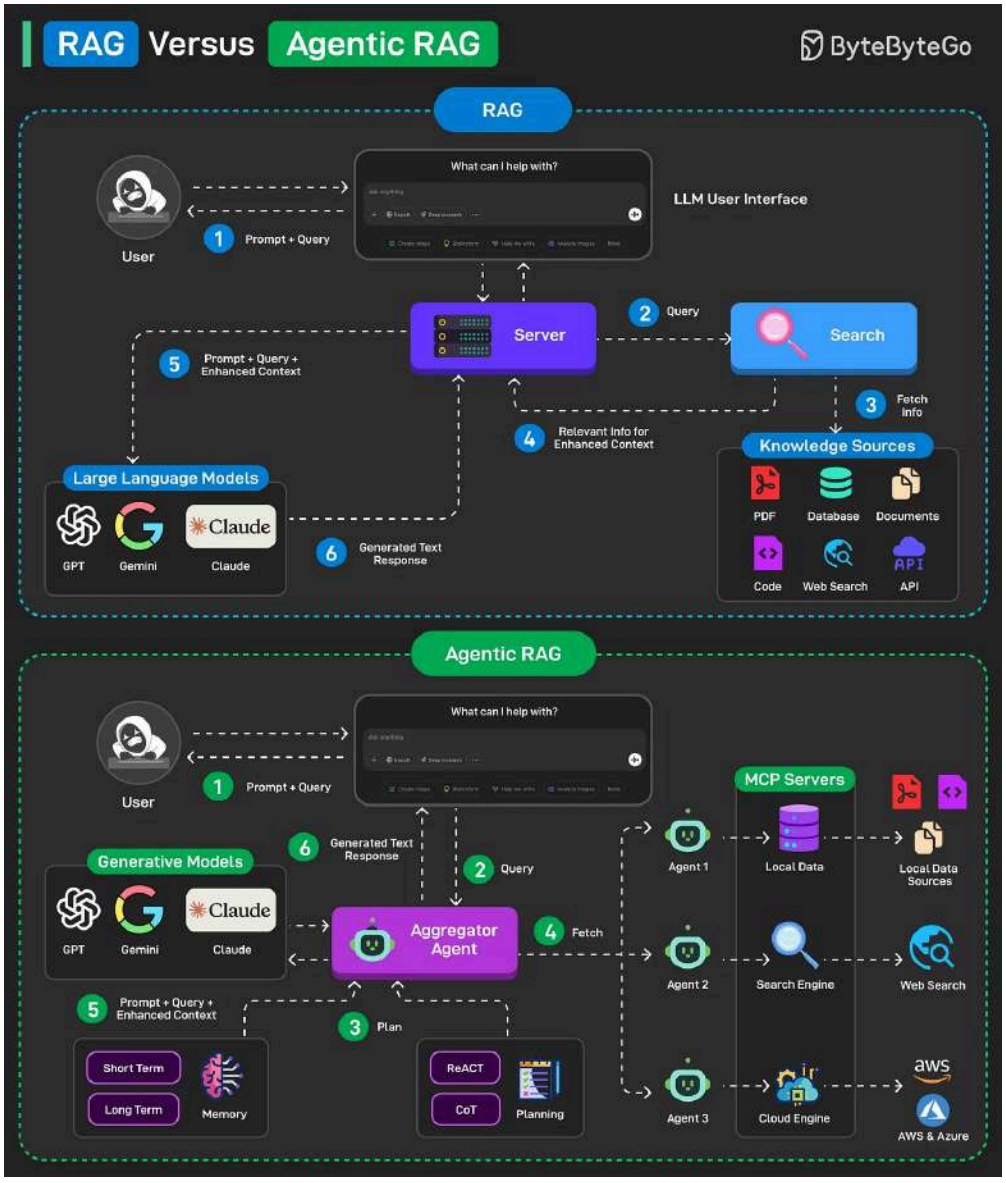
1. A producer (usually an application or service) sends messages to the RabbitMQ broker, which manages message routing and delivery.

2. Within the broker, messages are sent to an exchange, which determines how they should be routed based on the type of exchange: Direct, Topic, or Fanout.

3. Bindings connect exchanges to queues using a binding key, which defines the rules for routing messages (for example, exact match or pattern-based)
4. Direct exchanges route messages to queues that match the routing key exactly, as shown with Queue 1.
5. Topic exchanges use patterns to route messages to matching queues.
6. Fanout exchanges broadcast messages to all bound queues, regardless of routing keys.
7. Finally, messages are pulled from the queues by a consumer, which processes them and can pass the results to other systems.

Over to you: What else will you add to the RabbitMQ process flow?

RAG vs Agentic RAG



RAG (Retrieval Augmented Generation) is a method that combines information retrieval with large language models to generate answers. Here's how RAG works on a high level:

1. The model retrieves relevant data from data sources and then extracts it to a vector database from the pre-indexed model.
2. Augment the prompts by retrieving information and merging it with the query prompt.
3. A Large Language Model (like GPT, Claude, or Gemini) understands the combined query and generates the final response.

A traditional RAG has a simple retrieval, limited adaptability, and relies on static knowledge, making it less flexible for dynamic and real-time information.

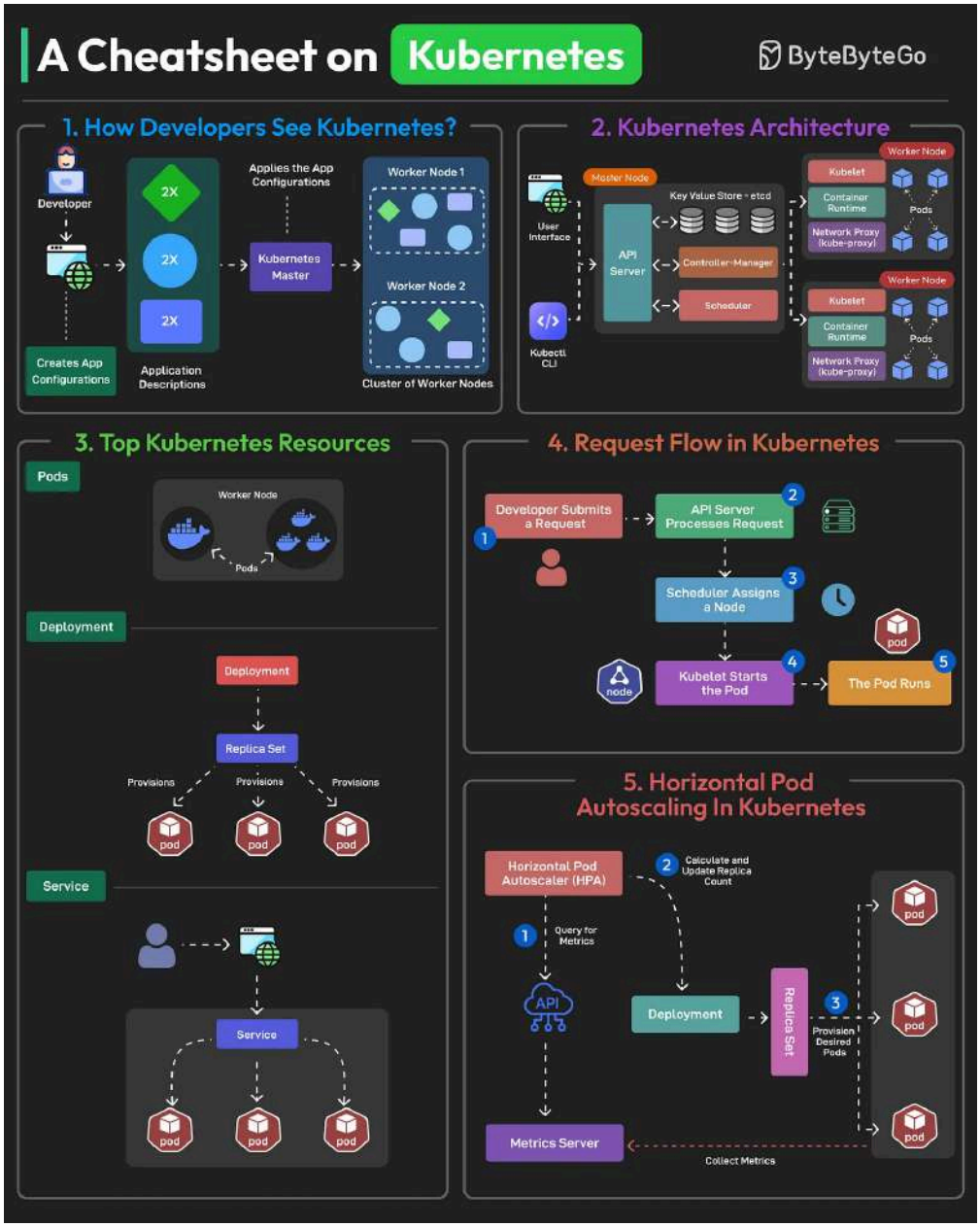
Agentic RAG improves on this by introducing AI agents that can make decisions, select tools, and even refine queries for more accurate and flexible responses. Here's how Agentic RAG works on a high level:

1. The user query is directed to an AI Agent for processing.
2. The agent uses short-term and long-term memory to track query context. It also formulates a retrieval strategy and selects appropriate tools for the job.
3. The data fetching process can use tools such as vector search, multiple agents, and MCP servers to gather relevant data from the knowledge base.
4. The agent then combines retrieved data with a query and system prompt. It passes this data to the LLM.
5. LLM processes the optimized input to answer the user's query.

Over to you: What else will you add to better understand RAG vs Agentic RAG?

A Cheatsheet on Kubernetes

Kubernetes (K8S) is an open-source container orchestration platform originally developed by Google and now maintained by the Cloud Native Computing Foundation (CNCF).



Developers working with Kubernetes create manifest files that describe their application and specify the number of instances, resource requirements, and other configurations.

Kubernetes operates using a control pane and a group of nodes.
The control plane is deployed on the master node and manages the overall state of the cluster. It consists of components like API Server, Etcd, Controller Manager, and Scheduler.

Nodes are the workers in a Kubernetes cluster. Each node contains components like the Kubelet and Kube-proxy and is responsible for running the containerized applications.

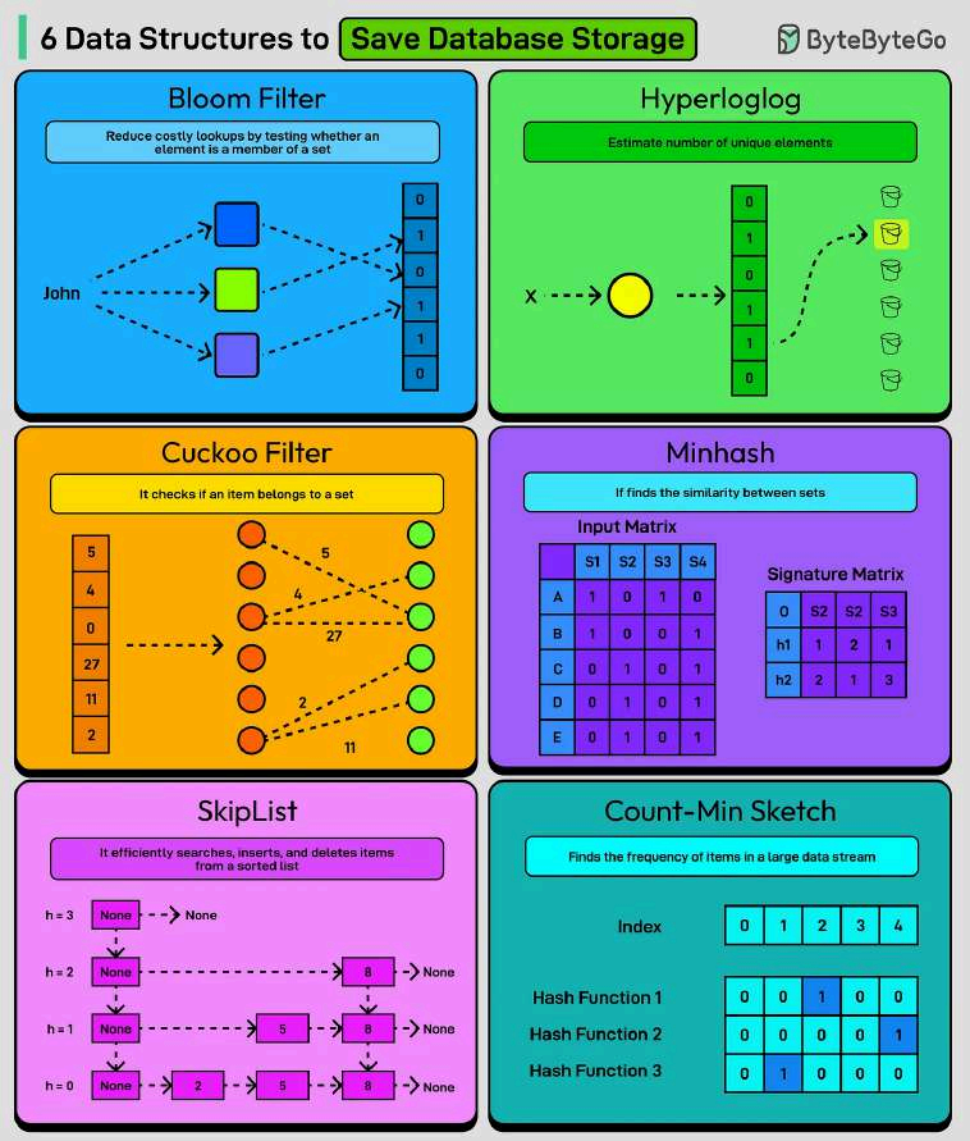
Some top Kubernetes resources are Pods, Deployments, Services, and Persistent Volumes.

1. Pods encapsulate one or more containers.
2. Deployment is a higher-level abstraction that manages Pods.
3. Service is an abstraction that provides a stable way to expose and access a set of Pods.
4. Volume is a storage resource that helps retain data between pod restarts or recreations.

The Horizontal Pod Autoscaler helps scale the number of Pods in a Deployment, ReplicaSet, or StatefulSet based on observed CPU utilization, memory usage, or custom metrics. It monitors resource usage and adjusts the number of replicas to match the desired resource targets.

Over to you: Have you used Kubernetes in your projects?

6 Data Structures to Save Storage



1. Bloom Filter
   A probabilistic data structure used to test whether an element is a member of a set.

2. HyperLogLog
   An algorithm that approximates the number of unique elements in a multi-set using minimal memory.

3. Cuckoo Filter
   A space-efficient alternative to Bloom filters that supports deletion and has better lookup performance.

4. Minhash
   A technique for quickly estimating the similarity between large sets using compressed hash signatures.

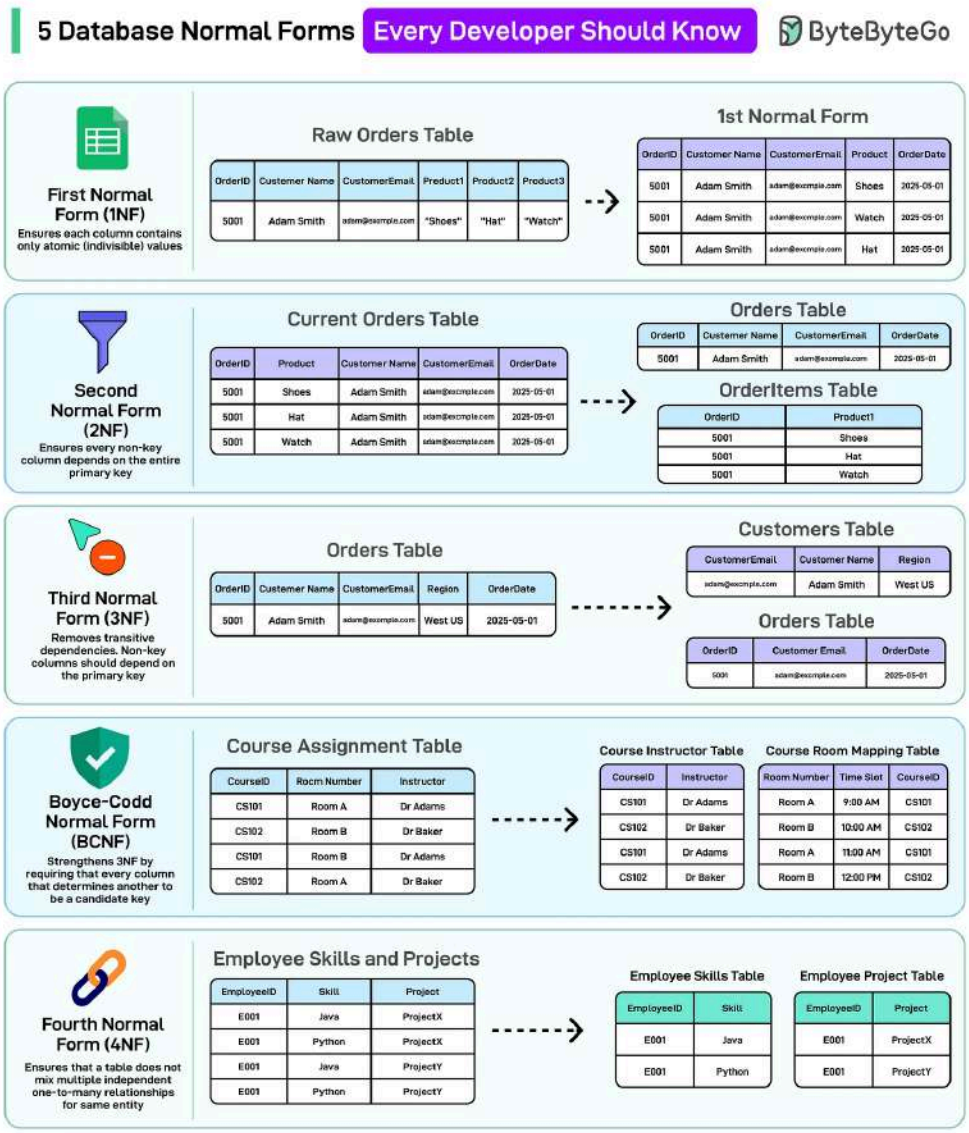
5. SkipList
   A layered linked list structure that allows fast search, insert, and delete operations.

6. Count-Min Sketch
   A probabilistic data structure that approximates the frequency of items in a large data stream

Over to you: Which other data structure will you add to the list?

5 Database Normal Forms Every Developer Should Know

Normalization aims to eliminate redundancy and enforce data integrity by organizing data into logical, dependency-driven forms.



1. First Normal Form (1NF): Removes repeating groups and ensures atomic values in each column.
2. Second Normal Form (2NF): Removes partial dependencies by ensuring all non-key columns depend on the full primary key.
3. Third Normal Form (3NF): Eliminates transitive dependencies, ensuring non-key columns depend only on primary keys.

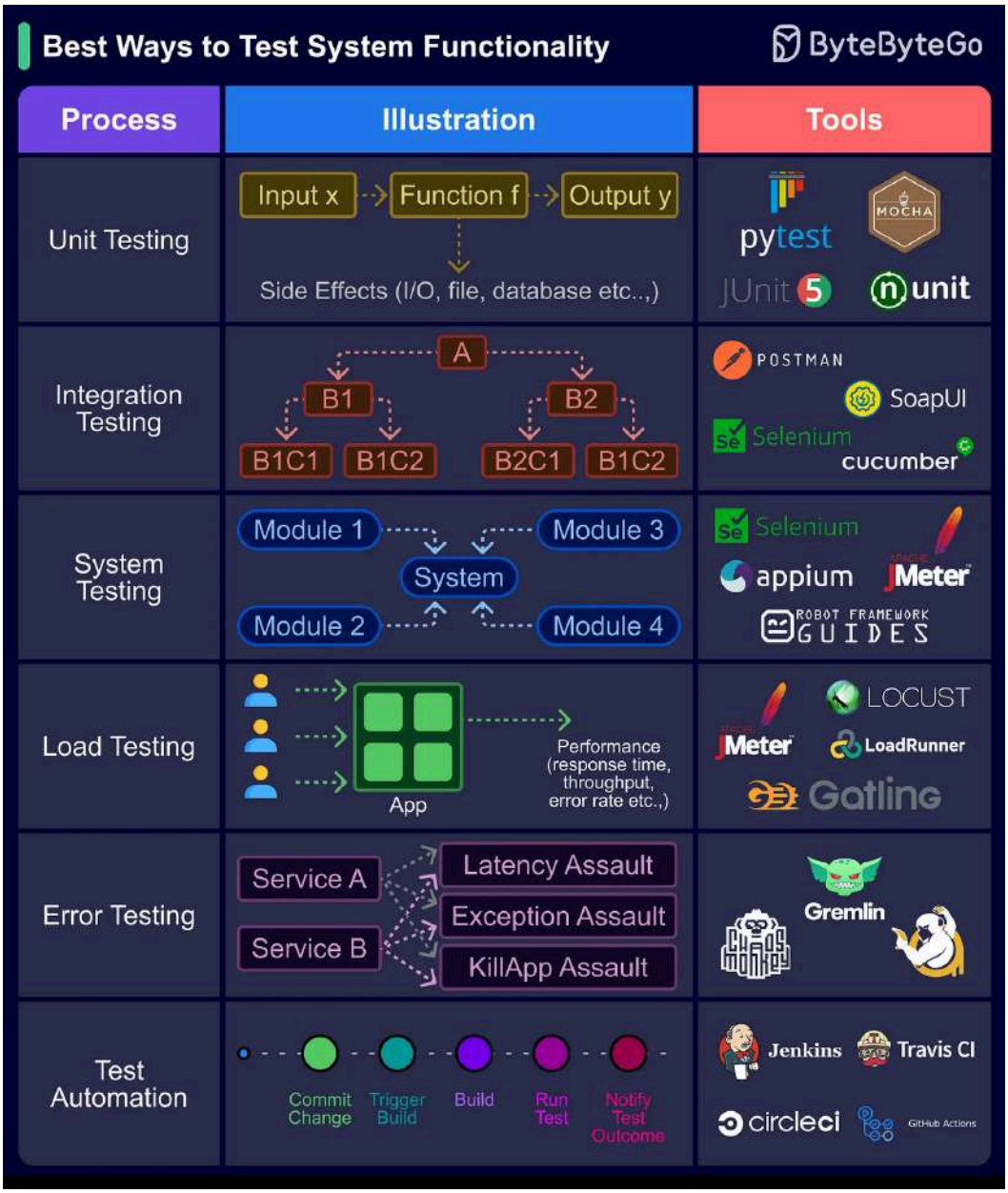
4. Boyce-Codd Normal Form (BCNF): Strengthens 3NF by removing anomalies that exist due to overlapping candidate keys. If one column depends on some other column, then that “other column” must be enough to identify each row uniquely.
5. Fourth Normal Form (4NF): Ensures that a table does not mix multiple independent one-to-many relationships for the same entity.

Over to you: Which other Normal Form will you add to the list?

Best ways to test system functionality

Testing system functionality is a crucial step in software development and engineering processes.

It ensures that a system or software application performs as expected, meets user requirements, and operates reliably.



Here we delve into the best ways:

1. Unit Testing: Ensures individual code components work correctly in isolation.
2. Integration Testing: Verifies that different system parts function seamlessly together.

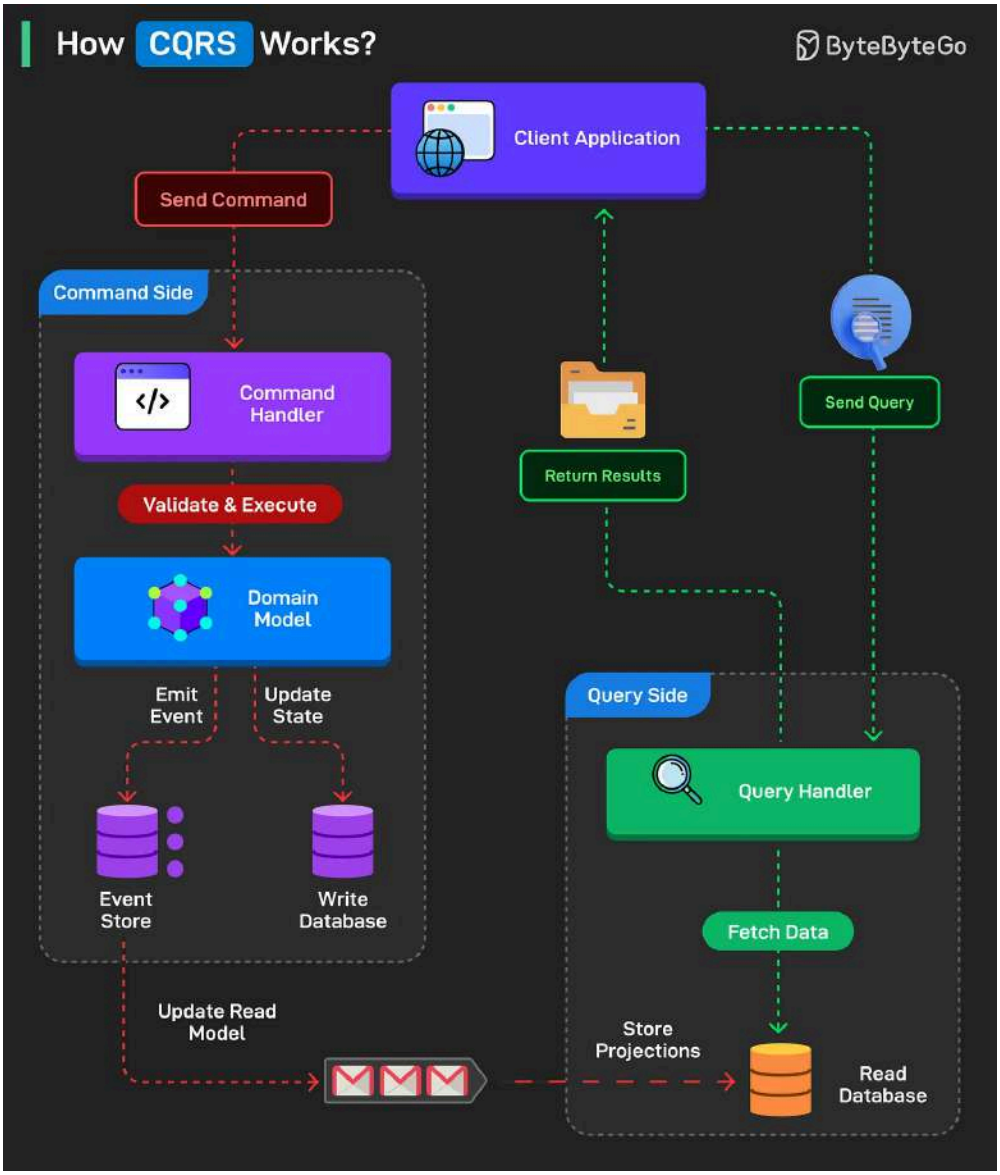
3. System Testing: Assesses the entire system's compliance with user requirements and performance.
4. Load Testing: Tests a system's ability to handle high workloads and identifies performance issues.
5. Error Testing: Evaluates how the software handles invalid inputs and error conditions.
6. Test Automation: Automates test case execution for efficiency, repeatability, and error reduction.

Over to you: How do you approach testing system functionality in your software development or engineering projects?

Over to you: what's your company's release process look like?

How CQRS Works?

CQRS (Command Query Responsibility Segregation) separates write (Command) and read (Query) operations for better scalability and maintainability.



Here's how it works:

1 - The client sends a command to update the system state. A Command Handler validates and executes logic using the Domain Model.

2 - Changes are saved in the Write Database and can also be saved to an Event Store. Events are emitted to update the Read Model asynchronously.

3 - The projections are stored in the Read Database. This database is eventually consistent with the Write Database.

4 - On the query side, the client sends a query to retrieve data.

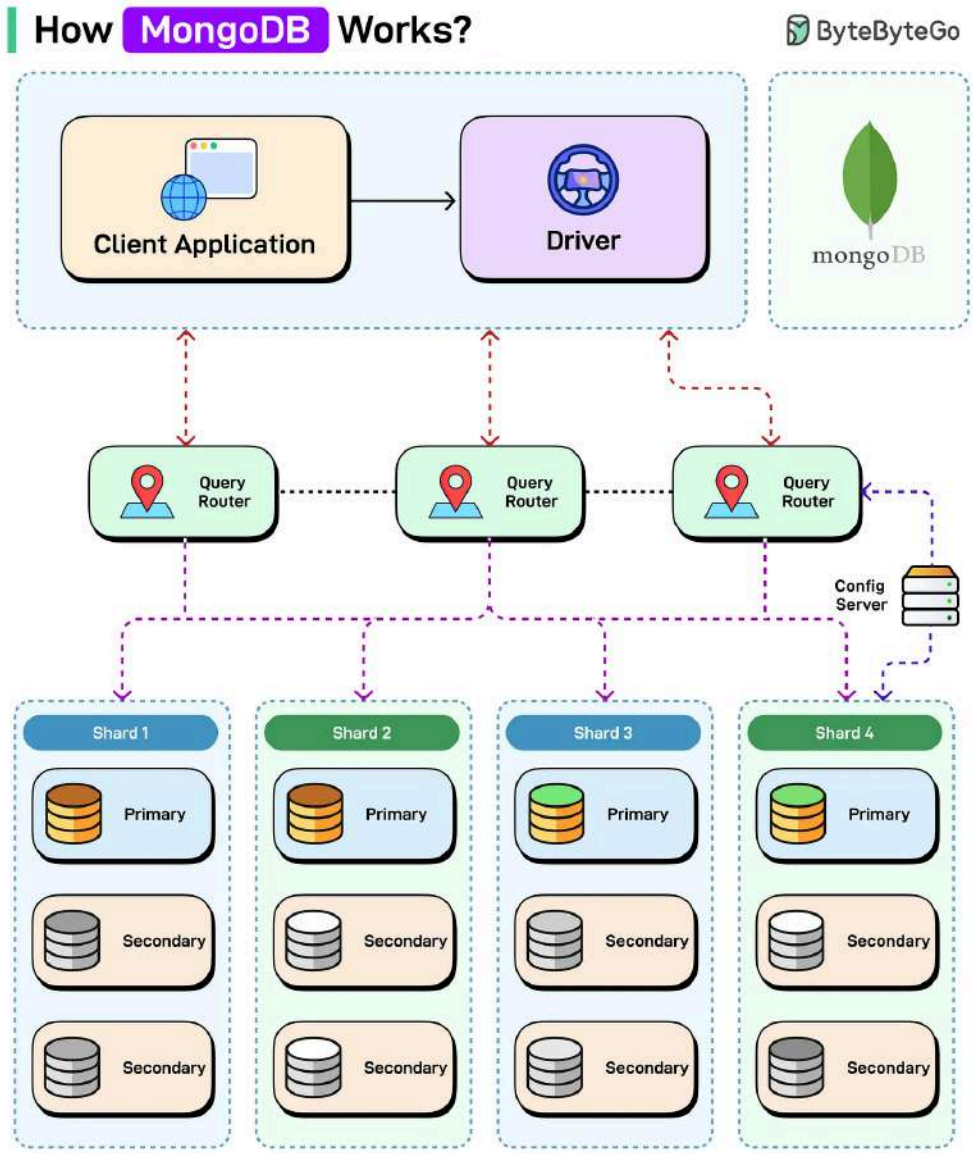
5 - A Query Handler fetches data from the Read Database, which contains precomputed projections.

6 - Results are returned to the client without hitting the write model or the write database.

Over to you: What else will you add to understand CQRS?

How MongoDB Works?

MongoDB is a popular NoSQL database designed for flexibility, scalability, and high performance. It stores data in a JSON-like format (BSON) and supports horizontal scaling through sharding and replication.



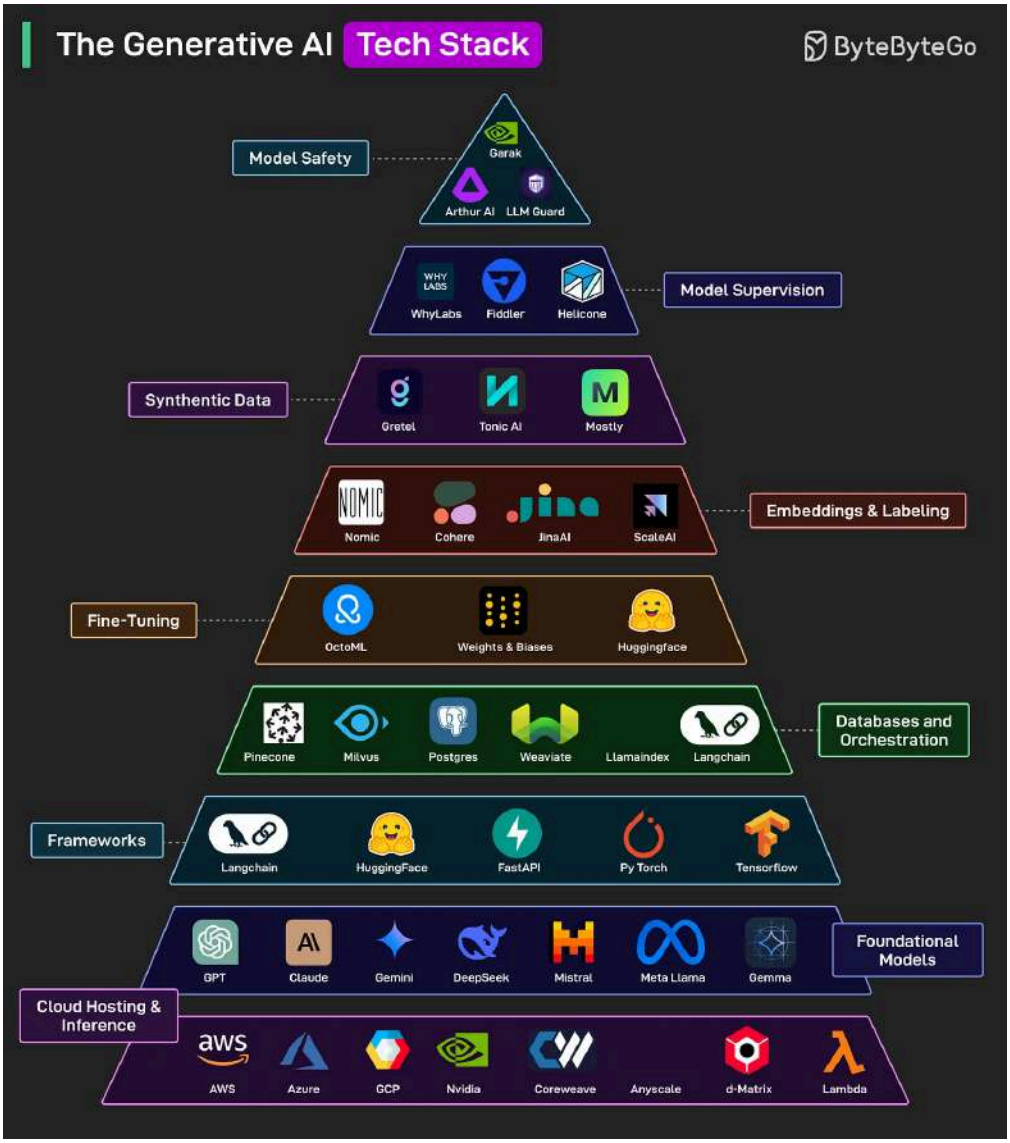
Here's how it works:

1. Client application connects via a MongoDB Driver to perform read/write operations.
2. The Query Router (mongos) acts as a mediator, directing queries to the appropriate shard based on the data's shard key.

3. Config servers store metadata and routing information. This helps query routers locate data across shards.
4. The data is distributed across multiple shards to support horizontal scaling.
5. Each shard is a replica set that consists of one primary node for handling writes and multiple secondary nodes for high availability and read scaling.
6. If a primary fails, a secondary is automatically elected to replace it and maintain availability.

Over to you: What else will you add to understand MongoDB's working better?

The Generative AI Tech Stack



GenAI refers to systems capable of creating new content, such as text, images, code, or music, by learning patterns from existing data. Here are the key building blocks for GenAI Tech Stack:

1. Cloud Hosting & Inference: Providers like AWS, GCP, Azure, and Nvidia offer the infrastructure to run and scale AI workloads.

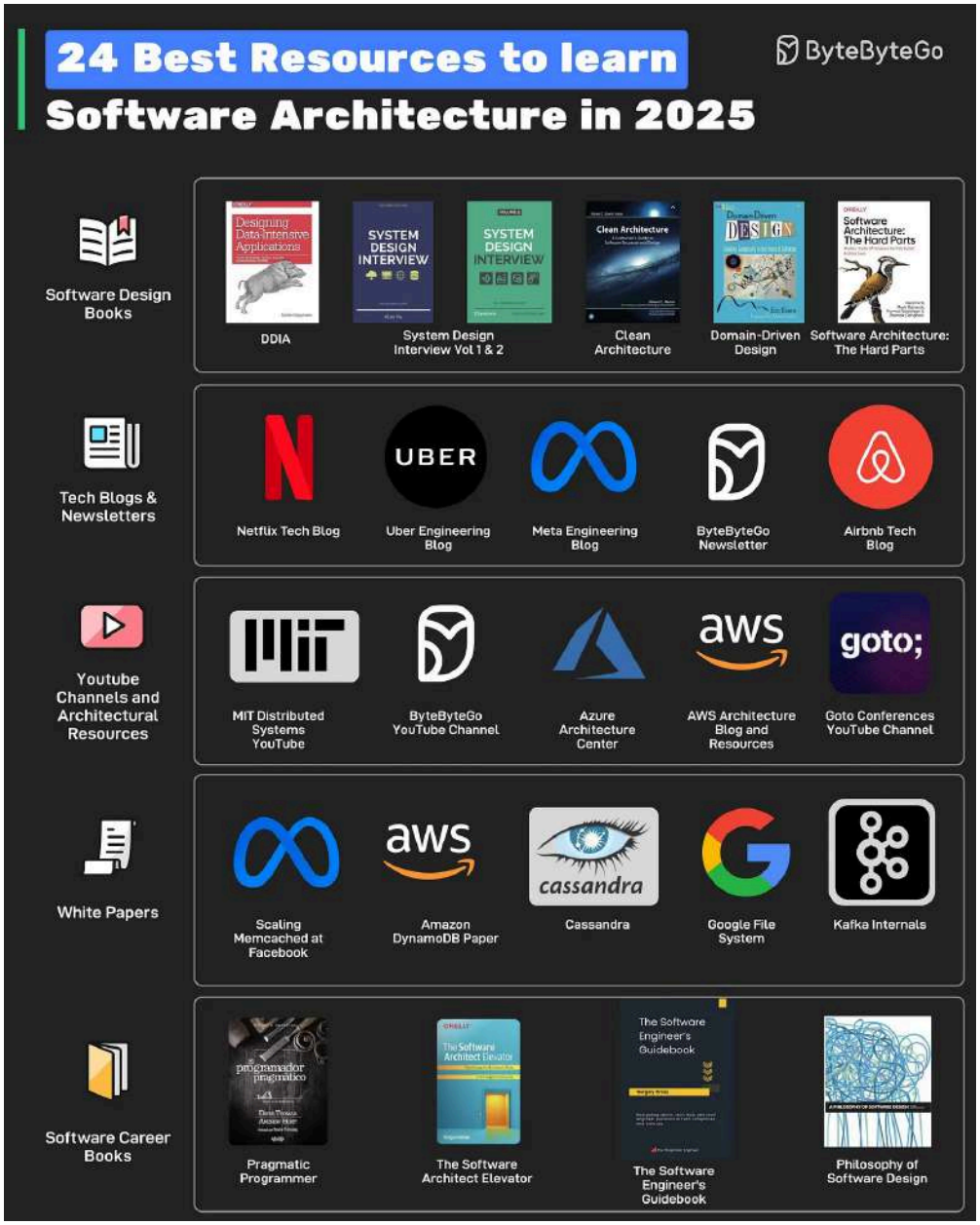
2. Foundational Models: Core LLMs (such as GPT, Claude, Mistral, Llama, Gemini, Deepseek) trained on massive data, form the base for all GenAI applications.

3. Frameworks: Tools like LangChain, PyTorch, and Hugging Face help build, deploy, and integrate models into apps.

4. Databases and Orchestration: Vector DBs (such as Pinecone, Weaviate), orchestration tools (such as LangChain, LlamaIndex) manage memory, retrieval, and logic flow.
5. Fine-Tuning: Platforms like Weights & Biases, OctoML, and Hugging Face enable training models for specific tasks or domains.
6. Embeddings and Labeling: Services like Cohere, Scale AI, Nomic, and JinaAI help generate and label vector representations to power search and RAG systems.
7. Synthetic Data: Tools like Gretel, Tonic AI, and Mostly AI create artificial datasets to enhance training.
8. Model Supervision: Monitor model performance, bias, and behavior. Tools such as Fiddler, Helicone, and WhyLabs help.
9. Model Safety: Helps ensure ethical, secure, and safe deployment of GenAI systems. Solutions like LLM Guard, Arthur AI, and Garak help with this.

Over to you: What else will you add to this list?

24 Good Resources to Learn Software Architecture in 2025



The resources can be divided into different types such as:

1. Software Design Books
   Some books that can help are DDIA, System Design Volume 1 & 2, Clean Architecture, Domain-Driven Design, and Software Architecture: the Hard Parts

2. Tech Blogs and Newsletters
   Read technical blogs by companies like Netflix, Uber, Meta, and Airbnb. Also, the ByteByteGo newsletter provides insights into software design every week.

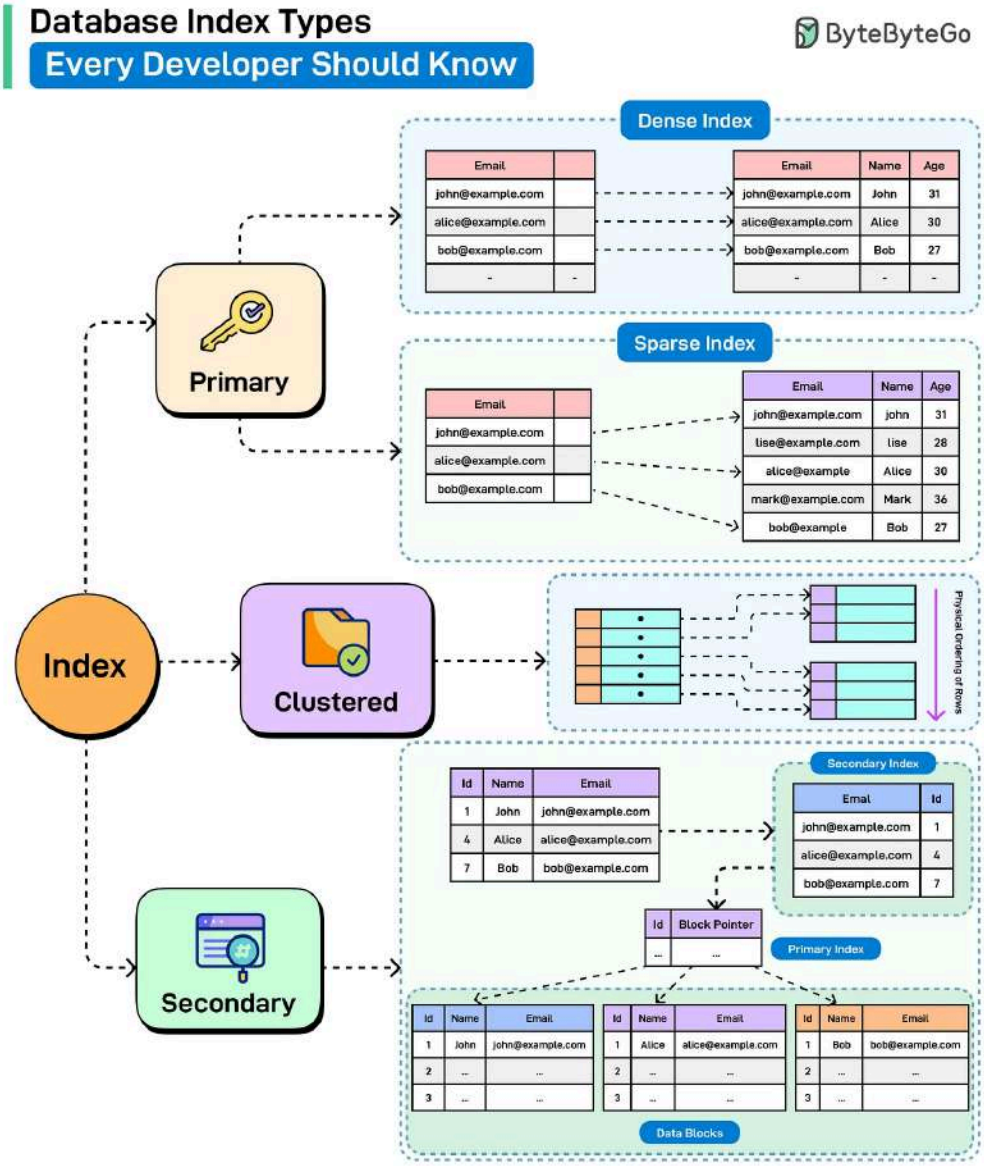
3. YouTube Channels and Architectural Resources
   YouTube channels like MIT Distributed Systems, Goto Conferences, and ByteByteGo can help with software architecture and system design. Azure Architecture Center and AWS Architecture Blog are other important resources.

4. WhitePapers
   For deeper insights, read whitepapers like Facebook Memcache Scaling, Cassandra, Amazon DynamoDB, Kafka, and Google File System.

5. Software Career Books
   A Software Architect also needs to develop holistic skills. Books about software career aspects such as Pragmatic Programmer, The Software Architect Elevator, The Software Engineer's Guidebook, and Philosophy of Software Design can help.

Over to you: Which other resources will you add to the list?

Database Index Types Every Developer Should Know



A database index is a derived structure that maps column values to the physical locations of rows in a table. Let’s look at some key index types:

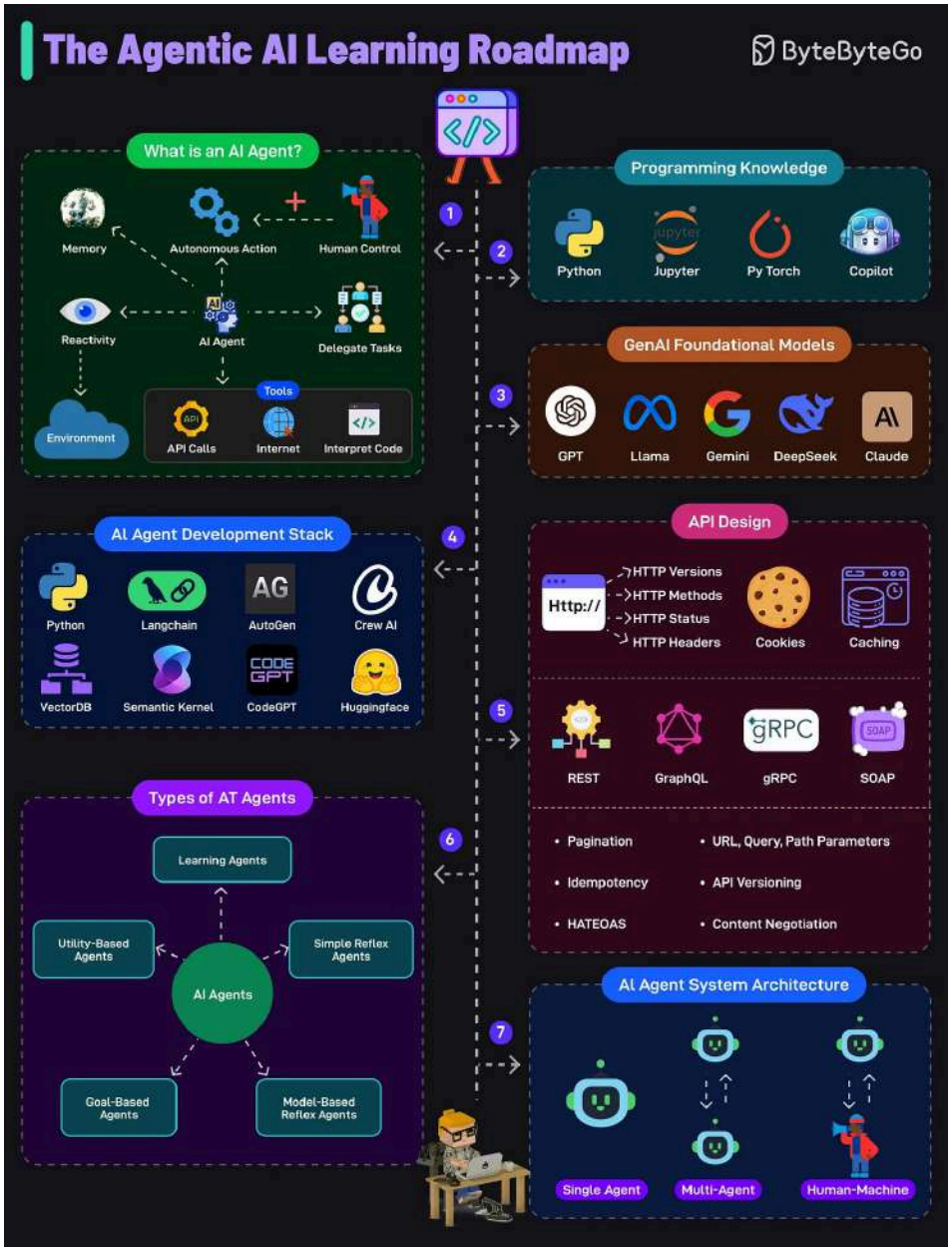
1. Primary Index
   This index is automatically created when a primary key is defined on a table. Such an index can be dense as well as sparse, though sparse is preferred in most scenarios.
   A dense index contains one entry for every row in the table. On the other hand, a sparse index contains entries for only some rows in the table.

2. Clustered Index
   A clustered index determines the physical order of rows in a table. Only one clustered index can exist on a table because data can only be stored in one order at a time. It is great for range queries, ordered scans, and I/O efficiency.

3. Secondary Index
   A non-clustered index is a separate structure that holds a copy of one or more columns along with pointers to the actual rows in the table. It doesn't affect how data is physically stored, and it can use the primary index to locate the records.

Over to you: Which other index type will you add to the list?

The Agentic AI Learning Roadmap



1. An AI Agent is a system capable of autonomous actions, reacting to its environment, using tools (APIs, Internet, code, etc), and can work under human guidance.

2. To build AI agents, one must know tools like Python, Jupyter, PyTorch, and GitHub Copilot. These enable coding, experimentation, and integration with AI libraries and APIs.

3. GenAI Foundational Models
   Familiarity with large models like GPT, Gemini, LLaMa, DeepSeek, and Claude is essential. These models provide the base intelligence that agents can use for reasoning, generation, and understanding.

4. AI Agent Development Stack
   Tools like Langchain, AutoGen, Crew AI, and frameworks like Semantic Kernel and Hugging Face power agent workflows. These components manage tasks, memory, and external tool integrations in agent pipelines.

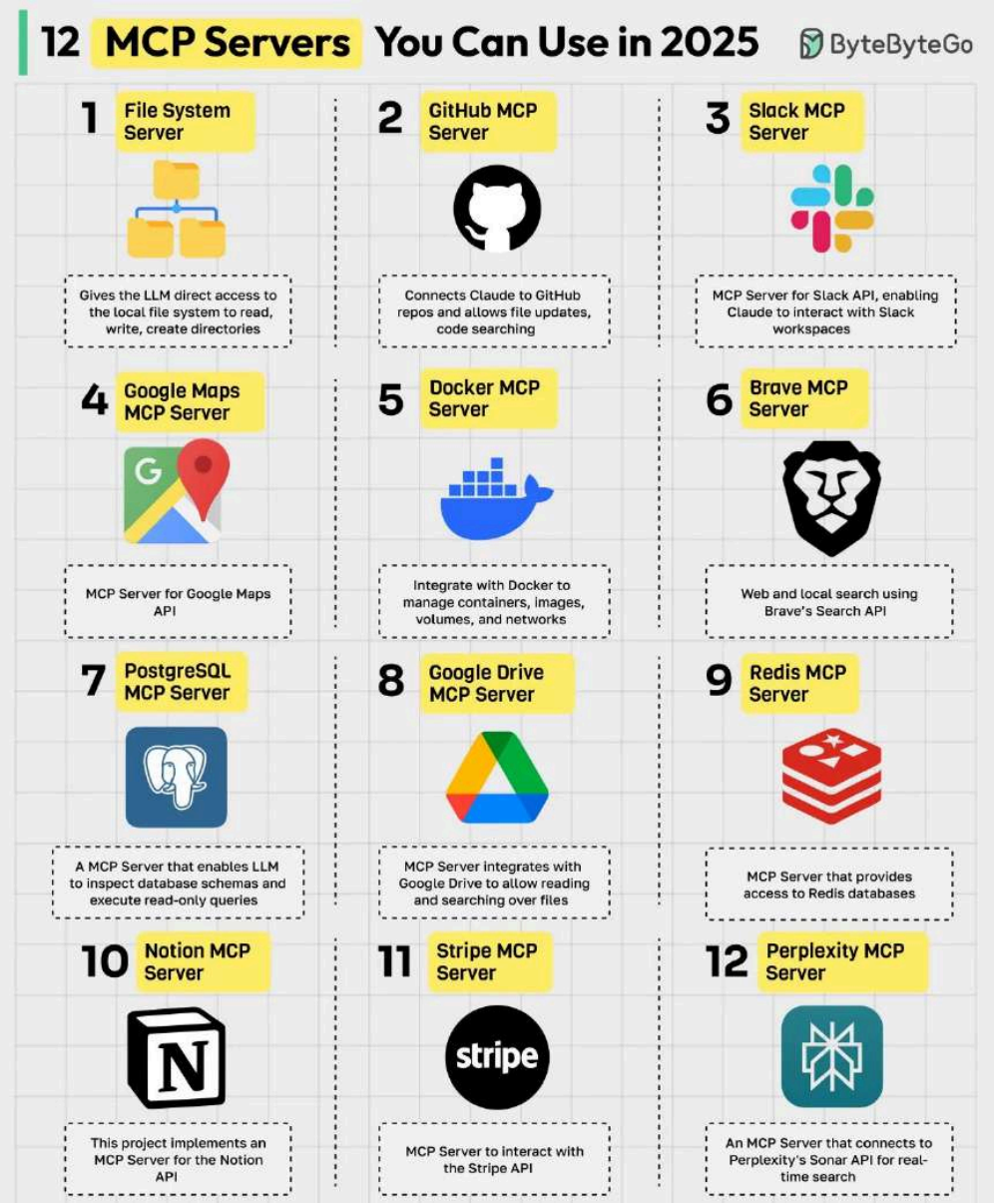
5. API Design
   Understanding API design approaches like REST, GraphQL, gRPC, and SOAP is crucial to building interoperable agents. Key concepts include HTTP methods, status codes, versioning, cookies, headers, and caching.

6. Type of AI Agents
   Learn about the several types of AI agents, such as simple reflex, model-based reflex, goal-based, utility-based, and learning agents. Each varies in complexity.

7. AI Agent System Architecture
   AI agents can operate as single agents, in multi-agent systems, or in human-machine collaboration. Architecture depends on the use case.

Over to you: What else will you add to the AI Agent Learning Roadmap?

12 MCP Servers You Can Use in 2025



MCP (Model Context Protocol) is an open standard that simplifies how AI models, particularly LLMs, interact with external data sources, tools, and services. An MCP server acts as a bridge between these AI models and external tools. Here are the top MCP servers:

1. File System MCP Server
   Allows the LLM to directly access the local file system to read, write, and create directories.

2. GitHub MCP Server
   Connects Claude to GitHub repos and allows file updates, code searching.

3. Slack MCP Server
   MCP Server for Slack API, enabling Claude to interact with Slack workspaces.

4. Google Maps MCP Server
   MCP Server for Google Maps API.

5. Docker MCP Server
   Integrate with Docker to manage containers, images, volumes, and networks.

6. Brave MCP Server
   Web and local search using Brave's Search API.

7. PostgreSQL MCP Server
   An MCP server that enables LLM to inspect database schemas and execute read-only queries.

8. Google Drive MCP Server
   An MCP server that integrates with Google Drive to allow reading and searching over files.

9. Redis MCP Server
   MCP Server that provides access to Redis databases.

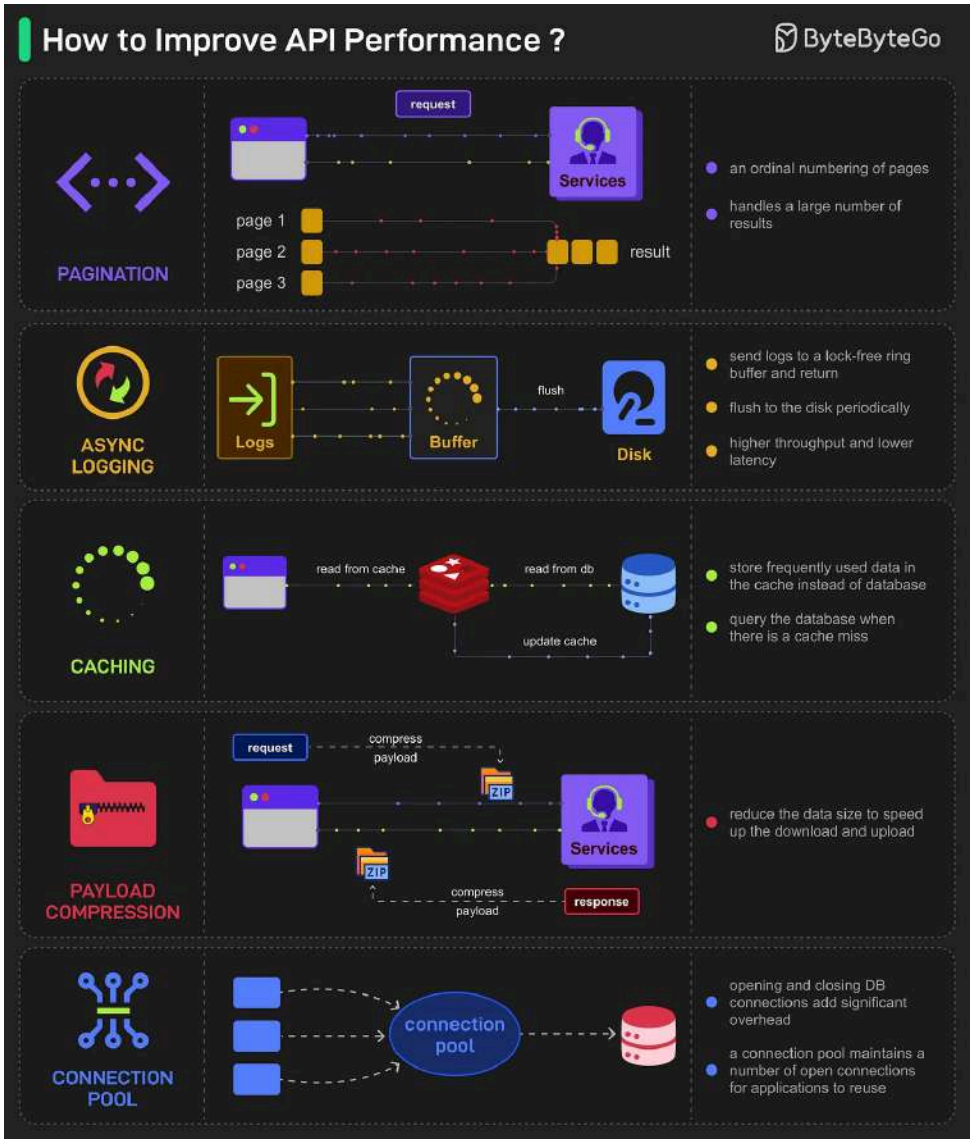
10. Notion MCP Server
    This project implements an MCP server for the Notion API.

11. Stripe MCP Server
    MCP Server to interact with the Stripe API.

12. Perplexity MCP Server
    An MCP Server that connects to Perplexity's Sonar API for real-time search.

Over to you: Which other MCP Server will you add to the list?

Top 5 common ways to improve API performance



- Result Pagination:
  This method is used to optimize large result sets by streaming them back to the client, enhancing service responsiveness and user experience.

- Asynchronous Logging:
  This approach involves sending logs to a lock-free buffer and returning immediately, rather than dealing with the disk on every call. Logs are periodically flushed to the disk, significantly reducing I/O overhead.

- Data Caching:
  Frequently accessed data can be stored in a cache to speed up retrieval. Clients check the cache before querying the database, with data storage solutions like Redis offering faster access due to in-memory storage.

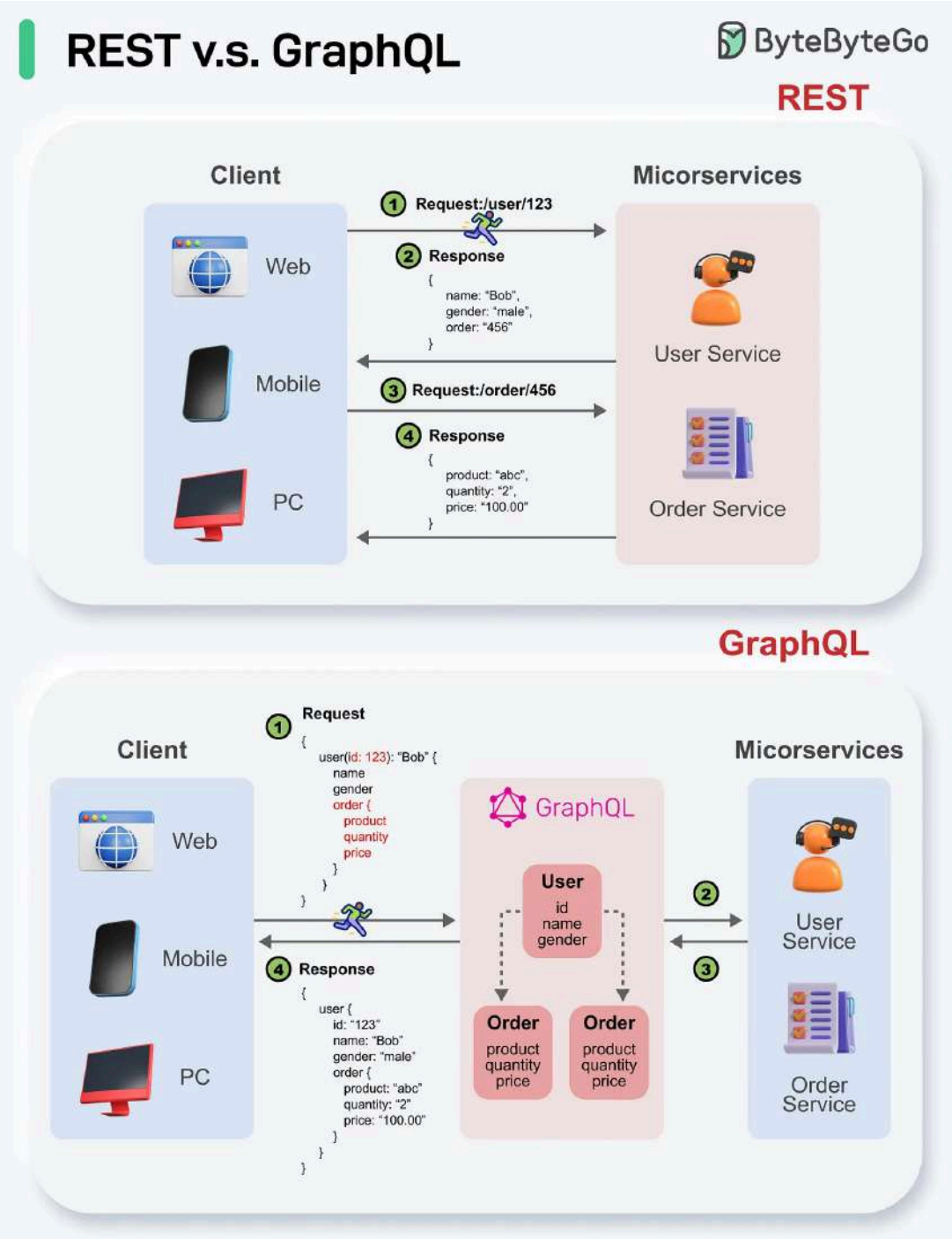
- Payload Compression:
  To reduce data transmission time, requests and responses can be compressed (e.g., using gzip), making the upload and download processes quicker.

- Connection Pooling:
  This technique involves using a pool of open connections to manage database interaction, which reduces the overhead associated with opening and closing connections each time data needs to be loaded. The pool manages the lifecycle of connections for efficient resource use.

Over to you: What other ways do you use to improve API performance?

REST API Vs. GraphQL

When it comes to API design, REST and GraphQL each have their own strengths and weaknesses.



REST

- Uses standard HTTP methods like GET, POST, PUT, DELETE for CRUD operations.

- Works well when you need simple, uniform interfaces between separate services/applications.
- Caching strategies are straightforward to implement.
- The downside is it may require multiple roundtrips to assemble related data from separate endpoints.

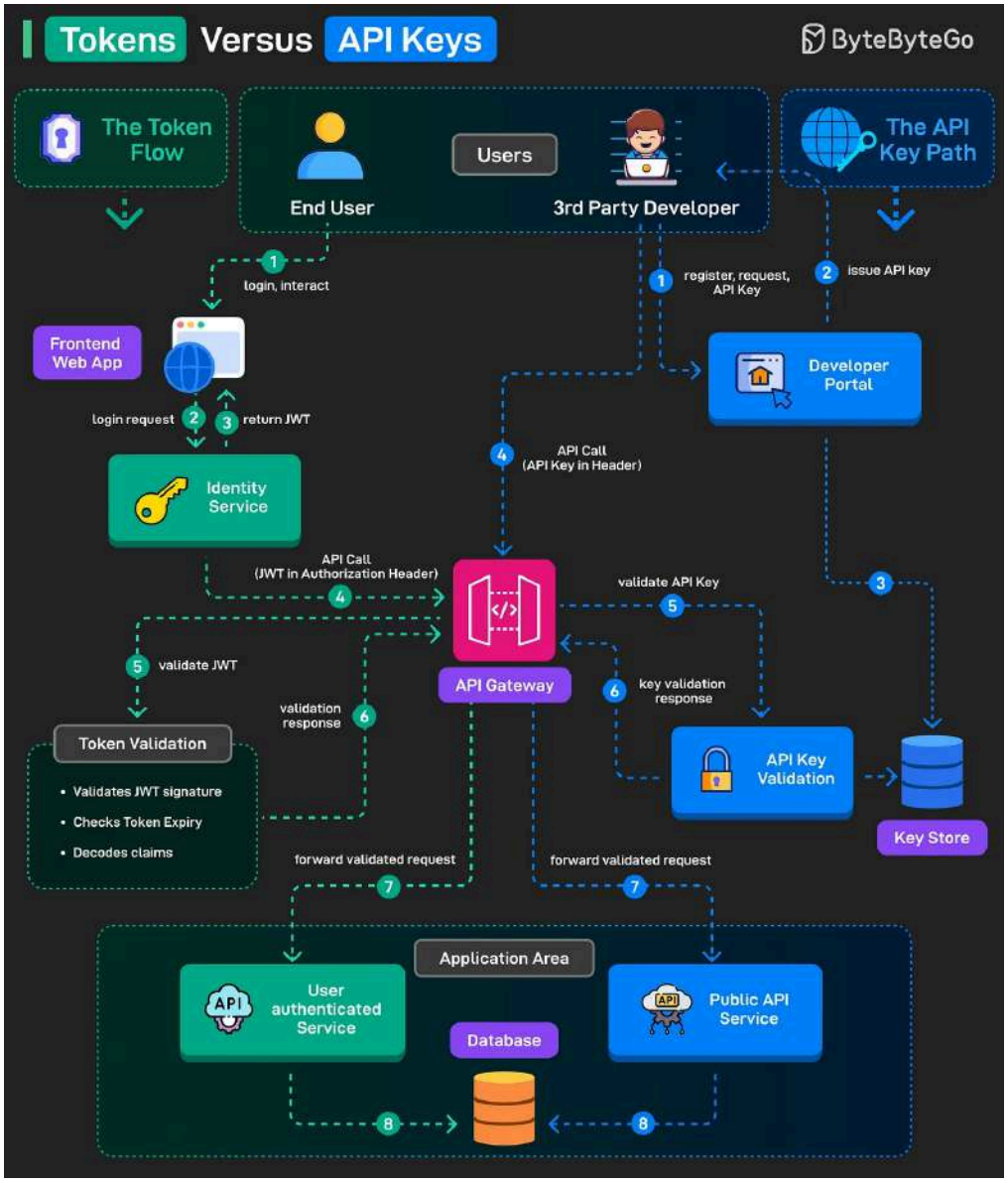
GraphQL

- Provides a single endpoint for clients to query for precisely the data they need.
- Clients specify the exact fields required in nested queries, and the server returns optimized payloads containing just those fields.
- Supports Mutations for modifying data and Subscriptions for real-time notifications.
- Great for aggregating data from multiple sources and works well with rapidly evolving frontend requirements.
- However, it shifts complexity to the client side and can allow abusive queries if not properly safeguarded
- Caching strategies can be more complicated than REST.

The best choice between REST and GraphQL depends on the specific requirements of the application and development team. GraphQL is a good fit for complex or frequently changing frontend needs, while REST suits applications where simple and consistent contracts are preferred.

Tokens vs API Keys

Both tokens (such as JWT) and API keys are used for authentication and authorization, but they serve different purposes. Let's understand the simplified flow for both.



The Token Flow

1. End user logs into the frontend web application.
2. Login credentials are sent to the Identity Service.
3. On successful authentication, a JWT token is issued and returned.
4. The frontend makes API calls with the JWT in the Authorization header.

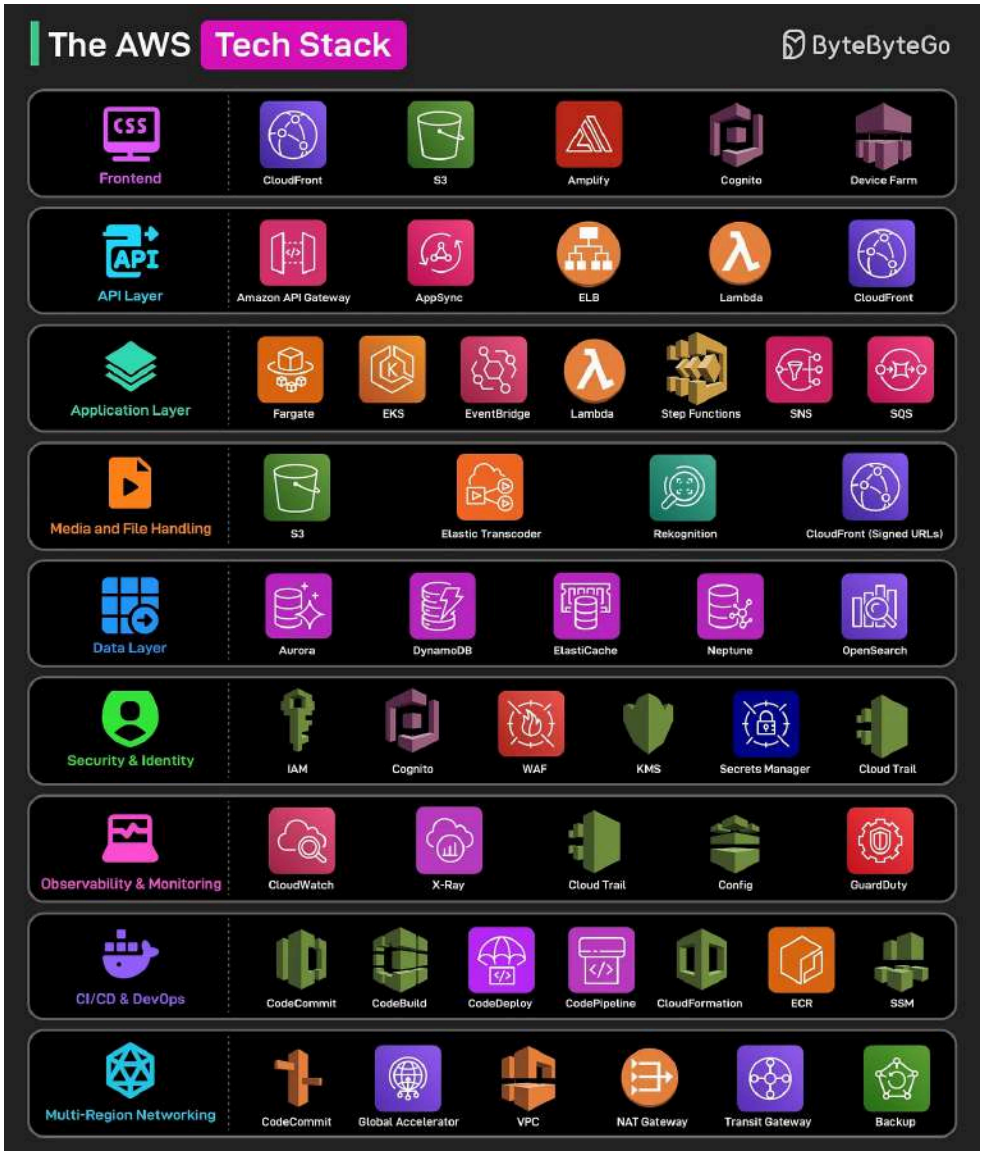
5. API Gateway intercepts the request and validates the JWT (signature, expiry, and claims).
6. If valid, the gateway sends a validation response.
7. The validated request is forwarded to the user-authenticated service.
8. The service processes the request and interacts with the database to return results.

The API Key Flow

1. A 3rd party developer registers on the Developer Portal.
2. The portal issues an API Key.
3. The key is also stored in a secure key store for later verification.
4. The developer app sends future API requests with the API Key in the header.
5. The API Gateway intercepts the request and sends the key to the API Key Validation service.
6. The validation service verifies the key from the key store and responds.
7. For valid API keys, the gateway forwards the request to the public API service.
8. The service processes it and accesses the database as needed.

Over to you: What else will you add to the explanation?

The AWS Tech Stack



1. Frontend
   Static websites are hosted on S3 and served globally via CloudFront for low latency. Other services that support the frontend development include Amplify, Cognito, and Device Farm.

2. API Layer
   API Gateway and AppSync expose REST and GraphQL APIs with built-in security and throttling. Other services that work in this area are Lambda, ELB, and CloudFront.

3. Application Layer
   This layer hosts business logic. Some services that are important in this layer are Fargate, EKS, Lambda, EventBridge, Step Functions, SNS, and SQS.

4. Media and File Handling
   Media is uploaded to S3, transcoded via Elastic Transcoder, and analyzed using Rekognition for moderation. CloudFront signed URLs ensure secure delivery of videos and files to authenticated users.

5. Data Layer
   The primary services for this layer are Aurora, DynamoDB, ElastiCache, Neptune, and OpenSearch.

6. Security and Identity
   Some AWS services that help in this layer of the stack are IAM, Cognito, WAF, KMS, Secrets Manager, and CloudTrail.

7. Observability and Monitoring
   CloudWatch monitors logs, metrics, and alarms. X-Ray provides tracing of request paths. CloudTrail captures API calls. Config ensures compliance, and GuardDuty detects security threats.

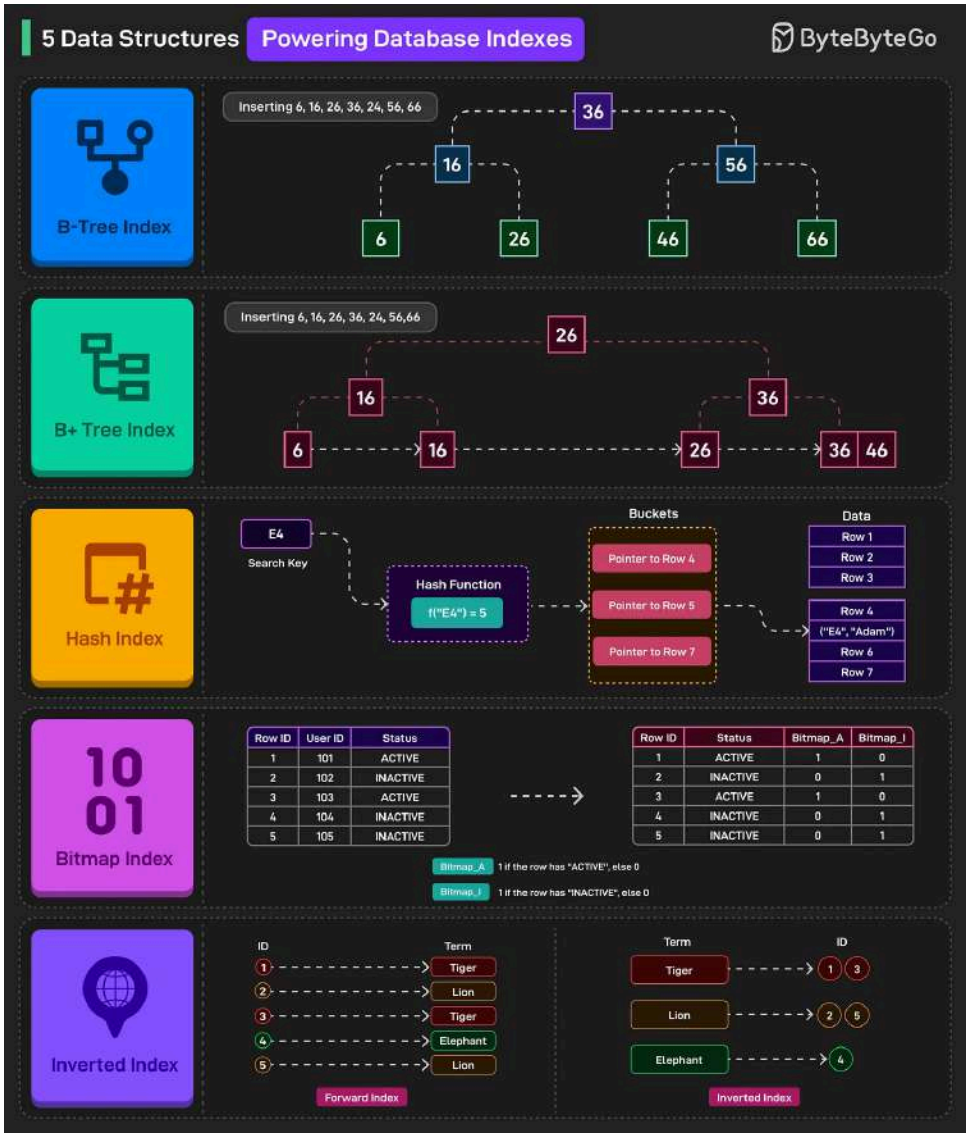
8. CI/CD and DevOps
   The key services used in this layer are CodeCommit, CodeBuild, CodeDeploy, CodePipeline, CloudFormation, ECR, and SSM.

9. Multi-Region Networking
   Route 53 and Global Accelerator ensure fast DNS and global routing. VPC segments the network while NAT and Transit Gateways handle secure traffic flow. AWS Backup provides disaster recovery across regions.

Over to you: Which other service will you add to the list?

5 Data Structures That Make DB Queries Super Fast

Data structures are crucial for database indexes because they determine how efficiently data can be searched, inserted, and retrieved, directly impacting query performance.



1. B-Tree Index
   B-Tree indexes use a balanced tree structure where keys and data pointers exist in internal and leaf nodes. They support efficient range and point queries through ordered traversal.

2. B+ Tree Index
   B+ Tree indexes store all data pointers in the leaf nodes, while internal nodes hold only keys to guide the search. Leaf nodes are linked for fast range queries via sequential access.

3. Hash Index
   Hash indexes apply a hash function to a search key to directly locate a bucket with pointers to data rows. They are optimized for equality searches but not for range queries.

4. Bitmap Index
   Bitmap indexes represent column values using bit arrays for each possible value, allowing fast filtering through bitwise operations. They’re ideal for low-cardinality categorical data.

5. Inverted Index
   Inverted indexes map each unique term to a list of row IDs containing that term, enabling fast full-text search.

12 MCP Servers You Can Use in 2025



MCP (Model Context Protocol) is an open standard that simplifies how AI models, particularly LLMs, interact with external data sources, tools, and services. An MCP server acts as a bridge between these AI models and external tools. Here are the top MCP servers:

1. File System MCP Server
   Allows the LLM to directly access the local file system to read, write, and create directories.

2. GitHub MCP Server
   Connects Claude to GitHub repos and allows file updates, code searching.

3. Slack MCP Server
   MCP Server for Slack API, enabling Claude to interact with Slack workspaces.

4. Google Maps MCP Server
   MCP Server for Google Maps API.

5. Docker MCP Server
   Integrate with Docker to manage containers, images, volumes, and networks.

6. Brave MCP Server
   Web and local search using Brave's Search API.

7. PostgreSQL MCP Server
   An MCP server that enables LLM to inspect database schemas and execute read-only queries.

8. Google Drive MCP Server
   An MCP server that integrates with Google Drive to allow reading and searching over files.

9. Redis MCP Server
   MCP Server that provides access to Redis databases.

10. Notion MCP Server
    This project implements an MCP server for the Notion API.

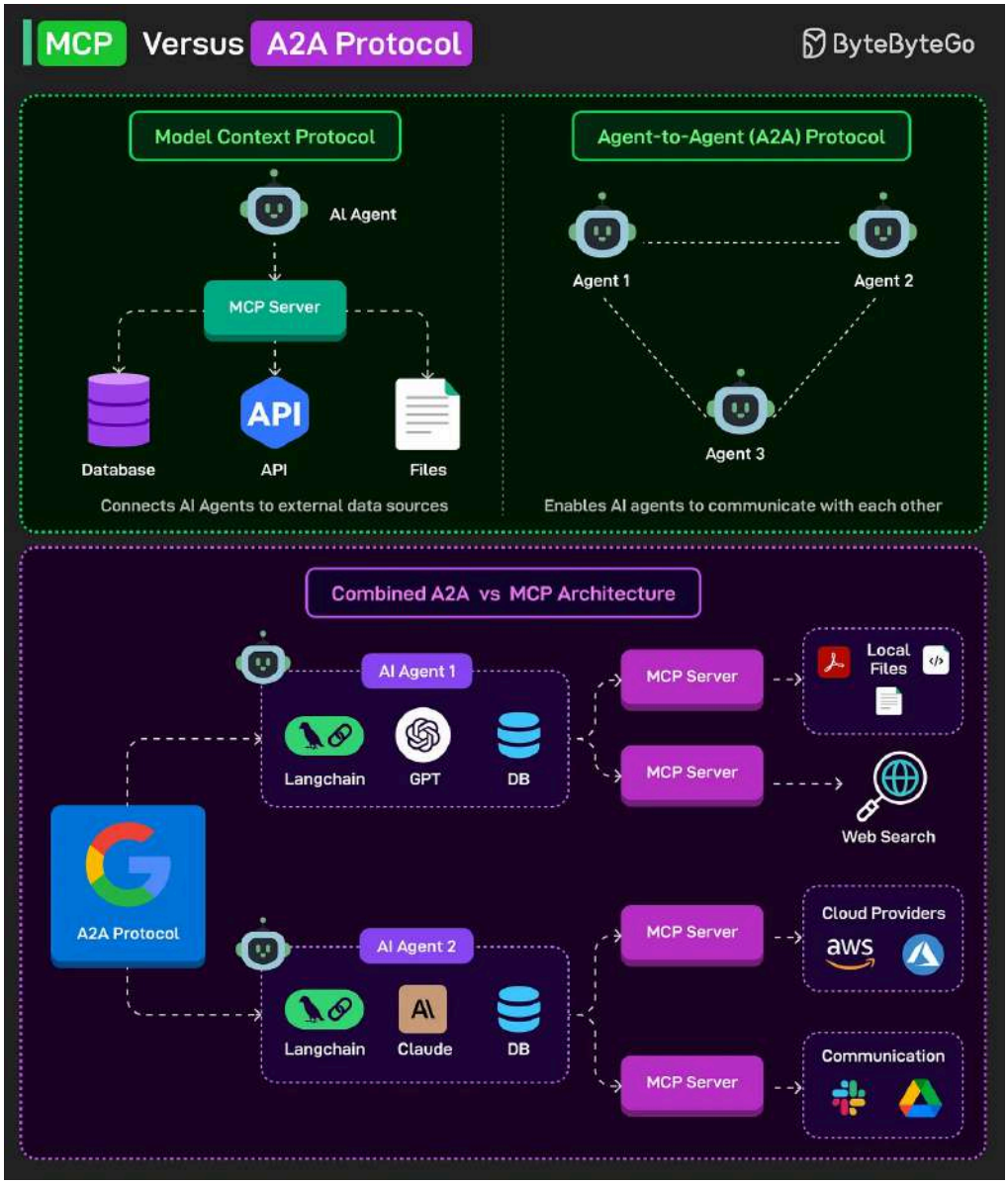
11. Stripe MCP Server
    MCP Server to interact with the Stripe API.

12. Perplexity MCP Server
    An MCP Server that connects to Perplexity's Sonar API for real-time search.

Over to you: Which other MCP Server will you add to the list?

MCP Versus A2A Protocol

The Model Context Protocol (MCP) connects AI agents to external data sources, such as databases, APIs, and files, via an MCP server, thereby enriching their responses with real-world context.



Google's Agent-to-Agent (A2A) Protocol enables AI agents to communicate and collaborate, allowing them to delegate tasks, share results, and enhance each other's capabilities.

MCP and A2A can be combined into a holistic architecture. Here's how it can work:

1. Each AI agent (using tools like Langchain with GPT or Claude) connects to external tools via MCP servers for data access.

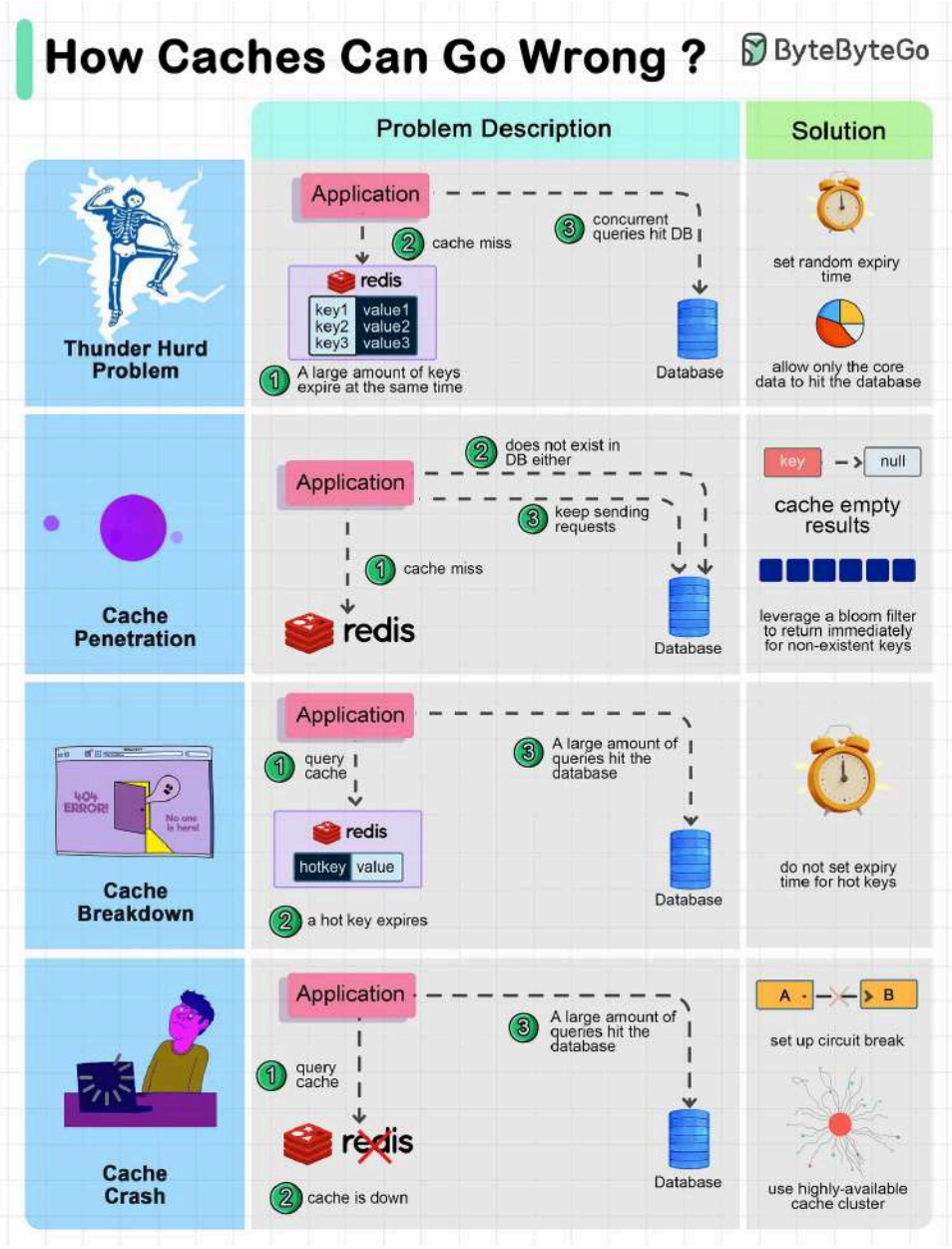
2. The external tools can comprise cloud APIs, local files, web search, or communication platforms like Slack.
3. Simultaneously, the AI agents can communicate with one another using the A2A protocol to coordinate actions, share intermediate outputs, and solve complex tasks collectively.

This architecture enables both rich external context (via MCP) and decentralized agent collaboration (via A2A).

Over to you: What else will you add to understand the MCP vs A2A Protocol capabilities?

How can Cache Systems go wrong?

The diagram below shows 4 typical cases where caches can go wrong and their solutions.



1. Thunder herd problem
   This happens when a large number of keys in the cache expire at the same time. Then the query requests directly hit the database, which overloads the database.

   There are two ways to mitigate this issue: one is to avoid setting the same expiry time for the keys, adding a random number in the configuration; the other is to allow only the core

business data to hit the database and prevent non-core data to access the database until the cache is backed up.

2. Cache penetration
   This happens when the key doesn't exist in the cache or the database. The application cannot retrieve relevant data from the database to update the cache. This problem creates a lot of pressure on both the cache and the database.

   To solve this, there are two suggestions. One is to cache a null value for non-existent keys, avoiding hitting the database. The other is to use a bloom filter to check the key existence first, and if the key doesn't exist, we can avoid hitting the database.

3. Cache breakdown
   This is similar to the thunder herd problem. It happens when a hot key expires. A large number of requests hit the database.

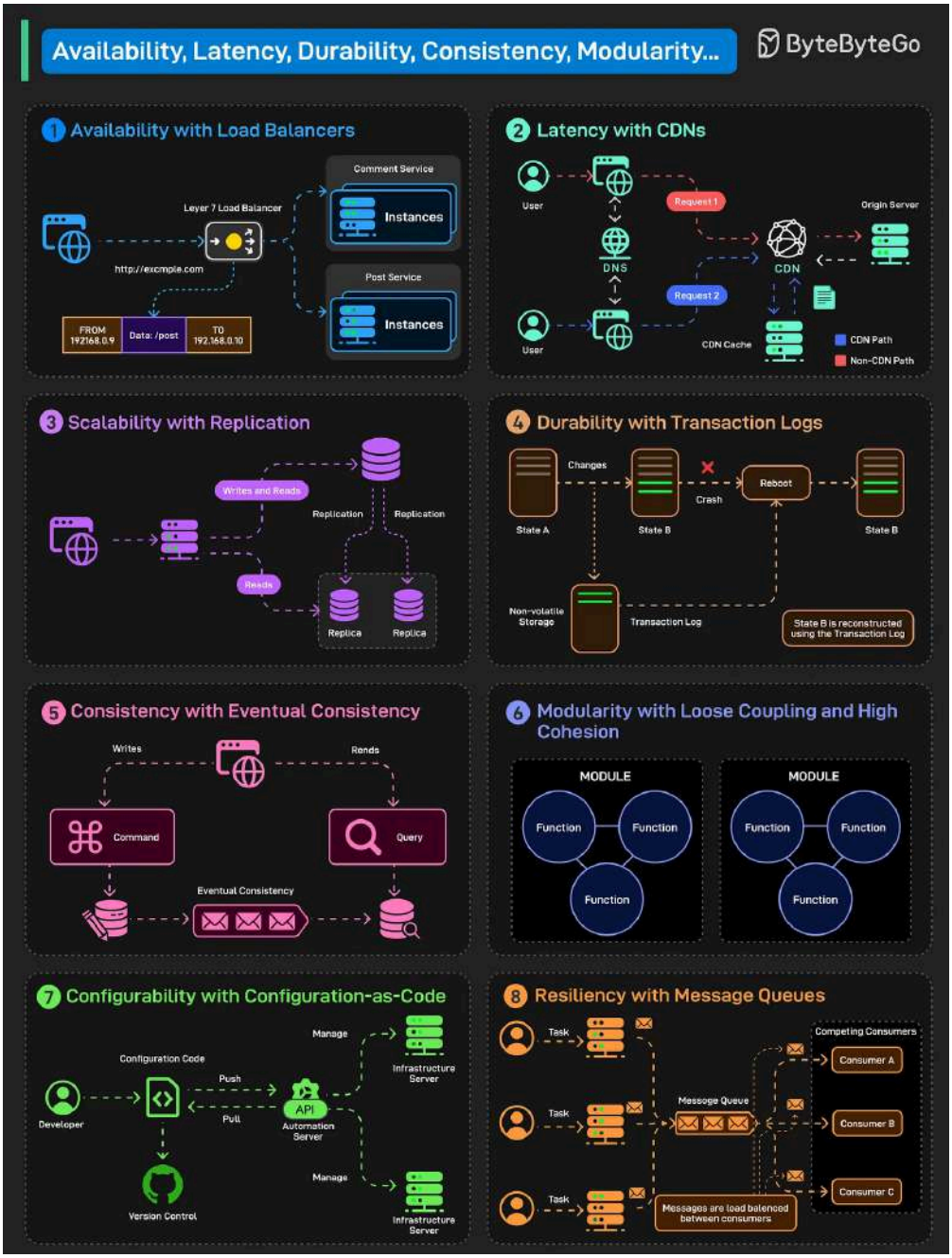
   Since the hot keys take up 80% of the queries, we do not set an expiration time for them.

4. Cache crash
   This happens when the cache is down and all the requests go to the database.

   There are two ways to solve this problem. One is to set up a circuit breaker, and when the cache is down, the application services cannot visit the cache or the database. The other is to set up a cluster for the cache to improve cache availability.

Over to you: Have you met any of these issues in production?

8 System Design Concepts Explained in 1 Diagram



Non-functional requirements define the quality attributes of a system that ensure it performs reliably under real-world conditions. Some key NFRs, along with the implementation approach, are as follows:

1. Availability with Load Balancers
   Availability ensures that a system remains operational and accessible to users at all times.

Using load balancers distributes traffic across multiple service instances to eliminate single points of failure

2. Latency with CDNs
   Latency refers to the time delay experienced in a system's response to a user request. CDNs reduce latency by caching content closer to users.

3. Scalability with Replication
   Scalability is the system's ability to handle increased load by adding resources. Replication distributes data across multiple nodes, enabling higher throughput and workload.

4. Durability with Transaction Log
   Durability guarantees that once data is committed, it remains safe even in the event of failure. Transaction logs persist all operations, allowing the system to reconstruct the state after a crash.

5. Consistency with Eventual Consistency
   Consistency means that all users see the same data. Eventual consistency allows temporary differences, but synchronizes replicas over time to a consistent state.

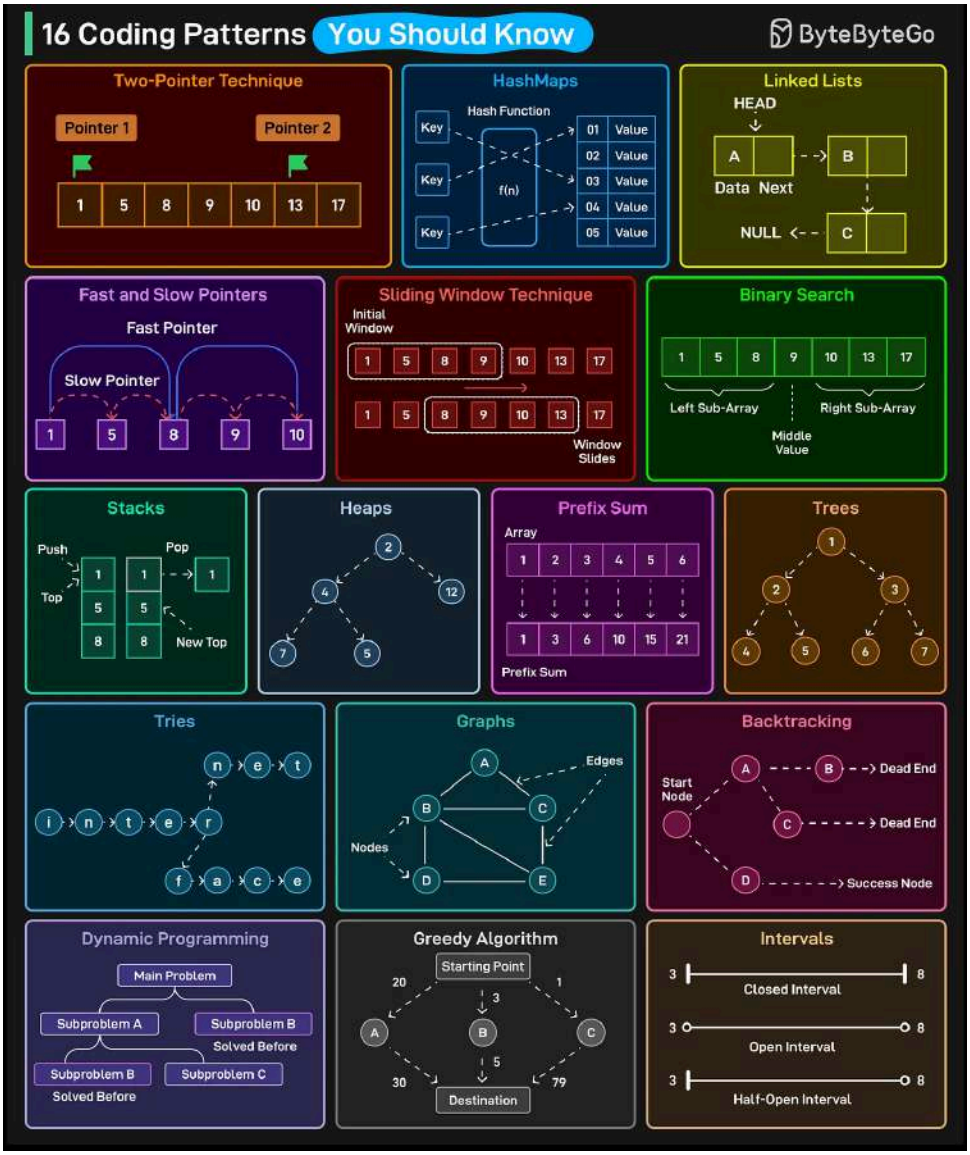
6. Modularity with Loose Coupling and High Cohesion
   Modularity promotes building systems with well-separated and self-contained components. Loose coupling and high cohesion help achieve the same.

7. Configurability
   Configurability allows a system to be easily adjusted or modified without altering core logic. Configuration-as-Code manages infra and app settings via version-controlled scripts.

8. Resiliency with Message Queues
   Resiliency is a system's ability to recover from failures and continue operating smoothly. Message queues decouple components and buffer tasks, enabling retries.

Over to you: Which other NFR or strategy will you add to the list?

16 Coding Patterns That Make Interviews Easy

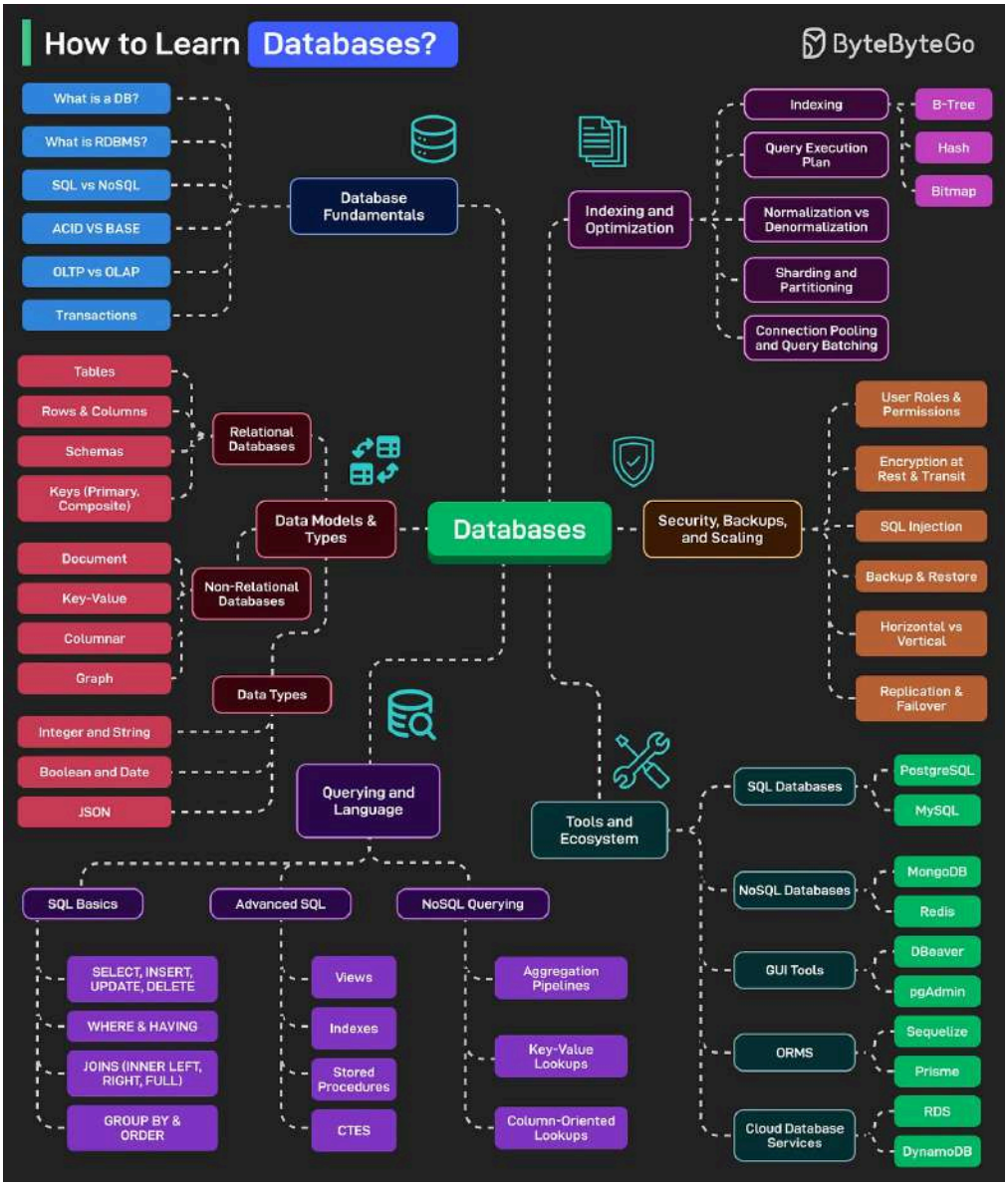


1. Two-Pointer Technique
2. HashMaps
3. Linked Lists
4. Fast and Slow Pointers
5. Sliding Window Technique
6. Binary Search
7. Stacks

8. Heaps
9. Prefix Sum
10. Trees
11. Tries
12. Graphs
13. Backtracking
14. Dynamic Programming
15. Greedy Algorithms
16. Intervals

Over to you: Which other coding pattern will you add to the list?

How to Learn Databases?



Databases power everything from websites and apps to enterprise systems. Here's a learning map that can help you master databases:

1. Database Fundamentals
   This includes topics like "What is a database", RDBMS, SQL vs NoSQL, ACID vs BASE, OLTP vs OLAP, Transactions, and Isolation Levels.

2. Data Models and Types
   Consists of topics like Relational Databases, Non-Relational Databases, and Data Types (Integer, String, Boolean, Date, JSON, etc).

3. Querying and Language
   This includes topics like SQL Basics (SELECT, INSERT, etc), Advanced SQL (Views, Indexes, CTEs, etc), and NoSQL Querying (Aggregation and Key-Value Lookups).

4. Indexing and Optimization
   Consists of topics like Indexing (B-Tree, Hash, and Bitmaps), Query Execution Plans, Denormalization vs Normalization, Sharding, Connecting Pooling, and Query Batching.

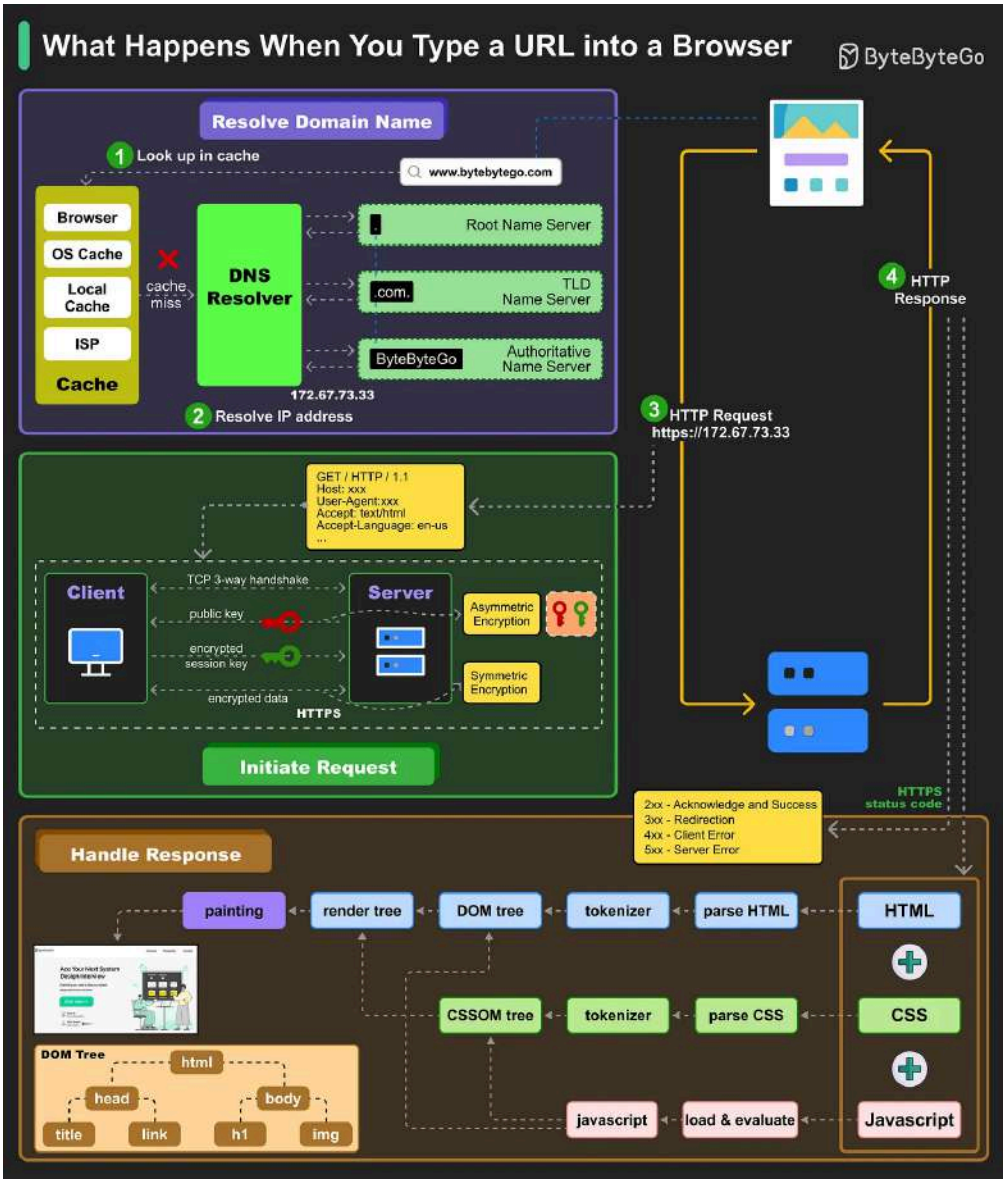
5. Security, Backups, and Scaling
   This includes topics like User Roles, Permissions, Encryption, SQL Injection, High Availability (Replication and Failover), Horizontal vs Vertical Scaling.

6. Tools and Ecosystem
   Consists of topics like Popular SQL Databases, NoSQL Database, GUI Tools, ORMs, Cloud DB services (RDS, DynamoDB, Google Cloud SQL, etc.)

Over to you: What else will you add to the list for learning databases?

What happens when you type a URL into a browser?

Let’s look at the process step by step.



Step 1: The user enters a URL (bytebytego .com) into the browser and hits Enter. The first thing we need to do is to translate the URL to an IP address. The mapping is usually stored in a cache, so the browser looks for the IP address in multiple layers of cache: the browser cache, OS cache, local cache, and ISP cache. If the browser couldn’t find the mapping in the cache, it will ask the DNS (Domain Name System) resolver to resolve it.

Step 2: If the IP address cannot be found at any of the caches, the browser goes to DNS servers to

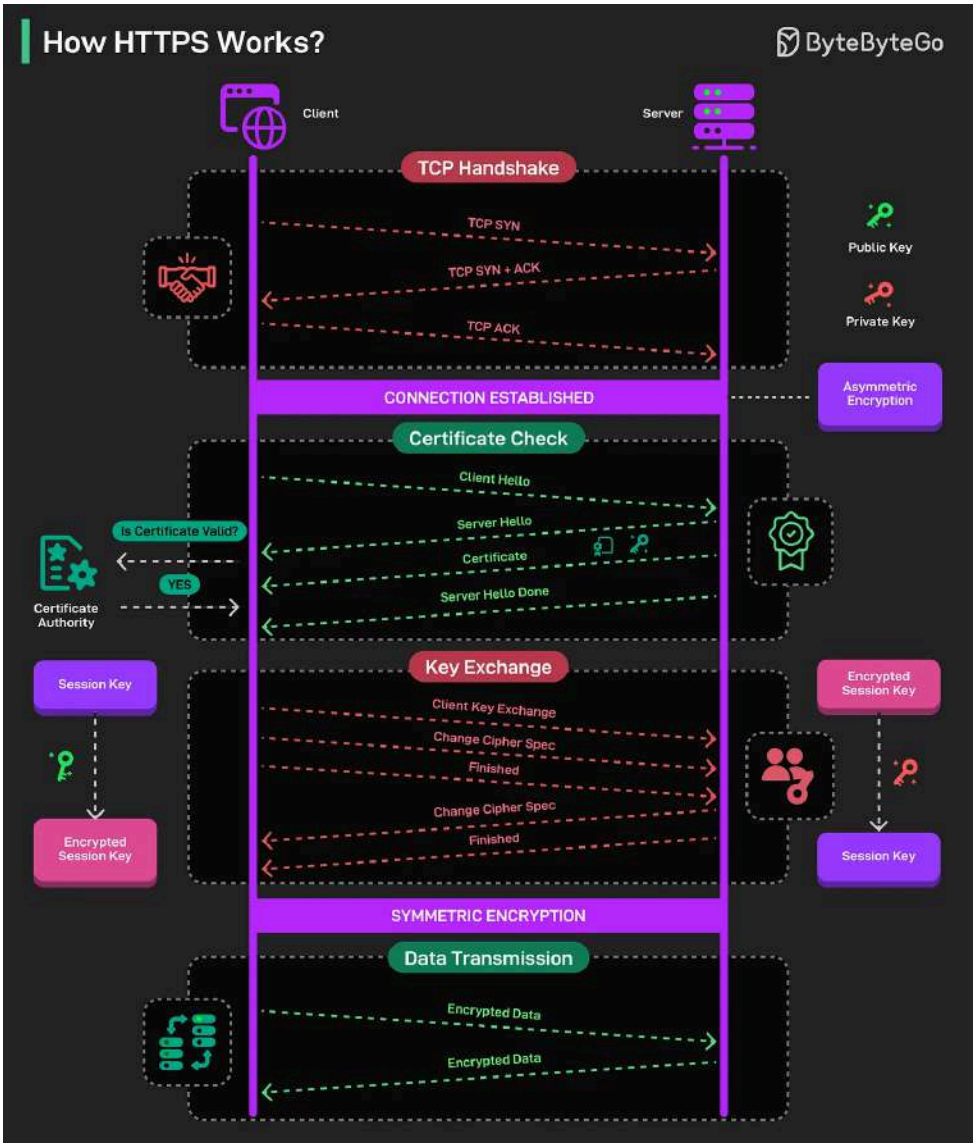
do a recursive DNS lookup until the IP address is found.

Step 3: Now that we have the IP address of the server, the browser sends an HTTP request to the server. For secure access of server resources, we should always use HTTPS. It first establishes a TCP connection with the server via TCP 3-way handshake. Then it sends the public key to the client. The client uses the public key to encrypt the session key and sends to the server. The server uses the private key to decrypt the session key. The client and server can now exchange encrypted data using the session key.

Step 4: The server processes the request and sends back the response. For a successful response, the status code is 200. There are 3 parts in the response: HTML, CSS and Javascript. The browser parses HTML and generates DOM tree. It also parses CSS and generates CSSOM tree. It then combines DOM tree and CSSOM tree to render tree. The browser renders the content and display to the user.

How HTTPS Works?

HTTPS ensures secure communication between your browser (client) and a website's server using encryption.



1. TCP Handshake: Before any secure communication occurs, the client and server first establish a basic connection using the TCP handshake process. At this point, the data is not encrypted.

2. Certificate Check: This stage is meant to verify the server's identity. The client starts the TLS handshake with a "hello" message that offers supported encryption algorithms. The server replies, choosing algorithms and sending its digital certificate, which contains the public key. The client verifies the certificate to ensure it's from a trusted Certificate Authority (CA).

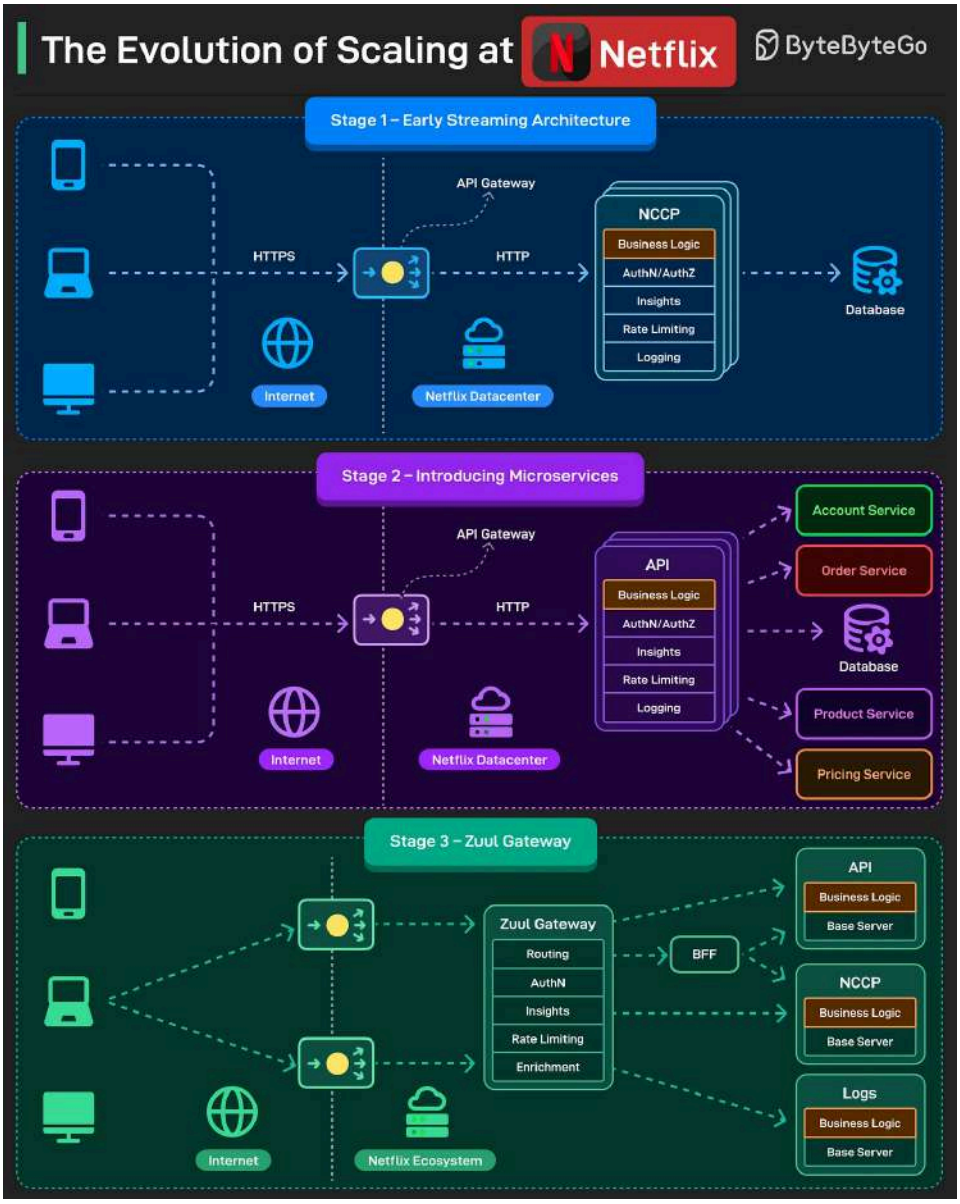
3. Key Exchange: Once the client validates the certificate, it starts the key exchange process. The client uses the server's public key (from the certificate) to encrypt the session key. The encrypted session key is sent to the server, and the server can decrypt it using its private key. The change cipher spec message signifies that from this point onward, all messages will be encrypted using the agreed-upon session key and cipher. This step uses asymmetric encryption to securely exchange the session key.

4. Data Transmission: Now, both sides switch to symmetric encryption (faster) using the shared session key. Messages are encrypted and decrypted with the same key. All data exchanged is now secure and private.

Over to you: Will you add anything more to explain the HTTPS process?

The Evolution of Scaling at Netflix

Over the years, Netflix went through several growth stages. With each stage, they had to evolve the scaling approach.



1. Stage 1 (Early Streaming Architecture)
   It was a typical 3-tier consisting of a client, an API, and a database.

   The API application was named NCCP (Netflix Content Control Protocol). It was a monolithic application that contained all functionalities.

2. Stage 2 (Introducing Microservices)
   As more features were added, maintaining the engineering velocity became critical. To

support this, the monolithic application was broken up into microservices.

Features were extracted from the NCCP application and developed as separate applications, but the orchestration logic remained in the NCCP application.

3. Stage 3 (Zuul Gateway)
   Next, Netflix split the application. The NCCP stayed there for the playback experience while other APIs started to handle other features.

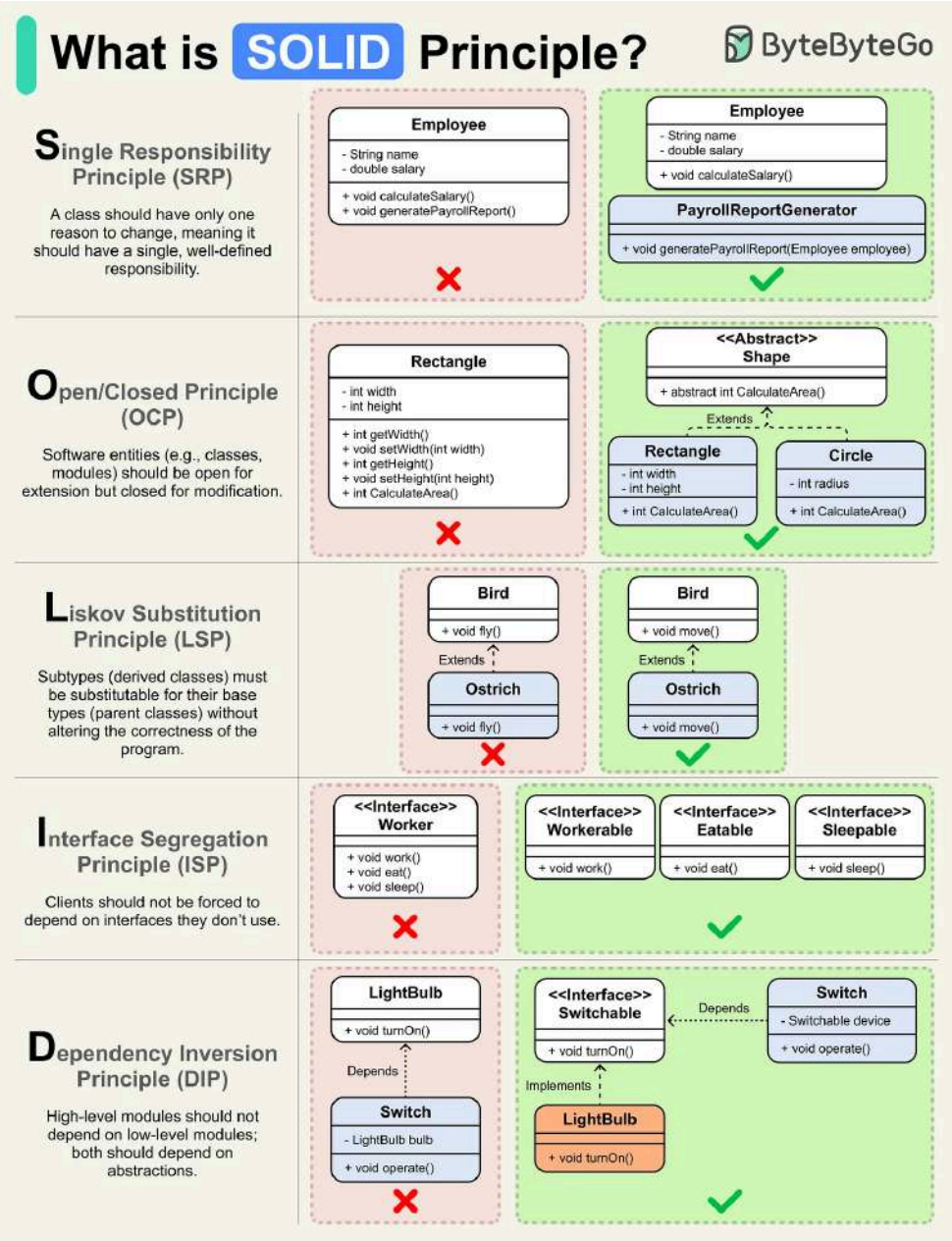
   A Zuul API Gateway was introduced to reduce the coupling between the client and the services.

Reference: Netflix TechBlog

Over to you: Have you come across any other nuance of how Netflix scaled to its current level?

What is the SOLID Principle?

SOLID offers guidelines to create software that is easy to understand, modify, and extend.



The SOLID acronym stands for:

S -> Single Responsibility Principle
O -> Open/Closed Principle
L -> Liskov Substitution Principle
I -> Interface Segregation Principle
D -> Dependency Inversion Principle

- The Single Responsibility Principle (SRP) states that a class should have only one reason to change. In other words, it should have a single, well-defined responsibility or task within a software system.
- The Open/Closed Principle (OCP) states that software entities, such as classes, should be open for extension but closed for modification. This means you can add new functionality without altering existing code.
- The Liskov Substitution Principle (LSP) states that objects of a derived class should be able to replace objects of the base class without affecting the correctness of the program.
- The Interface Segregation Principle (ISP) emphasizes that classes or components that use interfaces should not be forced to depend on interfaces they don't use.
- The Dependency Inversion Principle (DIP) states that high-level modules (or classes) should not depend on low-level modules; both should depend on abstractions, such as interfaces.

Over to you: Which SOLID principle do you find most useful or challenging in your daily development work?

Common HTTP Status Codes

HTTP status codes are issued by a server in response to a client's request. These concise responses from servers are super useful when debugging our web apps, mobile apps, or APIs.



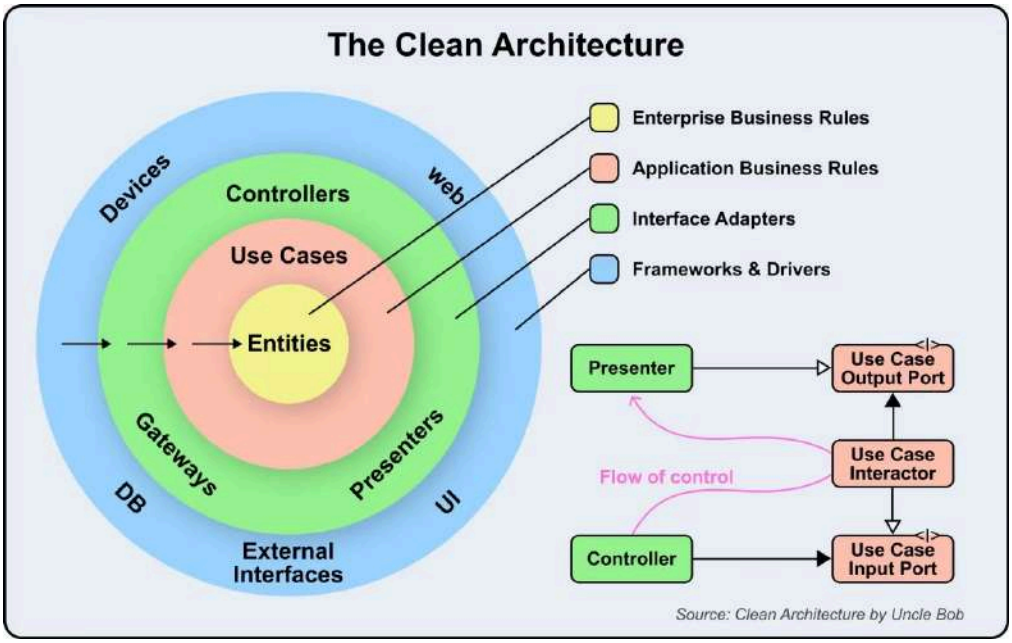
- 1xx Informational: Signals client's request received, and processing continues (e.g., 100 Continue).
- 2xx Success: Confirms request was successfully completed (e.g., 200 OK).
- 3xx Redirection: Requires client action, like following redirects (e.g., 301 Moved Permanently).

- 4xx Client Error: Indicates client-caused request failures (e.g., 404 Not Found).
- 5xx Server Error: Shows server failures in processing requests (e.g., 500 Internal Server Error).

Over to you: Which category confuses you most, and why?

How Clean Architecture Works?

Clean Architecture is a way of organizing software applications so that they are easy to understand, maintain, and scale. Clean Architecture has 4 layers, each with a clear role. The dependencies between these layers always point inward, ensuring a loosely coupled, testable, and maintainable system.

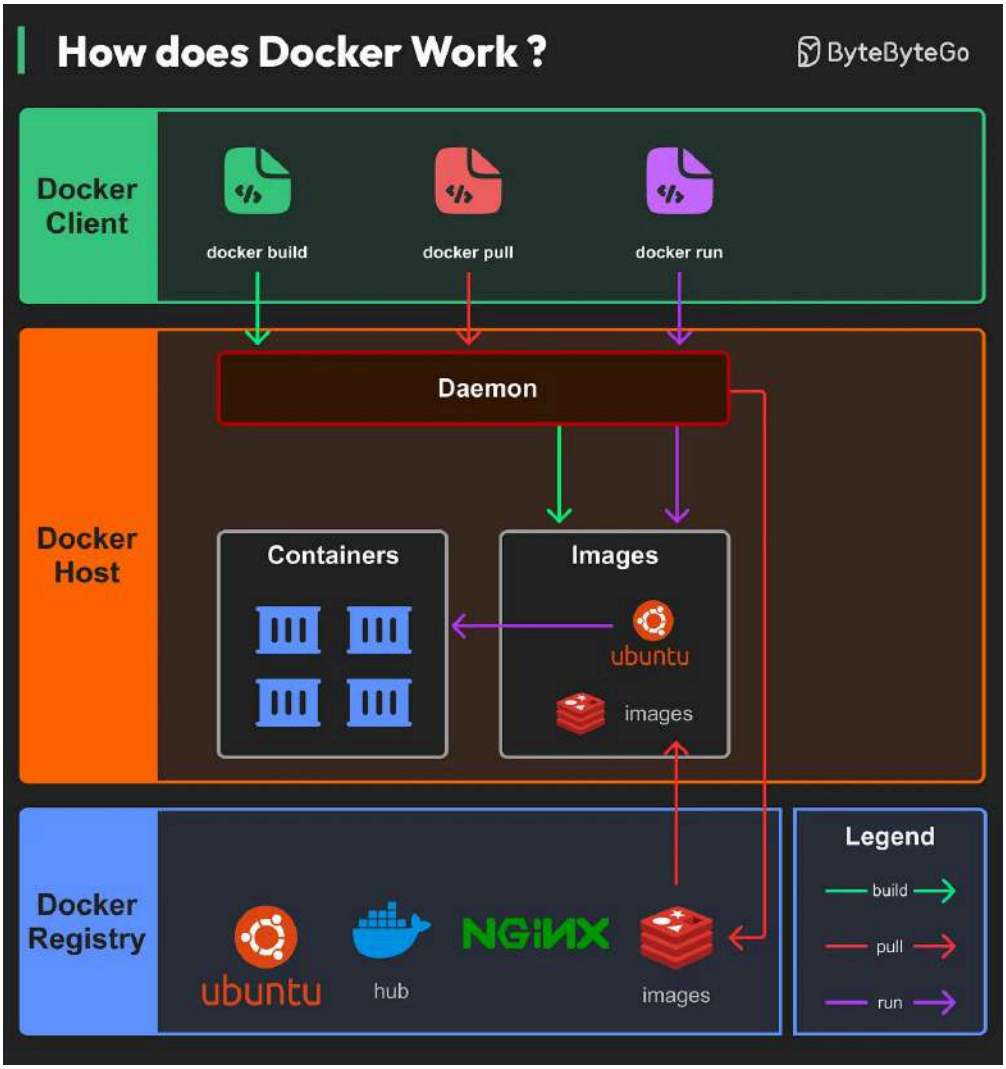


Let's look at the layers:

1. Entities: These are the core rules and logic of the business. They represent the most stable part of the system and are independent of any external systems or frameworks.
2. Use Cases: These define the specific operations your application can perform. They orchestrate the flow of data between entities and external interfaces but don't care about how the data is displayed or stored.
3. Interface Adapters: This layer includes Controllers, Presenters, and Gateways. It converts data from the outer layers (like the web, database, or UI) into a form that the use cases and entities can understand.
4. Frameworks and Drivers: These are external tools and systems like the database, web UI, or devices. They sit on the outside and depend on the inner layers.

Over to you: What else will you add to understand the working of Clean Architecture?

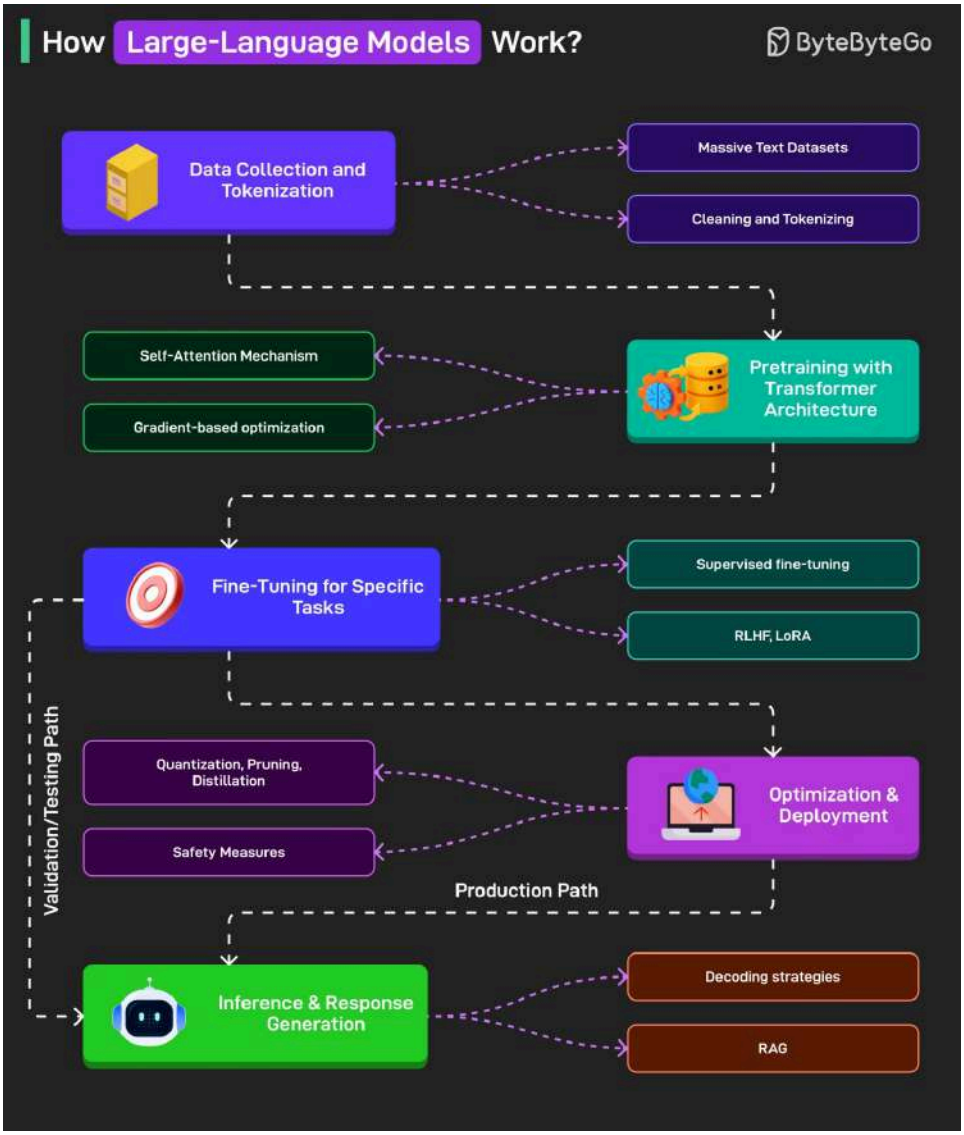
How does Docker Work?



Docker's architecture comprises three main components:

- Docker Client
  This is the interface through which users interact. It communicates with the Docker daemon.
- Docker Host
  Here, the Docker daemon listens for Docker API requests and manages various Docker objects, including images, containers, networks, and volumes.
- Docker Registry
  This is where Docker images are stored. Docker Hub, for instance, is a widely-used public registry.

How Large Language Models Work?



1. Data Collection and Tokenization.
   LLMs begin by ingesting massive volumes of text from sources like books, websites, and code. This text is cleaned and broken down into smaller units (tokens) to prepare it for training.

2. Pre-training with Transformer Architecture
   Using the transformer model and self-attention mechanisms, the system learns to predict the next token in a sequence. This process adjusts billions of parameters to develop a broad understanding of language.

3. Fine-Tuning for Specific Tasks
   The pretrained model is then tailored for specific applications through fine-tuning techniques,

aligning its behavior with human preferences or specialized tasks. Some methods are RLHF, LoRA, and so on.

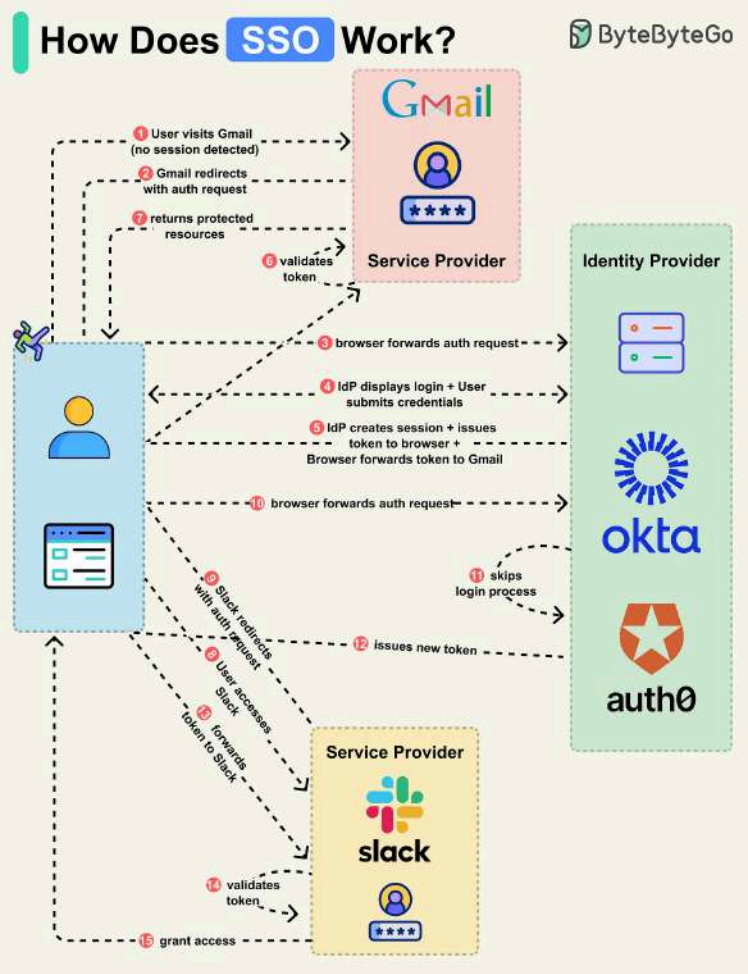
4. Inference and Response Generation
   The model generates responses based on input prompts at inference time by predicting the next most likely tokens. Advanced strategies and retrieval mechanisms can improve fluency and factual accuracy.

5. Optimization and Deployment
   Before deployment, the model is compressed and filtered to ensure speed, efficiency, and safety. It's then deployed in environments like cloud platforms and edge devices for real-world use.

Over to you: What else will you add to this high level flow?

How Does SSO Work?

Single Sign-On (SSO) is an authentication scheme. It allows a user to log in to different systems using a single ID.



Let’s walk through a typical SSO login flow:

Step 1: A user accesses a protected resource on an application like Gmail, which is a Service Provider (SP).

Step 2: The Gmail server detects that the user is not logged in and redirects the browser to the company’s Identity Provider (IdP) with an authentication request.

Step 3: The browser sends the user to the IdP.

Step 4: The IdP shows the login page where the user enters their login credentials.

Step 5: The IdP creates a secure token and returns it to the browser. The IdP also creates a session for future access. The browser forwards the token to Gmail.

Step 6: Gmail validates the token to ensure it comes from the IdP.

Step 7: Gmail returns the protected resource to the browser based on what the user is allowed to access.

This completes the basic SSO login flow. Let's see what happens when the user navigates to another SSO-integrated application, like Slack.

Step 8-9: The user accesses Slack, and the Slack server detects that the user is not logged in. It redirects the browser to the IdP with a new authentication request.

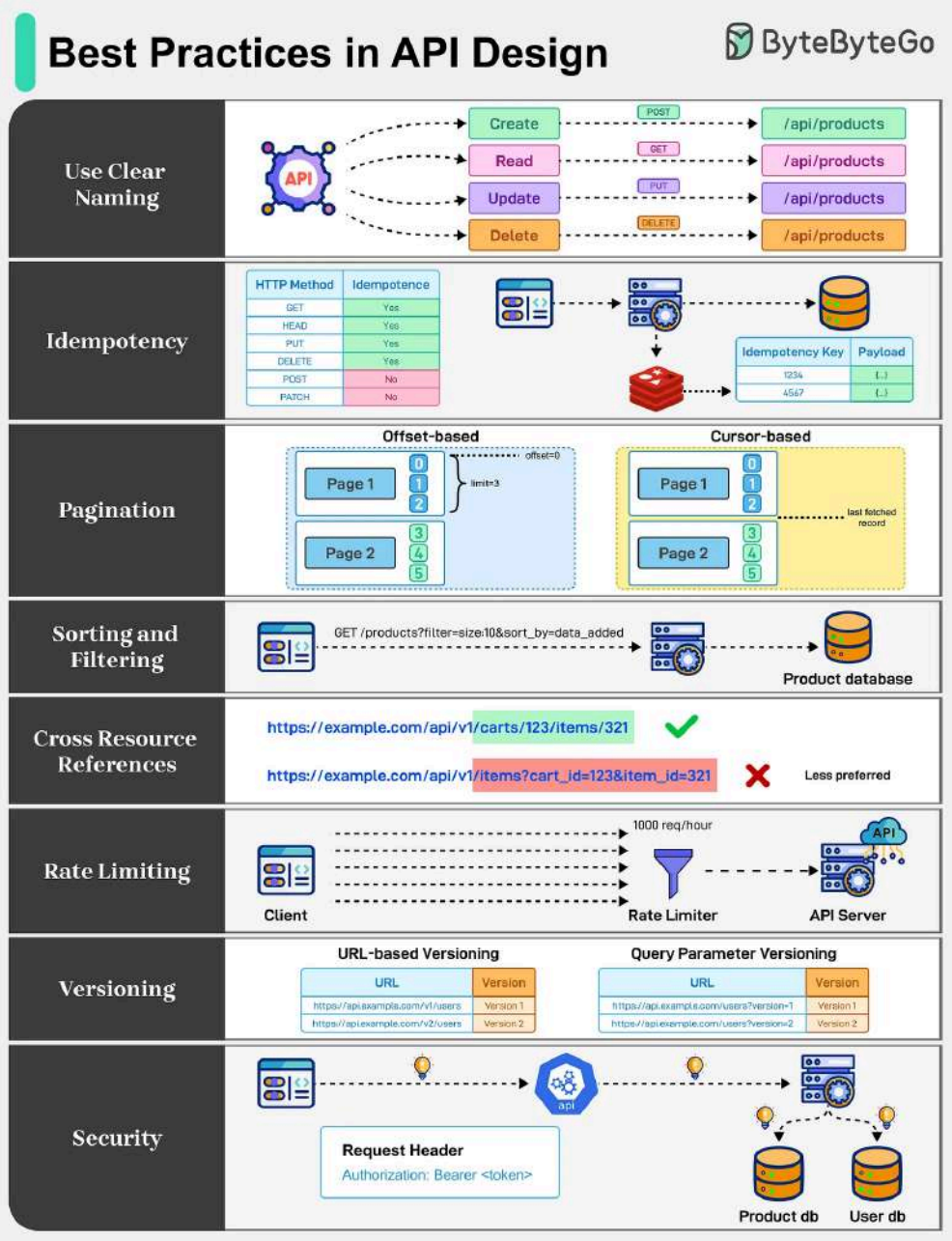
Step 10: The browser sends the user back to the IdP.

Step 11-13: Since the user has already logged in with the IdP, it skips the login process and instead creates a new token for Slack. The new token is sent to the browser, which forwards it to Slack.

Step 14-15: Slack validates the token and grants the user access accordingly.

Over to you: Would you like to see an example flow for another application?

Best Practices in API Design



APIs are the backbone of communication over the Internet. Well-designed APIs behave consistently, are predictable, and grow without friction. Some best practices to keep in mind are as follows:

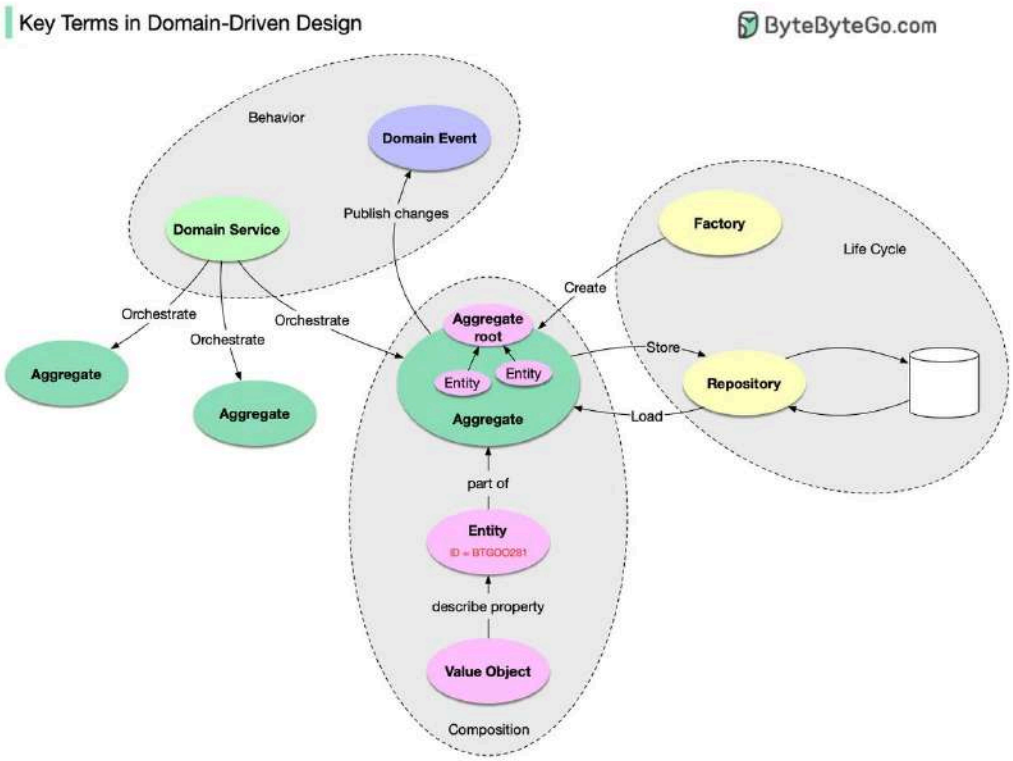
1. Use Clear Naming: When building an API, choose straightforward and logical names. Be consistent and stick with intuitive URLs that denote collections.
2. Idempotency: APIs should be idempotent. They ensure safe retries by making repeated requests to produce the same result, especially for POST operations.

3. Pagination: APIs should support pagination to prevent performance bottlenecks and payload bloat. Some common pagination strategies are offset-based and cursor-based.
4. Sorting and Filtering: Query strings are an effective way to allow sorting and filtering of API responses. This makes it easy for developers to see what filters and sort orders are applied.
5. Cross Resource References: Use clear linking between connected resources. Avoid excessively long query strings that make the API harder to understand.
6. Rate Limiting: Rate limiting is used to control the number of requests a user can make to an API within a certain timeframe. This is crucial for maintaining the reliability and availability of the API.
7. Versioning: When modifying API endpoints, proper versioning to support backward compatibility is important.
8. Security: API security is mandatory for well-designed APIs. Use proper authentication and authorization with APIs using API Keys, JWTs, OAuth2, and other mechanisms.

Over to you: did we miss anything important?

Key Terms in Domain-Driven Design

Have you heard of Domain-Driven Design (DDD), a major software design approach?



DDD was introduced in Eric Evans' classic book "Domain-Driven Design: Tackling Complexity in the Heart of Software". It explained a methodology to model a complex business. In this book, there is a lot of content, so I'll summarize the basics.

The composition of domain objects:

- Entity: a domain object that has ID and life cycle.
- Value Object: a domain object without ID. It is used to describe the property of Entity.
- Aggregate: a collection of Entities that are bounded together by Aggregate Root (which is also an entity). It is the unit of storage.

The life cycle of domain objects:

- Repository: storing and loading the Aggregate.
- Factory: handling the creation of the Aggregate.

Behavior of domain objects:

- Domain Service: orchestrate multiple Aggregate.

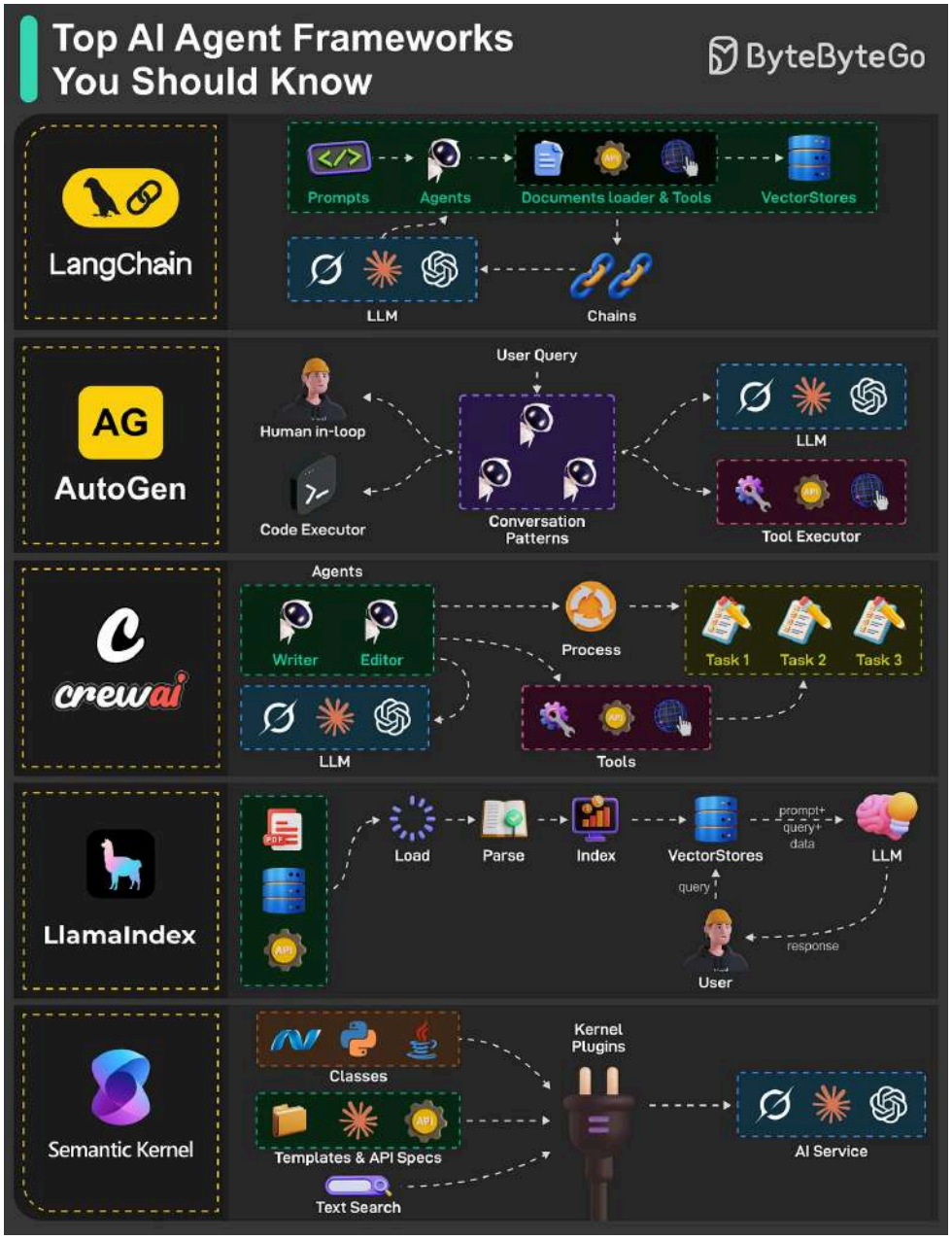
- Domain Event: a description of what has happened to the Aggregate. The publication is made public so others can consume and reconstruct it.

Congratulations on getting this far. Now you know the basics of DDD. If you want to learn more, I highly recommend the book. It might help to simplify the complexity of software modeling.

Over to you: do you know how to check the equality of two Value Objects? How about two Entities?

Top AI Agent Frameworks You Should Know

Building smart, independent AI systems is easier with agent frameworks that combine large language models (LLMs) with tools like APIs, web access, or code execution.



- LangChain
  LangChain makes it simple to connect LLMs with external tools like APIs and vector

databases. It allows developers to create chains for sequential task execution and document loaders for context-aware responses.

- AutoGen
  AutoGen allows you to develop AI agents that can chat with each other or involve humans in the loop. It is like a collaborative workspace where agents can run code, pull in data from tools, or get human feedback to complete a task.

- CrewAI
  As the name suggests, CrewAI is all about teamwork. It orchestrates teams of AI agents with roles like writers and editors, processing tasks in a structured workflow. It utilizes LLMs and tools (APIs, Internet, code, etc) to efficiently manage complex task execution and data flow.

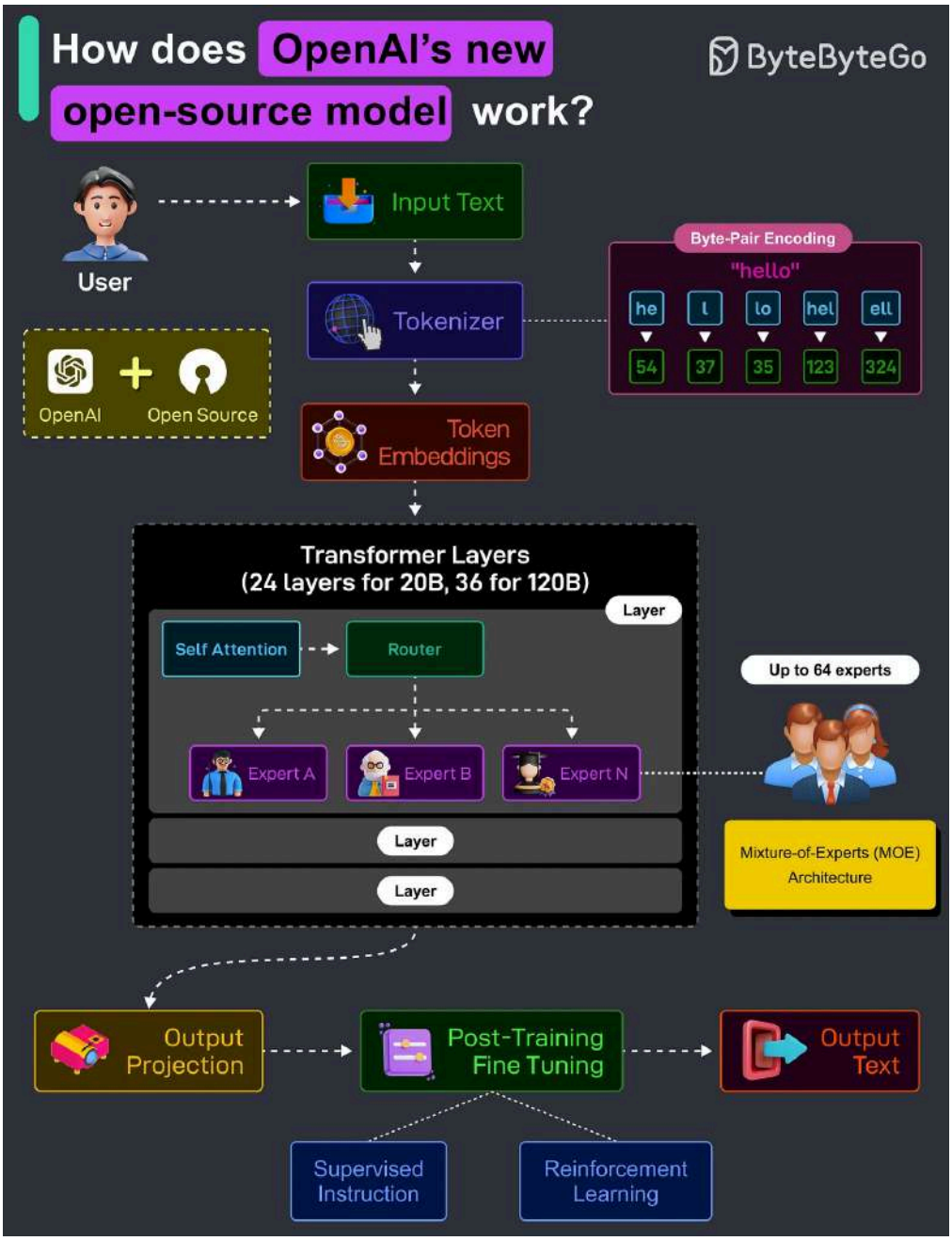
- LlamaIndex
  This framework indexes and queries data from documents, APIs, and vector stores to enhance agent responses. It parses and loads data, enabling LLMs to provide context-aware answers, making it ideal for enterprise document search systems and intelligent assistants that access private knowledge bases.

- Semantic Kernel
  Semantic Kernel connects AI services (OpenAI, Calude, Hugging Face models, etc) with a plugin-based architecture that supports skills, templates, and API integrations for flexible workflows. It supports text search and custom workflows for applications.

Over to you: Which AI agent framework have you explored or plan to use?

How OpenAI’s GPT-OSS 120B and 20B Models Work?

OpenAI has recently released two LLMs, GPT-OSS-120B (120 billion parameters) and GPT-OSS-20B (20 billion parameters). These are fully open-source models and are provided under an Apache 2.0 license.

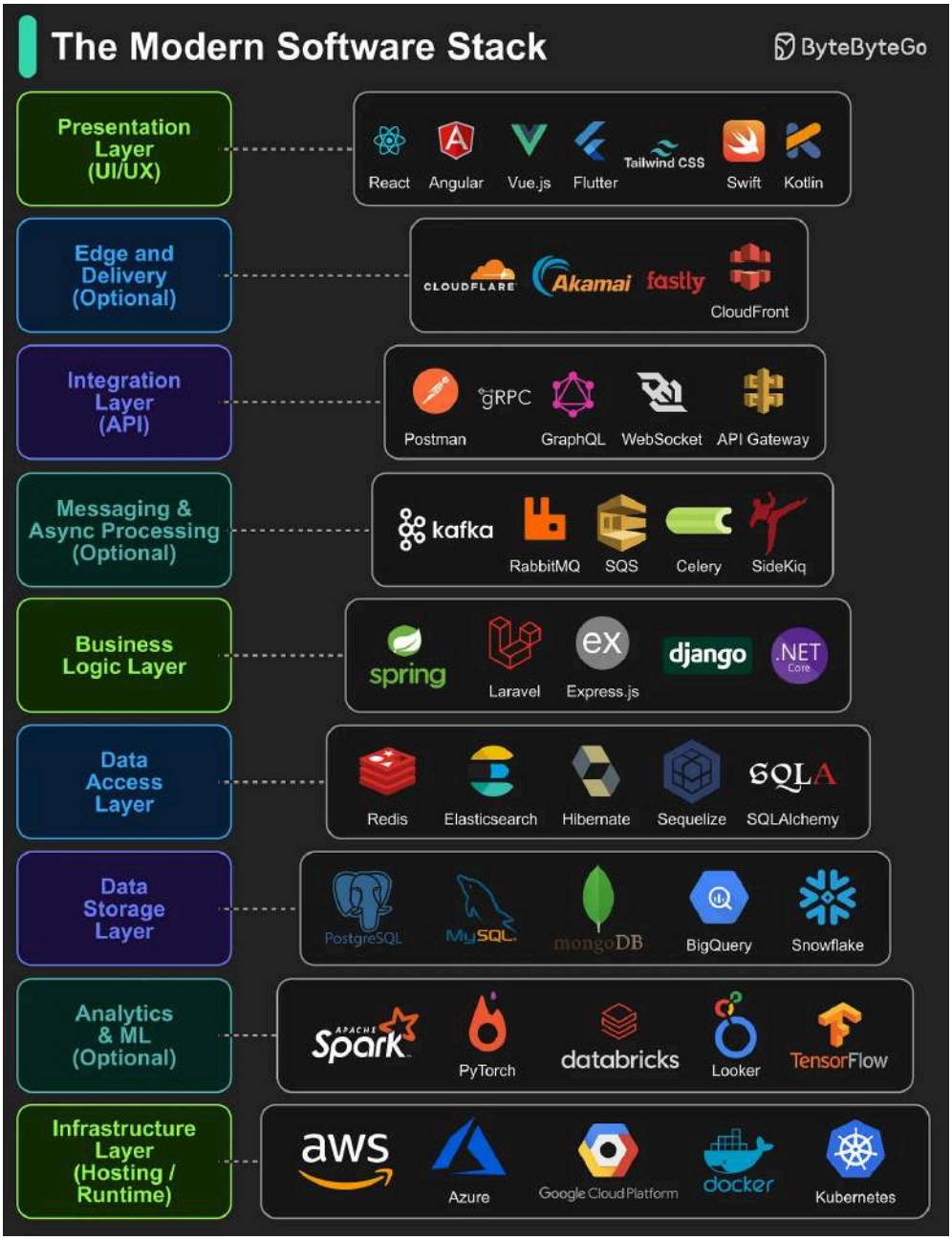


These models aim to deliver strong real-world performance at low cost. Here’s how they work:

1. The user provides some input, such as a question or a task. For example, “Explain quantum mechanics in a simple manner”.
2. The raw text is converted into numerical tokens using Byte-Pair Encoding (BPE). BPE splits the text into frequently occurring subword units. Since it operates at the byte level, it can handle any input, including text, code, emojis, and more.
3. Each token is mapped to a vector (a list of numbers) using a learned embedding table. This vectorized form is what the model understands and processes.
4. Transformer layers are where the real computation happens. The 20B Model has 24 Transformer layers, and the 120B Model has 36 Transformer layers. Each layer includes a self-attention module, router, and experts (MoE).
5. The self-attention module lets the model understand relationships between words across the entire input.
6. The LLM uses a Mixture-of-Experts (MOE) architecture. Instead of using all model weights like in traditional models, a router chooses the 2 best “experts” out of a pool of up to 64 total experts. Each expert is a small feedforward network trained to specialize in certain types of inputs. Only 2 experts are activated per token, thereby saving compute while improving quality.
7. After passing through all layers, the model projects the internal representation back into token probabilities, predicting the next word or phrase.
8. To make the raw model safe and helpful, it undergoes supervised fine-tuning and reinforcement learning.
9. Finally, the model generates a response based on the predicted tokens, returning coherent output to the user based on the context.

Over to you: Have you used OpenAI’s open-source models?

The Modern Software Stack



In today's world, building software means working across multiple layers, each with its own role, tools, and technologies. Here are 9 layers that make up most modern applications:

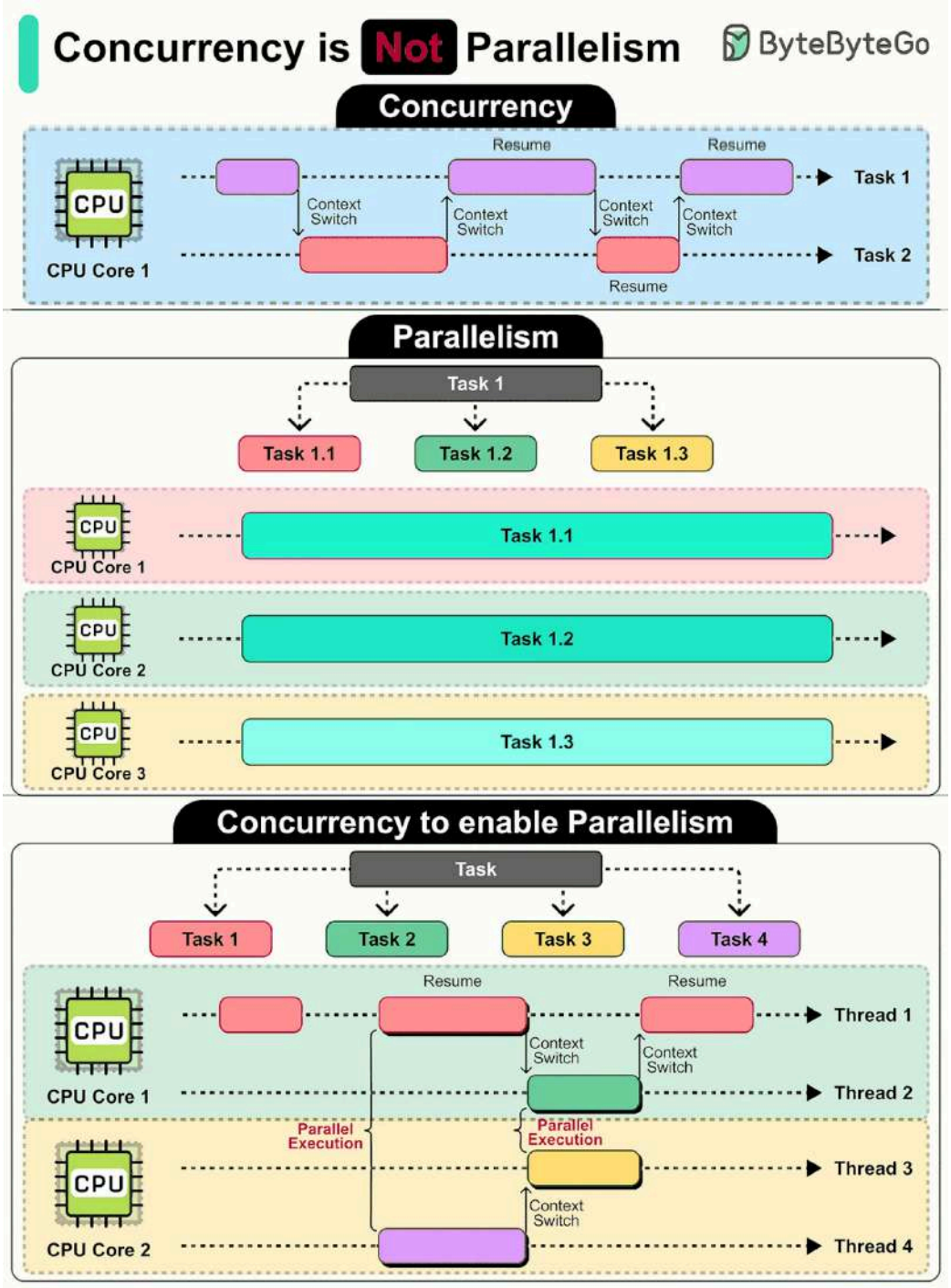
1. Presentation Layer (UI/UX): Handles how users interact with the application, focusing on visuals, layout, and usability.

2. Edge and Delivery (Optional): Brings content closer to users through global delivery networks, reducing latency and improving performance.

3. Integration Layer (API): Defines how different parts of the system communicate, enabling interoperability between components.
4. Messaging & Async Processing (Optional): Processes tasks and events in the background to improve scalability and responsiveness.
5. Business Logic Layer: Implements the core rules, workflows, and decision-making processes of the application.
6. Data Access Layer: Acts as a bridge between application logic and stored data, ensuring secure and efficient retrieval or updates.
7. Data Storage Layer: Stores, organizes, and manages the application's structured and unstructured data.
8. Analytics & ML (Optional): Analyzes data to generate insights, predictions, and intelligent features.
9. Infrastructure Layer (Hosting / Runtime): Provides the computing environment and resources for deploying, running, and scaling the application.

Over to you: Which layer do you think is the most underrated in modern software development?

Concurrency is NOT Parallelism



Concurrency: It's a design approach where tasks can start, run, and complete in overlapping periods, even on a single CPU core. It is about managing multiple tasks at the same time.

The CPU rapidly switches between tasks (context switching), creating the illusion that tasks are progressing simultaneously, though they are not.

Concurrency is great for tasks that involve waiting, like I/O operations. It allows other tasks to progress during the wait, improving overall efficiency.

Parallelism: Refers to the simultaneous execution of multiple tasks, using multiple CPU cores.

Parallelism excels at heavy computations like data analysis or rendering graphics, where tasks can be divided and run simultaneously on different cores.

How They Work Together

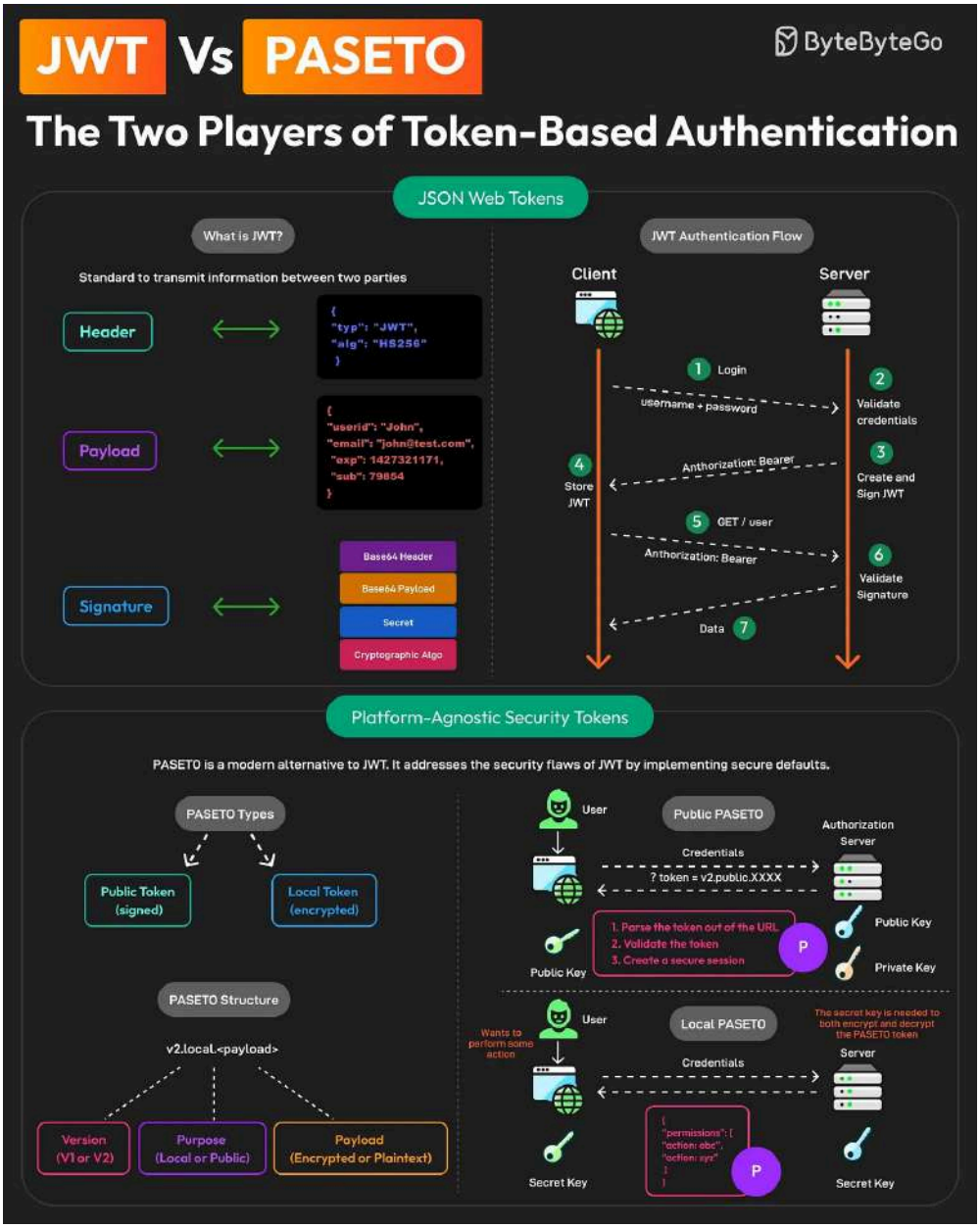
It's important to note that while concurrency and parallelism are different concepts, they are closely related. A well-designed concurrent program can scale to use multiple cores for parallelism when needed.

By understanding the differences and interplay between concurrency and parallelism, we can design more efficient systems and create better-performing applications.

Over to you: Have you encountered any challenges with concurrency or parallelism?

JWT vs PASETO: The Two Players of Token-Based Authentication

Token-based authentication has become quite popular over the years. Traditionally, JWTs have dominated this space.



But now, a new player is making waves: PASETO, or Platform-Agnostic Security Tokens.

So, what's the difference between the two?

1. JWTs
   JWT or JSON Web Tokens is an open standard for securely transmitting information between two parties.

   A JWT consists of a Header, Payload, and Signature.

   JWTs can be used to implement stateless authentication between client and server applications.

2. PASETO
   PASETO is a modern alternative to JWT. It addresses JWT's security flaws by implementing secure defaults.

   Unlike JWT, PASETO enforces strong, cryptographically sound algorithms, reducing the risk of vulnerabilities.

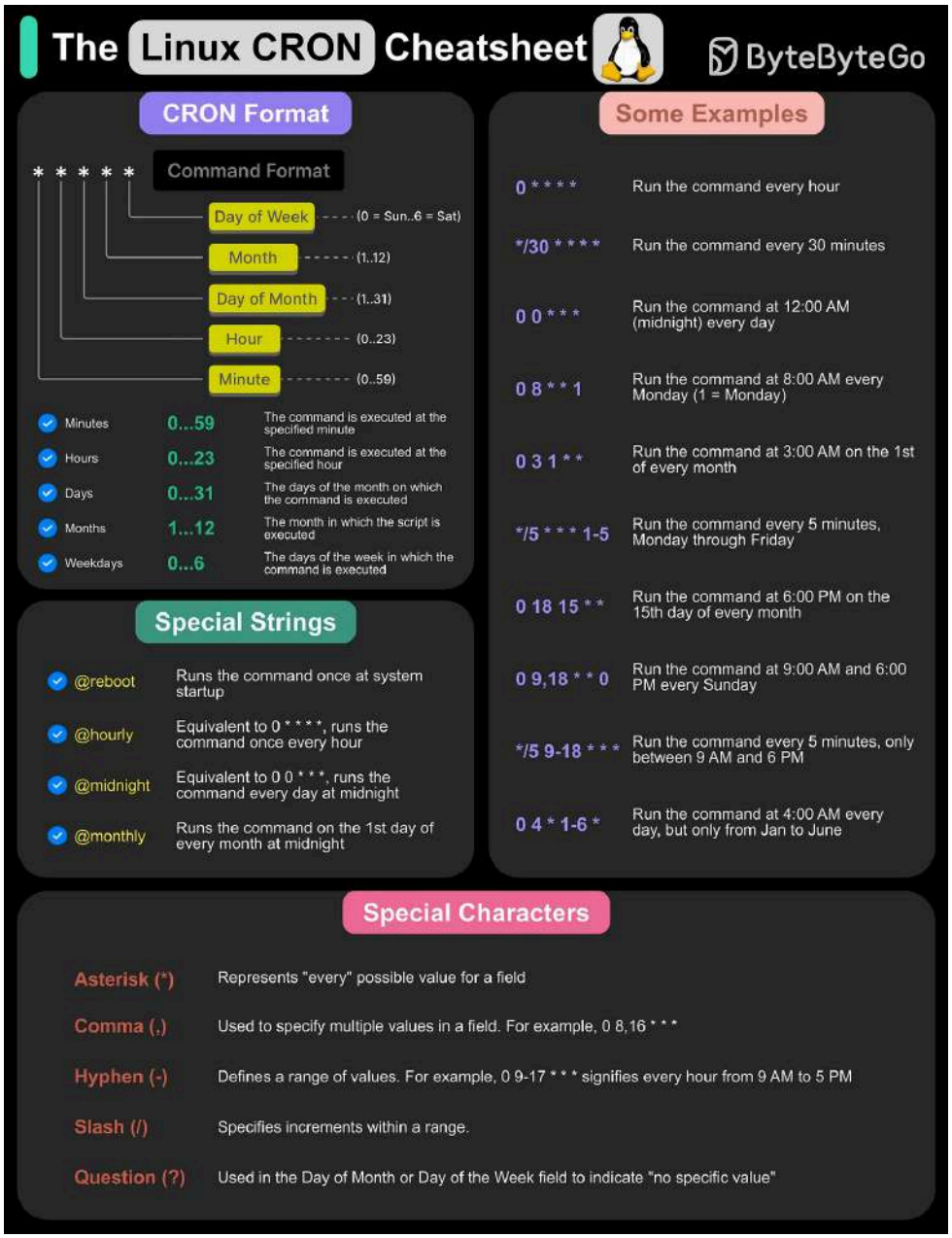
   A PASETO typically consists of Version, Purpose, and Payload. There are two types of PASETO:

   - Public PASETO: They are signed using asymmetric cryptography and ensure the integrity of the data, but not its confidentiality.
   - Local PASETO: They are encrypted using symmetric encryption algorithms, ensuring the confidentiality of the data contained within the token.

Over to you: Have you used JWTs or PASETO in your applications?

The Linux Cron Cheatsheet

Cron is a time-based job scheduler that allows users to automate repetitive tasks by running commands or scripts at specified intervals. It uses a Cron expression, a syntax defining the schedule, which consists of five fields (minute, hours, day, month, weekday).



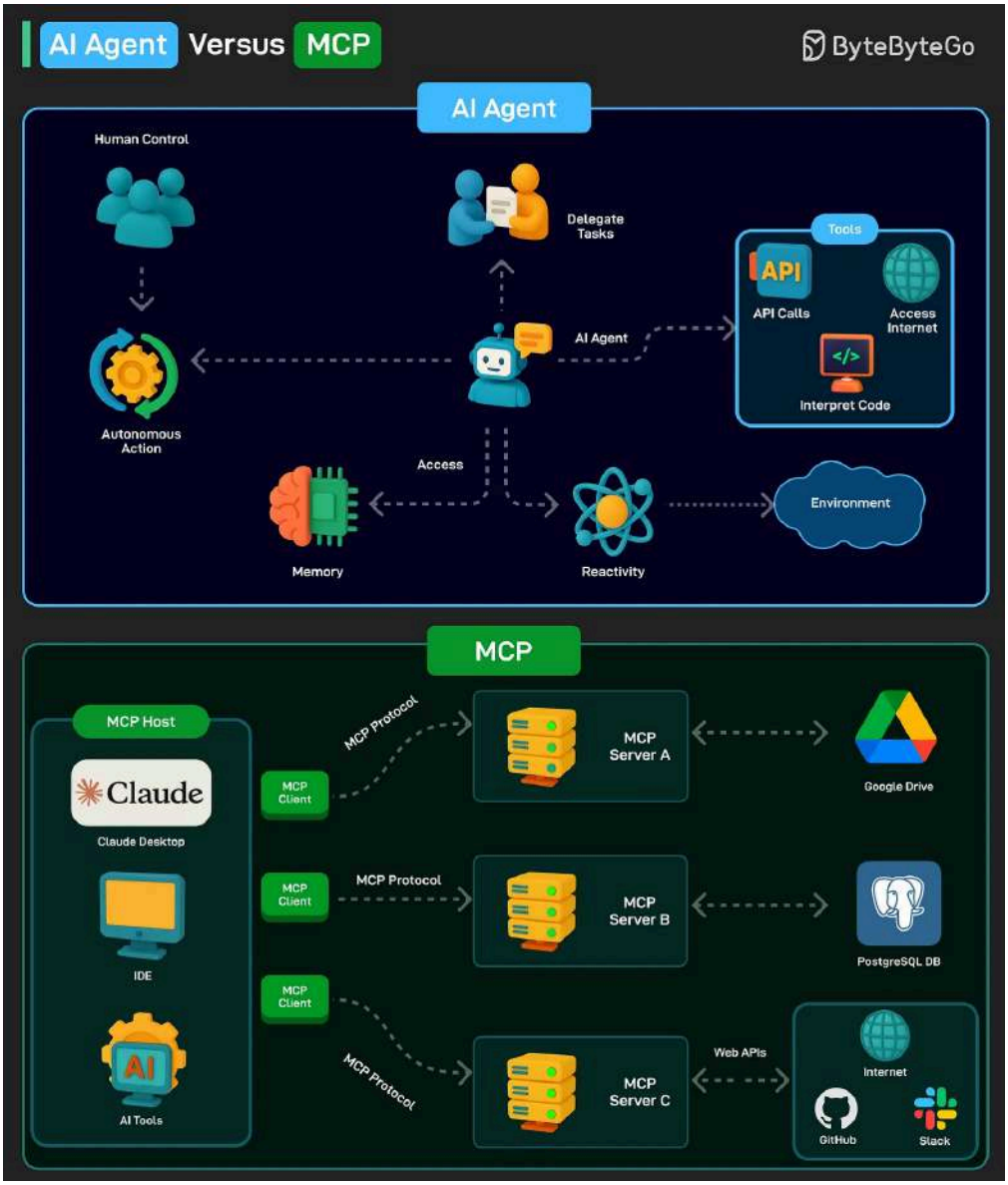
Cron is widely for system maintenance, backups, log rotation, and automation of tasks like sending emails or running scripts at regular intervals. In this cheatsheet, we cover:

1. The Cron Format

2. Some Cron Examples
3. Special Cron Strings
4. Special characters

Over to you: Have you used Cron?

AI Agent versus MCP



An AI agent is a software program that can interact with its environment, gather data, and use that data to achieve predetermined goals. AI agents can choose the best actions to perform to meet those goals.

Key characteristics of AI agents are as follows:

1. An agent can perform autonomous actions without constant human intervention. Also, they can have a human in the loop to maintain control.
2. Agents have a memory to store individual preferences and allow for personalization. It can also store knowledge. An LLM can undertake information processing and decision-making functions.

3. Agents must be able to perceive and process the information available from their environment.

Model Context Protocol (MCP) is a new system introduced by Anthropic to make AI models more powerful.

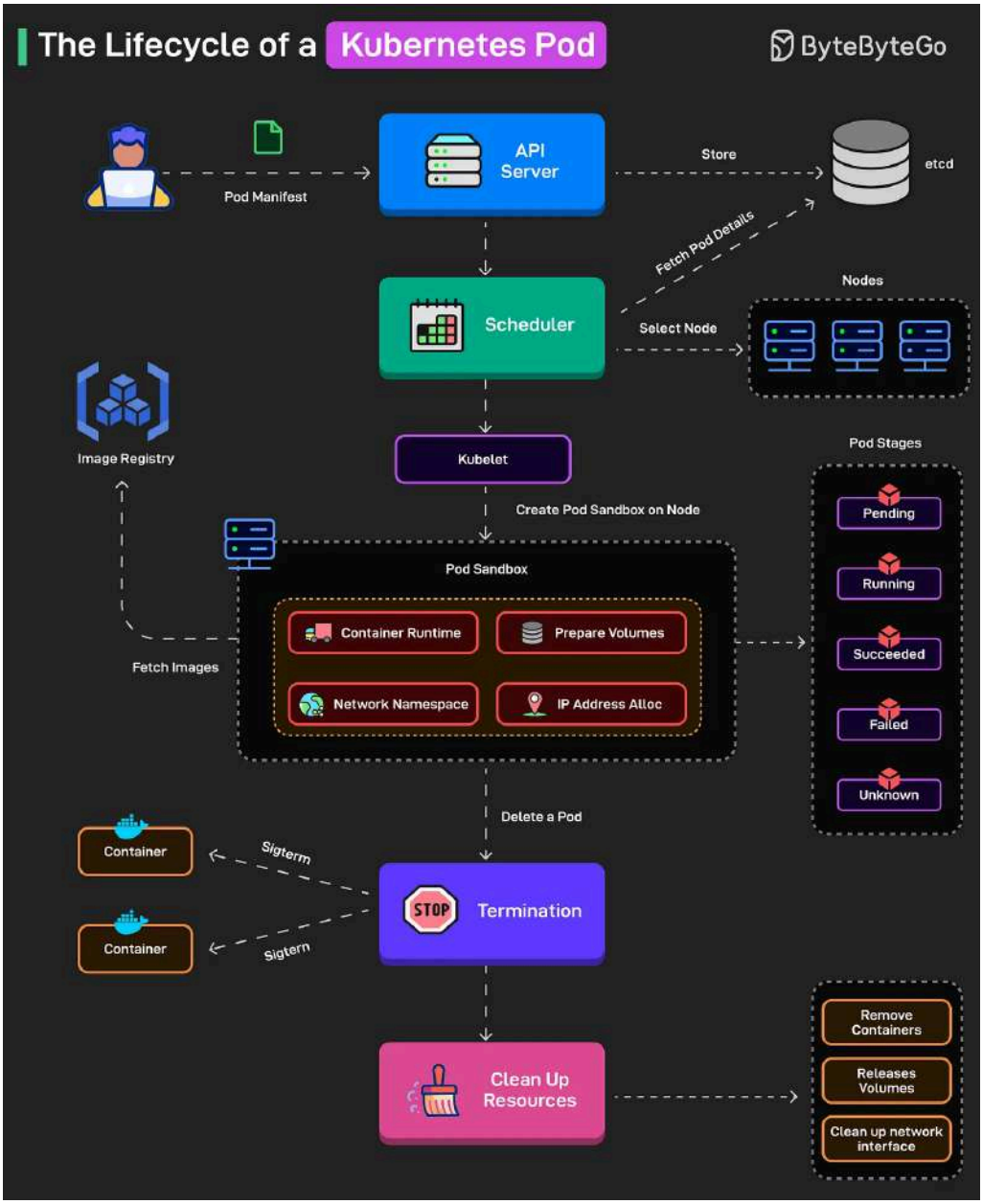
It is an open standard that allows AI models (like Claude) to connect to databases, APIs, file systems, and other tools without needing custom code for each new integration.

MCP follows a client-server model with 3 key components:

1. Host: AI applications like Claude
2. MCP Client: Component inside an AI model (like Claude) that allows it to communicate with MCP servers
3. MCP Server: Middleman that connects an AI model to an external system

Over to you: Have you used AI Agents or MCP?

The Lifecycle of a Kubernetes Pod



The Pod manifest is submitted to the API server and stored in etcd.

1. The scheduler selects a node for the Pod based on resources, affinity rules, and binds the Pod to that node.
2. The kubelet prepares the Pod by creating its network namespace, assigning an IP, mounting volumes, and pulling images if needed.
3. Containers move from Waiting to Running, with kubelet monitoring health probes for liveness and readiness.

4. Kubernetes tracks the Pod's high-level phase from Pending to Running to Succeeded/Failed/Unknown.
5. Upon termination, Kubernetes sends SIGTERM (and SIGKILL if needed) for the individual containers in the Pod.
6. After termination, the resources are cleaned up, and the Pod details are removed from etcd.

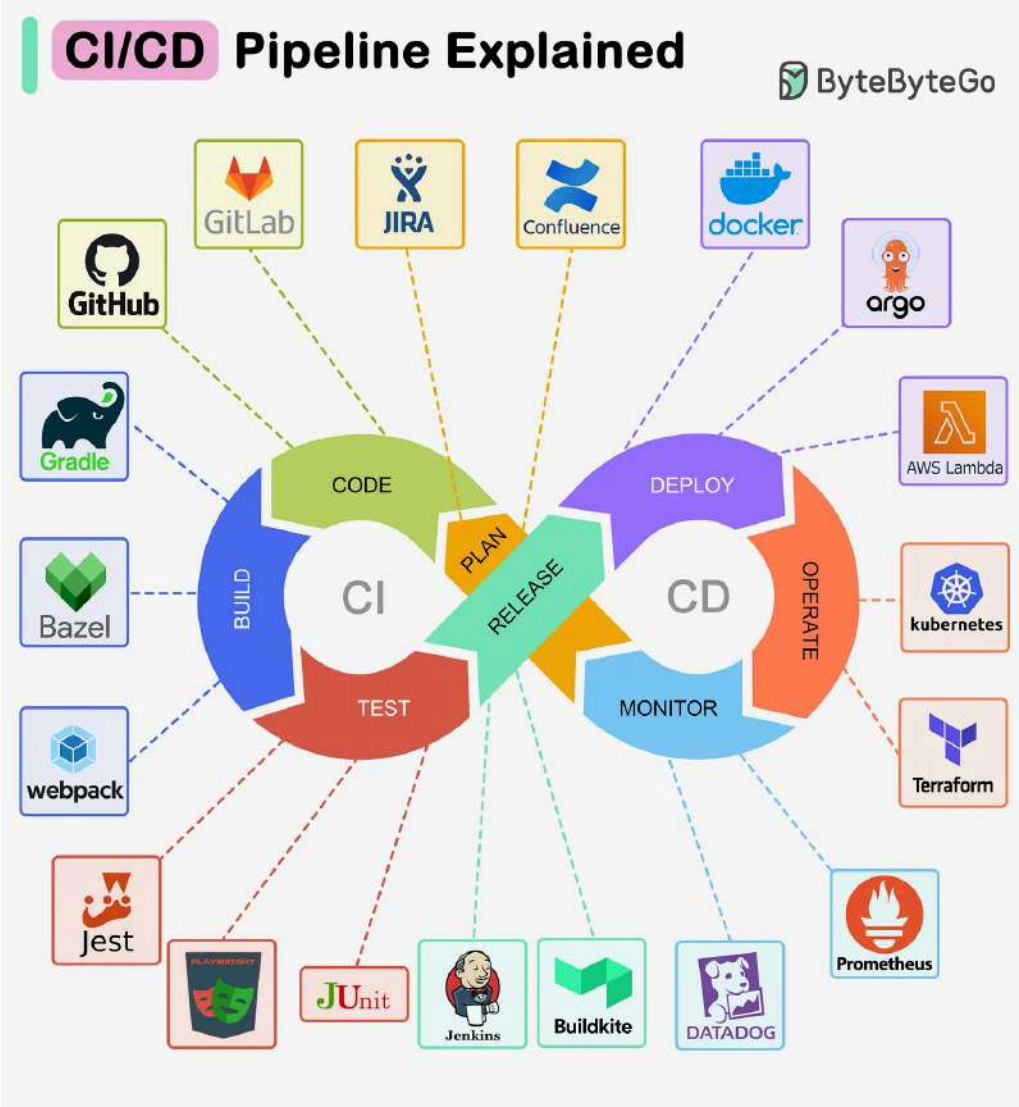
Over to you: What else will you add to understand the Kubernetes Pod Lifecycle?

CI/CD Pipeline Explained

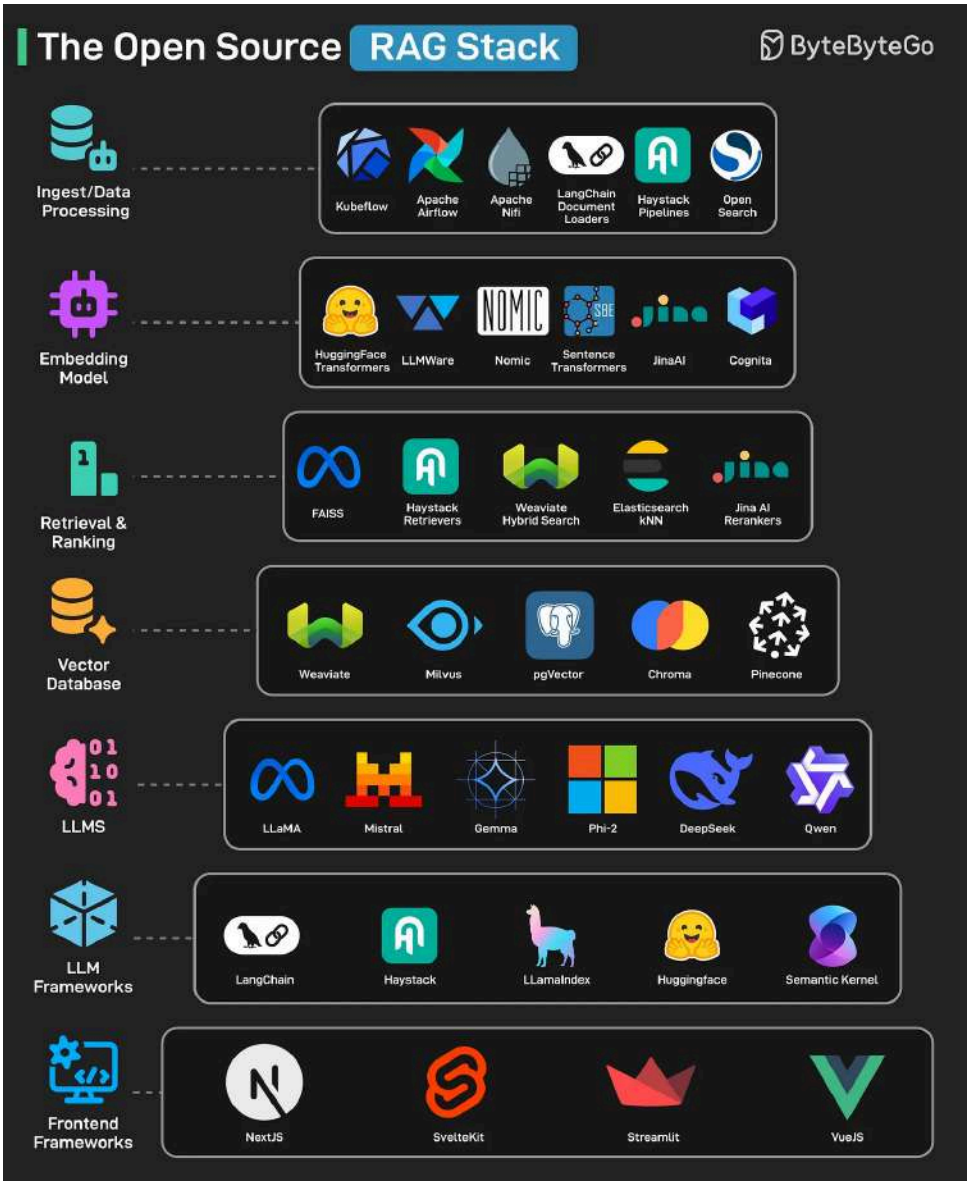
A CI/CD pipeline is a tool that automates the process of building, testing, and deploying software.

It integrates the different stages of the software development lifecycle, including code creation and revision, testing, and deployment, into a single, cohesive workflow.

The diagram below illustrates some of the tools that are commonly used.



The Open Source RAG Stack



1. Frontend Frameworks: Used to build frontend interfaces for the RAG apps. Some tools that can help are NextJS, VueJS, SvelteKit, and Streamlit.

2. LLM Frameworks: High-level orchestration of LLM pipelines, prompts, and chains. This includes tools such as LangChain, LlamaIndex, Haystack, HuggingFace, and Semantic Kernel.

3. LLMs: Generates final responses using large language models. Some open-source options include Llama, Mistral, Gemma, Phi-2, DeepSeek, Qwen, etc.

4. Retrieval and Ranking: Retrieves relevant chunks and ranks them based on relevance. Tools like Meta FAISS, Haystack Retrievers, Weaviate Hybrid Search, ElasticSearch kNN, and JinaAI Rerankers can help.

5. Vector Database: Stores and enables similarity search over vector embeddings. Common options include Weaviate, Milvus, Postgres pgVector, Chroma, and Pinecone.

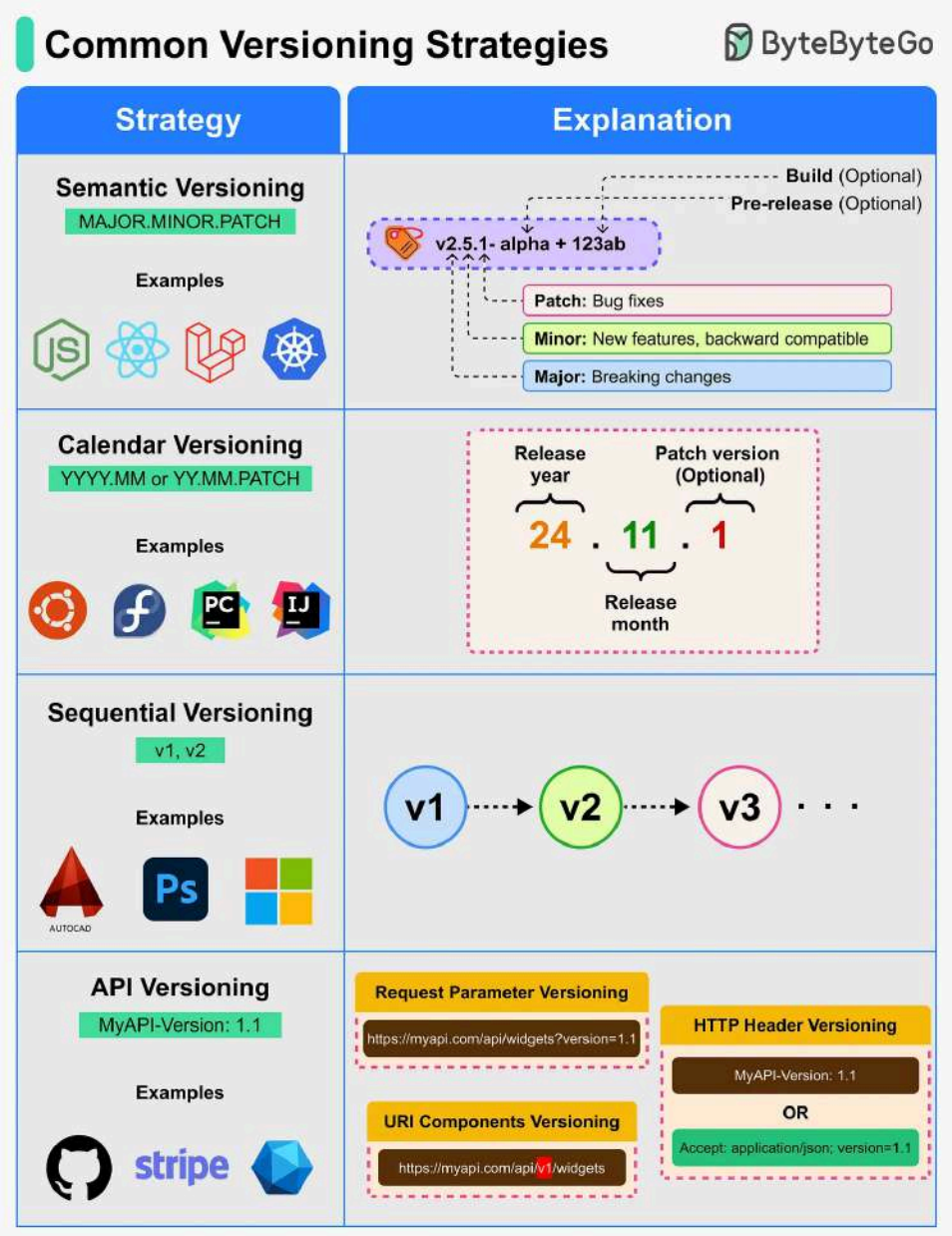
6. Embedding Mode: Converts text/data into vector representations using ML. Some open-source tools include HuggingFace Transformers, LLMWare, Nomic, Sentence Transformers, JinaAI, and Cognita.

7. Ingest/Data Processing: Extracts, cleanses, and prepares raw data for indexing and retrieval. Tools like Kubeflow, Apache Airflow, Apache NiFi, LangChain Document Loads, Haystack Pipelines, and OpenSearch can help.

Over to you: Which other open-source tool will you add to the list?

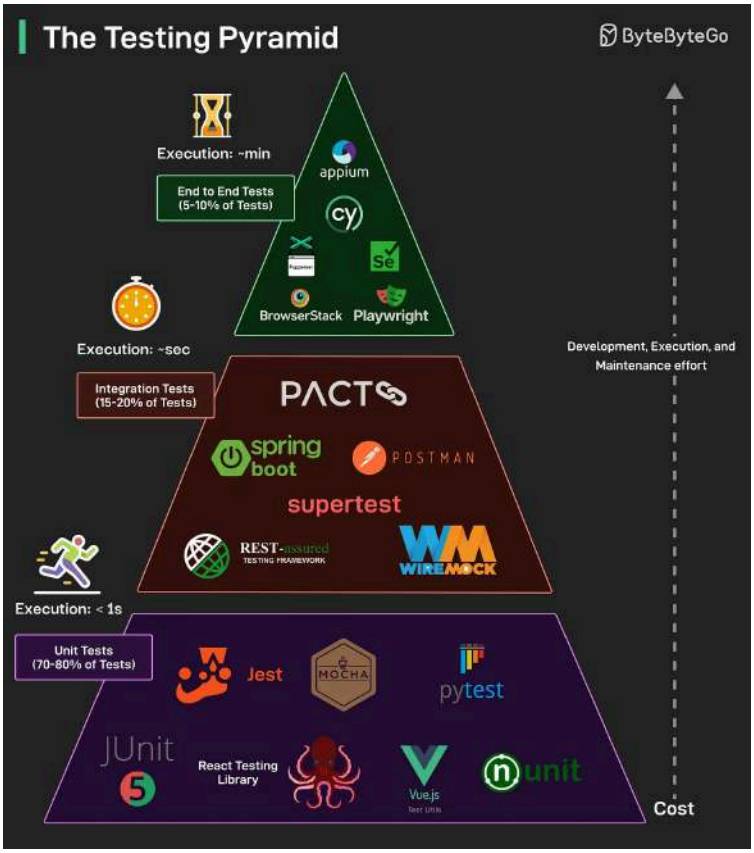
What are some of the most popular versioning strategies?

Versioning helps developers communicate release changes clearly, whether it's for software packages, APIs, or operating systems.



- Semantic Versioning (SemVer): A versioning scheme that conveys meaning about the scope of changes in a release using the format MAJOR.MINOR.PATCH.
- Calendar Versioning (CalVer): A scheme that uses the release date (year and month) as the version number to indicate when the release occurred.

The Testing Pyramid



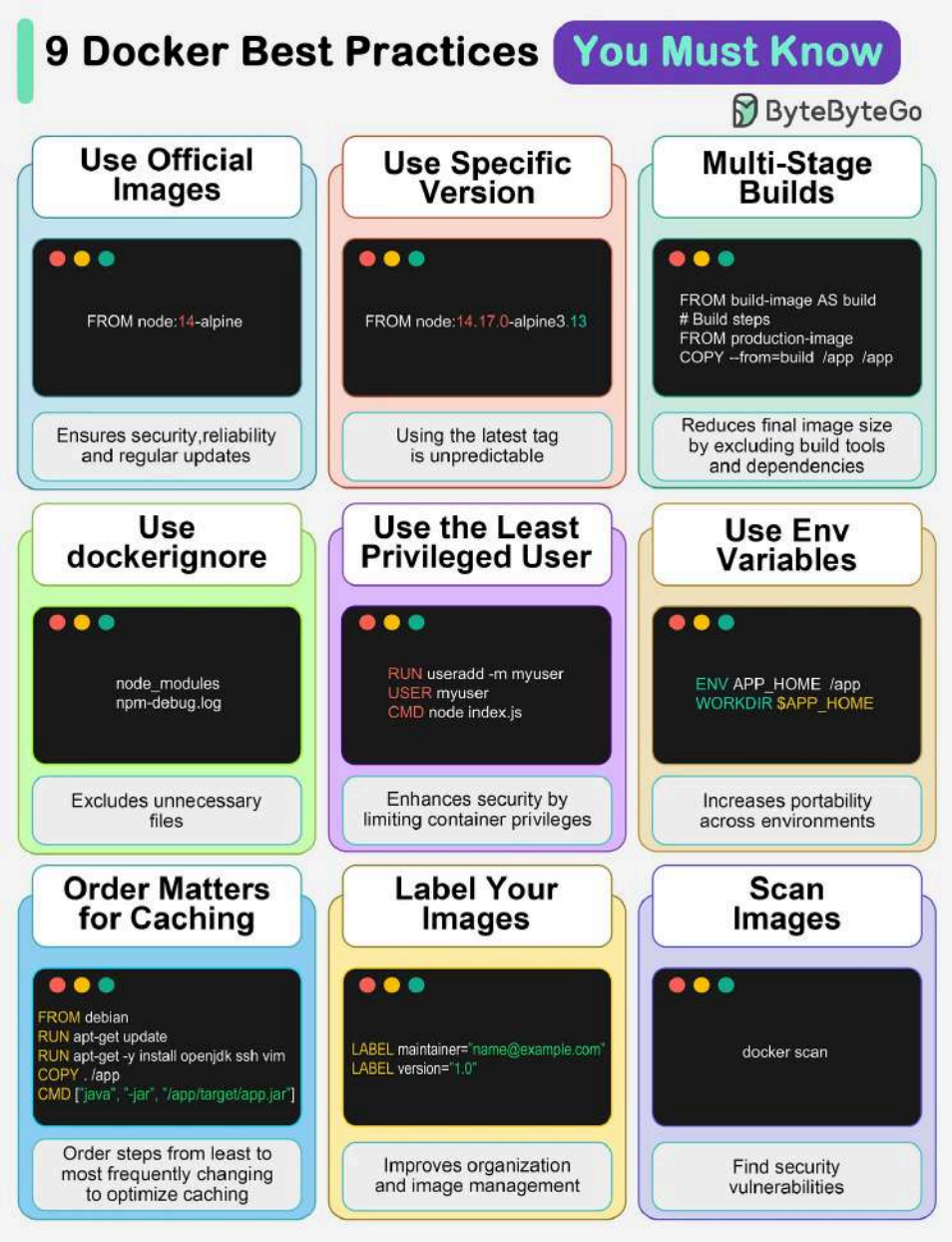
Testing is the backbone of reliable software. The Testing Pyramid is a widely accepted strategy for structuring tests into three key layers:

- Unit Tests: These are the foundation of the pyramid. Unit tests are fast, isolated, and low-cost to write and maintain. They test individual functions, methods, or components.
- Integration Tests: These tests validate interactions between components, such as APIs, databases, and external services. They are slower than unit tests and require more setup.
- E2E Tests: These simulate real user flows from start to finish across the full system. They are expensive to write and maintain and tend to be slow to execute.

As you go up the pyramid, the cost of test development, execution, and maintenance increases.

Over to you: Which layer do you find most valuable in your testing strategy, and why?

9 Docker Best Practices You Should Know



1. Use official images
   This ensures security, reliability, and regular updates.

2. Use a specific image version
   The default latest tag is unpredictable and causes unexpected behavior.

3. Multi-Stage builds
   Reduces final image size by excluding build tools and dependencies.

4. Use .dockerignore
   Excludes unnecessary files, speeds up builds, and reduces image size.

5. Use the least privileged user
   Enhances security by limiting container privileges.

6. Use environment variables
   Increases flexibility and portability across different environments.

7. Order matters for caching
   Order your steps from least to most frequently changing to optimize caching.

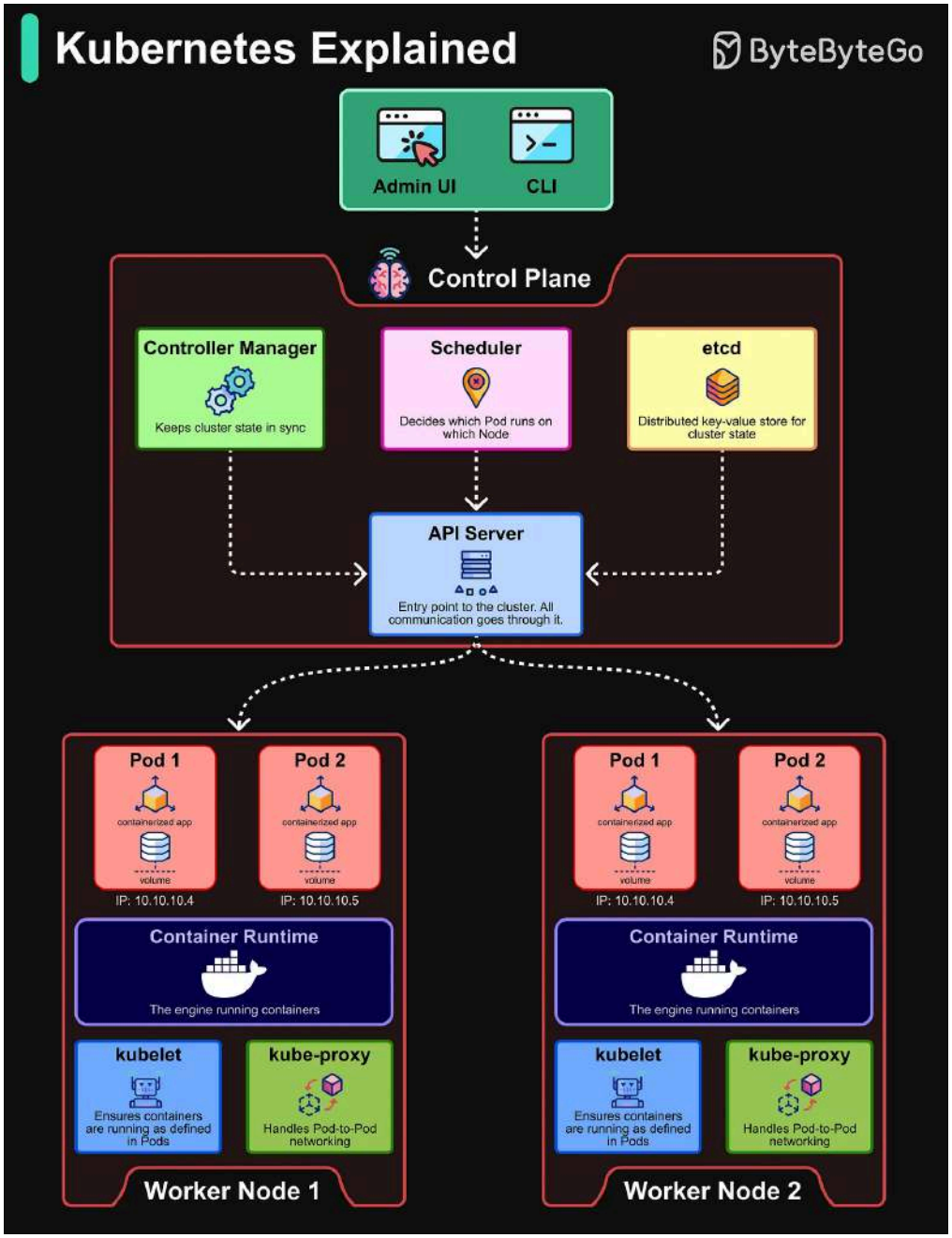
8. Label your images
   It improves organization and helps with image management.

9. Scan images
   Find security vulnerabilities before they become bigger problems.

Over to you: Which other Docker best practices will you add to the list?

Kubernetes Explained

Kubernetes is the de facto standard for container orchestration. It automates the deployment, scaling, and management of containerized applications.



Control Plane:

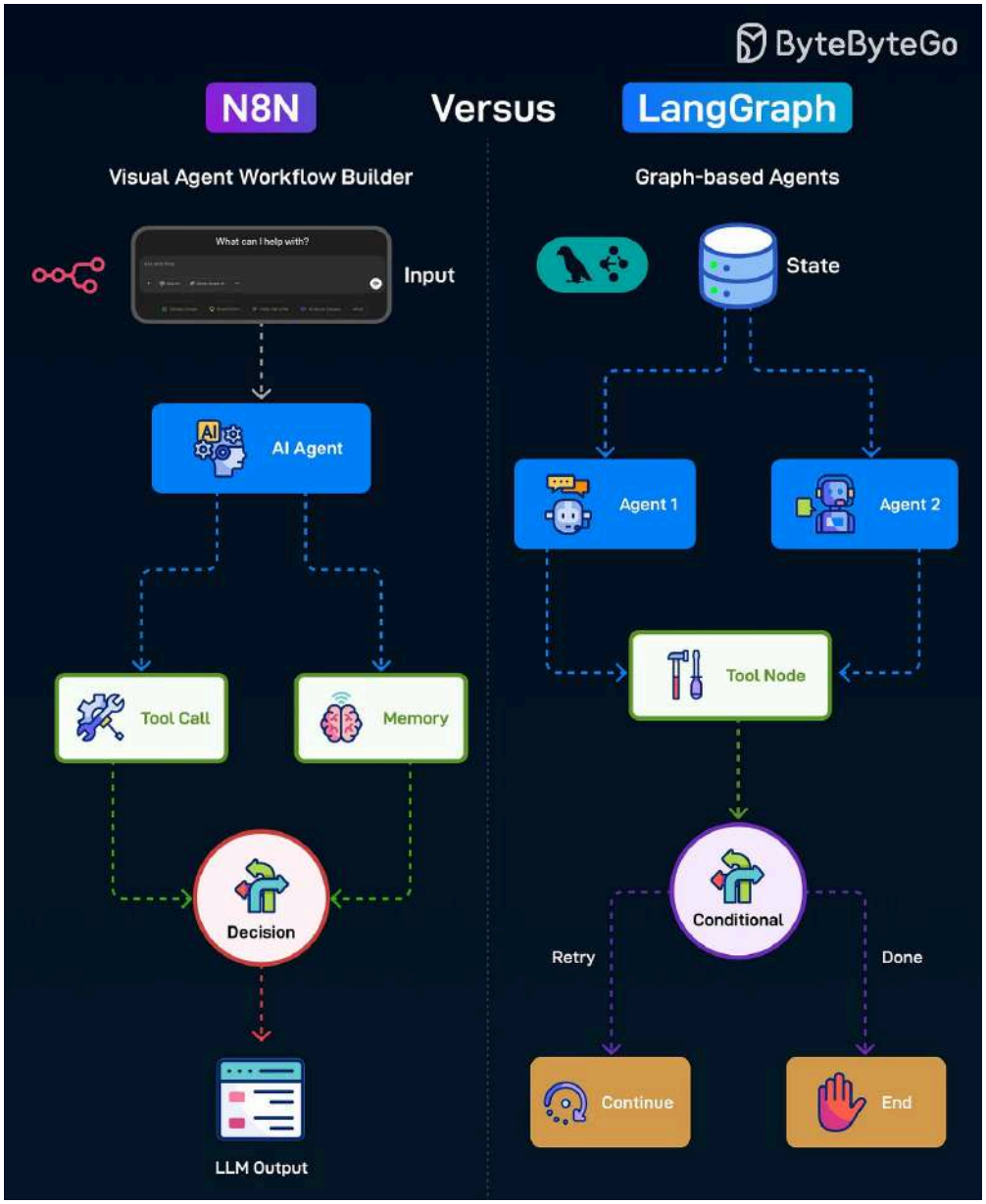
- API Server: Acts as the communication hub between users, the control plane, and worker nodes.

- Scheduler: Decides which Pod runs on which Node.
- Controller Manager: Keeps the cluster state in sync.
- etcd: A distributed key-value store that holds the cluster's state.

Worker Nodes:

- Pods: The smallest deployable unit in Kubernetes, representing one or more containers.
- Container Runtime: The engine (like Docker or containerd) that runs the containers.
- kubelet: Ensures containers are running as defined in Pods.
- kube-proxy: Handles networking between Pods and ensures communication.

N8N versus LangGraph



N8N is an open-source automation tool that lets you visually build workflows by connecting different services, APIs, and AI tools in a sequence. Here's how it works:

1. Starts with Input from the user.
2. Passes it to an AI Agent for processing.
3. The AI Agent can either make a Tool Call or access Memory.
4. A Decision node chooses the next action and produces the final LLM output for the user.

LangGraph is a Python framework for building AI Agent workflows using a flexible graph structure that supports branching, looping, and multi-agent collaboration. Here's how it works:

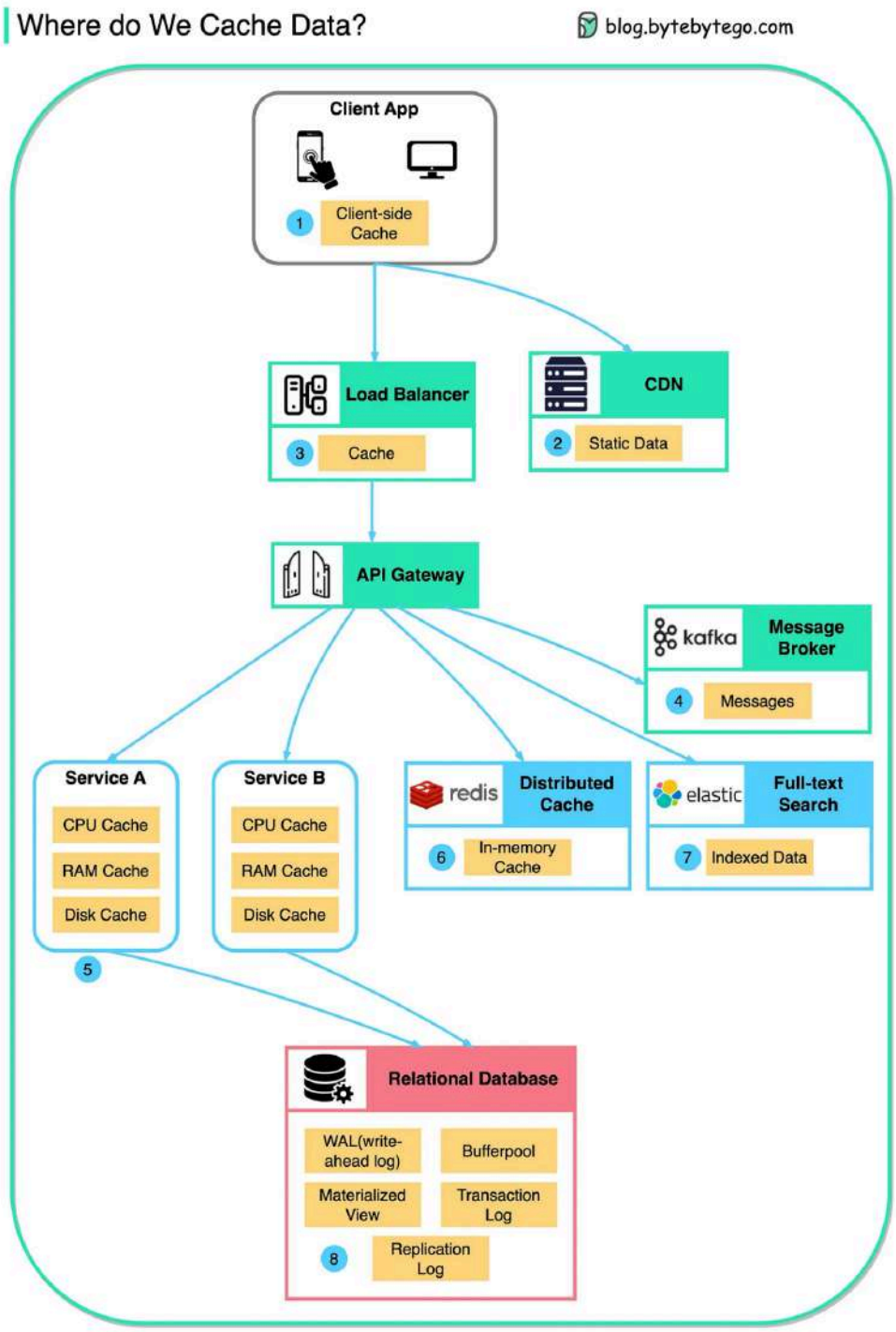
1. Starts with a shared State containing workflow context.
2. Can route tasks to different agents.
3. Agents interact with a Tool Node to perform tasks.
4. A Conditional node decides whether to retry or mark the process done.

Over to you: Have you used N8N or LangGraph?

Where Do We Cache Data?

Data is cached everywhere, from the front end to the back end!

This diagram illustrates where we cache data in a typical architecture.

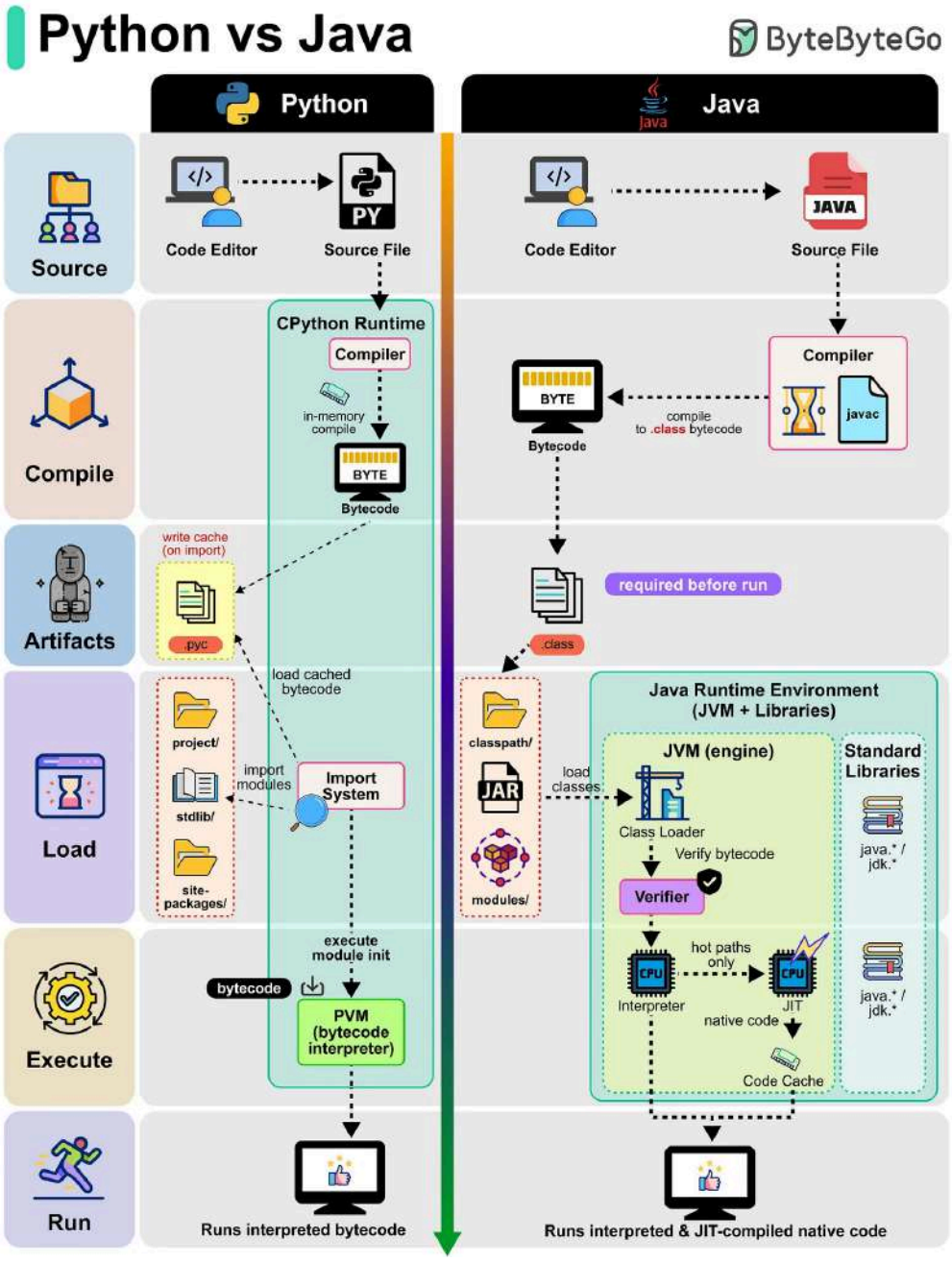


There are multiple layers along the flow.

1. Client apps: HTTP responses can be cached by the browser. We request data over HTTP for the first time, and it is returned with an expiry policy in the HTTP header; we request data again, and the client app tries to retrieve the data from the browser cache first.
2. CDN: CDN caches static web resources. The clients can retrieve data from a CDN node nearby.
3. Load Balancer: The load Balancer can cache resources as well.
4. Messaging infra: Message brokers store messages on disk first, and then consumers retrieve them at their own pace. Depending on the retention policy, the data is cached in Kafka clusters for a period of time.
5. Services: There are multiple layers of cache in a service. If the data is not cached in the CPU cache, the service will try to retrieve the data from memory. Sometimes the service has a second-level cache to store data on disk.
6. Distributed Cache: Distributed cache like Redis hold key-value pairs for multiple services in memory. It provides much better read/write performance than the database.
7. Full-text Search: we sometimes need to use full-text searches like Elastic Search for document search or log search. A copy of data is indexed in the search engine as well.
8. Database: Even in the database, we have different levels of caches:
    - WAL(Write-ahead Log): data is written to WAL first before building the B tree index
    - Bufferpool: A memory area allocated to cache query results
    - Materialized View: Pre-compute query results and store them in the database tables for better query performance
    - Transaction log: record all the transactions and database updates
    - Replication Log: used to record the replication state in a database cluster

Over to you: With the data cached at so many levels, how can we guarantee the sensitive user data is completely erased from the systems?

Python vs Java



Ever wondered what happens behind the scenes when you run a Python script or a Java program? Let’s find out:

Python (CPython Runtime):

- Python source code (.py) is compiled into bytecode automatically in memory.

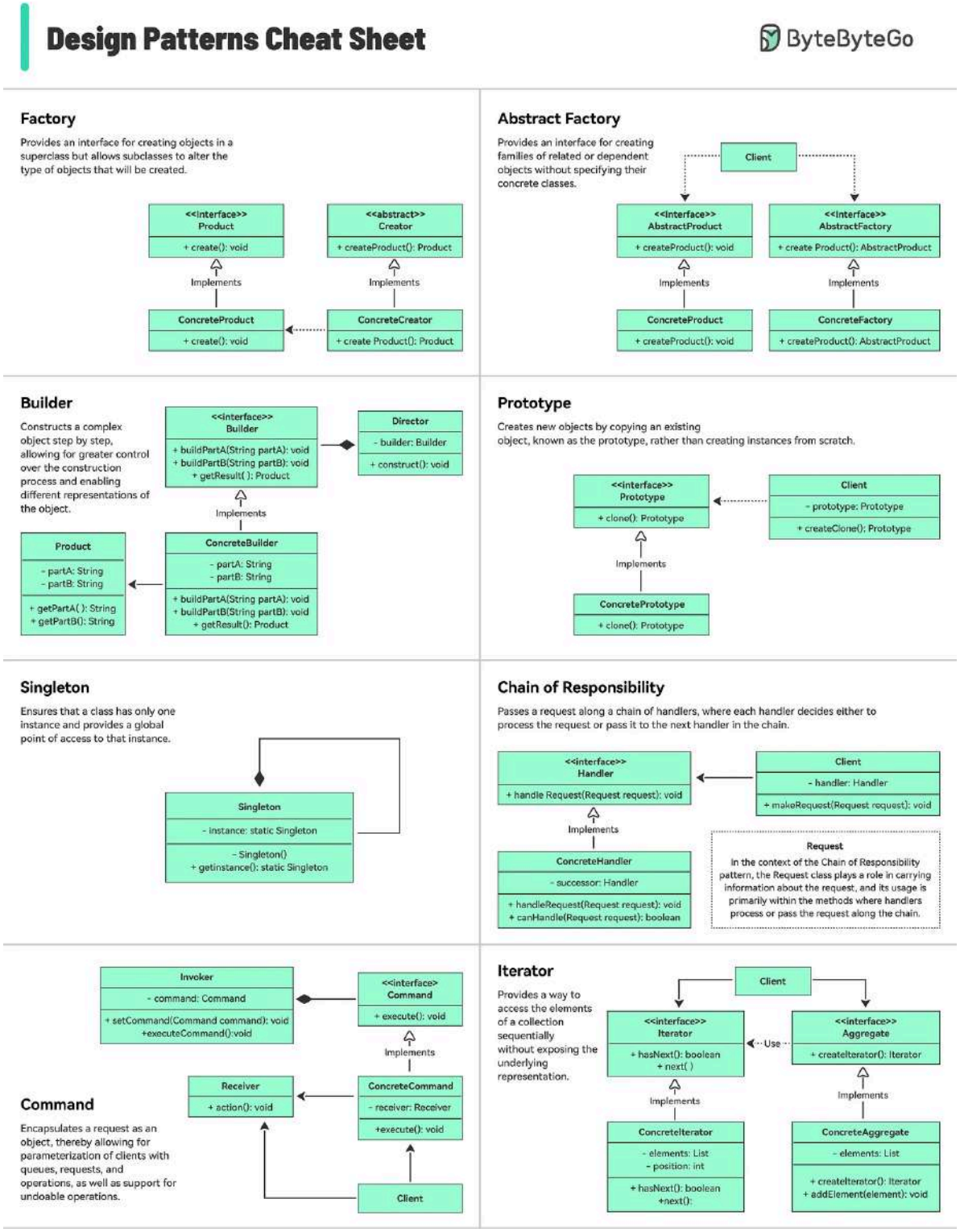
- Bytecode can also be cached in .pyc files, making re-runs faster by using the cached version.
- The Import System loads modules and dependencies.
- The Python Virtual Machine (PVM) interprets the bytecode line by line, making Python flexible but relatively slower.

Java (JVM Runtime):

- Java source code (.java) is compiled into .class bytecode using javac.
- The Class Loader loads bytecode into the Java Runtime Environment (JVM).
- Bytecode is verified and executed.
- JVM uses both an Interpreter and a JIT Compiler, frequently used code (hot paths) is converted into native machine code, making Java faster.

Design Patterns Cheat Sheet

The cheat sheet briefly explains each pattern and how to use it.

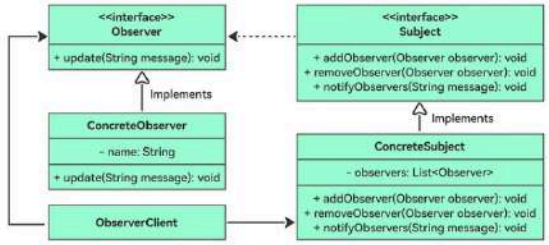


Design Patterns Cheat Sheet

ByteByteGo

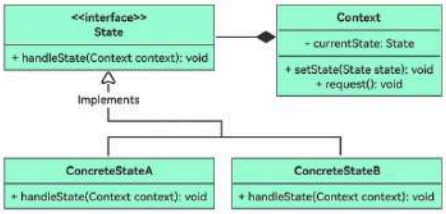
Observer

Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.



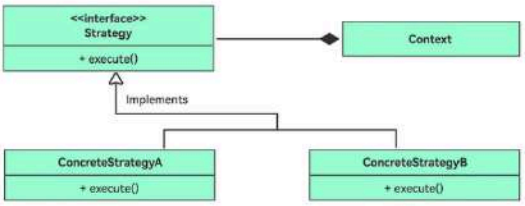
State

Allows an object to alter its behavior when its internal state changes, effectively encapsulating state-specific logic into separate classes.



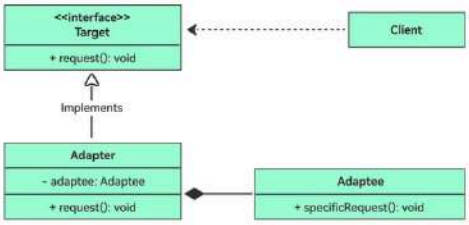
Strategy

Defines a family of algorithms, encapsulates each one, and makes them interchangeable, allowing the algorithm to vary independently from clients that use it.



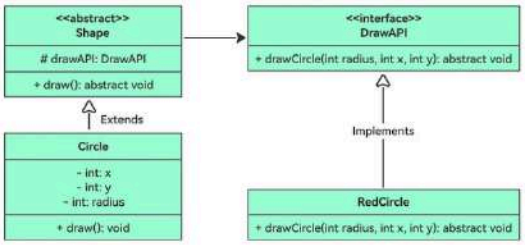
Adapter

Allows incompatible interfaces to work together by acting as a bridge that converts one interface into another expected by the client.



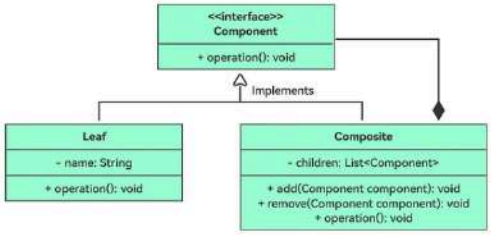
Bridge

Decouples an abstraction from its implementation, allowing them to vary independently by placing them in separate class hierarchies.



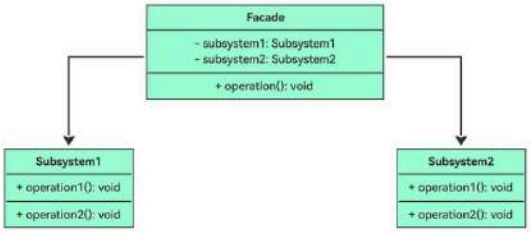
Composite

Allows individual objects and compositions of objects to be treated uniformly by organizing objects into tree structures to represent part-whole hierarchies.



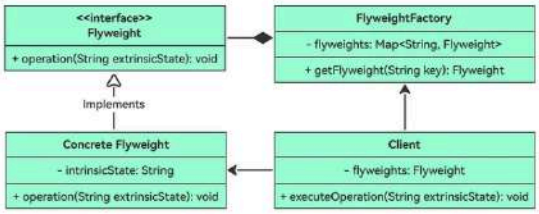
Facade

Provides a simplified, unified interface to a complex subsystem making it easier to use and understand.



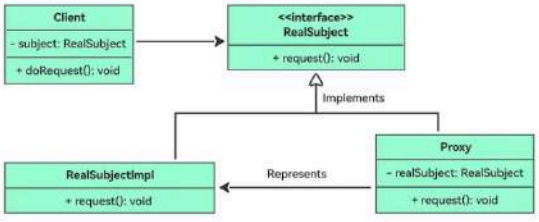
Flyweight

Minimizes memory usage by sharing as much data as possible with other similar objects, using a fine-grained instance for efficient resource management.



Proxy

Provides a surrogate or placeholder for another object to control access to it, enhancing functionality like lazy initialization, access control, or logging.

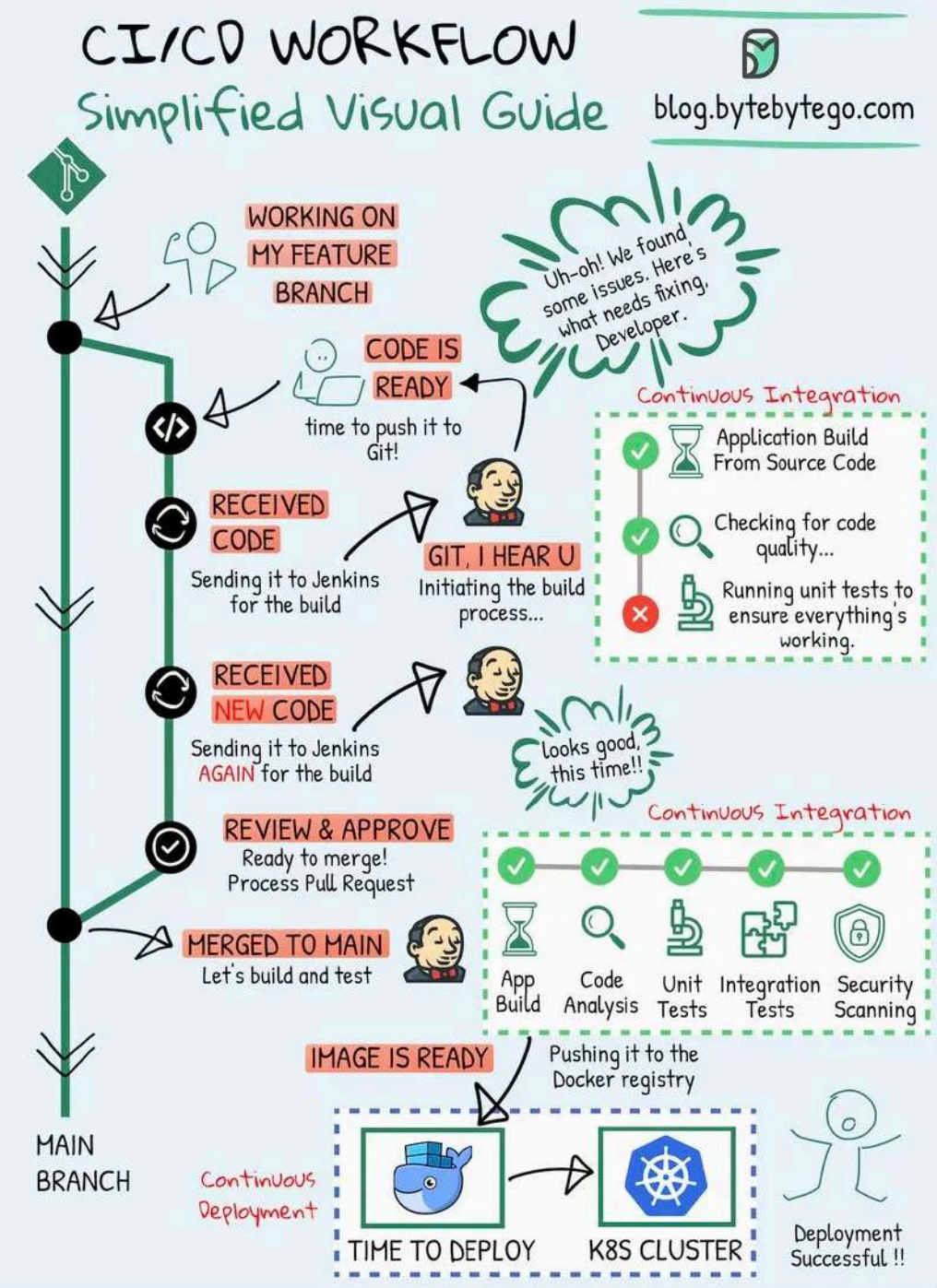


What's included?

- Factory
- Builder
- Prototype
- Singleton
- Chain of Responsibility
- And many more!

CI/CD Simplified Visual Guide

Whether you're a developer, a DevOps specialist, a tester, or involved in any modern IT role, CI/CD pipelines have become an integral part of the software development process.

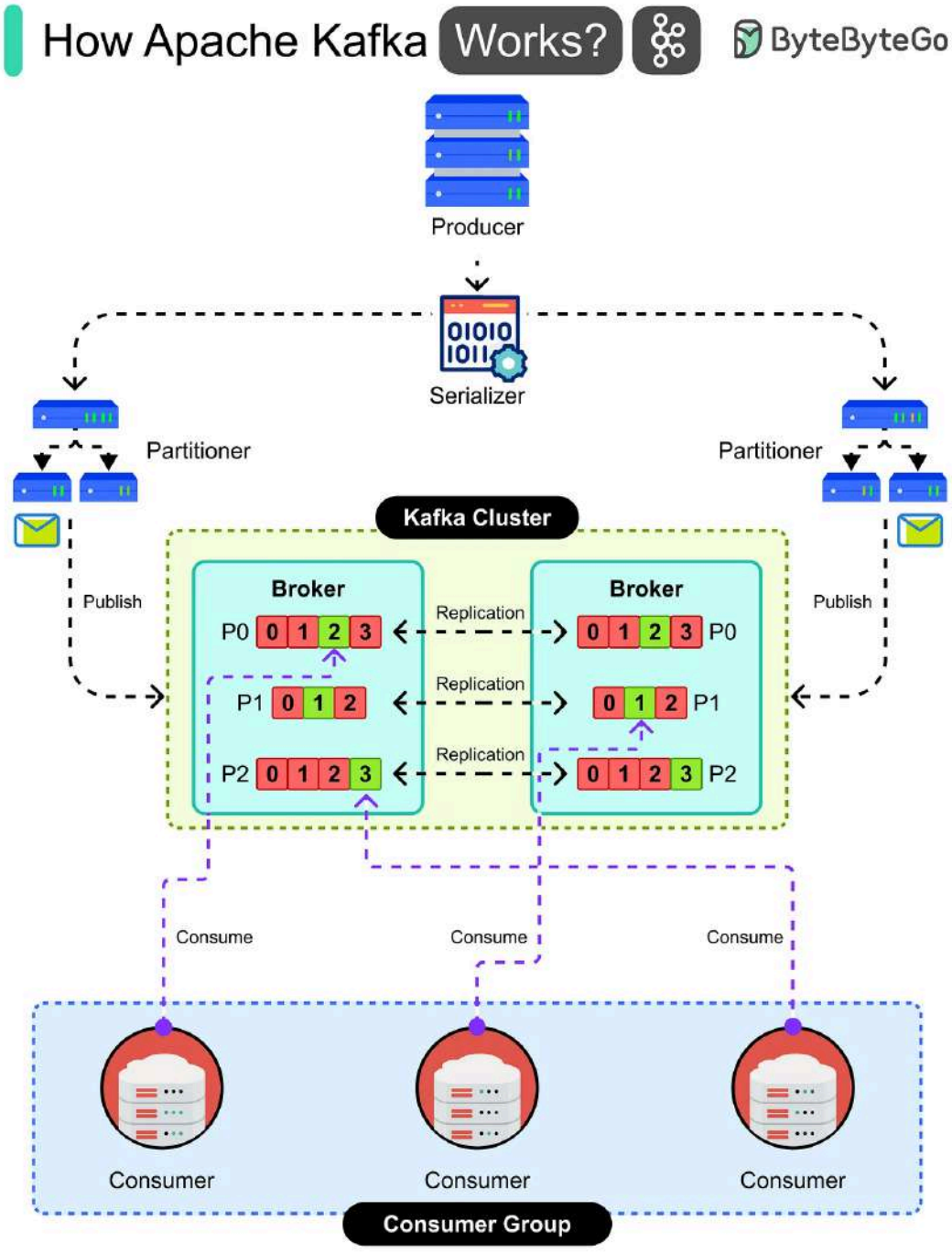


Continuous Integration (CI) is a practice where code changes are frequently combined into a shared repository. This process includes automatic checks to ensure the new code works well with the existing code.

Continuous Deployment (CD) takes care of automatically putting these code changes into real-world use. It makes sure that the process of moving new code to production is smooth and reliable.

This visual guide is designed to help you grasp and enhance your methods for creating and delivering software more effectively.

How Apache Kafka Works?

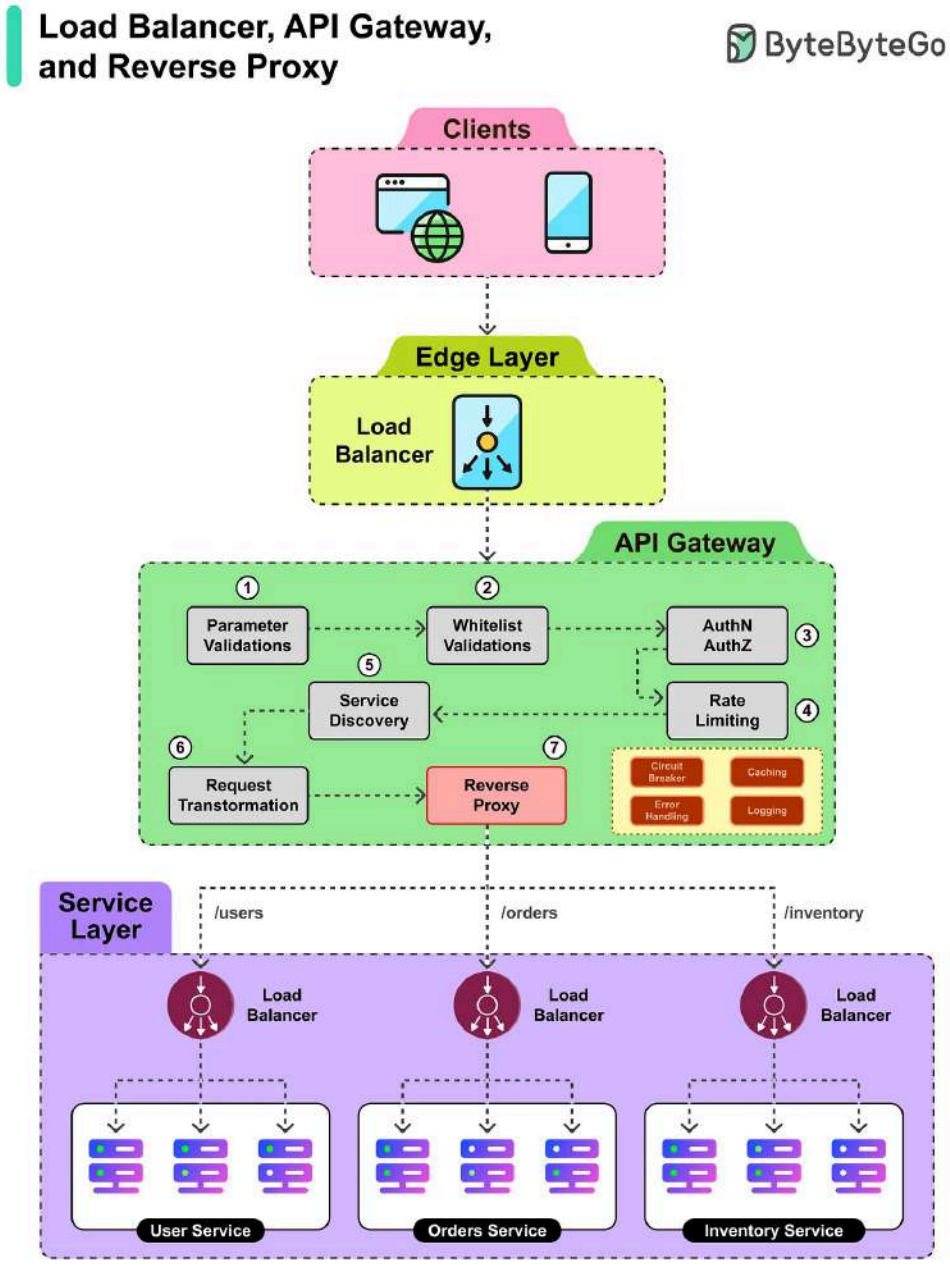


Apache Kafka is a distributed event streaming platform that lets producers publish data and consumers subscribe to it in real-time. Here's how it works:

1. A producer application creates data, like website clicks or payment events.
2. The data is converted by a serializer into bytes so Kafka can handle it.

3. A partitioner decides which topic partition the message should go to.
4. The message is published into a Kafka cluster made of multiple brokers.
5. Each broker stores partitions of topics and replicates them to others for safety.
6. Messages inside partitions are stored in order and available for reading.
7. A consumer group subscribes to the topic and takes responsibility for processing data.
8. Each consumer in the group reads from different partitions to balance the work.
9. Consumers process the data in real-time, such as updating dashboards or triggering actions.

Load Balancers vs API Gateways vs Reverse Proxy! And how can they Work Together?

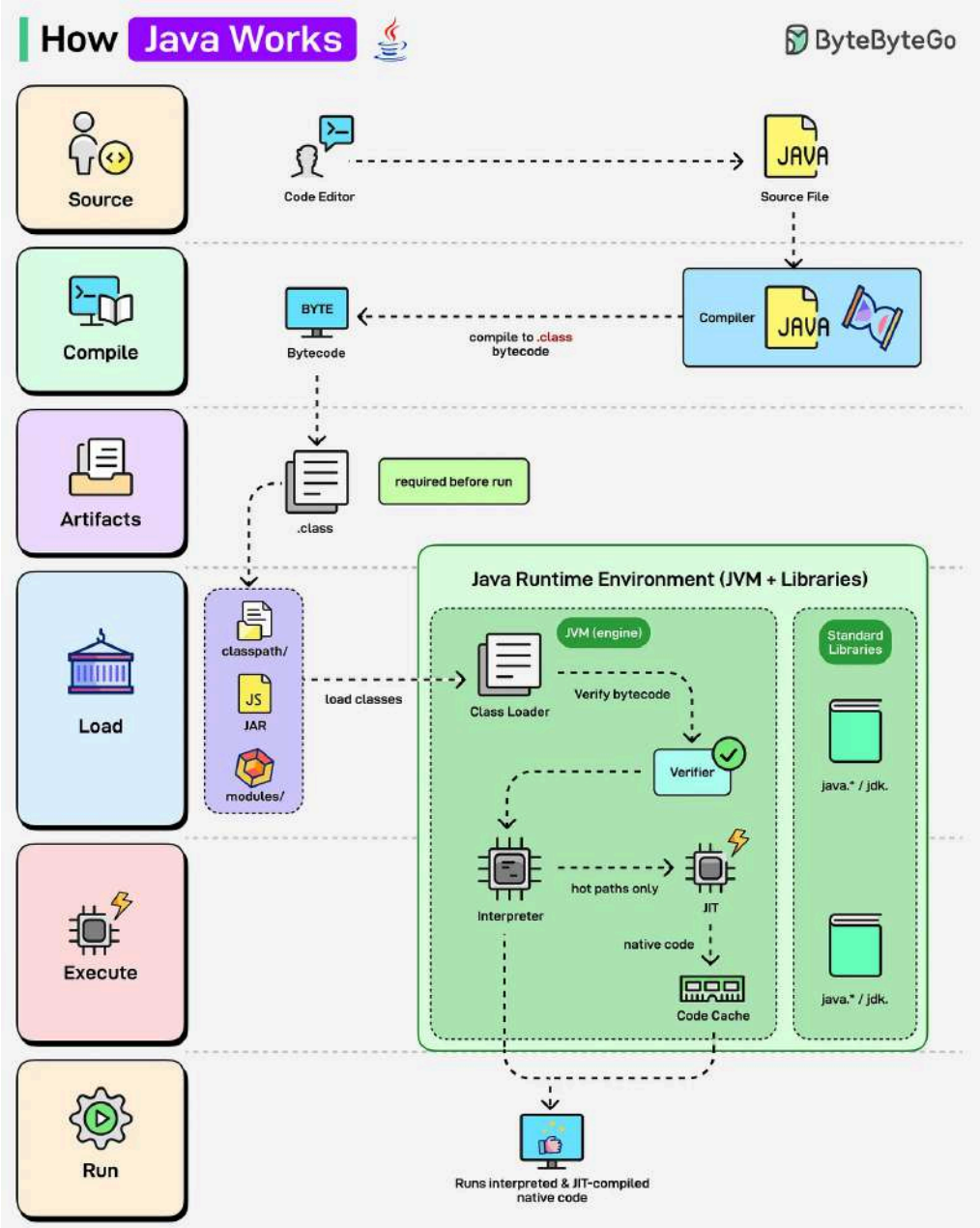


While load balancers, API gateways, and reverse proxies often overlap each other's functionalities, they also have specific roles that can be leveraged in a project.

1. A client request first hits the edge load balancer, which provides a single entry point and distributes traffic to the API Gateway.

2. The API Gateway takes over and performs initial parameter validations to ensure the request is well-formed.
3. Next, the whitelist verification checks if the source is trusted and allowed to access the APIs.
4. Authentication and authorization validate the identity of the requester and their permissions.
5. Rate limiting ensures the client is not overwhelming the system with too many requests.
6. Finally, the reverse proxy inside the gateway forwards the request to the correct service endpoint.
7. At the service layer, another load balancer distributes the request across multiple instances of the target microservice.
8. The chosen service instance processes the request and returns the response to the client.

How Java Works



Ever wondered what happens behind the scenes when you run a Java program? Let’s find out:

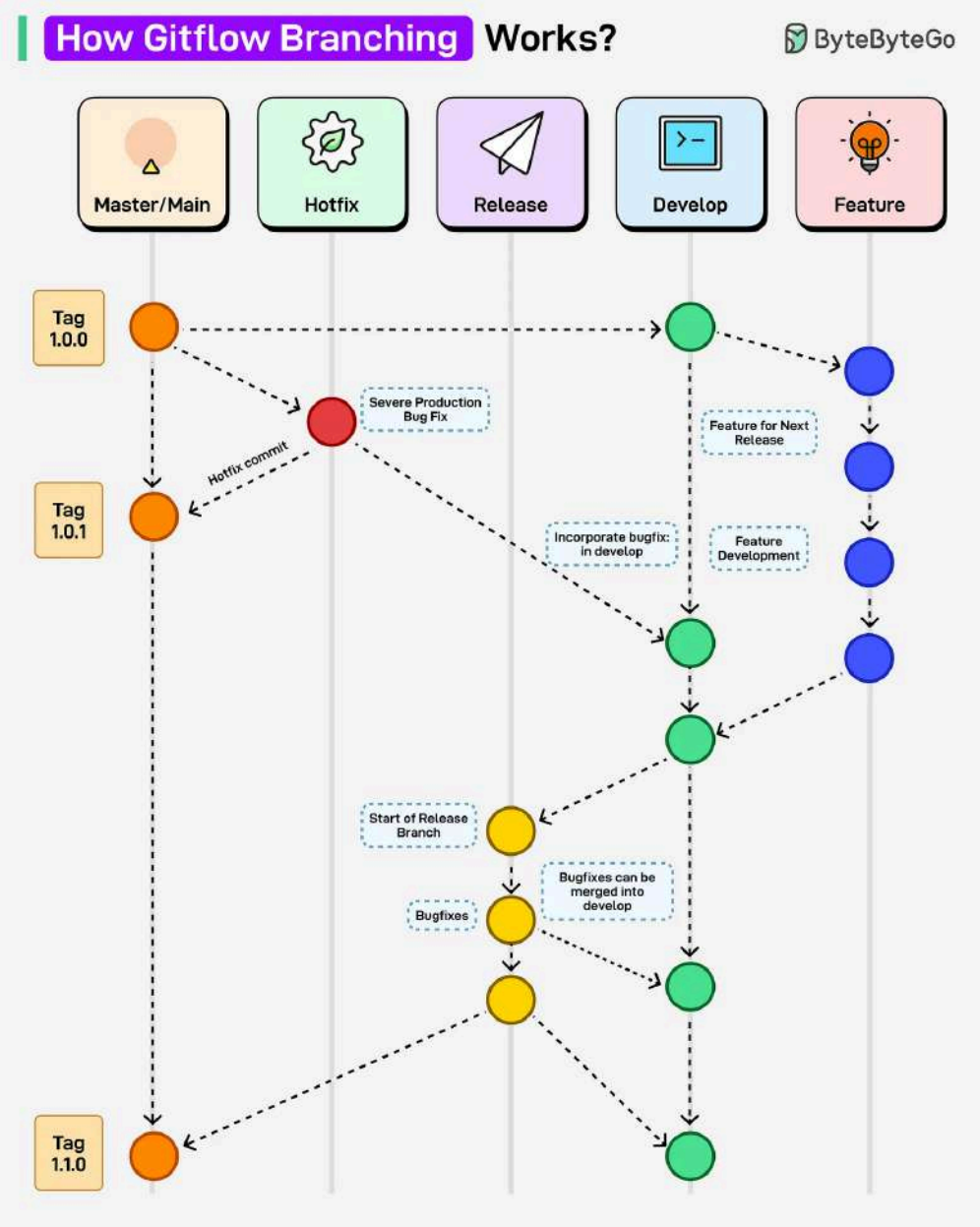
Java (JVM Runtime):

- Java source code (.java) is compiled into .class bytecode using javac.
- The Class Loader loads bytecode into the Java Runtime Environment (JVM).
- Bytecode is verified and executed.

- JVM uses both an Interpreter and a JIT Compiler, frequently used code (hot paths) is converted into native machine code, making Java faster.

How Gitflow Branching Works?

Gitflow branching strategy is a Git workflow that organizes development into dedicated branches for features, releases, hotfixes, and the main production line. It is an ideal Git branching strategy for projects with regular versioned deployments.



Here's how it works:

1. Development starts on the develop branch, where new features are integrated.

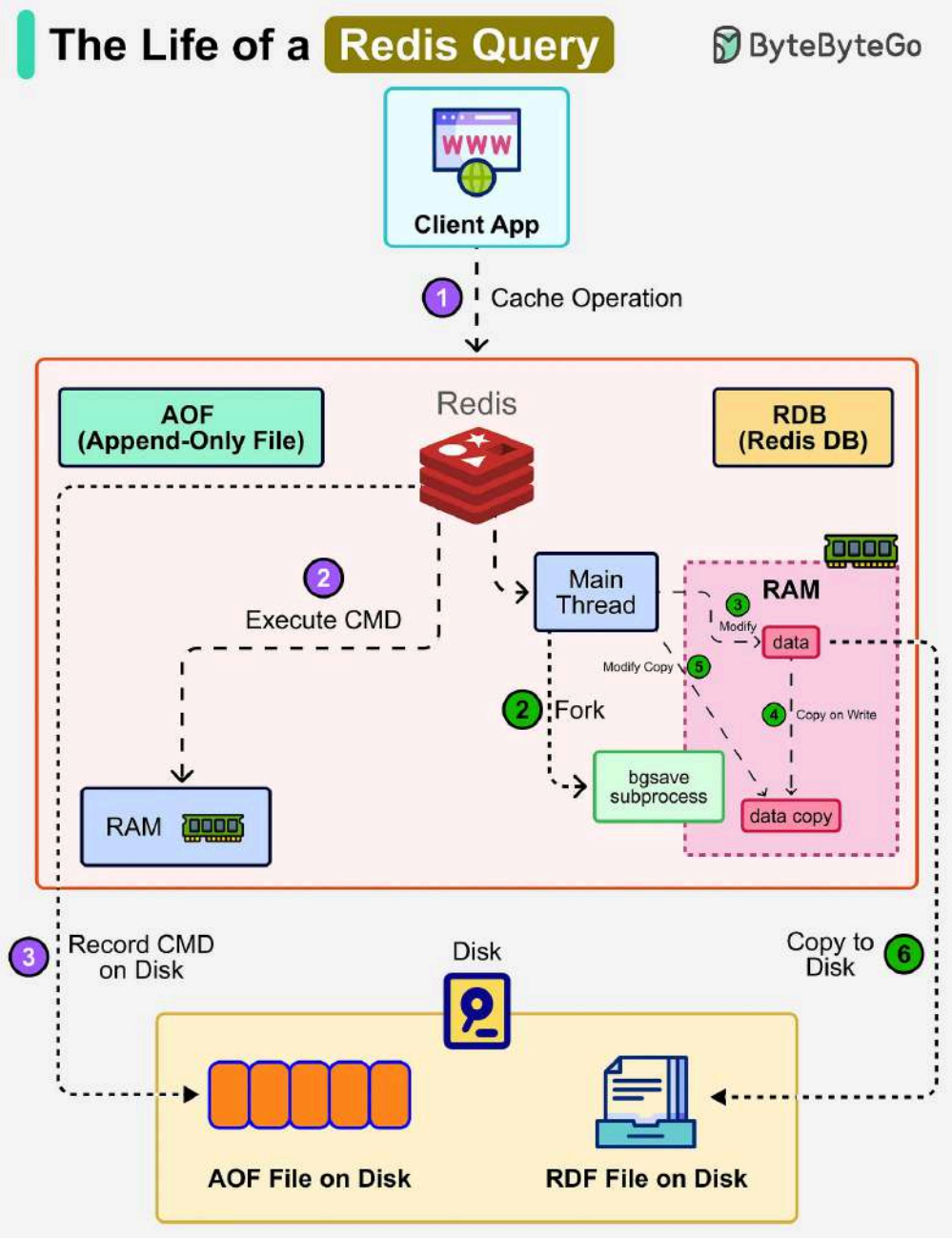
2. Each feature branch is created from develop and merged back once the feature is complete.

3. When preparing for a release, a release branch is created from develop for final bug fixes.

4. Release branches are merged into main (for production) and back into develop to keep a consistent history.

5. For urgent fixes, a hotfix branch is created directly from main, then merged back into both main and develop.

The Life of a Redis Query



Redis is an in-memory database, which means all data lives in RAM for speed. However, if the server crashes or restarts, data could be lost. To solve this problem, Redis provides two persistence mechanisms to write data to disk:

1. AOF (Append-Only File)
   When a client sends a command, Redis first executes it in memory (RAM). After that, Redis logs the command by appending it to an AOF file on disk. This ensures every operation can be replayed later to rebuild the dataset. Since the command is executed first and logged

afterward, writes are non-blocking. The recovery process uses the event log to replay the recorded commands.

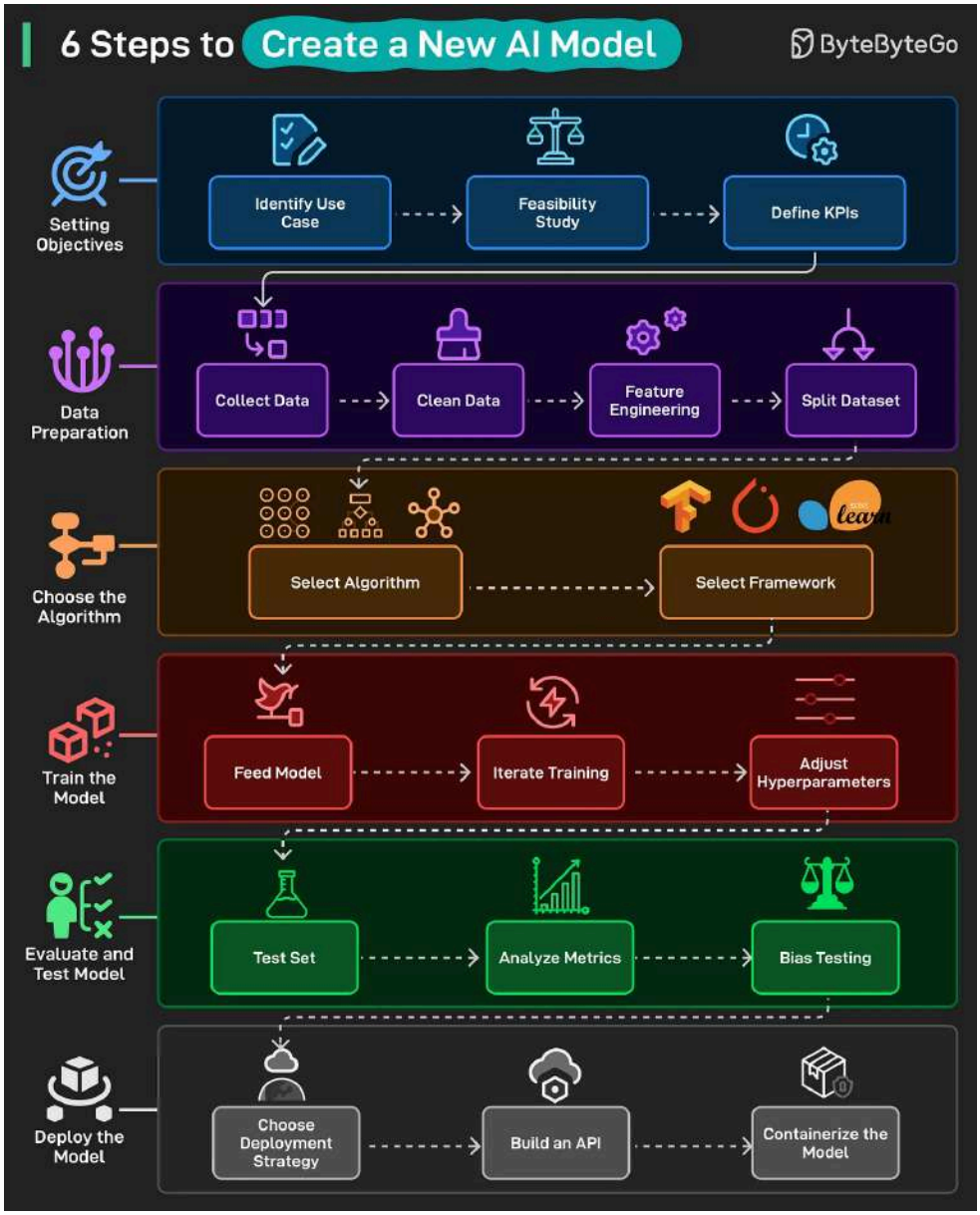
2. RDB (Redis Database)
   Instead of writing every command, Redis can periodically take snapshots of the entire dataset.

   The main thread forks a subprocess (bgsave) that shares all the in-memory data of the main thread. The bgsave subprocess reads the data from the main thread and writes it to the RDB file.

   Redis uses copy-on-write. When the main thread modifies data, a copy of the data is created, and the process works on that so that writes don't get blocked. The snapshot is then written as an RDB file on disk, allowing Redis to quickly reload the snapshot into memory when needed.

3. Mixed Approach
   In production, Redis often uses both AOF and RDB. RDB provides fast reloads with compact snapshots. AOF guarantees durability by recording every operation since the last snapshot.

6 Steps to Create a New AI Model

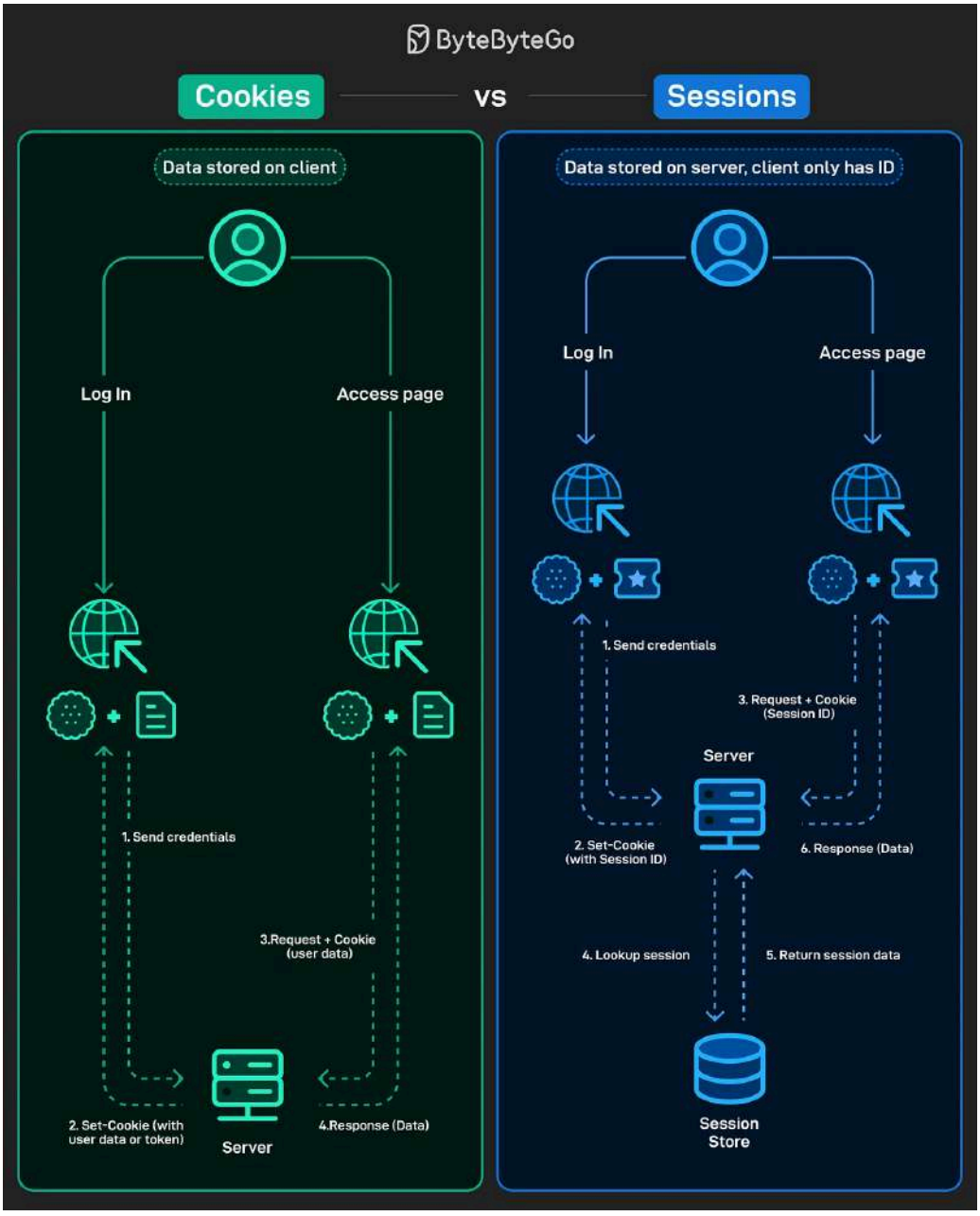


1. Setting Objectives: Define the problem to be solved by the AI model by identifying use cases, checking feasibility, and setting clear KPIs for success.
2. Data Preparation: Gather and clean raw data, engineer useful features, and split the data into training, validation, and test sets.
3. Choose the Algorithm: Pick the right algorithm for your problem and select a framework (for example, TensorFlow, PyTorch, Sci-kit Learn.
4. Train the Model: Feed data into the model, iterate training, and tune hyperparameters until performance improves.

5. Evaluate and Test Model: Test on unseen data, analyze performance metrics, and check for bias or unfair outcomes.
6. Deploy the Model: Select a deployment strategy, build an API, and containerize the model for production use.

Cookies vs Sessions

Every web app needs a way to remember users after they log in, and the way they manage this makes a big difference in performance, scalability, and security of that web application.



Cookies:

- Store data directly on the client (like user info or tokens).
- Each request sends the cookie back to the server.

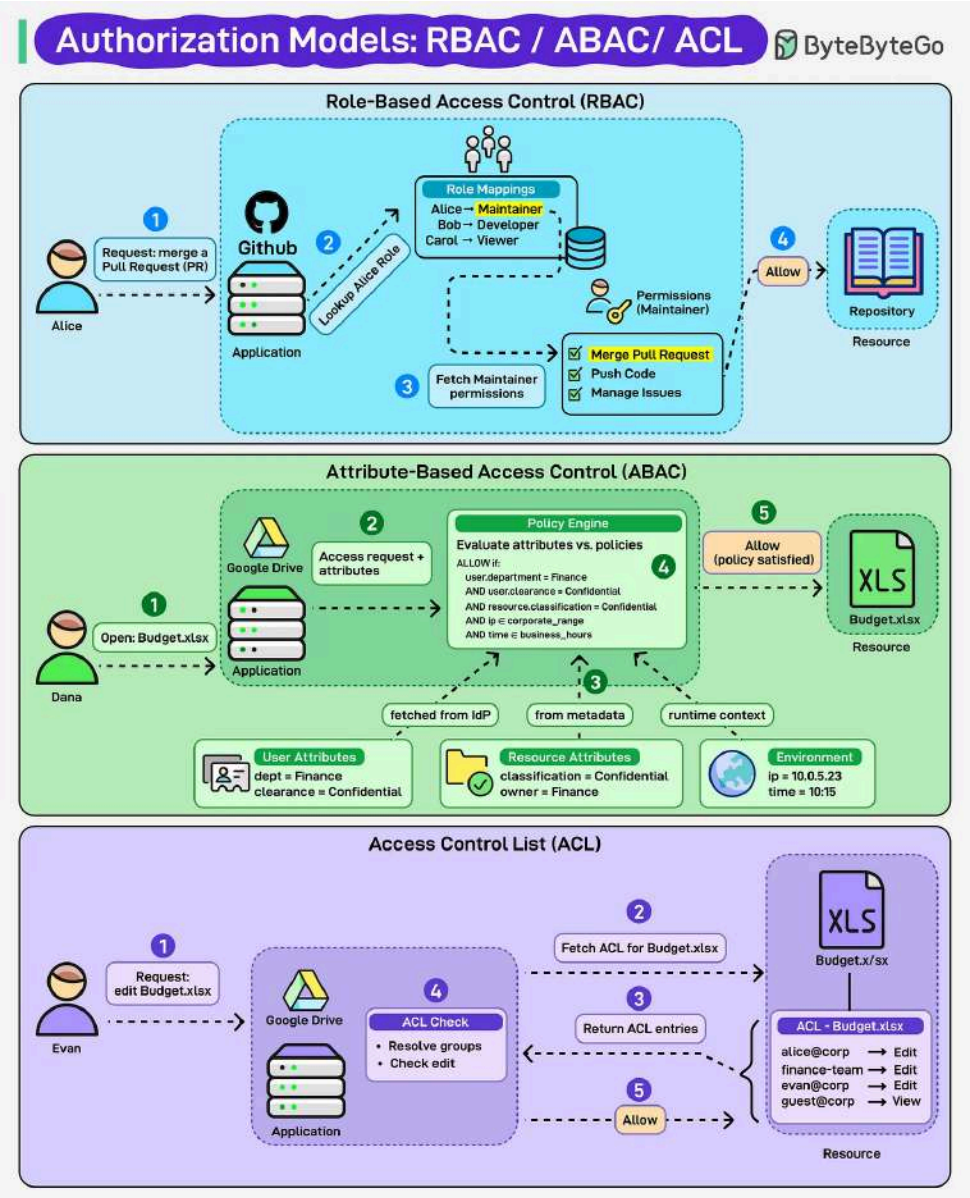
- Simpler to implement, but can expose data if not secured properly.

Sessions:

- Store data on the server; the client only keeps a Session ID in a cookie.
- Each request uses the Session ID, and the server looks up the data in its session store.
- More secure, but requires extra server-side storage and management.

Access Control Clearly Explained

Access control decides who gets in and who's locked out — but the rules vary.

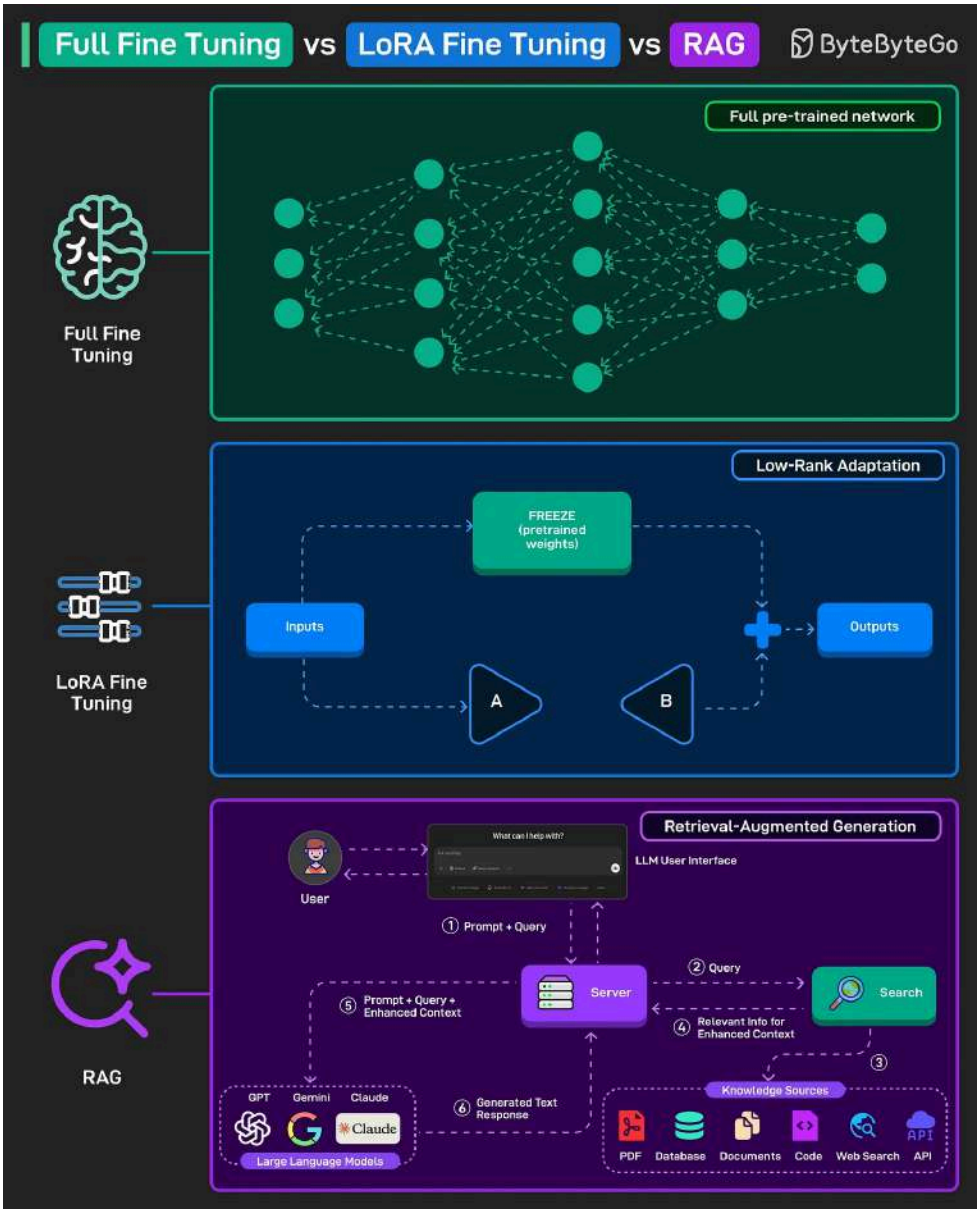


- Role-Based Access Control (RBAC): Access based on roles (e.g., Maintainer, Viewer). Simple, scalable.
- Attribute-Based Access Control (ABAC): Access based on attributes (user, resource, environment). Flexible, but complex.
- Access Control List (ACL): Explicit permissions for each user or group. Direct, but hard to manage at scale.

RBAC = roles.
ABAC = attributes.
ACL = explicit permissions.

Full Fine-Tuning vs LoRA vs RAG

All three are different ways of adapting a large pre-trained language model to new tasks or knowledge.



1. Full Fine-Tuning
   The entire pre-trained model is retrained on new data. It means that all the weights of the network are updated, which gives high accuracy but requires more computing power and storage.

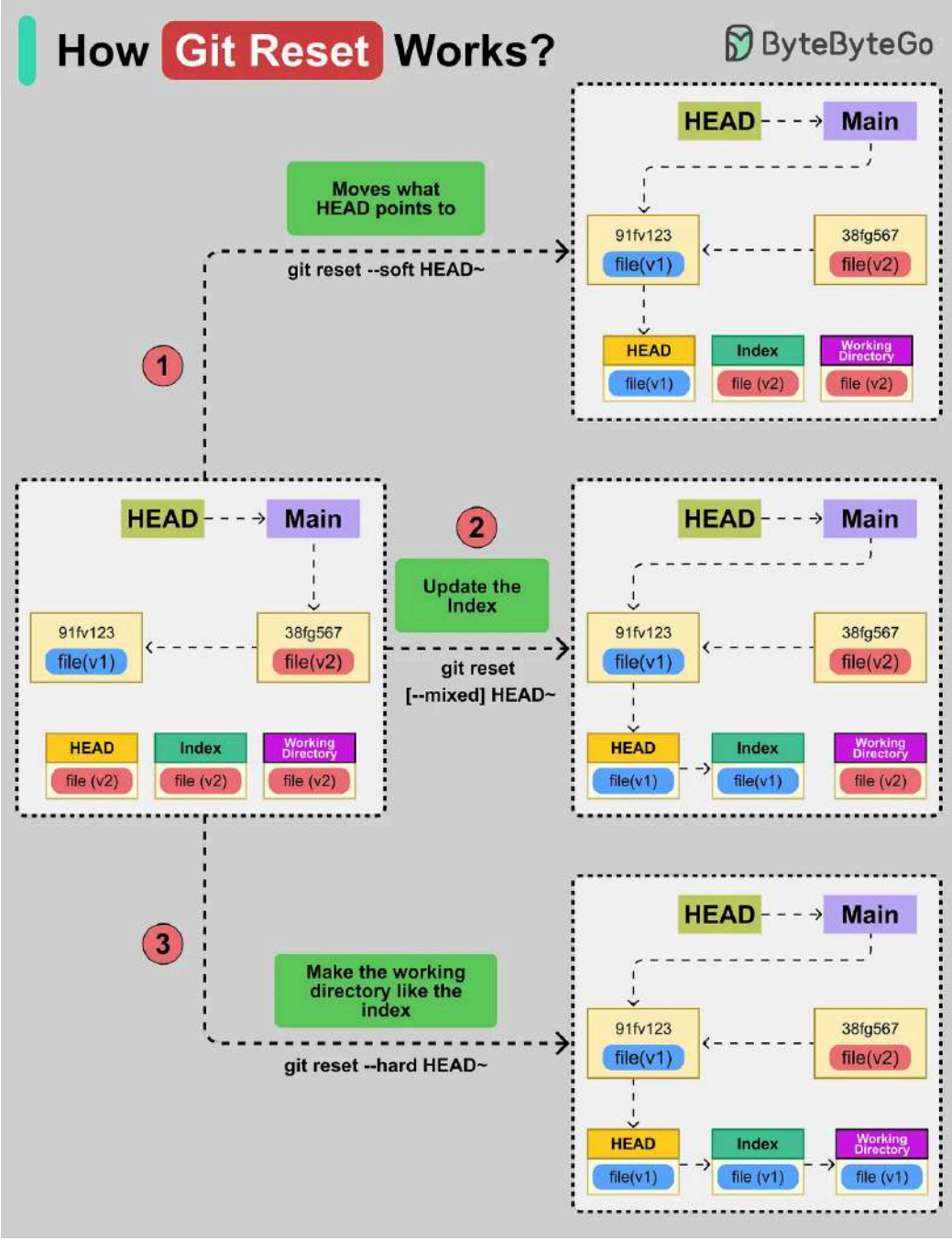
2. LoRA Fine-Tuning
   LoRA stands for Low-Rank Adaptation. Instead of changing the whole model, the main

weights are frozen, and only small additional layers are trained. This is much faster and cheaper than full fine-tuning while still adapting the model to new tasks.

3. RAG
   RAG stands for retrieval-augmented generation. The model doesn't retrain. Instead, it retrieves information from external sources (databases, web, documents, etc) whenever needed. The query is enhanced with relevant context before being passed to the model, making it more accurate and up-to-date without heavy retraining.

How Git Reset Works?

Git Reset moves your current git branch (HEAD) to a different commit and can make the index and working directory match it.



There are three options available with the ‘git reset’ command:

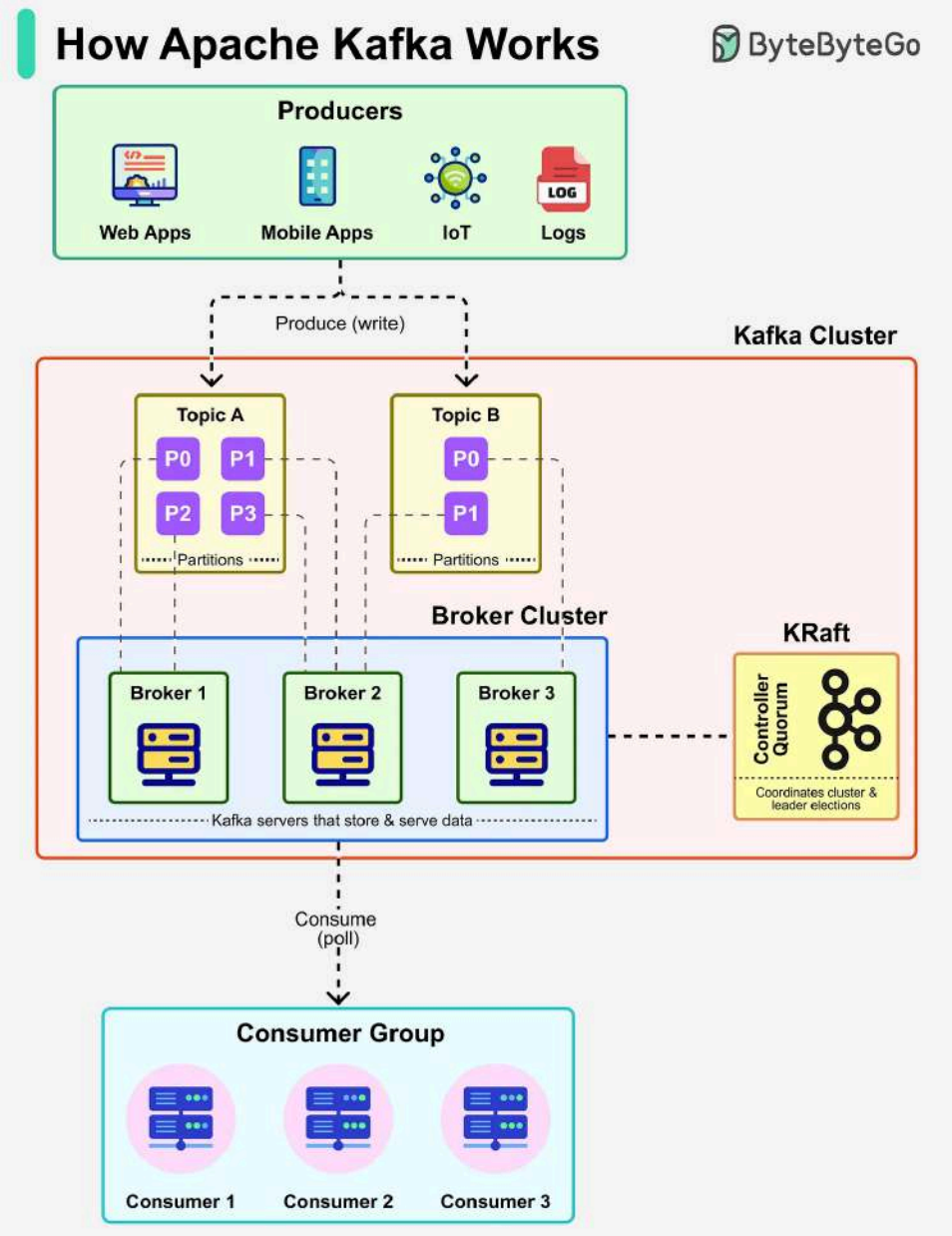
1. Move HEAD (--soft)
   Moves what the HEAD points to. However, the index and working files stay the same. For example, the ‘git reset --soft HEAD~’ command moves the pointer back one commit.

2. Updating the Index (--mixed)
   This is the default option. It updates the index with the contents of whatever snapshot HEAD now points to, but leaves your working files alone.

3. Updating the Working Directory (--hard)
   This option moves the HEAD, resets the index, and makes the working directory match that commit. The command ‘git reset --hard HEAD~’ sets everything (pointer, index, files) to the older commit.

Apache Kafka Explained (At the high level)

From Netflix to Uber to LinkedIn, Apache Kafka is the backbone of their real-time data infrastructure. It is a distributed event streaming platform built to handle massive streams of data with low latency and high reliability.

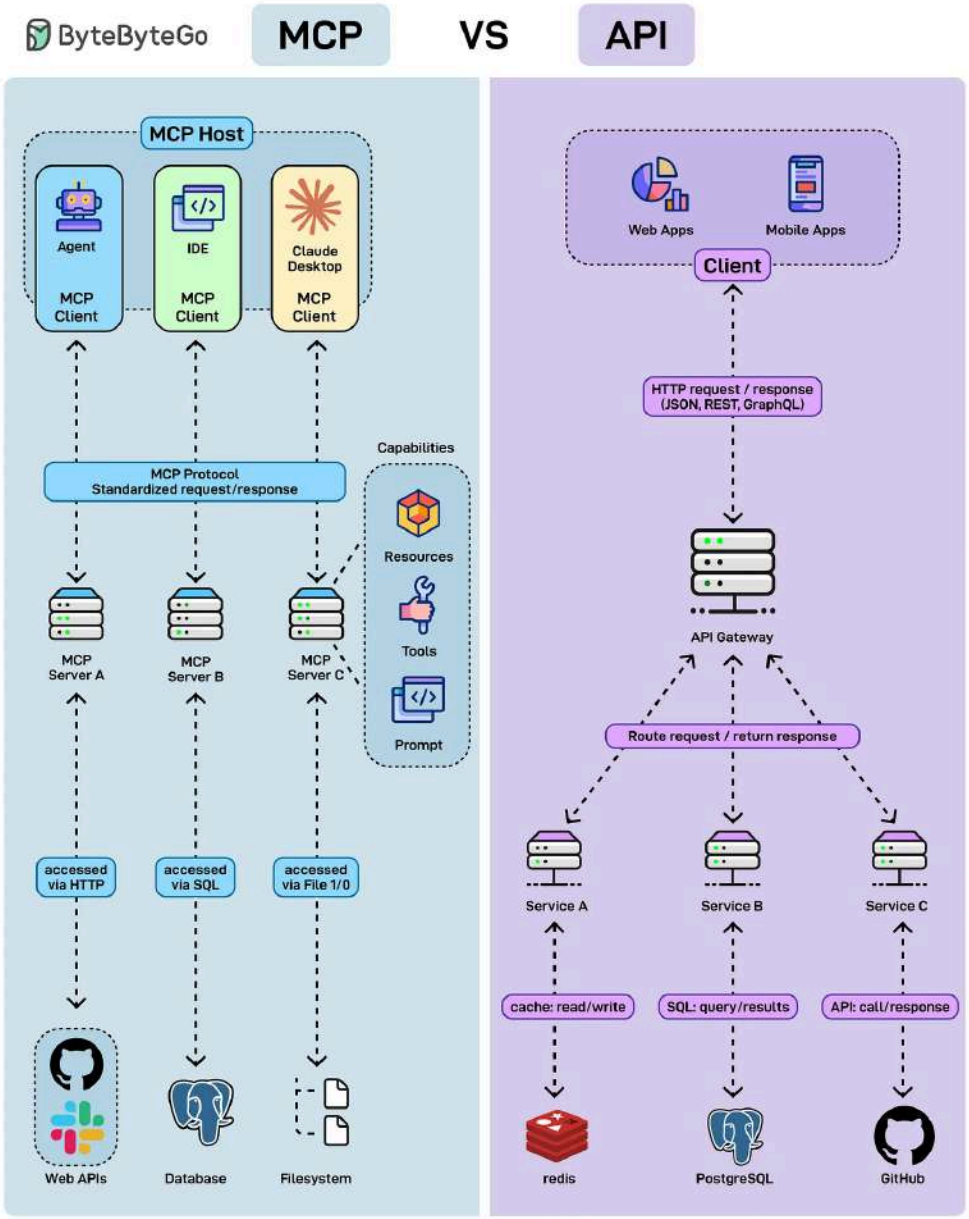


- Producers: Applications (web, mobile, IoT, logs, etc.) that publish messages to Kafka topics.
- Topics & Partitions: Messages are organized into topics, which are split into partitions for scalability and parallelism.

- Broker Cluster: Kafka brokers store and serve partitioned data. Multiple brokers form a cluster to ensure reliability and fault tolerance.
- KRaft (Controller Quorum): Coordinates cluster metadata and leader elections — ensuring the cluster remains consistent.
- Consumer Groups: Applications that subscribe to topics and consume messages.

MCP vs API: what's the difference?

APIs have been the backbone of software-to-software communication for decades. Now, a new player — the Model Context Protocol (MCP) — is emerging as an AI-native protocol designed for agents, IDEs, and LLMs.



API (Application Programming Interface):

- Purpose: Enables software-to-software communication.
- Discovery: Requires documentation.

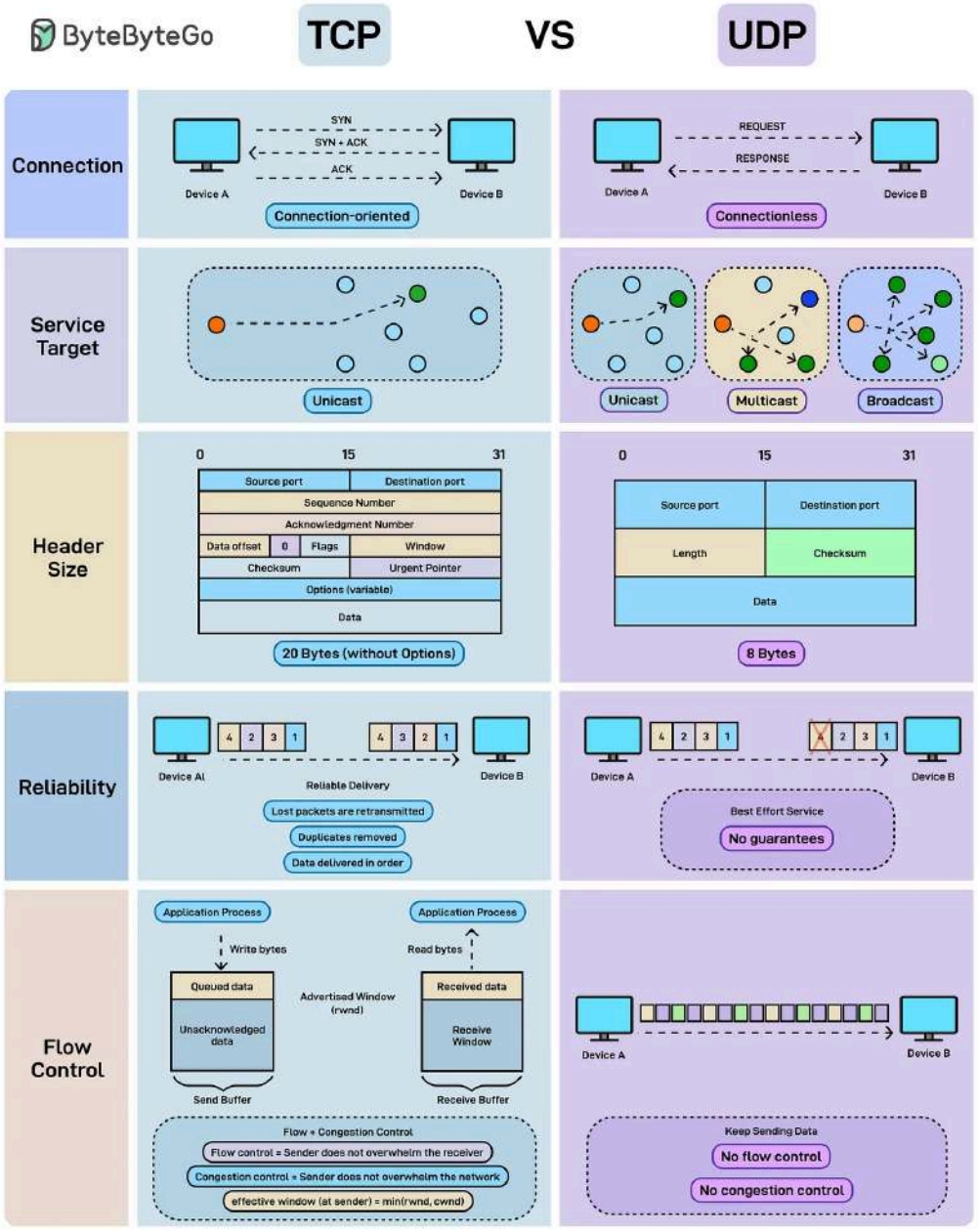
- Standardization: Varies — REST, GraphQL, gRPC, etc.

MCP (Model Context Protocol):

- Purpose: Enables AI-native communication between clients (agents, IDEs, LLMs) and servers.
- Discovery: Self-describing (no external docs needed).
- Standardization: One uniform protocol for resources, tools, and prompts.

TCP vs UDP

Every time data moves across the internet, it chooses between accuracy and speed. That's TCP vs UDP.

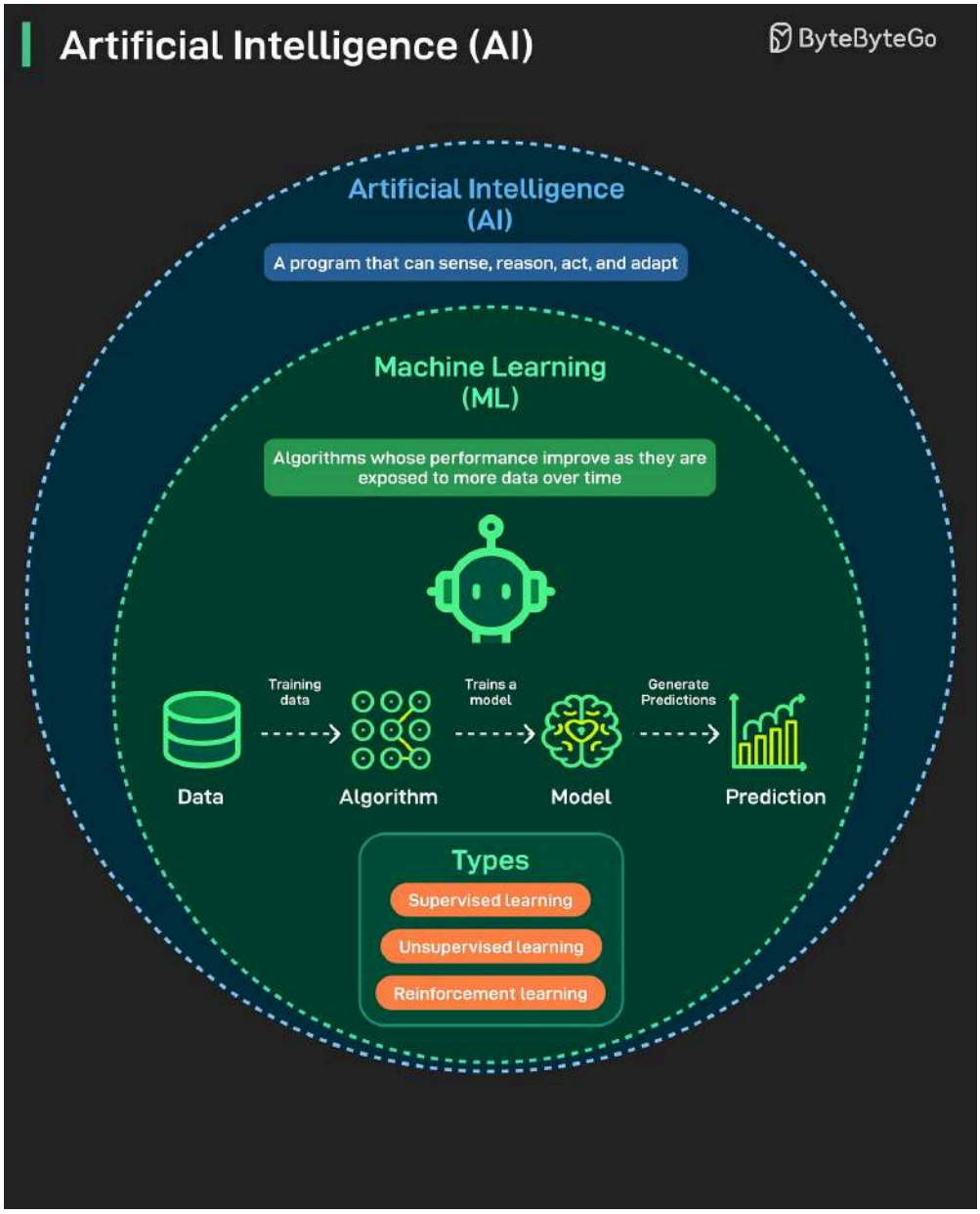


TCP: Connection-oriented and reliable. It ensures ordered, duplicate-free delivery with flow and congestion control, making it ideal for web browsing, email, and file transfers.

UDP: Connectionless and lightweight. It sends packets without guarantees of delivery or order, but with minimal overhead. It is perfect for gaming, streaming, and real-time communication.

A piece of knowledge each day: AI and Machine Learning (ML)

What is AI? What is ML? Are they the same thing? We will clear the common confusion in this post.



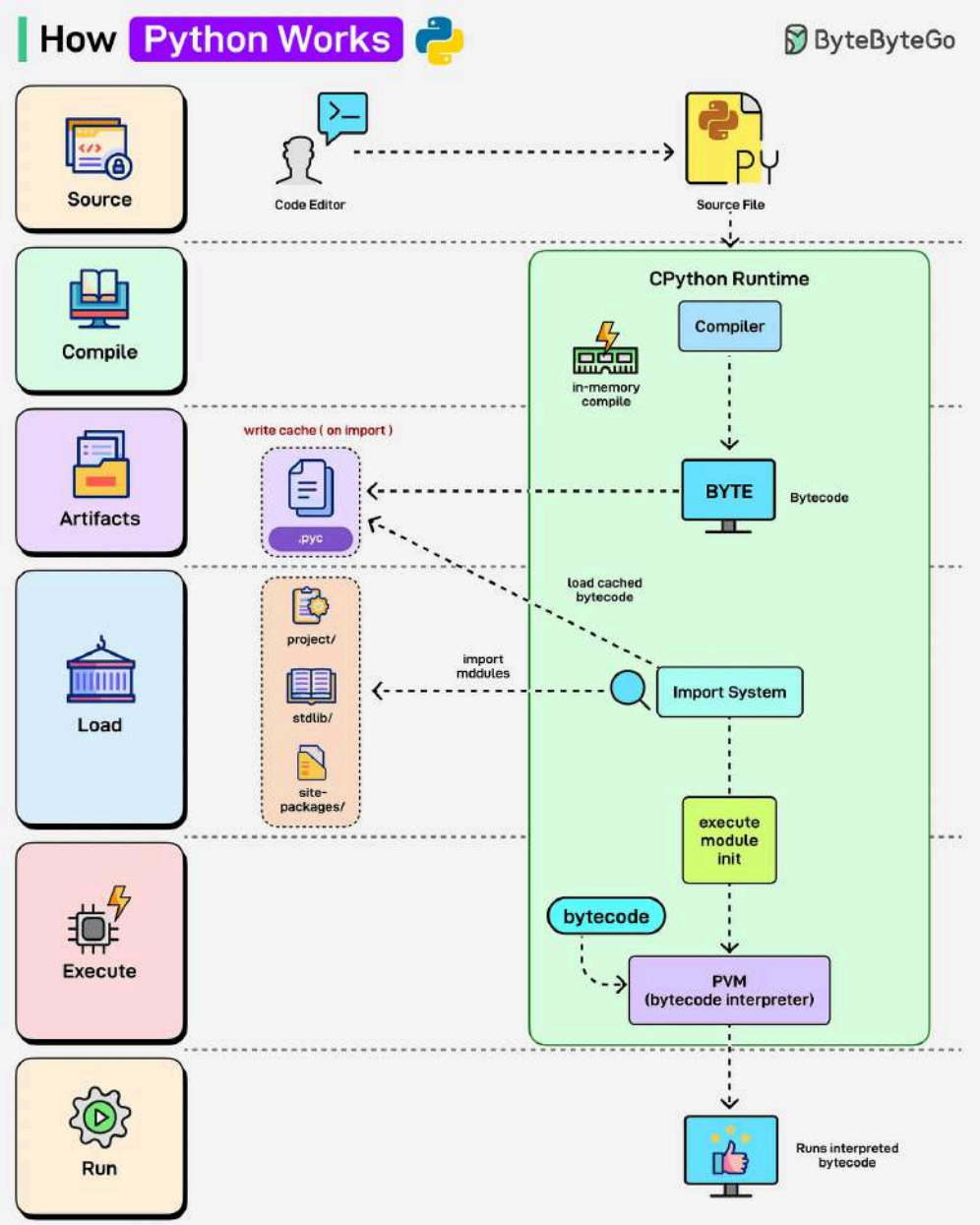
AI and ML are often treated as if they are the same thing. They are not. AI is the bigger field. It is about creating programs that can sense, reason, act, and adapt. Any system that shows intelligent behavior can fall under AI.

ML is a subset of AI. It focuses on algorithms that learn from data and improve with experience. This is where most of the progress in recent years has happened. Some common use cases of ML are

recommendation engines, fraud detection, and image recognition. Most of what we interact with daily.

The biggest breakthrough in recent years came from ML, and when the media talks about the AI revolution, they are mostly talking about advances in ML, particularly deep learning.

How Python Works



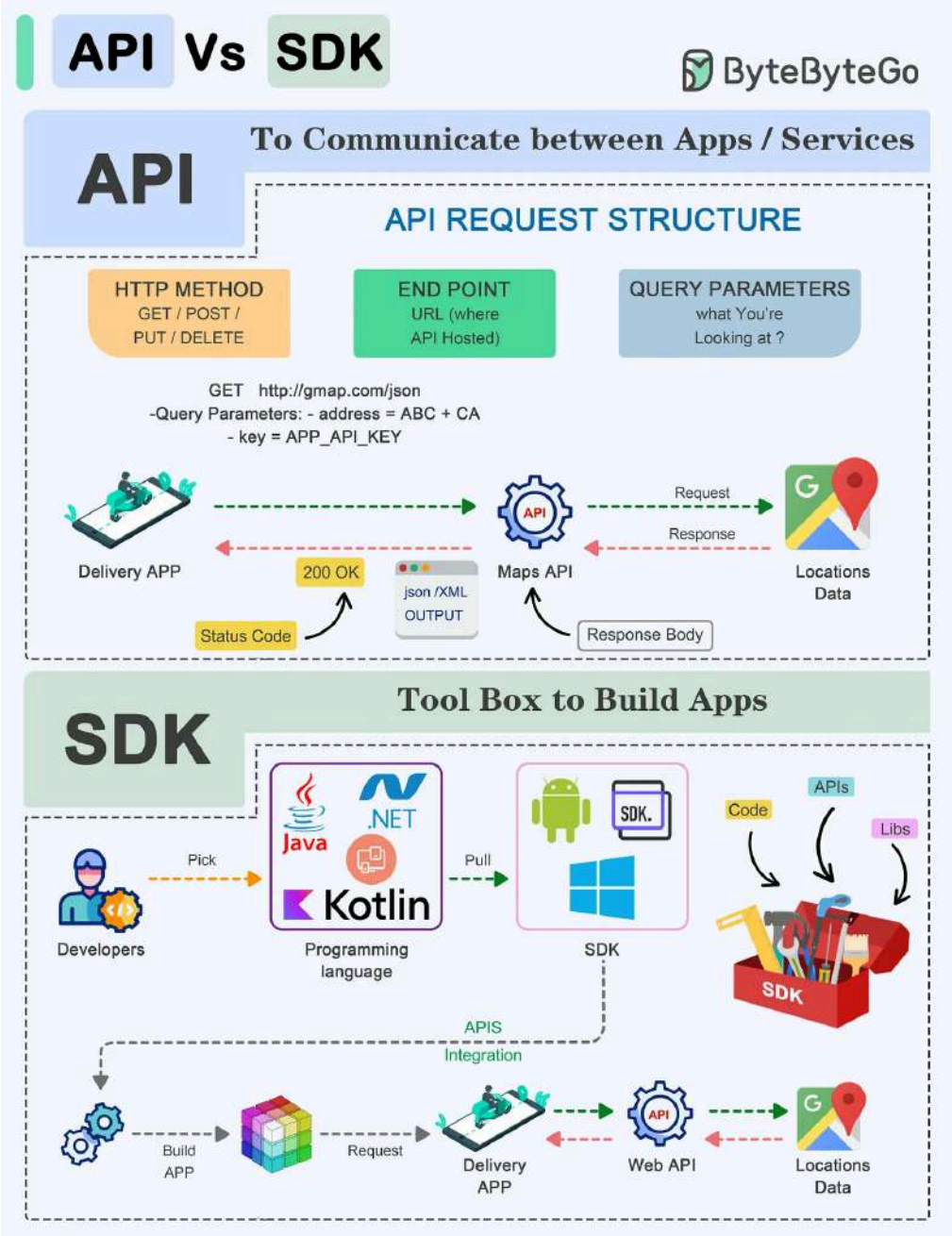
Ever wondered what happens behind the scenes when you run a Python script? Let's find out:

Python (CPython Runtime):

- Python source code (.py) is compiled into bytecode automatically in memory.
- Bytecode can also be cached in .pyc files, making re-runs faster by using the cached version.
- The Import System loads modules and dependencies.

- The Python Virtual Machine (PVM) interprets the bytecode line by line, making Python flexible but relatively slower.

API Vs SDK!



API (Application Programming Interface) and SDK (Software Development Kit) are essential tools in the software development world, but they serve distinct purposes:

API:
An API is a set of rules and protocols that allows different software applications and services to communicate with each other.

1. It defines how software components should interact.

2. Facilitates data exchange and functionality access between software components.

3. Typically consists of endpoints, requests, and responses.

SDK:
An SDK is a comprehensive package of tools, libraries, sample code, and documentation that assists developers in building applications for a particular platform, framework, or hardware.

1. Offers higher-level abstractions, simplifying development for a specific platform.

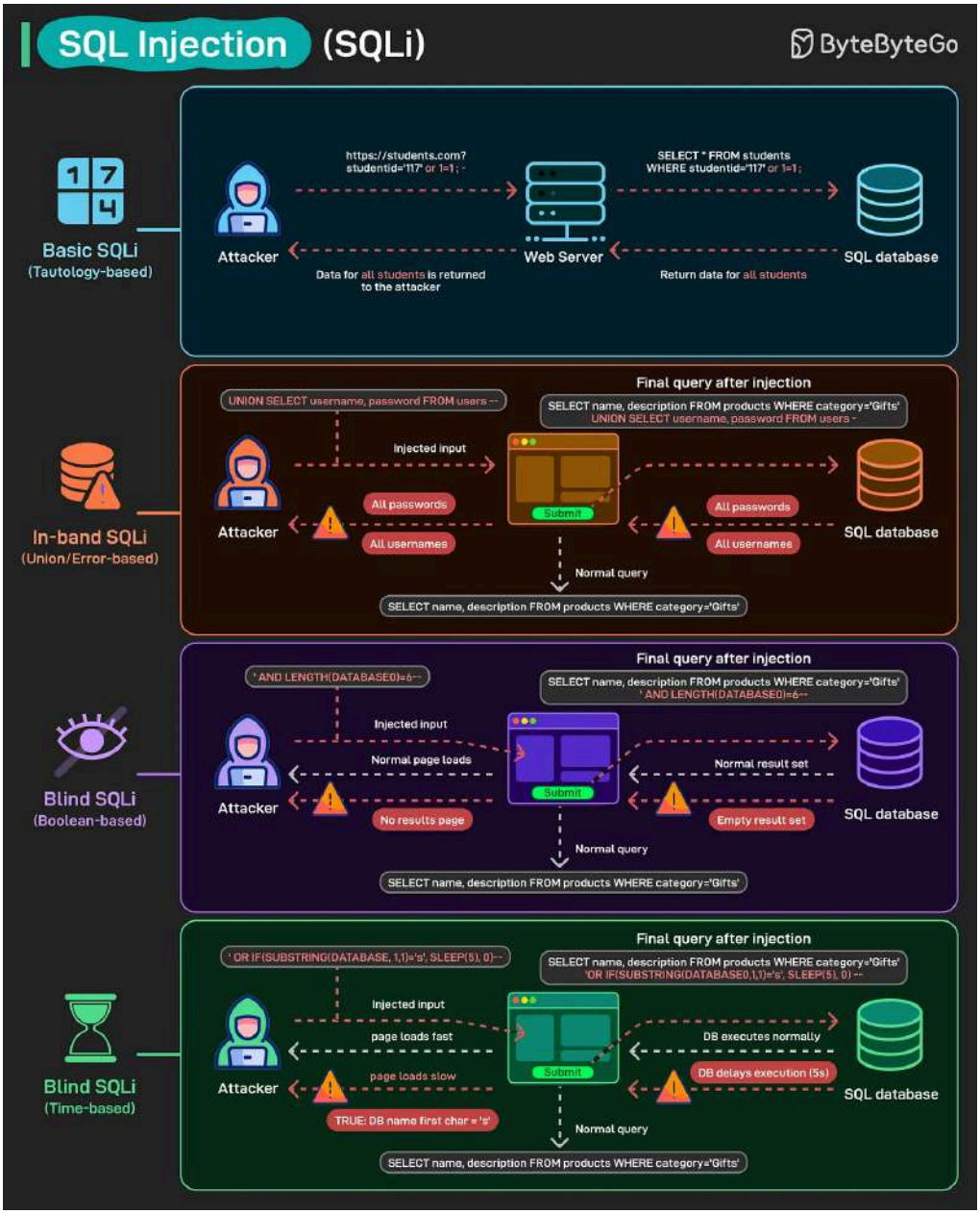
2. Tailored to specific platforms or frameworks, ensuring compatibility and optimal performance on that platform.

3. Offer access to advanced features and capabilities specific to the platform, which might be otherwise challenging to implement from scratch.

The choice between APIs and SDKs depends on the development goals and requirements of the project.

SQL Injection (SQLi)

SQL Injection is one of the oldest and most dangerous web vulnerabilities. With just a few crafted inputs, attackers can manipulate database queries and gain access to sensitive data.



Basic SQLi (Tautology-based): Attackers inject conditions like 1=1 to bypass authentication and retrieve all records.

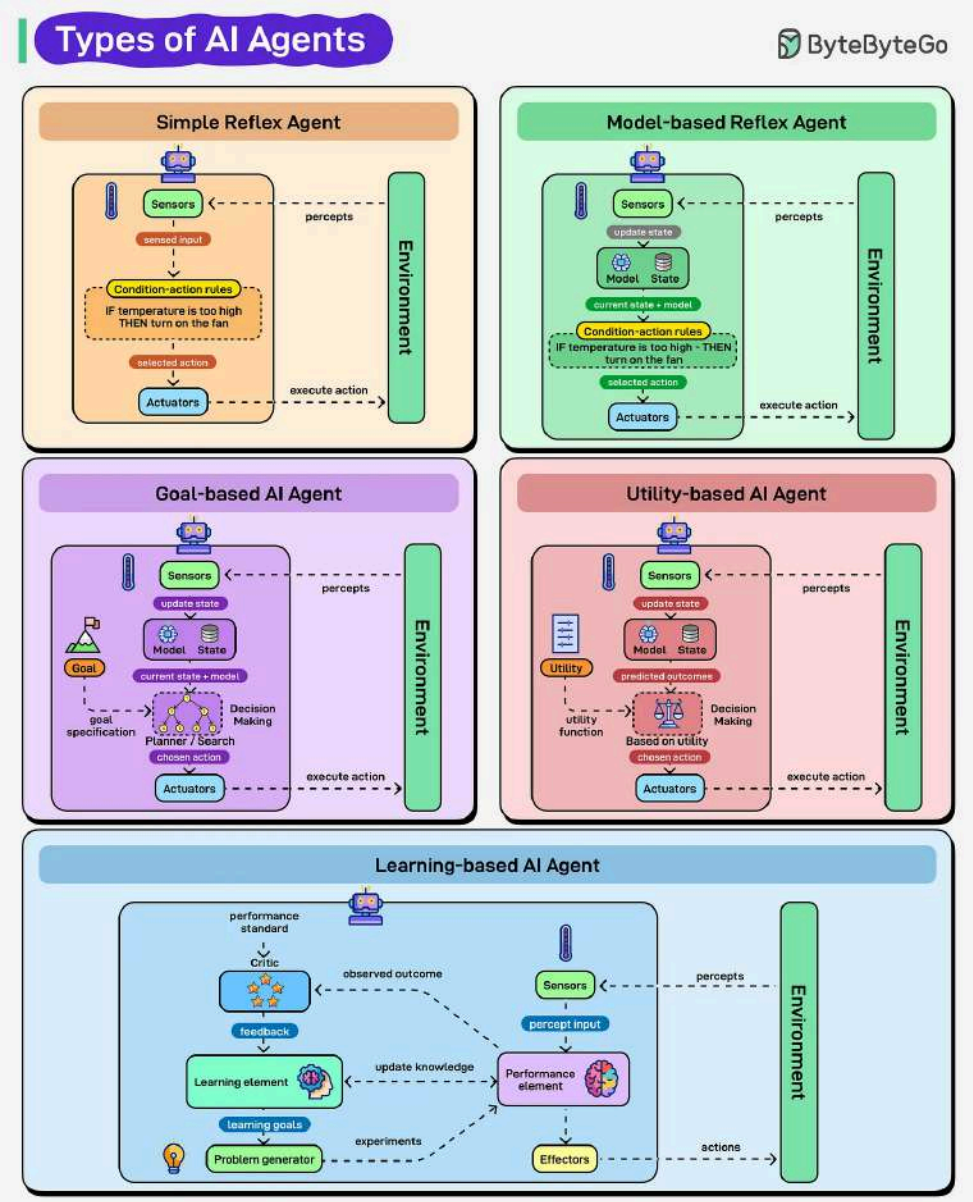
In-band SQLi (Union/Error-based): Attackers use UNION SELECT or leverage error messages to extract usernames, passwords, or other sensitive data directly.

Blind SQLi (Boolean-based): No direct output is shown, but attackers infer database information based on whether a page returns results or not.

Blind SQLi (Time-based): Attackers use commands like SLEEP(5) to measure server response delays and extract data incrementally.

Types of AI Agents

AI agents don’t all think and act in the same way. They range from simple rule-followers to systems that learn and adapt. Each type marks a step forward in how machines perceive, decide, and act.

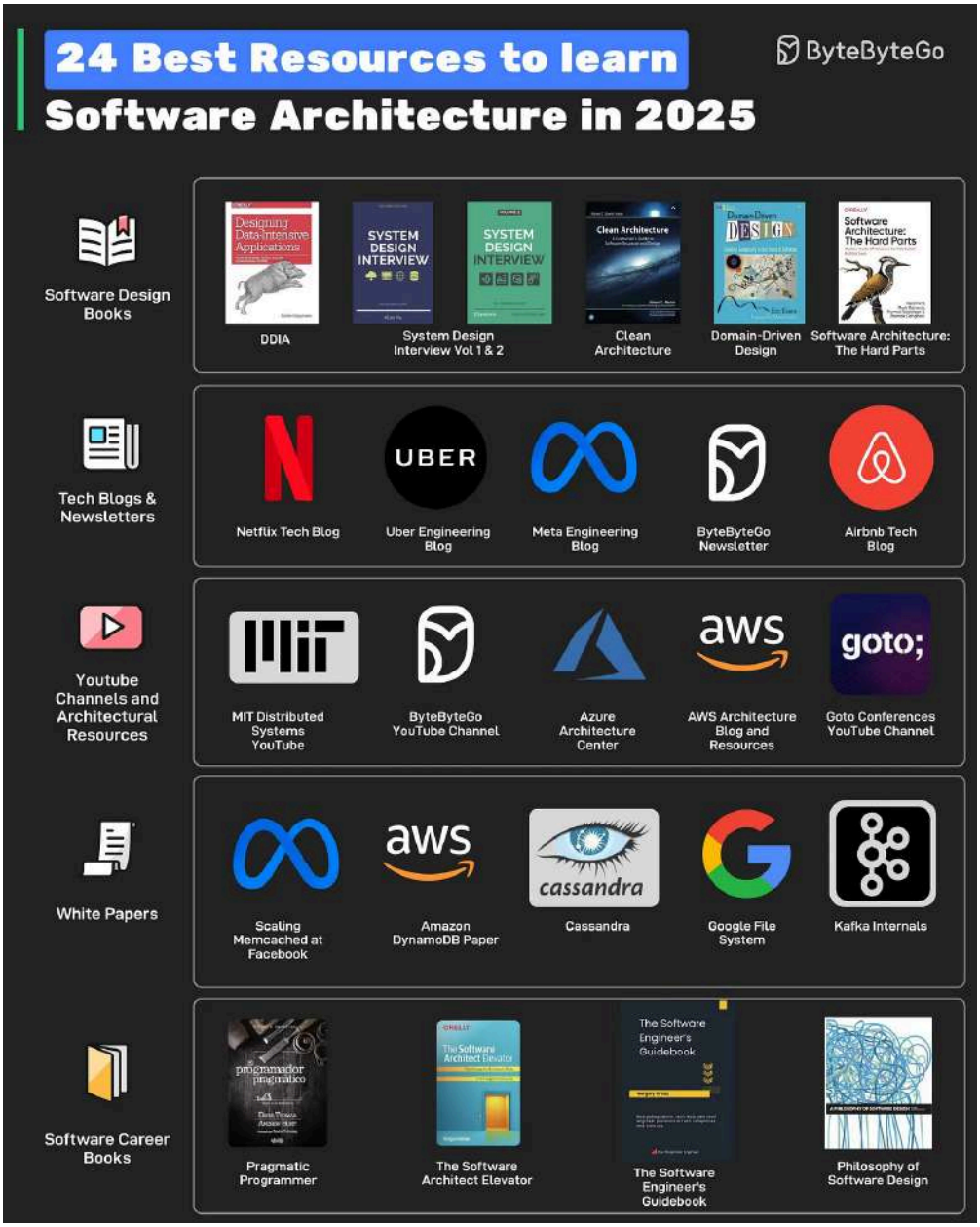


1. Simple Reflex Agents: These follow condition–action rules. For example, if the temperature is high, turn on the fan. No memory, no thinking, just instant reaction. They are fast and simple.

2. Model-based Reflex Agents: These maintain an internal understanding of their environment. They are not just reacting to immediate inputs, they have a model that helps them make sense of what is happening beyond what they can see right now.

3. Goal-based Agents: Here, the focus shifts to goals. Decisions are made based on whether an action brings the agent closer to its objective.
4. Utility-based Agents: These go a step further by weighing different outcomes. They choose the action that offers the best overall result, balancing trade-offs along the way.
5. Learning Agents: These are the most advanced. They improve continuously, using feedback to adapt and perform better over time.

24 Good Resources to Learn Software Architecture in 2025



The resources can be divided into different types such as:

1. Software Design Books
   Some books that can help are DDIA, System Design Volume 1 & 2, Clean Architecture, Domain-Driven Design, and Software Architecture: the Hard Parts

2. Tech Blogs and Newsletters
   Read technical blogs by companies like Netflix, Uber, Meta, and Airbnb. Also, the ByteByteGo newsletter provides insights into software design every week.

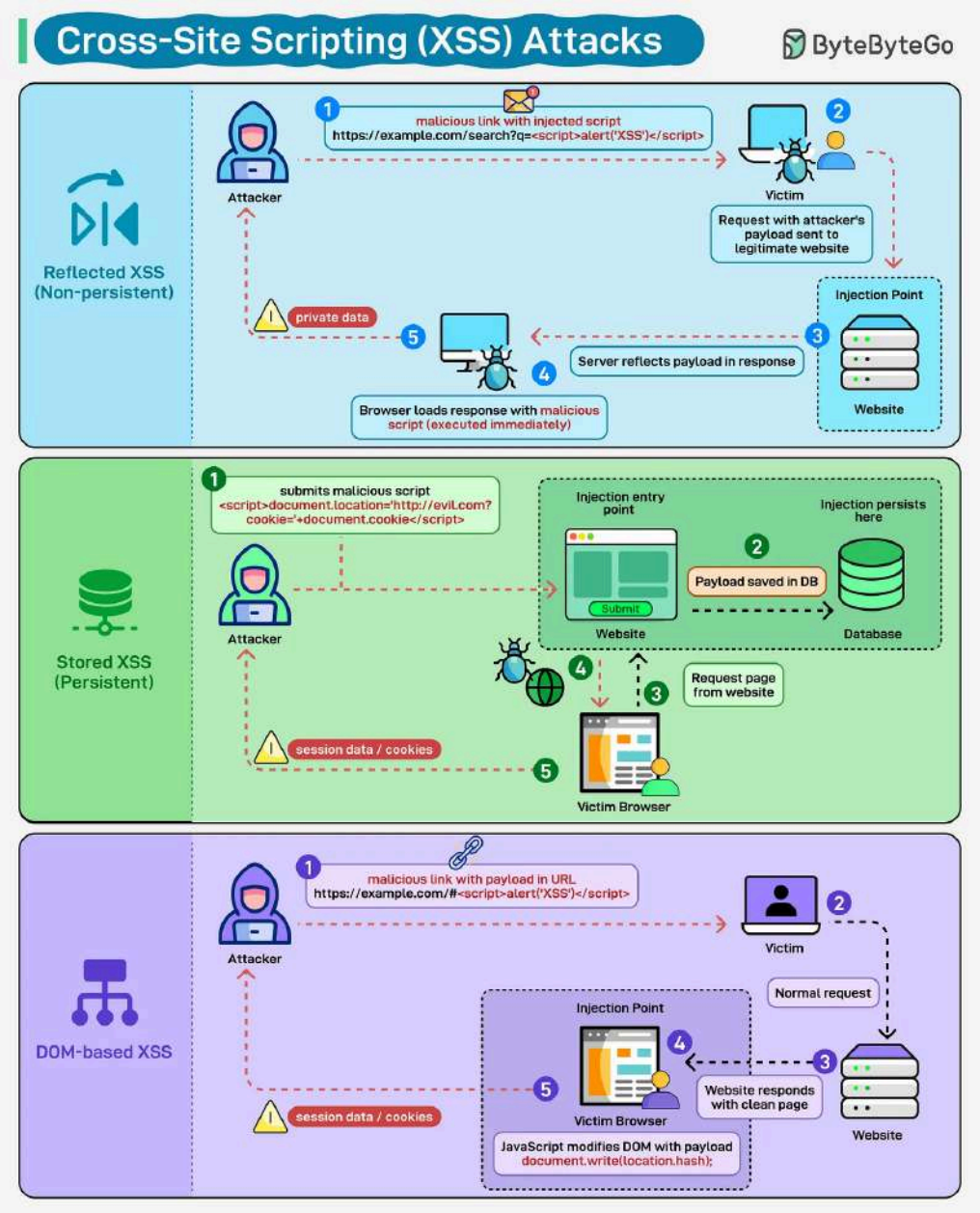
3. YouTube Channels and Architectural Resources
   YouTube channels like MIT Distributed Systems, Goto Conferences, and ByteByteGo can help with software architecture and system design. Azure Architecture Center and AWS Architecture Blog are other important resources.

4. WhitePapers
   For deeper insights, read whitepapers like Facebook Memcache Scaling, Cassandra, Amazon DynamoDB, Kafka, and Google File System.

5. Software Career Books
   A Software Architect also needs to develop holistic skills. Books about software career aspects such as Pragmatic Programmer, The Software Architect Elevator, The Software Engineer's Guidebook, and Philosophy of Software Design can help.

Cross-Site Scripting (XSS) Attacks

A small script can cause big damage. XSS lets attackers inject malicious code into web pages and hijack user sessions, steal cookies, or manipulate the browser.

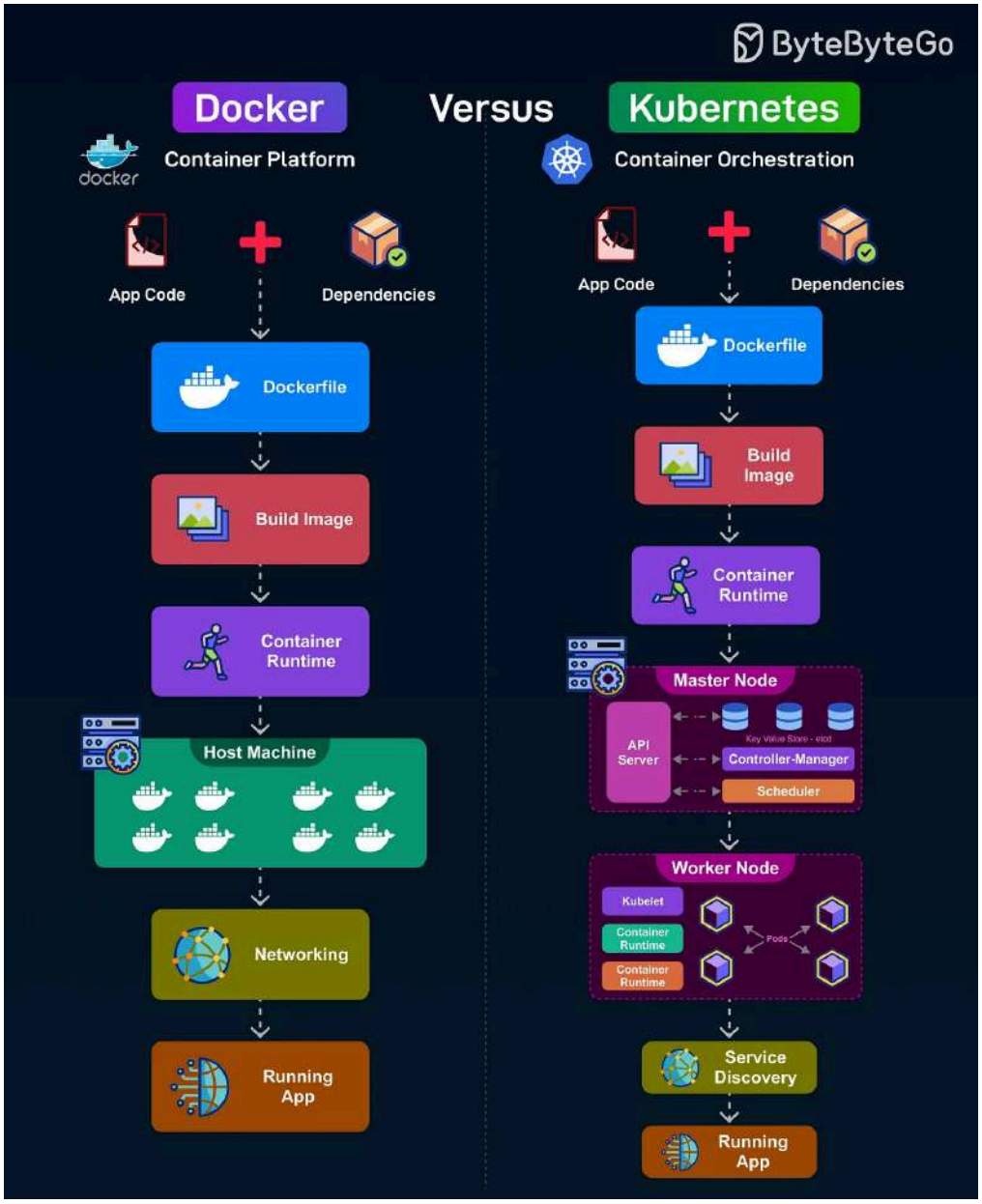


Reflected XSS: This type of attack happens when someone clicks a malicious link. The payload sits in the URL, gets reflected back in the response, and executes. It's often used in phishing campaigns because you need to trick someone into clicking.

Stored XSS: The malicious code gets saved in your database, maybe in a comment field or user profile. Then it runs automatically every time someone loads that page.

DOM-based XSS: The payload manipulates the DOM directly in the browser without ever hitting your server. Makes it harder to catch with traditional server-side validation.

Docker vs Kubernetes



Docker is a container platform that lets you package applications with their dependencies and run them on a single machine. Here's how it works:

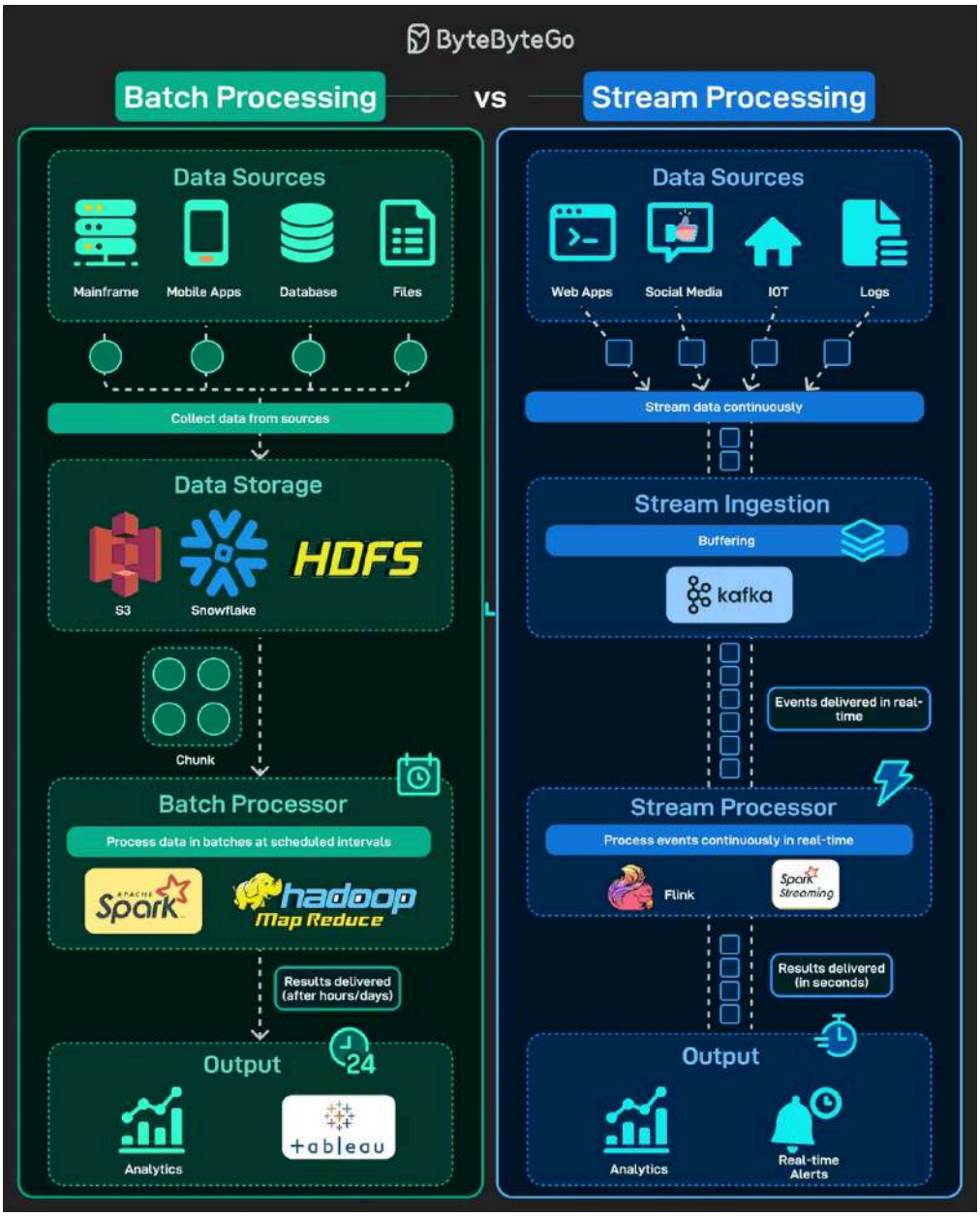
1. Starts with the app code and dependencies written into a Dockerfile.
2. The build image step creates a portable container image.
3. A container runtime runs the image directly on the host machine.
4. Networking connects containers and external services and produces the final running app.

Kubernetes is a container orchestration platform that manages containerized applications across a cluster of machines for scalability and resilience. Here's how it works:

1. Starts with the app code and dependencies in a Dockerfile.
2. The build image is passed to a container runtime supported by Kubernetes.
3. A master node runs the API server, etcd (key-value store), controller manager, and scheduler to coordinate the cluster.
4. Worker nodes run the actual containers inside the Pods, managed by Kubelet and kube-proxy for networking.
5. Produces a running app that is distributed, scalable, and self-healing in nature.

Batch vs Stream Processing

Data never stops flowing, but how we process it makes all the difference. When building data systems, two main approaches emerge: batch processing and stream processing. Both have their place, depending on whether you need accuracy over time or immediate insights.



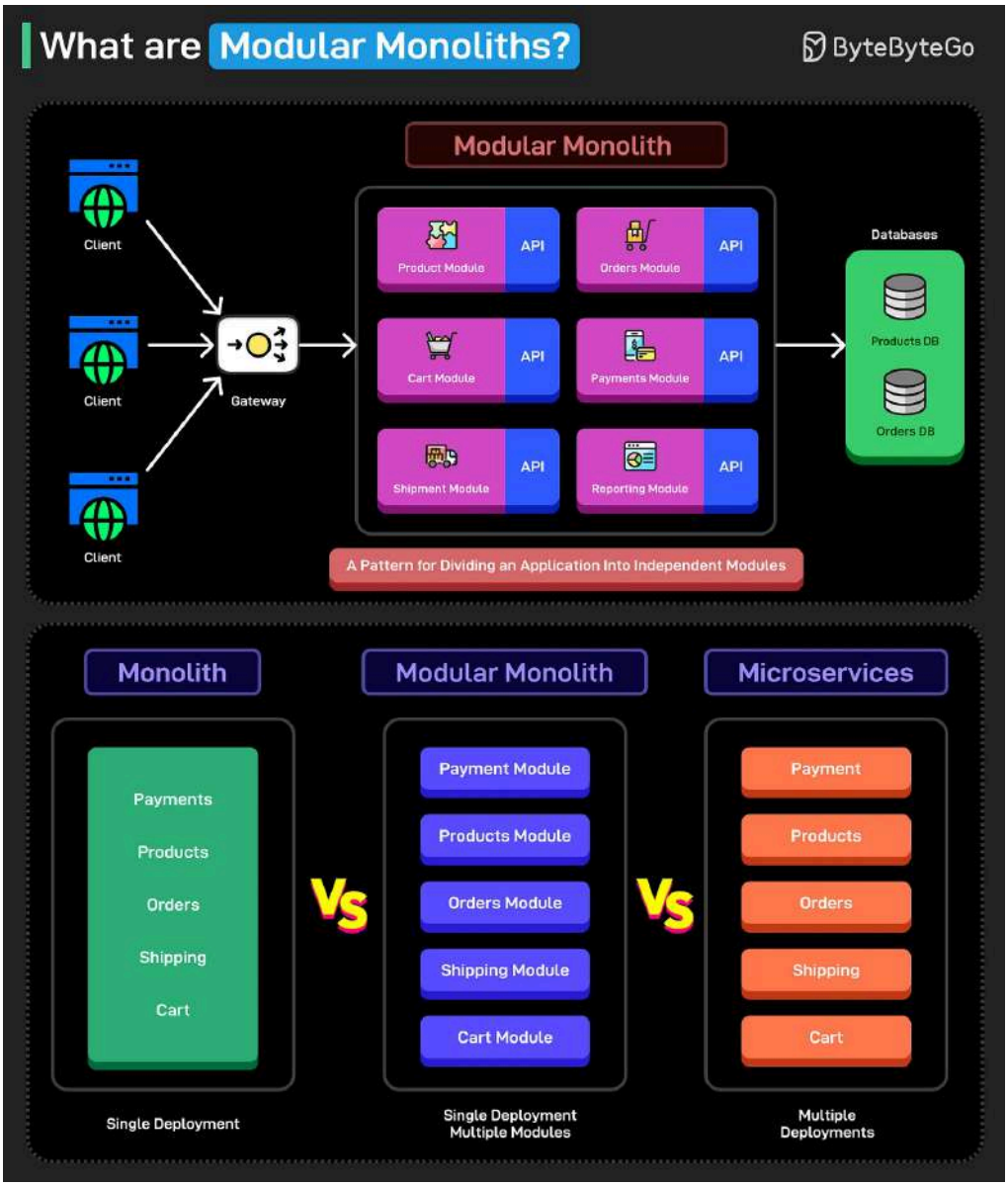
Batch Processing: Collects data in chunks and processes it at scheduled intervals, great for reports and historical analysis.

Stream Processing: Processes events continuously in real-time, powering dashboards, alerts, and recommendations.

Batch = high-volume, historical accuracy.
Stream = low-latency, real-time action.

What are Modular Monoliths?

A modular monolith is an architectural pattern that divides an application into independent modules or components. Each of these modules has well-defined boundaries that group related functionalities. Doing so results in better cohesion.



Here are the main characteristics of modular monoliths:

1. The modules are independent.
2. Each module provides a specific functionality.
3. Each module should expose a well-defined interface.

What is the main advantage of modular monoliths?

A monolith clubs all the functionalities into one big deployment unit. This makes monoliths simple to understand.

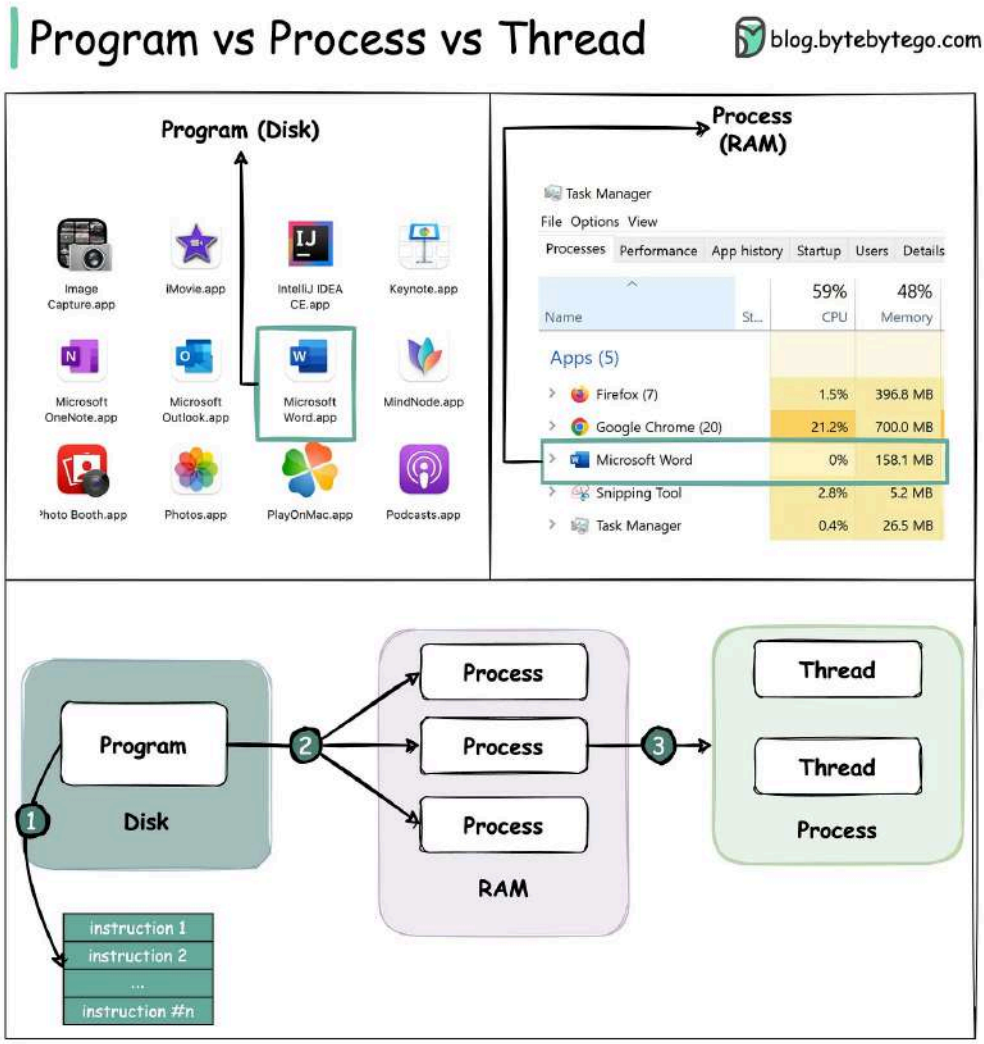
Microservices distribute the functionalities into separate deployable units. This makes them more scalable.

However, A modular monolith divides the application into modules that are part of the same deployment unit. They combine the simplicity of traditional monolithic applications with the flexibility of microservices.

In other words, you can have the best of both worlds with modular monoliths.

So - have you used modular monoliths to build applications?

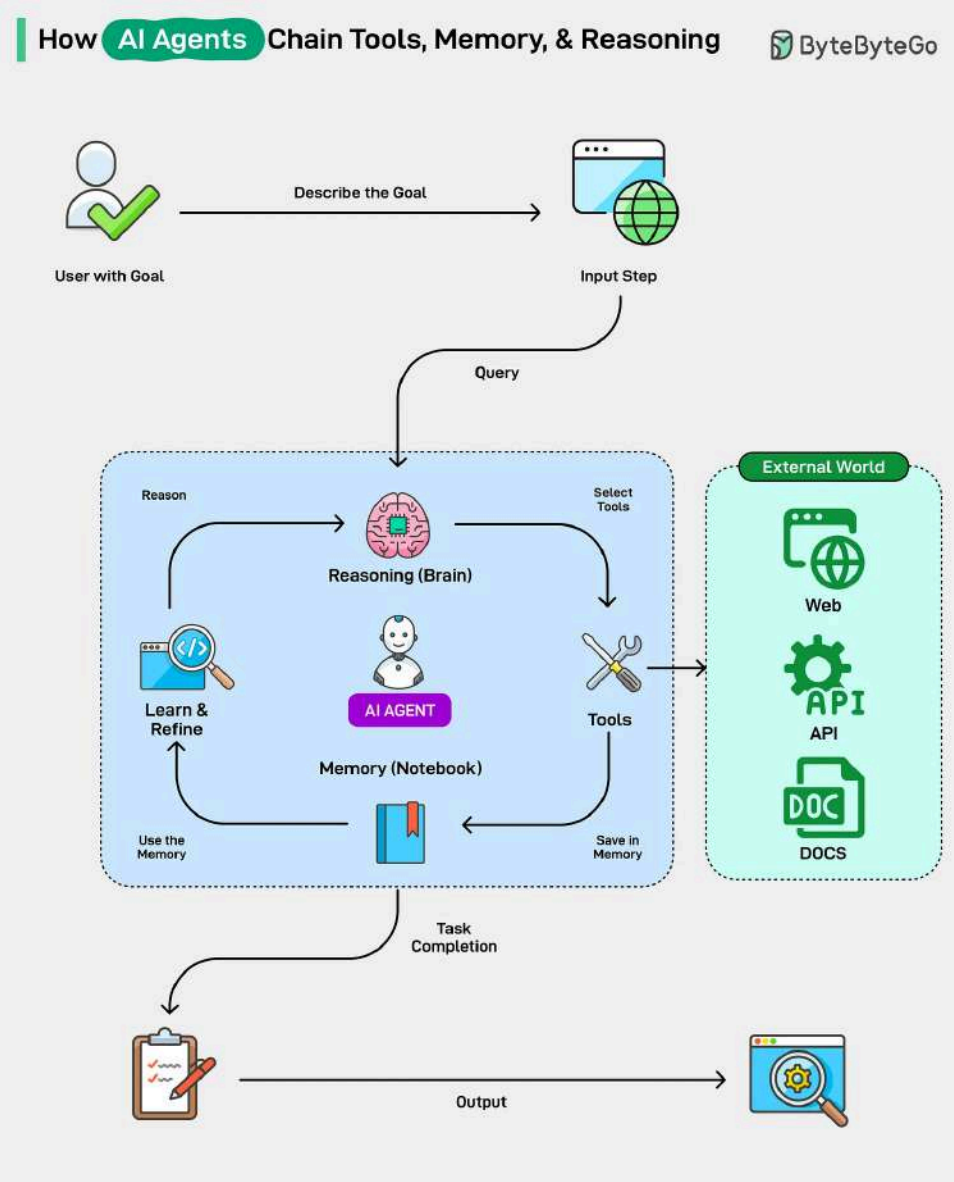
Popular interview question: What is the difference between Process and Thread?



Main differences between process and thread:

- Processes are usually independent, while threads exist as process subsets.
- Each process has its own memory space. Threads that belong to the same process share the same memory.
- A process is a heavyweight operation. It takes more time to create and terminate.
- Context switching is more expensive between processes.
- Inter-thread communication is faster for threads.

How AI Agents Chain Tools, Memory, and Reasoning?



1. Reasoning: This is the brain of the AI agent. It receives the user's goal as a query and uses its reasoning agent (using frameworks like ReAct) to break the goal into a series of smaller, logical steps.

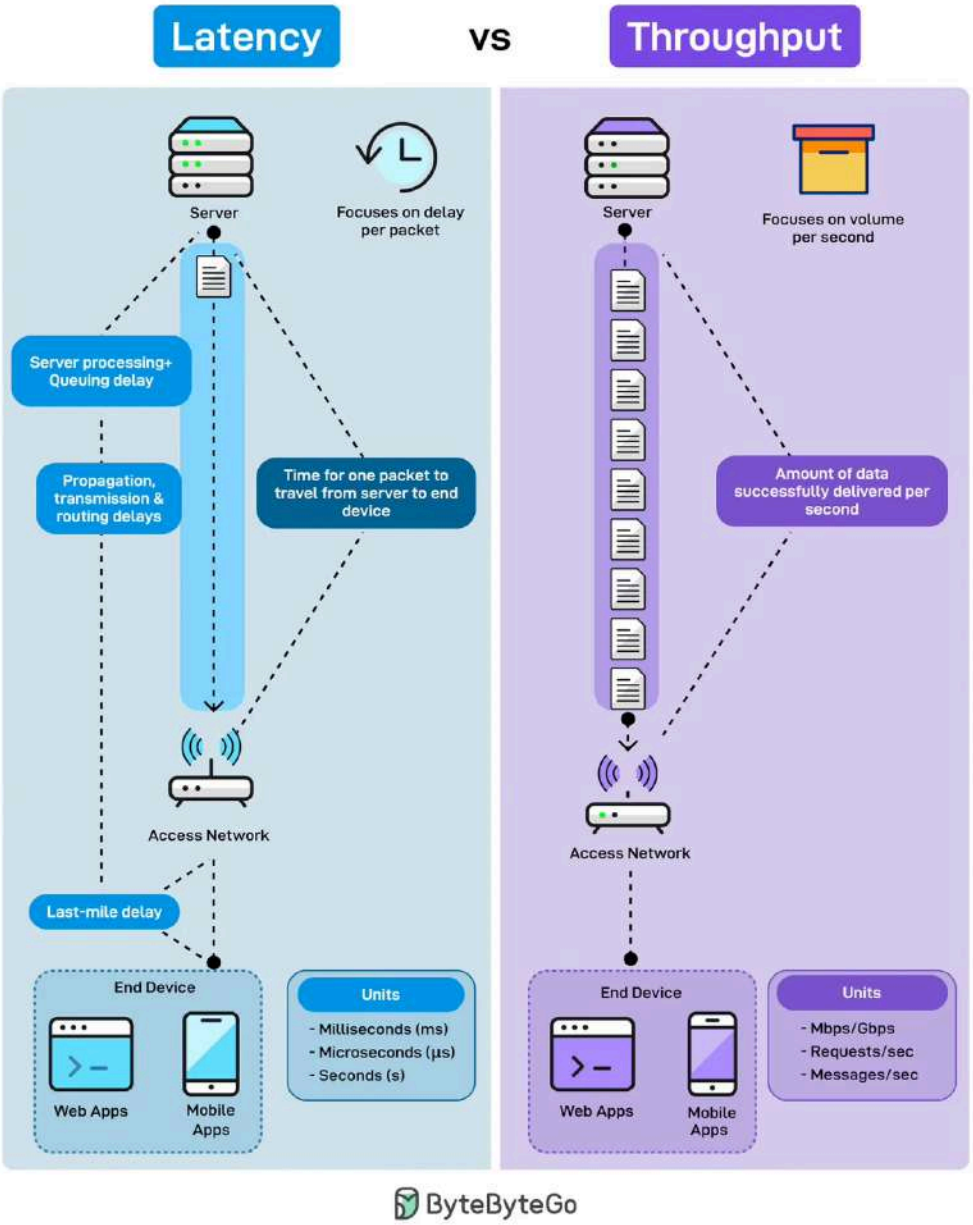
2. Tools: For each step, the agent selects an appropriate tool. These can be external programs or APIs that it can call upon, such as a web search, a calculator, a code interpreter, or documents.

3. Memory: This is like the notebook. The agent records the outcome of each tool's use in its memory. This notebook allows it to learn from results, maintain context over long conversations, and refine its plan based on new findings.

This continuous loop of reasoning, acting with a tool, and remembering the outcomes creates a chain. Finally, when the agent is satisfied that it has fulfilled the user's query, it responds to the user.

Latency vs. Throughput

Ever wondered why your app feels slow even when the bandwidth looks fine? Latency and throughput explain two very different stories of performance.



Latency measures the delay per packet. It is what users feel when they click a button. It's responsiveness. It is the time for one request to travel from the server to the end device. This includes server processing time, queuing delays, propagation through the network, transmission delays, and the last-mile connection to the user's device.

Throughput measures volume per second. It is how much data successfully gets delivered in a given timeframe. Not how fast each packet moves, but how many packets flow through the pipe.

Throughput is capacity. High throughput means the system handles the load without choking.

Top 20 System Design Concepts You Should Know

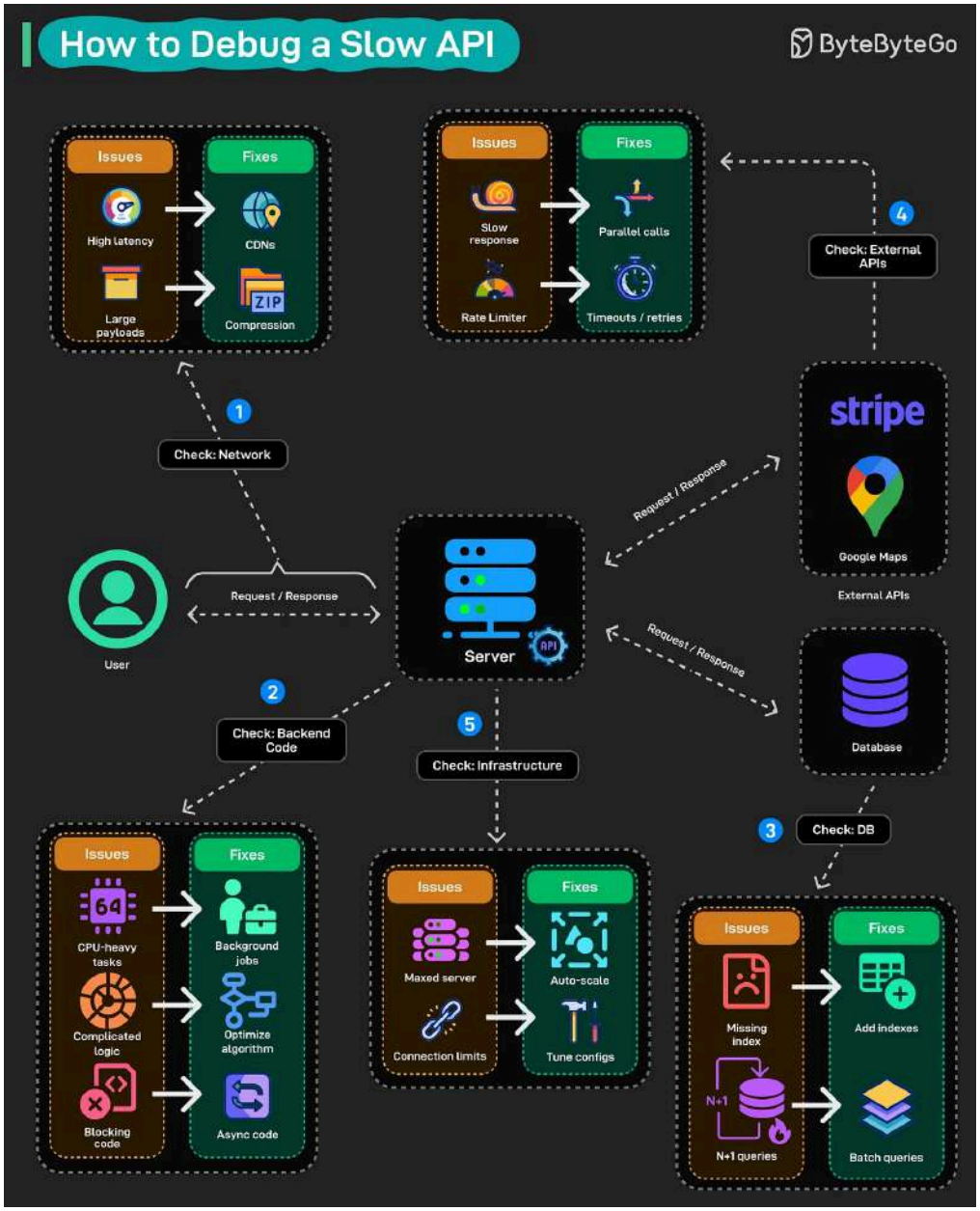


1. Load Balancing: Distributes traffic across multiple servers for reliability and availability.
2. Caching: Stores frequently accessed data in memory for faster access.
3. Database Sharding: Splits databases to handle large-scale data growth.
4. Replication: Copies data across replicas for availability and fault tolerance.
5. CAP Theorem: Trade-off between consistency, availability, and partition tolerance.
6. Consistent Hashing: Distributes load evenly in dynamic server environments.
7. Message Queues: Decouples services using asynchronous event-driven architecture.

8. Rate Limiting: Controls request frequency to prevent system overload.
9. API Gateway: Centralized entry point for routing API requests.
10. Microservices: Breaks systems into independent, loosely coupled services.
11. Service Discovery: Locates services dynamically in distributed systems.
12. CDN: Delivers content from edge servers for speed.
13. Database Indexing: Speeds up queries by indexing important fields.
14. Data Partitioning: Divides data across nodes for scalability and performance.
15. Eventual Consistency: Guarantees consistency over time in distributed databases
16. WebSockets: Enables bi-directional communication for live updates.
17. Scalability: Increases capacity by upgrading or adding machines.
18. Fault Tolerance: Ensures system availability during hardware/software failures.
19. Monitoring: Tracks metrics and logs to understand system health.
20. Authentication & Authorization: Controls user access and verifies identity securely.

How to Debug a Slow API?

Your API is slow. Users are complaining. And you have no idea where to start looking. Here is the systematic approach to track down what is killing your API.



Start with the network: High latency? Throw a CDN in front of your static assets. Large payloads? Compress your responses. These are quick wins that don't require touching code.

Check your backend code next: This is where most slowdowns hide. CPU-heavy operations should run in the background. Complicated business logic that needs simplification. Blocking synchronous calls that should be async. Profile it, find the hot paths, fix them.

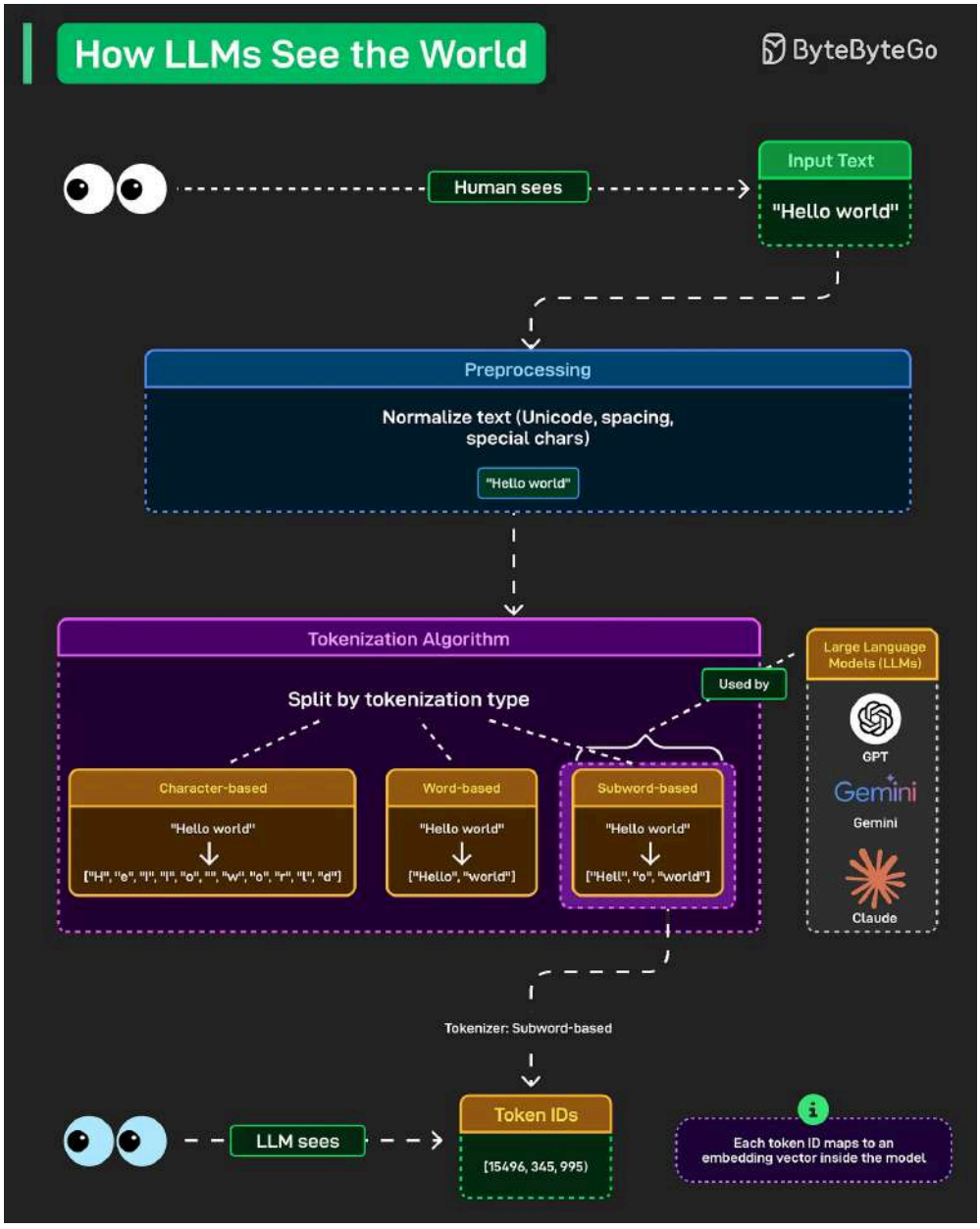
Check the database: Missing indexes are the classic culprit. Also watch for N+1 queries, where you are hammering the database hundreds of times when one batch query would do.

Don’t forget external APIs: That Stripe call, that Google Maps request, they are outside your control. Make parallel calls where you can. Set aggressive timeouts and retries so one slow third-party doesn’t tank your whole response.

Finally, check your infrastructure: Maxed-out servers need auto-scaling. Connection pool limits need tuning. Sometimes the problem isn’t your code at all, it’s that you are trying to serve 10,000 requests with resources built for 100.

The key is being methodical. Don’t just throw solutions at the wall. Measure first, identify the actual bottleneck, then fix it.

How LLMs See the World



When you type “Hello world” into ChatGPT or Claude, the model isn’t processing those letters and spaces like you’re reading this post right now. It’s converting everything into numbers through a process most people never think about.

Preprocessing comes first. Text gets normalized. Unicode characters, spacing quirks, and special symbols, they all get cleaned up and standardized. “Hello world” becomes a consistent format that the model can actually work with.

Then comes tokenization. This is where things get interesting. The model splits text into tokens, and there are different approaches.

1. Character-based tokenization breaks everything down to individual characters. “Hello world” becomes [”H”, “e”, “l”, “l”, “o”, “ “, “w”, “o”, “r”, “l”, “d”]. Simple but inefficient.

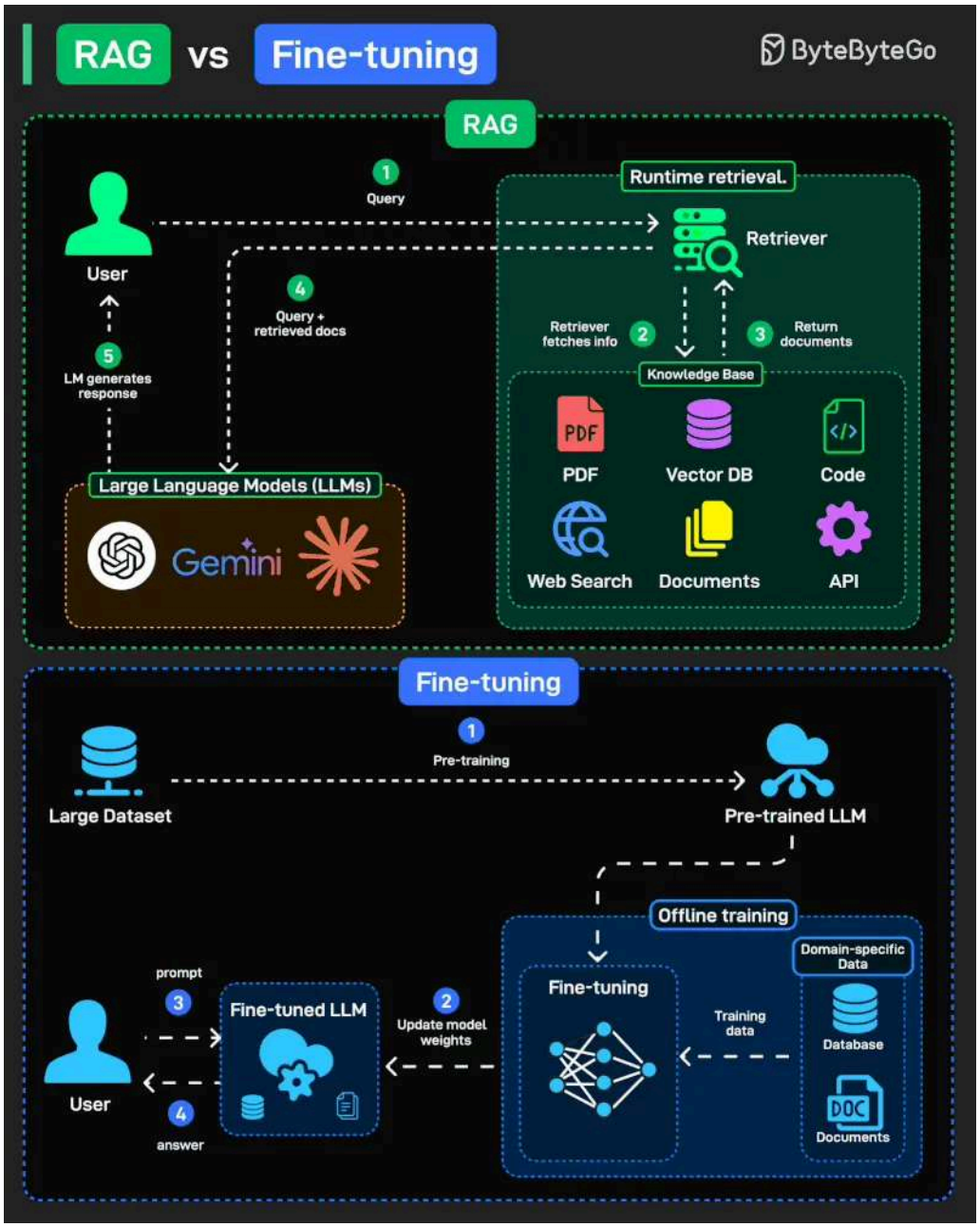
2. Word-based splits on whole words. [”Hello”, “world”]. Cleaner but struggles with rare words and creates massive vocabularies.

3. Subword-based is what modern LLMs actually use. GPT, Gemini, Claude, they all rely on this. “Hello world” becomes something like [”Hell”, “o”, “world”]. It balances efficiency with flexibility, handling rare words by breaking them into known subword pieces.

The final step is Token IDs. Those subwords get mapped to numbers like [15496, 345, 995]. Each token ID corresponds to an embedding vector inside the model. That’s what the neural network actually processes.

RAG vs Fine-tuning: Which one should you use?

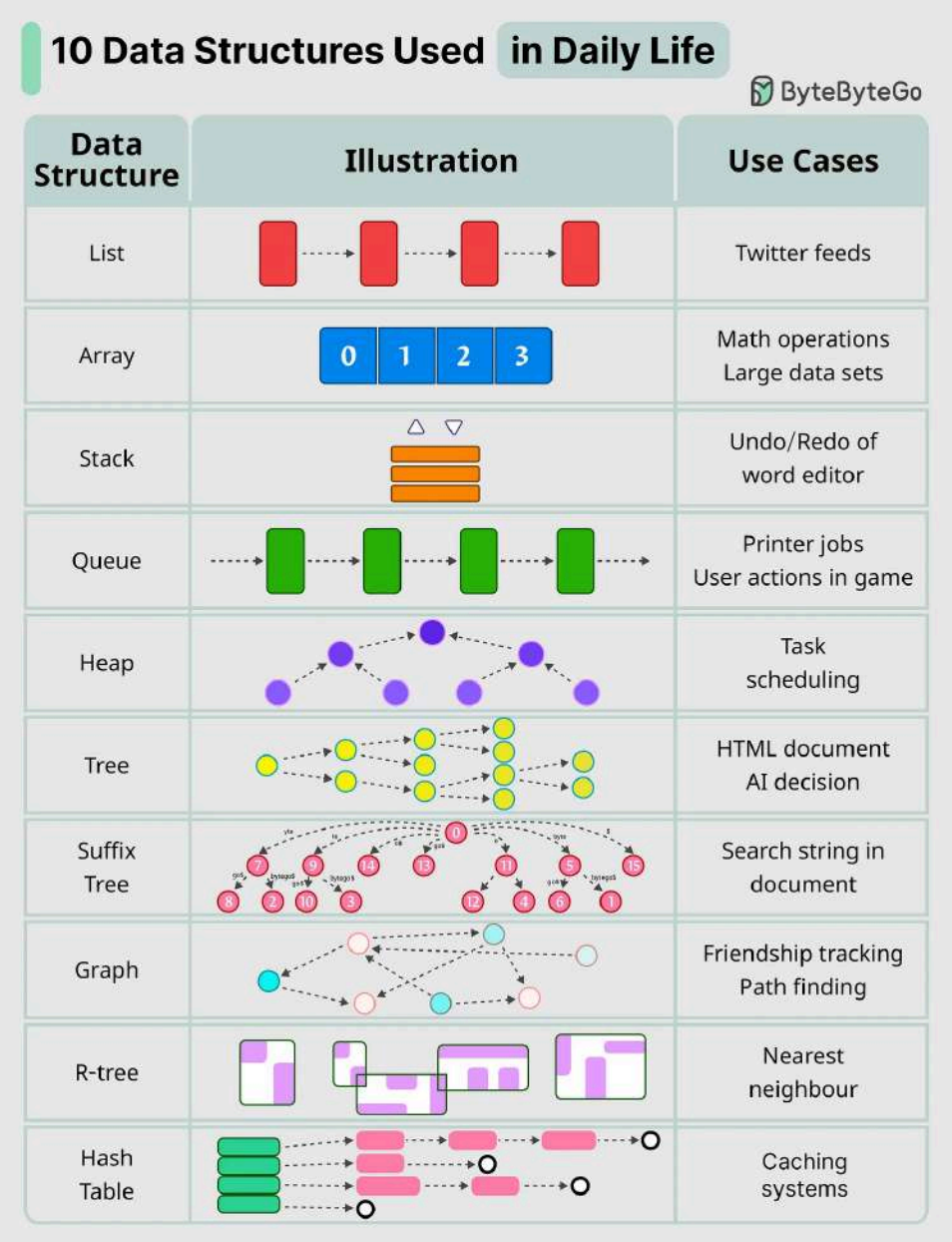
When it comes to adapting Large Language Models (LLMs) to new tasks, two popular approaches stand out: Retrieval-Augmented Generation (RAG) and Fine-tuning. They solve the same problem, making models more useful, but in very different ways.



RAG (Retrieval-Augmented Generation): Fetches knowledge at runtime from external sources (docs, DBs, APIs). Flexible, always fresh.

Fine-tuning: Offline training that updates model weights with domain-specific data, making the model an expert in your field.

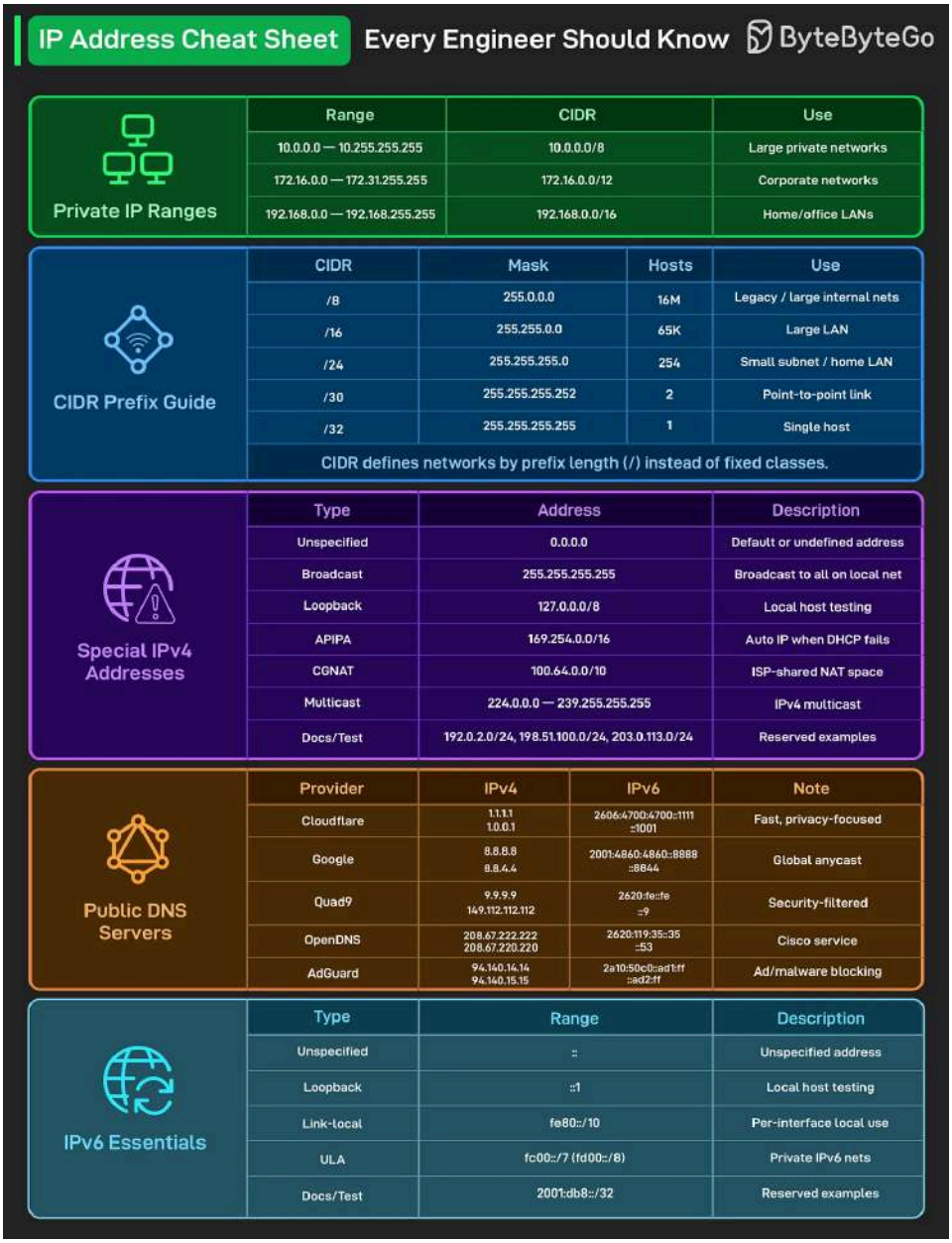
10 Key Data Structures We Use Every Day



- list: keep your Twitter feeds
- stack: support undo/redo of the word editor
- queue: keep printer jobs, or send user actions in-game
- hash table: cashing systems
- Array: math operations

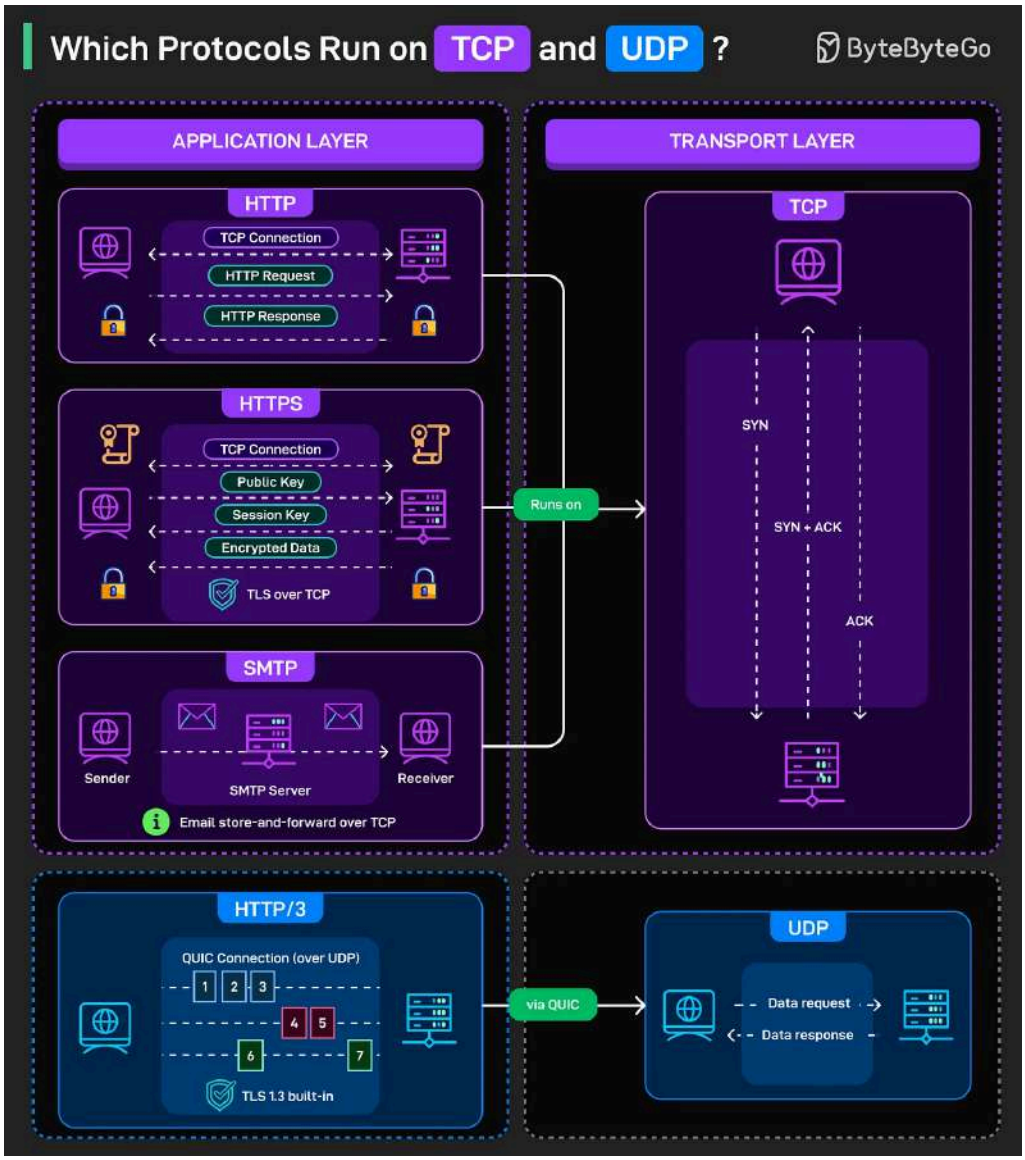
- heap: task scheduling
- tree: keep the HTML document, or for AI decision
- suffix tree: for searching string in a document
- graph: for tracking friendship, or path finding
- r-tree: for finding the nearest neighbor
- vertex buffer: for sending data to GPU for rendering

IP Address Cheat Sheet Every Engineer Should Know



Which Protocols Run on TCP and UDP

Every message sent over the internet has two layers of communication, one that carries the data (transport) and one that defines what the data means (application). TCP and UDP sit at the transport layer, but they serve completely different purposes.



TCP is connection-oriented. It guarantees delivery, maintains order, and handles retransmission when packets get lost.

- HTTP runs on TCP. The browser opens a TCP connection, sends an HTTP request, waits for the HTTP response, and closes the connection (or keeps it alive for subsequent requests). Every web page you have ever loaded used this pattern.

- HTTPS adds TLS over TCP. The TCP connection happens first. Then comes the TLS handshake with public key exchange, session key negotiation, and finally encrypted data transfer.
- SMTP uses TCP for email. Messages flow from sender to SMTP server to receiver over TCP connections. Email can't afford to lose data mid-transmission, so TCP's reliability is essential.

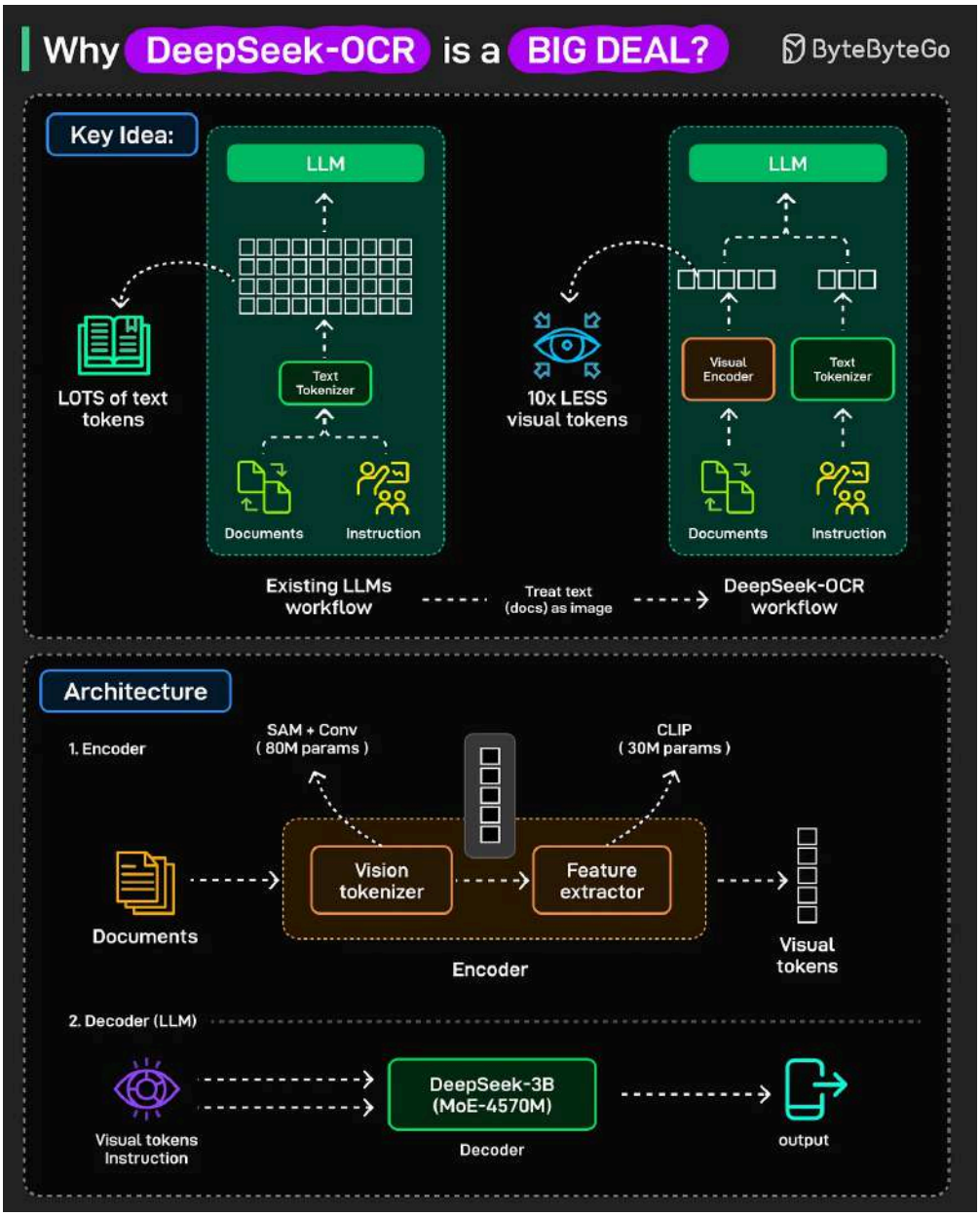
UDP is connectionless. No handshake. No guaranteed delivery. No order preservation. Just fire data requests and responses into the network and hope they arrive. Sounds chaotic, but it's fast.

- HTTP/3 runs over QUIC, which uses UDP. This seems backwards until you realize QUIC reimplements the reliability features of TCP inside UDP, but with better performance. Multiple streams over one connection. Built-in TLS 1.3. Faster connection establishment. The numbered streams in the diagram show parallel data flows that don't block each other.

Why is DeepSeek-OCR such a BIG DEAL?

Existing LLMs struggle with long inputs because they can only handle a fixed number of tokens, known as the context window, and attention cost grows quickly as inputs get longer.

DeepSeek-OCR takes a new approach.



Instead of sending long context directly to an LLM, it turns it into an image, compresses that image into visual tokens, and then passes those tokens to the LLM.

Fewer tokens lead to lower computational cost from attention and a larger effective context window. This makes chatbots and document models more capable and efficient.

How is DeepSeek-OCR built? The system has two main parts:

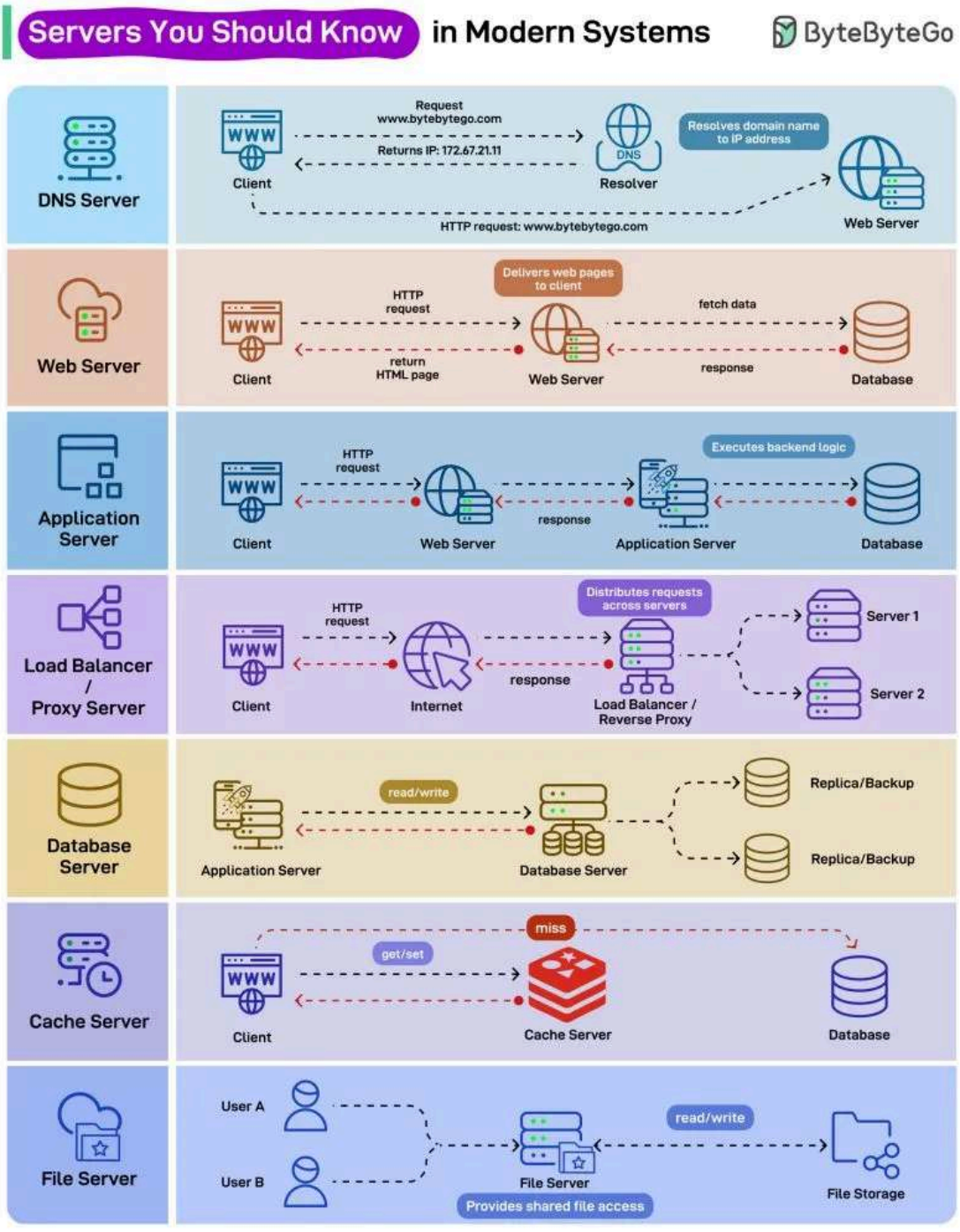
1. Encoder: It processes an image of text, extracts the visual features, and compresses them into a small number of vision tokens.
2. Decoder: A Mixture of Experts language model that reads those tokens and generate text one token at a time, similar to a standard decoder-only transformer.

When to use it?

DeepSeek-OCR shows that text can be efficiently compressed using visual representations.

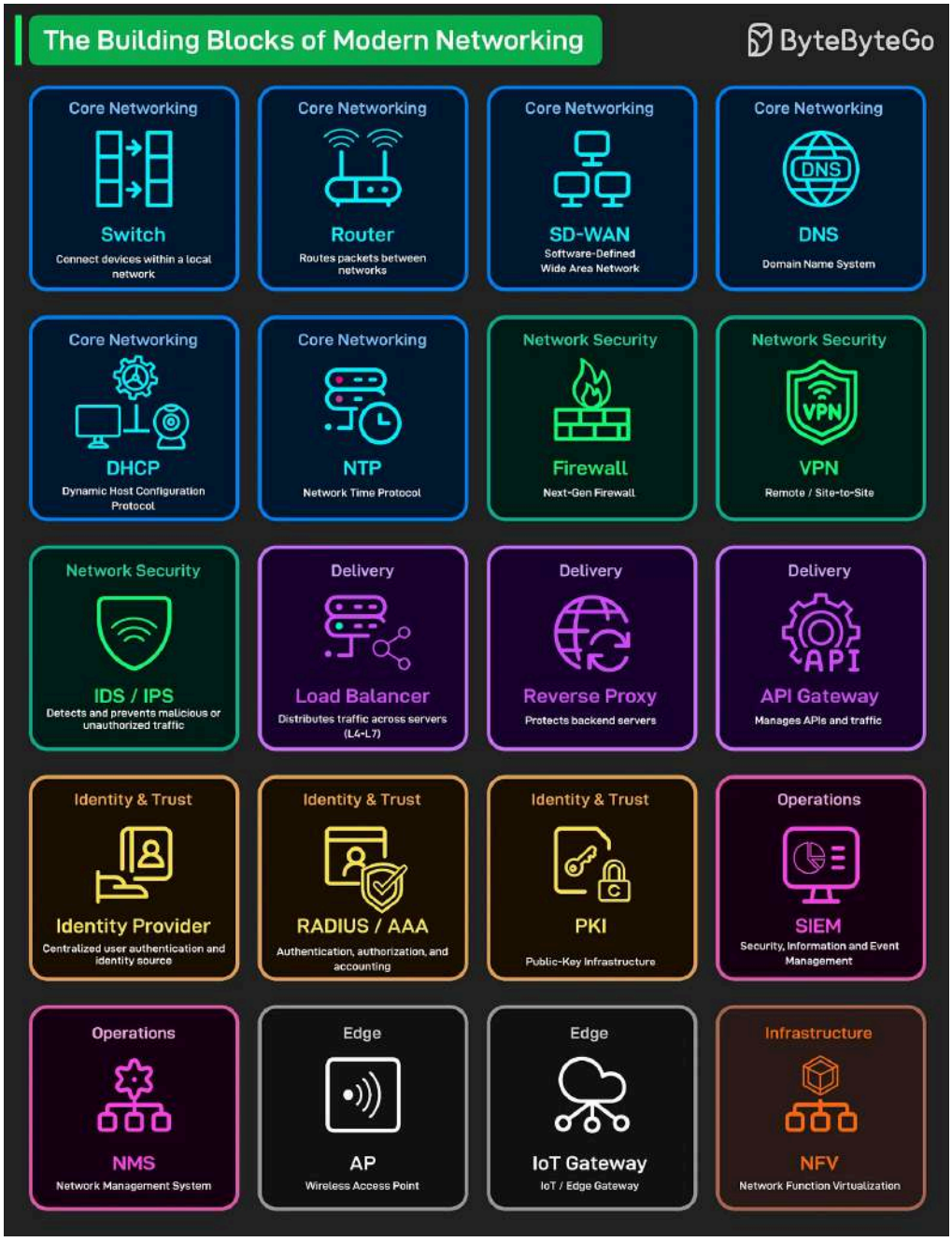
It is especially useful for handling very long documents that exceed standard context limits. You can use it for context compression, standard OCR tasks, or deep parsing, such as converting tables and complex layouts into text.

Servers You Should Know in Modern Systems



The Building Blocks of Modern Networking

Every modern network, from home Wi-Fi to global cloud infrastructure, is built on a few essential components. Here's a quick overview of the Building Blocks of Modern



Networking:

- Core Networking: Switches connect devices within a local network. Every office has dozens of these. Routers move packets between different networks. Your gateway to the internet and beyond. SD-WAN is how modern companies connect branch offices. Software-defined,

flexible, way cheaper.
DNS translates domain names to IP addresses. DHCP hands out IP addresses automatically. NTP keeps clocks synchronized across all systems.

- Network Security: Firewalls are your first line of defense. Next-gen versions can inspect traffic at the application level. VPNs create encrypted tunnels for remote access and site-to-site connections. Remote work runs on this. IDS/IPS detects and blocks malicious traffic before it reaches your servers.

- Delivery (Traffic management): Load Balancers distribute requests across multiple servers. One server goes down? Users never notice. Reverse Proxy sits in front of your backend servers, handling SSL termination and caching. API Gateway manages all your API traffic.

- Identity & Trust: Identity Provider is your single source of truth for user authentication. Think Okta, Azure AD, Auth0. RADIUS/AAA handles network device authentication. PKI manages digital certificates and encryption keys. HTTPS wouldn't exist without it.

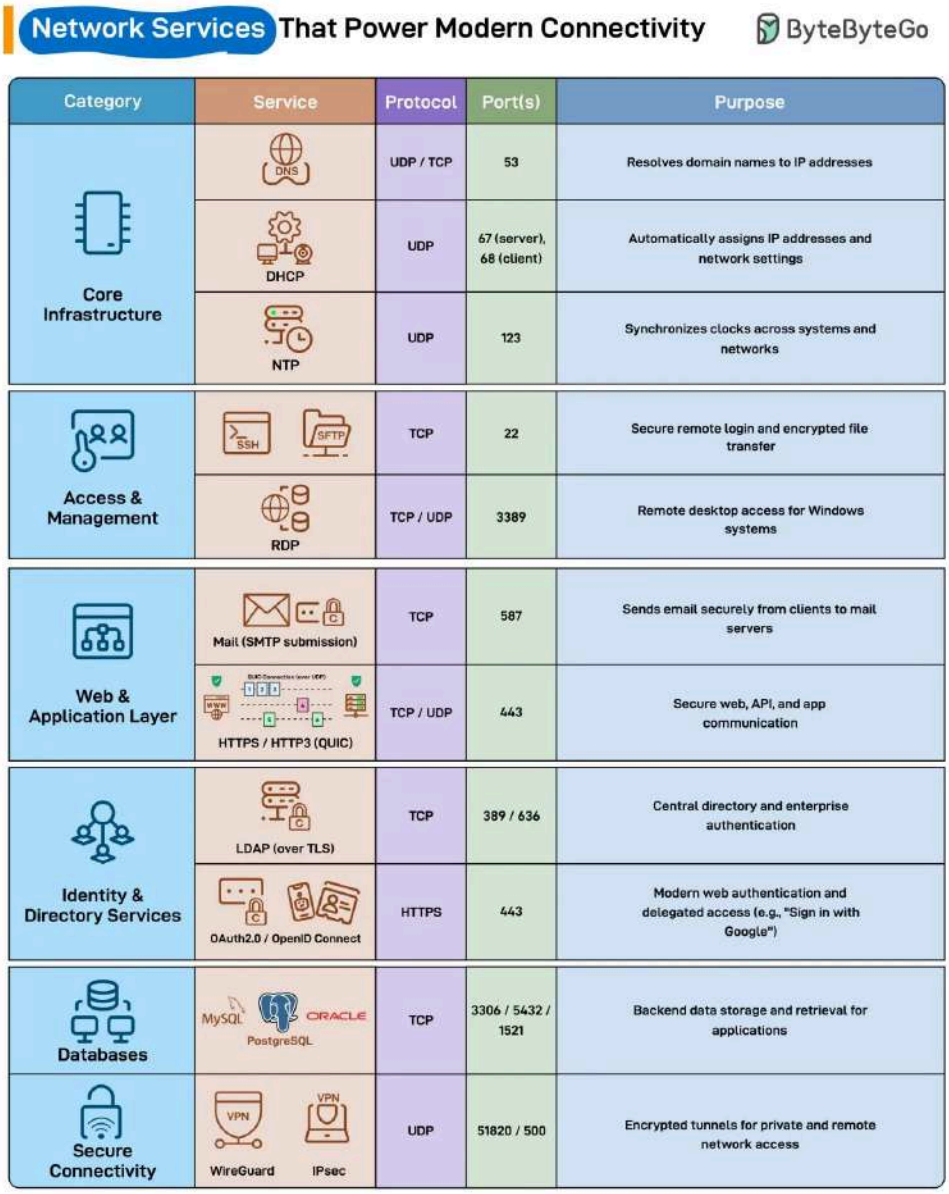
- Operations: SIEM collects and analyzes security events from across your entire infrastructure. NMS monitors network health and performance. Alerts you before users start complaining.

- Edge: Access Points provide WiFi coverage. IoT Gateway connects sensors, cameras, and smart devices to your network. The bridge between operational tech and IT.

- Infrastructure: NFV runs network functions as software instead of dedicated hardware. Virtual firewalls, virtual routers, virtual everything.

Network Services That Power Modern Connectivity

Every time you open a browser, send an email, or connect to a VPN, these network services quietly make it possible.

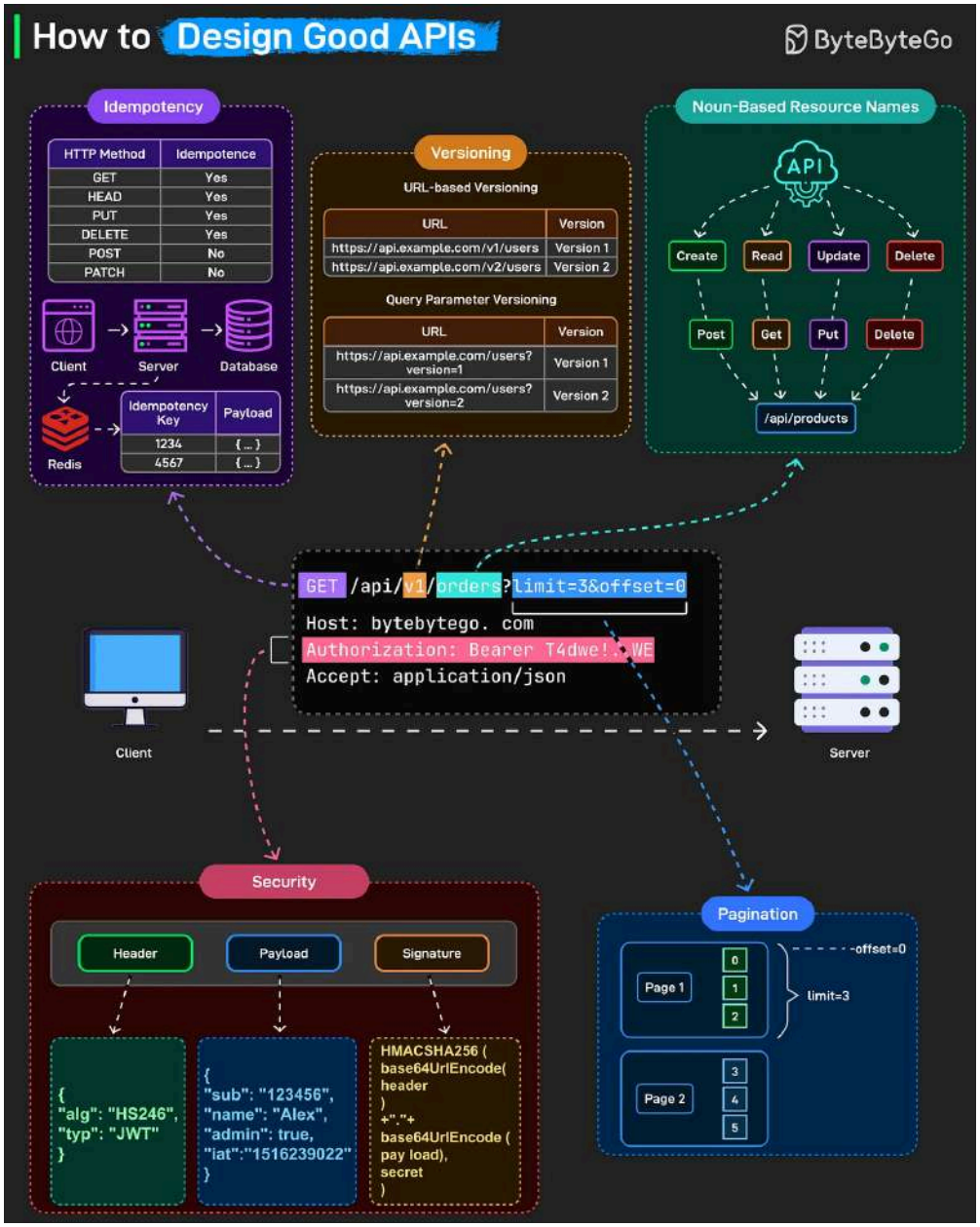


- DNS: Resolves domain names to IP addresses so users can reach websites without memorizing numbers.
- DHCP: Automatically assigns IPs and network settings to devices joining the network.
- NTP: Keeps clocks synchronized across systems to ensure consistent logs and authentication.

- SSH: Enables secure remote login and encrypted file transfers over port 22.
- RDP: Allows remote desktop access to Windows systems through port 3389.
- Mail (SMTP submission): Sends emails securely from clients to mail servers.
- HTTPS / HTTP3 (QUIC): Secures web, API, and app communication over encrypted channels.
- LDAP (over TLS): Acts as a central directory for enterprise logins and access control.
- OAuth2.0 / OpenID Connect: Powers modern authentication flows like “Sign in with Google.”
- MySQL / PostgreSQL / Oracle: Handle backend data storage and retrieval for web and mobile applications.
- WireGuard / IPsec: Create encrypted tunnels for private and remote network access.

How to Design Good APIs

A well-designed API feels invisible, it just works. Behind that simplicity lies a set of consistent design principles that make APIs predictable, secure, and scalable.



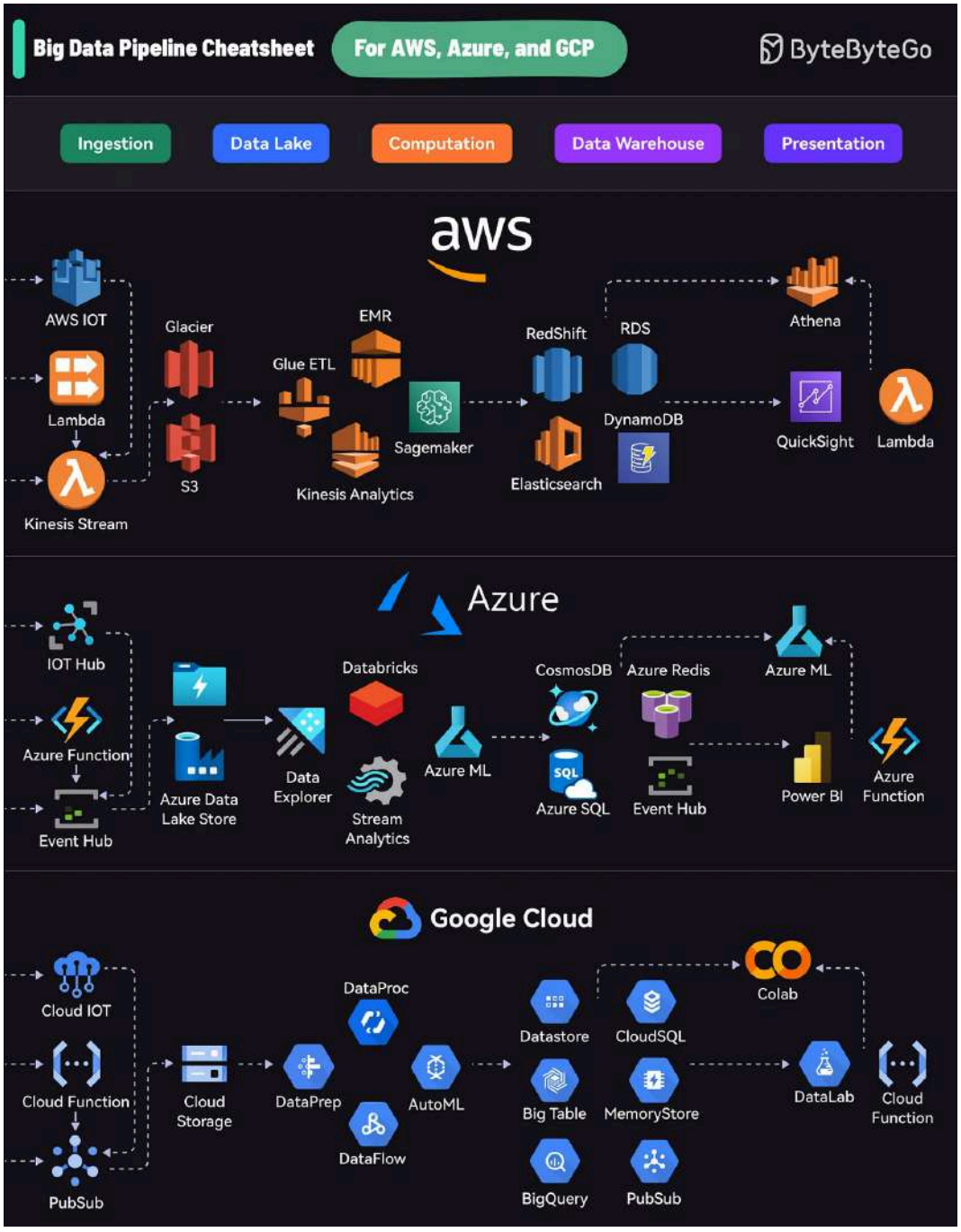
Here's what separates good APIs from terrible ones:

- Idempotency: GET, HEAD, PUT, and DELETE should be idempotent. Send the same request twice, get the same result. No unintended side effects. POST and PATCH are not idempotent. Each call creates a new resource or modifies the state differently.

Use idempotency keys stored in Redis or your database. Client sends the same key with retries, server recognizes it and returns the original response instead of processing again.

- Versioning
- Noun-based resource names: Resources should be nouns, not verbs. “/api/products”, not “/api/getProducts”.
- Security: Secure every endpoint with proper authentication. Bearer tokens (like JWTs) include a header, payload, and signature to validate requests. Always use HTTPS and verify tokens on every call.
- Pagination: When returning large datasets, use pagination parameters like “?limit=10&offset=20” to keep responses efficient and consistent.

Big Data Pipeline Cheatsheet for AWS, Azure, and Google Cloud



Each platform offers a comprehensive suite of services that cover the entire lifecycle:

1. Ingestion: Collecting data from various sources
2. Data Lake: Storing raw data

3. Computation: Processing and analyzing data
4. Data Warehouse: Storing structured data
5. Presentation: Visualizing and reporting insights

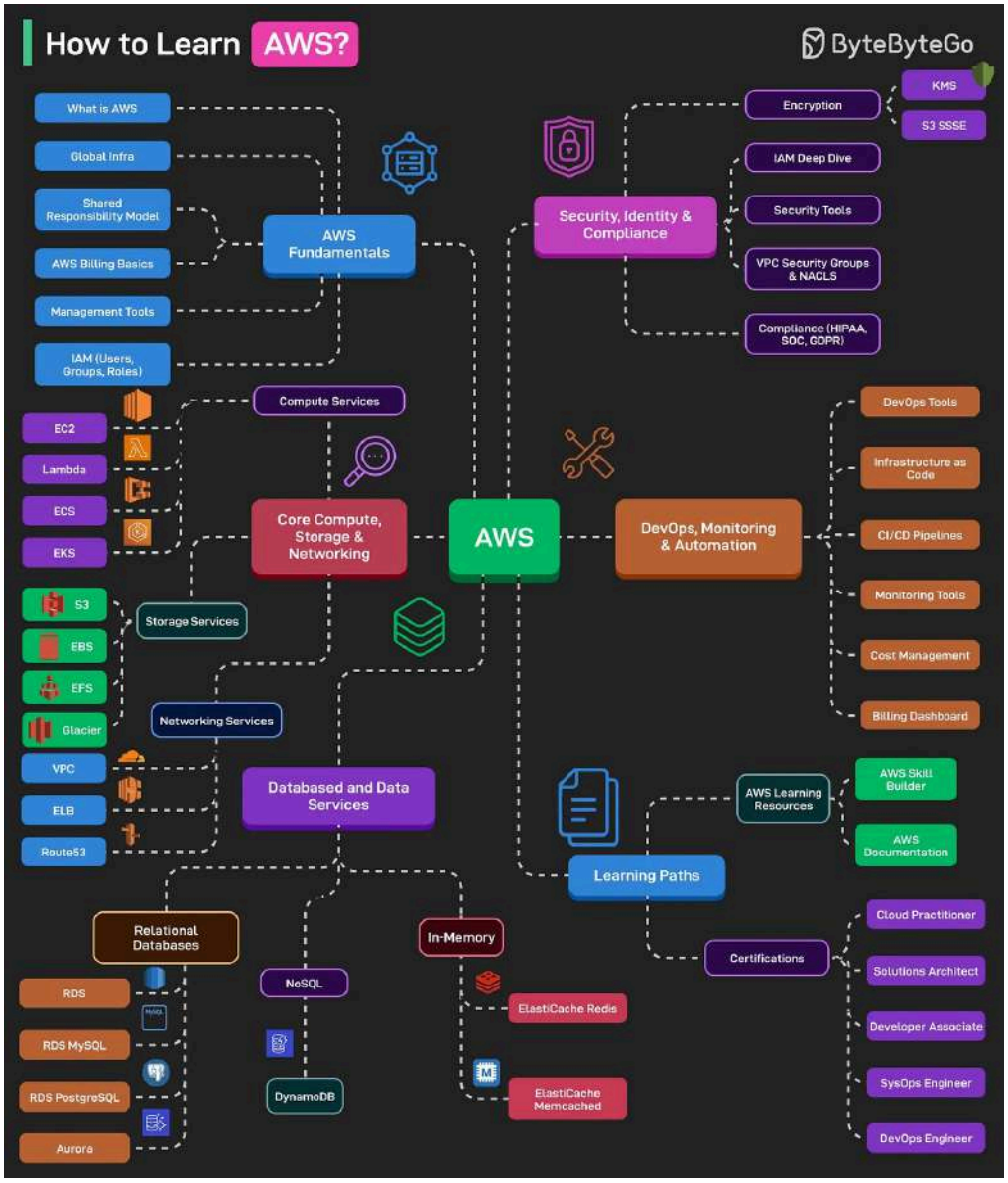
AWS uses services like Kinesis for data streaming, S3 for storage, EMR for processing, RedShift for warehousing, and QuickSight for visualization.

Azure's pipeline includes Event Hubs for ingestion, Data Lake Store for storage, Databricks for processing, Cosmos DB for warehousing, and Power BI for presentation.

GCP offers PubSub for data streaming, Cloud Storage for data lakes, DataProc and DataFlow for processing, BigQuery for warehousing, and Data Studio for visualization.

How to Learn AWS?

AWS is one of the most popular cloud platforms. When AWS goes down, a large part of the Internet goes down.



Here's a learning map that can help you master AWS:

1. AWS Fundamentals
   This includes topics like "What is AWS?", Global Infrastructure, AWS Billing, Management, and IAM basics.

2. Core Compute, Storage & Networking
   This includes compute services like EC2, Lambda, ECS, EKS, Storage Services (such as S3, EBS, EFS, Glacier), and Networking Services (such as VPC, ELB, Route 53).

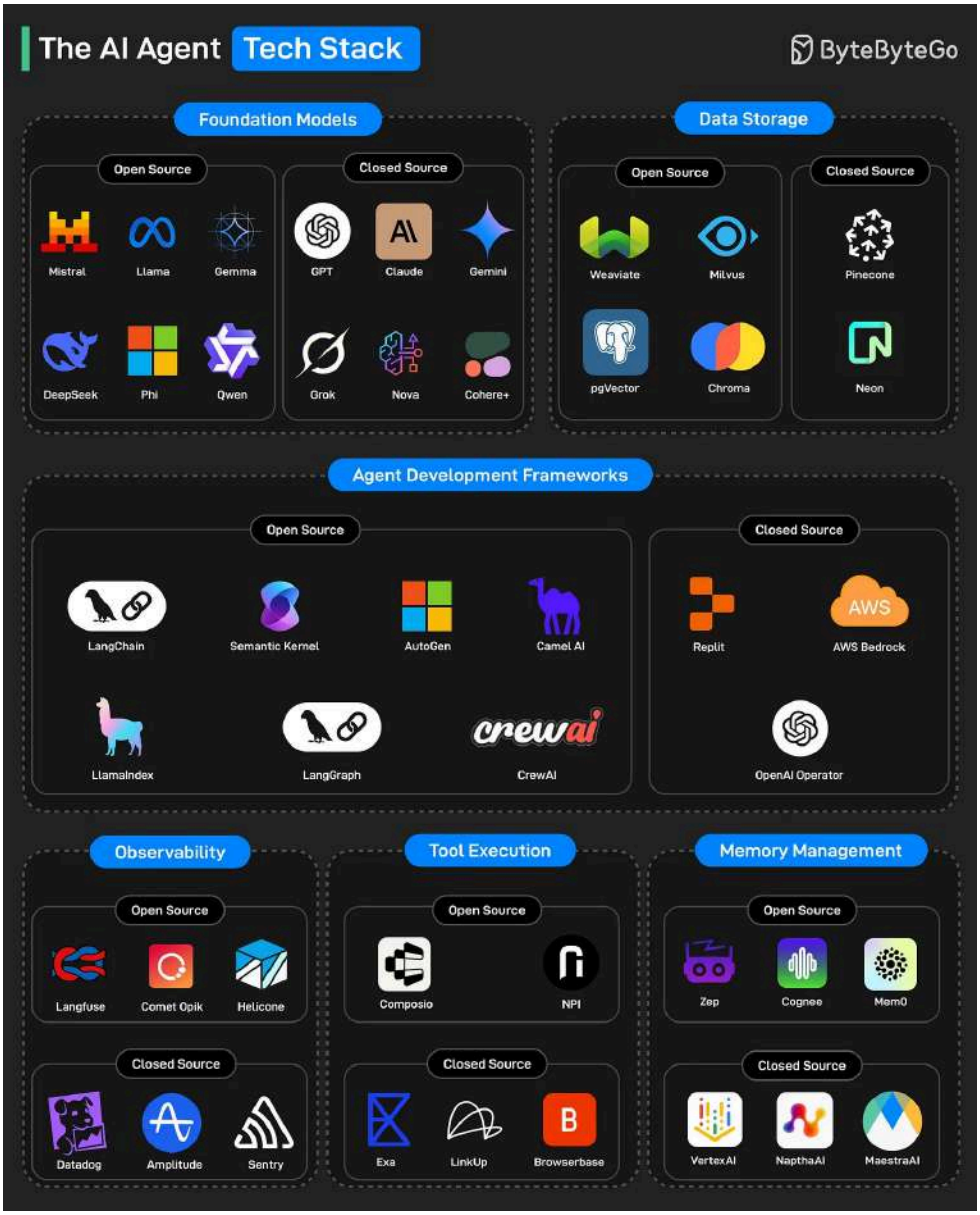
3. Databases and Data Services
   This includes topics like Relational Databases (RDS MySQL and PostgreSQL), NoSQL, and In-Memory Databases like ElastiCache (Redis and Memcached).

4. Security, Identity & Compliance
   Consists of topics like IAM Deep Dive, Encryption (KMS, S3 SSE), Security Tools, VPC Security Groups, and Compliance-related tools for HIPAA, SOC, and GDPR.

5. DevOps, Monitoring & Automation
   This includes topics like DevOps Tools (CodeCommit, CodeBuild, CodePipeline), Infrastructure as Code, CI/CD Pipelines, Monitoring Tools (CloudWatch, CloudTrail), and Cost Management and Billing Dashboard

6. Learning Paths and Certifications
   Consists of topics like AWS Learning Resources, such as Skill Builder and documentation, and certification paths such as Cloud Practitioner, Solutions Architect Associate, Developer Associate, SysOps, and DevOps Engineer.

The AI Agent Tech Stack

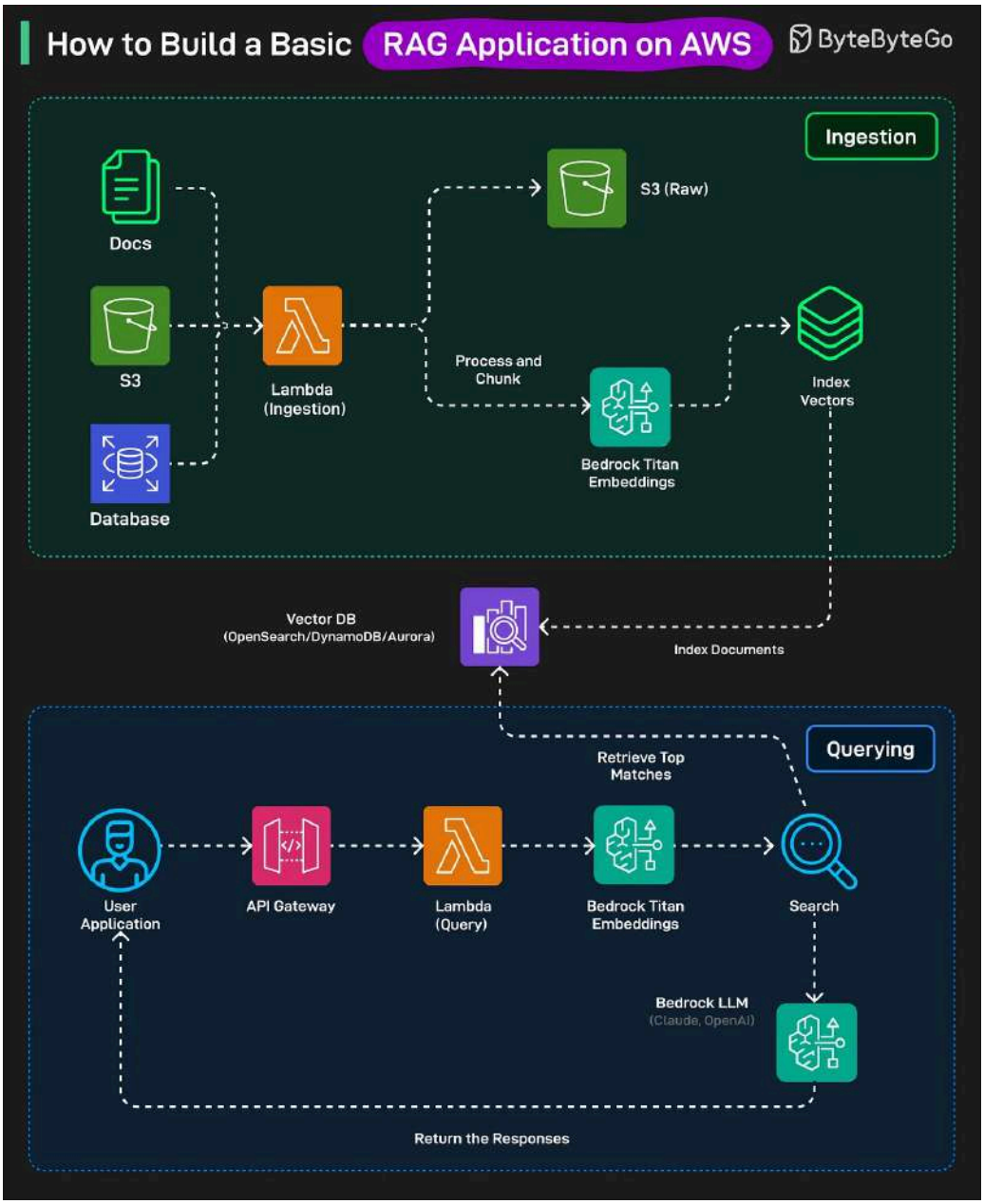


1. Foundation Models: Large-scale pre-trained language models that serve as the “brains” of AI agents, enabling capabilities like reasoning, text generation, coding, and question answering.
2. Data Storage: This layer handles vector databases and memory storage systems used by AI agents to store and retrieve context, embeddings, or documents.
3. Agent Development Frameworks: These frameworks help developers build, orchestrate, and manage multi-step AI agents and their workflows.
4. Observability: This category enables monitoring, debugging, and logging of AI agent behavior and performance in real-time.

5. Tool Execution: These platforms allow AI agents to interface with real-world tools (for example, APIs, browsers, external systems) to complete complex tasks.
6. Memory Management: These systems manage long-term and short-term memory for agents, helping them retain useful context and learn from past interactions.

How to Build a Basic RAG Application on AWS?

RAG is an AI pattern that combines a search step with text generation. It retrieves relevant information from a knowledge source (like a vector database) and then uses an LLM to generate accurate, context-aware responses.



Ingestion Stage

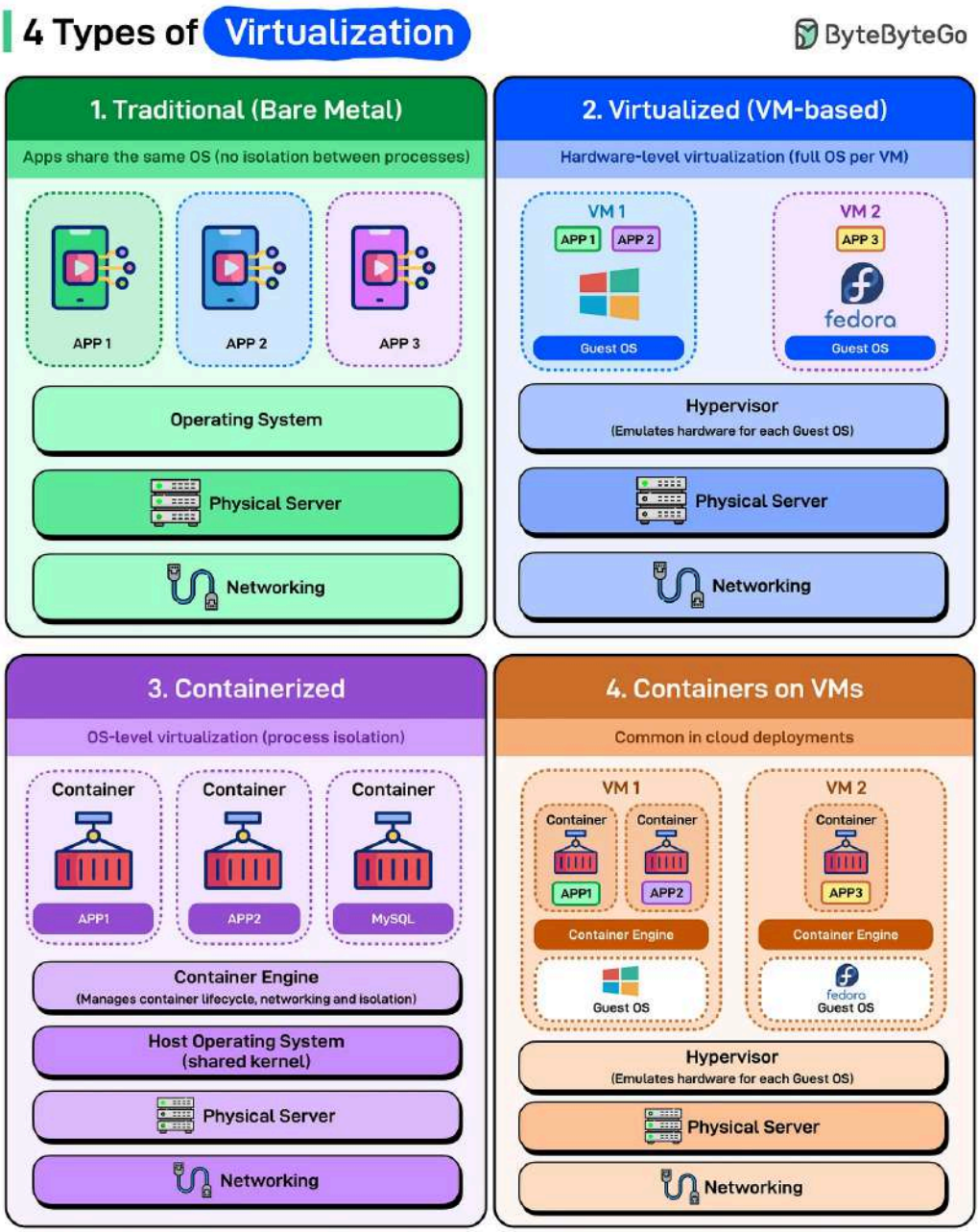
1. All raw documents (PDFs, text, etc) are first stored in Amazon S3.
2. When a file is added, AWS Lambda runs an ingestion function. This function cleans and splits the document into smaller chunks.

3. Each chunk is sent to Amazon Bedrock's Titan embeddings model, which converts it into vector representations
4. These embeddings, along with metadata, are stored in a vector database such as OpenSearch serverless, DynamoDB

Querying Stage:

1. A user sends a question through the app frontend, which goes to API Gateway and then a Lambda query function.
2. The question is converted to an embedding using Amazon Bedrock Titan Embeddings.
3. This embedding is compared against the stored document embeddings in the vector database to find the most relevant chunks.
4. The relevant chunks and the user's questions are sent to an LLM (like Claude or OpenAI on Bedrock) to generate an answer.
5. The generated response is sent back to the user through the same API.

Types of Virtualization



Virtualization didn't just make servers efficient, it changed how we build, scale, and deploy everything. Here's a quick breakdown of the four major types of virtualization you'll find in modern systems:

1. Traditional (Bare Metal): Applications run directly on the operating system. No virtualization layer, no isolation between processes. All applications share the same OS kernel, libraries, and resources.

2. Virtualized (VM-based): Each VM runs its own complete operating system. The hypervisor sits on physical hardware and emulates entire machines for each guest OS. Each VM thinks it has dedicated hardware even though it's sharing the same physical server.

3. Containerized: Containers share the host operating system's kernel but get isolated runtime environments. Each container has its own filesystem, but they're all using the same underlying OS. The container engine (Docker, containerd, Podman) manages lifecycle, networking, and isolation without needing separate operating systems for each application. Lightweight and fast. Containers start in milliseconds because you're not booting an OS. Resource usage is dramatically lower than VMs.

4. Containers on VMs: This is what actually runs in production cloud environments. Containers inside VMs, getting benefits from both. Each VM runs its own guest OS with a container engine inside. The hypervisor provides hardware-level isolation between VMs. The container engine provides lightweight application isolation within VMs.

This is the architecture behind Kubernetes clusters on AWS, Azure, and GCP. Your pods are containers, but they're running inside VMs you never directly see or manage.

Cloudflare vs. AWS vs. Azure

Cloudflare is much more than just a CDN and DDoS protection service. Let's do a quick comparison of Cloudflare, AWS, and Azure.



Cloudflare has rapidly expanded beyond its traditional CDN roots, launching a suite of modern developer-first services like Workers, R2, D1, and so on. These offerings position it as a serious edge-native alternative to other cloud providers.

Here are the key cloud capabilities that Cloudflare supports:

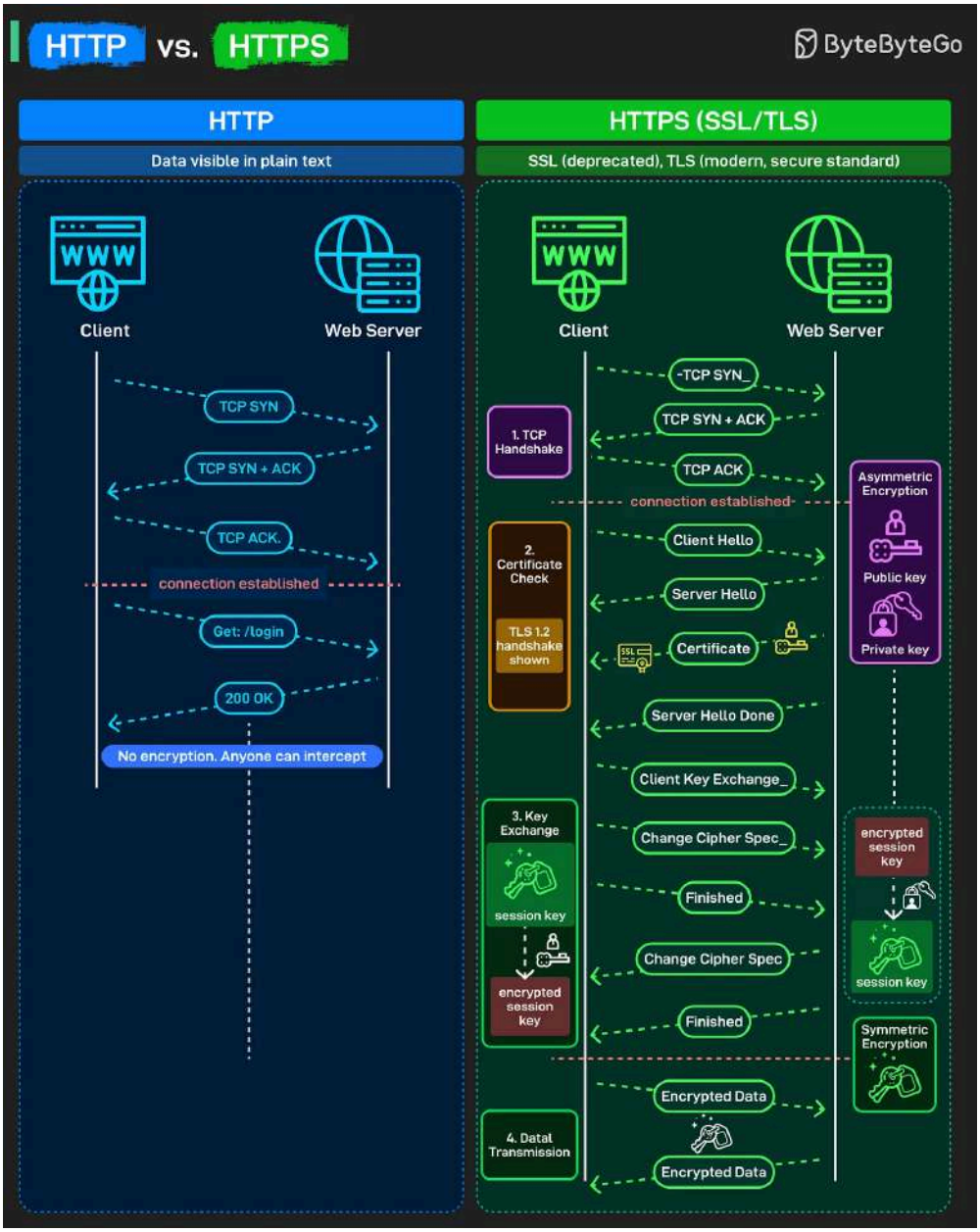
1. Edge Compute and Serverless

2. Object and Blob Storage
3. Relational Databases
4. Containers
5. Sandboxes
6. Workflows
7. AI Agents SDK
8. Vector and AI search
9. Data Connectivity
10. AI Infrastructure
11. Content Delivery Network
12. DNS
13. Load Balancing

Popular Backend Tech Stack



HTTP vs. HTTPS



When you open a website, the difference between HTTP and HTTPS decides whether your data travels safely or in plain sight. Here's what actually happens under the hood:

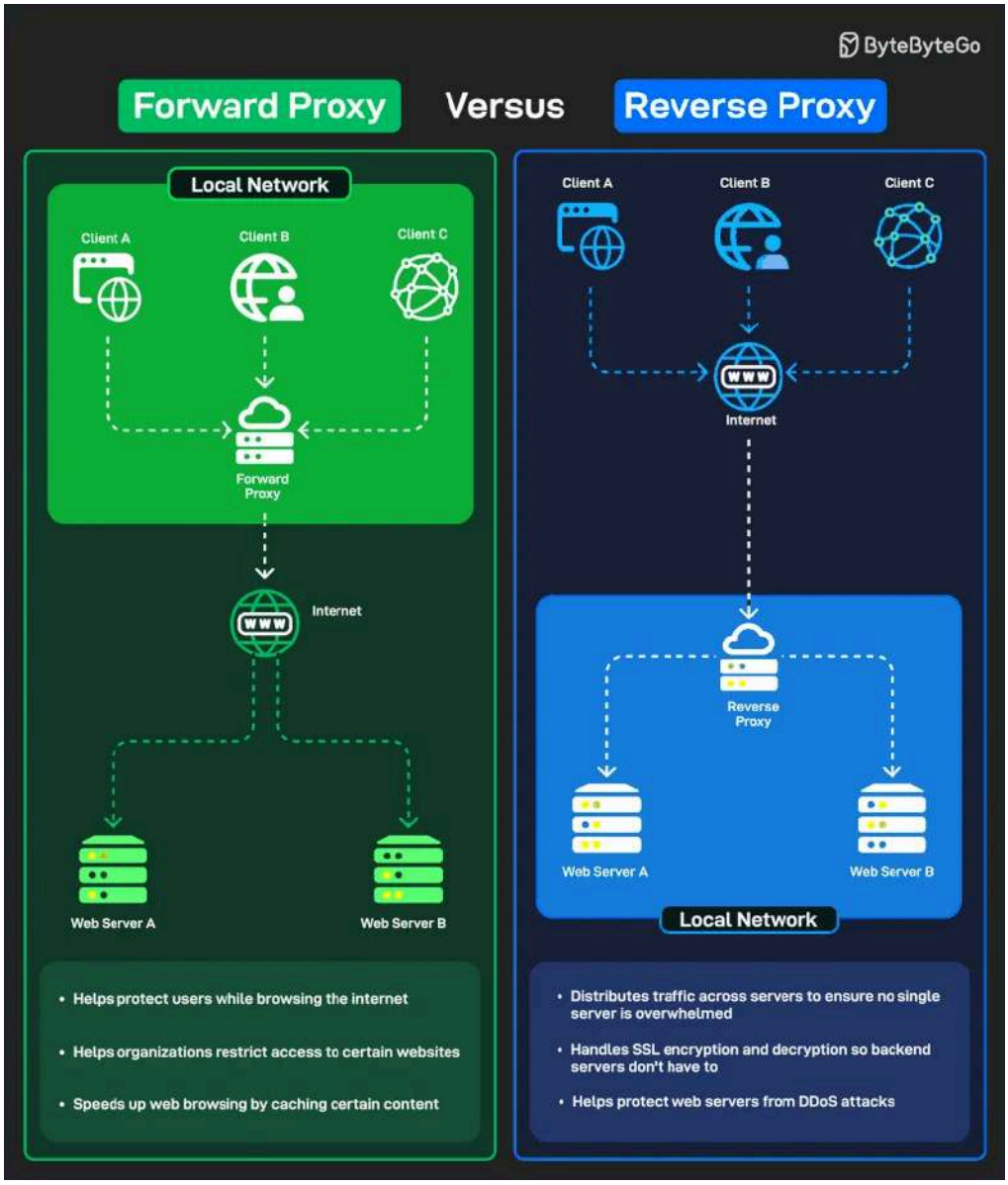
HTTP:

- Sends data in plain text, anyone on the network can intercept it.
- The client and server perform a simple TCP handshake: SYN, SYN-ACK, ACK
- Fast but completely insecure. Passwords, tokens, and forms can all be read in transit.

HTTPS (SSL/TLS):

- Step 1: TCP Handshake: Standard connection setup.
- Step 2: Certificate Check: Client says hello. Server responds with hello and its SSL/TLS certificate. That certificate contains the server's public key and is signed by a trusted Certificate Authority.
  Your browser verifies this certificate is legitimate, not expired, and actually belongs to the domain you're trying to reach. This proves you're talking to the real server, not some attacker pretending to be it.
- Step 3: Key Exchange: Here's where asymmetric encryption happens. The server has a public key and a private key. Client generates a session key, encrypts it with the server's public key, and sends it over. Only the server can decrypt this with its private key.
  Both sides now have the same session key that nobody else could have intercepted. This becomes the symmetric encryption key for the rest of the session.
- Step 4: Data Transmission: Now every request and response gets encrypted with that session key using symmetric encryption.

Forward Proxy versus Reverse Proxy



A forward proxy sits between clients (users) and the internet. It acts on behalf of users, hiding their identity or filtering traffic before it reaches the external web.

Some applications of a forward proxy are:

1. Protect users while browsing the internet.
2. Helps organizations restrict access to certain websites.
3. Speeds up web browsing by caching frequently accessed content.

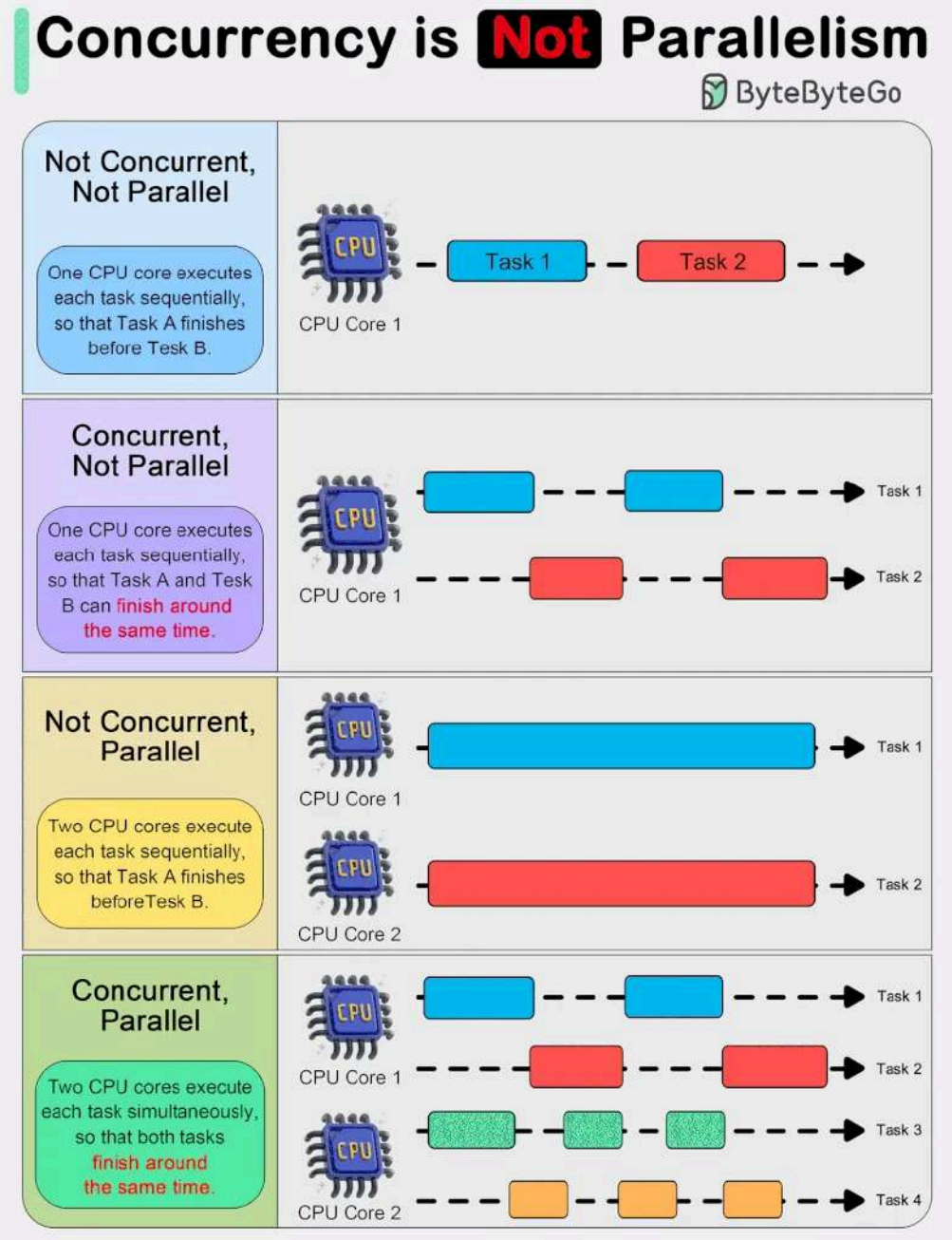
A reverse proxy sits between the internet (clients) and backend servers. It acts on behalf of servers, handling incoming traffic.

Some applications of a reverse proxy are:

1. Distributes traffic across multiple servers to ensure no single server is overwhelmed.
2. Handles SSL encryption/decryption so backend servers don’t have to.
3. Helps protect backend servers from DDoS attacks.

Things Every Developer Should Know: Concurrency is NOT parallelism

In system design, it is important to understand the difference between concurrency and parallelism.



As Rob Pyke(one of the creators of GoLang) stated:“ Concurrency is about **dealing with** lots of things at once. Parallelism is about **doing** lots of things at once.” This distinction emphasizes that concurrency is more about the **design** of a program, while parallelism is about the **execution**.

Concurrency is about dealing with multiple things at once. It involves structuring a program to handle multiple tasks simultaneously, where the tasks can start, run, and complete in overlapping time periods, but not necessarily at the same instant.

Concurrency is about the composition of independently executing processes and describes a program's ability to manage multiple tasks by making progress on them without necessarily completing one before it starts another.

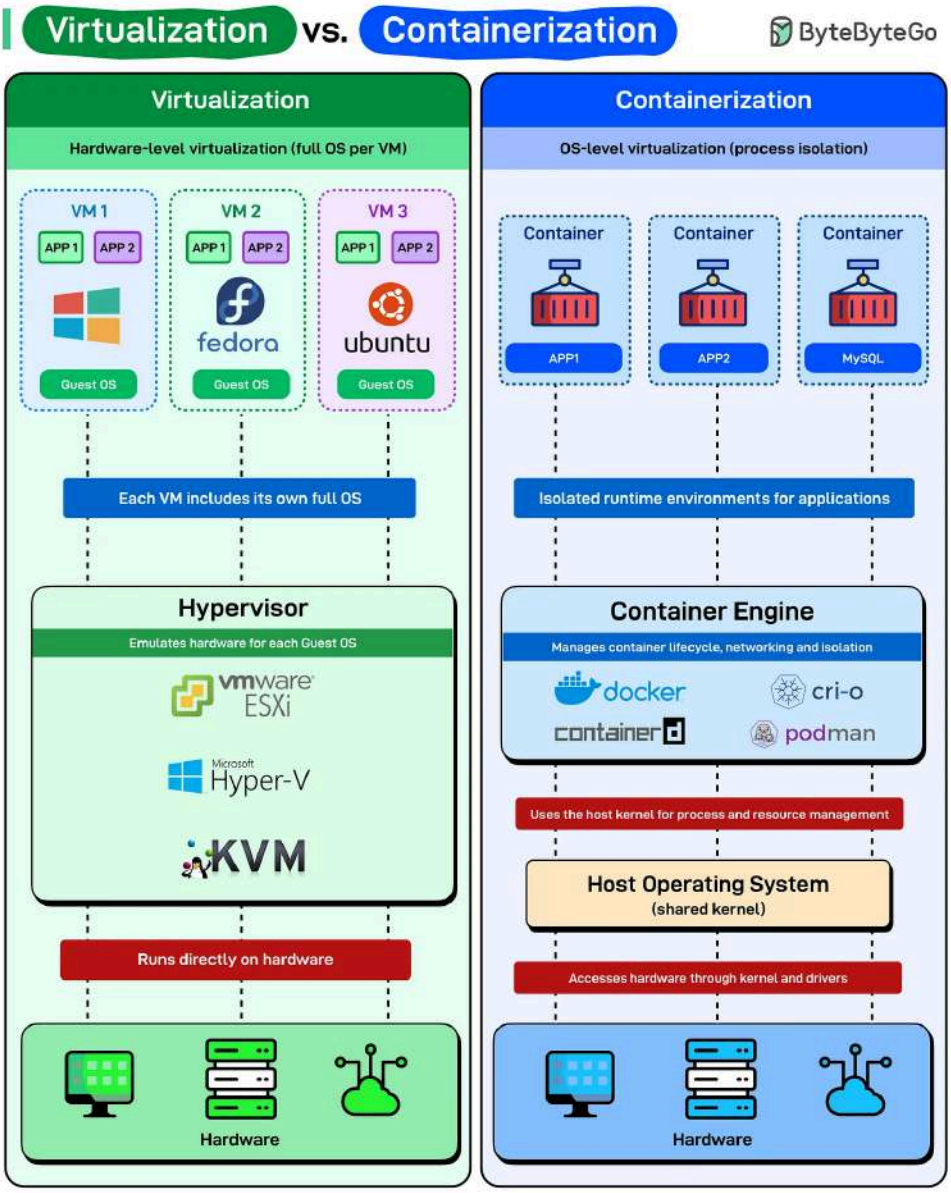
Parallelism, on the other hand, refers to the simultaneous execution of multiple computations. It is the technique of running two or more tasks or computations at the same time, utilizing multiple processors or cores within a computer to perform several operations concurrently. Parallelism requires hardware with multiple processing units, and its primary goal is to increase the throughput and computational speed of a system.

In practical terms, concurrency enables a program to remain responsive to input, perform background tasks, and handle multiple operations in a seemingly simultaneous manner, even on a single-core processor. It's particularly useful in I/O-bound and high-latency operations where programs need to wait for external events, such as file, network, or user interactions.

Parallelism, with its ability to perform multiple operations at the same time, is crucial in CPU-bound tasks where computational speed and throughput are the bottlenecks. Applications that require heavy mathematical computations, data analysis, image processing, and real-time processing can significantly benefit from parallel execution.

Virtualization vs. Containerization

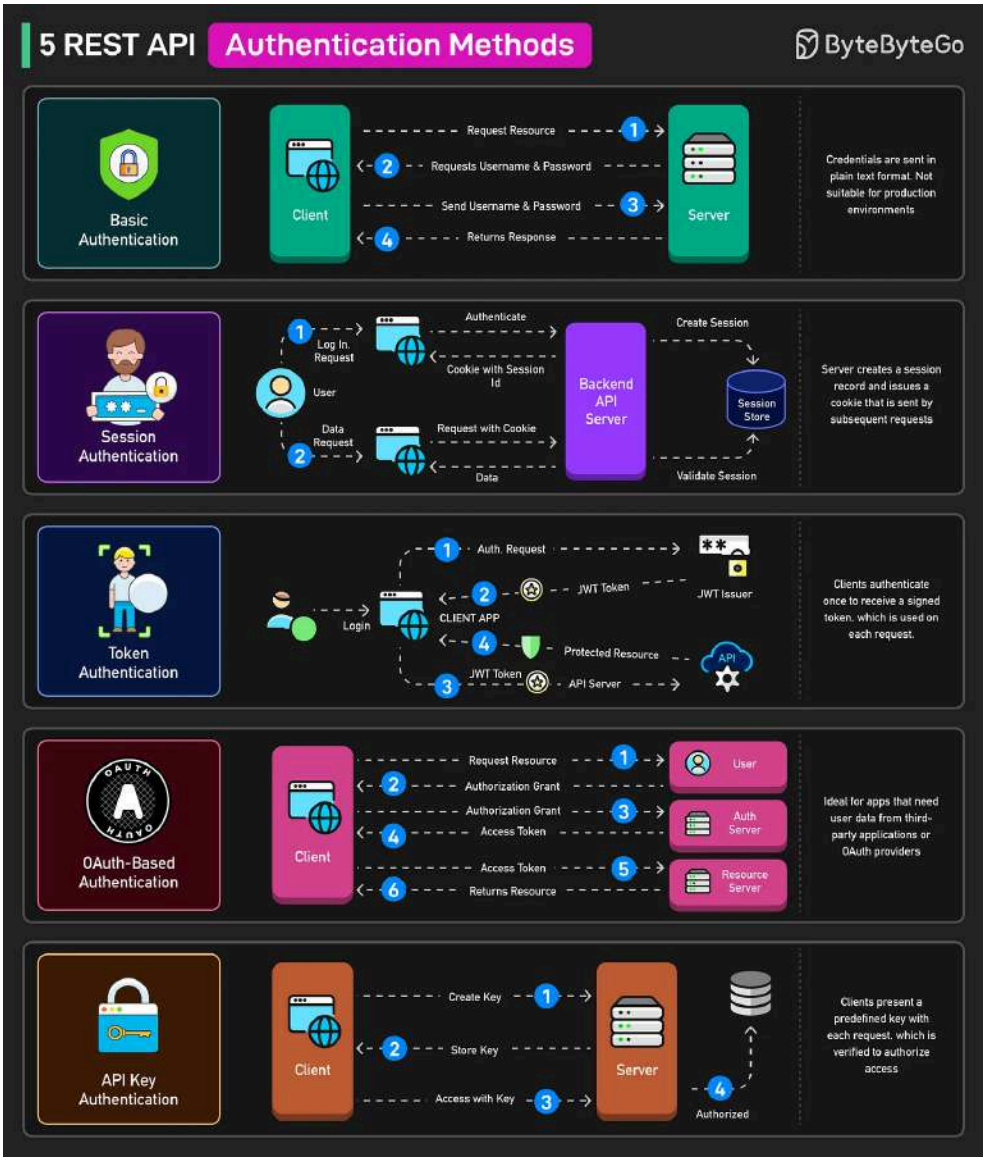
Before containers simplified deployment, virtualization changed how we used hardware. Both isolate workloads, but they do it differently.



- Virtualization (Hardware-level isolation): Each virtual machine runs a complete operating system, Windows, Fedora, or Ubuntu, with its own kernel, drivers, and libraries. The hypervisor (VMware ESXi, Hyper-V, KVM) sits directly on hardware and emulates physical machines for each guest OS.
  This makes VMs heavy but isolated. Need Windows and Linux on the same box? VMs handle it easily. Startup time for a typical VM is in minutes because you're booting an entire operating system from scratch.

- Containerization (OS-level isolation): Containers share the host operating system's kernel. No separate OS per container. Just isolated processes with their own filesystem and dependencies.
  The container engine (Docker, containerd, CRI-O, Podman) manages lifecycle, networking, and isolation, but it all runs on top of a single shared kernel. Lightweight and fast. Containers start in milliseconds because you're not booting an OS, just launching a process.
  But here's the catch: all containers on a host must be compatible with that host's kernel. Can't run Windows containers on a Linux host (without nested virtualization tricks).

5 REST API Authentication Methods



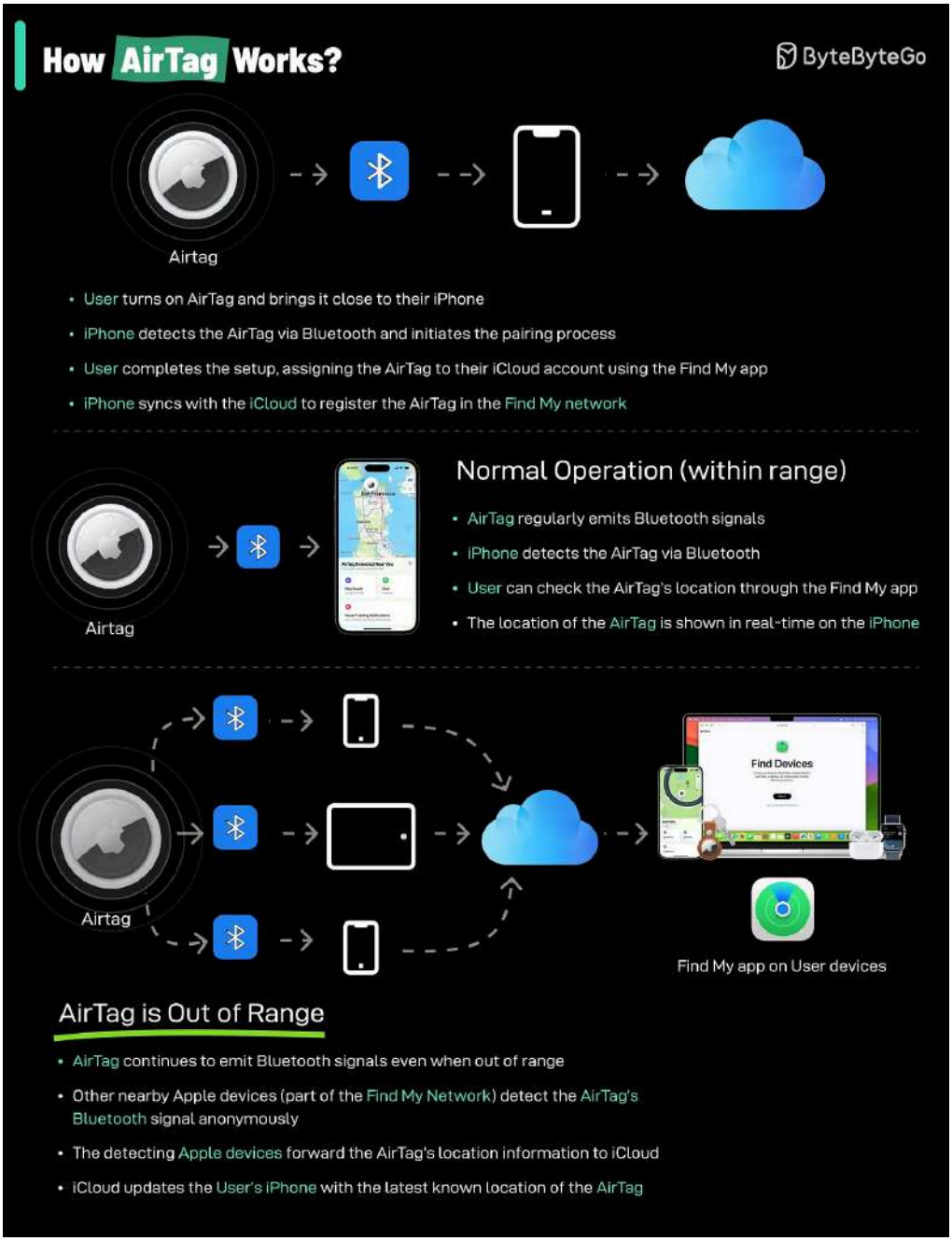
1. Basic Authentication: Clients include a Base64-encoded username and password in every request header, which is simple but insecure since credentials are transmitted in plaintext. Useful in quick prototypes or internal services over secure networks.

2. Session Authentication: After login, the server creates a session record and issues a cookie. Subsequent requests send that cookie so the server can validate user state. Used in traditional web-apps.

3. Token Authentication: Clients authenticate once to receive a signed token, then present the token on each request for stateless authentication. Used in single-page applications and modern APIs that require scalable, stateless authentication.

4. OAuth-Based Authentication: Clients obtain an access token via an authorization grant from an OAuth provider, then use that token to call resource servers on the user's behalf. Used in cases of third-party integrations or apps that need delegated access to user data.
5. API Key Authentication: Clients present a predefined key (often in headers or query strings) with each request. The server verifies the key to authorize access. Used in service-to-service or machine-to-machine APIs where simple credential checks are sufficient.

How do AirTags work?



AirTags work by leveraging a combination of Bluetooth technology and the vast network of Apple devices to help you locate your lost items.
Here's a breakdown of how they function:

1. Bluetooth Signal: Each AirTag emits a secure Bluetooth signal that can be detected by nearby Apple devices (iPhones, iPads, etc.) within the Find My network.

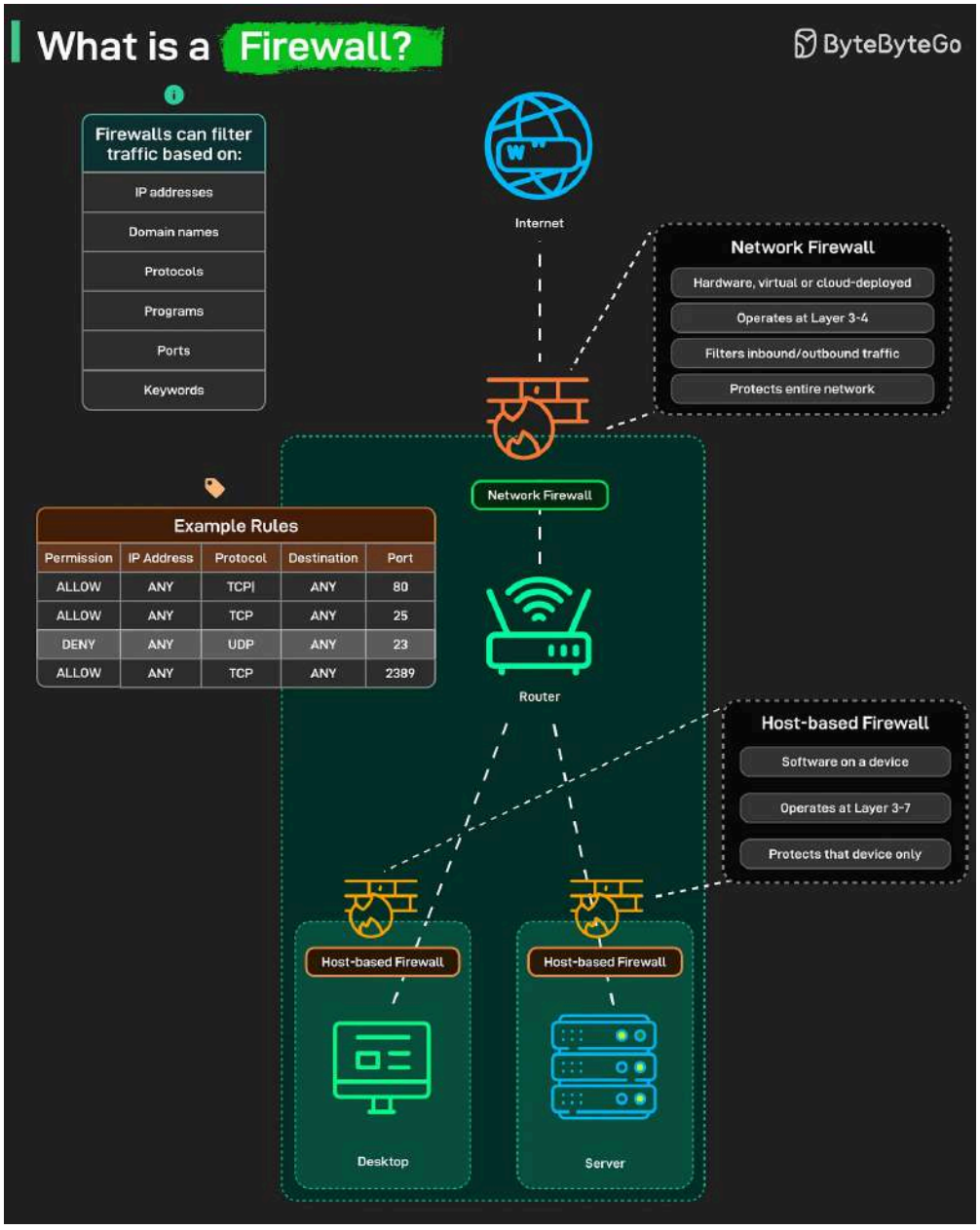
2. Find My Network: When an AirTag comes within range of an Apple device in the Find My network, that device anonymously and securely relays the AirTag's location information to iCloud.

3. Location Tracking: You can then use the Find My app on your own Apple device to see the approximate location of your AirTag on a map.

Limitations:
Please note that AirTags rely on Bluetooth technology and the presence of Apple devices within the Find My network. If your AirTag is in an area with few Apple devices, its location may not be updated as frequently or accurately.

What is a Firewall?

Every time you connect to the Internet, a firewall quietly decides what can come in and what must stay out. A firewall is your network's first line of defense. It filters traffic based on rules you define, by IP address, protocol, port, program, or even keywords. Every packet that tries to enter or leave your network passes through this checkpoint.



There are two main types:

1. Network Firewall: Sits at the network edge between your infrastructure and the internet. Can be physical hardware, virtualized software, or cloud-deployed service. Operates at Layer 3-4

of the OSI model. Filters traffic based on IP addresses, protocols, and ports before it ever reaches your internal network.

Protects the entire network at once. This is your first line of defense. Internet traffic hits the network firewall before it reaches your router, before it touches any internal systems.

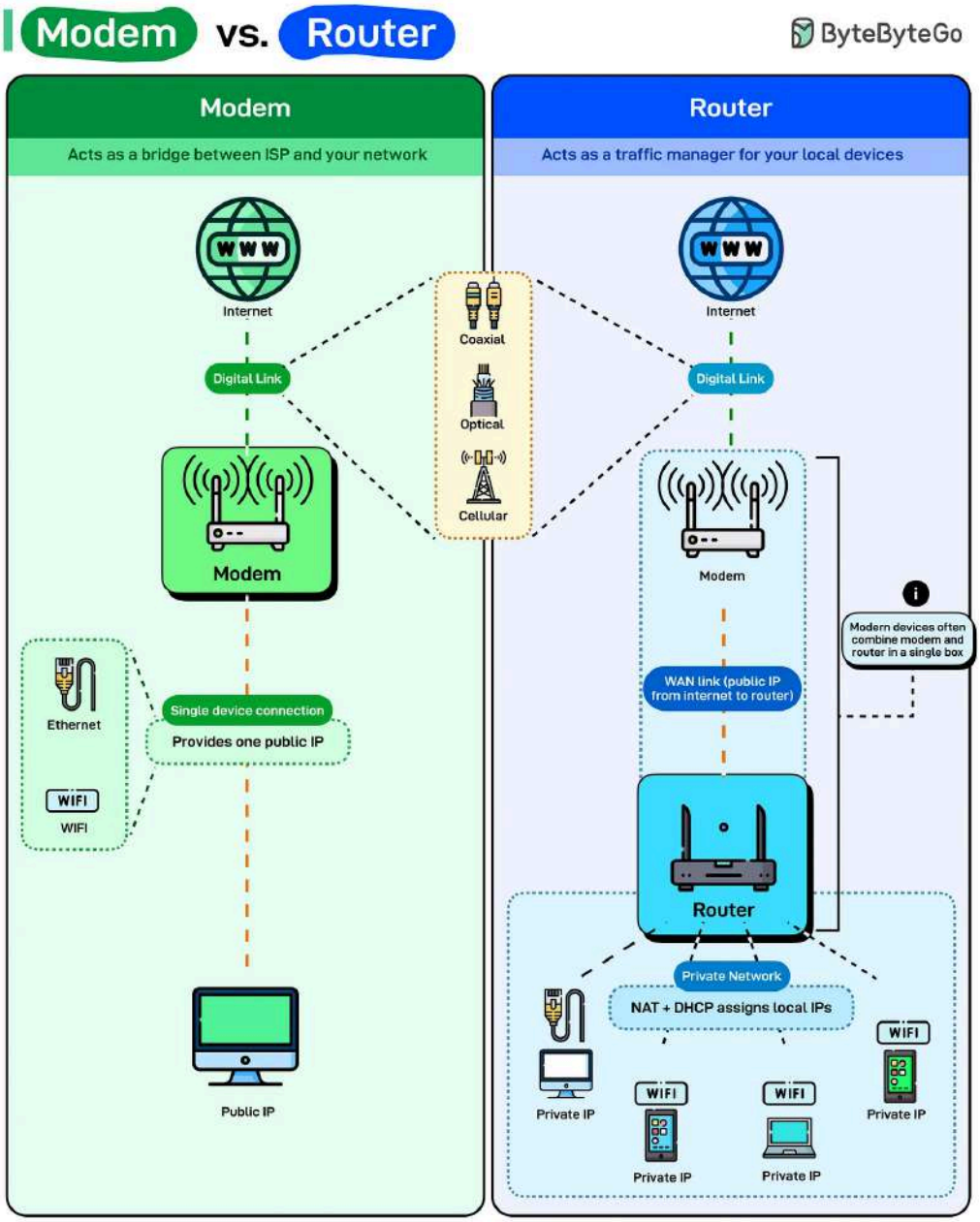
2. Host-based Firewall: This runs as software on individual devices, like your laptop or a server. It works at Layer 3–7, inspecting packets more deeply and protecting only that specific device.

   Your desktop has its own host firewall. Your server has its own. Each one is configured independently. It’s your last layer of defense in case something slips past the network firewall.

   Together, they form a layered shield, keeping unwanted traffic out while letting legitimate communication flow freely.

Modem vs. Router

Most people think their WiFi router gives them internet. It doesn't. Your router is just managing traffic inside your home. The actual internet connection comes from the modem.



Here's what each one actually does:

- Modem: The modem connects you to your Internet Service Provider (ISP). It translates signals between your ISP's network and your home network. Depending on the service type, the digital link may use coaxial cable, fiber optic, or cellular connections. The modem converts those signals into data your devices can understand.

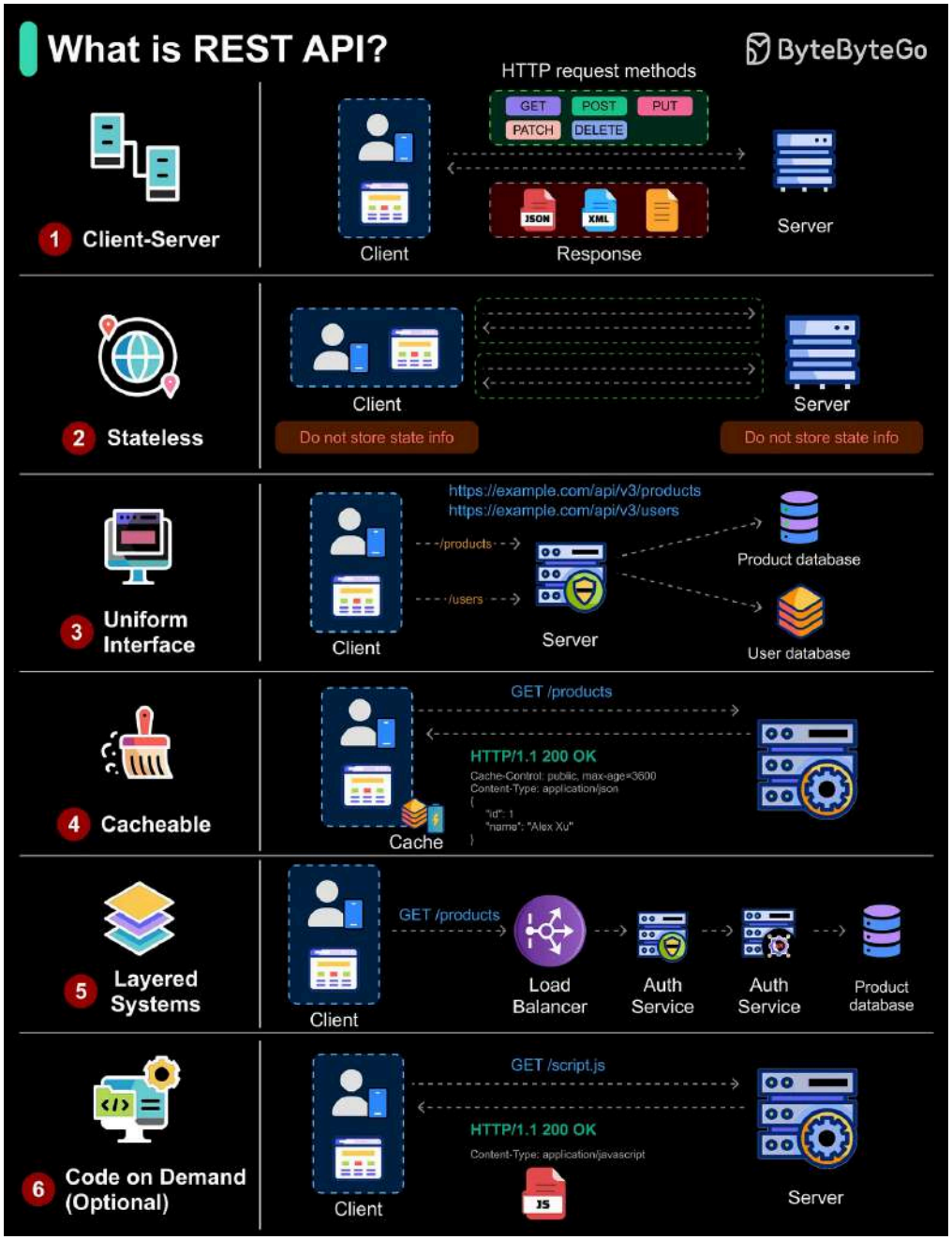
It provides one public IP address, meaning one connection to the internet. If you plug a single device directly into a modem via Ethernet, that device gets internet access with a public IP.

- Router: The router creates a private network inside your home. It takes that single public IP from the modem and shares it across multiple devices using Network Address Translation (NAT). Every device on your network gets a private IP address, usually something like 192.168.1.x. The router keeps track of which device requested what data and routes responses back to the right device.

  DHCP assigns those private IPs automatically. Your phone connects to WiFi, the router gives it an IP address, and suddenly it can talk to the internet through the router.

Modern devices often combine both functions, a modem-router combo, but understanding the distinction helps when you're troubleshooting slow speeds or network drops.

What is a REST API?



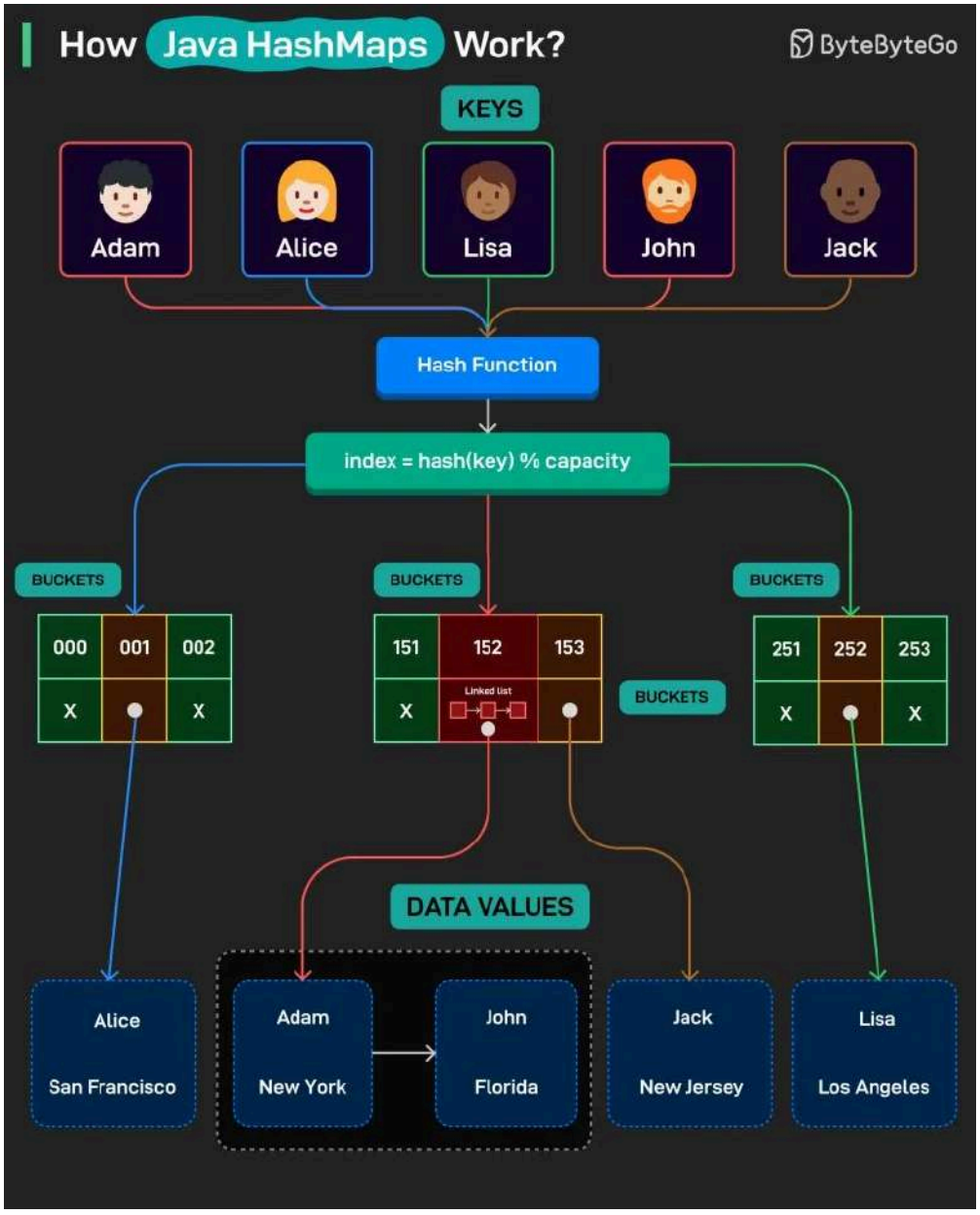
REST (Representational State Transfer) is an architectural style for building APIs that use HTTP for communication. To be considered RESTful, an API should follow six key constraints:

- Client-Server: Separates the user interface from data storage and processing. This allows each side to evolve independently.
- Stateless: Each request contains all the information needed to process it; the server does not store session state between requests.

- Uniform Interface: Consistent resource naming and formats across the API, e.g., “/products”, “/users”.
- Cacheable: Responses explicitly indicate if they can be cached (e.g., Cache-Control) to improve performance.
- Layered System: Requests may pass through multiple layers (load balancers, auth services, etc.) before reaching the API server, but to the client it appears as a single endpoint.
- Code on Demand (Optional): The server can send executable code (like JavaScript) to the client to extend its functionality.

How Java HashMaps Work?

A HashMap is a data structure that stores key-value pairs, allowing fast lookup, insertion, and deletion. It uses a hash function to map each key to a bucket index, making access efficient.

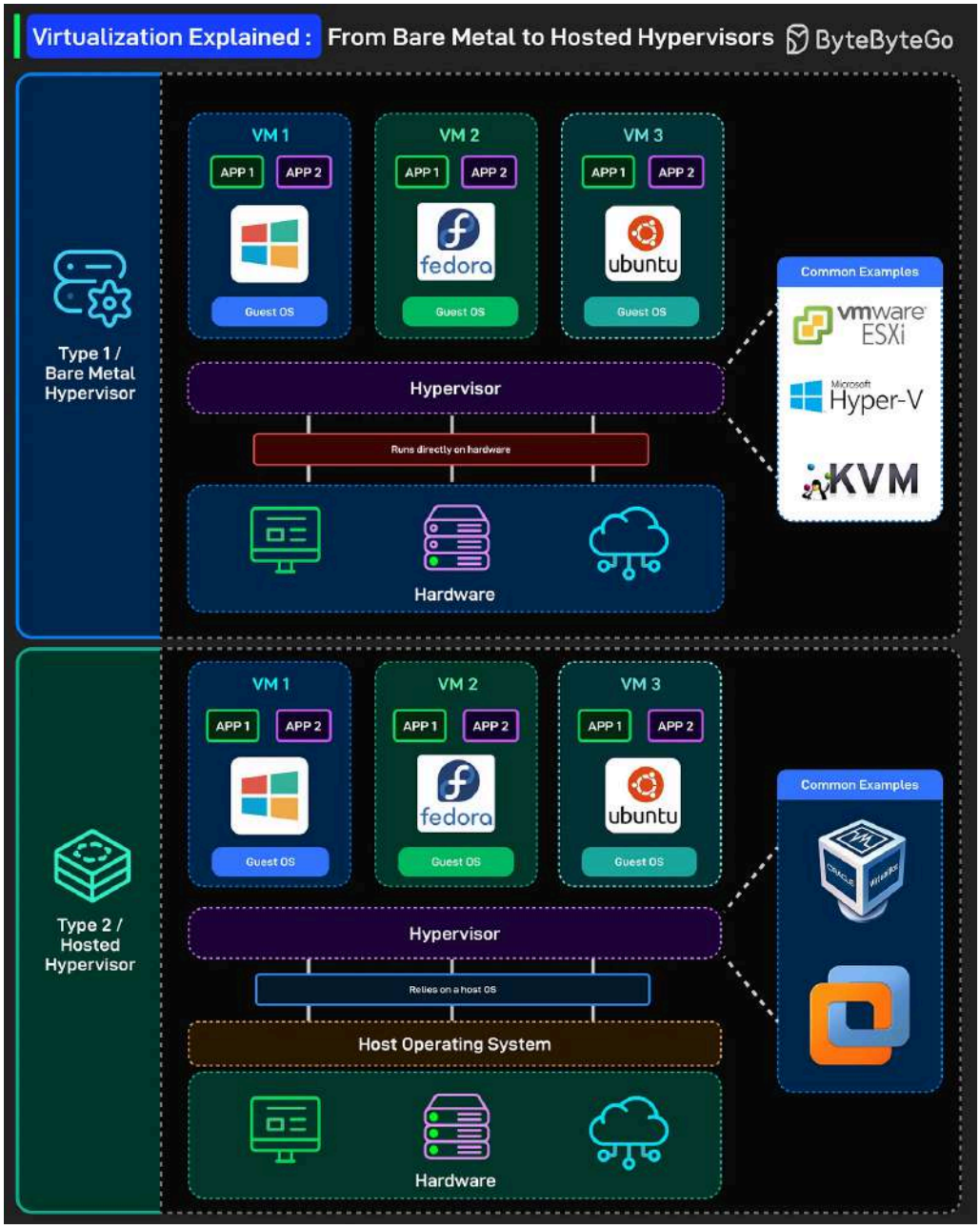


Here's how it works:

1. Keys are given to a HashMap
2. A hash function converts each key into a number that points to a bucket index that points to the data value.

3. When two keys map to the same bucket index, a collision occurs.
4. Collisions are handled by linking entries together in a chain within the same bucket. This is done using a linked list or even trees in some cases.
5. Finally, values are retrieved quickly by hashing the key and looking up the correct bucket.

Virtualization Explained: From Bare Metal to Hosted Hypervisors



Before containers and serverless, there was virtualization, the foundation of modern cloud computing. Here's a simple breakdown of how virtualization works, from bare metal to hosted hypervisors:

- Type 1 / Bare Metal Hypervisor: The hypervisor runs directly on hardware. No operating system underneath it. The hypervisor is the operating system layer. VMware ESXi, Microsoft Hyper-V, and KVM are all Type 1 hypervisors. They boot up on physical servers and manage VMs with direct hardware access.

  Each VM gets its own full operating system. Windows, Fedora, Ubuntu. Complete isolation between VMs. One crashes? The others keep running.

  This is what runs in data centers and cloud providers. When you spin up an EC2 instance on AWS, it's running on Type 1 virtualization (they use their custom Nitro hypervisor now, but same concept).

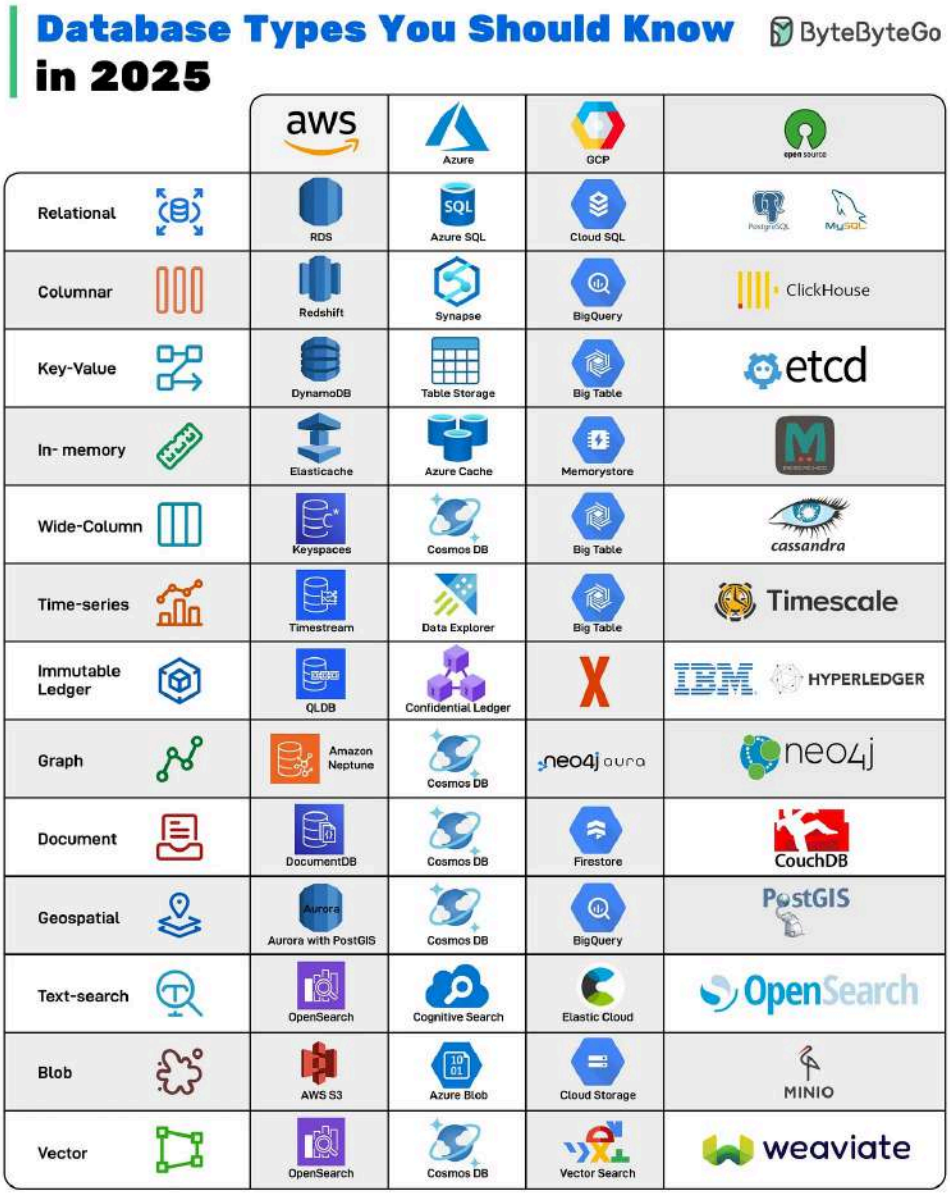
- Type 2 / Hosted Hypervisor: The hypervisor runs as an application on top of a host operating system. You're running Windows or macOS, then you install VirtualBox or VMware Workstation on top of it.

  Guest VMs still get full operating systems, but now there's an extra layer. Your VM talks to the hypervisor, which talks to the host OS, which talks to the hardware.

  More overhead, slightly lower performance, but way more convenient for development and testing. This is what you use on your laptop. Running a Linux VM on your MacBook? That's Type 2.

Database Types You Should Know in 2025

There’s no such thing as a one-size-fits-all database anymore. Modern applications rely on multiple database types, from real-time analytics to vector search for AI. Knowing which type to use can make or break your system’s performance.

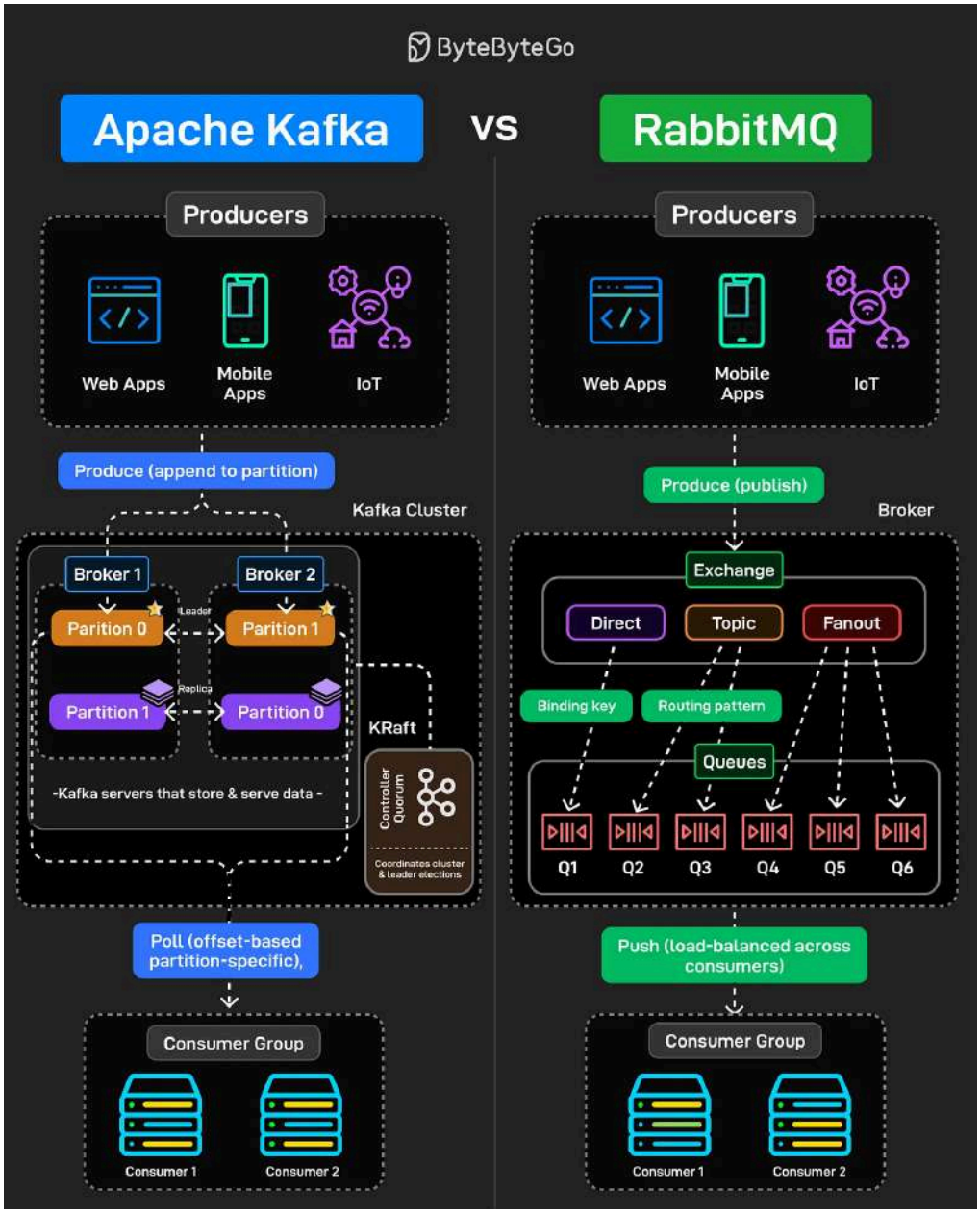


- Relational: Traditional row-and-column databases, great for structured data and transactions.
- Columnar: Optimized for analytics, storing data by columns for fast aggregations.
- Key-Value: Stores data as simple key–value pairs, enabling fast lookups.

- In-memory: Stores data in RAM for ultra-low latency lookups, ideal for caching or session management.
- Wide-Column: Handles massive amounts of semi-structured data across distributed nodes.
- Time-series: Specialized for metrics, logs, and sensor data with time as a primary dimension.
- Immutable Ledger: Ensures tamper-proof, cryptographically verifiable transaction logs.
- Graph: Models complex relationships, perfect for social networks and fraud detection
- Document: Flexible JSON-like storage, great for modern apps with evolving schemas.
- Geospatial: Manages location-aware data such as maps, routes, and spatial queries.
- Text-search: Full-text indexing and search with ranking, filters, and analytics.
- Blob: Stores unstructured objects like images, videos, and files.
- Vector: Powers AI/ML apps by enabling similarity search across embeddings.

Apache Kafka vs. RabbitMQ

Kafka and RabbitMQ both handle messages, but they solve fundamentally different problems. Understanding the difference matters when designing distributed systems.



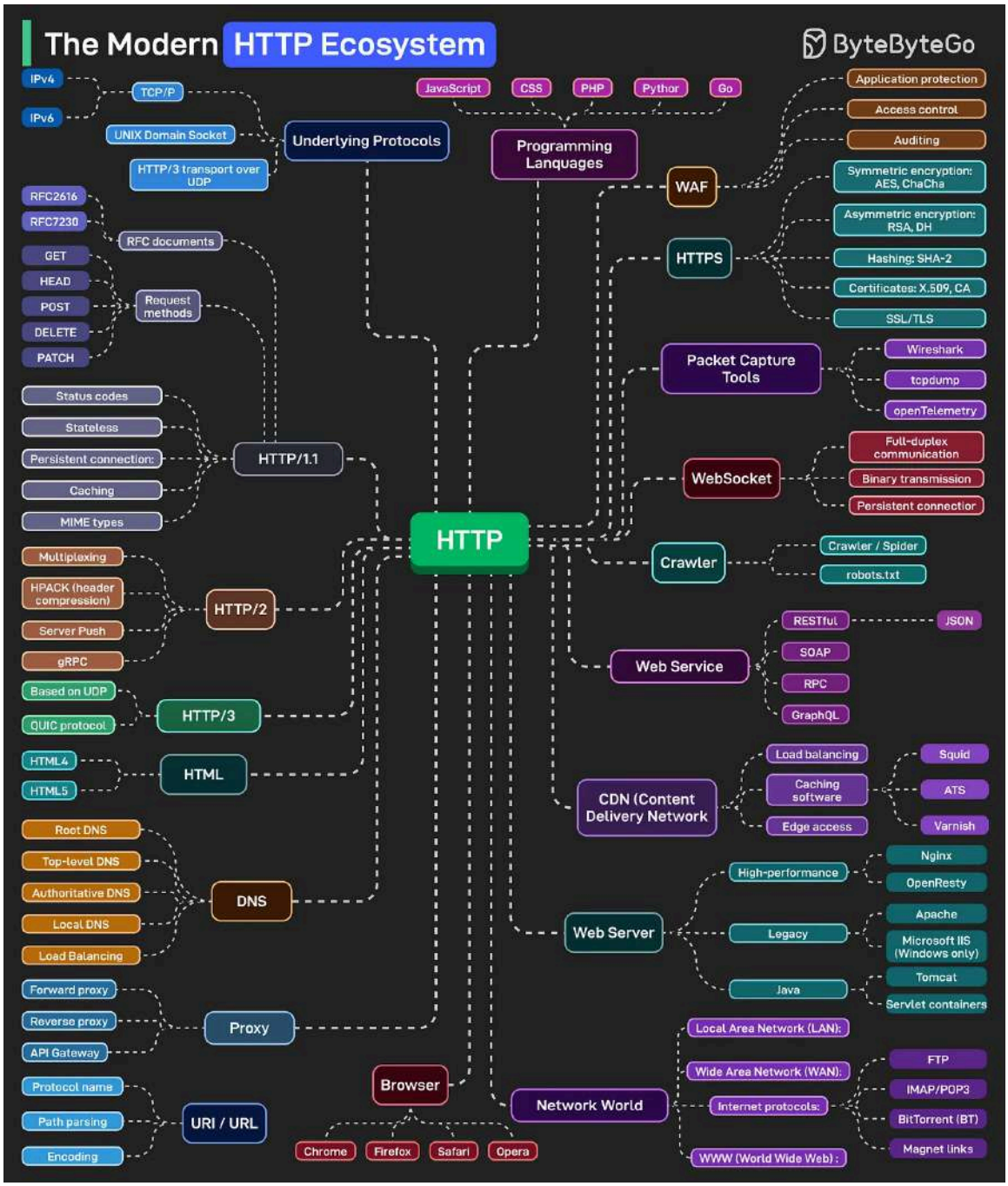
Kafka is a distributed log. Producers append messages to partitions. Those messages stick around based on retention policy, not because someone consumed them. Consumers pull messages at their own pace using offsets. You can rewind, replay, reprocess everything. It is designed for high throughput event streaming where multiple consumers need the same data independently.

RabbitMQ is a message broker. Producers publish messages to exchanges. Those exchanges route to queues based on binding keys and patterns (direct, topic, fanout). Messages get pushed to consumers and then deleted once acknowledged. It is built for task distribution and traditional messaging workflows.

The common mistake is using Kafka like a queue or RabbitMQ like an event log. They’re different tools built for different use cases.

The HTTP Mindmap

HTTP has evolved from HTTP/1.1 to HTTP/2, and now HTTP/3, which uses the QUIC protocol over UDP for improved performance. Today, it's the backbone of almost everything on the internet, from browsers and APIs to streaming, cloud, and AI systems.



At the foundation, we have underlying protocols. TCP/IP for IPv4 and IPv6 traffic. Unix domain sockets for local communication. HTTP/3 running over UDP instead of TCP. These handle the actual data transport before HTTP even comes into play.

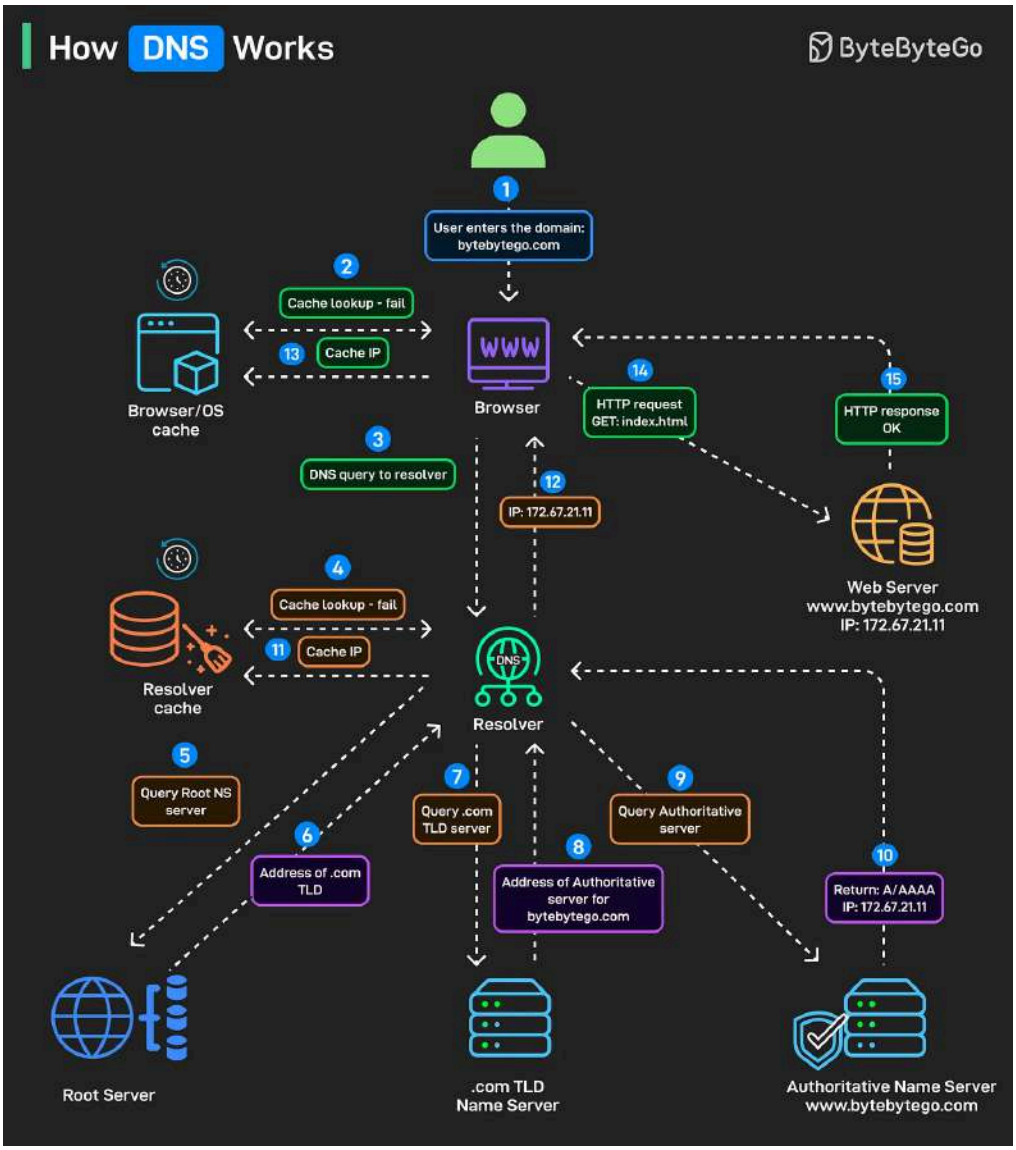
Security wraps around everything. HTTPS isn't optional anymore. WebSockets power real-time connections. Web servers manage workloads. CDNs distribute content globally. DNS resolves everything to IPs. Proxies (forward, reverse, and API gateways) route, filter, and secure traffic in between.

Web services exchange data in different formats, REST with JSON, SOAP for enterprise systems, RPC for direct calls, and GraphQL for flexible queries. Crawlers and bots index the web, guided by robots.txt files that set the boundaries.

The network world connects everything. LANs, WANs, and protocols like FTP for file transfers, IMAP/POP3 for email, and BitTorrent for peer-to-peer communication. For observability, packet capture tools like Wireshark, tcpdump, and OpenTelemetry let developers peek under the hood to understand performance, latency, and behavior across the stack.

How DNS Works

You type a domain name and hit enter, but what actually happens before that webpage loads is more complex than most people realize. DNS is the phonebook of the internet, and every request you make triggers a chain of lookups across multiple servers.



Step 1: Someone types bytebytego. com into their browser and presses enter.

Step 2: Before doing anything, the browser looks for a cached IP address. Operating system cache gets checked too.

Step 3: Cache miss triggers a DNS query. The browser sends a query to the configured DNS resolver, usually provided by your ISP or a service like Google DNS or Cloudflare.

Step 4: Resolver checks its own cache.

Step 5-6: If the resolver doesn’t have the answer cached, it asks the root servers, “Where can I find .com?” For bytebytego. com, the root server responds with the address of the .com TLD name server.

Step 7-8: Resolver queries the .com TLD server. TLD server returns the authoritative server address.

Step 9-10: This server has the actual A/AAAA record mapping the domain to an IP address. The resolver finally gets the answer: 172. 67. 21. 11 for bytebytego. com.

Step 11-12: The IP gets cached at the resolver level for future lookups, and returned to the browser.

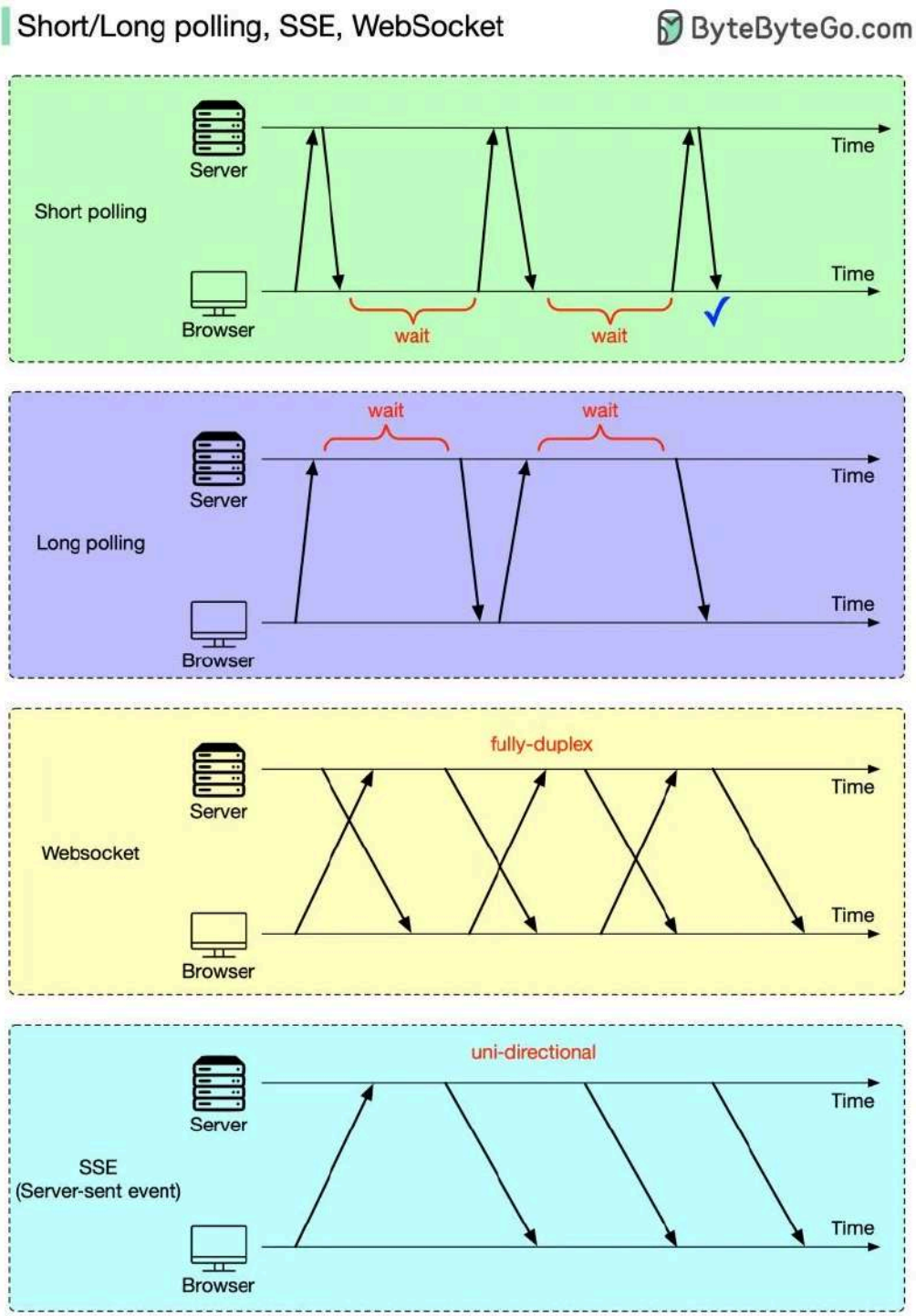
Step 13-14: The browser stores this for its own future use, and uses the IP to make the actual HTTP request.

Step 15: The web server returns the requested content.

All this happens in milliseconds, before your first page even starts loading.

Can a web server provide real-time updates?

An HTTP server cannot automatically initiate a connection to a browser. As a result, the web browser is the initiator. What should we do next to get real-time updates from the HTTP server?

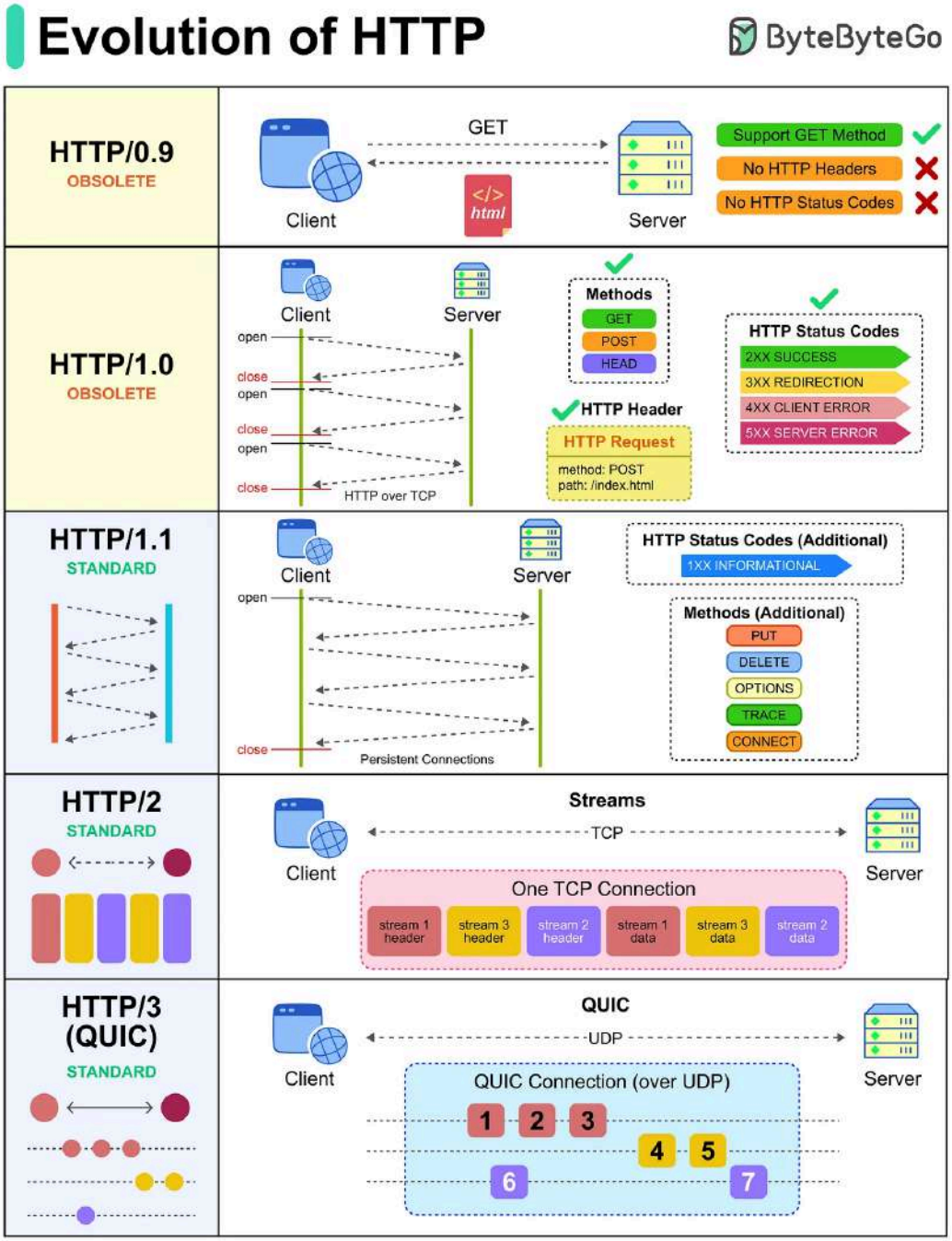


Both the web browser and the HTTP server could be responsible for this task.

- Web browsers do the heavy lifting: short polling or long polling. With short polling, the browser will retry until it gets the latest data. With long polling, the HTTP server doesn’t return results until new data has arrived.
- HTTP server and web browser cooperate: WebSocket or SSE (server-sent event). In both cases, the HTTP server could directly send the latest data to the browser after the connection is established. The difference is that SSE is uni-directional, so the browser cannot send a new request to the server, while WebSocket is fully-duplex, so the browser can keep sending new requests.

Evolution of HTTP

The Hypertext Transfer Protocol (HTTP) has evolved over the years to meet the needs of modern applications, from simple text delivery to high-performance, real-time experiences.



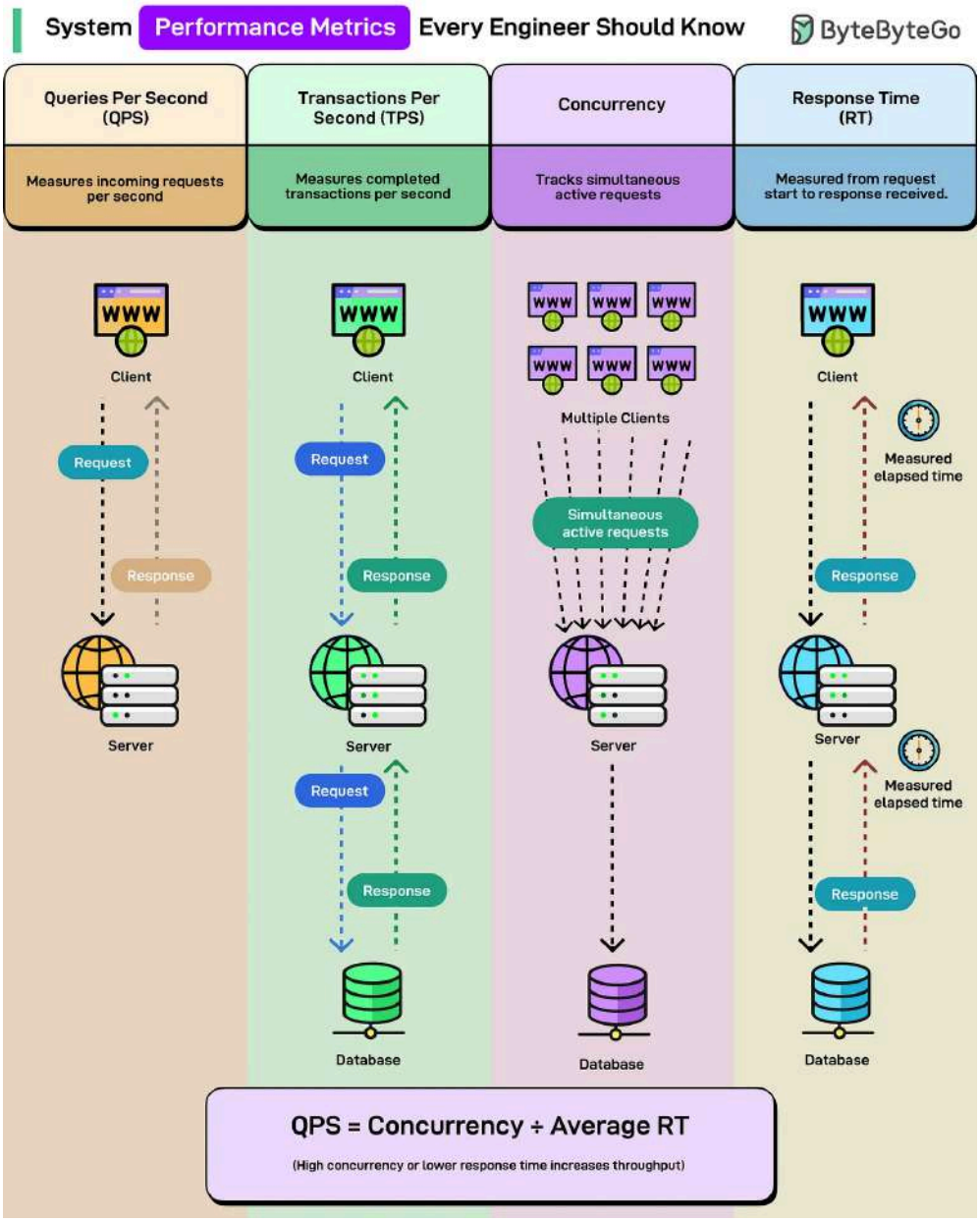
Here is how HTTP has progressed:

- HTTP/0.9: Built to fetch simple HTML documents with a single GET request.

- HTTP/1.0: Added headers and status codes to support richer interactions, but every request still required a new connection.
- HTTP/1.1: Introduced persistent connections and more methods, making the web faster and more efficient for everyday browsing.
- HTTP/2: Solved performance bottlenecks with multiplexing, enabling multiple requests to share one connection.
- HTTP/3 (QUIC): Shifted to UDP with QUIC to reduce latency and improve reliability, especially for mobile and real-time apps.

System Performance Metrics Every Engineer Should Know

Your API is slow. But how slow, exactly? You need numbers. Real metrics that tell you what's actually broken and where to fix it.



Here are the four core metrics every engineer should know when analyzing system performance:

- Queries Per Second (QPS): How many incoming requests your system handles per second. Your server gets 1,000 requests in one second? That's 1,000 QPS. Sounds straightforward

until you realize most systems can't sustain their peak QPS for long without things starting to break.

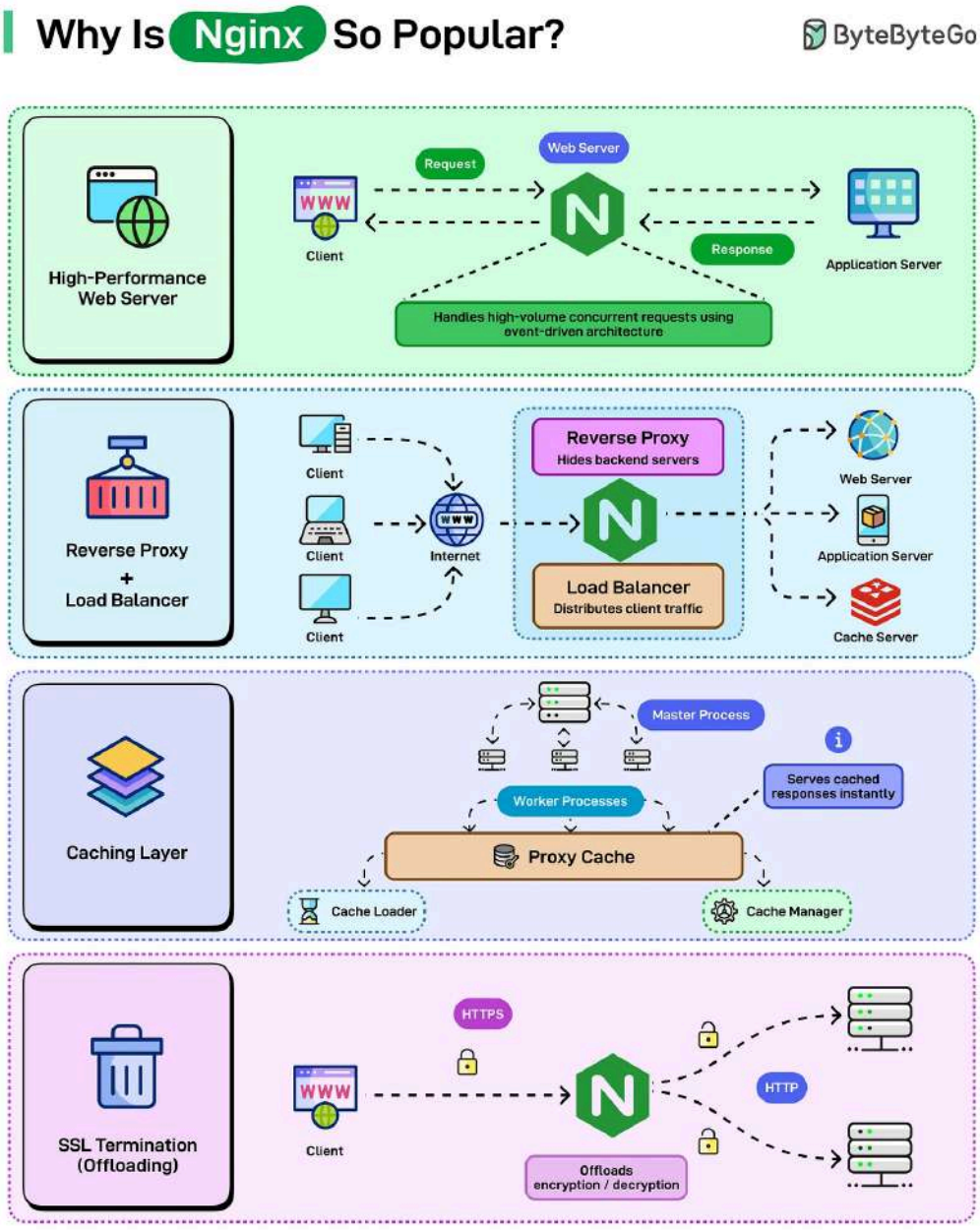
- Transactions Per Second (TPS): How many completed transactions your system processes per second. A transaction includes the full round trip, i.e., the request goes out, hits the database, and comes back with a response.
  TPS tells you about actual work completed, not just requests received. This is what your business cares about.

- Concurrency: How many simultaneous active requests your system is handling at any given moment. You could have 100 requests per second, but if each takes 5 seconds to complete, you're actually handling 500 concurrent requests at once.
  High concurrency means you need more resources, better connection pooling, and smarter thread management.

- Response Time (RT): The elapsed time from when a request starts until the response is received. Measured at both the client level and server level.
  A simple relationship ties them all together: QPS = Concurrency ÷ Average Response Time
  More concurrency or lower response time = higher throughput.

Why Is Nginx So Popular?

Apache dominated web servers for 20 years, then Nginx showed up and changed everything. Now Nginx powers some of the largest sites on the internet, including Netflix, Airbnb, Dropbox, and WordPress. com. Not because it's newer or trendier, but because it solves problems that Apache couldn't handle efficiently.



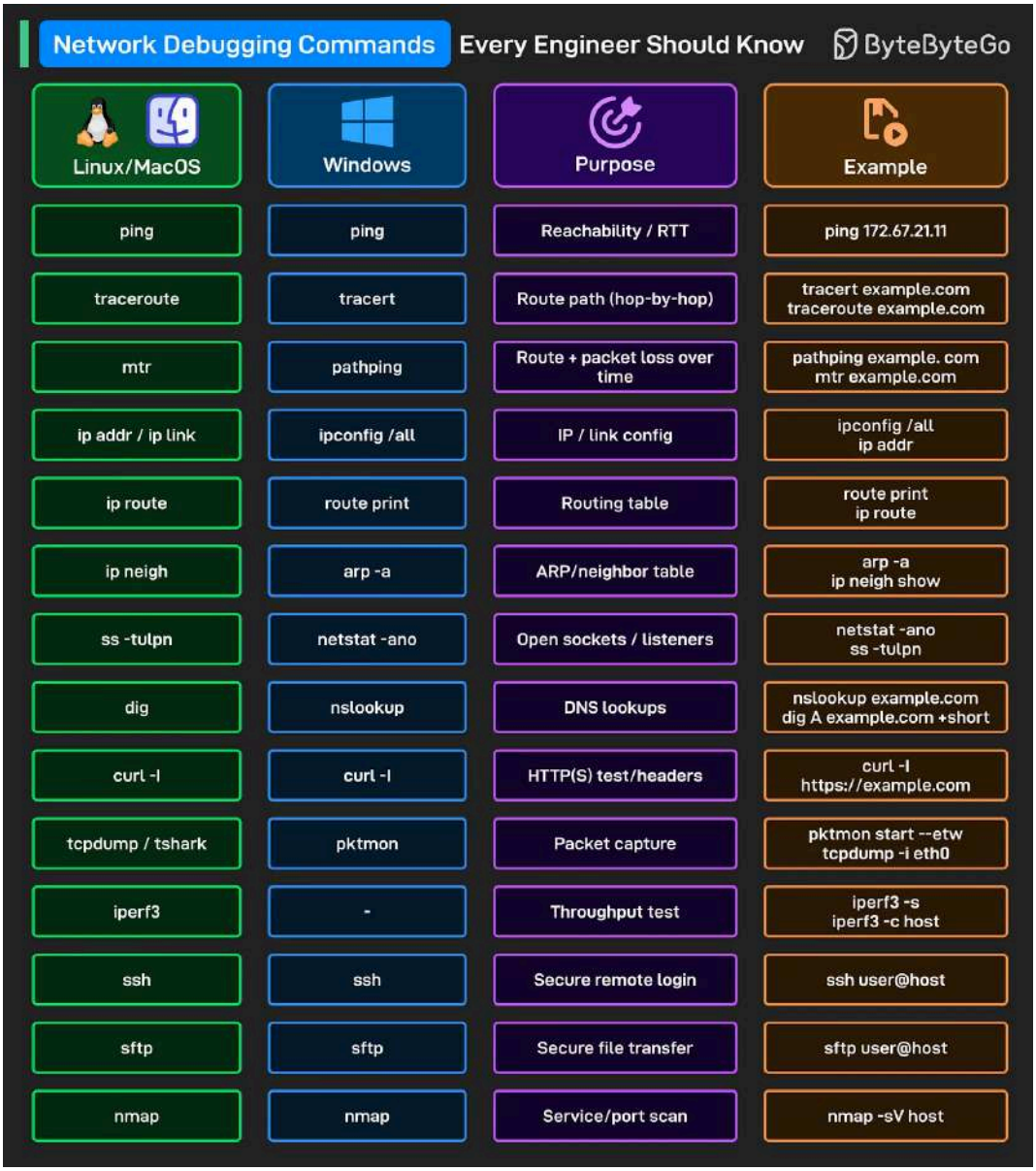
Here’s what makes Nginx so popular:

- High-Performance Web Server

- Reverse Proxy & Load Balancer
- Caching Layer
- SSL Termination (Offloading)

Network Debugging Commands Every Engineer Should Know

When someone says “It’s a network issue,” these commands help you find what’s wrong fast.

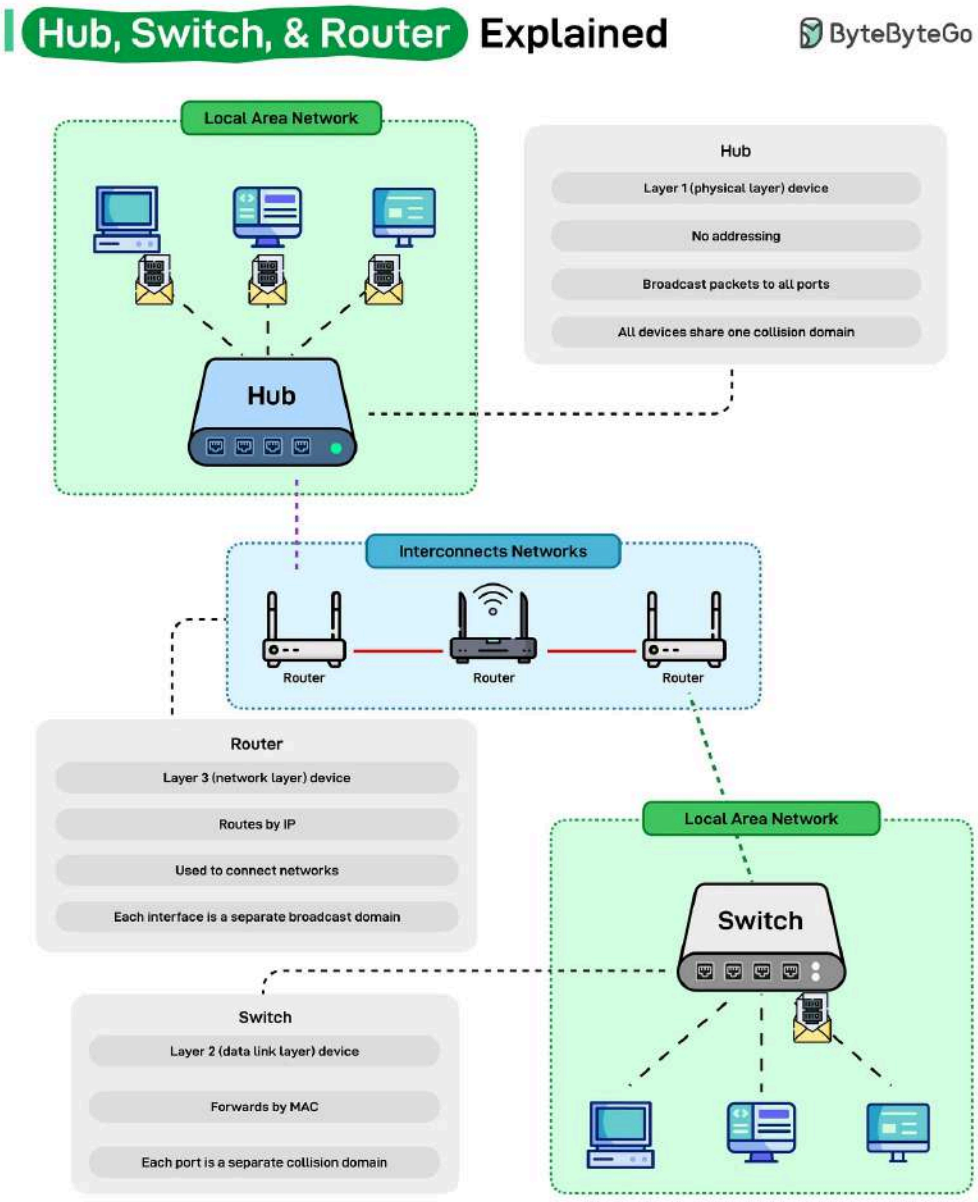


- ping: Checks if the destination responds and reports the round-trip time for basic reachability.
- traceroute / tracert: Shows each hop on the path so you can see where packets slow down or stop.
- mtr / pathping: Continuously measures latency and loss per hop to catch intermittent issues.

- ip addr, ip link / ipconfig /all: Prints local IPs, MACs, and interface status so you can verify the machine's network identity.
- ip route: Reveals the routing table to confirm which gateway and next hop the system will use.
- ip neigh: Displays IP-to-MAC entries to detect duplicates or stale ARP records on the LAN.
- ss -tulpn: Lists listening sockets and PIDs so you can confirm a service is actually bound to the expected port.
- dig: Resolves DNS records to verify the exact IPs clients will connect to.
- curl -I: Fetches only HTTP(S) headers to check status codes, redirects, and cache settings.
- tcpdump / tshark: Captures packets so you can inspect real traffic and validate what's sent and received.
- iperf3: Measures end-to-end throughput between two hosts to separate bandwidth limits from app issues.
- ssh: Opens a secure shell on the remote machine to run checks and apply fixes directly.
- sftp: Transfers files securely so you can pull logs or push artifacts during an incident.
- nmap: Scans open ports and probes versions to confirm which services are exposed and responding.

Hub, Switch, & Router Explained

Every home and office network relies on these three devices, hub, switch, and router, yet their roles are often mixed up.



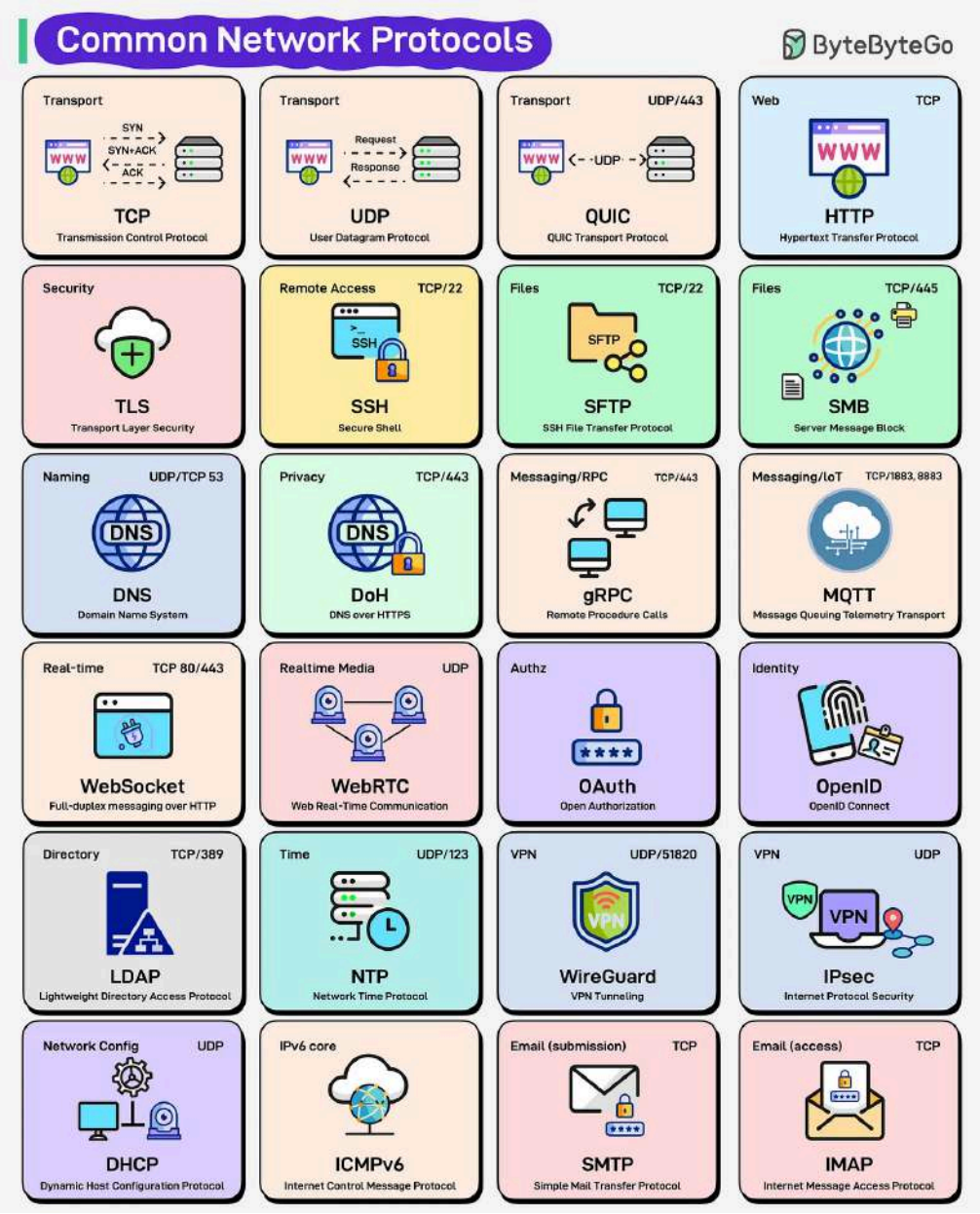
- A hub operates at Layer 1 (Physical Layer). It's the simplest of the three, it doesn't understand addresses or data types. When a packet arrives, it simply broadcasts it to every connected device, creating one big collision domain. That means all devices compete for the same bandwidth, making hubs inefficient in modern networks.

- A switch works at Layer 2 (Data Link Layer). It learns MAC addresses and forwards frames only to the correct destination device. Each port on a switch acts as its own collision domain, improving efficiency and speeding up communication within a LAN.
- A router operates at Layer 3 (Network Layer). It routes packets based on IP addresses and connects different networks together, for example, your home network to the Internet. Each router interface forms a separate broadcast domain, keeping local and external traffic isolated.

Understanding how these three layers work together is the foundation of every modern network, from your home Wi-Fi to the global Internet backbone.

Common Network Protocols Every Engineer Should Know

Ever wonder what actually happens when you click "Send" on an email or join a video call? Every click, message, and API call on the internet relies on network protocols. They define how data moves, who can talk, and how securely it all happens.



At the foundation are transport protocols: TCP ensures reliable delivery, UDP prioritizes speed, and QUIC brings both worlds together over UDP.

On top of that, HTTP powers the web, TLS secures it, and DNS translates names into addresses.

Need remote access? That’s SSH. File transfers? SFTP or SMB.

Real-time chat and media? WebSocket, WebRTC, and MQTT keep data flowing live.

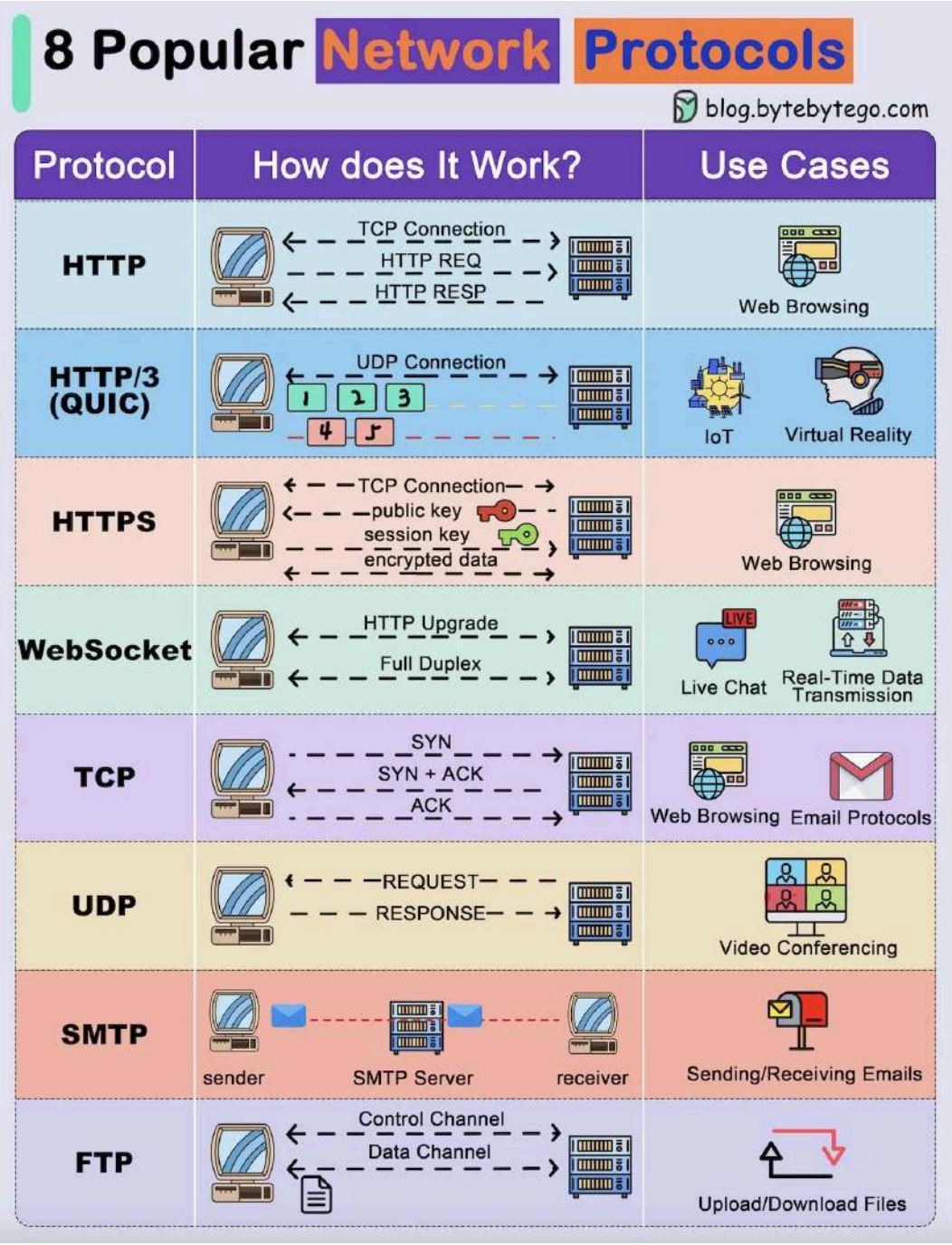
For identity and access, OAuth and OpenID handle authorization and authentication.

In the backend, DHCP, NTP, ICMPv6, and LDAP quietly keep everything synchronized, addressed, and discoverable.

From simple emails (SMTP, IMAP) to encrypted VPNs (WireGuard, IPsec), these protocols form the invisible language that keeps the internet connected and secure.

8 Popular Network Protocols

Network protocols are the key to transferring data between two systems in a network.



1. FTP (File Transfer Protocol)
   Uses separate control and data channels to upload and download files between a client and server.

2. TCP (Transmission Control Protocol)
   Establishes a reliable connection using a 3-way handshake (SYN, SYN+ACK, ACK) for accurate data delivery.

3. UDP (User Datagram Protocol)
   Sends lightweight, connectionless packets (requests and responses) with minimal latency. Ideal for fast transmissions.

4. HTTP (HyperText Transfer Protocol)
   Uses TCP to request and receive web resources (HTML, images) through HTTP requests and responses.

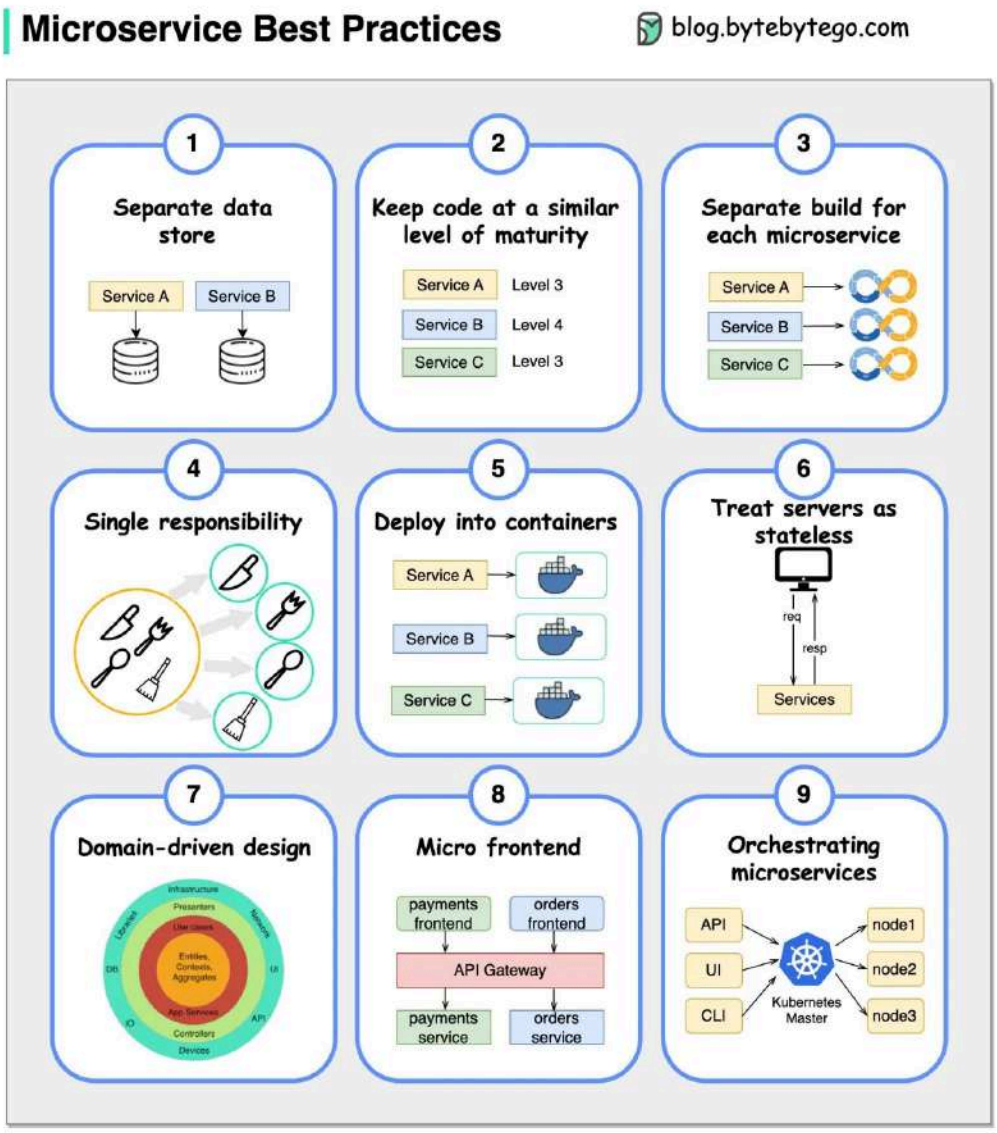
5. HTTP/3 (QUIC)
   Built on top of UDP, it enables faster and more reliable connections by multiplexing data streams and reducing latency.

6. HTTPS (Secure HTTP)
   Secures HTTP with encryption using public and session keys over a TCP connection, thereby protecting web data.

7. SMTP (Simple Mail Transfer Protocol)
   Transfer emails from a sender to a recipient through an SMTP server. It is commonly used for email delivery.

8. WebSocket
   Upgrades an HTTP connection to a full-duplex channel for real-time, bidirectional communication like live chats.

A picture is worth a thousand words: 9 best practices for developing microservices



When we develop microservices, we need to follow the following best practices:

1. Use separate data storage for each microservice
2. Keep code at a similar level of maturity
3. Separate build for each microservice
4. Assign each microservice with a single responsibility
5. Deploy into containers

6. Design stateless services
7. Adopt domain-driven design
8. Design micro frontend
9. Orchestrating microservices